hushuqi 算法竞赛模板 (中)

ICPC/CCPC Algorithm Templates

目录

5. 图论 1
5.1 图的表示与基础遍历 . . . 1
5.1.001 邻接表、链式前向星、邻接矩阵 . . . 1
5.1.002 DFS / BFS . . . 2
5.1.003 拓扑排序 . . . 3
5.1.004 二分图判定 . . . 4
5.1.005 DAG 最长路 . . . 5
5.1.006 DAG 删点最长路 . . . 7
5.1.007 差分约束 . . . 9
5.2 最短路 . . . 10
5.2.001 Dijkstra：稀疏图与稠密图 . . . 10
5.2.002 Bellman-Ford . . . 12
5.2.003 SPFA、SLF 优化与负环判定/输出 . . . 13
5.2.004 Floyd 与最小环 . . . 14
5.2.005 Johnson 全源最短路 . . . 16
5.2.006 区间图最短路 . . . 18
5.2.007 平面图对偶最短路 . . . 21
5.2.008 最短路径树 . . . 22
5.2.009 单源最短路/次短路计数 . . . 23
5.2.010 第 K 短路（Dijkstra / A*） . . . 26
5.2.011 0-1 BFS . . . 28
5.2.012 第 K 短游走（Eppstein） . . . 29
5.2.013 第 K 短简单路 Yen . . . 32
5.3 连通性 . . . 36
5.3.001 Tarjan 强连通分量 . . . 36
5.3.002 Kosaraju 强连通分量 . . . 38
5.3.003 Bitset 优化 Kosaraju . . . 39
5.3.004 割边与边双连通分量 . . . 40
5.3.005 割点与点双连通分量 . . . 43
5.3.006 缩点与块割树 . . . 45
5.3.007 圆方树、广义圆方树 . . . 46
5.3.008 无向图连通度、点连通度、边连通度 . . . 46
5.3.009 最佳/最小点割集与边割集 . . . 48
5.3.010 仙人掌森林分解 . . . 48
5.3.011 补图连通块 . . . 51
5.3.012 离线增量 SCC . . . 52
5.3.013 三边连通分量 . . . 54
5.4 2-SAT . . . 56
5.4.001 Tarjan 2-SAT . . . 56
5.4.002 Kosaraju 2-SAT . . . 58
5.4.003 字典序或答案不唯一时的构造 . . . 60
5.5 最小生成结构 . . . 62

5.5.001 Kruskal . . . 62
5.5.002 次小生成树 . . . 64
5.5.003 K 棵树的最小生成森林 . . . 68
5.5.004 带修改最小生成树 . . . 69
5.5.005 最小乘积生成树 . . . 70
5.5.006 切比雪夫距离最小生成树 . . . 74
5.5.007 平面欧氏距离最小生成树 . . . 76
5.5.008 最小树形图 / 朱刘算法 . . . 78
5.5.009 最小斯坦纳树 . . . 80
5.5.010 Kruskal 重构树 . . . 82
5.5.011 增量最小生成森林 . . . 86
5.5.012 最小直径生成树 . . . 91
5.5.013 Prim：稀疏图与稠密图 . . . 95
5.5.014 Borůvka . . . 97
5.6 二分图匹配与覆盖 . . . 98
5.6.001 匈牙利算法 . . . 98
5.6.002 Bitset 优化匈牙利 . . . 100
5.6.003 Hopcroft-Karp . . . 102
5.6.004 CSR Hopcroft-Karp . . . 105
5.6.005 二分图最大权/最小权匹配（KM） . . . 108
5.6.006 二分图边染色 . . . 112
5.6.007 最小点覆盖、最小边覆盖 . . . 117
5.6.008 最大独立集 . . . 119
5.6.009 DAG 最小不相交链覆盖 . . . 121
5.6.010 DAG 最大互不可达集 . . . 122
5.6.011 最大权闭合子图 . . . 124
5.7 一般图匹配 . . . 125
5.7.001 一般图最大匹配（Blossom） . . . 125
5.7.002 一般图最大权匹配 . . . 129
5.7.003 Tutte Matrix . . . 130
5.7.004 Edmonds Matrix . . . 131
5.7.005 稳定婚姻 . . . 131
5.7.006 一般图最大权匹配（带权 Blossom） . . . 133
5.7.007 稀疏稳定匹配（不完全偏好） . . . 141
5.8 网络流与割 . . . 143
5.8.001 Dinic 最大流 . . . 143
5.8.002 FIFO 预流推进 . . . 146
5.8.003 SPFA 多路增广最小费用流 . . . 149
5.8.004 原始对偶最小费用流 . . . 152
5.8.005 有上下界可行流、最大流、最小流 . . . 156
5.8.006 最大流最小割与建图模型 . . . 158
5.8.007 Stoer-Wagner 全局最小割 . . . 160
5.8.008 Gomory-Hu 最小割树 . . . 162
5.8.009 ISAP 最大流 . . . 163

5.8.010 HLPP / 最高标号预流推进 . . . 166
5.8.011 费用缩放最小费用 b 流与带费用上下界 . . . 172
5.8.012 动态星增广图全局最小割 . . . 177
5.8.013 CSR 优化原始对偶最小费用流 . . . 182
5.8.014 有向图全局最小割 . . . 188
5.9 欧拉图与环 . . . 189
5.9.001 有向/无向欧拉路径与欧拉回路 . . . 189
5.9.002 字典序最小欧拉路径 . . . 194
5.9.003 BEST 定理与有向欧拉回路计数 . . . 194
5.9.004 无向图最小环 . . . 195
5.9.005 最小平均环 / 最小密度环 . . . 196
5.9.006 三元环、四元环计数 . . . 199
5.9.007 简单环存在性、输出与计数 . . . 205
5.9.008 基环树 . . . 209
5.9.009 函数图与后继跳跃 . . . 212
5.10 特殊图与图论结论 . . . 216
5.10.001 弦图、完美消除序列与判错 . . . 216
5.10.002 欧拉公式与圆上弦分侧 . . . 222
5.10.003 支配树（DAG / 一般图） . . . 223
5.10.004 竞赛图与 Landau 定理 . . . 227
5.10.005 度序列可图性 / Erdős-Gallai . . . 227
5.10.006 最大团、极大团计数 . . . 228
5.10.007 最大独立集与点染色 . . . 233
5.10.008 DSATUR 实践型精确点染色 . . . 234
5.10.009 有向图最小点基 . . . 237
5.10.010 有向无环定向与色多项式 . . . 238
5.10.011 拟阵交 . . . 244
5.10.012 小规模暴力双极定向 . . . 246
5.10.013 线性 st 编号 . . . 248
5.10.014 宽度 2 树分解 . . . 251
5.10.015 Ramsey 定理 . . . 253
5.10.016 外环加弦平面性判定（2-SAT） . . . 254
5.10.017 Havel-Hakimi / Gale-Ryser 构造 . . . 255
5.10.018 小图一般图同构 . . . 258

6. 树论与动态树 262
6.1 树的基础性质 . . . 262
6.1.001 树的直径、重心、中心 . . . 262
6.1.002 最近公共祖先：倍增 . . . 265
6.1.003 最近公共祖先：树链剖分 . . . 267
6.1.004 最近公共祖先：ST 表 / RMQ . . . 267
6.1.005 最近公共祖先：Tarjan 离线 . . . 270
6.1.006 树上距离、K 级祖先 . . . 271
6.1.007 树上路径交 . . . 273
6.1.008 树哈希与基环树哈希 . . . 274

6.1.009 Prüfer 序列与树的计数 . . . 279
6.1.010 树的邻接表转前序表 . . . 281
6.1.011 公共区间分解树 . . . 282
6.2 树上分治与启发式合并 . . . 286
6.2.001 点分治 . . . 286
6.2.002 点分树 . . . 289
6.2.003 边分治 . . . 292
6.2.004 DSU on Tree . . . 295
6.2.005 长链剖分 . . . 297
6.2.006 毛毛虫剖分 . . . 297
6.3 树链与虚树 . . . 297
6.3.001 轻重链剖分 / HLD . . . 297
6.3.002 换根树剖 . . . 300
6.3.003 虚树 . . . 303
6.3.004 树上路径/子树数据结构 . . . 305
6.3.005 树上差分 . . . 308
6.4 动态树 . . . 310
6.4.001 Link-Cut Tree . . . 310
6.4.002 带子树信息的 LCT . . . 315
6.4.003 动态割边、割点 LCT . . . 320
6.4.004 全局平衡二叉树 . . . 326
6.4.005 动态 DP . . . 329
6.4.006 离线动态图连通块点权和 . . . 333
6.4.007 Euler Tour Tree . . . 336
6.5 树形 DP . . . 341
6.5.001 树上背包 . . . 341
6.5.002 单调队列优化树上背包 . . . 343
6.5.003 长链剖分 DP 合并 . . . 344
6.5.004 树上依赖背包 . . . 346
6.5.005 有根树最小逆序拓扑序 . . . 348
6.5.006 换根 DP / Rerooting . . . 351

7. 数论 354
7.1 整数运算与模运算 . . . 354
7.1.001 快速幂 . . . 354
7.1.002 GCD、快速 GCD、LCM . . . 355
7.1.003 扩展欧几里得 . . . 356
7.1.004 线性同余方程与同余方程组 . . . 357
7.1.005 CRT / exCRT . . . 358
7.1.006 模逆元：快速幂、exgcd、线性与批量 . . . 359
7.1.007 静态取模类 . . . 360
7.1.008 动态取模类 . . . 360
7.1.009 Barrett 约减 . . . 363
7.1.010 Montgomery 模乘 . . . 363
7.1.011 Long Long 防溢出乘法 . . . 366

7.1.012 快速整除判定 . . . 366
7.1.013 模意义真分数还原 . . . 367
7.1.014 整数 k 次方根 . . . 368
7.1.015 线性同余不等式 . . . 369
7.1.016 基于值域优化的 GCD . . . 370
7.1.017 高斯整数与最大公约数 . . . 372
7.2 素数与因数分解 . . . 374
7.2.001 埃氏筛、线性筛 . . . 374
7.2.002 区间筛 . . . 376
7.2.003 Miller-Rabin . . . 377
7.2.004 Pollard-Rho . . . 379
7.2.005 质因数分解 . . . 381
7.2.006 约数枚举、约数个数和 . . . 382
7.2.007 因数最多的数 . . . 383
7.2.008 阶乘最后非零位 . . . 383
7.2.009 阶乘模合数 . . . 384
7.2.010 哥德巴赫猜想及竞赛用结论 . . . 386
7.2.011 麦乐鸡定理 / 两数 Frobenius 问题 . . . 387
7.3 算术函数与筛法 . . . 387
7.3.001 欧拉函数 . . . 387
7.3.002 莫比乌斯函数 . . . 388
7.3.003 积性函数 . . . 389
7.3.004 Dirichlet 卷积 . . . 390
7.3.005 莫比乌斯反演 . . . 391
7.3.006 杜教筛 . . . 392
7.3.007 Min_25 筛 . . . 395
7.3.008 Meissel-Lehmer 质数计数 . . . 398
7.3.009 整除分块 . . . 400
7.3.010 调和级数分块 . . . 401
7.3.011 素数、$\omega(n)$ 与 $d(n)$ . . . 401
7.3.012 商集上的 Dirichlet 卷积与除法 . . . 402
7.3.013 无平方因子数计数 . . . 407
7.3.014 Lucy 素数前缀筛 . . . 409
7.3.015 Powerful Number 筛 . . . 410
7.4 离散对数与剩余 . . . 412
7.4.001 BSGS / exBSGS . . . 412
7.4.002 Pohlig-Hellman . . . 415
7.4.003 原根与全部原根筛 . . . 416
7.4.004 二次剩余 . . . 417
7.4.005 K 次剩余 . . . 419
7.4.006 扩展欧拉定理与欧拉降幂 . . . 420
7.4.007 Carmichael 函数 . . . 423
7.4.008 固定模批量离散对数 . . . 424
7.4.009 Hensel 简单根提升 . . . 426

7.5 类欧几里得 . . . 428
7.5.001 `floor_sum` . . . 428
7.5.002 类欧几里得格点统计 . . . 429
7.5.003 万能欧几里得 . . . 429
7.5.004 `Min of Mod of Linear` . . . 434
7.5.005 Stern-Brocot Tree / Farey 序列 . . . 435
7.5.006 Pell 方程 . . . 437
7.5.007 圆上整点 . . . 438

8. 组合数学 439
8.1 组合数 . . . 439
8.1.001 阶乘、逆阶乘与组合数 . . . 439
8.1.002 杨辉三角 . . . 440
8.1.003 Lucas 定理 . . . 441
8.1.004 扩展 Lucas . . . 442
8.1.005 任意模数组合数 . . . 445
8.1.006 Vandermonde 卷积 . . . 446
8.1.007 容斥原理 . . . 447
8.1.008 扩展 min-max 容斥与 KMAX-MIN 反演 . . . 448
8.1.009 单位根反演 . . . 450
8.1.010 球盒模型与鸽巢原理 . . . 451
8.1.011 q-二项式与 q-Lucas . . . 453
8.1.012 二项式恒等式与反演 . . . 455
8.1.013 斯特林反演 . . . 456
8.2 经典数列与计数 . . . 457
8.2.001 Fibonacci / Lucas 数 . . . 457
8.2.002 Catalan 数 . . . 458
8.2.003 Motzkin 数 . . . 459
8.2.004 Bell 数 . . . 459
8.2.005 Stirling 数 . . . 460
8.2.006 Eulerian 数 . . . 462
8.2.007 Bernoulli 数与幂和 . . . 463
8.2.008 错排数 . . . 463
8.2.009 分拆数与五边形数 . . . 464
8.2.010 有根树、无根树计数 . . . 465
8.2.011 Matrix-Tree / Kirchhoff 生成树计数 . . . 466
8.2.012 Pólya / Burnside 置换计数 . . . 469
8.2.013 杨氏矩阵与钩子公式 . . . 470
8.2.014 固定 k 的第一类 Stirling 数 . . . 472
8.2.015 固定 k 的第二类 Stirling 数 . . . 473
8.2.016 小素数大下标第一类 Stirling 数 . . . 474
8.2.017 小素数大下标第二类 Stirling 数 . . . 476
8.2.018 LGV 引理与不相交格路计数 . . . 478
8.3 生成与构造 . . . 480
8.3.001 排列组合生成 . . . 480

8.3.002 康托展开与逆展开 481
8.3.003 勾股数枚举 483
8.3.004 阿达马矩阵 484
8.3.005 幻方 484

5. 图论

5.1 图的表示与基础遍历

5.1.001 邻接表、链式前向星、邻接矩阵

图的存储方式决定后续所有算法的常数与可行性，本条给出三种基本形态。邻接表只存实际出现的边，稀疏图上遍历与空间都最省；邻接矩阵按点对直接下标索引，$O(1)$ 判边和改权，代价是 $O(V^2)$ 空间，只在稠密图或需要常数时间查边权时用；链式前向星用一维边数组 `e` 加头指针 `head` 把同起点的边串成链，加边时新边接到链首，天然得到稳定的边编号。普通 `add` 只加入一条边；需要用 `id^1` 取得反向边时，必须改用 `addResEdge` 连续插入正反边。三者不解决任何最短路或连通性问题，只是承载它们的数据结构。

```cpp
// AdjEdge[0..1] 依次是边的终点与边权。
using AdjEdge = array<int, 2>;
using AdjList = vector<vector<AdjEdge>>;
using AdjMatrix = vector<vector<int>>;

class ForwardStar
{
    // head[u] 保存点 u 的第一条出边编号，-1 表示没有出边。
    vector<int> head;
    // e[i][0..3] 依次是第 i 条边的终点、边权、下一条同起点边和起点。
    vector<array<int, 4>> e;

  public:
    ForwardStar(int n) : head(n, -1)
    {
        // n 是点数；构造一张没有边的图，无返回值。
    }

    int add(int u, int v, int w = 1)
    {
        // u、v 是边的起点和终点，w 是边权；加入一条有向边并返回边编号。
        e.push_back({v, w, head[u], u});
        head[u] = e.size() - 1;
        return head[u];
    }

    int addResEdge(int u, int v, int w, int rw = 0)
    {
        // u、v 是正向边端点，w、rw 是正反边初值；返回正向边编号。
        int id = e.size();
        add(u, v, w);
        add(v, u, rw);
        return id;
    }

    const vector<int> &heads() const
    {
```

```
        // 无参数；返回各点第一条出边编号。
        return head;
    }

    const vector<array<int, 4>> &edges() const
    {
        // 无参数；返回链式前向星的边表。
        return e;
    }
};
```

邻接表用 `AdjList g(n)`，无边权时退化为 `vector<vector<int>>`；矩阵用 `AdjMatrix g(n, vector<int>(n, INF))` 并把对角置零；链式前向星构造 `ForwardStar g(n)` 后用 `add(u,v,w)` 加一条有向边，无向边加两次。`addResEdge(u,v,w,rw)` 返回正向边编号 `id`，且只对这一对边保证反向边编号为 `id^1`。邻接表与前向星空间 $O(V+E)$、加边均摊 $O(1)$，矩阵空间 $O(V^2)$、加边 $O(1)$。无权图上求可达性或最少边数直接在邻接表上跑 5.1.002 DFS / BFS；非负边权最短路交给 5.2.001 Dijkstra（稠密图正需要邻接矩阵版）；有负权则改 5.2.002 Bellman-Ford。需要成对反向边、按编号回退的网络流一律成对插边，见 5.8.001 Dinic。

5.1.002 DFS / BFS

两种最基本的遍历。BFS 用队列按边数分层扩展，首次到达某点即是无权图上的最短边数，`d[v]==-1` 兼作未访问标记；DFS 沿一条分支深入到底再回溯，天然给出访问顺序、时间戳与 DFS 搜索树，是后面 Tarjan 系连通性算法的骨架。两者都只处理「能否到达、几条边」，一旦边带非负权就得换 Dijkstra。DFS 的递归深度可达点数量级，链状图需确认系统栈足够，必要时使用 1.2.007 手动扩大栈空间。

```
vector<int> bfs(const vector<vector<int>> &g, int s)
{
    // g 是无权图邻接表，s 是起点；返回到各点的最少边数，不可达点为 -1。
    vector<int> d(g.size(), -1);
    queue<int> q;
    d[s] = 0;
    q.push(s);
    while (!q.empty())
    {
        int u = q.front();
        q.pop();
        for (auto v : g[u])
        {
            if (d[v] == -1)
            {
                d[v] = d[u] + 1;
                q.push(v);
            }
        }
    }
    return d;
}

```

```cpp
vector<int> dfs(const vector<vector<int>> &g, int s)
{
    // g 是图邻接表，s 是起点；返回从 s 出发的递归 DFS 首次访问顺序。
    vector<int> ord, vis(g.size());
    auto work = [&](auto &&self, int u) -> void
    {
        vis[u] = 1;
        ord.push_back(u);
        for (auto v : g[u])
        {
            if (!vis[v])
            {
                self(self, v);
            }
        }
    };
    work(work, s);
    return ord;
}
```

`bfs(g,s)` 返回各点最少边数（不可达为 -1），`dfs(g,s)` 返回首次访问顺序，点编号须在 $[0,n)$。二者均 $O(V+E)$ 时间、$O(V)$ 额外空间，递归 DFS 另有最坏 $O(V)$ 调用栈。边带非负权时最少边数不再等于最短路，改用 5.2.001 Dijkstra；有向无环图上要求「每条边从前指向后」的顺序而非任意访问序时用 5.1.003 拓扑排序。

5.1.003 拓扑排序

拓扑排序把 DAG 的点排成每条边都从前指向后的线性序，是 DAG 上一切递推（最长路、计数、DP）的前置条件，同时兼任判环。这里用 Kahn 算法：先把入度为零的点入队，每取出一个点就删去其全部出边、令新的零入度点入队；若最终排出的点数不足 n，剩下的点必然卡在某个有向环里，据此返回空。队列换成优先队列即可求字典序最小拓扑序，当前普通队列版不保证这一点。

```cpp
optional<vector<int>> topo(const vector<vector<int>> &g)
{
    // g 是有向图邻接表；返回一种拓扑序，图含有向环时返回空 optional。
    vector<int> deg(g.size()), ord;
    for (auto &e : g)
    {
        for (auto v : e)
        {
            deg[v]++;
        }
    }
    queue<int> q;
    for (int i = 0; i < (int)g.size(); i++)
    {
        if (deg[i] == 0)
        {
            q.push(i);
```

```
        }
    }
    while (!q.empty())
    {
        int u = q.front();
        q.pop();
        ord.push_back(u);
        for (auto v : g[u])
        {
            if (--deg[v] == 0)
            {
                q.push(v);
            }
        }
    }
    if ((int)ord.size() != (int)g.size())
    {
        return nullopt;
    }
    return ord;
}
```

`topo(g)` 返回空表示图含有向环，否则返回含全部点的一种拓扑序；$O(V+E)$ 时间、$O(V)$ 额外空间。有向边 `0→1,0→2,1→3,2→3` 的结果必以 0 开头、3 结尾，再加 `3→0` 成环后返回空。排出序后即可按序做 5.1.005 DAG 最长路 的松弛；要求字典序最小方案时把 `queue` 换 `priority_queue`；判环也可用 5.3.001 Tarjan 强连通分量：存在大小大于 1 的强连通分量，或某个单点分量中的点有自环，才说明图中有环。

5.1.004 二分图判定

二分图判定即检查无向图能否两染色使每条边两端异色，等价于「不含奇环」。做法是对每个未染色连通块从颜色 0 起 BFS，沿边把颜色异或 1 传播；一旦碰到已染成同色的邻点，说明存在奇环（树上到两端的路径加这条边构成奇数长度环），返回空。每个连通块独立处理，因此非连通图也能整体给出一组染色。

```
optional<vector<int>> bipartite(const vector<vector<int>> &g)
{
    // g 是无向图邻接表；返回每个点的 0/1 颜色，存在奇环时返回空 optional。
    vector<int> col(g.size(), -1);
    for (int s = 0; s < (int)g.size(); s++)
    {
        if (col[s] != -1)
        {
            continue;
        }
        queue<int> q;
        col[s] = 0;
        q.push(s);
        while (!q.empty())
        {
            int u = q.front();
```

```
            q.pop();
            for (auto v : g[u])
            {
                if (col[v] == -1)
                {
                    col[v] = col[u] ^ 1;
                    q.push(v);
                }
                else if (col[v] == col[u])
                {
                    return nullopt;
                }
            }
        }
    }
    return col;
}
```

无向边须在邻接表里加两个方向再调 `bipartite(g)`：空返回值表示存在奇环，否则返回 $0/1$ 染色；$O(V+E)$ 时间、$O(V)$ 额外空间。四边形 `0-1-2-3-0` 返回合法二染色，三角形 `0-1-2-0` 返回空。判定为二分图后，最大匹配走 5.6.001 匈牙利算法、最小点覆盖走 5.6.007 最小点覆盖、最小边覆盖；若染色本身要动态维护（加边判奇环）则改用带权并查集或 5.5.001 Kruskal 里的并查集扩展。

5.1.005 DAG 最长路

DAG 最长路把拓扑序当成 DP 顺序：按序处理点 u 时，所有指向 u 的路径都已算完，于是沿出边更新终点即可。函数复用 Kahn 拓扑框架先排序兼验 DAG 前提，拓扑序未覆盖全部点时返回空；这表示输入不满足本算法的定义域，并不声称任意有环图的路径权都会无限增大。参数 `s=-1` 时所有点初值为 0，求任意起点的全局最长路；指定 `s` 时用内层 `optional` 区分不可达点，路径和类型由 `Dist` 指定。

```
template <class Dist = long long>
optional<vector<optional<Dist>>> dagLongest(
    const vector<vector<array<int, 2>>> &g, int s = -1)
{
    // g[u] 的每项依次是终点和边权，s 是指定起点；返回各点最长路，图不是 DAG 时返回空。
    vector<int> deg(g.size()), ord;
    for (auto &e : g)
    {
        for (auto [v, w] : e)
        {
            deg[v]++;
        }
    }
    queue<int> q;
    for (int i = 0; i < (int)g.size(); i++)
    {
        if (deg[i] == 0)
        {
            q.push(i);
```

```
        }
    }
    while (!q.empty())
    {
        int u = q.front();
        q.pop();
        ord.push_back(u);
        for (auto [v, w] : g[u])
        {
            if (--deg[v] == 0)
            {
                q.push(v);
            }
        }
    }
    if ((int)ord.size() != (int)g.size())
    {
        return nullopt;
    }
    vector<optional<Dist>> d(g.size());
    if (s == -1)
    {
        fill(d.begin(), d.end(), 0);
    }
    else
    {
        d[s] = 0;
    }
    for (auto u : ord)
    {
        if (!d[u])
        {
            continue;
        }
        for (auto [v, w] : g[u])
        {
            Dist nd = *d[u] + (Dist)w;
            if (!d[v] || nd > *d[v])
            {
                d[v] = nd;
            }
        }
    }
    return d;
}
```

`g[u]` 每项 `array<int,2>` 为终点和边权。`dagLongest(g,s)` 的外层空值表示图不是 DAG；成功时返回各点的最长路，指定 `s` 后内层空值表示该点不可达，`s=-1` 时每个点都可作为起点。调用者必须保证所有参与比较的路径和都可由 `Dist` 表示；需要更宽距离时调用 `dagLongest<__int128>(g,s)`。算法用 $O(V+E)$ 时间、$O(V)$ 额外空间。图不保证无环时可先用 5.3.001 Tarjan 强连通分量 缩点；要求删去某点后的最长路见 5.1.006 DAG 删点最长路；求最短路而非最长路时对非负权用 5.2.001 Dijkstra。

5.1.006 DAG 删点最长路

要对每个点求「删掉它之后剩余 DAG 的最长路径边数」，逐点重跑 DP 是 $O(nm)$，本条用拓扑前后缀把它降到近线性。先拓扑排序，`left[v]` 记以 v 结尾的最长路径长、`right[u]` 记以 u 开头的最长路径长；删去拓扑位置为 i 的点后，一条不经过它的最长路径要么整段落在 i 左侧（前缀 `left` 最大值）、要么整段落在右侧（后缀 `right` 最大值）、要么是一条「跨过位置 i」的边 $u \to v$（`position[u]<i<position[v]`）拼接 `left[u]+1+right[v]`。跨越边按左端点挂到 `starts`，扫描时用大根堆维护当前仍跨过 i 的边的最大贡献，堆顶右端点越界即弹出，于是每个点的答案取前缀、后缀、活跃跨越边三者最大值。

```cpp
class DelDAG
{
    // n 是图的点数。
    int n;
    // graph 是输入 DAG 的邻接表。
    vector<vector<int>> graph;
    // edges 保存全部有向边的起点和终点。
    vector<pair<int, int>> edges;

  public:
    explicit DelDAG(int n) : n(n), graph(n)
    {
        // n 是图的点数；构造一张没有边的 DAG。
    }
    void addEdge(int u, int v)
    {
        // u、v 是边的起点和终点；加入一条有向边。
        graph[u].push_back(v);
        edges.push_back({u, v});
    }

    // 依次删除每个点，返回剩余图中的最长路径边数。
    vector<int> solve() const
    {
        // 无参数；返回依次删除每个点后，剩余 DAG 的最长路径边数。
        vector<int> indegree(n);
        for (auto [u, v] : edges)
        {
            ++indegree[v];
        }
        queue<int> q;
        for (int u = 0; u < n; ++u)
        {
            if (!indegree[u])
            {
                q.push(u);
            }
        }
        vector<int> order;
        while (!q.empty())
        {
            int u = q.front();
```

```
            q.pop();
            order.push_back(u);
            for (int v : graph[u])
            {
                if (!--indegree[v])
                {
                    q.push(v);
                }
            }
        }
        assert((int)order.size() == n); // 调试检查，可删：输入必须是 DAG。
        vector<int> position(n), left(n), right(n);
        for (int i = 0; i < n; ++i)
        {
            position[order[i]] = i;
        }
        for (int u : order)
        {
            for (int v : graph[u])
            {
                left[v] = max(left[v], left[u] + 1);
            }
        }
        for (int u : order | views::reverse)
        {
            for (int v : graph[u])
            {
                right[u] = max(right[u], right[v] + 1);
            }
        }
        vector<vector<pair<int, int>>> starts(n + 1);
        for (auto [u, v] : edges)
        {
            int begin = position[u] + 1, end = position[v];
            if (begin < end)
            {
                starts[begin].push_back({left[u] + 1 + right[v], end});
            }
        }
        vector<int> prefix(n + 1), suffix(n + 1), answer(n);
        for (int i = 0; i < n; ++i)
        {
            prefix[i + 1] = max(prefix[i], left[order[i]]);
        }
        for (int i = n - 1; i >= 0; --i)
        {
            suffix[i] = max(suffix[i + 1], right[order[i]]);
        }
        priority_queue<pair<int, int>> active;
        for (int i = 0; i < n; ++i)
        {
            for (auto item : starts[i])
```

```
            {
                active.push(item);
            }
            while (!active.empty() && active.top().second <= i)
            {
                active.pop();
            }
            answer[order[i]] = max(prefix[i], suffix[i + 1]);
            if (!active.empty())
            {
                answer[order[i]] = max(answer[order[i]], active.top().first);
            }
        }
        return answer;
    }
};
```

`DelDAG g(n)` 建图，`addEdge(u,v)` 加有向边，调用 `solve()` 前必须保证输入是 DAG；代码中的断言只用于调试时发现误用。返回数组第 i 项是删去点 i 后剩余图的最长路径边数。拓扑与前后缀 DP 用 $O(n+m)$ 时间，跨越边用堆维护，合计 $O(n+m\log m)$ 时间、$O(n+m)$ 空间。只需一次性最长路时用 5.1.005 DAG 最长路；若删的是边而非点，跨越集合需按被删边重新定义。

5.1.007 差分约束

差分约束求一组变量满足所有形如 $x_v - x_u \le w$ 的不等式，本质是把每条约束看成边 $u \to v$ 权 w 的松弛 $x_v \le x_u + w$，一组可行解就是最短路。所有初值置零等价于加入一个向每点连零权边的超级源，于是 Bellman-Ford 跑到收敛即得解；若第 n 轮仍能松弛，说明存在负环，约束不相容，返回空。代码就是朴素 n 轮松弛，某轮无更新即提前收敛返回。注意本条只判可行性，若要「最大化」或「最小化」某变量还需按题意选择连边方向和取最长/最短路。

```
template <class Dist = long long>
optional<vector<Dist>> diffCons(int n, const vector<array<int, 3>> &a)
{
    // n 是变量个数；a[i][0..2] 依次是 u、v、w，表示 x[v]-x[u]<=w；无解返回空，否则返回一组可行 x。
    vector<Dist> x(n);
    if (n == 0)
    {
        return x;
    }
    for (int k = 0; k < n; k++)
    {
        bool upd = false;
        for (auto [u, v, w] : a)
        {
            assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
            Dist nx = x[u] + (Dist)w;
            if (x[v] > nx)
            {
                x[v] = nx;
```

```
                upd = true;
            }
        }
        if (!upd)
        {
            return x;
        }
    }
    return nullopt;
}
```

`diffCons(n,a)` 中每个 `array<int,3>` 依次为 u, v, w，返回空表示约束含负环无解，否则返回一组满足全部约束的 `Dist` 势值；默认 `Dist=long long`，调用者须保证可行势值和每次松弛均可表示，需要宽势值时调用 `diffCons<__int128>(n,a)`。因初值全零，它给出一组逐次向下松弛得到的可行解，整体平移仍合法。算法用 $O(VE)$ 时间、$O(V)$ 额外空间。约束 `{0,1,3},{1,2,4},{2,0,-8}` 无解，把 -8 改成 -7 后可行。约束稠密、需要队列优化时见 5.2.003 SPFA；全部边非负、只是普通最短路则用 5.2.001 Dijkstra。

5.2 最短路

5.2.001 Dijkstra：稀疏图与稠密图

Dijkstra 求非负边权图的单源最短路。稀疏版使用不支持 decrease-key 的二叉堆，更新后直接压入新记录，再用 `du≠d[u]` 跳过旧记录；堆内最多有 $O(E)$ 个记录，所以按当前实现严格写作 $O((V+E)\log E)$，简单图下因 $E = O(V^2)$ 可化成常见的 $O((V+E)\log V)$。稠密版直接用 $O(V^2)$ 扫描选点，省去堆常数。

```
// WEdge[0..1] 依次是有向边终点和非负边权。
using WEdge = array<int, 2>;

template <class Dist = long long>
vector<optional<Dist>> dijkstraSp(const vector<vector<WEdge>> &g, int s)
{
    // g 是稀疏图邻接表，s 是源点；返回各点最短距离，不可达点为空。
    int n = g.size();
    vector<optional<Dist>> d(n);
    priority_queue<pair<Dist, int>, vector<pair<Dist, int>>, greater<>> q;
    d[s] = 0;
    q.push({0, s});
    while (!q.empty())
    {
        auto [du, u] = q.top();
        q.pop();
        if (!d[u] || du ≠ *d[u])
        {
            continue;
        }
        for (auto [v, w] : g[u])
        {
```

```
            assert(w >= 0); // 调试检查，可删：Dijkstra 要求边权非负。
            Dist ew = (Dist)w;
            Dist nd = du + ew;
            if (!d[v] || nd < *d[v])
            {
                d[v] = nd;
                q.push({nd, v});
            }
        }
    }
    return d;
}

template <class Dist = long long>
vector<optional<Dist>> dijkstraDn(
    const vector<vector<optional<int>>> &w, int s)
{
    // w 是邻接矩阵，空值表示缺边，s 是源点；返回各点最短距离，不可达点为空。
    int n = w.size();
    vector<optional<Dist>> d(n);
    vector<bool> vis(n);
    d[s] = 0;
    for (int it = 0; it < n; it++)
    {
        int u = -1;
        for (int i = 0; i < n; i++)
        {
            if (!vis[i] && d[i] && (u < 0 || *d[i] < *d[u]))
            {
                u = i;
            }
        }
        if (u < 0)
        {
            break;
        }
        vis[u] = true;
        for (int v = 0; v < n; v++)
        {
            if (w[u][v])
            {
                assert(*w[u][v] >= 0); // 调试检查，可删：Dijkstra 要求边权非负。
                Dist z = (Dist)*w[u][v];
                Dist nd = *d[u] + z;
                if (!d[v] || nd < *d[v])
                {
                    d[v] = nd;
                }
            }
        }
    }
    return d;
```

```
}
```

稀疏图调 `dijkstraSp(g,s)`,`g[u]` 每项 `WEdge` 为终点与非负边权;稠密图调 `dijkstraDn(w,s)`,空 `optional` 表示缺边，对角填 0。两版都用空距离表示不可达；调用者必须保证松弛中间量与有限最短路可由 `Dist` 表示，需要更宽距离时显式调用 `dijkstraSp<__int128>` 或 `dijkstraDn<__int128>`。上面的五边样例从 0 出发距离为 `0,2,3,4`。稀疏版严格界为 $O((V+E)\log E)$，简单图下常写 $O((V+E)\log V)$；稠密版为 $O(V^2)$，额外空间均为 $O(V)$。含负边时改用 5.2.002 Bellman-Ford。

5.2.002 Bellman-Ford

Bellman-Ford 允许负边权求单源最短路：反复扫描全部边，最多 $V-1$ 轮即得到无负环时的最短路；若第 V 轮仍能松弛，被松弛点位于或可达一个负环。距离用 `optional<Dist>` 分开表示不可达和有限值，避免有限哨兵截断合法答案；`p` 记前驱，检测到负环后先沿前驱回退 n 步进入环，再收集一组环上顶点。默认 `Dist=long long`，需要宽距离时显式实例化。`DEdge` 是后续 SPFA 和 Johnson 共用的有向边类型。

```
struct DEdge
{
    // x、y 是有向边的起点和终点。
    int x, y;
    // w 是边权。
    int w;
};

template <class Dist = long long> struct Bellman
{
    // d[v] 是源点到 v 的 Dist 最短距离，空值表示不可达。
    vector<optional<Dist>> d;
    // cycle 是一组源点可达的负环顶点，空数组表示没有这种负环。
    vector<int> cycle;
};

template <class Dist = long long>
Bellman<Dist> bellmanFord(int n, const vector<DEdge> &e, int s)
{
    // n 是点数，e 是边表，s 是源点；返回最短距离及一组可达负环。
    vector<optional<Dist>> d(n);
    vector<int> p(n, -1);
    d[s] = 0;
    int x = -1;
    for (int z = 0; z < n; z++)
    {
        x = -1;
        for (auto [u, v, w] : e)
        {
            if (!d[u])
            {
                continue;
            }
            Dist nd = *d[u] + (Dist)w;
```

```
            if (!d[v] || nd < *d[v])
            {
                d[v] = nd;
                p[v] = u;
                x = v;
            }
        }
    }
    vector<int> cycle;
    if (x != -1)
    {
        for (int i = 0; i < n; i++)
        {
            x = p[x];
        }
        int y = x;
        do
        {
            cycle.push_back(y);
            y = p[y];
        } while (y != x);
        reverse(cycle.begin(), cycle.end());
    }
    return {d, cycle};
}
```

`bellmanFord(n,e,s)` 返回的 `Bellman<Dist>::d` 用空值表示不可达，`cycle` 非空时给出一组从 s 可达的负环顶点。调用者必须保证松弛中间量与有限距离可由 `Dist` 表示，需要更宽距离时调用 `bellman-Ford<__int128>`。算法最坏用 $O(nm)$ 时间、$O(n+m)$ 空间。需要同时区分有限距离、不可达与可由负环到达的负无穷点时，调用本条附带的 `negShortest(n,edges,s)`；它维护最短路树并传播负环影响，随机数据上较快，但最坏复杂度不作多项式保证，要求稳定上界时仍用 Bellman-Ford。边全非负时用 5.2.001 Dijkstra；只需队列式判负环时见 5.2.003 SPFA、SLF 优化与负环判定/输出；全源稀疏图见 5.2.005 Johnson 全源最短路。

5.2.003 SPFA、SLF 优化与负环判定/输出

SPFA 是 Bellman-Ford 的队列优化：只把成功松弛的点重新入队，避免每轮盲扫全部边。负环判定维护当前最短路的边数 `len`，一旦某点的边数达到 n，这条更短路径必然含环，返回空。它最坏仍是 $O(nm)$，容易被构造数据卡满；稳定性要求高时用 Bellman-Ford，边全非负时用 Dijkstra。本条复用 5.2.002 Bellman-Ford 的 `DEdge`，距离默认是 `Dist=long long`，并在负数加法前检查上下界。

```
// 依赖 5.2.002 Bellman-Ford
template <class Dist = long long>
optional<vector<optional<Dist>>> spfa(
    int n, const vector<DEdge> &edges, int s)
{
    // n 是点数，edges 是边表，s 是源点；有可达负环时返回空，否则返回最短距离。
    vector<vector<pair<int, Dist>>> e(n);
    for (auto [x, y, w] : edges)
```

```
    {
        e[x].push_back({y, w});
    }
    vector<optional<Dist>> d(n);
    vector<int> in(n), len(n);
    queue<int> q;
    d[s] = 0;
    in[s] = 1;
    q.push(s);
    while (!q.empty())
    {
        int x = q.front();
        q.pop();
        in[x] = 0;
        for (auto [y, w] : e[x])
        {
            Dist nd = *d[x] + w;
            if (d[y] && *d[y] <= nd)
            {
                continue;
            }
            d[y] = nd;
            len[y] = len[x] + 1;
            if (len[y] >= n)
            {
                return nullopt;
            }
            if (!in[y])
            {
                in[y] = 1;
                q.push(y);
            }
        }
    }
    return d;
}
```

`spfa(n,edges,s)` 的外层空值表示源点可达负环；成功时，内层空距离表示该点不可达，其余距离默认是 `long long`。调用者必须保证松弛中间量与有限最短路可由 `Dist` 表示；需要更宽距离时调用 `spfa<__int128>`。最坏用 $O(nm)$ 时间、$O(n+m)$ 空间。要输出具体负环顶点，用 5.2.002 Bellman-Ford 的前驱回溯版；边全非负时 5.2.001 Dijkstra 更稳。

5.2.004 Floyd 与最小环

Floyd 按「允许经过的中间点集合逐个扩大」做 DP：处理到第 k 个中间点时，用经过 k 的路径更新所有点对，全程 $O(n^3)$，适合稠密图全点对最短路，也容忍负边，但最短路语义要求相关路径上没有负环。无向图最小环利用同一层次结构：在把 k 并入中间点集合之前，$d[i][j]$ 只经过编号小于 k 的点，此时组合 $d[i][j]+a[i][k]+a[k][j]$ 得到一个以 k 为最大编号点的环。必须先统计环，再用 k 更新距离。矩阵空值表示缺边或不可达，有限值类型由 `Dist` 指定。

```cpp
template <class Dist = long long>
vector<vector<optional<Dist>>> floyd(vector<vector<optional<Dist>>> d)
{
    // d 是初始距离矩阵，空值表示没有路径；返回全点对最短距离。
    int n = d.size();
    for (int k = 0; k < n; k++)
    {
        for (int i = 0; i < n; i++)
        {
            if (!d[i][k])
            {
                continue;
            }
            for (int j = 0; j < n; j++)
            {
                if (d[k][j])
                {
                    Dist nd = *d[i][k] + *d[k][j];
                    if (!d[i][j] || nd < *d[i][j])
                    {
                        d[i][j] = nd;
                    }
                }
            }
        }
    }
    return d;
}

template <class Dist = long long>
optional<Dist> minCycle(const vector<vector<optional<Dist>>> &a)
{
    // a 是非负权无向简单图的邻接矩阵；返回最小环权值，无环时返回空。
    int n = a.size();
    auto d = a;
    optional<Dist> ans;
    for (int k = 0; k < n; k++)
    {
        for (int i = 0; i < k; i++)
        {
            for (int j = i + 1; j < k; j++)
            {
                if (d[i][j] && a[i][k] && a[k][j])
                {
                    Dist cur = *d[i][j] + *a[i][k] + *a[k][j];
                    if (!ans || cur < *ans)
                    {
                        ans = cur;
                    }
                }
            }
        }
```

```
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (d[i][k] && d[k][j])
                {
                    Dist nd = *d[i][k] + *d[k][j];
                    if (!d[i][j] || nd < *d[i][j])
                    {
                        d[i][j] = nd;
                    }
                }
            }
        }
    }
    return ans;
}
```

`floyd(d)` 要求对角为 0、缺边为空，返回全点对最短距离；调用后若 `d[i][i]<0`，说明从 i 可达负环。`minCycle(a)` 接受非负权无向简单图邻接矩阵，无环时返回空值。调用者必须保证有限路径与环权可由 `Dist` 表示；需要更宽距离时传入 `optional<__int128>` 矩阵。两者均用 $O(n^3)$ 时间、$O(n^2)$ 空间。稀疏图全源最短路见 5.2.005 Johnson 全源最短路；单源见 5.2.001 Dijkstra；负环输出见 5.2.002 Bellman-Ford。

5.2.005 Johnson 全源最短路

Johnson 让含负边的稀疏图也能用 Dijkstra 求全源最短路。关键是势函数重标权：先跑一次 Bellman-Ford 求出每点势 $h(u)$（等价于加超级源到各点的最短路），把边权重定义为 $w'(u,v) = w(u,v) + h(u) - h(v)$。由三角不等式 $h(v) \le h(u) + w$ 知 $w' \ge 0$，且任意路径 $u \rightsquigarrow v$ 的新权只比原权多常数 $h(u) - h(v)$，最短路结构不变；于是从每点跑 Dijkstra，最后减去 $h(v) - h(source)$ 还原真实距离。Bellman-Ford 第 n 轮仍能松弛则存在负环，返回空。

```
// 依赖 5.2.002 Bellman-Ford
template <class Dist = long long> struct Johnson
{
    // n 是图的点数。
    int n;
    // e 保存全部有向边。
    vector<DEdge> e;

    explicit Johnson(int n) : n(n)
    {
        // n 是图的点数；构造一张没有边的图。
    }
    void addEdge(int u, int v, int w)
    {
        // u、v 是边的起点和终点，w 是边权；加入一条有向边。
        e.push_back({u, v, w});
    }

```

```
    optional<vector<vector<optional<Dist>>>> solve() const
    {
        // 无参数；返回全点对最短距离，图含负环时返回空。
        vector<Dist> h(n);
        for (int k = 0; k < n; k++)
        {
            bool upd = false;
            for (auto [u, v, w] : e)
            {
                Dist nh = h[u] + (Dist)w;
                if (h[v] > nh)
                {
                    h[v] = nh;
                    upd = true;
                }
            }
            if (!upd)
            {
                break;
            }
            if (k == n - 1)
            {
                return nullopt;
            }
        }
        vector<vector<pair<int, Dist>>> g(n);
        for (auto [u, v, w] : e)
        {
            Dist nw = (Dist)w + h[u] - h[v];
            assert(nw >= 0); // 调试检查，可删：势函数应使重标权非负。
            g[u].push_back({v, nw});
        }
        vector ans(n, vector<optional<Dist>>(n));
        for (int s = 0; s < n; s++)
        {
            priority_queue<pair<Dist, int>, vector<pair<Dist, int>>, greater<>> q;
            ans[s][s] = 0;
            q.push({0, s});
            while (!q.empty())
            {
                auto [du, u] = q.top();
                q.pop();
                if (!ans[s][u] || du != *ans[s][u])
                {
                    continue;
                }
                for (auto [v, w] : g[u])
                {
                    Dist nd = du + w;
                    if (!ans[s][v] || nd < *ans[s][v])
                    {
                        ans[s][v] = nd;
```

```
                        q.push({nd, v});
                    }
                }
            }
            for (int v = 0; v < n; ++v)
            {
                if (ans[s][v])
                {
                    ans[s][v] = *ans[s][v] + h[v] - h[s];
                }
            }
        }
        return ans;
    }
};
```

`Johnson<> g(n)` 建图，`addEdge(u,v,w)` 允许负边；`solve()` 的外层空值表示图含负环，成功时矩阵中的空值表示两点不可达，有限距离默认是 `long long`。势、重标边权、堆键和答案都使用 `Dist`；调用者必须保证这些中间量均可表示，需要更宽距离时构造 `Johnson<__int128>`。势函数用 $O(nm)$ 时间，随后 n 次 Dijkstra 用 $O(n(m+n)\log n)$ 时间，总空间为 $O(n^2+m)$。全部边非负时直接逐点调用 5.2.001 Dijkstra；稠密图改用 5.2.004 Floyd 与最小环。

5.2.006 区间图最短路

当题目要求「一个点向一段连续下标批量连同权边」或反向时，逐点连边是 $O(n)$，用线段树优化建图能压到 $O(\log n)$ 条。建两棵零权线段树：向外树的父节点向两个儿子连边、叶子连到原点，代表「从某区间节点能走到区间内每个点」；向内树方向相反，代表「区间内每点都能汇到该区间节点」。一个区间在线段树上被拆成 $O(\log n)$ 个节点，于是「点 u 到区间 $[l,r)$」只需从 u 向向外树的 $O(\log n)$ 个覆盖节点各连一条边，「区间到点」对称地连向内树。最后在这张含辅助点的图上跑 Dijkstra，权值必须非负。

```
template<class Dist = long long>
class IntGraph
{
    // n 是原始点数，g 是包含线段树辅助点的带权有向图。
    int n;
    vector<vector<array<int, 2>>> g;
    // out[p]、in[p] 是原区间节点 p 对应的向外树、向内树图节点编号。
    vector<int> out, in;

    int node()
    {
        // 无参数；新增一个空图节点并返回编号。
        g.push_back({});
        return g.size() - 1;
    }

    void edge(int u, int v, int w)
    {
        // u、v 是图节点，w 是非负边权；加入 u 到 v 的有向边，无返回值。
```

```
        g[u].push_back({v, w});
    }

    void build(int p, int l, int r)
    {
        // p 是线段树节点，[l,r) 是原点区间；建立两棵方向相反的零权树，无返回值。
        out[p] = node();
        in[p] = node();
        if (r - l == 1)
        {
            edge(out[p], l, 0);
            edge(l, in[p], 0);
            return;
        }
        int m = (l + r) / 2;
        build(p << 1, l, m);
        build(p << 1 | 1, m, r);
        edge(out[p], out[p << 1], 0);
        edge(out[p], out[p << 1 | 1], 0);
        edge(in[p << 1], in[p], 0);
        edge(in[p << 1 | 1], in[p], 0);
    }

    void pointRange(int p, int l, int r, int u, int ql, int qr, int w)
    {
        // p、[l,r) 是线段树状态，u 是起点，[ql,qr) 是终点范围，w 是边权；加入压缩边。
        if (qr <= l || r <= ql)
        {
            return;
        }
        if (ql <= l && r <= qr)
        {
            edge(u, out[p], w);
            return;
        }
        int m = (l + r) / 2;
        pointRange(p << 1, l, m, u, ql, qr, w);
        pointRange(p << 1 | 1, m, r, u, ql, qr, w);
    }

    void rangePoint(int p, int l, int r, int ql, int qr, int v, int w)
    {
        // p、[l,r) 是线段树状态，[ql,qr) 是起点范围，v 是终点，w 是边权；加入压缩边。
        if (qr <= l || r <= ql)
        {
            return;
        }
        if (ql <= l && r <= qr)
        {
            edge(in[p], v, w);
            return;
        }
```

```
        int m = (l + r) / 2;
        rangePoint(p << 1, l, m, ql, qr, v, w);
        rangePoint(p << 1 | 1, m, r, ql, qr, v, w);
    }

public:
    IntGraph(int n) : n(n), g(n), out(4 * n), in(4 * n)
    {
        // n 是正的原始点数；建立区间图辅助节点。
        assert(n > 0); // 调试检查，可删。
        build(1, 0, n);
    }

    void addEdge(int u, int v, int w)
    {
        // u、v 是原始点，w 是非负边权；加入单点到单点边。
        assert(0 <= u && u < n && 0 <= v && v < n && w >= 0); // 调试检查，可删。
        edge(u, v, w);
    }

    void addPointRng(int u, int l, int r, int w)
    {
        // u 是起点，[l,r) 是终点范围，w 是非负边权；表示 r-l 条逻辑边。
        assert(0 <= u && u < n && 0 <= l && l <= r && r <= n && w >= 0); // 调试检查，可删。
        pointRange(1, 0, n, u, l, r, w);
    }

    void addRngPoint(int l, int r, int v, int w)
    {
        // [l,r) 是起点范围，v 是终点，w 是非负边权；表示 r-l 条逻辑边。
        assert(0 <= l && l <= r && r <= n && 0 <= v && v < n && w >= 0); // 调试检查，可删。
        rangePoint(1, 0, n, l, r, v, w);
    }

    vector<optional<Dist>> dijkstra(int s) const
    {
        // s 是原始源点；返回到全部原始点的最短路，不可达点为空。
        assert(0 <= s && s < n); // 调试检查，可删。
        vector<optional<Dist>> d(g.size());
        priority_queue<pair<Dist, int>, vector<pair<Dist, int>>, greater<>> q;
        d[s] = 0;
        q.push({0, s});
        while (!q.empty())
        {
            auto [du, u] = q.top();
            q.pop();
            if (!d[u] || du ≠ *d[u])
            {
                continue;
            }
            for (auto [v, w] : g[u])
            {
```

```
                Dist nd = du + (Dist)w;
                if (!d[v] || nd < *d[v])
                {
                    d[v] = nd;
                    q.push({nd, v});
                }
            }
        }
        d.resize(n);
        return d;
    }
};
```

构造 `IntGraph q(n)` 时要求 $n > 0$，三个加边接口都要求端点有效、区间左闭右开且边权非负；断言仅用于调试。`IntGraph<Dist>::dijkstra(s)` 返回原始点距离，空值表示不可达；调用者必须保证有限距离可由 `Dist` 表示，需要更宽距离时可显式使用 `IntGraph<__int128>`。$n = 5$、$0 \to [1,4)$ 权 7、$[1,3) \to 4$ 权 2、$3 \to 4$ 权 1 时，从 0 的距离为 `0,7,7,7,8`。辅助点与边为 $O(n)$，每次区间连边新增 $O(\log n)$ 条实际边，Dijkstra 用 $O((V+E)\log V)$ 时间。没有区间需求时直接用 5.2.001 Dijkstra。

5.2.007 平面图对偶最短路

在已知平面嵌入（面及相邻关系）时，「穿越一系列边从面 s 走到面 t 的最小总代价」可转成对偶图最短路：原图每条边恰分隔两个面，就在这两个面的对偶点间连一条权等于穿越代价的边，面路径的最短路即最小穿越代价，这正是平面图 s-t 最小割转最短路的经典技巧。函数本体就是一次记录前驱的 Dijkstra，最后回溯出一条面序列。前提是面信息已经给出——只有原图点边邻接而无嵌入时无法直接用。

```
template<class Dist = long long>
struct DualPath
{
    // dist 是最小穿越代价。
    Dist dist;
    // path 是从起始面到终止面的面编号序列。
    vector<int> path;
};

template<class Dist = long long>
optional<DualPath<Dist>> dualShortest(
    int faces, const vector<array<int, 3>> &dualEdges, int s, int t)
{
    // faces 是面数，边记录相邻面与代价，s、t 是起终面；不可达时返回空。
    assert(faces > 0 && 0 <= s && s < faces && 0 <= t && t < faces); // 调试检查，可删。
    vector<vector<array<int, 2>>> g(faces);
    for (auto [u, v, w] : dualEdges)
    {
        assert(0 <= u && u < faces && 0 <= v && v < faces && w >= 0); // 调试检查，可删。
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
    vector<optional<Dist>> d(faces);
```

```
    vector<int> pre(faces, -1);
    priority_queue<pair<Dist, int>, vector<pair<Dist, int>>, greater<>> q;
    d[s] = 0;
    q.push({0, s});
    while (!q.empty())
    {
        auto [du, u] = q.top();
        q.pop();
        if (!d[u] || du != *d[u])
        {
            continue;
        }
        for (auto [v, w] : g[u])
        {
            Dist nd = du + (Dist)w;
            if (!d[v] || nd < *d[v])
            {
                d[v] = nd;
                pre[v] = u;
                q.push({nd, v});
            }
        }
    }
    if (!d[t])
    {
        return nullopt;
    }
    vector<int> path;
    for (int u = t; u != -1; u = pre[u])
    {
        path.push_back(u);
    }
    reverse(path.begin(), path.end());
    return DualPath<Dist>{*d[t], path};
}
```

先给所有面编号，为每条可穿越原边提供 {左面，右面，代价}，并保证面编号有效、代价非负。`dualShortest(faces,dualEdges,s,t)` 默认以 `long long` 保存距离，返回 `DualPath<Dist>`，其中 `dist` 是最小代价，`path` 是一条最短面路径；不连通时返回空。调用者必须保证路径代价可由 `Dist` 表示，需要更宽距离时可显式调用 `dualShortest<__int128>`。算法用 $O((F+D)\log F)$ 时间、$O(F+D)$ 空间，建立平面嵌入的成本另计。它与原图 s-t 最小割对应，一般图最小割见 5.8.006 最大流最小割与建图模型。

5.2.008 最短路径树

最短路径树从每个可达非源点选一条满足 $d[v]=d[u]+w$ 的父边，便于回溯路径或继续做树上处理。不能在 Dijkstra 的等长松弛中各自挑最小父点：零权等距分量内可能互相选为父亲而成环。这里先求完整距离，再从源点沿最短路子图做一次 BFS；只有首次到达点 v 时才记录父边，因此父指针一定构成一棵有根树，零权环也不会破坏结构。

```cpp
template<class Dist = long long>
pair<vector<optional<Dist>>, vector<int>> shortestTree(
    const vector<vector<WEdge>> &g, int s)
{
    // g 是非负权有向图，s 是源点；返回最短距离与一棵最短路径树的父节点。
    int n = g.size();
    assert(0 <= s && s < n); // 调试检查，可删。
    auto d = dijkstraSp<Dist>(g, s);
    vector<int> par(n, -1), vis(n);
    queue<int> q;
    vis[s] = 1;
    q.push(s);
    while (!q.empty())
    {
        int u = q.front();
        q.pop();
        for (auto [v, w] : g[u])
        {
            Dist z = (Dist)w;
            if (!vis[v] && d[v] && *d[u] + z == *d[v])
            {
                vis[v] = 1;
                par[v] = u;
                q.push(v);
            }
        }
    }
    return {d, par};
}
```

本条依赖 5.2.001 Dijkstra：稀疏图与稠密图 的 `WEdge` 与 `dijkstraSp`。`Dist` 默认是 `long long`；若最短距离可能超出 64 位，可显式调用 `shortestTree<__int128>`。返回值为 `{d,par}`，空距离表示不可达，源点和不可达点的父亲均为 -1；从任一可达点沿 `par` 必能回到源点。算法用 $O((V+E)\log V)$ 时间、$O(V)$ 额外空间。含负边时先用 5.2.002 Bellman-Ford 求距离，再按相同办法遍历最短路子图。

5.2.009 单源最短路/次短路计数

这条统计从源点到每点的最短、严格次短两种游走长度及各自条数。先用 Dijkstra 式扩展只求每点最小的两个不同距离，再建立两层状态图：状态 $(v,0/1)$ 表示到 v 的最短/次短距离，原边恰好把一个已知距离转成另一个已知距离时才加入状态边，最后按拓扑序计数。这样相同距离的全部前驱都会在后继之前汇总，不依赖优先队列对并列元素的弹出顺序。边权要求非负，且这张可达状态图必须无环；若有零权环，按游走语义会产生无限条等长方案，函数返回空。

```cpp
template<class Dist = long long>
struct ShortestCnt
{
    // d[v][0..1] 是到 v 的最短、严格次短距离，空值表示不存在。
    vector<array<optional<Dist>, 2>> d;
    // cnt[v][0..1] 是对应距离的游走条数模 mod。
```

```
    vector<array<int, 2>> cnt;
};

template<class Dist = long long>
optional<ShortestCnt<Dist>> shortestCnt(
    const vector<vector<pair<int, int>>> &g, int s, int mod)
{
    // g 是非负权邻接表，s 是源点，mod 是正模数；零权环导致计数无穷时返回空。
    int n = g.size();
    assert(0 <= s && s < n); // 调试检查，可删。
    assert(mod > 0); // 调试检查，可删。
    ShortestCnt<Dist> z{vector<array<optional<Dist>, 2>>(n), vector<array<int, 2>>(n)};
    using Q = pair<Dist, int>;
    priority_queue<Q, vector<Q>, greater<>> q;
    z.d[s][0] = 0;
    q.push({0, s});
    while (!q.empty())
    {
        auto [du, u] = q.top();
        q.pop();
        if ((!z.d[u][0] || du != *z.d[u][0]) &&
            (!z.d[u][1] || du != *z.d[u][1]))
        {
            continue;
        }
        for (auto [v, w] : g[u])
        {
            assert(w >= 0); // 调试检查，可删。
            Dist ew = (Dist)w;
            Dist nd = du + ew;
            if (!z.d[v][0] || nd < *z.d[v][0])
            {
                z.d[v][1] = z.d[v][0];
                z.d[v][0] = nd;
                q.push({nd, v});
            }
            else if (nd > *z.d[v][0] && (!z.d[v][1] || nd < *z.d[v][1]))
            {
                z.d[v][1] = nd;
                q.push({nd, v});
            }
        }
    }
    vector<vector<int>> e(2 * n);
    vector<int> deg(2 * n), use(2 * n);
    for (int u = 0; u < n; u++)
    {
        for (int k = 0; k < 2; k++)
        {
            if (!z.d[u][k])
            {
                continue;
```

```
            }
            int x = 2 * u + k;
            use[x] = 1;
            for (auto [v, w] : g[u])
            {
                Dist ew = (Dist)w;
                Dist nd = *z.d[u][k] + ew;
                for (int p = 0; p < 2; p++)
                {
                    if (z.d[v][p] && nd == *z.d[v][p])
                    {
                        e[x].push_back(2 * v + p);
                        deg[2 * v + p]++;
                    }
                }
            }
        }
    }
    queue<int> que;
    int states = 0, done = 0;
    for (int x = 0; x < 2 * n; x++)
    {
        if (use[x])
        {
            states++;
            if (deg[x] == 0)
            {
                que.push(x);
            }
        }
    }
    z.cnt[s][0] = 1 % mod;
    while (!que.empty())
    {
        int x = que.front();
        que.pop();
        done++;
        for (auto y : e[x])
        {
            int v = y / 2, k = y % 2;
            int add = z.cnt[x / 2][x % 2];
            if (z.cnt[v][k] >= mod - add)
            {
                z.cnt[v][k] -= mod - add;
            }
            else
            {
                z.cnt[v][k] += add;
            }
            if (--deg[y] == 0)
            {
                que.push(y);
```

```
            }
        }
    }
    if (done != states)
    {
        return nullopt;
    }
    return z;
}
```

`shortestCnt(g,s,mod)` 要求 `mod>0`，返回空表示可达的最短/次短状态图含环，按游走语义计数无穷；否则 `d[v][0/1]` 与 `cnt[v][0/1]` 分别是最短/严格次短距离及其条数模 `mod`。调用者必须保证距离中间量可由 `Dist` 表示，需要更宽距离时可显式调用 `shortestCnt<__int128>`。不存在的名次为空。算法用 $O((n+m)\log n)$ 时间、$O(n+m)$ 空间。只要一棵最短路径树时用 5.2.008 最短路径树，只要距离时用 5.2.001 Dijkstra。

5.2.010 第 K 短路（Dijkstra / A*）

第 K 短路按总长从小到大枚举源到汇的游走（允许重复点边），本条用 A* 求。先在反图上跑一次 Dijkstra，得到每点到汇 t 的精确最短距离 $h(u)$，它是可采纳且一致的估价；主搜索的优先队列按 $f=$「已走距离 $+h(u)$」取最小，当汇点第 k 次出队时的实际距离就是第 K 短。用 `cnt[u]` 限制每点最多扩展 k 次即可保证复杂度。`keep0` 控制 $s=t$ 时那条长度为零的空游走是否计入排名。零权环下路径条数无限，需靠这种「游走」定义配合出队计数才有良定义。

```
struct KEdge
{
    // u、v 是有向边的起点和终点。
    int u, v;
    // w 是非负边权。
    int w;
};

template<class Dist = long long>
optional<Dist> kthWalk(
    int n, const vector<KEdge> &e, int s, int t,
    unsigned long long k, bool keep0 = true)
{
    // n 是点数，e 是边表，s、t 是起终点，k 从 1 开始；返回第 k 小游走长度。
    assert(k >= 1 && 0 <= s && s < n && 0 <= t && t < n); // 调试检查，可删。
    vector<vector<pair<int, int>>> g(n), rg(n);
    for (auto [u, v, w] : e)
    {
        assert(w >= 0); // 调试检查，可删。
        g[u].push_back({v, w});
        rg[v].push_back({u, w});
    }
    vector<optional<Dist>> h(n);
    priority_queue<pair<Dist, int>, vector<pair<Dist, int>>, greater<>> dq;
    h[t] = 0;
```

```
    dq.push({0, t});
    while (!dq.empty())
    {
        auto [d, u] = dq.top();
        dq.pop();
        if (!h[u] || d ≠ *h[u])
        {
            continue;
        }
        for (auto [v, w] : rg[u])
        {
            Dist ew = (Dist)w;
            Dist nd = d + ew;
            if (!h[v] || nd < *h[v])
            {
                h[v] = nd;
                dq.push({nd, v});
            }
        }
    }
    if (!h[s])
    {
        return nullopt;
    }
    if (s == t && keep0)
    {
        if (k == 1)
        {
            return (Dist)0;
        }
        k--;
    }
    using State = tuple<Dist, Dist, int>;
    priority_queue<State, vector<State>, greater<>> q;
    vector<unsigned long long> cnt(n);
    for (auto [v, w] : g[s])
    {
        if (!h[v])
        {
            continue;
        }
        Dist ew = (Dist)w;
        Dist nd = ew;
        q.push({nd + *h[v], nd, v});
    }
    while (!q.empty())
    {
        auto [f, d, u] = q.top();
        q.pop();
        if (cnt[u] == k)
        {
            continue;
```

```
        }
        cnt[u]++;
        if (u == t && cnt[u] == k)
        {
            return d;
        }
        for (auto [v, w] : g[u])
        {
            if (h[v] && cnt[v] < k)
            {
                Dist ew = (Dist)w;
                Dist nd = d + ew;
                q.push({nd + *h[v], nd, v});
            }
        }
    }
    return nullopt;
}
```

`kthWalk` 要求边权非负且 $k \geq 1$，返回第 k 小有向游走长度，不足 k 条或不可达时返回空。调用者必须保证实际距离、估价与二者之和可由 `Dist` 表示，需要更宽距离时可显式调用 `kthWalk<__int128>`。`keep0=false` 会在 $s = t$ 时跳过唯一的空游走；函数先把源点的每条出边放入队列，因此不会用 `k+1` 表示这个特例。反图 Dijkstra 用 $O((n+m)\log n)$ 时间，主搜索至多弹出每点 k 次、扫描每条出边 k 次，最坏用 $O(km\log(km))$ 时间和 $O(km+n)$ 空间。它枚举允许重复点边的游走；要求简单路径时应改用专门的 K 短简单路算法，不能直接套本条。

5.2.011 0-1 BFS

边权只取 0 或 1 时，用双端队列替代 Dijkstra 的优先队列。队首弹出的点具有当前最小距离：沿 0 边松弛得到的点仍属于当前距离层，压到队首；沿 1 边松弛得到下一层，压到队尾。因此每个点第一次正式弹出时距离已经确定，`done` 让它只扫描一次出边；距离用 `optional` 区分不可达，不借有限哨兵承担加法。若出现负边或大于 1 的边权，这个队列次序便不再成立。

```
// BinaryEdge[0] 是终点，BinaryEdge[1] 是只能取 0 或 1 的边权。
using BinaryEdge = array<int, 2>;

template <class Dist = long long> vector<optional<Dist>> zeroOneBfs(const
 ↪  vector<vector<BinaryEdge>> &g, int s)
{
    // g 是 0-1 权有向图，s 是源点；返回最短距离，不可达点为空。
    int n = g.size();
    assert(0 <= s && s < n); // 调试检查，可删
    vector<optional<Dist>> d(n);
    vector<bool> done(n);
    deque<int> q;
    d[s] = (Dist)0;
    q.push_front(s);
    while (!q.empty())
    {
```

```
        int u = q.front();
        q.pop_front();
        if (done[u])
        {
            continue;
        }
        done[u] = true;
        for (auto [v, w] : g[u])
        {
            assert(0 <= v && v < n && (w == 0 || w == 1)); // 调试检查，可删
            Dist nd = *d[u] + (Dist)w;
            if (!d[v].has_value() || nd < *d[v])
            {
                d[v] = nd;
                if (w == 0)
                {
                    q.push_front(v);
                }
                else
                {
                    q.push_back(v);
                }
            }
        }
    }
    return d;
}
```

`zeroOneBfs(g,s)` 使用 0 下标有向邻接表，建无向边时需向两端各加入一项；调用者须保证最短距离能由 `Dist` 表示，需要更宽结果时显式调用 `zeroOneBfs<__int128>`。设点数为 n、边数为 m，时间 $O(n+m)$、额外空间 $O(n)$。全为 1 时普通 5.1.002 BFS 更短，一般非负权改用 5.2.001 Dijkstra，存在负权则用 5.2.002 Bellman-Ford 或 5.2.003 SPFA。

```
signed main()
{
    vector<vector<BinaryEdge>> g(6);
    g[0] = {{1, 1}, {2, 0}};
    g[2] = {{1, 0}, {3, 1}};
    g[1] = {{3, 1}};
    g[3] = {{4, 0}};
    auto d = zeroOneBfs(g, 0);
    assert(d[0] == 0 && d[1] == 0 && d[2] == 0);
    assert(d[3] == 1 && d[4] == 1 && !d[5]);
}
```

5.2.012 第 K 短游走（Eppstein）

当 n, m, k 都达到 3×10^5 时，5.2.010 的逐次 A* 扩展会变成 $O(km \log(km))$，只适合小规模或很小的名次。这里保留它的接口和教学价值，另用 Eppstein 的偏离边枚举处理大规模有向非负权图。先在反图上以 t 为源运行 Dijkstra，选出一棵到汇最短路树；非树边 $u \to v$ 的偏离代价为 $w + d(v) - d(u) \geq 0$。

沿最短路树把各点可用偏离边合并为持久左偏堆，堆节点的左右儿子替换当前偏离边，堆根再接入终点处的偏离堆，便恰好枚举全部游走。

```cpp
struct EppsteinEdge
{
    // u、v 是有向边端点，w 是非负边权。
    int u, v, w;
};

template<class Dist = long long>
vector<Dist> eppWalks(int n, const vector<EppsteinEdge> &e, int s, int t,
                              unsigned long long k, bool keep0 = true)
{
    // n 是点数，e 是边表，s、t 是起终点，k 是要枚举的项数；返回前至多 k 小游走长度。
    assert(k >= 1 && 0 <= s && s < n && 0 <= t && t < n);
    vector<vector<array<int, 3>>> g(n), rg(n);
    for (int i = 0; i < (int)e.size(); i++)
    {
        auto [u, v, w] = e[i];
        assert(0 <= u && u < n && 0 <= v && v < n && w >= 0);
        g[u].push_back({v, w, i});
        rg[v].push_back({u, w, i});
    }
    vector<optional<Dist>> d(n);
    vector<int> par(n, -1), pe(n, -1);
    priority_queue<pair<Dist, int>, vector<pair<Dist, int>>, greater<>> q;
    d[t] = 0;
    q.push({0, t});
    while (!q.empty())
    {
        auto [du, u] = q.top();
        q.pop();
        if (!d[u] || du != *d[u])
        {
            continue;
        }
        for (auto [v, w, id] : rg[u])
        {
            Dist nd = du + (Dist)w;
            if (!d[v] || nd < *d[v])
            {
                d[v] = nd;
                par[v] = u;
                pe[v] = id;
                q.push({nd, v});
            }
        }
    }
    if (!d[s]) return {};
    struct Node
    {
        // l、r 是左偏堆儿子编号，rk 是右链长度，to 是偏离边终点，val 是偏离代价。
```

```
        int l = 0, r = 0, rk = 1, to = -1;
        Dist val{};
    };
    vector<Node> hp(1);
    hp[0].rk = 0;
    auto meld = [&](auto &&self, int a, int b) -> int
    {
        // a、b 是持久左偏堆根编号；返回合并后新堆根，不修改旧版本。
        if (!a) return b;
        if (!b) return a;
        if (hp[a].val > hp[b].val) swap(a, b);
        int x = hp.size();
        hp.push_back(hp[a]);
        hp[x].r = self(self, hp[x].r, b);
        if (hp[hp[x].l].rk < hp[hp[x].r].rk) swap(hp[x].l, hp[x].r);
        hp[x].rk = hp[hp[x].r].rk + 1;
        return x;
    };
    vector<vector<int>> child(n);
    for (int u = 0; u < n; u++)
    {
        if (par[u] != -1) child[par[u]].push_back(u);
    }
    vector<int> root(n);
    queue<int> bfs;
    bfs.push(t);
    while (!bfs.empty())
    {
        int u = bfs.front();
        bfs.pop();
        if (par[u] != -1) root[u] = meld(meld, root[u], root[par[u]]);
        for (auto [v, w, id] : g[u])
        {
            if (id == pe[u] || !d[v]) continue;
            // extra 是偏离最短路树后增加的非负长度。
            Dist extra = (Dist)w - *d[u] + *d[v];
            hp.push_back({0, 0, 1, v, extra});
            root[u] = meld(meld, root[u], (int)hp.size() - 1);
        }
        for (int v : child[u])
        {
            bfs.push(v);
        }
    }
    hp.push_back({0, 0, 1, s, *d[s]});
    priority_queue<pair<Dist, int>, vector<pair<Dist, int>>, greater<>> pq;
    pq.push({*d[s], (int)hp.size() - 1});
    bool skip0 = s == t && !keep0;
    vector<Dist> ans;
    ans.reserve(k);
    while (!pq.empty())
    {
```

```
        auto [len, x] = pq.top();
        pq.pop();
        if (skip0)
        {
            skip0 = false;
        }
        else
        {
            ans.push_back(len);
            if (ans.size() == k) return ans;
        }
        for (int y : {hp[x].l, hp[x].r})
        {
            if (y) pq.push({len + hp[y].val - hp[x].val, y});
        }
        int r = root[hp[x].to];
        if (r) pq.push({len + hp[r].val, r});
    }
    return ans;
}

template<class Dist = long long>
optional<Dist> eppWalk(int n, const vector<EppsteinEdge> &e, int s, int t,
                                unsigned long long k, bool keep0 = true)
{
    // 参数含义同 eppWalks；返回第 k 小游走长度，不足 k 条时返回空。
    auto ans = eppWalks<Dist>(n, e, s, t, k, keep0);
    if (ans.size() < k) return nullopt;
    return ans.back();
}
```

`eppWalks` 一次返回前至多 k 项，`eppWalk` 是其第 k 项包装；二者与朴素 `kthWalk` 一样枚举允许重边、重点和同权平行边的游走，`keep0` 只影响 $s = t$ 的零长空游走。前者建反图最短路和持久堆用 $O((n+m)\log n)$ 时间，随后连续输出 k 项用 $O(k\log(n+m))$ 时间，持久堆空间为 $O(m\log n)$；后者实现较短，适合 k 很小或仅作原型。官方大规模第 K 短游走应调用批量接口。两者都要求非负边权；要枚举简单路径时必须使用专门算法。

5.2.013 第 K 短简单路 Yen

求有向图（边权非负）中 s 到 t 代价最小的前 k 条简单路径，路径按边编号区分，所以平行边给出不同路径。Yen 算法逐条产生答案：第一条是普通最短路；已得到第 i 条路径 $A_i = (v_0, \dots, v_\ell)$ 后，对每个偏离点 v_j：

1. 根路径取 A_i 的前 j 条边；
2. 对所有已输出且前 j 条边与根路径相同的路径，禁用它们的第 $j+1$ 条边，保证新路径与已有路径不同；
3. 禁用根路径上 v_j 之前的所有点，保证拼接后仍是简单路径；
4. 求 v_j 到 t 的最短路，与根路径拼成候选。

候选放入按（代价，边序列）排序的集合，每轮取出最小者作为下一条答案。任何第 $i+1$ 短简单路都与某条已输出路径有一段最长公共前缀，并在该前缀末端偏离，因此一定作为候选出现过。

```
struct YenPaths
{
    struct Path
    {
        __int128 cost = 0;    // cost 是路径边权和。
        vector<int> vertices; // vertices 是从起点到终点的顶点序列。
        vector<int> edges;    // edges 是依次经过的边编号，平行边据此区分。

        bool operator<(const Path &o) const
        {
            // 先比代价，再比边编号序列，使候选集合有确定的顺序。
            return tie(cost, edges) < tie(o.cost, o.edges);
        }
    };

    int n; // n 是点数。
    vector<array<int, 3>> edges; // edges[id] 的三维依次是有向边的起点、终点和非负边权。
    vector<vector<int>> adj; // adj[u] 是从 u 出发的边编号。

    YenPaths(int n) : n(n), adj(n)
    {
        // n 是点数，点编号为 [0,n)；构造空图。
    }

    int add(int u, int v, int w)
    {
        // 加入有向边 u->v，w 为非负边权；返回边编号。
        assert(0 <= u && u < n && 0 <= v && v < n && w >= 0); // 调试检查，可删。
        edges.push_back({u, v, w});
        adj[u].push_back(edges.size() - 1);
        return edges.size() - 1;
    }

    optional<Path> shortest(int s, int t, const vector<char> &banV, const vector<char> &banE)
    ↪ const
    {
        // 在删去 banV 中的点和 banE 中的边后求 s 到 t 的最短路；不可达时返回空。
        const __int128 INF = (__int128)1 << 120;
        if (banV[s] || banV[t])
        {
            return nullopt;
        }
        vector<__int128> d(n, INF);
        vector<int> pre(n, -1); // pre[v] 是最短路树上进入 v 的边编号。
        vector<char> done(n);
        priority_queue<pair<__int128, int>, vector<pair<__int128, int>>, greater<>> q;
        d[s] = 0;
        q.push({0, s});
        while (!q.empty())
```

```
        {
            auto [du, u] = q.top();
            q.pop();
            if (done[u])
            {
                continue;
            }
            done[u] = true;
            if (u == t)
            {
                break;
            }
            for (int id : adj[u])
            {
                auto [x, v, w] = edges[id];
                if (banE[id] || banV[v] || done[v])
                {
                    continue;
                }
                if (du + w < d[v])
                {
                    d[v] = du + w;
                    pre[v] = id;
                    q.push({d[v], v});
                }
            }
        }
        if (d[t] == INF)
        {
            return nullopt;
        }
        Path p;
        p.cost = d[t];
        for (int u = t;; u = edges[pre[u]][0])
        {
            p.vertices.push_back(u);
            if (u == s)
            {
                break;
            }
            p.edges.push_back(pre[u]);
        }
        reverse(p.vertices.begin(), p.vertices.end());
        reverse(p.edges.begin(), p.edges.end());
        return p;
    }

    vector<Path> solve(int s, int t, int k) const
    {
        // 返回 s 到 t 代价最小的前 k 条互不相同的简单路径，按代价升序；不足 k 条时全部返回。
        assert(0 <= s && s < n && 0 <= t && t < n); // 调试检查，可删。
        vector<Path> ans;
```

```
        vector<char> banV(n), banE(edges.size());
        auto first = shortest(s, t, banV, banE);
        if (k <= 0 || !first)
        {
            return ans;
        }
        ans.push_back(*first);
        set<Path> cand;                // cand 是尚未输出的候选路径。
        set<vector<int>> seen{first->edges}; // seen 是已生成过的边序列，避免重复入候选。
        while ((int)ans.size() < k)
        {
            const Path &last = ans.back();
            __int128 root = 0; // root 是 last 前 i 条边的代价，即根路径代价。
            fill(banV.begin(), banV.end(), 0);
            for (int i = 0; i < (int)last.edges.size(); i++)
            {
                // 偏离点是 last.vertices[i]：禁掉所有共享这段根路径的已选路径的下一条边，
                // 根路径上更早的点已在 banV 中，保证拼接后仍是简单路径。
                fill(banE.begin(), banE.end(), 0);
                for (const Path &p : ans)
                {
                    if ((int)p.edges.size() > i && equal(last.edges.begin(), last.edges.begin()
                     ↪  + i, p.edges.begin()))
                    {
                        banE[p.edges[i]] = 1;
                    }
                }
                auto spur = shortest(last.vertices[i], t, banV, banE);
                if (spur)
                {
                    Path p;
                    p.cost = root + spur->cost;
                    p.edges.assign(last.edges.begin(), last.edges.begin() + i);
                    p.edges.insert(p.edges.end(), spur->edges.begin(), spur->edges.end());
                    p.vertices.assign(last.vertices.begin(), last.vertices.begin() + i);
                    p.vertices.insert(p.vertices.end(), spur->vertices.begin(),
↪  spur->vertices.end());
                    if (seen.insert(p.edges).second)
                    {
                        cand.insert(move(p));
                    }
                }
                banV[last.vertices[i]] = 1;
                root += edges[last.edges[i]][2];
            }
            if (cand.empty())
            {
                break;
            }
            ans.push_back(*cand.begin());
            cand.erase(cand.begin());
        }
```

```
        return ans;
    }
};
```

`YenPaths g(n)` 后用 `g.add(u,v,w)` 加入有向边并得到边编号，`g.solve(s,t,k)` 按代价升序返回至多 k 条路径，每条含 `cost`、`vertices` 与 `edges`；不足 k 条时全部返回，$s=t$ 时只有一条零长路径。无向图把每条边按两个方向各加一次即可。设路径边数不超过 n，每条答案做至多 n 次 Dijkstra，时间 $O(kn\,m\log m)$；候选集合最多 $O(kn)$ 条路径，空间 $O(kn^2+m)$。

允许重复经过点的第 k 短路（游走）见 5.2.010 第 K 短路（Dijkstra / A*）与 5.2.012 第 K 短游走（Eppstein），它们比 Yen 快但不能保证简单。下面是 Yen 论文常用的 6 点示例：

```
signed main()
{
    // C=0,D=1,E=2,F=3,G=4,H=5, 求 C 到 H 的前 3 条简单路。
    YenPaths g(6);
    for (auto [u, v, w] : vector<array<int, 3>>{{0, 1, 3}, {0, 2, 2}, {1, 3, 4}, {2, 1, 1}, {2, 3, 2},
                                                {2, 4, 3}, {3, 4, 2}, {3, 5, 1}, {4, 5, 2}})
    {
        g.add(u, v, w);
    }
    auto p = g.solve(0, 5, 3);
    assert(p.size() == 3);
    assert(p[0].cost == 5 && p[0].vertices == vector<int>({0, 2, 3, 5}));
    assert(p[1].cost == 7 && p[1].vertices == vector<int>({0, 2, 4, 5}));
    assert(p[2].cost == 8 && p[2].vertices == vector<int>({0, 1, 3, 5}));
}
```

5.3 连通性

5.3.001 Tarjan 强连通分量

Tarjan 一次 DFS 就求出有向图的强连通分量。给每点记进入时间戳 dfn_u 与「不经过已封闭分量、能回溯到的最早时间戳」low_u，并把访问中的点压入栈。回溯时若 $low_u=dfn_u$，说明 u 是当前栈中一个封闭强连通分量的根——从栈顶连续弹出直到弹出 u，这批点两两可达且不可能再与栈外点互达，即一个 SCC。分量编号按弹栈顺序递增，天然是缩点 DAG 的逆拓扑序，缩点后可直接做 DAG DP。

```
vector<int> tarjanSCC(const vector<vector<int>> &g)
{
    // g 是有向图邻接表；返回每点的强连通分量编号，编号按分量弹栈顺序递增。
    int n = g.size(), ti = 0, cc = 0;
    vector<int> dfn(n), low(n), st, bel(n, -1);
    vector<bool> on(n);
    auto dfs = [&](auto &&self, int u) -> void
    {
        // u 是当前点；完成以 u 为根的 Tarjan 搜索并弹出已封闭分量，无返回值。
        dfn[u] = low[u] = ++ti;
```

```
        st.push_back(u);
        on[u] = true;
        for (auto v : g[u])
        {
            if (!dfn[v])
            {
                self(self, v);
                chmin(low[u], low[v]);
            }
            else if (on[v])
            {
                chmin(low[u], dfn[v]);
            }
        }
        if (low[u] == dfn[u])
        {
            return;
        }
        while (true)
        {
            int v = st.back();
            st.pop_back();
            on[v] = false;
            bel[v] = cc;
            if (v == u)
            {
                break;
            }
        }
        cc++;
    };
    for (int u = 0; u < n; u++)
    {
        if (!dfn[u])
        {
            dfs(dfs, u);
        }
    }
    return bel;
}
```

`tarjanSCC(g)` 返回每点的分量编号，允许图不连通、自环和重边；编号只保证「同分量相等」，缩点建 DAG 时须按分量编号重新连边去重。$O(V+E)$ 时间与空间。有向边 `0→1,1→0,1→2,2→3,3→2` 得两个 SCC `{0,1}` 与 `{2,3}`；5.3.002 Kosaraju 强连通分量、5.3.003 Bitset 优化 Kosaraju 解决同一问题，各版都应单独用这个已知分组检查。需要按分量拓扑序正序处理时，Kosaraju 直接给出正序；无向图找桥/割点是另一套 low 值判据，见 5.3.004 割边与边双连通分量、5.3.005 割点与点双连通分量。

5.3.002 Kosaraju 强连通分量

Kosaraju 用两遍 DFS 求 SCC，是 Tarjan 的替代。第一遍在原图 DFS，按结束时间把点压入 `ord`；缩点 DAG 的源分量必然最后结束。第二遍按结束时间逆序（`ord` 反转）在反图 DFS，每从一个未定点出发所能到达的点集恰是一个 SCC——因为反图上从源分量出发无法越过 DAG 的边界。它比 Tarjan 直观、但需要显式建反图，分量编号按缩点 DAG 的拓扑正序递增，正好与 Tarjan 相反。

```cpp
vector<int> kosarajuSCC(const vector<vector<int>> &g)
{
    // g 是有向图邻接表；返回每点的强连通分量编号，编号按缩点 DAG 拓扑序递增。
    int n = g.size(), cc = 0;
    vector<vector<int>> rg(n);
    for (int u = 0; u < n; u++)
    {
        for (auto v : g[u])
        {
            rg[v].push_back(u);
        }
    }
    vector<int> ord, bel(n, -1);
    vector<bool> vis(n);
    auto dfs1 = [&](auto &&self, int u) -> void
    {
        // u 是原图当前点；按 DFS 结束时间把点加入 ord，无返回值。
        vis[u] = true;
        for (auto v : g[u])
        {
            if (!vis[v])
            {
                self(self, v);
            }
        }
        ord.push_back(u);
    };
    auto dfs2 = [&](auto &&self, int u) -> void
    {
        // u 是反图当前点；把本次可达点标为同一强连通分量，无返回值。
        bel[u] = cc;
        for (auto v : rg[u])
        {
            if (bel[v] == -1)
            {
                self(self, v);
            }
        }
    };
    for (int u = 0; u < n; u++)
    {
        if (!vis[u])
        {
            dfs1(dfs1, u);
```

```
        }
    }
    reverse(ord.begin(), ord.end());
    for (auto u : ord)
    {
        if (bel[u] == -1)
        {
            dfs2(dfs2, u);
            cc++;
        }
    }
    return bel;
}
```

`kosarajuSCC(g)` 返回每点分量编号，编号按缩点 DAG 拓扑正序递增；$O(V+E)$ 时间与空间，需额外存反图，递归深度可达点数（深链要防爆栈）。只关心分组且想省掉反图时用 5.3.001 Tarjan 强连通分量；需要拓扑正序编号（如按序 DP）则 Kosaraju 更省事。点数很小的稠密图可用位运算并行的 5.3.003 Bitset 优化 Kosaraju。

5.3.003 Bitset 优化 Kosaraju

当点数小到能压位时，可以用位集传递闭包求 SCC。`reach[u]` 是 u 能到达的点集，用位集版 Floyd（枚举中间点 k，若 u 能到 k 就把 $reach[k]$ 或进 $reach[u]$）求出全部可达关系；随后 u, v 同属一个 SCC 当且仅当互相可达，互达是等价关系可直接分组。它把 SCC 转成 $O(V^3/64)$ 的闭包运算，只在稠密小图（V 几百内）划算，不是一般大图的默认做法——那时线性的 Tarjan/Kosaraju 远快。

```
vector<int> bitsetSCC(const vector<vector<int>> &g)
{
    // g 是点数较小的稠密有向图；返回每点强连通分量编号。
    int n = g.size(), w = (n + 63) / 64;
    vector reach(n, vector<unsigned long long>(w));
    for (int u = 0; u < n; u++)
    {
        reach[u][u / 64] |= 1ULL << (u % 64);
        for (auto v : g[u])
        {
            reach[u][v / 64] |= 1ULL << (v % 64);
        }
    }
    for (int k = 0; k < n; k++)
    {
        for (int u = 0; u < n; u++)
        {
            if (!(reach[u][k / 64] >> (k % 64) & 1))
            {
                continue;
            }
            for (int b = 0; b < w; b++)
            {
```

```
                reach[u][b] |= reach[k][b];
            }
        }
    }
    vector<int> bel(n, -1);
    int cc = 0;
    for (int u = 0; u < n; u++)
    {
        if (bel[u] != -1)
        {
            continue;
        }
        for (int v = u; v < n; v++)
        {
            bool uv = reach[u][v / 64] >> (v % 64) & 1;
            bool vu = reach[v][u / 64] >> (u % 64) & 1;
            if (uv && vu)
            {
                bel[v] = cc;
            }
        }
        cc++;
    }
    return bel;
}
```

`bitsetSCC(g)` 返回每点分量编号（编号按首个未定点顺序给出，不承诺拓扑序），仅在 $O(V^2/64)$ 位内存与 $O(V^3/64)$ 时间可接受时使用。点数一大就该回到线性的 5.3.001 Tarjan 强连通分量 或 5.3.002 Kosaraju 强连通分量；此处顺带求出的传递闭包本身可复用（如判两点可达性），这是位集法相对前两者的额外收益。

5.3.004 割边与边双连通分量

桥是删掉后连通块数增加的边，边双连通分量是删去所有桥后剩下的极大连通块。判据用 DFS 树的 low 值：树边 $u \to v$ 满足 $low_v > dfn_u$ 时是桥，因为 v 的子树没有任何返祖边能绕过这条树边回到 u 或更早。重边必须靠边编号区分（`id==pe` 跳的是父边编号而非父点，才不会把两条平行边误判成桥）。求出桥后再做一次「不走桥」的遍历，每个连通块即一个边双。

```
class EdgeBCC
{
    // n 是原图点数，ti 是 DFS 时间戳。
    int n, ti = 0;
    // ed[i][0..1] 依次是第 i 条无向边的两个端点。
    vector<array<int, 2>> ed;
    // g[u] 的每项依次是邻点与无向边编号。
    vector<vector<array<int, 2>>> g;
    // dfn[u] 是进入时间，low[u] 是不经过父边能到达的最早时间。
    vector<int> dfn, low;
    // bridge[i] 标记第 i 条边是否为桥。
```

```
    vector<bool> bridge;

    void dfs(int u, int pe)
    {
        // u 是当前点，pe 是进入 u 的边编号；计算 low 并标记桥，无返回值。
        dfn[u] = low[u] = ++ti;
        for (auto [v, id] : g[u])
        {
            if (id == pe)
            {
                continue;
            }
            if (!dfn[v])
            {
                dfs(v, id);
                chmin(low[u], low[v]);
                if (low[v] > dfn[u])
                {
                    bridge[id] = true;
                }
            }
            else
            {
                chmin(low[u], dfn[v]);
            }
        }
    }

  public:
    EdgeBCC(int n) : n(n), g(n), dfn(n), low(n)
    {
        // n 是点数；构造一张无边图，无返回值。
    }

    int addEdge(int u, int v)
    {
        // u、v 是 0 下标端点；加入无向边并返回边编号，允许重边。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        int id = ed.size();
        ed.push_back({u, v});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        return id;
    }

    void build()
    {
        // 无参数；重新计算全部桥，无返回值。
        ti = 0;
        fill(dfn.begin(), dfn.end(), 0);
        fill(low.begin(), low.end(), 0);
        bridge.assign(ed.size(), false);
```

```
        for (int u = 0; u < n; u++)
        {
            if (!dfn[u])
            {
                dfs(u, -1);
            }
        }
    }

    const vector<bool> &bridges() const
    {
        // 无参数；返回按边编号排列的桥标记。
        return bridge;
    }

    vector<int> comps() const
    {
        // 无参数；返回删除全部桥后每个点的边双连通分量编号。
        vector<int> bel(n, -1);
        int cc = 0;
        for (int s = 0; s < n; s++)
        {
            if (bel[s] != -1)
            {
                continue;
            }
            queue<int> q;
            q.push(s);
            bel[s] = cc++;
            while (!q.empty())
            {
                int u = q.front();
                q.pop();
                for (auto [v, id] : g[u])
                {
                    if (!bridge[id] && bel[v] == -1)
                    {
                        bel[v] = bel[u];
                        q.push(v);
                    }
                }
            }
        }
        return bel;
    }
};
```

构造 `EdgeBCC q(n)`，逐边 `addEdge(u,v)` 后 `build()` 重算，之后 `bridges()[id]` 判第 `id` 条边是否为桥、`comps()` 返回各点的边双编号；加边后必须重新 `build()`。$O(V+E)$ 时间与空间。点数 5、边 `0-1,1-2,2-0,1-3,3-4`：桥是后两条，边双里 $0,1,2$ 同号而 $3,4$ 各自成块。求的是割点/点双而非割边/边双时用 5.3.005 割点与点双连通分量（判据是 $low_v \geq dfn_u$，多一个等号）；把边双缩点连桥可得块割树，见

5.3.006 缩点与块割树。

5.3.005 割点与点双连通分量

割点是删去后连通块数增加的点，点双连通分量是极大的「任意两点间有两条内部不相交路径」的块；一个割点可同属多个点双。判据是树边 $u \to v$ 满足 $low_v \geq dfn_u$（比桥的严格大于多一个等号）：此时把边栈弹到这条树边为止，弹出的边所涉点集就是一个点双，且 u 是割点。根节点特殊——它只有在 DFS 儿子数 > 1 时才是割点。孤立点单独作为一个点双输出。栈存的是边编号而非点，才能正确处理割点被多个点双共享的情形。

```
class VertexBCC
{
    // n 是原图点数，ti 是 DFS 时间戳。
    int n, ti = 0;
    // ed[i][0..1] 依次是第 i 条无向边的两个端点。
    vector<array<int, 2>> ed;
    // g[u] 的每项依次是邻点与边编号。
    vector<vector<array<int, 2>>> g;
    // dfn、low 分别保存进入时间和 Lowlink 值，cut 标记割点。
    vector<int> dfn, low;
    vector<bool> cut;
    // st 保存尚未归入点双的边编号，bcc 保存各点双的点集。
    vector<int> st;
    vector<vector<int>> bcc;

    void dfs(int u, int pe)
    {
        // u 是当前点，pe 是父边编号；划分点双并标记割点，无返回值。
        dfn[u] = low[u] = ++ti;
        int son = 0;
        for (auto [v, id] : g[u])
        {
            if (id == pe)
            {
                continue;
            }
            if (!dfn[v])
            {
                son++;
                st.push_back(id);
                dfs(v, id);
                chmin(low[u], low[v]);
                if (low[v] >= dfn[u])
                {
                    if (pe != -1 || son > 1)
                    {
                        cut[u] = true;
                    }
                    vector<int> cur;
                    while (true)
```

```
                    {
                        int x = st.back();
                        st.pop_back();
                        cur.push_back(ed[x][0]);
                        cur.push_back(ed[x][1]);
                        if (x == id)
                        {
                            break;
                        }
                    }
                    sort(cur.begin(), cur.end());
                    cur.erase(unique(cur.begin(), cur.end()), cur.end());
                    bcc.push_back(move(cur));
                }
            }
            else if (dfn[v] < dfn[u])
            {
                st.push_back(id);
                chmin(low[u], dfn[v]);
            }
        }
        if (pe == -1 && son == 0)
        {
            bcc.push_back({u});
        }
    }

public:
    VertexBCC(int n) : n(n), g(n), dfn(n), low(n), cut(n)
    {
        // n 是点数；构造一张无边图，无返回值。
    }

    int addEdge(int u, int v)
    {
        // u、v 是 0 下标端点；加入无向边并返回边编号，允许重边。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        int id = ed.size();
        ed.push_back({u, v});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        return id;
    }

    void build()
    {
        // 无参数；清空旧结果并重新求割点和点双，无返回值。
        ti = 0;
        fill(dfn.begin(), dfn.end(), 0);
        fill(low.begin(), low.end(), 0);
        fill(cut.begin(), cut.end(), false);
        st.clear();
```

```
        bcc.clear();
        for (int u = 0; u < n; u++)
        {
            if (!dfn[u])
            {
                dfs(u, -1);
            }
        }
    }

    const vector<bool> &cutVertices() const
    {
        // 无参数；返回按点编号排列的割点标记。
        return cut;
    }

    const vector<vector<int>> &comps() const
    {
        // 无参数；返回全部点双连通分量，每项是去重后的原图点编号。
        return bcc;
    }
};
```

构造 `VertexBCC q(n)`，加完边后 `build()`，从 `cutVertices()[u]` 读割点标记、`comps()` 读各点双点集（每项去重）；自环需按题意预处理。$O(V+E)$ 时间与空间，每条边只入栈出栈一次。对边 `0-1,1-2,2-0,1-3,3-4` 的图，割点为 $1,3$，点双点集为 `{0,1,2}`、`{1,3}`、`{3,4}`。把点双与割点建成树即 5.3.006 缩点与块割树 或 5.3.007 圆方树、广义圆方树；求割边/边双而非割点/点双时用 5.3.004 割边与边双连通分量。

5.3.006 缩点与块割树

块割树把每个点双缩成一个块节点，与它所含的原图点相连，从而把「原图经割点串联的块结构」变成一棵树。因为不同点双至多共享一个割点，加入块节点后整体无环——原图每个连通块对应一棵块割树，圆点（原图点）与方点（块节点）交替，故也称二分结构。构造只是把每个点双 i 建成新点 $n+i$，向其点集里每个原图点连无向边。

```
vector<vector<int>> blockCutTree(int n, const vector<vector<int>> &bcc)
{
    // n 是原图点数，bcc[i] 是第 i 个点双的点集；返回 n+|bcc| 个点的块割树邻接表。
    vector<vector<int>> tr(n + bcc.size());
    for (int i = 0; i < (int)bcc.size(); i++)
    {
        int b = n + i;
        for (auto u : bcc[i])
        {
            tr[u].push_back(b);
            tr[b].push_back(u);
        }
    }
    return tr;
```

```
}
```

先由 5.3.005 割点与点双连通分量 得到 bcc，再 `blockCutTree(n, bcc)` 返回 $n + |bcc|$ 个点的邻接表：编号 $< n$ 是原图点，其余依次对应点双。$O(V + S)$ 时间与空间，S 为全部点双大小之和，无向图中 $S = O(V + E)$。同一份拓扑若保留「圆点/方点」语义用于树上路径限制，就是 5.3.007 圆方树、广义圆方树——两者代码一致，区别在方点是否被赋予点双相关的权信息。

5.3.007 圆方树、广义圆方树

圆方树把原图每个点作为圆点、每个点双作为方点，圆点与包含它的方点相连。它与 5.3.006 缩点与块割树 的拓扑完全相同，价值在语义：路径一旦进入某方点就代表在对应点双内自由移动，于是「经过割点的连通/路径限制」被转化为树上路径问题，可套 LCA、树上差分等工具。广义圆方树在此基础上按题意给方点赋权（如点双大小、内部最短环长），因此方点权不能简单复制某个圆点的权。

实现直接复用 5.3.006 缩点与块割树 的 `blockCutTree(n,bcc)`：返回图中编号 $[0, n)$ 的点作为圆点，第 i 个点双对应编号 $n + i$ 的方点。圆方树与块割树在这里是同一份拓扑，差别只在使用语义；广义圆方树还要按题意给每个方点汇总点双权值，不能复制某个圆点权。构造用 $O(V + S)$ 时间与空间，S 为全部点双大小之和；仙人掌的环结构见 5.3.010 仙人掌森林分解。

5.3.008 无向图连通度、点连通度、边连通度

全局连通度问至少删多少条边或多少个点才能使无向图不连通。边连通度 λ 就是全局最小割，调用 Stoer-Wagner。点连通度 κ 用拆点最大流：每点拆成入点、出点并连容量 1，指定源汇的内部边和原图边容量取 n，再枚举所有非相邻点对；完全图单独返回 $n - 1$。全局值不能用一次任意源汇最小割代替。本条只保留建模入口，底层统一引用 5.8 的主实现。

```
// 依赖 5.8.007 Stoer-Wagner 全局最小割
template <class T>
struct Flow;

struct GlobalCut;

GlobalCut stoerWagner(vector<vector<int>> a);

int edgeConn(int n, const vector<tuple<int, int, long long>> &e)
{
    // e 是非负权无向边；返回全局边连通度。
    vector a(n, vector<int>(n));
    for (auto [u, v, w] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && w >= 0); // 调试检查，可删。
        if (u != v)
        {
            a[u][v] += w;
            a[v][u] = a[u][v];
        }
    }
```

```
    return stoerWagner(move(a)).w;
}

template <class F = Flow<long long>>
int vertexConn(const vector<vector<int>> &g)
{
    // g 是简单无向图邻接表；返回全局点连通度。
    int n = g.size();
    if (n <= 1)
    {
        return 0;
    }
    vector a(n, vector<bool>(n));
    int m = 0;
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u])
        {
            assert(0 <= v && v < n); // 调试检查，可删。
            if (u == v)
            {
                continue;
            }
            if (!a[u][v])
            {
                a[u][v] = true;
                a[v][u] = true;
                m++;
            }
        }
    }
    if (m == n * (n - 1) / 2)
    {
        return n - 1;
    }
    int ans = n - 1;
    for (int s = 0; s < n; s++)
    {
        for (int t = s + 1; t < n; t++)
        {
            if (a[s][t])
            {
                continue;
            }
            F f(2 * n);
            for (int u = 0; u < n; u++)
            {
                f.add(2 * u, 2 * u + 1, u == s || u == t ? n : 1);
            }
            for (int u = 0; u < n; u++)
            {
                for (int v = u + 1; v < n; v++)
```

```
                {
                    if (a[u][v])
                    {
                        f.add(2 * u + 1, 2 * v, n);
                        f.add(2 * v + 1, 2 * u, n);
                    }
                }
            }
            ans = min(ans, f.flow(2 * s + 1, 2 * t));
        }
    }
    return ans;
}
```

本条提前声明接口，完整实现仍统一放在 5.8.001 Dinic 最大流 的 `Flow` 与 5.8.007 Stoer-Wagner 全局最小割；调用点须写在这两条定义之后。`edgeConn(n,e)` 接受非负 64 位边权，自环不影响割；调用者必须保证平行边与收缩过程中的容量和可由 64 位有符号整数表示。`vertexConn(g)` 处理简单无向图，$n \le 1$ 返回 0，完全图返回 $n-1$。边连通度为 $O(n^3)$；点连通度按通用界为 $O(n^4(n+m))$，仅适合小图。

5.3.009 最佳/最小点割集与边割集

要真正取出一组割而不只是割的大小，靠的是最大流最小割的对偶：把「删点」和「删边」都翻译成容量约束，最小割中被用满的那批容量就是要删的对象。求边割时直接给每条边赋容量、原点保持不动，跑源汇最大流后残量图里从源可达一侧与不可达一侧之间的满流边即为边割集。求点割时先把每个可删点 u 拆成 $u_{in} \to u_{out}$ 一条容量为其代价的内部边、原图的边设成无穷容量，这样最小割只能切在拆点边上，被切的拆点边对应的原点就是点割集；指定点对要断开时把该点对的拆点边设成无穷以禁止割掉端点自身。

这套建模直接复用 5.8.001 Dinic 最大流 的 `Flow` 与 5.8.007 Stoer-Wagner 全局最小割，不再重复代码。求指定点对边割时跑 `Flow::flow`，再用 `cut(源)` 分出残量源侧，跨越两侧的正容量原边就是一组最小边割；不指定点对时，`GlobalCut.side` 已给出全局最小割的一侧。求点割时在拆点图上取被割断的内部边，枚举非相邻点对取最小者。

复杂度与 5.3.008 相同：边割由 Stoer-Wagner 求得，$O(n^3)$ 时间、$O(n^2)$ 空间；点割枚举 $O(n^2)$ 个点对，拆点图有 $2n$ 个点和 $O(n+m)$ 条边，代入本章 Dinic 的通用界后为 $O(n^4(n+m))$ 时间、$O(n+m)$ 空间；输出割集只需在此基础上再做一次 $O(n+m)$ 的残量图遍历。不同评测机上的可用规模会受常数影响，但不改变这里的渐进上界。若只要连通度数值而不要具体割集，直接调用 5.3.008 即可；需要把断开代价推广到「删点或删边任选」的混合模型时，仍是本条的拆点加边建图，只是给拆点边与原边各自赋上对应代价。

5.3.010 仙人掌森林分解

仙人掌是每条边至多属于一个简单环的简单无向图；本实现也接受多个连通块，得到仙人掌森林。DFS 维护 `dfn`、`low` 与边栈，当 `low[v]>=dfn[u]` 时弹出一个点双分量。仙人掌中每个含多条边的点双必须恰为一个简单环，所以其边数等于不同端点数。端点去重使用时间戳数组，整段保持线性复杂度。标题只写实际提供的仙人掌分解，不再保留没有对应代码的 “AFL” 。

```cpp
struct Cactus
{
    // n 是图的点数，ti 是 DFS 时间戳。
    int n, ti = 0;
    // e[id] 是第 id 条无向边的两个端点。
    vector<pair<int, int>> e;
    // g[u] 的每项是邻点及无向边编号。
    vector<vector<pair<int, int>>> g;
    // Tarjan 时间戳、边栈及端点去重时间戳。
    vector<int> dfn, low, st, mark;
    int tag = 0;
    // cycles 的每项是一条简单环的边编号集合。
    vector<vector<int>> cycles;
    // ok 表示每个非桥点双是否都是单环。
    bool ok = true;

    Cactus(int n) : n(n), g(n), dfn(n), low(n), mark(n)
    {
        // n 是图的点数；构造一张没有边的图。
    }

    void add(int u, int v)
    {
        // u、v 是简单无向边端点；加入一条边。
        assert(0 <= u && u < n && 0 <= v && v < n && u != v); // 调试检查，可删。
        int id = e.size();
        e.push_back({u, v});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
    }

    void dfs(int u, int pe)
    {
        // u 是当前点，pe 是父边编号；划分以 u 为根的点双。
        dfn[u] = low[u] = ++ti;
        for (auto [v, id] : g[u])
        {
            if (id == pe)
            {
                continue;
            }
            if (!dfn[v])
            {
                st.push_back(id);
                dfs(v, id);
                low[u] = min(low[u], low[v]);
                if (low[v] >= dfn[u])
                {
                    vector<int> q;
                    while (true)
                    {
                        int x = st.back();
```

```
                        st.pop_back();
                        q.push_back(x);
                        if (x == id)
                        {
                            break;
                        }
                    }
                    if (q.size() > 1)
                    {
                        tag++;
                        int cnt = 0;
                        for (int x : q)
                        {
                            for (int v : {e[x].first, e[x].second})
                            {
                                if (mark[v] != tag)
                                {
                                    mark[v] = tag;
                                    cnt++;
                                }
                            }
                        }
                        if (cnt != (int)q.size())
                        {
                            ok = false;
                        }
                        else
                        {
                            cycles.push_back(q);
                        }
                    }
                }
            }
            else if (dfn[v] < dfn[u])
            {
                st.push_back(id);
                low[u] = min(low[u], dfn[v]);
            }
        }
    }

    bool build()
    {
        // 无参数；重新分解全部点双，返回图是否为仙人掌。
        ti = 0;
        fill(dfn.begin(), dfn.end(), 0);
        fill(low.begin(), low.end(), 0);
        fill(mark.begin(), mark.end(), 0);
        st.clear();
        cycles.clear();
        ok = true;
        tag = 0;
```

```
        for (int i = 0; i < n; i++)
        {
            if (!dfn[i])
            {
                dfs(i, -1);
            }
        }
        return ok;
    }
};
```

用 `add` 建简单无向图，`build()` 可重复调用；每次都会清空全部运行态。返回 `true` 时，`cycles` 的每项是一条环的原边编号，桥边不出现；返回 `false` 表示某个点双不是单环。允许图不连通，但不允许自环或重边。时间与空间均为 $O(n+m)$。一般点双见 5.3.005 割点与点双连通分量，只有一个环的连通图可用 5.9.008 基环树。

5.3.011 补图连通块

当原图稀疏而补图稠密（补图有近 n^2 条边）时，显式建补图再跑连通块不可行。诀窍是维护一个「尚未归类点」的有序集合 `left`，每次从中取一点做 BFS：先用一个时间戳 `tim` 把该点在原图中的邻居打上标记，然后遍历 `left`，凡是没被打上标记的点就是补图邻居，从 `left` 里删掉并入队。由于每个点一旦入队就永久离开 `left`，`left` 的总删除次数是 $O(n)$，而每次 BFS 扩展时对某点的原图邻居打标记的总量是 $O(m)$，于是不必真正枚举补图的 $O(n^2)$ 条边就能把补图连通块全部找出。

```
vector<vector<int>> complementCC(const vector<vector<int>> &g)
{
    // g 是原图邻接表；返回补图的全部连通块，每个点恰出现一次。
    int n = g.size();
    set<int> left;
    for (int u = 0; u < n; u++)
    {
        left.insert(u);
    }
    vector<int> mark(n);
    int tim = 0;
    vector<vector<int>> ans;
    while (!left.empty())
    {
        int s = *left.begin();
        left.erase(s);
        ans.push_back({});
        queue<int> q;
        q.push(s);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            ans.back().push_back(u);
            tim++;
```

```
            for (int v : g[u])
            {
                mark[v] = tim;
            }
            for (auto it = left.begin(); it != left.end();)
            {
                int v = *it;
                if (mark[v] != tim)
                {
                    it = left.erase(it);
                    q.push(v);
                }
                else
                {
                    it++;
                }
            }
        }
    }
    return ans;
}
```

`complementCC(g)` 传入原图邻接表，返回补图的全部连通块，每个点恰出现一次；`g` 里的重复边或自环不影响结果，因为判定只看「是否被标记」。集合遍历配合 `set` 的删除，最坏 $O((n+m)\log n)$ 时间、$O(n+m)$ 空间；把 `set` 换成带链表的并查集式结构可去掉对数因子。这里求的是无向图补图连通性，与原图上的连通分量（直接 BFS / 5.3.004 割边与边双连通分量 的连通框架）互补；若要的是有向图强连通而非补图连通，应改用 5.3.001 Tarjan 强连通分量。

5.3.012 离线增量 SCC

已知有向边按编号 $0, 1, \dots, m-1$ 依次加入，要求每条边最早在哪个前缀中处于有向环。对时间区间分治：在中点加入当前候选中编号小于中点的边并求 SCC；端点已经同分量的边进入左半继续找更早时间，其余边把端点缩成当前分量后进入右半。每层所有候选边只出现一次，缩点保证右半保留后续成环所需的全部可达关系。

```
// 依赖 5.3.001 Tarjan 强连通分量
vector<int> incrSCC(int n, const vector<pair<int, int>> &e)
{
    // e[i] 是第 i 次加入的有向边；返回它最早成环时的前缀长度，m+1 表示始终不成环。
    int m = e.size();
    vector<int> ans(m, m + 1), id(n, -1);
    vector<tuple<int, int, int>> all;
    for (int i = 0; i < m; i++)
    {
        auto [u, v] = e[i];
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        all.push_back({i, u, v});
    }
    auto solve = [&](auto &&self, const vector<tuple<int, int, int>> &a, int l, int r) -> void
```

```
    {
        if (a.empty() || r == l + 1)
        {
            return;
        }
        int mid = (l + r) / 2;
        int cnt = 0;
        for (auto [i, u, v] : a)
        {
            if (id[u] == -1)
            {
                id[u] = cnt++;
            }
            if (id[v] == -1)
            {
                id[v] = cnt++;
            }
        }
        vector<vector<int>> g(cnt);
        for (auto [i, u, v] : a)
        {
            if (i < mid)
            {
                g[id[u]].push_back(id[v]);
            }
        }
        vector<int> bel = tarjanSCC(g);
        vector<tuple<int, int, int>> left, right;
        for (auto [i, u, v] : a)
        {
            int x = id[u], y = id[v];
            if (i < mid && bel[x] == bel[y])
            {
                chmin(ans[i], mid);
                left.push_back({i, x, y});
            }
            else
            {
                right.push_back({i, bel[x], bel[y]});
            }
        }
        for (auto [i, u, v] : a)
        {
            id[u] = id[v] = -1;
        }
        self(self, left, l, mid);
        self(self, right, mid, r);
    };
    solve(solve, all, 0, m + 1);
    return ans;
}
```

`incrSCC(n,e)` 的答案 `t` 表示加入 `e[0..t)` 后该边的两端首次同属一个 SCC；自环答案是 `i+1`，始终不成环时为 `m+1`。这是离线算法，不支持插边后立即索取当前全部分量编号；那种小规模需求可每次重跑 5.3.001 Tarjan 强连通分量。时间 $O((n+m)\log(m+1))$、空间 $O(n+m)$，递归层数 $O(\log m)$。

下面用五条边检查三点环、之后并入第四点的环，以及始终不能成环的边：

```
signed main()
{
    vector<pair<int, int>> e{{0, 1}, {1, 2}, {2, 0}, {2, 3}, {3, 2}, {0, 0}};
    assert(incrSCC(4, e) == vector<int>({3, 3, 3, 5, 5, 6}));
}
```

5.3.013 三边连通分量

三边连通要求任意两点间至少有三条边不相交路径，强于 5.3.004 割边与边双连通分量 的两边连通。DFS 维护返祖边数 `deg`、最低返祖时间 `low` 与尚未并入的祖先链 `path`；一条子树只有至多一条返祖边时可把它的约束上提，否则按最低返祖边把整段祖先链与当前点合并。并查集的代表即最终三边连通分量。

```
vector<vector<int>> threeEdgeCC(const vector<vector<int>> &g)
{
    // g 是允许重边和自环的无向邻接表；返回任意顺序的三边连通点集划分。
    int n = g.size(), timer = 0;
    vector<int> in(n), out(n), low(n, n), deg(n), path(n, n), nxt(n);
    vector<char> vis(n);
    iota(nxt.begin(), nxt.end(), 0);
    auto find = [&](auto &&self, int x) -> int
    {
        return nxt[x] == x ? x : nxt[x] = self(self, nxt[x]);
    };
    auto join = [&](int x, int y)
    {
        x = find(find, x);
        y = find(find, y);
        if (x != y)
        {
            nxt[y] = x;
            deg[x] += deg[y];
        }
    };
    auto dfs = [&](auto &&self, int v, int p) -> void
    {
        vis[v] = true;
        in[v] = timer++;
        for (int w : g[v])
        {
            if (w == v)
            {
                continue;
            }
            if (w == p)
```

```
            {
                p = n;
                continue;
            }
            if (vis[w])
            {
                if (in[w] < in[v])
                {
                    deg[v]++;
                    low[v] = min(low[v], in[w]);
                }
                else
                {
                    deg[v]--;
                    int u = path[v];
                    while (u != n && in[u] <= in[w] && in[w] < out[u])
                    {
                        join(v, u);
                        u = path[u];
                    }
                    path[v] = u;
                }
                continue;
            }
            self(self, w, v);
            if (path[w] == n && deg[w] <= 1)
            {
                deg[v] += deg[w];
                low[v] = min(low[v], low[w]);
            }
            else
            {
                if (deg[w] == 0)
                {
                    w = path[w];
                }
                if (low[w] < low[v])
                {
                    low[v] = low[w];
                    swap(w, path[v]);
                }
                while (w != n)
                {
                    join(v, w);
                    w = path[w];
                }
            }
        }
        out[v] = timer;
    };
    for (int i = 0; i < n; i++)
    {
```

```
        if (!vis[i])
        {
            dfs(dfs, i, n);
        }
    }
    vector<vector<int>> ans(n);
    for (int i = 0; i < n; i++)
    {
        ans[find(find, i)].push_back(i);
    }
    ans.erase(remove_if(ans.begin(), ans.end(), [](const vector<int> &a)
                        { return a.empty(); }), ans.end());
    return ans;
}
```

`threeEdgeCC(g)` 的 `g` 是无向邻接表，每条无向边必须在两个端点各出现一次；允许重边和自环，返回分量顺序与分量内点顺序均未规定。时间 $O(n+m\alpha(n))$、额外空间 $O(n+m)$，适用于 $n, m \le 2\times 10^5$。只需要按桥切开的边双连通分量时用 5.3.004 割边与边双连通分量，不能把它替代本条。

5.4 2-SAT

5.4.001 Tarjan 2-SAT

2-SAT 要给每个布尔变量定真值以同时满足一批二元子句。把变量 x 拆成两个文字点 $2x$（假）与 $2x+1$（真），子句 $a \vee b$ 等价于 $\neg a \to b$ 及其逆否 $\neg b \to a$。可满足当且仅当没有变量的两个文字落在同一强连通分量里。Tarjan 按分量弹栈顺序编号：缩点 DAG 的汇分量先弹出、编号更小，因此每对互否文字中选编号更小者为真；代码里的比较式正按这个方向赋值。

```
class TarjanTwoSAT
{
    // n 是布尔变量数，g 是 2n 个文字构成的蕴含图。
    int n;
    vector<vector<int>> g;

  public:
    TarjanTwoSAT(int n) : n(n), g(2 * n)
    {
        // n 是变量数；构造没有子句的 2-SAT，无返回值。
    }

    void imply(int x, bool xv, int y, bool yv)
    {
        // x=xv 是前件，y=yv 是后件；加入蕴含及其逆否边。
        int a = 2 * x + xv, b = 2 * y + yv;
        g[a].push_back(b);
        g[b ^ 1].push_back(a ^ 1);
    }

    void addOr(int x, bool xv, int y, bool yv)
```

```
    {
        // x=xv、y=yv 是两个文字；加入二元子句 `(x=xv) or (y=yv)`。
        imply(x, !xv, y, yv);
    }

    optional<vector<bool>> solve() const
    {
        // 无参数；返回一组可行变量值，不可满足时返回空。
        int ti = 0, cc = 0;
        vector<int> dfn(2 * n), low(2 * n), st, bel(2 * n, -1);
        vector<bool> on(2 * n);
        auto dfs = [&](auto &&self, int u) -> void
        {
            // u 是当前文字；执行 Tarjan SCC 并给封闭分量编号，无返回值。
            dfn[u] = low[u] = ++ti;
            st.push_back(u);
            on[u] = true;
            for (auto v : g[u])
            {
                if (!dfn[v])
                {
                    self(self, v);
                    low[u] = min(low[u], low[v]);
                }
                else if (on[v])
                {
                    low[u] = min(low[u], dfn[v]);
                }
            }
            if (low[u] != dfn[u])
            {
                return;
            }
            while (true)
            {
                int v = st.back();
                st.pop_back();
                on[v] = false;
                bel[v] = cc;
                if (v == u)
                {
                    break;
                }
            }
            cc++;
        };
        for (int u = 0; u < 2 * n; u++)
        {
            if (!dfn[u])
            {
                dfs(dfs, u);
            }
```

```
        }
        vector<bool> ans(n);
        for (int x = 0; x < n; x++)
        {
            if (bel[2 * x] == bel[2 * x + 1])
            {
                return nullopt;
            }
            ans[x] = bel[2 * x] > bel[2 * x + 1];
        }
        return ans;
    }
};
```

构造 `TarjanTwoSAT q(n)`，用 `addOr(x,xv,y,yv)` 加二元子句，或用 `imply(x,xv,y,yv)` 加蕴含；后者会同时加入逆否边，不能把它当成只插一条普通有向边的接口。`solve()` 无解时返回空，否则返回变量取值。算法用 $O(V+E)$ 时间与空间，$V=2n$ 为文字数。Kosaraju 版本见 5.4.002 Kosaraju 2-SAT，字典序最小构造见 5.4.003 字典序或答案不唯一时的构造。

5.4.002 Kosaraju 2-SAT

同样的 2-SAT，只是把求 SCC 换成 Kosaraju：第一遍在原图上记录结束顺序，第二遍按逆结束序在反图上染色。本实现的分量编号按缩点 DAG 的拓扑正序递增，因此每对互否文字中选编号更大、拓扑更靠后的文字为真，正好与 Tarjan 版的编号比较方向相反。它同时保存正反两张图，逻辑直观，代价是边存储翻倍。

```
class Kosaraju2SAT
{
    // n 是变量数，g、rg 是文字蕴含图及其反图。
    int n;
    vector<vector<int>> g, rg;

    void edge(int u, int v)
    {
        // u、v 是文字节点；加入 u 到 v 的蕴含边及反边索引，无返回值。
        g[u].push_back(v);
        rg[v].push_back(u);
    }

  public:
    Kosaraju2SAT(int n) : n(n), g(2 * n), rg(2 * n)
    {
        // n 是变量数；构造没有子句的 2-SAT，无返回值。
    }

    void imply(int x, bool xv, int y, bool yv)
    {
        // x=xv 是前件，y=yv 是后件；加入蕴含及其逆否边。
        int a = 2 * x + xv, b = 2 * y + yv;
```

```
        edge(a, b);
        edge(b ^ 1, a ^ 1);
    }

    void addOr(int x, bool xv, int y, bool yv)
    {
        // x=xv、y=yv 是两个文字；加入二元子句。
        imply(x, !xv, y, yv);
    }

    optional<vector<bool>> solve() const
    {
        // 无参数；返回一组可行变量值，不可满足时返回空。
        vector<int> ord, bel(2 * n, -1);
        vector<bool> vis(2 * n);
        auto dfs1 = [&](auto &&self, int u) -> void
        {
            // u 是原图文字；记录 DFS 结束顺序，无返回值。
            vis[u] = true;
            for (auto v : g[u])
            {
                if (!vis[v])
                {
                    self(self, v);
                }
            }
            ord.push_back(u);
        };
        int cc = 0;
        auto dfs2 = [&](auto &&self, int u) -> void
        {
            // u 是反图文字；标记当前强连通分量，无返回值。
            bel[u] = cc;
            for (auto v : rg[u])
            {
                if (bel[v] == -1)
                {
                    self(self, v);
                }
            }
        };
        for (int u = 0; u < 2 * n; u++)
        {
            if (!vis[u])
            {
                dfs1(dfs1, u);
            }
        }
        reverse(ord.begin(), ord.end());
        for (auto u : ord)
        {
            if (bel[u] == -1)
```

```
            {
                dfs2(dfs2, u);
                cc++;
            }
        }
        vector<bool> ans(n);
        for (int x = 0; x < n; x++)
        {
            if (bel[2 * x] == bel[2 * x + 1])
            {
                return nullopt;
            }
            ans[x] = bel[2 * x] < bel[2 * x + 1];
        }
        return ans;
    }
};
```

接口与 5.4.001 Tarjan 2-SAT 完全一致，只把类型名换成 `Kosaraju2SAT`：`addOr` / `imply` 加约束、`solve()` 返回 `optional<vector<bool>>`。$O(V+E)$ 时间与空间，因存反图边数常数约为 Tarjan 版两倍。追求单遍 DFS、更省内存时用 5.4.001 Tarjan 2-SAT；需要字典序最小解或增量固定变量时见 5.4.003 字典序或答案不唯一时的构造。底层 SCC 原理见 5.3.002 Kosaraju 强连通分量。

5.4.003 字典序或答案不唯一时的构造

当 2-SAT 有多组解而要求字典序最小时，不能直接用一次 SCC 的赋值（它给出的是任意可行解）。做法是把「可满足性判定」和「构造」分开：先验证空约束下整体可满足，再按变量顺序逐个试着把 x_i 钉成 0——通过在蕴含图里加一条 $\neg(x_i{=}0) \to (x_i{=}0)$ 的强制边使该值成为必然，若加钉后整体仍可满足就固定为 0，否则回退为 1。因为每一步都保留了「已固定前缀 + 当前位取更小」仍可满足的不变量，最终得到的就是字典序最小解。判定函数每次都从头重建蕴含图并跑一遍 Kosaraju，故必须与构造循环彻底解耦。

```
// 知识点 5.4.003：2-SAT 的字典序最小可行赋值。
namespace lex_two_sat
{
optional<vector<int>> lexTwoSat(int n, const vector<array<int, 4>> &clauses)
{
    // n 是变量数；clauses[i] 四维依次是 u、u 的目标布尔值、v、v 的目标布尔值，表示二者至少一个成立。
    auto feasible = [&](const vector<int> &fix) -> bool
    {
        // fix[x] 为 -1 或强制布尔值；返回加入强制条件后公式是否可满足。
        vector<vector<int>> g(2 * n), rg(2 * n);
        auto add = [&](int u, int v)
        {
            // u、v 是文字编号；加入蕴含 u⇒v，无返回值。
            g[u].push_back(v);
            rg[v].push_back(u);
        };
        for (auto [u, a, v, b] : clauses)
```

```
        {
            assert(0 <= u && u < n && 0 <= v && v < n && (a == 0 || a == 1) && (b == 0 || b ==
            ↪ 1)); // 调试检查，可删。
            int x = 2 * u + a, y = 2 * v + b;
            add(x ^ 1, y);
            add(y ^ 1, x);
        }
        for (int i = 0; i < n; i++)
        {
            if (fix[i] >= 0)
            {
                add(2 * i + (fix[i] ^ 1), 2 * i + fix[i]);
            }
        }
        vector<int> vis(2 * n), ord, id(2 * n, -1);
        auto dfs = [&](auto &&self, int u) -> void
        {
            // self 是递归自身，u 是文字；完成正图后序遍历，无返回值。
            vis[u] = 1;
            for (int v : g[u])
            {
                if (!vis[v])
                {
                    self(self, v);
                }
            }
            ord.push_back(u);
        };
        auto rdfs = [&](auto &&self, int u, int c) -> void
        {
            // self 是递归自身，u 是文字，c 是分量编号；标记反图分量，无返回值。
            id[u] = c;
            for (int v : rg[u])
            {
                if (id[v] < 0)
                {
                    self(self, v, c);
                }
            }
        };
        for (int i = 0; i < 2 * n; i++)
        {
            if (!vis[i])
            {
                dfs(dfs, i);
            }
        }
        reverse(ord.begin(), ord.end());
        int c = 0;
        for (int u : ord)
        {
            if (id[u] < 0)
```

```
            {
                rdfs(rdfs, u, c++);
            }
        }
        for (int i = 0; i < n; i++)
        {
            if (id[2 * i] == id[2 * i + 1])
            {
                return false;
            }
        }
        return true;
    };
    vector<int> ans(n, -1);
    if (!feasible(ans))
    {
        return nullopt;
    }
    for (int i = 0; i < n; i++)
    {
        ans[i] = 0;
        if (!feasible(ans))
        {
            ans[i] = 1;
        }
    }
    return ans;
}
}
```

`lexTwoSat(n, clauses)` 中 `clauses[i]` 是四元组 (u, a, v, b),表示子句「$x_u{=}a$ 或 $x_v{=}b$」($a, b \in \{0, 1\}$);返回 `optional<vector<int>>`，无解为空，否则是字典序最小赋值（每位取 0 或 1）。逐变量各做一次判定、每次判定一遍 SCC，最坏 $O(n(V+E))$ 时间、$O(V+E)$ 空间，其中 $V = 2n$。只需任意一组解而不在意字典序时，用一次 5.4.001 Tarjan 2-SAT 或 5.4.002 Kosaraju 2-SAT 即可，快 n 倍；本条只在明确要字典序最小或要按优先级增量固定变量时才用。

5.5 最小生成结构

5.5.001 Kruskal

最小生成树的正确性来自割性质：任意割上的最轻边都属于某棵 MST。Kruskal 把边按权升序扫描，只接受连接不同分量的边；接受 $n-1$ 条后成树，不足则原图不连通。这里同时给出本节唯一的并查集底座 `DSU`，后续最小生成结构均复用它。`kruskal` 用可配置的累计类型保存总权，避免把通常不需要的宽整数带入默认实例。稠密图可另写邻接矩阵 Prim，把每轮选边降为扫描点集；本条只提供 Kruskal，不在标题中承诺未给出的实现。

```
struct DSU
{
```

```
    // f[x] 是并查集父亲，s[x] 仅在根处保存连通块大小。
    vector<int> f, s;

    DSU(int n = 0)
    {
        // n 是元素个数；建立 n 个单点集合。
        init(n);
    }

    void init(int n)
    {
        // n 是元素个数；清空旧状态并建立 n 个单点集合。
        f.resize(n);
        iota(f.begin(), f.end(), 0);
        s.assign(n, 1);
    }

    int find(int x)
    {
        // x 是元素编号；返回所在集合的根并压缩路径。
        return x == f[x] ? x : f[x] = find(f[x]);
    }

    bool merge(int x, int y)
    {
        // x、y 是元素编号；合并所在集合，原本不连通时返回 true。
        x = find(x);
        y = find(y);
        if (x == y)
        {
            return false;
        }
        if (s[x] < s[y])
        {
            swap(x, y);
        }
        f[y] = x;
        s[x] += s[y];
        return true;
    }

    bool same(int x, int y)
    {
        // x、y 是元素编号；返回二者是否属于同一集合。
        return find(x) == find(y);
    }
    int size(int x)
    {
        // x 是元素编号；返回所在集合的元素个数。
        return s[find(x)];
    }
};
```

```cpp

struct MEdge
{
    // x、y 是无向边的两个端点。
    int x, y;
    // w 是边权。
    int w;
};

optional<pair<int, vector<MEdge>>> kruskal(int n, vector<MEdge> e)
{
    // n 是点数，e 是无向边集；返回 MST 总权与所选边，图不连通时返回空。
    sort(e.begin(),
         e.end(),
         [](const MEdge &a, const MEdge &b)
         {
             return a.w < b.w;
         });
    DSU d(n);
    int ans = 0;
    vector<MEdge> use;
    for (auto v : e)
    {
        if (!d.merge(v.x, v.y))
        {
            continue;
        }
        ans += v.w;
        use.push_back(v);
    }
    if ((int)use.size() + 1 != n && n)
    {
        return nullopt;
    }
    return pair{ans, use};
}
```

`kruskal(n,e)` 中 `MEdge` 保存端点 `x,y` 和边权 `w`；图不连通时返回空，否则第一项是总权，第二项是所选边。调用者必须保证生成树总权可由 64 位有符号整数表示；确实需要更宽类型时，直接把 `ans` 改为 `__int128`。排序主导 $O(m \log m)$ 时间，空间为 $O(n+m)$。`DSU` 与 `MEdge` 供 5.5.003 K 棵树的最小生成森林、5.5.004 带修改最小生成树 和 5.5.010 Kruskal 重构树 复用；严格次小值见 5.5.002 次小生成树。

5.5.002 次小生成树

先求一棵 MST，任何非树边 (u, v, w) 加进来都会与树上 $u \to v$ 的唯一路径形成一个环，换掉环上一条树边就得到另一棵生成树，权值增量为 w 减去被删边权。要让增量最小又非零，就删这条路径上权值最大的树边；据此枚举所有非树边取最小增量即得次小生成树。「不同边集次小」允许增量为零（存在等权替换），「严格次小」要求权值真正变大：当 w 恰等于路径最大边权时，最大边换不出正增量，必须改删路径上与最大值不同的次大边权，因此倍增维护每段路径的最大与严格次大两个边权。实现用倍增 LCA 同时

上跳求这两个量。

```cpp
// 依赖 5.5.001 Kruskal
struct MstTop2
{
    // a[0..1] 是一段路径上最大、严格次大的边权，不存在时为空。
    array<optional<int>, 2> a;
};

MstTop2 mstMergeTop(const MstTop2 &a, const MstTop2 &b)
{
    // a、b 是两段路径的最大值记录；返回合并后的两个互异最大值。
    vector<int> v;
    for (auto x : a.a)
    {
        if (x)
        {
            v.push_back(*x);
        }
    }
    for (auto x : b.a)
    {
        if (x)
        {
            v.push_back(*x);
        }
    }
    sort(v.begin(), v.end(), greater<>());
    v.erase(unique(v.begin(), v.end()), v.end());
    MstTop2 c;
    for (int i = 0; i < min<int>(2, v.size()); i++)
    {
        c.a[i] = v[i];
    }
    return c;
}

struct MstVariants
{
    // mst 是最小生成树权值。
    int mst;
    // other 是不同边集的最小权值，不存在时为空。
    optional<int> other;
    // strict 是严格大于 mst 的最小生成树权值，不存在时为空。
    optional<int> strict;
};

optional<MstVariants> secondMST(int n, const vector<array<int, 3>> &edges)
{
    // edges[i][0..2] 是端点 u、v 与边权；图不连通时返回空。
    if (n == 0)
    {
```

```
        return MstVariants{0, nullopt, nullopt};
    }
    int m = edges.size();
    vector<int> ord(m);
    iota(ord.begin(), ord.end(), 0);
    sort(ord.begin(),
         ord.end(),
         [&](int x, int y)
         {
             return edges[x][2] < edges[y][2];
         });
    DSU dsu(n);
    vector<bool> used(m);
    vector<vector<array<int, 2>>> g(n);
    int mst = 0;
    int cnt = 0;
    for (auto id : ord)
    {
        auto [u, v, w] = edges[id];
        if (!dsu.merge(u, v))
        {
            continue;
        }
        used[id] = true;
        mst += w;
        cnt++;
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
    if (cnt != n - 1)
    {
        return nullopt;
    }
    int lg = bit_width((unsigned long long)max<int>(1, n));
    vector up(lg, vector<int>(n));
    vector mx(lg, vector<MstTop2>(n));
    vector<int> dep(n);
    vector<int> par(n, -1), seq{0};
    par[0] = 0;
    for (int i = 0; i < (int)seq.size(); i++)
    {
        int u = seq[i];
        up[0][u] = par[u];
        for (auto [v, w] : g[u])
        {
            if (v == par[u])
            {
                continue;
            }
            par[v] = u;
            dep[v] = dep[u] + 1;
            mx[0][v].a[0] = w;
```

```
            seq.push_back(v);
        }
    }
    for (int j = 1; j < lg; j++)
    {
        for (int u = 0; u < n; u++)
        {
            mx[j][u] = mstMergeTop(mx[j - 1][u], mx[j - 1][up[j - 1][u]]);
            up[j][u] = up[j - 1][up[j - 1][u]];
        }
    }
    auto pathMax = [&](int u, int v) -> MstTop2
    {
        // u、v 是树上点；返回路径边权中最大的两个互异值。
        MstTop2 ans;
        if (dep[u] < dep[v])
        {
            swap(u, v);
        }
        for (int j = lg - 1; j >= 0; j--)
        {
            if (dep[u] - (1LL << j) >= dep[v])
            {
                ans = mstMergeTop(ans, mx[j][u]);
                u = up[j][u];
            }
        }
        if (u == v)
        {
            return ans;
        }
        for (int j = lg - 1; j >= 0; j--)
        {
            if (up[j][u] != up[j][v])
            {
                ans = mstMergeTop(ans, mx[j][u]);
                ans = mstMergeTop(ans, mx[j][v]);
                u = up[j][u];
                v = up[j][v];
            }
        }
        ans = mstMergeTop(ans, mx[0][u]);
        return mstMergeTop(ans, mx[0][v]);
    };
    optional<int> other, strict;
    auto upd = [] (optional<int> &x, int v)
    {
        // x 是当前最优候选，v 是新候选；保留较小值。
        if (!x || v < *x)
        {
            x = v;
        }
```

```
    };
    for (int id = 0; id < m; id++)
    {
        if (used[id])
        {
            continue;
        }
        auto [u, v, w] = edges[id];
        auto top = pathMax(u, v);
        if (top.a[0])
        {
            int cur = mst + w - *top.a[0];
            upd(other, cur);
            if (w > *top.a[0])
            {
                upd(strict, cur);
            }
            else if (w == *top.a[0] && top.a[1])
            {
                upd(strict, mst + w - *top.a[1]);
            }
        }
    }
    return MstVariants{mst, other, strict};
}
```

`secondMST(n,edges)` 的边记录依次是 u, v, w；图不连通时返回空，否则 `mst` 是 MST 权，`other` 是不同边集的最小权，`strict` 是严格更大的最小权，后两者不存在时各自为空。调用者必须保证生成树总权和替换后权值可由 64 位有符号整数表示。倍增建表和非树边查询合计 $O((V+E)\log V)$ 时间与空间。四点环加一条权 2 的对角边时三项依次为 $3, 3, 4$。本条复用 5.5.001 Kruskal 的 `DSU`。

5.5.003 K 棵树的最小生成森林

要把 n 个点组织成恰好 k 棵树的森林且总权最小，只需跑 Kruskal 到连通块数从 n 降到 k 时停手。按边权升序选不成环的边，在任意连通块数下都是该块数的最小权森林。实现复用 5.5.001 Kruskal 的 `DSU` 与 `MEdge`；`k=n` 必须在扫描边前直接返回零，否则第一次成功合并就再也回不到 n 个连通块。

```
// 依赖 5.5.001 Kruskal
optional<int> minKForest(int n, int k, vector<MEdge> e)
{
    // n 是点数，k 是目标连通块数，e 是无向边集；返回最小森林权值。
    assert(n >= 1 && 1 <= k && k <= n); // 调试检查，可删。
    if (k == n)
    {
        return 0;
    }
    sort(e.begin(),
         e.end(),
         [](auto a, auto b)
```

```
        {
            return a.w < b.w;
        });
    DSU d(n);
    int cc = n;
    int ans = 0;
    for (auto [u, v, w] : e)
    {
        if (!d.merge(u, v))
        {
            continue;
        }
        ans += w;
        if (--cc == k)
        {
            return ans;
        }
    }
    return cc == k ? optional<int>(ans) : nullopt;
}
```

`minKForest(n,k,e)` 要求 $n \geq 1$ 且 $1 \leq k \leq n$；图自身连通块数超过 k 时返回空，否则返回最小森林总权。调用者必须保证答案可由 64 位有符号整数表示。四点环上 $k = 2, 1, 4$ 的答案依次为 $2, 3, 0$。排序主导 $O(E \log E)$ 时间，额外空间为 $O(V)$；$k = 1$ 就是 5.5.001 Kruskal。

5.5.004 带修改最小生成树

这里给出最朴素的带修改版本：`DynamicMST` 保存全部边，`set(id,w)` 直接改权，`query()` 调用 5.5.003 K 棵树的最小生成森林 的 `minKForest(n,1,e)` 重跑 Kruskal。修改是 $O(1)$，查询承担全部排序成本，适合查询少的场景；修改和查询都密集时应换 LCT 或离线分治。本条复用 5.5.001 Kruskal 的 `MEdge`，不再定义同名边类型或复制森林算法。

```
// 依赖 5.5.001 Kruskal
// 依赖 5.5.003 K 棵树的最小生成森林
struct DynamicMST
{
    // n 是图的点数。
    int n;
    // e 是当前无向边集。
    vector<MEdge> e;

    DynamicMST(int n, vector<MEdge> e) : n(n), e(move(e))
    {
        // n 是点数，e 是初始边集；保存当前图。
    }

    void set(int id, int w)
    {
        // id 是边编号，w 是新边权；修改该边。
```

```
        e[id].w = w;
    }

    optional<int> query() const
    {
        // 无参数；返回当前 MST 总权，图不连通时返回空。
        return minKForest(n, 1, e);
    }
};
```

构造 `DynamicMST dm(n,e)`，边使用 `MEdge{x,y,w}`；`set(id,w)` 改边权，`query()` 返回 MST 总权，不连通时为空。调用者必须保证答案可由 64 位有符号整数表示。修改用 $O(1)$ 时间，查询用 $O(m \log m)$ 时间、$O(n)$ 额外空间。静态一次查询见 5.5.001 Kruskal，次小值见 5.5.002 次小生成树。

5.5.005 最小乘积生成树

每条边带两种非负代价，要最小化生成树两种总代价之积 $X \cdot Y$。把每棵生成树映射成平面点 (X, Y)，最优点落在这些点的下凸壳上；用 $aX + bY$ 作边权跑一次 MST，就得到沿方向 (a, b) 的支撑点。先以 $(1, 0)$、$(0, 1)$ 求凸壳两端，再用垂直于端点连线的方向继续找中间支撑点。两种权和与差先在 `__int128` 中形成；线性权、叉积和最终乘积只需比较两个 128 位乘积，辅助结构用四个 64 位肢精确完成比较，避免再次溢出且不依赖额外大整数库。底层复用 5.5.001 Kruskal 的 `DSU`。

```
// 依赖 5.5.001 Kruskal
struct MstU256
{
    // a 从低到高保存四个 64 位肢。
    array<unsigned long long, 4> a{};

    static MstU256 mul(unsigned __int128 x, unsigned __int128 y)
    {
        // x、y 是无符号 128 位整数；返回精确的 256 位乘积。
        unsigned long long p[2] = {(unsigned long long)x, (unsigned long long)(x >> 64)};
        unsigned long long q[2] = {(unsigned long long)y, (unsigned long long)(y >> 64)};
        MstU256 z;
        for (int i = 0; i < 2; i++)
        {
            unsigned __int128 c = 0;
            for (int j = 0; j < 2; j++)
            {
                unsigned __int128 t = (unsigned __int128)p[i] * q[j] + z.a[i + j] + c;
                z.a[i + j] = (unsigned long long)t;
                c = t >> 64;
            }
            z.a[i + 2] = (unsigned long long)c;
        }
        return z;
    }
};

```

```cpp
int mstProdCmp(__int128 a, __int128 b, __int128 c, __int128 d)
{
    // 返回 a*b 与 c*d 的大小关系。
    auto mag = [](__int128 x) -> unsigned __int128
    {
        return x < 0 ? 0 - (unsigned __int128)x : (unsigned __int128)x;
    };
    auto x = MstU256::mul(mag(a), mag(b));
    auto y = MstU256::mul(mag(c), mag(d));
    bool nx = (a < 0) != (b < 0), ny = (c < 0) != (d < 0);
    if (x.a == array<unsigned long long, 4>{})
    {
        nx = false;
    }
    if (y.a == array<unsigned long long, 4>{})
    {
        ny = false;
    }
    if (nx != ny)
    {
        return nx ? -1 : 1;
    }
    int s = 0;
    for (int i = 3; i >= 0 && !s; i--)
    {
        s = (x.a[i] > y.a[i]) - (x.a[i] < y.a[i]);
    }
    return nx ? -s : s;
}

struct ProductMST
{
    using Cost = pair<__int128, __int128>;

    struct E
    {
        // u、v 是无向边端点。
        int u, v;
        // x、y 是两种非负边权。
        int x, y;
    };

    // n 是图的点数。
    int n;
    // e 是全部无向边。
    vector<E> e;

    static bool better(const Cost &a, const Cost &b)
    {
        // a、b 是两棵生成树的两种总权；返回 a 的乘积是否更优。
        int c = mstProdCmp(a.first, a.second, b.first, b.second);
        return c != 0 ? c < 0 : a < b;
```

```
    }

    ProductMST(int n = 0) : n(n)
    {
        // n 是图的点数；构造一张没有边的图。
    }

    void add(int u, int v, int x, int y)
    {
        // u、v 是端点，x、y 是两种非负边权；加入一条无向边。
        assert(x >= 0 && y >= 0); // 调试检查，可删。
        e.push_back({u, v, x, y});
    }

    optional<Cost> mst(__int128 a, __int128 b) const
    {
        // a、b 是线性目标系数；返回该方向下 MST 的两种原始总权。
        vector<int> p(e.size());
        iota(p.begin(), p.end(), 0);
        sort(p.begin(),
             p.end(),
             [&](int i, int j)
             {
                 int c = mstProdCmp(a,
                                    (__int128)e[i].x - e[j].x,
                                    b,
                                    (__int128)e[j].y - e[i].y);
                 if (c != 0)
                 {
                     return c < 0;
                 }
                 if (e[i].x != e[j].x)
                 {
                     return e[i].x < e[j].x;
                 }
                 return e[i].y < e[j].y;
             });
        DSU d(n);
        Cost s{};
        int z = 0;
        for (int i : p)
        {
            if (d.merge(e[i].u, e[i].v))
            {
                s.first += (__int128)e[i].x;
                s.second += (__int128)e[i].y;
                z++;
            }
        }
        if (z != n - 1)
        {
            return nullopt;
```

```
        }
        return s;
    }

    optional<Cost> solve() const
    {
        // 无参数；返回使两种边权总和乘积最小的生成树两项权值和。
        if (n <= 1)
        {
            return Cost{0, 0};
        }
        auto a = mst(1, 0);
        auto b = mst(0, 1);
        if (!a || !b)
        {
            return nullopt;
        }
        Cost ans = better(*a, *b) ? *a : *b;
        auto divide = [&](auto &&self, Cost l, Cost r) -> void
        {
            __int128 x = l.second - r.second;
            __int128 y = r.first - l.first;
            auto c = mst(x, y);
            if (!c || *c == l || *c == r)
            {
                return;
            }
            int cr = mstProdCmp(r.first - l.first,
                                c->second - l.second,
                                r.second - l.second,
                                c->first - l.first);
            if (cr >= 0)
            {
                return;
            }
            if (better(*c, ans))
            {
                ans = *c;
            }
            self(self, l, *c);
            self(self, *c, r);
        };
        divide(divide, *a, *b);
        return ans;
    }
};
```

构造 `ProductMST pm(n)`，用 `add(u,v,x,y)` 加非负双权边，`solve()` 返回 `pair<__int128,__int128>` 形式的最优 (X, Y)，不连通时为空。每次 Kruskal 用 $O(m \log m)$ 时间，若下凸壳访问了 h 个支撑点，总时间为 $O(hm \log m)$、空间为 $O(n + m + h)$。两种代价成固定线性关系时直接用 5.5.001 Kruskal。

5.5.006 切比雪夫距离最小生成树

平面上 n 个点两两以切比雪夫距离 $\max(|\Delta x|, |\Delta y|)$ 连边求 MST，完全图有 $O(n^2)$ 条边不可直接跑。关键结论是曼哈顿 / 切比雪夫距离的 MST 只需保留每个点在八分之一平面内的最近邻这类候选边——切比雪夫经 $(x+y,\ x-y)$ 变换即化为曼哈顿距离，而曼哈顿 MST 只要在四个（对称后八个）方向的每个锥区里为每点连一条最近点边就够了。用按坐标排序 + 树状数组维护「某区间内 $x+y$ 最小的点」即可在 $O(n \log n)$ 内生成 $O(n)$ 条候选边，再对这批稀疏边跑 Kruskal。

```cpp
// 依赖 5.5.001 Kruskal
// CP[0..1] 依次是原点的 x、y 坐标。
using CP = array<int, 2>;

struct GeometricMST
{
    __int128 weight = 0; // 最小生成树总权。
    vector<pair<int, int>> edge; // 所选边的原点编号。
};

GeometricMST chebyMST(const vector<CP> &input)
{
    // input[i][0..1] 是原点坐标；返回切比雪夫 MST 总权与所选边。
    struct Point
    {
        // x、y 是变换后的坐标，id 是原点编号。
        __int128 x, y;
        int id;
    };
    struct Edge
    {
        // w 是候选边权，u、v 是原点编号。
        __int128 w;
        int u, v;
    };
    int n = input.size();
    vector<Point> p;
    for (int i = 0; i < n; ++i)
    {
        p.push_back({(__int128)input[i][0] + input[i][1],
                     (__int128)input[i][0] - input[i][1], i});
    }
    vector<Edge> e;
    for (int dir = 0; dir < 4; ++dir)
    {
        ranges::sort(p,
                     [](const Point &a, const Point &b)
                     {
                         return a.x + a.y != b.x + b.y ? a.x + a.y < b.x + b.y : a.x < b.x;
                     });
        map<__int128, int> sweep;
        for (int i = 0; i < n; ++i)
        {
```

```
            for (auto it = sweep.lower_bound(-p[i].y); it != sweep.end();)
            {
                int j = it->second;
                if (p[i].x - p[j].x < p[i].y - p[j].y)
                {
                    break;
                }
                __int128 dx = p[i].x - p[j].x;
                __int128 dy = p[i].y - p[j].y;
                if (dx < 0)
                {
                    dx = -dx;
                }
                if (dy < 0)
                {
                    dy = -dy;
                }
                e.push_back({(dx + dy) / 2, p[i].id, p[j].id});
                it = sweep.erase(it);
            }
            sweep[-p[i].y] = i;
        }
        for (auto &v : p)
        {
            if (dir & 1)
            {
                v.x = -v.x;
            }
            else
            {
                swap(v.x, v.y);
            }
        }
    }
    ranges::sort(e,
                 {},
                 [](const Edge &a)
                 {
                     return a.w;
                 });
    DSU d(n);
    GeometricMST ans;
    for (auto [w, u, v] : e)
    {
        if (!d.merge(u, v))
        {
            continue;
        }
        ans.weight += w;
        ans.edge.push_back({u, v});
    }
    return ans;
```

```
}

__int128 chebyshevMst(const vector<CP> &input)
{
    // input[i][0..1] 是原点坐标；返回切比雪夫 MST 总权。
    return chebyMST(input).weight;
}
```

`chebyMST(input)` 返回总权和一组原下标 MST 边，`chebyshevMst(input)` 仅取总权；`CP` 存 64 位整点坐标，坐标和差、候选距离与返回总权均为 `__int128`。候选边生成和 Kruskal 各用 $O(n \log n)$ 时间，空间为 $O(n)$。原坐标 Manhattan 距离等于变换后坐标的切比雪夫距离，故需要 Manhattan MST 边时先传入 `(x+y,x-y)`。欧氏距离版本见 5.5.007 平面欧氏距离最小生成树。

5.5.007 平面欧氏距离最小生成树

二维点两两以欧氏距离连边求 MST，完全图 $O(n^2)$ 条边。这里用稠密图 Prim 避免显式建边：维护每个未选点到当前生成树的最短距离 `d[i]`，每轮 $O(n)$ 扫出最近的未选点加入，再用它 $O(n)$ 更新其余 `d`，边权按坐标即时算平方欧氏距离（比较用平方避免开根、累加时才开根）。总共 $O(n^2)$，无需存边表，适合中等点数；点数很大时应改用 Delaunay 三角剖分把候选边降到 $O(n)$。

```
// 依赖 5.5.001 Kruskal
long double euclideanMST(const vector<array<long double, 2>> &p)
{
    // p[i][0..1] 依次是第 i 个点的 x、y 坐标；返回完全图欧氏距离最小生成树总长。
    int n = p.size();
    if (n == 0)
    {
        return 0;
    }
    vector<long double> dis(n, numeric_limits<long double>::infinity());
    vector<bool> vis(n);
    dis[0] = 0;
    long double ans = 0;
    for (int t = 0; t < n; t++)
    {
        int u = -1;
        for (int i = 0; i < n; i++)
        {
            if (!vis[i] && (u == -1 || dis[i] < dis[u]))
            {
                u = i;
            }
        }
        vis[u] = true;
        ans += sqrtl(dis[u]);
        for (int v = 0; v < n; v++)
        {
            if (vis[v])
            {
```

```
                continue;
            }
            long double dx = p[u][0] - p[v][0], dy = p[u][1] - p[v][1];
            dis[v] = min(dis[v], dx * dx + dy * dy);
        }
    }
    return ans;
}

vector<pair<int, int>> euclidMST(const vector<array<long long, 2>> &p)
{
    // p[i][0..1] 是整数点坐标；返回完全图欧氏距离 MST 的原点编号边集。
    int n = p.size();
    if (n <= 1)
    {
        return {};
    }
    map<pair<long long, long long>, int> first;
    vector<IP> unique;
    vector<int> rep;
    vector<MEdge> edge;
    for (int i = 0; i < n; i++)
    {
        pair key{p[i][0], p[i][1]};
        auto [it, fresh] = first.emplace(key, i);
        if (!fresh)
        {
            edge.push_back({it->second, i, 0});
        }
        else
        {
            unique.push_back({p[i][0], p[i][1]});
            rep.push_back(i);
        }
    }
    auto tri = delaunay(unique);
    for (auto [x, y] : tri.edge)
    {
        long long dx = unique[x].x - unique[y].x;
        long long dy = unique[x].y - unique[y].y;
        edge.push_back({rep[x], rep[y], dx * dx + dy * dy});
    }
    auto mst = kruskal(n, move(edge));
    assert(mst.has_value()); // 调试检查，可删；Delaunay 边与重合点零边必然连通。
    vector<pair<int, int>> ans;
    for (auto [x, y, w] : mst->second)
    {
        ans.push_back({x, y});
    }
    return ans;
}
```

`euclideanMST(p)` 保留稠密 Prim 小规模版，返回浮点总长，时间 $O(n^2)$、空间 $O(n)$。大规模整数点调用 `euclidMST(p)`：先用零边连接重合点，再依赖 11.3.011 Delaunay 三角剖分生成 $O(n)$ 条候选边并按距离平方 Kruskal，返回原下标边集；时间 $O(n \log n)$、空间 $O(n)$，要求坐标差四次方满足 Delaunay 条目的 `__int128` 范围。单位正方形的两版 MST 总权均为 3。

5.5.008 最小树形图 / 朱刘算法

最小树形图要在有向图里求一棵以 `root` 为根、能到达所有点的生成树，使总权最小；每个非根点恰有一条入边。朱刘算法的思想是：先给每个非根点选权值最小的入边，累加这些入边权得到一个下界。如果这些入边不成环，它们就直接构成答案；否则出现有向环，说明环上的点互相「抢」了对方的最小入边，需要把整个环收缩成一个超级点，并把所有指向环内点 v 的边权减去 v 当前选中的最小入边权（这样重新在收缩图上选最小入边时，等价于「换掉环上某条边、改从环外进入」的代价差），然后在收缩后的更小图上递归重复。每轮至少收缩掉一个环、点数严格减少，故最多 $n-1$ 轮。代码用 `pre` 记每点当前最小入边来源，用一次遍历给环染色 `id` 完成收缩。

```
struct DEdgeMST
{
    // u、v 是有向边的起点和终点。
    int u, v;
    // w 是当前收缩图中的边权。
    __int128 w;
};

optional<__int128> directedMST(int n, int root, vector<DEdgeMST> e)
{
    // n 是点数，root 是树根，e 是有向边集；返回最小树形图权值，不存在时返回空。
    assert(n >= 1 && 0 <= root && root < n); // 调试检查，可删。
    __int128 ans = 0;
    while (true)
    {
        vector<optional<__int128>> in(n);
        vector<int> pre(n, -1);
        for (auto [u, v, w] : e)
        {
            if (u != v && (!in[v] || w < *in[v]))
            {
                in[v] = w;
                pre[v] = u;
            }
        }
        in[root] = 0;
        for (int u = 0; u < n; u++)
        {
            if (!in[u])
            {
                return nullopt;
            }
            ans += *in[u];
        }
```

```
        int cnt = 0;
        vector<int> id(n, -1), vis(n, -1);
        for (int s = 0; s < n; s++)
        {
            int u = s;
            while (vis[u] != s && id[u] < 0 && u != root)
            {
                vis[u] = s;
                u = pre[u];
            }
            if (u != root && id[u] < 0)
            {
                for (int v = pre[u]; v != u; v = pre[v])
                {
                    id[v] = cnt;
                }
                id[u] = cnt++;
            }
        }
        if (!cnt)
        {
            return ans;
        }
        for (int u = 0; u < n; u++)
        {
            if (id[u] < 0)
            {
                id[u] = cnt++;
            }
        }
        for (auto &[u, v, w] : e)
        {
            int old = v;
            u = id[u];
            v = id[v];
            if (u != v)
            {
                w -= *in[old];
            }
        }
        root = id[root];
        n = cnt;
    }
}
```

`directedMST(n,root,e)` 要求 $n \geq 1$ 且 `root` 是合法点编号，返回以 `root` 为根的最小树形图总权，不存在时为空；`DEdgeMST::w` 和返回值均为 `__int128`。参数 `e` 按值传入，收缩只修改函数内副本，不会改调用方边集。算法至多收缩 $n-1$ 轮，用 $O(nm)$ 时间、$O(n+m)$ 空间，适合小规模、需要宽权值或只要总权的场景。大规模且需要父节点时使用同条目的 `dmstTree(n,root,e)`：它以配对堆维护每个收缩点的入边，`DMSTFastEdge` 用 `long long` 存边权，返回 `DMSTResult{weight,parent}`，其中 `parent[root]=root`。该版本在 $O((n+m)\log n)$ 期望时间和 $O(n+m)$ 空间内恢复一棵最优树，适用于 $n, m \leq 2 \times 10^5$；

两版并存，不能以快速版替代需要 `__int128` 总权的旧接口。有向根树与 5.5.001 Kruskal 的无向 MST 不同，不能互换。下面的四点图含一个非根有向环，最优值为 6；第二张图没有到点 2 的入边，故无解。

```
signed main()
{
    vector<DEdgeMST> e{{0, 1, 4}, {0, 2, 5}, {1, 2, 1},
                       {2, 3, 1}, {3, 1, 1}, {1, 3, 3}};
    assert(directedMST(4, 0, e) == 6);
    assert(!directedMST(3, 0, {{0, 1, 1}}));
}
```

5.5.009 最小斯坦纳树

斯坦纳树问题要连通指定的 k 个关键点，允许借道其余「斯坦纳点」，求最小总边权——当 k 等于全部点时就退化成 MST。关键点少时用状压 DP：`f[S][u]` 表示「已连通关键点集合 S、且当前树含点 u」的最小代价。转移分两种：一是子集合并，把 S 拆成两个不相交子集 a、b，在同一个汇点 u 处拼接，`f[S][u] = min(f[a][u]+f[b][u])`；二是在固定 S 后沿图边做一次 Dijkstra 松弛 `f[S][v] = min(f[S][u]+w(u,v))`，让这棵树「长」到相邻点上。初值把每个关键点单独成集的代价设 0，最终答案是 `f[full]` 在所有汇点上的最小值。

```
struct Steiner
{
    struct E
    {
        // to 是邻点编号。
        int to;
        // w 是非负边权。
        int w;
    };

    // n 是图的点数。
    int n;
    // e 是无向图邻接表。
    vector<vector<E>> e;

    Steiner(int n = 0) : n(n), e(n)
    {
        // n 是图的点数；构造一张没有边的图。
    }

    void add(int u, int v, int w)
    {
        // u、v 是端点，w 是非负边权；加入一条无向边。
        assert(w >= 0); // 调试检查，可删。
        e[u].push_back({v, w});
        e[v].push_back({u, w});
    }

    optional<int> solve(const vector<int> &key) const
```

```
    {
        // key 是关键点编号列表；返回连接全部关键点的最小权值，不可连通时返回空。
        size_t k = key.size();
        if (k == 0)
        {
            return 0;
        }
        assert(k < 63); // 调试检查，可删。
        size_t z = (size_t)1 << k;
        vector f(z, vector<optional<int>>(n));
        for (size_t i = 0; i < k; i++)
        {
            assert(0 <= key[i] && key[i] < n); // 调试检查，可删。
            f[(size_t)1 << i][key[i]] = 0;
        }
        for (size_t s = 1; s < z; s++)
        {
            for (size_t a = (s - 1) & s; a; a = (a - 1) & s)
            {
                size_t b = s ^ a;
                if (!b || a > b)
                {
                    continue;
                }
                for (int u = 0; u < n; u++)
                {
                    if (f[a][u] && f[b][u])
                    {
                        int nd = *f[a][u] + *f[b][u];
                        if (!f[s][u] || nd < *f[s][u])
                        {
                            f[s][u] = nd;
                        }
                    }
                }
            }
            priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> q;
            for (int u = 0; u < n; u++)
            {
                if (f[s][u])
                {
                    q.push({*f[s][u], u});
                }
            }
            while (!q.empty())
            {
                auto [d, u] = q.top();
                q.pop();
                if (!f[s][u] || d != *f[s][u])
                {
                    continue;
                }
```

```
                for (auto [v, w] : e[u])
                {
                    int nd = d + w;
                    if (!f[s][v] || nd < *f[s][v])
                    {
                        f[s][v] = nd;
                        q.push({nd, v});
                    }
                }
            }
        }
        optional<int> ans;
        for (auto x : f.back())
        {
            if (x && (!ans || *x < *ans))
            {
                ans = *x;
            }
        }
        return ans;
    }
};
```

构造 `Steiner g(n)` 后用 `add(u,v,w)` 加非负权无向边，`solve(key)` 返回最小权；关键点为空时为 0，关键点不能互相连通时为空。调用者必须保证边权和可由 64 位有符号整数表示，并且 $2^k n$ 的内存可承受。子集合并用 $O(3^k n)$ 时间，全部 Dijkstra 用 $O(2^k m \log n)$ 时间，空间为 $O(2^k n + m)$。关键点覆盖全部点时直接用 5.5.001 Kruskal。

5.5.010 Kruskal 重构树

把 5.5.001 Kruskal 的加边过程显式建成一棵二叉树。边按权升序处理，每当一条边成功合并两个连通块，就新建一个「代表这条边」的内部结点，其点权取该边权，并令它做两个块根的父亲。n 个原始点最终成为叶子，共得到 $2n-1$ 个结点（图连通时为一棵树，否则为森林）。两条关键性质：

- $\mathrm{lca}(u,v)$ 的点权 $=$ u,v 间所有路径上「最大边权」的最小值，即瓶颈路权、也等于最小生成树上 $u \to v$ 路径的最大边；
- 「从 u 出发只走权 $\le W$ 的边所能到达的点集」，恰为 u 在重构树上向上跳到「点权 $\le W$ 的最高祖先」，该祖先子树内的全部叶子。

若改按降序加边，则对应最大瓶颈（路径最小边的最大化）。第二条性质配合倍增找祖先 + DFS 序，能把「限制边权的可达性/第 K 大」类询问转成子树问题。

```
struct KruskalTree
{
    // n 是原始点数，tot 是当前重构树结点数。
    int n, tot;
    // val[x] 是内部结点代表的边权，叶子处为空。
    vector<optional<int>> val;
    // ch[x] 是内部结点的两个孩子，叶子为 {-1,-1}。
```

```
    vector<array<int, 2>> ch;
    // fa 是建树时使用的并查集父亲。
    vector<int> fa;
    // dep、up 是 build 后的深度和倍增祖先表。
    vector<int> dep;
    vector<vector<int>> up;

    KruskalTree(int n) : n(n), tot(n), val(2 * n), ch(2 * n, {-1, -1}), fa(2 * n)
    {
        // n 是原始点数；建立 n 个叶子。
        iota(fa.begin(), fa.end(), 0);
    }

    int find(int x)
    {
        // x 是当前结点；迭代查找并压缩所在块的根。
        int r = x;
        while (fa[r] != r)
        {
            r = fa[r];
        }
        while (fa[x] != x)
        {
            int p = fa[x];
            fa[x] = r;
            x = p;
        }
        return r;
    }

    void addEdge(int u, int v, int w)
    {
        // u、v 是原图端点，w 是当前边权；按非降权序合并两个块。
        int ru = find(u);
        int rv = find(v);
        if (ru == rv)
        {
            return;
        }
        int c = tot++;
        val[c] = w;
        ch[c] = {ru, rv};
        fa[ru] = c;
        fa[rv] = c;
        fa[c] = c;
    }

    void build()
    {
        // 无参数；加完所有边后重建深度和 LCA 倍增表。
        int lg = bit_width((unsigned long long)max<int>(1, tot));
        up.assign(lg, vector<int>(tot, -1));
```

```
        dep.assign(tot, 0);
        vector<char> vis(tot);
        for (int i = 0; i < tot; i++)
        {
            int r = find(i);
            if (vis[r])
            {
                continue;
            }
            vis[r] = 1;
            auto dfs = [&](auto &&self, int x) -> void
            {
                for (int c : {ch[x][0], ch[x][1]})
                {
                    if (c < 0)
                    {
                        continue;
                    }
                    dep[c] = dep[x] + 1;
                    up[0][c] = x;
                    self(self, c);
                }
            };
            dfs(dfs, r);
        }
        for (int k = 1; k < lg; k++)
        {
            for (int x = 0; x < tot; x++)
            {
                if (up[k - 1][x] >= 0)
                {
                    up[k][x] = up[k - 1][up[k - 1][x]];
                }
            }
        }
    }

    int lca(int u, int v)
    {
        // u、v 是原图点；返回重构森林 LCA，不连通时返回 -1。
        if (find(u) != find(v))
        {
            return -1;
        }
        if (dep[u] < dep[v])
        {
            swap(u, v);
        }
        int d = dep[u] - dep[v];
        for (int k = 0; d; k++)
        {
            if (d & 1)
```

```
            {
                u = up[k][u];
            }
            d >>= 1;
        }
        if (u == v)
        {
            return u;
        }
        for (int k = (int)up.size() - 1; k >= 0; k--)
        {
            if (up[k][u] != up[k][v])
            {
                u = up[k][u];
                v = up[k][v];
            }
        }
        return up[0][u];
    }

    optional<int> bottleneck(int u, int v)
    {
        // u、v 是原图点；返回瓶颈路权，不连通时返回空，同一点返回 0。
        if (u == v)
        {
            return 0;
        }
        int l = lca(u, v);
        if (l < 0)
        {
            return nullopt;
        }
        return val[l];
    }
};
```

把边按权非降序逐条传给 `addEdge(u,v,w)`，再调用 `build()`；之后 `bottleneck(u,v)` 返回瓶颈路权，不连通时为空。排序与建树用 $O(m \log m + n)$ 时间，查询用 $O(\log n)$ 时间，空间为 $O(n \log n)$。下面的图中点 4 孤立，路径 $0 \to 2$ 与 $0 \to 3$ 的瓶颈分别为 4 与 5。

```
signed main()
{
    vector<array<int, 3>> e{{2, 1, 2}, {4, 0, 1},
                            {5, 2, 3}, {7, 0, 2}};
    KruskalTree t(5);
    for (auto [w, u, v] : e)
    {
        t.addEdge(u, v, w);
    }
    t.build();
    assert(t.bottleneck(0, 0) == 0);
```

```
    assert(t.bottleneck(0, 2) == 4);
    assert(t.bottleneck(0, 3) == 5);
    assert(!t.bottleneck(0, 4));
}
```

5.5.011 增量最小生成森林

边只增加时，当前两端不连通的新边必进入最小生成森林；已经连通时，若新边比森林路径上的最重边更轻，就用它替换该最重边。把每条入选边建成一个 LCT 边点，原图端点连到边点，路径最大值便能同时给出权值和待删边编号。下面是 6.4.001 Link-Cut Tree 改成 “路径最大边” 的短专用版，不处理删去任意历史边或修改旧边权。

```
struct IncrMSF
{
    struct Node
    {
        array<int, 2> ch{}; // ch[0..1] 是 splay 左右儿子，0 表示空。
        int fa = 0; // fa 是 splay 父亲或所在链的路径父亲。
        bool rev = false, edge = false; // rev 是子树翻转标记，edge 表示本节点代表一条边。
        long long val = 0, mx = 0; // val 是边权（点节点为 0），mx 是 splay 子树内边节点的最大边权。
        int who = 0, u = 0, v = 0; // who 是取到 mx 的边节点（无边为 0），u、v 是边节点的两个端点。
    };

    int n, cc; // n 是点数，cc 是当前连通块数。
    int sum = 0; // sum 是当前最小生成森林总权。
    vector<Node> t; // t[1..n] 是点节点，之后依次是新建的边节点，t[0] 为空。

    IncrMSF(int n) : n(n), cc(n), t(n + 1)
    {
        // n 是点数；建立没有边的森林。
        assert(n >= 0); // 调试检查，可删。
    }

    bool isRoot(int x)
    {
        // x 是节点；返回 x 是否为所在 splay 的根。
        int f = t[x].fa;
        return !f || (t[f].ch[0] != x && t[f].ch[1] != x);
    }

    void pull(int x)
    {
        // x 是节点；由自身与儿子重算 mx 与取到它的边节点 who。
        t[x].who = t[x].edge ? x : 0;
        t[x].mx = t[x].val;
        for (int c : t[x].ch)
        {
            if (t[c].who && (!t[x].who || t[c].mx > t[x].mx))
            {
                t[x].who = t[c].who;
```

```
                t[x].mx = t[c].mx;
            }
        }
    }

    void reverseNode(int x)
    {
        // x 是节点（0 时忽略）；交换左右儿子并翻转标记。
        if (x)
        {
            swap(t[x].ch[0], t[x].ch[1]);
            t[x].rev ^= 1;
        }
    }

    void push(int x)
    {
        // x 是节点；把翻转标记下传给儿子。
        if (t[x].rev)
        {
            reverseNode(t[x].ch[0]);
            reverseNode(t[x].ch[1]);
            t[x].rev = false;
        }
    }

    void rotate(int x)
    {
        // x 是非根节点；把 x 旋转到其父亲的位置。
        int y = t[x].fa, z = t[y].fa, k = t[y].ch[1] == x, w = t[x].ch[k ^ 1];
        if (!isRoot(y))
        {
            t[z].ch[t[z].ch[1] == y] = x;
        }
        t[x].fa = z;
        t[x].ch[k ^ 1] = y;
        t[y].fa = x;
        t[y].ch[k] = w;
        if (w)
        {
            t[w].fa = y;
        }
        pull(y);
        pull(x);
    }

    void splay(int x)
    {
        // x 是节点；先自顶向下下传标记，再把 x 旋到所在 splay 的根。
        auto pushAll = [&](auto &&self, int y) -> void
        {
            if (!isRoot(y))
```

```
            {
                self(self, t[y].fa);
            }
            push(y);
        };
        pushAll(pushAll, x);
        while (!isRoot(x))
        {
            int y = t[x].fa, z = t[y].fa;
            if (!isRoot(y))
            {
                rotate((t[y].ch[1] == x) == (t[z].ch[1] == y) ? y : x);
            }
            rotate(x);
        }
    }

    void access(int x)
    {
        // x 是节点；打通根到 x 的偏好路径。
        int y = 0;
        while (x)
        {
            splay(x);
            int p = t[x].fa;
            t[x].ch[1] = y;
            pull(x);
            y = x;
            x = p;
        }
    }

    void makeRoot(int x)
    {
        // x 是节点；令 x 成为所在树的根。
        access(x);
        splay(x);
        reverseNode(x);
    }

    int findRoot(int x)
    {
        // x 是节点；返回 x 所在树的根。
        access(x);
        splay(x);
        while (true)
        {
            push(x);
            if (!t[x].ch[0])
            {
                break;
            }
```

```
            x = t[x].ch[0];
        }
        splay(x);
        return x;
    }

    void linkNode(int x, int y)
    {
        // x、y 在不同树中；连接 x 与 y。
        makeRoot(x);
        t[x].fa = y;
    }

    void cutNode(int x, int y)
    {
        // x、y 之间有树边；删除这条边。
        makeRoot(x);
        access(y);
        splay(y);
        assert(t[y].ch[0] == x && !t[x].ch[1]); // 调试检查，可删。
        t[y].ch[0] = t[x].fa = 0;
        pull(y);
    }

    int pathMax(int x, int y)
    {
        // x、y 连通；返回 x 到 y 路径上权值最大的边节点。
        makeRoot(x);
        access(y);
        splay(y);
        return t[y].who;
    }

    int newEdge(int u, int v, long long w)
    {
        // u、v 是 1 起端点，w 是边权；新建边节点并返回其编号。
        Node x;
        x.edge = true;
        x.val = x.mx = w;
        x.u = u;
        x.v = v;
        t.push_back(x);
        int id = t.size() - 1;
        t[id].who = id;
        return id;
    }

    bool add(int u, int v, long long w)
    {
        // u、v 是端点，w 是新边权；森林边集改变时返回 true。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        if (u == v)
```

```
        {
            return false;
        }
        u++;
        v++;
        if (findRoot(u) != findRoot(v))
        {
            int e = newEdge(u, v, w);
            linkNode(u, e);
            linkNode(e, v);
            sum += w;
            cc--;
            return true;
        }
        int e = pathMax(u, v);
        if (t[e].val <= w)
        {
            return false;
        }
        long long old = t[e].val;
        cutNode(e, t[e].u);
        cutNode(e, t[e].v);
        int ne = newEdge(u, v, w);
        linkNode(u, ne);
        linkNode(ne, v);
        sum += w - old;
        return true;
    }

    int weight() const
    {
        // 无参数；返回当前最小生成森林总权。
        return sum;
    }

    int comps() const
    {
        // 无参数；返回当前连通块数。
        return cc;
    }
};
```

构造 `IncrMSF q(n)`，按到达顺序调用 `q.add(u,v,w)`；`weight()` 始终是当前所有连通块的最小生成森林总权，`comps()` 给连通块数。调用者必须保证每次总权都可由 64 位有符号整数表示。每条边至多新建一个边点，单次加入均摊 $O(\log(n+m))$，总空间 $O(n+m)$；任意旧边修改仍用 5.5.004 带修改最小生成树 的重算版或另写完整动态算法。

下面按顺序加入边，检查成环时拒绝较重边、替换较重树边、负边以及自环：

```
signed main()
{
```

```
    IncrMSF q(4);
    q.add(0, 1, 5);
    q.add(1, 2, 4);
    q.add(0, 2, 10);
    assert(q.weight() == 9 && q.comps() == 2);
    q.add(0, 2, 1);
    q.add(2, 3, -2);
    q.add(0, 3, 6);
    q.add(3, 3, -7);
    assert(q.weight() == 3 && q.comps() == 1);
}
```

5.5.012 最小直径生成树

一般连通非负权无向图中，最小直径生成树不一定是 MST。最优树可以视为从某个绝对中心向各点选最短路：中心要么是顶点，要么位于一条原图边内部。先从每个顶点跑 Dijkstra 得全源最短路并按距离降序排列点；顶点中心的直径是两倍最大距离，边 uv 上的中心则扫描两端最远点归属变化的位置，候选直径由 $d(u,x)+w(u,v)+d(v,y)$ 给出。所有边权先乘二，使边内部的半整数中心仍能精确表示，最终恢复中心最短路树并把答案除二。

```
struct MinDiamTree
{
    struct Edge
    {
        int to; // to 是边的另一端。
        long long w; // w 是加倍后的边权。
    };

    struct State
    {
        long long d; // d 是当前距离。
        int u, fa; // u 是出堆点，fa 是它在最短路树上的前驱。

        bool operator>(const State &o) const
        {
            // o 是另一个状态；按 (d,u,fa) 字典序比较，供小根堆使用。
            return tie(d, u, fa) > tie(o.d, o.u, o.fa);
        }
    };

    int n; // n 是点数。
    vector<vector<Edge>> g; // g 是邻接表，边权已加倍以避免中点出现半整数。
    bool valid = true; // valid 为假表示加入过非法边或点数非正。
    __int128 totalWeight2 = 0; // totalWeight2 是全部加倍边权之和，用于检查溢出。

    MinDiamTree(int n_) : n(max(0LL, n_)), g(max(0LL, n_))
    {
        // n_ 是点数；构造空图，点数非正时标记为无效。
        valid = n_ > 0;
```

```
    }

    bool addEdge(int u, int v, long long w)
    {
        // u、v 是端点，w 是非负边权；合法时加入无向边并返回 true，否则标记无效并返回 false。
        if (u < 0 || u >= n || v < 0 || v >= n || w < 0 ||
            (__int128)w * 2 + totalWeight2 >= numeric_limits<long long>::max() / 4)
        {
            valid = false;
            return false;
        }
        totalWeight2 += (__int128)w * 2;
        g[u].push_back({v, w * 2});
        g[v].push_back({u, w * 2});
        return true;
    }

    pair<vector<long long>, vector<int>> dijkstra(int s, const vector<vector<Edge>> &a) const
    {
        // s 是源点，a 是邻接表；返回各点距离与最短路树前驱（源点前驱为自身，不可达为 -1）。
        int m = a.size();
        const long long INF = numeric_limits<long long>::max() / 4;
        vector<long long> dis(m, INF);
        vector<int> fa(m, -1);
        vector<char> vis(m);
        priority_queue<State, vector<State>, greater<State>> q;
        q.push({0, s, s});
        while (!q.empty())
        {
            auto [d, u, p] = q.top();
            q.pop();
            if (vis[u])
            {
                continue;
            }
            vis[u] = true;
            dis[u] = d;
            fa[u] = p;
            for (auto [v, w] : a[u])
            {
                if (!vis[v])
                {
                    if (d <= INF - w) q.push({d + w, v, u});
                }
            }
        }
        return {dis, fa};
    }

    optional<pair<long long, vector<pair<int, int>>>> solve() const
    {
        // 无参数；返回最小直径与一组生成树边，图非法或不连通时返回空。
```

```
        if (!valid || n == 0) return nullopt;
        const long long INF = numeric_limits<long long>::max() / 4;
        vector<vector<long long>> dis(n);
        vector<vector<int>> ord(n, vector<int>(n));
        long long bestP = INF;
        int center = 0;
        for (int s = 0; s < n; s++)
        {
            dis[s] = dijkstra(s, g).first;
            iota(ord[s].begin(), ord[s].end(), 0);
            stable_sort(ord[s].begin(), ord[s].end(), [&](int x, int y)
            {
                return dis[s][x] > dis[s][y];
            });
            if (dis[s][ord[s][0]] >= INF) return nullopt;
            long long cur = dis[s][ord[s][0]] * 2;
            if (cur < bestP)
            {
                bestP = cur;
                center = s;
            }
        }

        long long bestEdge = INF, left = 0, right = 0;
        int cu = 0, cv = 0;
        for (int u = 0; u < n; u++)
        {
            for (auto [v, w] : g[u])
            {
                if (v <= u)
                {
                    continue;
                }
                int y = ord[u][0];
                for (int x : ord[u])
                {
                    if (dis[v][x] > dis[v][y])
                    {
                        long long cur = dis[v][y] + dis[u][x] + w;
                        if (cur < bestEdge)
                        {
                            bestEdge = cur;
                            cu = u;
                            cv = v;
                            left = cur / 2 - dis[u][x];
                            right = w - left;
                        }
                        y = x;
                    }
                }
            }
        }
```

```
134
135         vector<vector<Edge>> a = g;
136         int root = center;
137         bool inside = bestEdge < bestP;
138         if (inside)
139         {
140             root = n;
141             a.push_back({});
142             a[root].push_back({cu, left});
143             a[root].push_back({cv, right});
144         }
145         auto fa = dijkstra(root, a).second;
146         vector<pair<int, int>> tree;
147         if (inside)
148         {
149             for (int u = 0; u < n; u++)
150             {
151                 if (u != cu && u != cv)
152                 {
153                     tree.push_back({u, fa[u]});
154                 }
155             }
156             tree.push_back({cu, cv});
157         }
158         else
159         {
160             for (int u = 0; u < n; u++)
161             {
162                 if (u != root)
163                 {
164                     tree.push_back({u, fa[u]});
165                 }
166             }
167         }
168         return pair<long long, vector<pair<int, int>>>{min(bestP, bestEdge) / 2, move(tree)};
169     }
170 };
```

构造 `MinDiamTree q(n)`、加入全部无向边后调用 `q.solve()`；成功返回 `{最小直径, 树边端点对}`，非法边或非连通图返回 `nullopt`。边权非负，所有加倍边权之和须小于 `LLONG_MAX/4`。平行边只按端点恢复，输出原边编号时应在该端点对中取最轻边；自环不会改善答案。n 次 Dijkstra 加边中心扫描共 $O(nm\log n+nm)$ 时间、$O(n^2+m)$ 空间，适合 $n,m\le 2000$。只求最小权和仍用 5.5.001 Kruskal。

下面检查单点与三角形；三角形两条权为 1 的边构成直径为 2 的最优生成树：

```
1 signed main()
2 {
3     MinDiamTree one(1);
4     auto [d0, t0] = *one.solve();
5     assert(d0 == 0 && t0.empty());
6
```

```
    MinDiamTree q(3);
    q.addEdge(0, 1, 1);
    q.addEdge(1, 2, 1);
    q.addEdge(0, 2, 3);
    auto [d, tree] = *q.solve();
    assert(d == 2 && tree.size() == 2);
}
```

5.5.013 Prim：稀疏图与稠密图

Prim 从一个点开始维护每个未选点到当前生成树的最轻连接边，每轮取连接代价最小的点并更新邻点。邻接表配优先队列适合稀疏图，邻接矩阵直接扫描适合边数接近 n^2 的稠密图；二者都保留父亲数组，避免只算权值却无法恢复树。负权无向边可以直接使用，图不连通时返回空。

```
struct PrimResult
{
    int sum;        // sum 保存最小生成树边权和。
    vector<int> fa; // fa[v] 保存 v 的父亲，根为 -1。
};

optional<PrimResult> primSparse(const vector<vector<pair<int, int>>> &g,
                                int s = 0)
{
    // g[u] 的每项是 {邻点, 边权}，s 是根；连通时返回 MST，否则返回空。
    int n = g.size();
    assert(n > 0 && 0 <= s && s < n); // 调试检查，可删
    vector<int> d(n, INF), fa(n, -1);
    vector<char> vis(n);
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> q;
    d[s] = 0;
    q.push({0, s});
    int sum = 0, cnt = 0;
    while (!q.empty())
    {
        auto [w, u] = q.top();
        q.pop();
        if (vis[u])
        {
            continue;
        }
        vis[u] = true;
        sum += w;
        cnt++;
        for (auto [v, c] : g[u])
        {
            if (!vis[v] && c < d[v])
            {
                d[v] = c;
                fa[v] = u;
                q.push({c, v});
```

```
            }
        }
    }
    if (cnt != n)
    {
        return nullopt;
    }
    return PrimResult{sum, move(fa)};
}

optional<PrimResult> primDense(const vector<vector<int>> &w, int s = 0)
{
    // w 是对称邻接矩阵，缺边填 INF，s 是根；连通时返回 MST，否则返回空。
    int n = w.size();
    assert(n > 0 && 0 <= s && s < n); // 调试检查，可删
    for (const auto &row : w)
    {
        assert((int)row.size() == n); // 调试检查，可删
    }
    vector<int> d(n, INF), fa(n, -1);
    vector<char> vis(n);
    d[s] = 0;
    int sum = 0;
    for (int t = 0; t < n; t++)
    {
        int u = -1;
        for (int v = 0; v < n; v++)
        {
            if (!vis[v] && (u < 0 || d[v] < d[u]))
            {
                u = v;
            }
        }
        if (u < 0 || d[u] == INF)
        {
            return nullopt;
        }
        vis[u] = true;
        sum += d[u];
        for (int v = 0; v < n; v++)
        {
            if (!vis[v] && w[u][v] < d[v])
            {
                d[v] = w[u][v];
                fa[v] = u;
            }
        }
    }
    return PrimResult{sum, move(fa)};
}
```

`primSparse(g,s)` 的时间为 $O((n+m)\log n)$、空间为 $O(n+m)$；`primDense(w,s)` 的时间为 $O(n^2)$、

除矩阵外空间为 $O(n)$。两版都要求无向边在邻接表或矩阵中对称出现，权值和须装入 64 位；只已有边表并需要排序时 5.5.001 Kruskal 通常更短，需要在线维护森林时用 5.5.011 增量最小生成森林。

```cpp
signed main()
{
    vector<vector<pair<int, int>>> g(4);
    auto add = [&](int u, int v, int w)
    {
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    };
    add(0, 1, 1), add(1, 2, 2), add(2, 3, 3), add(0, 3, 9);
    assert(primSparse(g)->sum == 6);
}
```

5.5.014 Borůvka

Borůvka 算法维护当前最小生成森林，并在每一轮为每个连通块选择一条权值最小的出边。割性质保证这些边都可以安全加入某棵最小生成树；合并所有仍连接不同块的候选边后，连通块数至少减半，因此最多进行 $O(\log n)$ 轮。它不需要预排序，适合边按批次扫描、并行寻找各块最轻边，或作为其他最小生成树算法的收缩阶段；普通一次性边表通常仍优先使用更短的 Kruskal。

以下实现显式依赖 5.5.001 Kruskal 已定义的 `DSU` 与 `MEdge`，不重复实现并查集和边记录：

```cpp
optional<pair<int, vector<MEdge>>> boruvka(int n, const vector<MEdge> &e)
{
    // n 是点数，e 是无向边集；返回 MST 总权与所选边，不连通时返回空。
    assert(n >= 0); // 调试检查，可删
    DSU d(n);
    int cnt = n, ans = 0;
    vector<MEdge> use;
    while (cnt > 1)
    {
        vector<int> best(n, -1);
        for (int i = 0; i < (int)e.size(); i++)
        {
            int x = d.find(e[i].x), y = d.find(e[i].y);
            if (x == y)
            {
                continue;
            }
            // 求最大生成树时，下面两处 < 要同时改成 >，其余逻辑不变。
            if (best[x] == -1 || e[i].w < e[best[x]].w)
            {
                best[x] = i;
            }
            if (best[y] == -1 || e[i].w < e[best[y]].w)
            {
                best[y] = i;
            }
```

```
        }
        bool upd = false;
        for (int x = 0; x < n; x++)
        {
            if (d.find(x) != x || best[x] == -1)
            {
                continue;
            }
            auto v = e[best[x]];
            if (d.merge(v.x, v.y))
            {
                ans += v.w;
                use.push_back(v);
                cnt--;
                upd = true;
            }
        }
        if (!upd)
        {
            return nullopt;
        }
    }
    return pair{ans, use};
}
```

调用 `boruvka(n,e)`，成功时返回最小生成树总权和按加入顺序排列的 $n-1$ 条边，图不连通时返回空；$n=0$ 或 $n=1$ 时返回权和 0 的空树。输入是无向边表，每条边只需出现一次，允许负权、重边和自环，端点必须落在 $[0,n)$，总权须能由 64 位 `int` 表示。每轮扫描全部 m 条边并做并查集操作，共 $O(m\log n\,\alpha(n))$ 时间、$O(n)$ 额外空间。求最大生成树时只需把代码中两处候选比较的 `<` 同时改为 `>`，无穷单位元和其余分支都不变；只改一处会破坏各连通块选择方向的一致性。已有边表且排序方便时使用 5.5.001 Kruskal，已有邻接表或矩阵时使用 5.5.013 Prim。

下面检查负权、重边和生成树恢复：

```
signed main()
{
    vector<MEdge> e{{0, 1, 4}, {1, 2, -2}, {2, 3, 5},
                    {0, 3, 10}, {0, 2, 1}, {0, 2, 8}};
    auto ans = boruvka(4, e);
    assert(ans.has_value());
    assert(ans->first == 4 && ans->second.size() == 3);
}
```

5.6 二分图匹配与覆盖

5.6.001 匈牙利算法

匈牙利算法求二分图最大基数匹配。核心是增广路：一条从未匹配左点出发、交替走「非匹配边、匹配边」并终止于未匹配右点的路径，把它上面的匹配 / 非匹配状态整体翻转，匹配数就增加 1。Berge 定理保证

当且仅当不存在增广路时匹配最大。实现逐个未匹配左点做一次 DFS 找增广路，用时间戳 `vis[v] == ti` 代替每轮清空访问标记（省去 $O(V)$ 重置）。代码短、常数小，适合中小稀疏图；稠密或大图应换下面的位运算版或 Hopcroft-Karp。

```
class Hungarian
{
    // nl、nr 是左右部点数，g[u] 保存左点 u 的全部邻点。
    int nl, nr;
    vector<vector<int>> g;
    // ml[u]、mr[v] 分别保存左右点当前匹配点，-1 表示未匹配。
    vector<int> ml, mr;
    // vis[v] 是本轮增广是否已经访问右点 v 的时间戳。
    vector<int> vis;
    int ti = 0;

    bool dfs(int u)
    {
        // u 是尝试增广的左点；找到增广路并改写匹配时返回 true。
        for (auto v : g[u])
        {
            if (vis[v] == ti)
            {
                continue;
            }
            vis[v] = ti;
            if (mr[v] == -1 || dfs(mr[v]))
            {
                ml[u] = v;
                mr[v] = u;
                return true;
            }
        }
        return false;
    }

  public:
    Hungarian(int nl, int nr) : nl(nl), nr(nr), g(nl), ml(nl, -1), mr(nr, -1), vis(nr)
    {
        // nl、nr 是左右部点数；构造空二分图，无返回值。
    }

    void addEdge(int u, int v)
    {
        // u 是左点，v 是右点；加入一条二分图边，无返回值。
        assert(0 <= u && u < nl && 0 <= v && v < nr); // 调试检查，可删。
        g[u].push_back(v);
    }

    int matching()
    {
        // 无参数；从空匹配重算并返回最大匹配大小。
        fill(ml.begin(), ml.end(), -1);
```

```
        fill(mr.begin(), mr.end(), -1);
        fill(vis.begin(), vis.end(), 0);
        ti = 0;
        for (int u = 0; u < nl; u++)
        {
            if (ml[u] == -1)
            {
                ti++;
                dfs(u);
            }
        }
        return count_if(ml.begin(),
                        ml.end(),
                        [](int v)
                        {
                            return v != -1;
                        });
    }

    const vector<int> &leftMatch() const
    {
        // 无参数；返回每个左点匹配的右点，未匹配为 -1。
        return ml;
    }

    const vector<int> &matchR() const
    {
        // 无参数；返回每个右点匹配的左点，未匹配为 -1。
        return mr;
    }
};
```

构造 `Hungarian q(nl,nr)` 后加边，`matching()` 每次从空匹配重算，之后用 `leftMatch()`、`matchR()` 读取互逆匹配数组。最坏用 $O(VE)$ 时间、$O(V+E)$ 空间。稠密图见 5.6.002 Bitset 优化匈牙利，大规模稀疏图见 5.6.003 Hopcroft-Karp。

5.6.002 Bitset 优化匈牙利

仍是 5.6.001 匈牙利算法 的增广路思想，只是把「遍历左点 u 的邻居中尚未访问者」用位运算加速：邻接集合按右点编号压进若干个 64 位块，本轮访问集合 `vis` 同样是位集，`g[u][b] & ~vis[b]` 一次取出该块里所有「相邻且未访问」的右点，`countr_zero` 逐位弹出。这样在稠密二分图上把内层邻居扫描的常数压到 $1/64$。匹配正确性完全来自增广路翻转，与朴素版一致。

```
class BitHungarian
{
    // nl、nr 是左右部点数，w 是存放 nr 位所需的 64 位块数。
    int nl, nr, w;
    // g[u][b] 的第 k 位表示左点 u 是否连接右点 64b+k。
    vector<vector<unsigned long long>> g;
```

```
    // ml、mr 是左右匹配点，vis 是一轮增广中访问过的右点位集。
    vector<int> ml, mr;
    vector<unsigned long long> vis;

    bool dfs(int u)
    {
        // u 是尝试增广的左点；找到增广路并改写匹配时返回 true。
        for (int b = 0; b < w; b++)
        {
            auto x = g[u][b] & ~vis[b];
            while (x)
            {
                int k = countr_zero(x);
                x &= x - 1;
                int v = b * 64 + k;
                if (v >= nr)
                {
                    continue;
                }
                vis[b] |= 1ULL << k;
                if (mr[v] == -1 || dfs(mr[v]))
                {
                    ml[u] = v;
                    mr[v] = u;
                    return true;
                }
                x &= ~vis[b];
            }
        }
        return false;
    }

public:
    BitHungarian(int nl, int nr)
        : nl(nl), nr(nr), w((nr + 63) / 64), g(nl), ml(nl, -1), mr(nr, -1), vis(w)
    {
        // nl、nr 是左右部点数；构造位集邻接矩阵，无返回值。
        for (auto &row : g)
        {
            row.assign(w, 0);
        }
    }

    void addEdge(int u, int v)
    {
        // u 是左点，v 是右点；把对应邻接位设为 1，无返回值。
        assert(0 <= u && u < nl && 0 <= v && v < nr); // 调试检查，可删。
        g[u][v / 64] |= 1ULL << (v % 64);
    }

    int matching()
    {
```

```
        // 无参数；从空匹配重算并返回最大匹配边数。
        fill(ml.begin(), ml.end(), -1);
        fill(mr.begin(), mr.end(), -1);
        int ans = 0;
        for (int u = 0; u < nl; u++)
        {
            fill(vis.begin(), vis.end(), 0);
            ans += dfs(u);
        }
        return ans;
    }

    const vector<int> &leftMatch() const
    {
        // 无参数；返回每个左点匹配的右点，未匹配为 -1。
        return ml;
    }

    const vector<int> &matchR() const
    {
        // 无参数；返回每个右点匹配的左点，未匹配为 -1。
        return mr;
    }
};
```

接口与 5.6.001 匈牙利算法 相同（`addEdge` 写位、`matching()` 返回最大匹配数），适合右部不太大且边稠密的图；右部极大而边稀疏时位集反而浪费内存，用普通邻接表版更省。保守最坏 $O(n_l^2\lceil n_r/64\rceil + n_l E)$ 时间、$O(n_l\lceil n_r/64\rceil + n_r)$ 空间。要更好的最坏渐进界用 5.6.003 Hopcroft-Karp。

5.6.003 Hopcroft-Karp

Hopcroft-Karp 把匈牙利的「逐条增广」升级成「逐层批量增广」。每轮先做一次 BFS，从所有未匹配左点出发按交替图给左点分层 `dep`；再用受层次约束的 DFS 一次找出一批点不相交的增广路同时翻转。这里完整展开可达层，并允许 DFS 在任意层接到未匹配右点；仍只沿层数递增的匹配边前进，故每轮得到极大增广路集。`dfs` 失败时把 `dep[u]` 置 -1 剪枝，`it` 记录本轮当前弧，避免重复扫描已经失败的边。邻接点与热点状态使用固定 32 位整数，不受公共前导的 `#define int long long` 放大；BFS 队列也预分配并复用。下面先给适合随时加边的邻接表版。

```
class HopcroftKarp
{
    // nl、nr 是左右部点数，g 是从左部出发的邻接表。
    int nl, nr;
    vector<vector<int>> g;
    // ml、mr 是左右匹配点；dep、it、que 分别是层数、当前弧和复用队列。
    vector<int> ml, mr, dep, it, que;

    bool bfs()
    {
        // 无参数；给交替图分层，存在增广路时返回 true。
```

```
        fill(dep.begin(), dep.end(), -1);
        int ql = 0, qr = 0;
        bool found = false;
        for (int u = 0; u < nl; u++)
        {
            if (ml[u] == -1)
            {
                dep[u] = 0;
                que[qr++] = u;
            }
        }
        while (ql < qr)
        {
            int u = que[ql++];
            for (int v : g[u])
            {
                int x = mr[v];
                if (x == -1)
                {
                    found = true;
                }
                else if (dep[x] == -1)
                {
                    dep[x] = dep[u] + 1;
                    que[qr++] = x;
                }
            }
        }
        return found;
    }

    bool dfs(int u)
    {
        // u 是当前左点；沿 BFS 层寻找增广路，成功时返回 true。
        for (int &i = it[u]; i < (int)g[u].size(); i++)
        {
            int v = g[u][i];
            int x = mr[v];
            if (x == -1)
            {
                ml[u] = v;
                mr[v] = u;
                return true;
            }
            if (x != -1 && dep[x] == dep[u] + 1 && dfs(x))
            {
                ml[u] = v;
                mr[v] = u;
                return true;
            }
        }
        dep[u] = -1;
```

```
        return false;
    }

public:
    HopcroftKarp(int nl, int nr)
        : nl(nl), nr(nr), g(nl), ml(nl, -1), mr(nr, -1), dep(nl), it(nl), que(nl)
    {
        // nl、nr 是左右部点数；构造空二分图，无返回值。
        assert(nl <= numeric_limits<int>::max() && nr <= numeric_limits<int>::max()); // 调试检
        ↪ 查，可删。
    }

    void reserve(int u, int m)
    {
        // u 是左点，m 是预计邻边数；预留空间但不改变图。
        assert(0 <= u && u < nl && m >= 0); // 调试检查，可删。
        g[u].reserve(m);
    }

    void addEdge(int u, int v)
    {
        // u 是左点，v 是右点；加入一条边，无返回值。
        assert(0 <= u && u < nl && 0 <= v && v < nr); // 调试检查，可删。
        g[u].push_back((int)v);
    }

    int matching()
    {
        // 无参数；从空匹配重算并返回最大匹配边数。
        fill(ml.begin(), ml.end(), -1);
        fill(mr.begin(), mr.end(), -1);
        int ans = 0;
        while (bfs())
        {
            fill(it.begin(), it.end(), 0);
            for (int u = 0; u < nl; u++)
            {
                if (ml[u] == -1)
                {
                    ans += dfs(u);
                }
            }
        }
        return ans;
    }

    vector<int> leftMatch() const
    {
        // 无参数；返回左点到右点的匹配数组，未匹配为 -1。
        return {ml.begin(), ml.end()};
    }

```

```
    vector<int> matchR() const
    {
        // 无参数；返回右点到左点的匹配数组，未匹配为 -1。
        return {mr.begin(), mr.end()};
    }
};
```

构造 `HopcroftKarp q(nl, nr)`，加边后 `matching()` 返回最大匹配数，`leftMatch()` / `matchR()` 取匹配（-1 未匹配）；点从 0 编号。已知左点 u 的邻边数时可先调用 `reserve(u,m)` 避免扩容；因内部使用 32 位编号，左右部点数都不能超过 `numeric_limits<int>::max()`。匹配数组接口按公共 `int` 类型返回一份副本。$O(E\sqrt{V})$ 时间、$O(V+E)$ 空间。它允许随时向各左点的邻接表追加边，接口最直接；若边在求解前一次给齐且更看重连续存储，改用 5.6.004 CSR Hopcroft-Karp。

5.6.004 CSR Hopcroft-Karp

CSR 版仍做完全相同的 BFS 分层和 DFS 批量增广，只把每个左点各自持有的邻接表压成连续的 `to` 数组。`start[u]` 与 `start[u+1]` 圈出左点 u 的邻边区间，免去 n_l 次独立分配并改善顺序扫描的局部性；代价是求解前必须按当前边集重建 CSR，不适合频繁加边、求解、再加边的交替流程。它是确定性实现，不增加失败概率，也不依赖平台指令。

```
// 依赖 5.6.001 匈牙利算法
// 依赖 5.6.002 Bitset 优化匈牙利
// 依赖 5.6.003 Hopcroft-Karp
class HopKarpCSR
{
    int nl, nr; // nl、nr 是左右部点数。
    // edge[i][0]、edge[i][1] 是第 i 条边的左右端点。
    vector<array<int, 2>> edge;
    // 左点 u 的邻边存放在 to[start[u]..start[u+1])。
    vector<int> start, to;
    // ml、mr 是左右匹配点，dep 是左点层数，lim 是本轮最短增广路长度。
    vector<int> ml, mr, dep;
    int lim = -1;

    void build()
    {
        // 无参数；按当前边集重建 CSR 邻接表，无返回值。
        start.assign(nl + 1, 0);
        for (auto e : edge)
        {
            start[e[0] + 1]++;
        }
        partial_sum(start.begin(), start.end(), start.begin());
        vector<int> cur = start;
        to.assign(edge.size(), 0);
        for (auto e : edge)
        {
            to[cur[e[0]]++] = e[1];
        }
```

```
    }

    bool bfs()
    {
        // 无参数；给全部最短增广路分层，存在增广路时返回 true。
        queue<int> q;
        fill(dep.begin(), dep.end(), -1);
        lim = -1;
        for (int u = 0; u < nl; u++)
        {
            if (ml[u] == -1)
            {
                dep[u] = 0;
                q.push(u);
            }
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            if (lim != -1 && dep[u] >= lim)
            {
                continue;
            }
            for (int i = start[u]; i < start[u + 1]; i++)
            {
                int x = mr[to[i]];
                if (x == -1)
                {
                    lim = dep[u] + 1;
                }
                else if (dep[x] == -1 && (lim == -1 || dep[u] + 1 < lim))
                {
                    dep[x] = dep[u] + 1;
                    q.push(x);
                }
            }
        }
        return lim != -1;
    }

    bool dfs(int u)
    {
        // u 是当前左点；沿 BFS 层寻找最短增广路，成功时返回 true。
        for (int i = start[u]; i < start[u + 1]; i++)
        {
            int v = to[i];
            int x = mr[v];
            if (x == -1 && dep[u] + 1 == lim)
            {
                ml[u] = v;
                mr[v] = u;
```

```
                return true;
            }
            if (x != -1 && dep[x] == dep[u] + 1 && dfs(x))
            {
                ml[u] = v;
                mr[v] = u;
                return true;
            }
        }
        dep[u] = -1;
        return false;
    }

public:
    HopKarpCSR(int nl, int nr) : nl(nl), nr(nr), ml(nl, -1), mr(nr, -1), dep(nl)
    {
        // nl、nr 是左右部点数；构造空边集，无返回值。
    }

    void addEdge(int u, int v)
    {
        // u 是左点，v 是右点；暂存一条边，无返回值。
        assert(0 <= u && u < nl && 0 <= v && v < nr); // 调试检查，可删。
        edge.push_back({u, v});
    }

    int matching()
    {
        // 无参数；重建 CSR，并从空匹配重算最大匹配边数。
        build();
        fill(ml.begin(), ml.end(), -1);
        fill(mr.begin(), mr.end(), -1);
        int ans = 0;
        while (bfs())
        {
            for (int u = 0; u < nl; u++)
            {
                if (ml[u] == -1)
                {
                    ans += dfs(u);
                }
            }
        }
        return ans;
    }

    const vector<int> &leftMatch() const
    {
        // 无参数；返回左点到右点的匹配数组，未匹配为 -1。
        return ml;
    }

```

```
    const vector<int> &matchR() const
    {
        // 无参数；返回右点到左点的匹配数组，未匹配为 -1。
        return mr;
    }
};
```

CSR 版与 5.6.003 Hopcroft-Karp 的正确性、$O(E\sqrt{V})$ 时间和 $O(V+E)$ 峰值空间界完全相同；它少做小 `vector` 分配，但每次 `matching()` 多做一次 $O(V+E)$ 建表。大规模静态稀疏图可优先试 CSR，动态图式调用仍用邻接表版；小规模或代码从简用 5.6.001 匈牙利算法，稠密图用 5.6.002 Bitset 优化匈牙利，带权最优匹配见 5.6.005 二分图最大权 / 最小权匹配（KM）。下面每个实现分别在一张已知答案为 3 的图上检查，不互相比较内部状态。

```
template <class F>
void testMatching()
{
    // F 是待检查的匹配结构；构造存在大小为 3 的完美匹配的图，无返回值。
    F g(3, 3);
    for (auto [u, v] : vector<array<int, 2>>{{0, 0}, {0, 1}, {1, 1}, {1, 2}, {2, 0}})
    {
        g.addEdge(u, v);
    }
    assert(g.matching() == 3);
}

signed main()
{
    testMatching<Hungarian>();
    testMatching<BitHungarian>();
    testMatching<HopcroftKarp>();
    testMatching<HopKarpCSR>();
    return 0;
}
```

5.6.005 二分图最大权/最小权匹配（KM）

带权二分图完备匹配（指派问题）要在 n 行 m 列（$n \le m$）的权值矩阵里给每行选一个不同的列，使总权最优。这里用最短增广路形式的匈牙利指派实现：为左点 `u` 和右点 `v` 维护可行顶标，保持任意边的归约代价 $c(i,j)-u_i-v_j \ge 0$；每次从一个新左点出发在「相等子图」（归约代价为 0 的边）里用最短增广路 + `way` 记录前驱地扩展，找不到增广时按未访问列的最小松弛量 `delta` 整体调整顶标，使至少一条新边进入相等子图，直到增广成功。求最大权时对权取负即可（`mx` 参数），故一份代码兼顾最大 / 最小。

```
template<class Acc = long long>
struct Assignment
{
    // cost 是最优匹配总权。
    Acc cost;
    // col[i] 是第 i 行匹配的列编号。
    vector<int> col;
```

```cpp
};

template<class Acc = long long>
optional<Assignment<Acc>> assignment(const vector<vector<long long>> &a, bool mx = false)
{
    // a 是权值矩阵，mx 表示求最大权；返回最优完备匹配，不存在时返回空。
    static_assert(numeric_limits<Acc>::is_integer && numeric_limits<Acc>::is_signed &&
                  sizeof(Acc) >= sizeof(long long));
    int n = a.size(), m = n ? a[0].size() : 0;
    for (const auto &row : a)
    {
        if ((int)row.size() != m)
        {
            return nullopt;
        }
    }
    if (n > m)
    {
        return nullopt;
    }
    auto add = []<class T>(T x, T y) -> optional<T>
    {
        // x、y 是同类型整数；和超出该类型范围时返回空。
        if ((y > 0 && x > numeric_limits<T>::max() - y) ||
            (y < 0 && x < numeric_limits<T>::min() - y))
        {
            return nullopt;
        }
        return x + y;
    };
    auto sub = []<class T>(T x, T y) -> optional<T>
    {
        // x、y 是同类型整数；差超出该类型范围时返回空。
        if ((y > 0 && x < numeric_limits<T>::min() + y) ||
            (y < 0 && x > numeric_limits<T>::max() + y))
        {
            return nullopt;
        }
        return x - y;
    };
    vector<Acc> u(n + 1), v(m + 1);
    vector<int> p(m + 1), way(m + 1);
    for (int i = 1; i <= n; i++)
    {
        p[0] = i;
        int j0 = 0;
        vector<optional<Acc>> d(m + 1);
        vector<bool> used(m + 1);
        do
        {
            used[j0] = true;
            int i0 = p[j0], j1 = 0;
```

```
            optional<Acc> delta;
            for (int j = 1; j <= m; j++)
            {
                if (used[j])
                {
                    continue;
                }
                Acc cur = (Acc)a[i0 - 1][j - 1];
                if (mx)
                {
                    auto z = sub((Acc)0, cur);
                    if (!z)
                    {
                        return nullopt;
                    }
                    cur = *z;
                }
                auto z = sub(cur, u[i0]);
                if (!z)
                {
                    return nullopt;
                }
                z = sub(*z, v[j]);
                if (!z)
                {
                    return nullopt;
                }
                cur = *z;
                if (!d[j] || cur < *d[j])
                {
                    d[j] = cur;
                    way[j] = j0;
                }
                if (!delta || *d[j] < *delta)
                {
                    delta = d[j];
                    j1 = j;
                }
            }
            assert(delta); // 调试检查，可删：n<=m 时总有未访问列。
            for (int j = 0; j <= m; j++)
            {
                if (used[j])
                {
                    auto x = add(u[p[j]], *delta);
                    if (!x)
                    {
                        return nullopt;
                    }
                    u[p[j]] = *x;
                    if (j)
                    {
```

```
                        x = sub(v[j], *delta);
                        if (!x)
                        {
                            return nullopt;
                        }
                        v[j] = *x;
                    }
                }
            }
            for (int j = 1; j <= m; j++)
            {
                if (!used[j] && d[j])
                {
                    auto x = sub(*d[j], *delta);
                    if (!x)
                    {
                        return nullopt;
                    }
                    d[j] = *x;
                }
            }
            j0 = j1;
        } while (p[j0]);
        do
        {
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        } while (j0);
    }
    vector<int> col(n);
    for (int j = 1; j <= m; j++)
    {
        if (p[j])
        {
            col[p[j] - 1] = j - 1;
        }
    }
    Acc cost = 0;
    for (int i = 0; i < n; i++)
    {
        auto x = add(cost, (Acc)a[i][col[i]]);
        if (!x)
        {
            return nullopt;
        }
        cost = *x;
    }
    return Assignment<Acc>{cost, col};
}
```

`assignment` 的 `Acc` 默认是 `long long`，必须是至少与输入同宽的有符号整数；`mx=true` 求最大权，否则

求最小权。矩阵的每个位置都是有效边，不能用极值表示缺边。返回的 `cost` 为 `Acc`，锯齿矩阵、行数大于列数或中间值越界时返回空；顶标或总权可能超过 64 位时可用 `assignment<__int128>(a,mx)`。算法用 $O(n^2m)$ 时间、$O(n+m)$ 额外空间。

```
signed main()
{
    vector<vector<long long>> a{{4, 1, 3, 8},
                                {2, 0, 5, 7},
                                {3, 2, 2, 6}};
    assert(assignment(a, false)->cost == 5);
    assert(assignment(a, true)->cost == 16);
}
```

5.6.006 二分图边染色

König 边染色定理：二分图的边色数恰等于最大度 Δ。构造性证明也给出算法：把两侧补齐成同样多的点、必要时添加虚拟边使每个点度数都恰为 Δ，得到一个 Δ-正则二分多重图；正则二分图必有完美匹配，取出一个完美匹配当作一种颜色、从图中删去，剩下的是 $(\Delta-1)$-正则图，递归 Δ 次即得恰 Δ 种颜色的合法边染色。实现里每轮用 Hopcroft-Karp 式的分层增广求一个覆盖所有点的完美匹配，真实边记颜色、虚拟边（`id == -1`）丢弃。

```
struct EdgeColor
{
    // 颜色数。
    int k;
    // c[i] 是第 i 条原边的颜色。
    vector<int> c;
};

struct BipEdgeColor
{
    struct E
    {
        // 左端点、右端点、原边编号；虚拟边编号为 -1。
        int u, v, id;
    };

    // 左、右部点数。
    int n, m;
    // 原图边集。
    vector<E> e;

    BipEdgeColor(int n, int m) : n(n), m(m)
    {
        // n、m 是左右部点数；构造空二分图。
    }

    void add(int u, int v)
    {
```

```cpp
        // u、v 是左右端点；按加入顺序给边编号。
        assert(0 <= u && u < n && 0 <= v && v < m); // 调试检查，可删。
        e.push_back({u, v, (int)e.size()});
    }

    EdgeColor work() const
    {
        // 返回最少颜色数及每条原边的颜色；先合并低度点，避免孤立点使补图膨胀。
        vector<int> dl(n), dr(m);
        int k = 0;
        for (auto x : e)
        {
            k = max(k, ++dl[x.u]);
            k = max(k, ++dr[x.v]);
        }
        if (!k) return {0, {}};
        auto compress = [&](const vector<int> &deg) -> pair<vector<int>, int>
        {
            // deg 是一侧原点度数；返回原点到合并点的映射及合并后点数。
            int sz = deg.size();
            vector<int> fa(sz, -1);
            auto find = [&](auto &&self, int u) -> int
            {
                // u 是并查集节点；返回所在集合代表。
                return fa[u] < 0 ? u : fa[u] = self(self, fa[u]);
            };
            priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> q;
            for (int i = 0; i < sz; i++)
            {
                q.push({deg[i], i});
            }
            while (q.size() > 1)
            {
                auto [x, u] = q.top(); q.pop();
                auto [y, v] = q.top(); q.pop();
                if (x + y > k)
                {
                    q.push({x, u});
                    q.push({y, v});
                    break;
                }
                if (fa[u] > fa[v]) swap(u, v);
                fa[u] += fa[v];
                fa[v] = u;
                q.push({x + y, u});
            }
            vector<int> id(sz), pos(sz, -1);
            int cnt = 0;
            for (int i = 0; i < sz; i++)
            {
                int u = find(find, i);
                if (pos[u] == -1) pos[u] = cnt++;
```

```
                id[i] = pos[u];
            }
            return {id, cnt};
        };
        auto [pl, nl] = compress(dl);
        auto [pr, nr] = compress(dr);
        int z = max(nl, nr);
        vector<E> es;
        es.reserve(2 * e.size() + 2 * z);
        vector<int> cl(z), cr(z);
        for (auto x : e)
        {
            x.u = pl[x.u];
            x.v = pr[x.v];
            es.push_back(x);
            cl[x.u]++;
            cr[x.v]++;
        }
        int u = 0, v = 0;
        while (u < z && v < z)
        {
            while (u < z && cl[u] == k)
            {
                u++;
            }
            while (v < z && cr[v] == k)
            {
                v++;
            }
            if (u == z || v == z) break;
            int cnt = min(k - cl[u], k - cr[v]);
            for (int i = 0; i < cnt; i++)
            {
                es.push_back({u, v, -1});
            }
            cl[u] += cnt;
            cr[v] += cnt;
        }
        vector<int> ans(e.size());
        auto perfect = [&](const vector<int> &ids) -> vector<int>
        {
            // ids 是正则二分图边编号；返回一组覆盖全部合并点的完美匹配边。
            vector<vector<int>> g(z);
            for (int id : ids)
            {
                g[es[id].u].push_back(id);
            }
            vector<int> ml(z, -1), mr(z, -1), dep(z), it(z);
            auto bfs = [&]() -> bool
            {
                // 无参数；给最短增广路分层，存在时返回 true。
                queue<int> q;
```

```
                fill(dep.begin(), dep.end(), -1);
                int lim = -1;
                for (int x = 0; x < z; x++)
                {
                    if (ml[x] == -1) dep[x] = 0, q.push(x);
                }
                while (!q.empty())
                {
                    int x = q.front(); q.pop();
                    if (lim != -1 && dep[x] >= lim) continue;
                    for (int id : g[x])
                    {
                        int y = mr[es[id].v];
                        if (y == -1) lim = dep[x] + 1;
                        else if (dep[y] == -1 && (lim == -1 || dep[x] + 1 < lim)) dep[y] =
                        ↪  dep[x] + 1, q.push(y);
                    }
                }
                return lim != -1;
            };
            auto dfs = [&](auto &&self, int x) -> bool
            {
                // x 是当前左点；沿分层交替路增广，成功时返回 true。
                for (int &i = it[x]; i < (int)g[x].size(); i++)
                {
                    int id = g[x][i], y = mr[es[id].v];
                    if (y == -1 || (dep[y] == dep[x] + 1 && self(self, y)))
                    {
                        ml[x] = id;
                        mr[es[id].v] = x;
                        return true;
                    }
                }
                dep[x] = -1;
                return false;
            };
            int got = 0;
            while (bfs())
            {
                fill(it.begin(), it.end(), 0);
                for (int x = 0; x < z; x++)
                {
                    if (ml[x] == -1 && dfs(dfs, x)) got++;
                }
            }
            assert(got == z); // 调试检查，可删：正则二分图总有完美匹配。
            return ml;
        };
        auto color = [&](auto &&self, vector<int> ids, int d, int first) -> void
        {
            // ids 是 d 正则图边集，d 是度数，first 是该集合第一种可用颜色。
            if (!d) return;
```

```
            if (d == 1)
            {
                for (int id : ids)
                {
                    if (es[id].id != -1) ans[es[id].id] = first;
                }
                return;
            }
            if (d & 1)
            {
                auto take = perfect(ids);
                vector<char> used(es.size());
                for (int id : take)
                {
                    used[id] = true;
                    if (es[id].id != -1) ans[es[id].id] = first;
                }
                vector<int> rest;
                rest.reserve(ids.size() - take.size());
                for (int id : ids)
                {
                    if (!used[id]) rest.push_back(id);
                }
                self(self, move(rest), d - 1, first + 1);
                return;
            }
            vector<vector<pair<int, int>>> g(2 * z);
            for (int id : ids)
            {
                int a = es[id].u, b = z + es[id].v;
                g[a].push_back({b, id});
                g[b].push_back({a, id});
            }
            vector<char> used(es.size());
            vector<int> ptr(2 * z), a, b;
            for (int s = 0; s < 2 * z; s++)
            {
                while (ptr[s] < (int)g[s].size() && used[g[s][ptr[s]].second])
                {
                    ptr[s]++;
                }
                if (ptr[s] == (int)g[s].size()) continue;
                vector<pair<int, int>> stk{{s, -1}}, tour;
                while (!stk.empty())
                {
                    int x = stk.back().first;
                    while (ptr[x] < (int)g[x].size() && used[g[x][ptr[x]].second])
                    {
                        ptr[x]++;
                    }
                    if (ptr[x] == (int)g[x].size())
                    {
```

```
                        int id = stk.back().second;
                        stk.pop_back();
                        if (id != -1) tour.push_back({0, id});
                    }
                    else
                    {
                        auto [y, id] = g[x][ptr[x]++];
                        if (!used[id]) used[id] = true, stk.push_back({y, id});
                    }
                }
                for (int i = 0; i < (int)tour.size(); i++)
                {
                    (i & 1 ? b : a).push_back(tour[i].second);
                }
            }
            self(self, move(a), d / 2, first);
            self(self, move(b), d / 2, first + d / 2);
        };
        vector<int> ids(es.size());
        iota(ids.begin(), ids.end(), 0);
        color(color, move(ids), k, 0);
        return {k, ans};
    }
};
```

构造 `BipEdgeColor g(n, m)`（左、右部点数），`add(u, v)` 加边（按加入顺序隐式编号），`work()` 返回 `EdgeColor`：`k` 是颜色数（即 Δ），`c[i]` 是第 i 条原边的颜色（0 到 $k-1$）。先把同侧度数和不超过 Δ 的点合并，故参与补齐的合并点和边数都是 $O(E)$；再把图补成 Δ 正则图，奇数度时取一组完美匹配、偶数度时沿欧拉回路二分。设 E 为原边数，Hopcroft-Karp 版本的完美匹配分解总时间为 $O(E\sqrt{E}\log\Delta)$、额外空间为 $O(E)$，不会因大量孤立点膨胀。它反复调用二分图完美匹配，底层增广与 5.6.003 Hopcroft-Karp 同构；一般（非二分）图的边染色可能需要 $\Delta+1$ 色（Vizing 定理），不能套用本条的 Δ 色结论。

5.6.007 最小点覆盖、最小边覆盖

König 定理：二分图最小点覆盖大小等于最大匹配数。要还原一组具体的最小点覆盖，先求一份最大匹配，再从所有未匹配左点出发沿「未匹配边向右、匹配边向左」交替搜索，记可达左点集 Z_L、可达右点集 Z_R；则最小点覆盖恰为 $(L\setminus Z_L)\cup Z_R$。最小边覆盖则由 Gallai 定理给出：无孤立点时其大小为 $|V|-$ 最大匹配，构造上先取匹配边、再给每个未被覆盖的点补一条任意关联边。任一侧存在孤立点时最小边覆盖不存在。

```
pair<vector<int>, vector<int>>
minVtxCover(const vector<vector<int>> &g, const vector<int> &ml, const vector<int> &mr)
{
    // g 是左部邻接表，ml、mr 是一份最大匹配；返回最小点覆盖中的左点表与右点表。
    vector<bool> vl(g.size()), vr(mr.size());
    queue<int> q;
    for (int u = 0; u < (int)g.size(); u++)
    {
        if (ml[u] == -1)
```

```
        {
            vl[u] = true;
            q.push(u);
        }
    }
    while (!q.empty())
    {
        int u = q.front();
        q.pop();
        for (auto v : g[u])
        {
            if (ml[u] == v || vr[v])
            {
                continue;
            }
            vr[v] = true;
            if (mr[v] != -1 && !vl[mr[v]])
            {
                vl[mr[v]] = true;
                q.push(mr[v]);
            }
        }
    }
    vector<int> l, r;
    for (int u = 0; u < (int)g.size(); u++)
    {
        if (!vl[u])
        {
            l.push_back(u);
        }
    }
    for (int v = 0; v < (int)mr.size(); v++)
    {
        if (vr[v])
        {
            r.push_back(v);
        }
    }
    return {l, r};
}

optional<vector<array<int, 2>>> minEdgeCover(const vector<vector<int>> &g, const vector<int>
↪ &ml, const vector<int> &mr)
{
    // g 是左部邻接表，ml、mr 是最大匹配；返回最小边覆盖，存在孤立点时返回空。
    vector<array<int, 2>> ans;
    vector<bool> usedR(mr.size());
    for (int u = 0; u < (int)g.size(); u++)
    {
        if (g[u].empty())
        {
            return nullopt;
```

```
        }
        if (ml[u] != -1)
        {
            ans.push_back({u, ml[u]});
            usedR[ml[u]] = true;
        }
    }
    for (int u = 0; u < (int)g.size(); u++)
    {
        if (ml[u] == -1)
        {
            ans.push_back({u, g[u][0]});
            usedR[g[u][0]] = true;
        }
    }
    vector<int> pre(mr.size(), -1);
    for (int u = 0; u < (int)g.size(); u++)
    {
        for (auto v : g[u])
        {
            pre[v] = u;
        }
    }
    for (int v = 0; v < (int)mr.size(); v++)
    {
        if (pre[v] == -1)
        {
            return nullopt;
        }
        if (!usedR[v])
        {
            ans.push_back({pre[v], v});
        }
    }
    return ans;
}
```

把原左部邻接表 `g` 和一份最大匹配 `ml` / `mr`（由 5.6.003 Hopcroft-Karp 或 5.6.001 匈牙利算法 求得）传入：`minVtxCover` 返回最小点覆盖的左点表与右点表；`minEdgeCover` 返回最小边覆盖的边列表（`array<int,2>` 为左右端点），任一侧有孤立点时返回空。两函数都只做一遍交替搜索与扫描，$O(V+E)$ 时间与输出空间，前置匹配的开销另计。最小点覆盖的补集就是最大独立集，构造见 5.6.008 最大独立集；这些都是二分图专属对偶，一般图的最小点覆盖是 NP 难、不能套用。

5.6.008 最大独立集

独立集是两两不相邻的点集。二分图里它有简洁对偶：最大独立集 = 全部点去掉一个最小点覆盖，大小 $|V|-$ 最大匹配，直接用 5.6.007 最小点覆盖、最小边覆盖 的点覆盖取补集即可。一般图没有这种多项式公式（最大独立集与最大团互补，都是 NP 难），只能搜索。这里在补图上求最大团：贪心着色把候选点分成若干独立色类，色数就是当前团还能扩大的上界；按色数倒序分支，界不可能超过已知最优时立即剪枝。

这比逐点选或不选的朴素分支适合 $n \leq 40$ 的在线约束。

```cpp
// 这是补图最大团的精确分支限界，只适合 n<=63 的一般图。
inline vector<int> maxIS(int n, const vector<pair<int, int>> &edges)
{
    // n 是点数，edges 是无向边；返回一组最大独立集顶点，允许重边和自环。
    assert(0 <= n && n <= 63); // 调试检查，可删：位集使用 unsigned long long。
    vector<unsigned long long> adj(n);
    vector<bool> loop(n);
    unsigned long long all = n ? (1ULL << n) - 1 : 0;
    for (int u = 0; u < n; u++)
    {
        adj[u] = all ^ (1ULL << u);
    }
    for (auto [u, v] : edges)
    {
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        if (u == v)
        {
            loop[u] = true;
        }
        else
        {
            adj[u] &= ~(1ULL << v);
            adj[v] &= ~(1ULL << u);
        }
    }
    vector<int> best, cur;
    auto dfs = [&](auto &&self, unsigned long long can) -> void
    {
        // can 是仍与当前选点全相邻的候选点集合；更新 best，无返回值。
        vector<int> ord, col;
        unsigned long long rem = can;
        int cnt = 0;
        while (rem)
        {
            cnt++;
            unsigned long long able = rem;
            while (able)
            {
                int u = countr_zero(able);
                ord.push_back(u);
                col.push_back(cnt);
                rem ^= 1ULL << u;
                able &= ~(1ULL << u);
                able &= ~adj[u];
                able &= rem;
            }
        }
        for (int i = (int)ord.size() - 1; i >= 0; i--)
        {
            if ((int)cur.size() + col[i] <= (int)best.size()) return;
```

```
            int u = ord[i];
            cur.push_back(u);
            unsigned long long nxt = 0;
            for (int j = 0; j < i; j++)
            {
                if (adj[u] >> ord[j] & 1ULL) nxt |= 1ULL << ord[j];
            }
            if (!nxt)
            {
                if (cur.size() > best.size()) best = cur;
            }
            else self(self, nxt);
            cur.pop_back();
        }
    };
    unsigned long long can = all;
    for (int u = 0; u < n; u++)
    {
        if (loop[u]) can &= ~(1ULL << u);
    }
    dfs(dfs, can);
    return best;
}
```

`maxIS(n, edges)` 返回一组最大独立集；平行边不影响结果，自环点会被排除，且要求 $n \le 63$。补图着色上界分支的最坏时间仍为指数级、额外空间 $O(n^2)$（递归候选序列），但在 $n \le 40$ 时比朴素逐点分支稳定得多。二分图应直接对 5.6.007 最小点覆盖、最小边覆盖 取补；更大的一般图要按稀疏性和约束另选专用算法。

5.6.009 DAG 最小不相交链覆盖

要用最少条点不相交有向路径覆盖 DAG 的全部点。经典转化：把每个点 x 拆成左副本 x_L 与右副本 x_R，原有向边 $x \to y$ 变成二分图边 x_L-y_R。二分图里的一条匹配边 x_L-y_R 表示「在覆盖中让 x 的后继紧接 y」，即把两个点串进同一条路径；于是每多一条匹配边就少一条路径，最小路径数 $= V -$ 最大匹配数。求出匹配后，从每个「右副本未被匹配」的点（即某条路径的起点）出发沿 `ml` 链一路串下去即可恢复每条路径。

```
// 依赖 5.6.003 Hopcroft-Karp
// DAG 的最小点不相交路径覆盖，返回路径中的原图顶点。
vector<vector<int>> dagPathCover(int n, const vector<pair<int, int>> &edges)
{
    // n 是点数，edges 是 DAG 边；返回一组最小点不相交路径覆盖。
    HopcroftKarp hk(n, n);
    for (auto [x, y] : edges)
    {
        hk.addEdge(x, y);
    }
    hk.matching();
    const auto &ml = hk.leftMatch();
```

```
    const auto &mr = hk.matchR();
    vector<vector<int>> ans;
    for (int s = 0; s < n; s++)
    {
        if (mr[s] != -1)
        {
            continue;
        }
        vector<int> a;
        for (int x = s; x != -1; x = ml[x])
        {
            a.push_back(x);
        }
        ans.push_back(move(a));
    }
    return ans;
}
```

`dagPathCover(n, edges)` 传入 DAG 的点数与有向边集，返回一组点不相交路径，每条路径按顶点顺序给出；路径条数即为最小值。拆点后二分图两侧各 n 个点，x、y 都沿用原图点号；本条直接复用 5.6.003 Hopcroft-Karp，不再内嵌第二份匹配器。输入必须是 DAG；若只给一般偏序的覆盖关系而边集未传递闭包，这里求的是原图路径覆盖，不是偏序链覆盖。二分图增广最坏 $O(m\sqrt{n})$，重建所有路径 $O(n)$，空间 $O(n+m)$。最大反链及传递闭包的区别见 5.6.010 DAG 最大互不可达集；带权版本或最小路径可重叠覆盖则要改用最小流建模，见 5.8.001 Dinic 最大流。

5.6.010 DAG 最大互不可达集

求 DAG 中两两不存在可达关系的最大点集，等价于其可达偏序上的最大反链。Dilworth 定理给出关键等式：最大反链大小等于最小链覆盖数。必须先求传递闭包，再把每个可达关系 $u \rightsquigarrow v$ 变成二分图边 $u_L - v_R$；只使用原图边求到的是最小路径覆盖，答案可能不同。

最大反链不能把二分图最小点覆盖的两个点集简单合并后取补，因为同一个原图点在左右部各有一个副本。设最小点覆盖为 $C_L \cup C_R$，正确恢复式是

$$A = \{x \mid x_L \notin C_L \text{ 且 } x_R \notin C_R\}.$$

任意可比的 $x < y$ 对应边 $x_L - y_R$，其至少一个端点在点覆盖中，所以 A 内两点必不可比；由 König 与 Dilworth 定理可知其大小恰为 $n - |M|$。

```
// 依赖 5.6.003 Hopcroft-Karp
// 依赖 5.6.007 最小点覆盖、最小边覆盖
vector<int> dagAntichain(int n, const vector<pair<int, int>> &edges)
{
    // n 是点数，edges 是 DAG 边；返回一组最大互不可达点集。
    vector reach(n, vector<bool>(n));
    for (auto [u, v] : edges)
    {
        reach[u][v] = true;
    }
    for (int k = 0; k < n; k++)
```

```
    {
        for (int i = 0; i < n; i++)
        {
            if (!reach[i][k])
            {
                continue;
            }
            for (int j = 0; j < n; j++)
            {
                reach[i][j] = reach[i][j] || reach[k][j];
            }
        }
    }

    vector<vector<int>> g(n);
    HopcroftKarp hk(n, n);
    for (int u = 0; u < n; u++)
    {
        for (int v = 0; v < n; v++)
        {
            if (reach[u][v])
            {
                g[u].push_back(v);
                hk.addEdge(u, v);
            }
        }
    }
    hk.matching();
    auto [cl, cr] = minVtxCover(g, hk.leftMatch(), hk.matchR());
    vector<bool> inL(n), inR(n);
    for (int u : cl)
    {
        inL[u] = true;
    }
    for (int v : cr)
    {
        inR[v] = true;
    }
    vector<int> ans;
    for (int u = 0; u < n; u++)
    {
        if (!inL[u] && !inR[u])
        {
            ans.push_back(u);
        }
    }
    return ans;
}
```

`dagAntichain(n, edges)` 返回一组最大反链。代码用 Floyd 求传递闭包，时间 $O(n^3 + n^2\sqrt{n})$、空间 $O(n^2)$；稀疏大图可把闭包部分换成从每点出发的 BFS，匹配与恢复不变。只需最小链覆盖方案时，把闭包边交给 5.6.009 DAG 最小不相交链覆盖；`dagPathCover` 返回的链数也等于本函数答案大小。二分图最

小点覆盖的构造见 5.6.007 最小点覆盖、最小边覆盖。带权反链需另建最小割模型，不能直接给这里的基数匹配加权。

5.6.011 最大权闭合子图

在带点权的有向图里选一个点集，要求「选了点 u 就必须选它的所有后继依赖点」，并最大化所选点权和——这是闭合性约束（选一个就得连带选下去），不是匹配。经典的最小割建模：新建源 s、汇 t，正权点 u 连 $s \to u$ 容量 w_u，负权点 u 连 $u \to t$ 容量 $-w_u$，每条依赖 $u \Rightarrow v$ 连 $u \to v$ 容量取一个大于全部正权之和的「无穷」值（代码里取 $1 + \sum_{w_u>0} w_u$）。因为依赖边容量足够大，任何有限割都不会切它，于是割对应的源侧点集一定是闭合的；最小割 $=$ 未选正权点的损失 $+$ 已选负权点的代价，答案就是正权总和减最小割。求出最小割后，残量网络里从 s 出发 BFS 可达的点即被选点集。

```cpp
template <class Cap = long long>
struct ClosureRes
{
    Cap val;          // 最大闭合子图点权和。
    vector<int> pick; // 一组被选择的原图点编号。
};

template <class Cap = long long>
ClosureRes<Cap> maxClosure(
    const vector<long long> &w, const vector<array<int, 2>> &need)
{
    // w[u] 是点权，need 中的 {u, v} 表示选 u 必须选 v；返回最优闭集。
    int n = w.size(), s = n, t = n + 1;
    Cap sum = 0;
    for (long long x : w)
    {
        if (x > 0)
        {
            sum += (Cap)x;
        }
    }

    Cap inf = sum + 1;
    Flow<Cap> fl(n + 2);
    for (int u = 0; u < n; u++)
    {
        if (w[u] > 0)
        {
            fl.add(s, u, (Cap)w[u]);
        }
        else if (w[u] < 0)
        {
            fl.add(u, t, -(Cap)w[u]);
        }
    }
    for (auto [u, v] : need)
    {
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
```

```
        fl.add(u, v, inf);
    }

    Cap flow = fl.flow(s, t);
    vector<bool> vis = fl.cut(s);
    vector<int> pick;
    for (int u = 0; u < n; u++)
    {
        if (vis[u])
        {
            pick.push_back(u);
        }
    }
    return ClosureRes<Cap>{sum - flow, pick};
}
```

`maxClosure<Cap>(w,need)` 中 `Cap` 默认是 `long long`，`val` 是最大权和，`pick` 是一组最优点。调用者必须保证正权总和、负权取反和人工边容量可由 `Cap` 表示；确实需要时可显式使用 `maxClosure<__int128>`。本条依赖 5.8.001 Dinic 最大流，建图 $O(V+E)$，Dinic 一般网络最坏 $O(V^2E)$，空间 $O(V+E)$。

5.7 一般图匹配

5.7.001 一般图最大匹配（Blossom）

在允许奇环的一般无向图里求最大基数匹配。5.6.001 匈牙利算法 与 5.6.003 Hopcroft-Karp 的二分图增广之所以失效，是因为奇环会让 BFS 交替森林里出现「同层两点相连」的边，此时沿两条路径回溯到最近公共祖先 b 之间的那个奇环就是一朵「花」（blossom）。Edmonds 的关键洞见：把整朵花缩成一个点后再找增广路，缩点不改变是否存在增广路——花里任意点都能既作偶点又作奇点参与增广。实现上不真的缩点，而是用 `base[]` 记录每个点当前所属花的基点，`lca` 求两条交替路径的汇合点，`mark` 沿环把途经点的 `base` 统一指向 b 并把奇点也拉进队列继续 BFS。找到未匹配端点后，沿 `p[]` 父链回溯并翻转匹配边即完成一次增广。逐点尝试增广，匹配数每次至多加一。

```
struct Blossom
{
    // 点数与无向邻接表。
    int n;
    vector<vector<int>> g;
    // 当前配偶、交替森林父点、花基点和 BFS 队列。
    vector<int> match, p, base, q;
    // used 标记偶层点是否入队，flower 标记本轮被收缩的基点。
    vector<bool> used, flower;

    Blossom(int n = 0) : n(n), g(n), match(n, -1), p(n), base(n), q(n), used(n), flower(n)
    {
        // n 是点数；构造空图。
    }

    void add(int u, int v)
```

```
    {
        // u、v 是端点；自环不参与匹配，直接忽略。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        if (u == v)
        {
            return;
        }
        g[u].push_back(v);
        g[v].push_back(u);
    }

    int lca(int a, int b)
    {
        // a、b 是同棵交替树中的偶层点；返回新花的基点。
        vector<bool> seen(n);
        while (true)
        {
            a = base[a];
            seen[a] = true;
            if (match[a] < 0)
            {
                break;
            }
            a = p[match[a]];
        }
        while (true)
        {
            b = base[b];
            if (seen[b])
            {
                return b;
            }
            b = p[match[b]];
        }
    }

    void mark(int v, int b, int x)
    {
        // 从 v 沿交替树走到基点 b，并令第一步父点为 x。
        while (base[v] != b)
        {
            flower[base[v]] = true;
            flower[base[match[v]]] = true;
            p[v] = x;
            x = match[v];
            v = p[match[v]];
        }
    }

    int path(int s)
    {
        // 从未匹配点 s 建交替树；返回另一个未匹配端点，不存在则返回 -1。
```

```
        fill(used.begin(), used.end(), false);
        fill(p.begin(), p.end(), -1);
        iota(base.begin(), base.end(), 0);
        int ql = 0, qr = 0;
        q[qr++] = s;
        used[s] = true;
        while (ql < qr)
        {
            int v = q[ql++];
            for (int u : g[v])
            {
                if (base[v] == base[u] || match[v] == u)
                {
                    continue;
                }
                if (u == s || (match[u] >= 0 && p[match[u]] >= 0))
                {
                    int b = lca(v, u);
                    fill(flower.begin(), flower.end(), false);
                    mark(v, b, u);
                    mark(u, b, v);
                    for (int i = 0; i < n; i++)
                    {
                        if (flower[base[i]])
                        {
                            base[i] = b;
                            if (!used[i])
                            {
                                used[i] = true;
                                q[qr++] = i;
                            }
                        }
                    }
                }
                else if (p[u] < 0)
                {
                    p[u] = v;
                    if (match[u] < 0)
                    {
                        return u;
                    }
                    u = match[u];
                    used[u] = true;
                    q[qr++] = u;
                }
            }
        }
        return -1;
    }

    int work()
    {
```

```
        // 从空匹配重算，返回最大匹配边数。
        fill(match.begin(), match.end(), -1);
        int ans = 0;
        for (int s = 0; s < n; s++)
        {
            if (match[s] >= 0)
            {
                continue;
            }
            int v = path(s);
            if (v < 0)
            {
                continue;
            }
            ans++;
            while (v >= 0)
            {
                int u = p[v];
                int w = u < 0 ? -1 : match[u];
                if (u >= 0)
                {
                    match[v] = u;
                    match[u] = v;
                }
                v = w;
            }
        }
        return ans;
    }
};
```

构造 `Blossom bl(n)`，`add(u, v)` 加无向边（点 0 下标），`work()` 返回最大匹配数并把方案写进 `match[]`（`match[u]` 为 u 的配偶，未匹配为 -1）。图可以不连通，也可以含平行边或自环；自环不可能进入普通图匹配，`add` 会直接忽略。时间 $O(n^3)$、空间 $O(n+m)$。图是二分图时这份代码仍正确，但用 5.6.003 Hopcroft-Karp 的 $O(m\sqrt{n})$ 更快；需要带权最大匹配见 5.7.002 一般图最大权匹配（本模板仅给 $n \le 24$ 的指数版），只求匹配大小可用 5.7.003 Tutte Matrix 的随机代数法。

下面用五环加一片叶子检查奇环收缩；最优方案可取 $(0,5),(1,2),(3,4)$，匹配数为 3。

```
signed main()
{
    Blossom b(6);
    for (int i = 0; i < 5; i++)
    {
        b.add(i, (i + 1) % 5);
    }
    b.add(0, 5);
    assert(b.work() == 3);
}
```

5.7.002 一般图最大权匹配

在一般无向图中求总权最大的匹配（不要求匹配数最大，允许空匹配）。理论上正解是带权 Blossom + 原始对偶，$O(n^3)$；但那份代码极长且容易写错，竞赛里点数往往很小，所以这里给一个更短、更可靠的状压 DP：以 $dp[S]$ 表示只用点集 S 时的最大权匹配。取 S 中最低位点 u，要么 u 不匹配（继承 $dp[S \setminus \{u\}]$），要么 u 与 S 中某个更高位点 v 匹配（$dp[S \setminus \{u,v\}] + w_{uv}$），两者取大。`take[S]` 记录 u 在最优解里配给谁，便于回溯匹配边。因为每步固定处理最低位点，转移无后效，$O(n2^n)$，`assert(n <= 24)` 卡住规模上限。

```cpp
pair<int, vector<pair<int, int>>>
weightMatch(const vector<vector<optional<long long>>> &w)
{
    // w[u][v] 是边权，空值表示无边；返回最大总权和一组匹配边。
    int n = w.size();
    assert(n <= 24); // 调试检查，可删：状态数必须能承受。
    for (const auto &row : w)
    {
        assert((int)row.size() == n); // 调试检查，可删。
    }
    size_t tot = 1ULL << n;
    vector<int> dp(tot);
    vector<int> take(tot, -1);
    dp[0] = 0;
    for (size_t s = 1; s < tot; s++)
    {
        int u = countr_zero((unsigned long long)s);
        dp[s] = dp[s ^ (1ULL << u)];
        for (int v = u + 1; v < n; v++)
        {
            if ((s >> v & 1) && w[u][v].has_value())
            {
                int z = dp[s ^ (1ULL << u) ^ (1ULL << v)] + *w[u][v];
                if (z > dp[s])
                {
                    dp[s] = z;
                    take[s] = v;
                }
            }
        }
    }
    vector<pair<int, int>> e;
    for (size_t s = tot - 1; s;)
    {
        int u = countr_zero((unsigned long long)s), v = take[s];
        s ^= 1ULL << u;
        if (v >= 0)
        {
            s ^= 1ULL << v;
            e.push_back({u, v});
        }
    }
```

```
    return pair{dp.back(), e};
}
```

`weightMatch(w)` 接受 $n \times n$ 对称矩阵，空值表示无边；返回最大总权和一组匹配边。算法允许空匹配，故全负权答案为 0；调用者必须保证匹配总权可由 64 位有符号整数表示。要求 $n \le 24$，但 $n = 24$ 时内存约 $272\,\mathrm{MiB}$，常见限制下宜取 $n \le 22$；时间 $O(n2^n)$、空间 $O(2^n)$。

5.7.003 Tutte Matrix

图 G 的 Tutte 矩阵是一个反对称矩阵：对每条边 (u, v) 引入一个不定元 x_{uv}，令 $A_{uv} = x_{uv}$、$A_{vu} = -x_{uv}$，非边处为 0。Tutte 定理说 $\det A$（作为多项式）恒为零当且仅当 G 没有完美匹配；Lovász 进一步证明 $\operatorname{rank} A = 2\nu(G)$，$\nu$ 是最大匹配数。多项式行列式无法直接算，但可用 Schwartz–Zippel 引理：把每个 x_{uv} 代入大素数域上的随机值，则以极高概率 rank 不掉——单次误判概率约 $\frac{n}{P}$。于是把随机化后的数值矩阵做一次高斯消元求秩，除以 2 就得最大匹配大小。代码里 `rankField` 是有限域高斯消元求秩，`tutteMatch` 填反对称随机矩阵后取 rank $/2$；固定 `seed` 保证结果可复现。

```
int rankField(vector<vector<Z>> a)
{
    // a 是有限域上的矩阵；返回矩阵秩。
    int n = a.size(), m = n ? a[0].size() : 0, r = 0;
    for (int c = 0; c < m && r < n; c++)
    {
        int p = r;
        while (p < n && a[p][c] == Z(0))
        {
            p++;
        }
        if (p == n)
        {
            continue;
        }
        swap(a[p], a[r]);
        Z iv = a[r][c].inv();
        for (int i = r + 1; i < n; i++)
        {
            Z q = a[i][c] * iv;
            for (int j = c; j < m; j++)
            {
                a[i][j] -= q * a[r][j];
            }
        }
        r++;
    }
    return r;
}

int tutteMatch(int n, const vector<pair<int, int>> &e, unsigned seed = 712367)
{
    // n 是点数，e 是无向边端点列表，seed 是固定随机种子；返回最大匹配大小，存在有限域随机误判概率。
```

```
    mt19937 rng(seed);
    vector a(n, vector<Z>(n));
    for (auto [u, v] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        if (u == v)
        {
            continue;
        }
        Z x = rng() % (Z::mod() - 1) + 1;
        a[u][v] += x;
        a[v][u] -= x;
    }
    return rankField(move(a)) / 2;
}
```

依赖 1.1.008 静态取模整数类的大素数域类型 `Z`。调用 `tutteMatch(n, e, seed)`，`e` 是无向边端点列表（点 0 下标）；平行边可重复出现，自环与普通图匹配无关，代码会忽略。返回最大匹配大小，不恢复边集；例如四点路径返回 2，三点路径返回 1。时间 $O(n^3+m)$、空间 $O(n^2)$，单次误判概率 $O(\frac{n}{P})$，P 是模数；可换几个 `seed` 取最大值进一步降低误判概率。需要确定性结果或恢复方案时改用 5.7.001 一般图最大匹配（Blossom）。

5.7.004 Edmonds Matrix

竞赛资料里「Edmonds Matrix」常指一般图匹配所用的 Tutte–Edmonds 反对称矩阵，与 5.7.003 Tutte Matrix 是同一个代数对象、同一份代码，这里不再重复打印，直接沿用那里的 `tutteMatch(n, e, seed)`：随机代入后求秩除以二即最大匹配大小，$O(n^3+m)$ 时间、$O(n^2)$ 空间，单次误判概率 $O(\frac{n}{P})$。需要注意的是「Edmonds 矩阵」在另一些资料里特指二分图的那个 x_{ij} 矩阵（左行右列，非反对称），其行列式非零当且仅当二分图有完美匹配——那是不同的对象，若题目按二分图定义使用，应改在二分图上建矩阵，而不能套用本条的一般图反对称版本。要确定性求解或恢复匹配边，用 5.7.001 一般图最大匹配（Blossom）。

5.7.005 稳定婚姻

给左右两侧各 n 人以及每人对另一侧从优到劣的完整偏好序列，求一个完美匹配，使不存在「阻塞对」——即不存在一对未配的 (x,y) 彼此都比各自现配偶更中意。稳定匹配一定存在，Gale–Shapley 算法给出其一：让尚未匹配的求婚方 x 按自己偏好从高到低依次向 y 求婚，y 若空闲则暂定，若已有对象则留下自己更偏好的那个、把另一个踢回求婚队列。每个 x 对每个 y 至多求婚一次，故必然终止；结论对求婚方最优、对被求婚方最劣，但都稳定。代码用 `rk[y][x]` 预存 y 眼中 x 的排名（越小越偏好）以 $O(1)$ 比较，`at[x]` 记 x 下一个要求婚的对象下标，队列 `q` 存当前空闲的求婚方。

```
struct Marriage
{
    // a[x]、b[y] 分别是左侧 x 与右侧 y 的配偶编号。
    vector<int> a, b;
};
```

```cpp

// pa[x] 是左侧 x 对右侧的偏好；pb[y] 是右侧 y 对左侧的偏好，均从优到劣。
Marriage stableMatch(const vector<vector<int>> &pa, const vector<vector<int>> &pb)
{
    // pa、pb 是左右两侧从优到劣的完整偏好表；返回两侧互逆的稳定配偶数组。
    int n = pa.size();
    assert((int)pb.size() == n); // 调试检查，可删。
    vector<vector<int>> rk(n, vector<int>(n));
    for (int y = 0; y < n; y++)
    {
        for (int i = 0; i < n; i++)
        {
            rk[y][pb[y][i]] = i;
        }
    }
    vector<int> a(n, -1), b(n, -1), at(n);
    queue<int> q;
    for (int x = 0; x < n; x++)
    {
        q.push(x);
    }
    while (!q.empty())
    {
        int x = q.front();
        q.pop();
        int y = pa[x][at[x]++];
        if (b[y] < 0)
        {
            a[x] = y;
            b[y] = x;
        }
        else if (rk[y][x] < rk[y][b[y]])
        {
            q.push(b[y]);
            a[b[y]] = -1;
            a[x] = y;
            b[y] = x;
        }
        else
        {
            q.push(x);
        }
    }
    return {a, b};
}
```

`stableMatch(pa, pb)` 中 `pa[x]`、`pb[y]` 都是长度 n 的排列，分别是左侧 x、右侧 y 从优到劣的对方编号序列；返回的 `Marriage` 里 `a[x]` 是左侧 x 的配偶、`b[y]` 是右侧 y 的配偶。要求两侧人数相等且偏好表是完整排列（每人都能接受所有对象），得到的是求婚方（左侧）最优稳定匹配；把 `pa`、`pb` 互换即得另一侧最优解。时间与空间 $O(n^2)$，瓶颈是 `rk` 表与偏好表本身。这属于稳定匹配问题，与追求总权最优的 5.6.005 KM 算法 或最大基数的一般图匹配 5.7.001 一般图最大匹配（Blossom） 目标不同，不能相互替代——稳

定解通常不是权最大解。

5.7.006 一般图最大权匹配（带权 Blossom）

上一条状压 DP 只适合 $n \le 24$。需要在一般图上直接处理更大点数时，使用带权 Blossom 的原始对偶算法：顶标 `lab` 保持边的对偶可行性，交替树上的零紧边扩展增广路，遇到奇环就收缩成 Blossom；顶标调整后再展开达到零顶标的花朵。实现按外部竞赛模板改成动态数组，保留矩阵松弛，时间 $O(n^3)$、空间 $O(n^2)$。边权非正时允许空匹配，因此不会为了凑匹配数强行加入负边。

```cpp
template <class W = long long>
struct WBlossom
{
    struct Edge
    {
        int u = 0, v = 0; // u、v 是边在当前收缩图中的两个端点。
        W w = 0;          // w 是边权。
    };

    int n, m, stamp = 0, head = 0, tail = 0; // n 是点数，m 是已用的点与花朵编号上界，stamp 是找 LCA 的时间戳，head、tail 是 BFS 队列首尾。
    W inf = numeric_limits<W>::max() / 4; // inf 是不可达权值。
    vector<vector<Edge>> e;          // e 是原点与花朵之间的最优代表边矩阵。
    vector<W> lab;                   // lab 是原点和花朵的对偶顶标。
    vector<int> link, slack, state;  // link 是匹配点，slack 是最松边，state 是交替树状态。
    vector<int> rep, fa, seen, q;    // rep 是当前花朵代表，fa 是交替树父边端点。
    vector<vector<int>> blossom;     // blossom[x] 按环序保存花朵 x 的子块。
    vector<vector<int>> base;        // base[x][v] 是花朵 x 内含原点 v 的直接子块。

    explicit WBlossom(int n)
        : n(n), m(n), e(2 * n + 5, vector<Edge>(2 * n + 5)),
          lab(2 * n + 5), link(2 * n + 5), slack(2 * n + 5),
          state(2 * n + 5), rep(2 * n + 5), fa(2 * n + 5), seen(2 * n + 5),
          q(2 * n + 5), blossom(2 * n + 5), base(2 * n + 5, vector<int>(2 * n + 5))
    {
        // n 是原图点数；构造最多含 n 个收缩花朵的工作区。
        assert(n >= 0);
        for (int i = 1; i < (int)e.size(); i++)
        {
            for (int j = 1; j < (int)e.size(); j++)
            {
                e[i][j] = {i, j, 0};
            }
        }
    }

    W delta(const Edge &x) const
    {
        // x 是边；返回它相对当前顶标的二倍松弛量。
        return lab[x.u] + lab[x.v] - x.w * 2;
    }
```

```
41
42      void updateSlack(int u, int v)
43      {
44          // u 是外点，v 是待更新块；保留连接 v 的最松边起点。
45          if (!slack[v] || delta(e[u][v]) < delta(e[slack[v]][v]))
46          {
47              slack[v] = u;
48          }
49      }
50
51      void setSlack(int v)
52      {
53          // v 是当前块；重新寻找从外层交替树连接到 v 的最松边。
54          slack[v] = 0;
55          for (int u = 1; u <= n; u++)
56          {
57              if (e[u][v].w > 0 && rep[u] != v && !state[rep[u]])
58              {
59                  updateSlack(u, v);
60              }
61          }
62      }
63
64      void enqueue(int u)
65      {
66          // u 是外层块；把其中全部原点加入交替树扫描队列。
67          if (u <= n)
68          {
69              q[++tail] = u;
70          }
71          else
72          {
73              for (int v : blossom[u])
74              {
75                  enqueue(v);
76              }
77          }
78      }
79
80      void modify(int u, int w)
81      {
82          // u 是块，w 是新代表；递归修改 u 内全部原点的代表。
83          rep[u] = w;
84          if (u > n)
85          {
86              for (int v : blossom[u])
87              {
88                  modify(v, w);
89              }
90          }
91      }
92
```

```
    int position(int u, int v)
    {
        // u 是花朵，v 是其中一个直接子块；调整环向后返回 v 的偶数下标。
        auto it = find(blossom[u].begin(), blossom[u].end(), v);
        int x = it - blossom[u].begin();
        if (x & 1)
        {
            reverse(blossom[u].begin() + 1, blossom[u].end());
            return (int)blossom[u].size() - x;
        }
        return x;
    }

    void setMatch(int u, int v)
    {
        // u、v 是两个待配对块；展开花朵 u 内部并建立对应匹配。
        link[u] = e[u][v].v;
        if (u <= n)
        {
            return;
        }
        Edge x = e[u][v];
        int z = base[u][x.u], k = position(u, z);
        for (int i = 0; i < k; i++)
        {
            setMatch(blossom[u][i], blossom[u][i ^ 1]);
        }
        setMatch(z, v);
        rotate(blossom[u].begin(), blossom[u].begin() + k, blossom[u].end());
    }

    void augment(int u, int v)
    {
        // u、v 是增广路首条边两端；沿交替树翻转匹配直到自由根。
        int x = rep[link[u]];
        setMatch(u, v);
        if (!x)
        {
            return;
        }
        setMatch(x, rep[fa[x]]);
        augment(rep[fa[x]], x);
    }

    int lca(int u, int v)
    {
        // u、v 是两个外层块；返回其交替树最低公共祖先，不相交时返回 0。
        ++stamp;
        while (u || v)
        {
            if (u)
            {
```

```
                if (seen[u] == stamp)
                {
                    return u;
                }
                seen[u] = stamp;
                u = rep[link[u]];
                if (u)
                {
                    u = rep[fa[u]];
                }
            }
            swap(u, v);
        }
        return 0;
    }

    void contract(int u, int a, int v)
    {
        // u、v 是零紧边两端，a 是交替树公共祖先；收缩对应奇环。
        int x = n + 1, i, j;
        while (x <= m && rep[x])
        {
            x++;
        }
        if (x > m)
        {
            m++;
        }
        lab[x] = state[x] = rep[x] = 0;
        link[x] = link[a];
        blossom[x].clear();
        blossom[x].push_back(a);
        for (i = u; i != a; i = rep[fa[j]])
        {
            blossom[x].push_back(i);
            blossom[x].push_back(j = rep[link[i]]);
            enqueue(j);
        }
        reverse(blossom[x].begin() + 1, blossom[x].end());
        for (i = v; i != a; i = rep[fa[j]])
        {
            blossom[x].push_back(i);
            blossom[x].push_back(j = rep[link[i]]);
            enqueue(j);
        }
        modify(x, x);
        for (i = 1; i <= m; i++)
        {
            e[x][i].w = e[i][x].w = 0;
        }
        fill(base[x].begin() + 1, base[x].begin() + n + 1, 0);
        for (int u0 : blossom[x])
```

```
        {
            for (int v0 = 1; v0 <= m; v0++)
            {
                if (!e[x][v0].w || delta(e[u0][v0]) < delta(e[x][v0]))
                {
                    e[x][v0] = e[u0][v0];
                    e[v0][x] = e[v0][u0];
                }
            }
            for (int v0 = 1; v0 <= n; v0++)
            {
                if (base[u0][v0])
                {
                    base[x][v0] = u0;
                }
            }
        }
        setSlack(x);
    }

    void expand(int u)
    {
        // u 是顶标降到零的内层花朵；按交替树方向展开。
        for (int x : blossom[u])
        {
            modify(x, x);
        }
        int a = base[u][e[u][fa[u]].u], r = position(u, a);
        for (int i = 0; i < r; i += 2)
        {
            int x = blossom[u][i], y = blossom[u][i + 1];
            fa[x] = e[y][x].u;
            state[x] = 1;
            state[y] = slack[x] = 0;
            setSlack(y);
            enqueue(y);
        }
        state[a] = 1;
        fa[a] = fa[u];
        for (int i = r + 1; i < (int)blossom[u].size(); i++)
        {
            state[blossom[u][i]] = -1;
            setSlack(blossom[u][i]);
        }
        rep[u] = 0;
    }

    bool onEdge(const Edge &x)
    {
        // x 是新出现的零紧边；扩展交替树、收花或增广，增广成功返回 true。
        int u = rep[x.u], v = rep[x.v];
        if (state[v] == -1)
```

```
        {
            fa[v] = x.u;
            state[v] = 1;
            int a = rep[link[v]];
            slack[v] = slack[a] = state[a] = 0;
            enqueue(a);
        }
        else if (!state[v])
        {
            int a = lca(u, v);
            if (!a)
            {
                augment(u, v);
                augment(v, u);
                return true;
            }
            contract(u, a, v);
        }
        return false;
    }

    bool bfs()
    {
        // 在当前匹配上寻找并执行一条增广路；不存在时返回 false。
        fill(state.begin() + 1, state.begin() + m + 1, -1);
        fill(slack.begin() + 1, slack.begin() + m + 1, 0);
        head = 1;
        tail = 0;
        for (int i = 1; i <= m; i++)
        {
            if (rep[i] == i && !link[i])
            {
                fa[i] = state[i] = 0;
                enqueue(i);
            }
        }
        if (head > tail)
        {
            return false;
        }
        while (true)
        {
            while (head <= tail)
            {
                int u = q[head++];
                if (state[rep[u]] == 1)
                {
                    continue;
                }
                for (int v = 1; v <= n; v++)
                {
                    if (e[u][v].w > 0 && rep[u] != rep[v])
```

```
                    {
                        if (delta(e[u][v]))
                        {
                            updateSlack(u, rep[v]);
                        }
                        else if (onEdge(e[u][v]))
                        {
                            return true;
                        }
                    }
                }
            }
            W x = inf;
            for (int i = n + 1; i <= m; i++)
            {
                if (rep[i] == i && state[i] == 1)
                {
                    x = min(x, lab[i] / 2);
                }
            }
            for (int i = 1; i <= m; i++)
            {
                if (rep[i] == i && slack[i] && rep[slack[i]] != i && state[i] != 1)
                {
                    x = min(x, delta(e[slack[i]][i]) >> (state[i] + 1));
                }
            }
            for (int i = 1; i <= n; i++)
            {
                if (state[rep[i]] != -1)
                {
                    lab[i] += (state[rep[i]] * 2 - 1) * x;
                    if (lab[i] <= 0)
                    {
                        return false;
                    }
                }
            }
            for (int i = n + 1; i <= m; i++)
            {
                if (rep[i] == i && state[i] != -1)
                {
                    lab[i] += (2 - state[i] * 4) * x;
                }
            }
            head = 1;
            tail = 0;
            for (int i = 1; i <= m; i++)
            {
                if (rep[i] == i && slack[i] && rep[slack[i]] != i &&
                    !delta(e[slack[i]][i]) && onEdge(e[slack[i]][i]))
                {
```

```
                    return true;
                }
            }
            for (int i = n + 1; i <= m; i++)
            {
                if (rep[i] == i && state[i] == 1 && !lab[i])
                {
                    expand(i);
                }
            }
        }
    }

    pair<W, vector<pair<int, int>>> solve(const vector<tuple<int, int, W>> &edges)
    {
        // edges 是 0 下标无向边 {u,v,w}；返回最大总权和及所选边端点。
        m = n;
        stamp = 0;
        fill(link.begin(), link.end(), 0);
        fill(seen.begin(), seen.end(), 0);
        for (int i = 1; i <= n; i++)
        {
            for (int j = 1; j <= n; j++)
            {
                e[i][j].w = 0;
            }
        }
        for (int i = 1; i <= n; i++)
        {
            rep[i] = i;
            blossom[i].clear();
        }
        W wm = 0;
        for (auto [u, v, w] : edges)
        {
            assert(0 <= u && u < n && 0 <= v && v < n && u != v);
            u++, v++;
            e[u][v].w = e[v][u].w = max(e[u][v].w, w);
            wm = max(wm, w);
        }
        for (int i = 1; i <= n; i++)
        {
            for (int j = 1; j <= n; j++)
            {
                base[i][j] = i == j ? i : 0;
                if (i != j)
                {
                    e[i][j].u = i;
                    e[i][j].v = j;
                }
            }
        }
```

```
        fill(lab.begin() + 1, lab.begin() + n + 1, wm);
        while (bfs())
        {
        }
        W ans = 0;
        vector<pair<int, int>> match;
        for (int i = 1; i <= n; i++)
        {
            if (link[i])
            {
                ans += e[i][link[i]].w;
                if (i < link[i])
                {
                    match.push_back({i - 1, link[i] - 1});
                }
            }
        }
        return {ans / 2, match};
    }
};
```

`edges` 是点编号从 0 开始的无向边三元组，重复边只保留较大权值；`solve` 返回最大总权值和匹配边集。权值必须能容纳顶标调整与答案累加，默认使用 `long long`；需要更宽中间量时显式实例化 `WBlossom<__int128>`。这是最大权匹配，不保证匹配边数最大；若只求最大基数，使用 5.7.001 一般图最大匹配（Blossom），小图还可用 5.7.002 一般图最大权匹配。固定四点完全图可检查返回值为 11：

```
signed main()
{
    vector<tuple<int, int, long long>> e{{0, 1, 4}, {0, 2, 5}, {0, 3, 1},
                                         {1, 2, 2}, {1, 3, 6}, {2, 3, 3}};
    WBlossom<long long> b(4);
    auto [v, m] = b.solve(e);
    assert(v == 11 && m.size() == 2);
}
```

5.7.007 稀疏稳定匹配（不完全偏好）

完整偏好表即使只运行 Gale–Shapley，也必须读入和保存 $\Theta(n^2)$ 个排名；当两侧人数不同、每个人只接受少量对象时，应只存可接受关系。设左右侧人数为 n_L, n_R，可接受边数为 m。每条边同时给出它在左端点和右端点偏好表中的严格排名，排名从 0 开始且越小越优；同一个人的排名必须恰为 $[0, d)$ 的排列，其中 d 是其可接受对象数。未列出的两人彼此不可接受，不会组成阻塞对。

算法仍让左侧依次求婚。代码先按 `leftRank` 将边线性放入连续偏好表；右侧只需保存当前对象的 `rightRank`，即可在 $O(1)$ 时间决定保留谁。每条边最多被求婚一次，因此返回的匹配对左侧最优且稳定。与需要按偏好分数排序的接口相比，显式排名省去 $O(m \log m)$ 的预处理；代价是调用方必须先给出合法的严格排名。

```cpp
struct StableEdge
{
    int left, right;           // left、right 是可接受关系的左右端点。
    int leftRank, rightRank;   // 两个排名分别表示双方对该关系的偏好，0 最优。
};

struct SparseMatch
{
    vector<int> left, right; // 两个数组分别记录左右侧配偶；未匹配位置为 -1。
};

SparseMatch sparseMatch(int nLeft, int nRight,
                                    const vector<StableEdge> &edges)
{
    // nLeft、nRight 是两侧人数，edges 给出全部可接受关系及双方严格排名；返回左侧最优稳定匹配。
    assert(nLeft >= 0 && nRight >= 0);
    vector<int> beginLeft(nLeft + 1), beginR(nRight + 1);
    for (auto [x, y, rx, ry] : edges)
    {
        assert(0 <= x && x < nLeft && 0 <= y && y < nRight);
        ++beginLeft[x + 1];
        ++beginR[y + 1];
    }
    partial_sum(beginLeft.begin(), beginLeft.end(), beginLeft.begin());
    partial_sum(beginR.begin(), beginR.end(), beginR.begin());

    // pref 按左侧排名连续存边；usedRight 检查每个右侧排名恰好出现一次。
    vector<pair<int, int>> pref(edges.size(), {-1, -1});
    vector<char> usedRight(edges.size());
    for (auto [x, y, rx, ry] : edges)
    {
        assert(0 <= rx && rx < beginLeft[x + 1] - beginLeft[x]);
        assert(0 <= ry && ry < beginR[y + 1] - beginR[y]);
        int px = beginLeft[x] + rx, py = beginR[y] + ry;
        assert(pref[px].first == -1 && !usedRight[py]);
        pref[px] = {y, ry};
        usedRight[py] = true;
    }

    vector<int> left(nLeft, -1), right(nRight, -1), heldRank(nRight, -1);
    vector<int> next = beginLeft, q;
    q.reserve(nLeft + edges.size());
    for (int x = 0; x < nLeft; ++x)
    {
        if (next[x] < beginLeft[x + 1])
        {
            q.push_back(x);
        }
    }
    for (int head = 0; head < (int)q.size(); ++head)
    {
        int x = q[head];
```

```
        auto [y, rank] = pref[next[x]++];
        if (right[y] == -1 || rank < heldRank[y])
        {
            int old = right[y];
            if (old != -1)
            {
                left[old] = -1;
                if (next[old] < beginLeft[old + 1])
                {
                    q.push_back(old);
                }
            }
            left[x] = y;
            right[y] = x;
            heldRank[y] = rank;
        }
        else if (next[x] < beginLeft[x + 1])
        {
            q.push_back(x);
        }
    }
    return {left, right};
}
```

`sparseMatch(nLeft,nRight,edges)` 使用 0 下标；每对可接受对象在 `edges` 中恰好出现一次，左右两端的排名都必须严格且连续。返回的 `left[x]` 与 `right[y]` 互逆，`-1` 表示没有可接受的稳定配偶。若允许并列偏好，需要另行规定弱稳定、强稳定或超稳定，不能直接把相同排名交给本实现。时间与额外空间均为 $O(n_L+n_R+m)$。若每个人接受另一侧全部对象且两侧同样大，输入已有 $\Theta(n^2)$ 项，使用 5.7.005 稳定婚姻 的双完整偏好表接口更自然；若目标是最大匹配边数或总权而不考虑偏好稳定性，分别使用 5.6.001 Hopcroft–Karp 或 5.6.005 KM 算法。固定的 3 对 4 示例中，左侧配偶为 $[1,0,2]$，右侧第 3 人未匹配：

```
signed main()
{
    vector<StableEdge> e{{0, 0, 0, 1}, {0, 1, 1, 0},
                         {1, 0, 0, 0}, {2, 2, 0, 0}};
    auto ans = sparseMatch(3, 4, e);
    assert((ans.left == vector<int>{1, 0, 2}));
    assert((ans.right == vector<int>{1, 0, 2, -1}));
}
```

5.8 网络流与割

5.8.001 Dinic 最大流

在有向容量网络里求源 s 到汇 t 的最大流。Dinic 每轮先 BFS 建残量层次图，再用当前弧优化的 DFS 推阻塞流。`flow` 的 `lim` 参数允许只继续增广指定流量，`cut(s)` 在最大流跑满后返回残量图的源侧。本条没有实现按容量位分阶段的 Scaling，因此标题只保留普通 Dinic；`Flow` 是本章其余流模型的统一底座。

```cpp
template <class T>
struct Flow
{
    struct E
    {
        // to 是终点，rev 是反向边在 e[to] 中的下标。
        int to, rev;
        // cap 是当前残量容量。
        T cap;
    };

    // n 是残量网络点数。
    int n;
    // e[u] 是点 u 发出的全部残量边。
    vector<vector<E>> e;
    // h 是层次图距离，cur[u] 是点 u 尚未尝试的第一条残量边下标。
    vector<int> h, cur;

    Flow(int n = 0) : n(n), e(n), h(n), cur(n)
    {
        // n 是残量网络点数；构造空网络。
    }

    int add(int x, int y, T c)
    {
        // x、y 是端点，c 是非负容量；返回正向边在 e[x] 中的下标。
        assert(0 <= x && x < n && 0 <= y && y < n && c >= 0); // 调试检查，可删。
        int id = e[x].size();
        int rev = e[y].size() + (x == y);
        e[x].push_back({y, rev, c});
        e[y].push_back({x, id, 0});
        return id;
    }

    bool bfs(int s, int t)
    {
        // s、t 是源汇点；重建层次图，存在增广路时返回 true。
        fill(h.begin(), h.end(), -1);
        queue<int> q;
        h[s] = 0;
        q.push(s);
        while (!q.empty())
        {
            int x = q.front();
            q.pop();
            for (auto [y, r, c] : e[x])
            {
                if (c > 0 && h[y] == -1)
                {
                    h[y] = h[x] + 1;
                    q.push(y);
                }
```

```
            }
        }
        return h[t] != -1;
    }

    T dfs(int x, int t, T f)
    {
        // x 是当前点，t 是汇点，f 是流量上限；返回沿层次图实际送出的流量。
        if (x == t)
        {
            return f;
        }
        T ans = 0;
        for (int &i = cur[x]; i < (int)e[x].size(); ++i)
        {
            auto &a = e[x][i];
            if (a.cap == 0 || h[a.to] != h[x] + 1)
            {
                continue;
            }
            T d = dfs(a.to, t, min(f - ans, a.cap));
            a.cap -= d;
            e[a.to][a.rev].cap += d;
            ans += d;
            if (ans == f)
            {
                break;
            }
        }
        return ans;
    }

    T flow(int s, int t, T lim = numeric_limits<T>::max())
    {
        // s、t 是源汇点，lim 是本次流量上限；返回现有残量网络上的新增流量。
        if (s == t)
        {
            return 0;
        }
        T ans = 0;
        while (ans < lim && bfs(s, t))
        {
            fill(cur.begin(), cur.end(), 0);
            ans += dfs(s, t, lim - ans);
        }
        return ans;
    }

    vector<bool> cut(int s) const
    {
        // s 是残量搜索起点；返回残量图中从 s 可达的点集。
        vector<bool> vis(n);
```

```
        queue<int> q;
        vis[s] = true;
        q.push(s);
        while (!q.empty())
        {
            int x = q.front();
            q.pop();
            for (auto [y, r, c] : e[x])
            {
                if (c > 0 && !vis[y])
                {
                    vis[y] = true;
                    q.push(y);
                }
            }
        }
        return vis;
    }
};
```

用 `Flow<T> fl(n)` 建 n 点网络（0 下标），`add(x, y, c)` 加容量非负的有向边并返回它在 `e[x]` 中的下标；自环也能正确维护反向边，但不会改变 s-t 最大流。`flow(s, t)` 返回本次在现有残量网络上新增的流，`s==t` 约定返回 0；`cut(s)` 返回当前残量图中从 s 可达的点集，只有最大流已经跑满时才对应最小割源侧。`T` 必须容纳总流量及残量加法；若每条容量是 `long long` 而容量总和可能越界，应直接用 `__int128`。同一对象再次调用 `flow` 会继续增广，独立求解需重建。Dinic 一般网络最坏 $O(V^2E)$，单位容量网络 $O(E\sqrt{E})$，二分图匹配网络 $O(E\sqrt{V})$，空间 $O(V+E)$。稠密图上可比较 5.8.002 FIFO 预流推进、5.8.009 ISAP 最大流 与 5.8.010 HLPP / 最高标号预流推进；边带费用改用 5.8.003 SPFA 多路增广最小费用流 或 5.8.004 原始对偶最小费用流；有上下界先用 5.8.005 有上下界可行流、最大流、最小流 转成普通网络；无指定源汇的全局最小割见 5.8.007 Stoer-Wagner 全局最小割。

Dinic、FIFO 预流推进、ISAP 与 HLPP 共用固定小图检查，见 5.8.010 HLPP / 最高标号预流推进 末尾。

5.8.002 FIFO 预流推进

预流推进（push-relabel）不寻找完整增广路，而允许中间点暂时有超额流：`ex[u]>0` 表示流入多于流出。高度 `h[u]` 限制流只能沿 `h[u]=h[v]+1` 的残量边推向邻点；没有允许边时就把 `u` 重标号到所有残量邻点的最低高度加一。源点高度初始化为 n 并饱和其出边，最终超额流要么退回源点，要么进入汇点。本实现按 FIFO 队列处理活跃点，配合当前弧 `cur`，最坏 $O(V^2E)$。它没有最高标号选择、全局重标号和 gap 启发式，因此不是 HLPP；标题与复杂度都按实际的 FIFO 预流推进书写。`PushRelabel` 与 5.8.001 Dinic 最大流 接口平行。

```
template <class T>
struct PushRelabel
{
    struct E
    {
        // 残量边终点与反向边下标。
        int v, r;
```

```
        // 当前残量容量。
        T c;
    };

    // 点数与残量邻接表。
    int n;
    vector<vector<E>> e;
    // 每个点的超额流。
    vector<T> ex;
    // 高度与当前弧。
    vector<int> h, cur;
    // 是否已在活跃队列中。
    vector<bool> in;
    // FIFO 活跃点队列。
    queue<int> q;

    PushRelabel(int n = 0) : n(n), e(n), ex(n), h(n), cur(n), in(n)
    {
        // n 是残量网络点数；构造空网络。
    }

    void add(int u, int v, T c)
    {
        // u、v 是端点，c 是非负容量；加入一对残量边。
        assert(0 <= u && u < n && 0 <= v && v < n && c >= 0); // 调试检查，可删。
        int id = e[u].size();
        int rev = e[v].size() + (u == v);
        e[u].push_back({v, rev, c});
        e[v].push_back({u, id, 0});
    }

    void active(int u, int s, int t)
    {
        // 超额为正且不是源汇时，把 u 加入活跃队列。
        if (u != s && u != t && ex[u] > 0 && !in[u])
        {
            in[u] = true;
            q.push(u);
        }
    }

    void push(int u, E &a, int s, int t)
    {
        // 若 a 是允许边，就从 u 沿 a 推出尽量多的超额流。
        if (!a.c || h[u] != h[a.v] + 1)
        {
            return;
        }
        T f = min(ex[u], a.c);
        a.c -= f;
        e[a.v][a.r].c += f;
        ex[u] -= f;
```

```
        ex[a.v] += f;
        active(a.v, s, t);
    }

    T flow(int s, int t)
    {
        // s、t 是源汇点；从初始网络求最大流并返回流量。
        if (s == t)
        {
            return 0;
        }
        h[s] = n;
        for (auto &a : e[s])
        {
            T f = a.c;
            a.c = 0;
            e[a.v][a.r].c += f;
            ex[a.v] += f;
            ex[s] -= f;
            active(a.v, s, t);
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            in[u] = false;
            while (ex[u] > 0)
            {
                if (cur[u] == (int)e[u].size())
                {
                    h[u] = 2 * n;
                    for (auto a : e[u])
                    {
                        if (a.c)
                        {
                            h[u] = min(h[u], h[a.v] + 1);
                        }
                    }
                    cur[u] = 0;
                    continue;
                }
                push(u, e[u][cur[u]], s, t);
                if (!e[u][cur[u]].c || h[u] != h[e[u][cur[u]].v] + 1)
                {
                    cur[u]++;
                }
            }
        }
        return ex[t];
    }
};
```

用 `PushRelabel<T> pr(n)` 建网络，`add(u, v, c)` 加有向边，`flow(s, t)` 返回最大流；`s==t` 约定返回 0，同一对象重复求解需重建。`T` 必须容纳总流量，容量总和可能超过 `long long` 时改用 `__int128`。FIFO 预流推进最坏 $O(V^2E)$，空间 $O(V+E)$。它代码短，稠密图上可与 Dinic、5.8.009 ISAP 最大流 和 5.8.010 HLPP / 最高标号预流推进 分别实测选型；需要最高标号、GAP 和全局重标号时应直接使用 HLPP。稀疏图和二分图匹配网络通常优先用 5.8.001 Dinic 最大流。带费用需求见 5.8.003 SPFA 多路增广最小费用流。

本实现与 Dinic、ISAP、HLPP 共用 5.8.010 HLPP / 最高标号预流推进 末尾的固定小图检查，不再重复驱动。

5.8.003 SPFA 多路增广最小费用流

最小费用流要求在发送指定流量（默认尽量发送最大流）的前提下，使 $\sum$(单位费用 $\times$ 流量) 最小。若当前流已是该流量下的最优解，沿残量图中的最短 $s \to t$ 路增广后，新流仍最优。残量反向边带负费用，因此本版每轮先用 SPFA 求最短距离 d，再把满足 $d_u + c_{uv} = d_v$ 的残量边称为紧边；从源点沿紧边用当前弧 DFS，一轮可同时推送多条等长最短路。每单位新增流的费用均为 d_t，所以批量增广仍保持最优性。零费用紧边可能成环，`vis` 只禁止当前递归路径重复点，下一轮 SPFA 会重建全部距离与当前弧。本实现常被笼统称作 ZKW 费用流，但它每轮重新运行 SPFA，没有使用经典 ZKW 的标号调整，故标题按实际过程命名。算法要求残量网络中不存在从 s 可达的负费用环。

```cpp
template <class T = long long, class Cost = long long>
struct SpfaCostFlow
{
    struct E
    {
        int to, rev;  // 终点与反向边下标。
        T cap;        // 当前残量容量。
        Cost cost;    // 单位流量费用。
    };

    int n;                // 残量网络点数。
    vector<vector<E>> e; // 每个点的残量边。
    // d 是本轮从源点出发的最短距离，空值表示不可达。
    vector<optional<Cost>> d;
    // in 标记 SPFA 队列中的点，cur 是紧边 DFS 的当前弧，vis 标记递归栈中的点。
    vector<int> in, cur, vis;

    SpfaCostFlow(int n) : n(n), e(n), d(n), in(n), cur(n), vis(n)
    {
        // n 是点数；构造空残量网络，无返回值。
    }

    void add(int u, int v, T cap, Cost cost)
    {
        // u、v 是端点，cap 是容量，cost 是单位费用；加入正向边和零容量反向边，无返回值。
        assert(0 <= u && u < n && 0 <= v && v < n && cap >= 0); // 调试检查，可删。
        assert(cost != numeric_limits<Cost>::min()); // 调试检查，可删。
        int id = e[u].size();
        int rev = e[v].size() + (u == v);
```

```
        e[u].push_back({v, rev, cap, cost});
        e[v].push_back({u, id, 0, -cost});
    }

    bool shortest(int s, int t)
    {
        // s、t 是源汇点；用 SPFA 建立残量最短距离，返回 t 是否可达。
        fill(d.begin(), d.end(), nullopt);
        fill(in.begin(), in.end(), 0);
        queue<int> q;
        d[s] = 0;
        q.push(s);
        in[s] = 1;
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            in[u] = 0;
            for (const auto &a : e[u])
            {
                if (a.cap == 0)
                {
                    continue;
                }
                Cost nd = *d[u] + a.cost;
                if (!d[a.to].has_value() || nd < *d[a.to])
                {
                    d[a.to] = nd;
                    if (!in[a.to])
                    {
                        q.push(a.to);
                        in[a.to] = 1;
                    }
                }
            }
        }
        return d[t].has_value();
    }

    T dfs(int u, int t, T lim)
    {
        // u、t 是当前点与汇点，lim 是流量上限；沿本轮紧边尽量增广并返回实际流量。
        if (u == t)
        {
            return lim;
        }
        vis[u] = 1;
        T f = 0;
        for (int &i = cur[u]; i < (int)e[u].size() && f < lim; i++)
        {
            auto &a = e[u][i];
            if (a.cap == 0 || vis[a.to] || !d[a.to].has_value() ||
```

```
                *d[u] + a.cost != *d[a.to])
            {
                continue;
            }
            T x = dfs(a.to, t, min(lim - f, a.cap));
            if (x == 0)
            {
                continue;
            }
            a.cap -= x;
            e[a.to][a.rev].cap += x;
            f += x;
        }
        vis[u] = 0;
        return f;
    }

    pair<T, Cost> flow(int s, int t, T lim = numeric_limits<T>::max())
    {
        // s、t 是源汇点，lim 是最多发送的流量；返回实际流量与最小费用。
        if (s == t)
        {
            return {0, 0};
        }
        assert(lim >= 0); // 调试检查，可删。
        T f = 0;
        Cost cost = 0;
        while (f < lim && shortest(s, t))
        {
            fill(cur.begin(), cur.end(), 0);
            fill(vis.begin(), vis.end(), 0);
            T x = dfs(s, t, lim - f);
            assert(x > 0); // 调试检查，可删：最短路存在时紧边 DFS 必须能增广。
            if (x == 0) break;
            f += x;
            cost += (Cost)x * *d[t];
        }
        return {f, cost};
    }
};
```

构造 `SpfaCostFlow<T,Cost> g(n)`，先用 `add(u,v,cap,cost)` 加完全部边，再调用 `flow`；容量与费用默认都是 `long long`。`flow(s,t,lim)` 最多发送 `lim`，返回 `{实际流量，最小费用}`。调用者必须保证反向费用、路径费用、单次乘积和总费用都可由 `Cost` 表示，且单位费用不等于 `numeric_limits<Cost>::min()`。`s==t` 约定返回 `{0,0}`，其余情况要求无源点可达的负费用环。同一边集上可以继续调用 `flow` 增广，但第一次求流后不要再加边。递归深度最坏为 $O(V)$；设 SPFA 轮数为 $K \le F$，其中 F 是整数容量下的最大流值，每轮 SPFA 最坏 $O(VE)$、紧边 DFS 按当前弧扫描 $O(E)$，故总时间最坏仍为 $O(FVE)$，空间为 $O(V+E)$。大量流能走费用相同的最短路时，K 可远小于 F；若最短费用大多互异或数据会卡 SPFA，则优先比较 5.8.004 原始对偶最小费用流。两版共用下一条末尾的检查。

5.8.004 原始对偶最小费用流

原始对偶法用 Johnson 势能把每轮最短路改成 Dijkstra。对残量边 (u, v) 定义约化费用 $c_{uv} + h_u - h_v$；若 h 是从源点出发的最短距离，则所有源点可达残量边的约化费用非负。首次调用先用一次 SPFA 初始化势能，所以允许负费用边，但仍要求不存在源点可达的负费用环。此后每轮 Dijkstra 求约化距离，并把可达点的距离累加到 `h`。势能保存在对象里，不会在下一次 `flow` 调用时清零，因此相同源汇可以分段继续增广。

```cpp
// 依赖 5.8.003 SPFA 多路增广最小费用流
template <class T = long long, class Cost = long long>
struct CostFlow
{
    struct E
    {
        // 终点与反向边下标。
        int to, rev;
        // 当前残量容量。
        T cap;
        // 单位流量费用。
        Cost cost;
    };

    // 点数、残量邻接表与跨调用保留的势能。
    int n;
    vector<vector<E>> e;
    vector<Cost> h;
    bool ready = false;
    int source = -1, sink = -1;

    CostFlow(int n = 0) : n(n), e(n), h(n)
    {
        // n 是残量网络点数；构造空网络。
    }

    void add(int x, int y, T cap, Cost cost)
    {
        // x、y 是端点，cap 是非负容量，cost 是单位费用；加入一对残量边。
        assert(0 <= x && x < n && 0 <= y && y < n && cap >= 0); // 调试检查，可删。
        assert(source == -1); // 调试检查，可删：所有边须在首次 flow 前加入。
        assert(cost != numeric_limits<Cost>::min()); // 调试检查，可删。
        int id = e[x].size();
        int rev = e[y].size() + (x == y);
        e[x].push_back({y, rev, cap, cost});
        e[y].push_back({x, id, 0, -cost});
        ready = false;
    }

    void init(int s)
    {
        // s 是源点；用 SPFA 求初始势能，要求不存在 s 可达的负费用环。
        vector<optional<Cost>> d(n);
```

```
        vector<bool> in(n);
        queue<int> q;
        d[s] = 0;
        q.push(s);
        in[s] = true;
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            in[u] = false;
            for (const auto &a : e[u])
            {
                Cost nd = *d[u] + a.cost;
                if (a.cap > 0 &&
                    (!d[a.to].has_value() || nd < *d[a.to]))
                {
                    d[a.to] = nd;
                    if (!in[a.to])
                    {
                        in[a.to] = true;
                        q.push(a.to);
                    }
                }
            }
        }
        fill(h.begin(), h.end(), 0);
        for (int u = 0; u < n; u++)
        {
            if (d[u].has_value())
            {
                h[u] = *d[u];
            }
        }
        ready = true;
    }

    pair<T, Cost> flow(int s, int t, T lim = numeric_limits<T>::max())
    {
        // s、t 是固定源汇，lim 是流量上限；返回本次新增流量与费用。
        if (s == t)
        {
            return {0, 0};
        }
        assert(lim >= 0); // 调试检查，可删。
        if (source == -1)
        {
            source = s;
            sink = t;
        }
        assert(source == s && sink == t); // 调试检查，可删：分段调用不能更换源汇。
        if (!ready)
        {
```

```
            init(s);
        }

        vector<int> pv(n), pe(n);
        T f = 0;
        Cost cost = 0;
        while (f < lim)
        {
            vector<optional<Cost>> dis(n);
            priority_queue<pair<Cost, int>, vector<pair<Cost, int>>, greater<>> q;
            dis[s] = 0;
            q.emplace(0, s);
            while (!q.empty())
            {
                auto [d, x] = q.top();
                q.pop();
                if (!dis[x].has_value() || d != *dis[x])
                {
                    continue;
                }
                for (int i = 0; i < (int)e[x].size(); ++i)
                {
                    auto &a = e[x][i];
                    if (a.cap == 0)
                    {
                        continue;
                    }
                    Cost nd = d + a.cost + h[x] - h[a.to];
                    assert(nd >= 0); // 调试检查，可删：势能应保证约化费用非负。
                    if (!dis[a.to].has_value() || nd < *dis[a.to])
                    {
                        dis[a.to] = nd;
                        pv[a.to] = x;
                        pe[a.to] = i;
                        q.emplace(nd, a.to);
                    }
                }
            }
            if (!dis[t].has_value())
            {
                break;
            }
            for (int i = 0; i < n; ++i)
            {
                if (dis[i].has_value())
                {
                    h[i] += *dis[i];
                }
            }
            T d = lim - f;
            for (int x = t; x != s; x = pv[x])
            {
```

```
                d = min(d, e[pv[x]][pe[x]].cap);
            }
            for (int x = t; x != s; x = pv[x])
            {
                auto &a = e[pv[x]][pe[x]];
                a.cap -= d;
                e[x][a.rev].cap += d;
            }
            f += d;
            cost += (Cost)d * (h[t] - h[s]);
        }
        return {f, cost};
    }
};
```

构造 `CostFlow<T,Cost> g(n)`，先用 `add(x,y,cap,cost)` 加完全部边，容量与费用默认都是 `long long`；`flow(s,t,lim)` 返回本次新增的 `{流量，最小费用}`。调用者必须保证反向费用、距离、势能、单次乘积和总费用都可由 `Cost` 表示，且单位费用不等于 `numeric_limits<Cost>::min()`。允许负费用边但不允许源点可达的负费用环；同一对象可以对相同 (s,t) 分段调用，不能中途更换源汇或继续加边。首次 SPFA 为 $O(VE)$，之后每次增广的 Dijkstra 为 $O(E\log V)$；整数容量下增广次数至多为最大流值 F，总复杂度 $O(VE+FE\log V)$，空间 $O(V+E)$。本版避免每轮 SPFA，最短费用大多互异或构造数据上更稳；它每轮只恢复一条路径，在大量等费用最短路并存时可能慢于 5.8.003 SPFA 多路增广最小费用流。两版各自核对下一段固定网络的已知答案；只求最大流用 5.8.001 Dinic 最大流。

本地性能检查使用 Windows 10、Ryzen 9 7945HX、GCC 14.2.0 与 `-O2 -pipe`，每个场景先预热一次，再重复 5 或 7 次取中位数，并把整个进程重复三轮。120×120 全零费用指派图上，多路 SPFA 为 0.486–0.541 ms，原始对偶为 4.540–4.714 ms；宽度 500、每点 8 条等费用边的分层图上，两者分别为 0.333–0.343 ms 与 14.016–14.803 ms。随机费用指派图上两者分别为 7.316–7.657 ms 与 8.122–8.297 ms，差距很小。这些结果只说明同费用最短路的批量增广收益，不替代 SPFA 与 Dijkstra 的最坏复杂度比较；

下面直接检查一张含负费用边的小图。最大流为 3，最小费用为 9；原始对偶版分两次调用时，第一单位费用为 0，余下两单位费用为 9。

```
template <class F>
void addFlowCase(F &g)
{
    // g 是四点费用流网络；加入含负费用边的固定样例。
    g.add(0, 1, 2, 1);
    g.add(0, 2, 1, 4);
    g.add(1, 2, 1, -2);
    g.add(1, 3, 1, 3);
    g.add(2, 3, 2, 1);
}

signed main()
{
    SpfaCostFlow<> a(4);
    addFlowCase(a);
    assert((a.flow(0, 3) == pair<long long, long long>{3, 9}));

```

```
    CostFlow<> b(4);
    addFlowCase(b);
    assert((b.flow(0, 3, 1) == pair<long long, long long>{1, 0}));
    assert((b.flow(0, 3) == pair<long long, long long>{2, 9}));
    return 0;
}
```

5.8.005 有上下界可行流、最大流、最小流

每条边带流量下界 l 与上界 r，先问是否存在满足所有 $l \le f_e \le r$ 与流量守恒的可行流，再在此基础上求指定源汇的最大流或最小流。标准做法是「拆下界」：让每条边先强制流 l，剩余可调容量为 $r-l$；强制的这部分会破坏守恒，于是给每个点算净盈亏 $d_v = \sum_{\text{入}} l - \sum_{\text{出}} l$，再建超级源 ss、超级汇 tt，对 $d_v > 0$ 的点连 $ss \to v$ 容量 d_v、$d_v < 0$ 的点连 $v \to tt$ 容量 $-d_v$。当 $ss \to tt$ 的最大流恰好灌满所有超级边（等于 $\sum_{d_v>0} d_v$）时可行流存在，每条原边的实际流量为 $l+$（其残量边反向容量）。若还要求原网络 $s \to t$ 的最大/最小流，先加一条 $t \to s$ 容量无穷的边把有源汇问题转成无源汇可行流问题，判可行后拆掉这条边，在残量网络上再对 $s \to t$（最大流）或 $t \to s$（最小流，从可行值里减去）增广。内部复用 5.8.001 Dinic 最大流 的 `Flow` 模板，此处不重复打印。

```
template <class Cap = long long>
struct BoundResult
{
    // 指定源汇时的流值；只求循环可行流时为 0。
    Cap value;
    // 每条原边的实际流量。
    vector<Cap> edge;
};

template <class Cap = long long>
struct BoundFlow
{
    struct E
    {
        // 端点与流量上下界。
        int u, v;
        Cap l, r;
    };

    // 点数与原图边集。
    int n;
    vector<E> e;

    BoundFlow(int n = 0) : n(n)
    {
        // n 是原图点数；构造空网络。
    }

    void add(int u, int v, Cap l, Cap r)
    {
        // u、v 是端点，l、r 是非负流量下界和上界。
```

```
        assert(0 <= u && u < n && 0 <= v && v < n && 0 <= l && l <= r); // 调试检查，可删。
        e.push_back({u, v, l, r});
    }

    optional<BoundResult<Cap>> solve(int s = -1, int t = -1, bool minimum = false) const
    {
        // s=t=-1 求循环可行流；否则 minimum 决定求最小流还是最大流。
        assert((s == -1 && t == -1) ||
               (0 <= s && s < n && 0 <= t && t < n && s != t)); // 调试检查，可删。
        int ss = n, tt = n + 1;
        Flow<Cap> f(n + 2);
        vector<Cap> d(n);
        vector<pair<int, int>> id;
        Cap sumR = 0;
        for (const auto &x : e)
        {
            id.push_back({x.u, f.add(x.u, x.v, x.r - x.l)});
            d[x.u] -= x.l;
            d[x.v] += x.l;
            sumR += x.r;
        }
        int art = -1;
        if (s != -1)
        {
            art = f.add(t, s, sumR + 1);
        }
        Cap need = 0;
        for (int i = 0; i < n; i++)
        {
            if (d[i] > 0)
            {
                f.add(ss, i, d[i]);
                need += d[i];
            }
            else if (d[i] < 0)
            {
                f.add(i, tt, -d[i]);
            }
        }
        if (f.flow(ss, tt) != need)
        {
            return nullopt;
        }
        Cap val = 0;
        if (s != -1)
        {
            auto &a = f.e[t][art];
            val = f.e[s][a.rev].cap;
            a.cap = 0;
            f.e[s][a.rev].cap = 0;
        }
        for (int u : {ss, tt})
```

```
        {
            for (auto &a : f.e[u])
            {
                f.e[a.to][a.rev].cap = 0;
                a.cap = 0;
            }
        }
        if (s != -1)
        {
            if (minimum)
            {
                val -= f.flow(t, s, val);
            }
            else
            {
                val += f.flow(s, t);
            }
        }
        vector<Cap> ans(e.size());
        for (int i = 0; i < (int)e.size(); i++)
        {
            auto [u, j] = id[i];
            auto &a = f.e[u][j];
            ans[i] = e[i].l + f.e[a.to][a.rev].cap;
        }
        return BoundResult<Cap>{val, ans};
    }
};
```

代码依赖 5.8.001 Dinic 最大流 的 `Flow`。构造 `BoundFlow<Cap> g(n)`，`Cap` 默认是 `long long`，`add(u,v,l,r)` 加一条同类型上下界边。`solve()` 求循环可行流；`solve(s,t,false)` 求非负 $s \to t$ 最大流，`solve(s,t,true)` 求非负最小流。只在无可行流时返回空，否则 `.value` 是流值，`.edge[i]` 是第 i 条原边的流量。调用者必须保证平衡量、所有上界之和、流值和边流可由 `Cap` 表示；人工边容量取所有上界之和加一，不使用可能偏小的固定 “无穷”。复杂度为两次 Dinic 的量级，最坏 $O(V^2E)$，空间 $O(V+E)$。带费用的上下界流还需累计下界的固定费用，并把增广器换成 5.8.003 SPFA 多路增广最小费用流 或 5.8.004 原始对偶最小费用流。

5.8.006 最大流最小割与建图模型

最大流最小割定理（s-t 最大流等于最小割容量）让一大类「每个对象二选一、带一元代价与二元关系代价」的最小化问题变成求最小割。约定割后与 s 同侧记为取 1、与 t 同侧记为取 0。三种常用建模砖块：一元代价——变量 u 取 0 花 c_0、取 1 花 c_1，连 $s \to u$ 容量 c_0、$u \to t$ 容量 c_1；蕴含约束 $u{=}1 \Rightarrow v{=}1$——连 $u \to v$ 硬约束边；异值代价——u, v 取值不同时付 c，连两条方向相反、容量均为 c 的边。`BinaryCut` 只保存这些建模项，求解时统一调用 5.8.001 Dinic 最大流，硬约束容量自动取全部有限容量之和加一。

```
// 依赖 5.8.001 Dinic 最大流
template <class Cap = long long>
struct BinaryCut
```

```
{
    // 变量数与内部源汇编号。
    int n, s, t;
    // 有限容量边、硬约束边及全部有限容量之和。
    vector<tuple<int, int, Cap>> edge;
    vector<pair<int, int>> hard;
    Cap sum = 0;

    BinaryCut(int n) : n(n), s(n), t(n + 1)
    {
        // n 是二元变量数；构造空模型。
    }

    void addCost(int u, Cap c0, Cap c1)
    {
        // c0、c1 是 u 取 0、1 的非负代价。
        assert(0 <= u && u < n && c0 >= 0 && c1 >= 0); // 调试检查，可删。
        edge.push_back({s, u, c0});
        edge.push_back({u, t, c1});
        sum += c0 + c1;
    }

    void imply(int u, int v)
    {
        // 加入 u=1 推出 v=1 的硬约束。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        hard.push_back({u, v});
    }

    void addDiffCost(int u, int v, Cap c)
    {
        // 二者取值不同时付非负代价 c。
        assert(0 <= u && u < n && 0 <= v && v < n && c >= 0); // 调试检查，可删。
        edge.push_back({u, v, c});
        edge.push_back({v, u, c});
        sum += c + c;
    }

    pair<Cap, vector<int>> solve() const
    {
        // 返回最小代价与一组取值；源点侧变量取 1。
        Cap inf = sum + 1;
        Flow<Cap> fl(n + 2);
        for (auto [u, v, c] : edge)
        {
            fl.add(u, v, c);
        }
        for (auto [u, v] : hard)
        {
            fl.add(u, v, inf);
        }
        Cap ans = fl.flow(s, t);
```

```
        vector<bool> side = fl.cut(s);
        vector<int> val(n);
        for (int u = 0; u < n; u++)
        {
            val[u] = side[u];
        }
        return pair<Cap, vector<int>>{ans, val};
    }
};
```

构造 `BinaryCut<Cap> q(n)`，`Cap` 默认是 `long long`；`addCost(u,c0,c1)` 加一元代价，`imply(u,v)` 加 $u{=}1 \Rightarrow v{=}1$，`addDiffCost(u,v,c)` 加异值代价，有限代价都使用 `Cap`。`solve()` 返回 `{最小代价，取值向量}`，可重复调用。调用者必须保证全部有限代价之和加一可由 `Cap` 表示；这个和加一就是硬约束容量，不依赖全局 `INF`。例如两变量代价为 $(3,0)$、$(0,2)$，再加 `imply(0,1)`，最优值为 2，取值为 $\{1,1\}$。Dinic 最坏 $O(V^2E)$，空间 $O(V+E)$。最大权闭合子图见 5.6.011 最大权闭合子图，全局最小割见 5.8.007 Stoer-Wagner 全局最小割，所有点对最小割见 5.8.008 Gomory-Hu 最小割树。

5.8.007 Stoer-Wagner 全局最小割

求无向带权图的全局最小割——把点集分成非空两部分、被切断的边权和最小，且不指定哪两个点必须分开。若枚举源汇跑 $\binom{n}{2}$ 次最大流太慢。Stoer–Wagner 用「最大邻接序」把它压到 $O(n^3)$：每一阶段从空集出发，反复把「与当前已选集合边权和最大」的点加入，直到所有点加完；记最后加入的点为 t、倒数第二个为 s，则 t 到已选集合的边权和恰是一个「s-t 最小割」的值（最小割阶段引理）。用它更新全局答案后，把 s、t 合并成一个点（边权相加）继续下一阶段，$n-1$ 阶段后剩一个点，期间记录的最小割即全局最小割。`GlobalCut.side` 保存取到最小值那一刻 t 所代表的原始点集合，即最小割的一侧。

```
struct GlobalCut
{
    // 最小割值与割的一侧。
    int w;
    vector<int> side;
};

GlobalCut stoerWagner(vector<vector<int>> g)
{
    // g 是无向图的对称非负权矩阵；返回全局最小割值和一侧点集。
    int n = g.size();
    if (n <= 1)
    {
        return {0, n ? vector<int>{0} : vector<int>{}};
    }
    for (int i = 0; i < n; i++)
    {
        assert((int)g[i].size() == n); // 调试检查，可删。
        for (int j = 0; j < n; j++)
        {
            assert(g[i][j] >= 0 && g[i][j] == g[j][i]); // 调试检查，可删。
        }
```

```
        g[i][i] = 0;
    }
    vector<int> v(n);
    vector<vector<int>> part(n);
    iota(v.begin(), v.end(), 0);
    for (int i = 0; i < n; i++)
    {
        part[i] = {i};
    }
    GlobalCut ans{numeric_limits<int>::max(), {}};
    while (v.size() > 1)
    {
        vector<int> w(n);
        vector<bool> used(n);
        int s = -1, t = -1;
        for (int z = 0; z < (int)v.size(); z++)
        {
            s = t;
            t = -1;
            for (int u : v)
            {
                if (!used[u] && (t < 0 || w[u] > w[t]))
                {
                    t = u;
                }
            }
            if (z + 1 == (int)v.size())
            {
                if (w[t] < ans.w)
                {
                    ans = {w[t], part[t]};
                }
                for (int u : v)
                {
                    if (u != s)
                    {
                        g[s][u] += g[t][u];
                        g[u][s] = g[s][u];
                    }
                }
                part[s].insert(part[s].end(), part[t].begin(), part[t].end());
                v.erase(find(v.begin(), v.end(), t));
                break;
            }
            used[t] = true;
            for (int u : v)
            {
                w[u] += g[t][u];
            }
        }
    }
    return ans;
```

```
}
```

`stoerWagner(g)` 传入 $n \times n$ 对称非负权矩阵，无边填 0，对角线会被忽略；返回的 `w` 是全局最小割值，`side` 是割的一侧。参数按值传递，算法会直接在副本上收缩；调用者必须保证任意收缩边权和可由 64 位有符号整数表示。$n \le 1$ 时不存在非平凡割，约定返回 0。时间 $O(n^3)$、空间 $O(n^2)$。第 5.3 节只引用本条，不再保留第二份实现；指定源汇最小割用 5.8.001 Dinic 最大流，所有点对最小割用 5.8.008 Gomory-Hu 最小割树，二元建模见 5.8.006 最大流最小割与建图模型。

下面检查四点带权图与不连通图。第一张图的全局最小割为孤立点 3，割值是 4；第二张图已有两个连通块，割值是 0。

```
signed main()
{
    vector<vector<int>> g{{0, 3, 2, 0},
                          {3, 0, 4, 1},
                          {2, 4, 0, 3},
                          {0, 1, 3, 0}};
    auto a = stoerWagner(g);
    assert(a.w == 4 && !a.side.empty() && a.side.size() < 4);

    vector<vector<int>> h{{0, 2, 0}, {2, 0, 0}, {0, 0, 0}};
    assert(stoerWagner(h).w == 0);
}
```

5.8.008 Gomory-Hu 最小割树

无向图有 $\binom{n}{2}$ 个点对最小割值，Gomory-Hu 树用 $n-1$ 条带权树边编码它们：树上 u, v 路径的最小边权等于原图 u-v 最小割。因此不同割值至多有 $n-1$ 个，并不保证树边权两两不同。构造只需 $n-1$ 次最大流。这里用 Gusfield 简化版：初始父点均为 0，依次求 i 到 `p[i]` 的最小割，再按残量源侧修正父指针。每轮都重建 5.8.001 Dinic 最大流 的网络。

```
// 依赖 5.8.001 Dinic 最大流
template <class Cap = long long>
vector<tuple<int, int, Cap>>
gomoryHu(int n, const vector<tuple<int, int, long long>> &edges)
{
    // edges 是非负权无向边；返回 Gomory-Hu 树边及割值。
    vector<int> p(n);
    vector<Cap> w(n);
    for (int s = 1; s < n; s++)
    {
        int t = p[s];
        Flow<Cap> f(n);
        for (auto [u, v, c] : edges)
        {
            assert(0 <= u && u < n && 0 <= v && v < n && c >= 0); // 调试检查，可删。
            f.add(u, v, (Cap)c);
            f.add(v, u, (Cap)c);
        }
```

```
        Cap z = f.flow(s, t);
        auto cut = f.cut(s);
        for (int i = s + 1; i < n; i++)
        {
            if (p[i] == t && cut[i])
            {
                p[i] = s;
            }
        }
        if (t && cut[p[t]])
        {
            p[s] = p[t];
            p[t] = s;
            w[s] = w[t];
            w[t] = z;
        }
        else
        {
            w[s] = z;
        }
    }
    vector<tuple<int, int, Cap>> ans;
    for (int i = 1; i < n; i++)
    {
        ans.push_back({i, p[i], w[i]});
    }
    return ans;
}
```

`gomoryHu<Cap>(n,edges)` 中 `Cap` 默认是 `long long`，平行边可直接重复加入；返回 $n-1$ 条树边 `(点，父点，割值)`。调用者必须保证任意点对最小割可由 `Cap` 表示。任意两点最小割等于树上路径最小边权，可再配合 6.1.001 LCA / 倍增与 ST 表 查询。算法调用 $n-1$ 次最大流；Dinic 最坏 $O(n^3m)$，空间 $O(n+m)$。只查一个点对时直接用 5.8.001 Dinic 最大流，只要所有点对中的最小值用 5.8.007 Stoer-Wagner 全局最小割。

5.8.009 ISAP 最大流

ISAP（Improved Shortest Augmenting Path）是 Dinic 之外的另一支高效最大流实现，很多稠密图上常数更小。它给每个点维护到汇点 t 的距离标号 `h[]`（先反向 BFS 从 t 初始化），DFS 只沿「$h_x = h_y + 1$」的允许边推流；当前点走不通时不重跑全局 BFS，而是局部重标号——把 h_x 抬到「所有仍有残量的出边终点 h 的最小值加一」，这样标号单调递增、总重标号次数 $O(V^2)$。关键加速是 GAP 优化：`gap[d]` 记距离标号恰为 d 的点数，一旦某层被抬空（`--gap[h[x]]==0`），说明 s 与 t 之间出现了断层、不可能再有增广路，立即把源点标号设为 n 提前终止整个算法。

```
template <class T>
struct ISAP
{
    struct E
    {
```

```
        int to, rev; // 终点与反向边下标。
        T cap;       // 当前残量容量。
    };

    int n, src = 0;       // n 是点数，src 是本次增广的源点。
    vector<vector<E>> e; // e[u] 是点 u 发出的全部残量边。
    vector<int> h, gap;  // h 是到汇点的距离标号，gap[d] 是标号为 d 的点数。
    vector<int> cur;     // cur[u] 是点 u 尚未尝试的第一条残量边下标。

    ISAP(int n = 0) : n(n), e(n), h(n), gap(n + 1), cur(n)
    {
        // n 是点数；构造空残量网络。
    }

    void add(int x, int y, T c)
    {
        // x、y 是端点，c 是非负容量；加入一对残量边。
        assert(0 <= x && x < n && 0 <= y && y < n && c >= 0); // 调试检查，可删。
        int id = e[x].size();
        int rev = e[y].size() + (x == y);
        e[x].push_back({y, rev, c});
        e[y].push_back({x, id, 0});
    }

    void bfs(int t)
    {
        // t 是汇点；在反向残量意义下初始化距离标号和 gap 计数，无返回值。
        fill(h.begin(), h.end(), n);
        fill(gap.begin(), gap.end(), 0);
        queue<int> q;
        h[t] = 0;
        q.push(t);
        while (!q.empty())
        {
            auto x = q.front();
            q.pop();
            for (auto [y, rev, cap] : e[x])
            {
                if (h[y] != n || e[y][rev].cap == 0)
                {
                    continue;
                }
                h[y] = h[x] + 1;
                q.push(y);
            }
        }
        for (auto d : h)
        {
            gap[d]++;
        }
    }

```

```
    T dfs(int x, int t, T lim)
    {
        // x、t、lim 是当前点、汇点和流量上限；返回本次实际增广量。
        if (x == t)
        {
            return lim;
        }
        T used = 0;
        for (auto &i = cur[x]; i < (int)e[x].size(); i++)
        {
            auto &a = e[x][i];
            if (a.cap == 0 || h[x] != h[a.to] + 1)
            {
                continue;
            }
            T d = dfs(a.to, t, min(lim - used, a.cap));
            a.cap -= d;
            e[a.to][a.rev].cap += d;
            used += d;
            if (used == lim || h[x] == n)
            {
                return used;
            }
        }
        if (--gap[h[x]] == 0)
        {
            h[src] = n;
            return used;
        }
        int nh = n - 1;
        for (const auto &a : e[x])
        {
            if (a.cap > 0)
            {
                nh = min(nh, h[a.to]);
            }
        }
        h[x] = nh + 1;
        gap[h[x]]++;
        cur[x] = 0;
        return used;
    }

    T flow(int s, int t)
    {
        // s、t 是源汇点；返回现有残量上新增的最大流。
        if (s == t)
        {
            return 0;
        }
        src = s;
        bfs(t);
```

```
        fill(cur.begin(), cur.end(), 0);
        T ans = 0;
        while (h[s] < n && ans < numeric_limits<T>::max())
        {
            ans += dfs(s, t, numeric_limits<T>::max() - ans);
        }
        return ans;
    }
};
```

构造 `ISAP<T> is(n)`，`add(u, v, c)` 加非负容量边，`flow(s, t)` 返回现有残量网络上新增的最大流；`s==t` 约定返回 0。`T` 必须容纳总流量，必要时用 `__int128`。最坏 $O(V^2E)$、空间 $O(V+E)$。它与 5.8.001 Dinic 最大流 同解，稠密图上也可比较 5.8.002 FIFO 预流推进 与 5.8.010 HLPP / 最高标号预流推进。带费用需求见 5.8.003 SPFA 多路增广最小费用流。本类没有 `cut(s)`，需要割侧点集时使用 `Flow`。

5.8.010 HLPP / 最高标号预流推进

HLPP（Highest Label Push-Relabel）也是预流推进，但活跃点总按当前最高标号处理。实现用当前弧 `cur`、按标号组织的活跃点桶、GAP 链表和全局重标号：最高标号桶避免 `priority_queue` 的对数开销，GAP 在某个标号层变空时把更高层一次标成不可达，全局重标号则按反向残量边 BFS 重建到汇点的精确距离。`flow(s,t)` 从当前加入的容量网络求一次最大流，`s==t` 约定返回 0；结束时可能仍保留无法到达汇点的预流超额，因此同一对象不得再次调用 `flow`。容量类型 `T` 必须为能表示非负容量和总流量的有符号整数。

```
// 依赖 5.8.001 Dinic 最大流
// 依赖 5.8.002 FIFO 预流推进
// 依赖 5.8.009 ISAP 最大流
template <class T = long long>
struct HLPP
{
    struct E
    {
        // to 是终点，rev 是反向边在 e[to] 中的下标。
        int to, rev;
        // cap 是当前残量容量。
        T cap;
    };

    // n 是点数，src、sink 是最近一次求流使用的源汇点。
    int n, src = -1, sink = -1;
    // e[u] 是点 u 发出的全部残量边。
    vector<vector<E>> e;
    // ex 是超额流，h 是标号，cur 是当前弧。
    vector<T> ex;
    vector<int> h, cur;
    // actHead/actNext 按标号维护活跃点链表，in 标记是否已经入链。
    vector<int> actHead, actNext;
    vector<char> in;
    // gapHead/gapPrev/gapNext 按标号维护全部顶点链表。
```

```
    vector<int> gapHead, gapPrev, gapNext;
    int maxH = -1, work = 0;
    bool used = false; // used 标记是否已经调用过 flow。

    HLPP(int n = 0)
        : n(n), e(n), ex(n), h(n), cur(n), actHead(n, -1), actNext(n, -1), in(n),
          gapHead(n, -1), gapPrev(n, -1), gapNext(n, -1)
    {
        // n 是残量网络点数；构造空网络。
    }

    void add(int u, int v, T c)
    {
        // u、v 是端点，c 是非负容量；加入一对残量边。
        assert(0 <= u && u < n && 0 <= v && v < n && c >= 0); // 调试检查，可删。
        int id = e[u].size();
        int rev = e[v].size() + (u == v);
        e[u].push_back({v, rev, c});
        e[v].push_back({u, id, 0});
    }

    void insertGap(int u)
    {
        // 把标号小于 n 的点 u 插入对应 GAP 链表。
        int d = h[u];
        gapPrev[u] = -1;
        gapNext[u] = gapHead[d];
        if (gapHead[d] != -1)
        {
            gapPrev[gapHead[d]] = u;
        }
        gapHead[d] = u;
    }

    void removeGap(int u)
    {
        // 从当前标号链表移除点 u。
        int d = h[u];
        if (gapPrev[u] == -1)
        {
            gapHead[d] = gapNext[u];
        }
        else
        {
            gapNext[gapPrev[u]] = gapNext[u];
        }
        if (gapNext[u] != -1)
        {
            gapPrev[gapNext[u]] = gapPrev[u];
        }
        gapPrev[u] = gapNext[u] = -1;
    }
```

```
78
79      void enqueue(int u)
80      {
81          // 把有超额且可达汇点的非源汇点加入最高标号活跃链表。
82          if (u == src || u == sink || ex[u] <= 0 || h[u] >= n || in[u])
83          {
84              return;
85          }
86          in[u] = true;
87          actNext[u] = actHead[h[u]];
88          actHead[h[u]] = u;
89          maxH = max(maxH, h[u]);
90      }
91
92      void relabelAll()
93      {
94          // 从汇点按反向残量边 BFS，重建标号、GAP 链表与活跃点链表。
95          fill(h.begin(), h.end(), n);
96          fill(cur.begin(), cur.end(), 0);
97          fill(actHead.begin(), actHead.end(), -1);
98          fill(actNext.begin(), actNext.end(), -1);
99          fill(in.begin(), in.end(), false);
100         fill(gapHead.begin(), gapHead.end(), -1);
101         fill(gapPrev.begin(), gapPrev.end(), -1);
102         fill(gapNext.begin(), gapNext.end(), -1);
103         maxH = -1;
104         queue<int> q;
105         h[sink] = 0;
106         q.push(sink);
107         while (!q.empty())
108         {
109             int u = q.front();
110             q.pop();
111             for (const auto &a : e[u])
112             {
113                 int v = a.to;
114                 if (v ≠ src && h[v] == n && e[v][a.rev].cap > 0)
115                 {
116                     h[v] = h[u] + 1;
117                     q.push(v);
118                 }
119             }
120         }
121         for (int u = 0; u < n; u++)
122         {
123             if (h[u] < n)
124             {
125                 insertGap(u);
126             }
127         }
128         for (int u = 0; u < n; u++)
129         {
```

```
            enqueue(u);
        }
        work = 0;
    }

    void gapRelabel(int old)
    {
        // old 层为空时，所有更高标号点都不可达汇点，统一标为 n。
        for (int d = old + 1; d < n; d++)
        {
            int u = gapHead[d];
            gapHead[d] = -1;
            while (u != -1)
            {
                int v = gapNext[u];
                h[u] = n;
                gapPrev[u] = gapNext[u] = -1;
                if (in[u])
                {
                    in[u] = false;
                    actNext[u] = -1;
                }
                u = v;
            }
            actHead[d] = -1;
        }
        maxH = min(maxH, old - 1);
    }

    void relabel(int u)
    {
        // 当前弧耗尽时把 u 改到残量邻点的最低标号加一。
        int old = h[u], nh = n;
        for (const auto &a : e[u])
        {
            if (a.cap > 0)
            {
                nh = min(nh, h[a.to] + 1);
            }
        }
        removeGap(u);
        bool gap = gapHead[old] == -1;
        if (gap)
        {
            gapRelabel(old);
        }
        h[u] = gap ? n : nh;
        cur[u] = 0;
        if (h[u] < n)
        {
            insertGap(u);
        }
```

```
        work++;
    }

    void push(int u, E &a)
    {
        // 沿允许边从 u 推出尽量多的超额流。
        T f = min(ex[u], a.cap);
        a.cap -= f;
        e[a.to][a.rev].cap += f;
        ex[u] -= f;
        ex[a.to] += f;
        enqueue(a.to);
    }

    void discharge(int u)
    {
        // 反复推流或重标号，直到 u 没有超额或已不可达汇点。
        while (ex[u] > 0 && h[u] < n)
        {
            if (cur[u] == (int)e[u].size())
            {
                relabel(u);
                continue;
            }
            E &a = e[u][cur[u]];
            if (a.cap == 0 || h[u] != h[a.to] + 1)
            {
                cur[u]++;
                continue;
            }
            push(u, a);
        }
    }

    T flow(int s, int t)
    {
        // s、t 是源汇点；首次调用时返回最大流。
        assert(0 <= s && s < n && 0 <= t && t < n); // 调试检查，可删。
        assert(!used); // 调试检查，可删。
        used = true;
        if (s == t)
        {
            return 0;
        }
        src = s;
        sink = t;
        ex.assign(n, 0);
        relabelAll();
        for (auto &a : e[s])
        {
            if (a.to == s || a.cap == 0)
            {
```

```
                continue;
            }
            T f = a.cap;
            a.cap = 0;
            e[a.to][a.rev].cap += f;
            ex[s] -= f;
            ex[a.to] += f;
            enqueue(a.to);
        }
        while (maxH >= 0)
        {
            if (actHead[maxH] == -1)
            {
                maxH--;
                continue;
            }
            int u = actHead[maxH];
            actHead[maxH] = actNext[u];
            actNext[u] = -1;
            in[u] = false;
            discharge(u);
            if (ex[u] > 0 && h[u] < n)
            {
                enqueue(u);
            }
            if (work >= 4 * max(1LL, n))
            {
                relabelAll();
            }
        }
        return ex[t];
    }
};
```

构造 `HLPP<T> hp(n)`，`add(u, v, c)` 加非负容量边，`flow(s, t)` 一次性返回最大流，同一对象不得再次调用；`s==t` 约定返回 0。最高标号预流推进最坏 $O(V^2E)$、空间 $O(V+E)$；数组桶、字节标志、当前弧、GAP 和约每 $4V$ 次重标号进行一次全局重标号，是本版保留的常数优化。它通常优先用于稠密网络或预流推进确有优势的场景；稀疏图、需要流量上限或割侧查询时优先使用 5.8.001 Dinic 最大流，需要 FIFO 调度的短实现使用 5.8.002 FIFO 预流推进，想保留距离标号与 GAP 但不需要活跃桶时使用 5.8.009 ISAP 最大流。HLPP 的全局重标号和 GAP 都维护同一份残量状态，不能把它们删掉后仍称为 HLPP。

下面让 Dinic、FIFO 预流推进、ISAP 与 HLPP 分别检查同一张已知答案为 5 的四点图；它们各自核对固定答案，不相互比较。

```
template <class F>
void testMaxFlow()
{
    // F 是待检查的最大流结构；直接核对一张固定图的答案。
    F f(4);
    f.add(0, 1, 3);
    f.add(0, 2, 2);
```

```
    f.add(1, 2, 1);
    f.add(1, 3, 2);
    f.add(2, 3, 4);
    assert(f.flow(0, 3) == 5);
}

signed main()
{
    testMaxFlow<Flow<long long>>();
    testMaxFlow<PushRelabel<long long>>();
    testMaxFlow<ISAP<long long>>();
    testMaxFlow<HLPP<long long>>();
    return 0;
}
```

5.8.011 费用缩放最小费用 b 流与带费用上下界

把每条边流量限制写成 $l_e \le f_e \le r_e$，并把固定的下界先放进节点超额 `ex`：边 $u \to v$ 的下界使 `ex[u]-=l`、`ex[v]+=l`。费用缩放维护约化费用 $c_{uv}+p_u-p_v$；每一轮先把所有负约化费用残量边饱和，再沿负约化费用边消除正超额，没有可用边时下调该点势能。把费用乘以 $n+1$ 后缩放到 $\varepsilon=1$，任意长度不超过 n 的负环都不可能残留，因此得到最小费用可行 b 流；负费用边和负环不需要额外假设。

```
template <class Cap = long long, class Cost = long long, class Ans = long long>
struct MinCostBFlow
{
    struct Edge
    {
        int to, rev; // to、rev 是终点与反向边下标。
        Cap cap;     // cap 是当前残量容量。
        Cost w;      // w 是乘以 n+1 后的费用。
    };

    struct Ref
    {
        int u, i;    // u、i 定位加入的正向边。
        Cap low;     // low 是原边下界。
        Cost cost;   // cost 是原单位费用。
    };
    struct Certificate
    {
        // value 是最小费用，pot 是满足互补松弛的点势，flow 按 add 顺序给出每条边流量。
        Ans value;
        vector<Cost> pot;
        vector<Cap> flow;
    };

    int n;                      // n 是点数。
    vector<vector<Edge>> g;     // g 是残量网络。
    vector<Cap> ex;             // ex 是当前流入减流出的超额。
    vector<Cost> pot;           // pot 是费用缩放势能。
```

```
    vector<Ref> ref;            // ref 保存原边，便于查询最终流量。

    MinCostBFlow(int n) : n(n), g(n), ex(n), pot(n)
    {
        // n 是点数；初始没有边和节点供给。
        assert(n >= 0); // 调试检查，可删
    }

    void add(int u, int v, Cap low, Cap high, Cost cost)
    {
        // u、v 是端点，low/high 是下界和上界，cost 是单位费用；加入一条带上下界边。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删
        assert(low <= high); // 调试检查，可删
        Cost scale = (Cost)n + 1;
        assert((__int128)cost * scale < numeric_limits<Cost>::max()); // 调试检查，可删
        assert((__int128)cost * scale > numeric_limits<Cost>::min()); // 调试检查，可删
        Cost w = cost * scale;
        int i = g[u].size(), j = g[v].size() + (u == v);
        g[u].push_back({v, j, high - low, w});
        g[v].push_back({u, i, 0, -w});
        ex[u] -= low;
        ex[v] += low;
        ref.push_back({u, i, low, cost});
    }

    void supply(int u, Cap x)
    {
        // u 是点，x 是要求的净流出量；正值表示必须流出，负值表示必须流入。
        assert(0 <= u && u < n); // 调试检查，可删
        ex[u] += x;
    }

    Cost reduced(int u, const Edge &e) const
    {
        // u、e 是残量边起点和边；返回缩放后的约化费用。
        return e.w + pot[u] - pot[e.to];
    }

    void push(int u, int i, Cap f)
    {
        // u、i 定位残量边，f 是沿边发送的流量。
        Edge &e = g[u][i];
        e.cap -= f;
        g[e.to][e.rev].cap += f;
        ex[u] -= f;
        ex[e.to] += f;
    }

    bool refine(Cost eps)
    {
        // eps 是当前费用精度；饱和负边并消除所有正超额。
        vector<int> cur(n);
```

```
        vector<char> in(n);
        deque<int> q;
        auto canDeficit = [&](int s)
        {
            // s 是当前超额点；判断残量图中是否仍有路径到任一缺口点。
            vector<char> vis(n);
            deque<int> z;
            z.push_back(s);
            vis[s] = true;
            while (!z.empty())
            {
                int u = z.front();
                z.pop_front();
                if (ex[u] < 0)
                {
                    return true;
                }
                for (const auto &e : g[u])
                {
                    if (e.cap > 0 && !vis[e.to])
                    {
                        vis[e.to] = true;
                        z.push_back(e.to);
                    }
                }
            }
            return false;
        };
        for (int u = 0; u < n; u++)
        {
            for (int i = 0; i < (int)g[u].size(); i++)
            {
                if (g[u][i].cap > 0 && reduced(u, g[u][i]) < 0)
                {
                    push(u, i, g[u][i].cap);
                }
            }
        }
        for (int u = 0; u < n; u++)
        {
            if (ex[u] > 0)
            {
                q.push_back(u);
                in[u] = true;
            }
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop_front();
            in[u] = false;
            while (ex[u] > 0)
```

```
            {
                if (cur[u] == (int)g[u].size())
                {
                    Cost best = numeric_limits<Cost>::max();
                    for (const auto &e : g[u])
                    {
                        if (e.cap > 0)
                        {
                            best = min(best, reduced(u, e));
                        }
                    }
                    if (best == numeric_limits<Cost>::max())
                    {
                        return false;
                    }
                    if (!canDeficit(u))
                    {
                        return false;
                    }
                    pot[u] -= best + eps;
                    cur[u] = 0;
                    continue;
                }
                Edge &e = g[u][cur[u]];
                if (e.cap == 0 || reduced(u, e) >= 0)
                {
                    cur[u]++;
                    continue;
                }
                int v = e.to;
                Cap f = min(ex[u], e.cap);
                push(u, cur[u], f);
                if (ex[v] > 0 && !in[v])
                {
                    q.push_back(v);
                    in[v] = true;
                }
            }
        }
        return true;
    }

    optional<Ans> solve()
    {
        // 无参数；返回最小费用可行 b 流，不存在可行流时返回空。
        Cap total = 0;
        for (Cap x : ex)
        {
            total += x;
        }
        if (total != 0)
        {
```

```
            return nullopt;
        }
        Cost mx = 1;
        for (int u = 0; u < n; u++)
        {
            for (const auto &e : g[u])
            {
                mx = max(mx, e.w >= 0 ? e.w : -e.w);
            }
        }
        Cost eps = 1;
        while (eps < mx)
        {
            assert(eps <= numeric_limits<Cost>::max() / 2); // 调试检查，可删
            eps *= 2;
        }
        while (eps)
        {
            if (!refine(eps))
            {
                return nullopt;
            }
            eps /= 2;
        }
        for (Cap x : ex)
        {
            if (x != 0)
            {
                return nullopt;
            }
        }
        Ans ans = 0;
        for (const auto &x : ref)
        {
            Cap flow = x.low + g[g[x.u][x.i].to][g[x.u][x.i].rev].cap;
            ans += (Ans)flow * x.cost;
        }
        return ans;
    }

    optional<Certificate> solveCert()
    {
        // 无参数；求最小费用并返回可直接验证的点势和每条原边流量，不可行时返回空。
        auto ans = solve();
        if (!ans) return nullopt;
        vector<Cost> d(n);
        // 最优残量网络无负环；从超级源以零边到各点做 Bellman-Ford，得到可行势。
        for (int it = 0; it + 1 < n; it++)
        {
            bool upd = false;
            for (int u = 0; u < n; u++)
            {
```

```
                for (const auto &x : g[u])
                {
                    if (x.cap > 0 && d[x.to] > d[u] + x.w / ((Cost)n + 1))
                    {
                        d[x.to] = d[u] + x.w / ((Cost)n + 1);
                        upd = true;
                    }
                }
            }
            if (!upd) break;
        }
        vector<Cap> flow;
        flow.reserve(ref.size());
        for (const auto &x : ref)
        {
            const Edge &a = g[x.u][x.i];
            flow.push_back(x.low + g[a.to][a.rev].cap);
        }
        return Certificate{*ans, d, flow};
    }
};
```

`add(u,v,l,r,c)` 加边，`supply(u,x)` 指定净流出量，`solve()` 返回最小费用；`supply(s,F)` 与 `supply(t,-F)` 就是固定流量为 F 的源汇费用流，原有 5.8.005 的无费用最大/最小可行流可先确定 F，再调用本条求对应最小费用。需要提交最优性证书时改用 `solveCert()`，其 `flow` 按 `add` 顺序给出边流量，`pot` 满足残量边的约化费用非负，故可直接验证容量、守恒与互补松弛。容量和总费用须装入模板参数，缩放后的 $(n+1)c$ 也须装入 `Cost`；默认实现用 `long long`，总费用可能超界时把 `Ans` 改为 `__int128`。费用缩放的经典复杂度为 $O(VE\log(VC))$ 的理论上界，证书恢复额外 $O(VE)$ 时间和 $O(V+E)$ 空间；它直接处理负费用环，不要求源点可达子图无负环。

```
signed main()
{
    MinCostBFlow<> g(2);
    g.add(0, 1, 1, 3, 2);
    g.add(1, 0, 0, 3, 1);
    assert(g.solve() == optional<long long>(3));

    MinCostBFlow<> h(2);
    h.add(0, 1, 0, 3, 2);
    h.supply(0, 2);
    h.supply(1, -2);
    assert(h.solve() == optional<long long>(4));
}
```

5.8.012 动态星增广图全局最小割

原图固定，再加入一个新星点并维护它到每个原点的非负边权；每次单点改权后输出全局最小割。先对原图做 Stoer-Wagner 收缩，得到叶子为原点的二叉收缩树。原边和星边在每个收缩树节点上分别表示对应割的贡献；一条星边恰给叶子到根路径全部加同一增量。对收缩树重链剖分后，路径加与全局最小值由懒

线段树维护，因此每次更新只改 $O(\log^2 n)$ 个区间。

```
struct DynStarCut
{
    struct Seg
    {
        int n; // n 是补齐到 2 的幂的叶子数。
        vector<int> mn, tag; // mn 是区间最小值，tag 是整段增量。
        Seg(int n = 0) { init(n); }
        void init(int m)
        {
            // m 是叶子数；把全部叶子初始化为无穷大。
            n = 1;
            while (n < m)
            {
                n <<= 1;
            }
            mn.assign(n * 2, numeric_limits<int>::max() / 4);
            tag.assign(n * 2, 0);
        }
        void add(int l, int r, int x, int u, int a, int b)
        {
            // [l,r) 加 x，u 覆盖 [a,b)。
            if (r <= a || b <= l)
            {
                return;
            }
            if (l <= a && b <= r)
            {
                mn[u] += x;
                tag[u] += x;
                return;
            }
            int m = (a + b) / 2;
            add(l, r, x, u * 2, a, m);
            add(l, r, x, u * 2 + 1, m, b);
            mn[u] = tag[u] + min(mn[u * 2], mn[u * 2 + 1]);
        }
        int point(int p, int u, int a, int b) const
        {
            // 返回位置 p 的当前值，u 覆盖 [a,b)。
            if (b - a == 1)
            {
                return mn[u];
            }
            int m = (a + b) / 2;
            return tag[u] + (p < m ? point(p, u * 2, a, m) : point(p, u * 2 + 1, m, b));
        }
        void add(int l, int r, int x) { add(l, r, x, 1, 0, n); }
        void set(int p, int x) { add(p, p + 1, x - point(p, 1, 0, n)); }
        int allMin() const { return mn[1]; }
    };
```

```

    int n; // n 是点数。
    vector<vector<int>> tree; // Stoer-Wagner 收缩树的子节点。
    vector<int> par, dep, heavy, head, pos, val; // HLD 父、深度、重儿子、链头、位置与星边权。
    Seg seg;

    void buildCutTree(vector<vector<pair<int, int>>> g)
    {
        // g 是原图的非零权邻接表；构造对应的收缩二叉树。
        tree.assign(2 * n - 1, {});
        vector<char> alive(2 * n - 1, true);
        for (int step = 0; step < n - 1; step++)
        {
            g.push_back({});
            vector<int> ord(n - step), cost(n + step);
            vector<char> used(n + step);
            priority_queue<pair<int, int>> q;
            for (int i = 0; i < n + step; i++)
            {
                if (alive[i])
                {
                    for (auto [j, w] : g[i])
                    {
                        cost[i] += w;
                    }
                    q.push({-cost[i], i});
                }
            }
            for (int k = 0; k < n - step; k++)
            {
                while (true)
                {
                    auto [neg, v] = q.top();
                    q.pop();
                    if (used[v])
                    {
                        continue;
                    }
                    used[v] = true;
                    ord[k] = v;
                    for (auto [to, w] : g[v])
                    {
                        if (!used[to])
                        {
                            cost[to] -= w;
                            q.push({-cost[to], to});
                        }
                    }
                    break;
                }
            }
            int x = ord[n - step - 1], y = ord[n - step - 2], z = n + step;
```

```
            g[x].clear();
            g[y].clear();
            alive[x] = alive[y] = false;
            for (int i = 0; i < n + step; i++)
            {
                if (!alive[i])
                {
                    continue;
                }
                for (auto &e : g[i])
                {
                    if (e.first == x || e.first == y)
                    {
                        e.first = z;
                        g[z].push_back({i, e.second});
                    }
                }
            }
            tree[z] = {x, y};
        }
    }

    int dfsSize(int u)
    {
        // u 是收缩树节点；设置父、深度和重儿子并返回子树大小。
        int size = 1, best = 0;
        for (int v : tree[u])
        {
            par[v] = u;
            dep[v] = dep[u] + 1;
            int s = dfsSize(v);
            size += s;
            if (s > best)
            {
                best = s;
                heavy[u] = v;
            }
        }
        return size;
    }

    void buildHld()
    {
        // 无参数；为收缩树建立重链编号并初始化线段树。
        int all = tree.size(), root = all - 1;
        par.assign(all, -1);
        dep.assign(all, 0);
        heavy.assign(all, -1);
        head.assign(all, -1);
        pos.assign(all, -1);
        dfsSize(root);
        queue<int> q;
```

```
        q.push(root);
        int timer = 0;
        while (!q.empty())
        {
            int h = q.front();
            q.pop();
            for (int u = h; u != -1; u = heavy[u])
            {
                head[u] = h;
                pos[u] = timer++;
                for (int v : tree[u])
                {
                    if (v != heavy[u])
                    {
                        q.push(v);
                    }
                }
            }
        }
        seg.init(all);
    }

    void addRootPath(int u, int x)
    {
        // u 是收缩树节点，x 是增量；把根到 u 的节点权整体加 x。
        while (u != -1)
        {
            seg.add(pos[head[u]], pos[u] + 1, x);
            u = par[head[u]];
        }
    }

    DynStarCut(int n, const vector<array<int, 3>> &ed, vector<int> a) : n(n), val(move(a))
    {
        // n 是原图点数，ed 每项为 {u,v,w}，val 是新星点到各原点的初始边权。
        assert(n >= 1 && (int)val.size() == n);
        vector<vector<pair<int, int>>> g(n);
        for (auto [u, v, w] : ed)
        {
            assert(0 <= u && u < n && 0 <= v && v < n && w >= 0);
            if (w)
            {
                g[u].push_back({v, w});
                g[v].push_back({u, w});
            }
        }
        buildCutTree(move(g));
        buildHld();
        vector<int> base(tree.size());
        for (auto [u, v, w] : ed)
        {
            if (w == 0)
```

```
            {
                continue;
            }
            while (u ≠ v)
            {
                if (dep[u] < dep[v])
                {
                    swap(u, v);
                }
                base[u] += w;
                u = par[u];
            }
        }
        for (int u = 0; u < (int)tree.size(); u++)
        {
            seg.set(pos[u], base[u]);
        }
        for (int u = 0; u < n; u++)
        {
            addRootPath(u, val[u]);
        }
    }

    int update(int u, int x)
    {
        // u 是被修改的原图点，x 是新星边权；更新后返回增广图的全局最小割值。
        assert(0 <= u && u < n && x >= 0);
        addRootPath(u, x - val[u]);
        val[u] = x;
        return seg.allMin();
    }
};
```

`DynStarCut(n,ed,val)` 中 `ed` 的每项为 `{u,v,w}`、`w\ge0`，`val[u]` 是星点到 `u` 的初始边权；`update(u,x)` 更新后返回最小割。预处理时间 $O(n^3+mn)$，额外空间 $O(n+m)$；单次更新 $O(\log^2 n)$，适用 $n,m\le 4000$、$Q\le 2\times 10^5$。静态图只求一次割值时 5.8.007 Stoer-Wagner 全局最小割 更短；本条的收缩树专门服务固定原图上的星边动态更新。

5.8.013 CSR 优化原始对偶最小费用流

本版仍是逐路增广的原始对偶法，但把运行期残量图一次性整理成 CSR：点 u 的边连续存放在 `[start[u],start[u+1])`。对残量边 (u,v) 使用约化费用 $c_{uv}+h_u-h_v$；正容量原边费用全非负时，零势能已经可行，直接跳过初始化 SPFA；否则只从源点运行一次 SPFA。每轮 Dijkstra 把约化费用为零的松弛放入数组栈 `zero`，只有正约化费用候选进入最小堆，并在汇点出堆时提前停止。更新已确定点的势能后，所有可达残量边仍保持非负约化费用。

性能版只把点号和边下标固定为 32 位有符号整数，以缩小 CSR 边和堆元素；容量、费用与返回值仍由 `Cap`、`Cost` 控制。打印版按全书约定使用普通 `int`，算法、分支与接口不变。

```cpp
template <class Cap = long long, class Cost = long long>
struct FastCostFlow
{
    struct Raw
    {
        // from、to 是端点；性能版固定为 32 位，打印版使用普通 int。
        int from, to;
        Cap cap;   // 原边容量。
        Cost cost; // 原边单位费用。
    };
    struct E
    {
        // to 是终点，rev 是反向边全局下标；性能版固定为 32 位，打印版使用普通 int。
        int to, rev;
        Cap cap;   // 当前残量容量。
        Cost cost; // 单位费用。
    };
    struct Q
    {
        Cost d;    // 当前约化距离。
        int u; // u 是终点；性能版固定为 32 位以缩小堆元素。
        bool operator<(const Q &o) const { return d > o.d; }
    };

    // n 与端点在性能版固定为 32 位；容量、费用及答案仍由模板参数控制。
    int n, source = -1, sink = -1;
    vector<Raw> raw;        // 首次求流前暂存的原边。
    vector<int> start; // CSR 中点 u 的边区间为 [start[u],start[u+1])。
    vector<E> e;            // 按起点连续排列的残量边。
    vector<Cost> h, d;      // h 是势能，d 是本轮约化距离。
    vector<int> pre;   // pre[v] 是最短路进入 v 的残量边下标。
    vector<int> zero;  // 尚未确定的零约化距离点。
    vector<Q> heap;         // 正约化距离候选的最小堆。
    vector<char> vis, in;  // vis 是已确定/已到达标记，in 是 SPFA 入队标记。
    bool built = false;     // 是否已经构造 CSR 残量图。
    bool neg = false;       // 是否存在正容量负费用原边。

    FastCostFlow(int n = 0) : n(n)
    {
        // n 是点数；构造一张空网络。
        assert(n >= 0); // 调试检查，可删。
        static_assert(numeric_limits<Cap>::is_signed);
        static_assert(numeric_limits<Cost>::is_signed);
    }

    void add(int u, int v, Cap cap, Cost cost)
    {
        // u、v 是端点，cap 是非负容量，cost 是单位费用；加入一条原边。
        assert(!built); // 调试检查，可删：首次 flow 后不能继续加边。
        assert(0 <= u && u < n && 0 <= v && v < n && cap >= 0); // 调试检查，可删。
        assert(cost != numeric_limits<Cost>::min()); // 调试检查，可删。
        raw.push_back({u, v, cap, cost});
```

```
        neg |= cap > 0 && cost < 0;
    }

    void build()
    {
        // 无参数；把原边一次性展开成按起点连续存储的 CSR 残量图。
        assert(raw.size() <= (size_t)numeric_limits<int>::max() / 2); // 调试检查，可删。
        start.assign(n + 1, 0);
        for (const auto &a : raw)
        {
            start[a.from + 1]++;
            start[a.to + 1]++;
        }
        for (int u = 0; u < n; u++)
        {
            start[u + 1] += start[u];
        }
        vector<int> ptr = start;
        e.resize(raw.size() * 2);
        for (const auto &a : raw)
        {
            int x = ptr[a.from]++;
            int y = ptr[a.to]++;
            e[x] = {a.to, y, a.cap, a.cost};
            e[y] = {a.from, x, 0, -a.cost};
        }
        vector<Raw>().swap(raw);
        h.assign(n, 0);
        d.resize(n);
        pre.resize(n);
        vis.resize(n);
        in.resize(n);
        zero.reserve(n);
        heap.reserve(e.size());
        built = true;
    }

    void init(int s)
    {
        // s 是源点；有负费用边时用一次 SPFA 建立可行势能，否则保留零势能。
        fill(h.begin(), h.end(), 0);
        if (!neg)
        {
            return;
        }
        fill(vis.begin(), vis.end(), 0);
        fill(in.begin(), in.end(), 0);
        queue<int> q;
        d[s] = 0;
        vis[s] = in[s] = 1;
        q.push(s);
        while (!q.empty())
```

```
        {
            int u = q.front();
            q.pop();
            in[u] = 0;
            for (int i = start[u]; i < start[u + 1]; i++)
            {
                const E &a = e[i];
                if (a.cap == 0)
                {
                    continue;
                }
                Cost nd = d[u] + a.cost;
                if (!vis[a.to] || nd < d[a.to])
                {
                    d[a.to] = nd;
                    vis[a.to] = 1;
                    if (!in[a.to])
                    {
                        in[a.to] = 1;
                        q.push(a.to);
                    }
                }
            }
        }
        for (int u = 0; u < n; u++)
        {
            if (vis[u])
            {
                h[u] = d[u];
            }
        }
    }

    bool dualRef(int s, int t)
    {
        // s、t 是固定源汇；求一条最短增广路并更新势能，返回 t 是否可达。
        const Cost inf = numeric_limits<Cost>::max();
        fill(d.begin(), d.end(), inf);
        fill(vis.begin(), vis.end(), 0);
        zero.clear();
        heap.clear();
        d[s] = 0;
        zero.push_back(s);
        size_t heapSize = 0;
        while (!zero.empty() || !heap.empty())
        {
            int u;
            if (!zero.empty())
            {
                u = zero.back();
                zero.pop_back();
            }
```

```
            else
            {
                while (heapSize < heap.size())
                {
                    heapSize++;
                    push_heap(heap.begin(), heap.begin() + heapSize);
                }
                u = heap.front().u;
                pop_heap(heap.begin(), heap.end());
                heap.pop_back();
                heapSize--;
            }
            if (vis[u])
            {
                continue;
            }
            vis[u] = 1;
            if (u == t)
            {
                break;
            }
            Cost du = d[u];
            for (int i = start[u]; i < start[u + 1]; i++)
            {
                const E &a = e[i];
                if (a.cap == 0)
                {
                    continue;
                }
                Cost w = a.cost + h[u] - h[a.to];
                assert(w >= 0); // 调试检查，可删：势能必须使可达残量边约化费用非负。
                Cost nd = du + w;
                if (nd < d[a.to])
                {
                    d[a.to] = nd;
                    pre[a.to] = i;
                    if (nd == du)
                    {
                        zero.push_back(a.to);
                    }
                    else
                    {
                        heap.push_back({nd, a.to});
                    }
                }
            }
        }
        if (!vis[t])
        {
            return false;
        }
        for (int u = 0; u < n; u++)
```

```
        {
            if (vis[u])
            {
                h[u] -= d[t] - d[u];
            }
        }
        return true;
    }

    pair<Cap, Cost> flow(int s, int t,
                         Cap lim = numeric_limits<Cap>::max())
    {
        // s、t 是固定源汇，lim 是本次最多发送的流量；返回本次新增流量与最小费用。
        assert(0 <= s && s < n && 0 <= t && t < n && lim >= 0); // 调试检查，可删。
        if (s == t)
        {
            return {0, 0};
        }
        if (source == -1)
        {
            source = s;
            sink = t;
            build();
            init(s);
        }
        assert(source == s && sink == t); // 调试检查，可删：分段调用不能更换源汇。
        Cap f = 0;
        Cost cost = 0;
        while (f < lim && dualRef(s, t))
        {
            Cap x = lim - f;
            for (int v = t; v != s; v = e[e[pre[v]].rev].to)
            {
                x = min(x, e[pre[v]].cap);
            }
            for (int v = t; v != s; v = e[e[pre[v]].rev].to)
            {
                E &a = e[pre[v]];
                a.cap -= x;
                e[a.rev].cap += x;
            }
            f += x;
            cost += (Cost)x * (h[t] - h[s]);
        }
        return {f, cost};
    }
};
```

构造 `FastCostFlow<Cap,Cost> g(n)` 后，先用 `add(u,v,cap,cost)` 加完全部边，再调用 `flow(s,t,lim)`；点编号为 $0 \dots n-1$，默认容量和费用均为 `long long`。返回值是本次新增的 `{流量，费用}`；相同源汇可以分段继续增广，`s==t` 返回 `{0,0}` 且不锁定源汇。首次有效 `flow` 会构造 CSR，此后不能继续加边或更换源汇。调用者必须保证边数可由性能版 32 位下标容纳，且反向费用、路径费用、势能、`Cap` 到 `Cost` 的转

换、单次乘积和总费用均不溢出；单位费用不能等于 `numeric_limits<Cost>::min()`。

算法允许负费用边，但要求残量网络中不存在从源点可达的负费用环；需要处理负环及上下界时改用 5.8.011 费用缩放最小费用 b 流。设增广次数为 A，整数容量下 $A \le F$，其中 F 是发送的总流量。若存在正容量负费用原边，初始化 SPFA 最坏 $O(VE)$；否则没有这一步。每轮最短路最坏 $O(E \log E + V)$，建图 $O(V + E)$，额外空间 $O(V + E)$。

这版适合费用较分散、增广轮数难以批量合并的图；大量同费用最短路并存时，5.8.003 SPFA 多路增广 可在一轮 DFS 中推送多条路径，通常更快。5.8.004 普通原始对偶 使用嵌套邻接表和标准优先队列，代码更直观，适合不需要极致常数的场合。完备二分图指派仍优先使用 5.6.005 二分图最大权 / 最小权匹配。

本地性能检查使用 Windows 10、Ryzen 9 7945HX、GCC 14.2.0 与 `-O2 -pipe`，每个场景预热一次后取 5 或 7 次中位数，并重复整个进程三轮。120×120 随机费用指派图上，本版为 2.273–2.330 ms，5.8.003 为 7.377–7.679 ms，5.8.004 为 8.185–8.521 ms；全零费用指派图上三者分别为 1.682–1.723 ms、0.495–0.523 ms、4.658–4.855 ms。宽度 500、每点 8 条等费用边的分层图上，本版为 0.717–0.798 ms，5.8.003 为 0.321–0.336 ms。结果包含构造和求流时间，只说明这些固定分布下的常数取舍；

下面检查一张含负费用边的小图，并验证同源汇分段调用。

```
signed main()
{
    FastCostFlow<> g(4);
    g.add(0, 1, 2, 1);
    g.add(0, 2, 1, 4);
    g.add(1, 2, 1, -2);
    g.add(1, 3, 1, 3);
    g.add(2, 3, 2, 1);
    assert((g.flow(0, 3, 1) == pair<long long, long long>{1, 0}));
    assert((g.flow(0, 3) == pair<long long, long long>{2, 9}));
    return 0;
}
```

5.8.014 有向图全局最小割

有向图的全局最小割是所有非空真子集 $S \subset V$ 上 $c(S, V \setminus S)$ 的最小值，其中 $c(S, T)$ 是从 S 指向 T 的边容量之和。任取一个非平凡割，点 0 要么在 S 中而某个 v 不在，要么不在 S 中而某个 v 在；前者至少是 $0 \to v$ 的最小割，后者至少是 $v \to 0$ 的最小割。所以只要对每个 $v \neq 0$ 求两个方向的最大流，取最小值。

实现直接复用 5.8.001 Dinic 最大流 的 `Flow<T>`，容量类型取 `__int128` 以免容量和溢出；残量网络上从源点可达的点就是最小割的源侧。

```
// 依赖 5.8.001 Dinic 最大流
pair<__int128, vector<bool>> dirMinCut(int n, const vector<array<int, 3>> &e)
{
    // n>=2 是点数，e[i][0..2] 依次是有向边起点、终点和非负容量；
    // 返回有向图全局最小割的容量，以及一组割的源侧点集 side（side[v] 为真表示 v 在源侧）。
    assert(n >= 2); // 调试检查，可删。
    __int128 best = (__int128)1 << 120;
```

```
    vector<bool> side;
    // 任一非平凡割都把点 0 与某点 v 分开：枚举 v 和两个方向各求一次最大流即可覆盖。
    for (int v = 1; v < n; v++)
    {
        for (int dir = 0; dir < 2; dir++)
        {
            Flow<__int128> f(n);
            for (auto [x, y, c] : e)
            {
                f.add(x, y, c);
            }
            int s = dir ? v : 0, t = dir ? 0 : v;
            __int128 cur = f.flow(s, t);
            if (cur < best)
            {
                best = cur;
                side = f.cut(s);
            }
        }
    }
    return {best, side};
}
```

`dirMinCut(n,e)` 要求 $n \geq 2$，`e[i]` 依次是起点、终点和非负容量；返回最小割容量及源侧点集 `side`（`side[v]` 为真表示 v 在源侧），割边是从源侧指向另一侧的边。总共 $2(n-1)$ 次最大流，时间为 Dinic 单次 $O(n^2m)$ 上界的 $2(n-1)$ 倍，空间 $O(n+m)$。

无向图的全局最小割用 5.8.007 Stoer-Wagner 全局最小割，所有点对的最小割用 5.8.008 Gomory-Hu 最小割树。Hao-Orlin 算法能在一次最大流的时间内求出有向全局最小割，但代码长得多，这里没有实现。下面的三点图中把 $\{0,1\}$ 与 $\{2\}$ 分开只需切断 $1 \to 2$：

```
signed main()
{
    auto [c, side] = dirMinCut(3, {{0, 1, 5}, {1, 2, 3}, {2, 0, 4}, {1, 0, 1}});
    assert(c == 3 && side == vector<bool>({1, 1, 0}));
}
```

5.9 欧拉图与环

5.9.001 有向/无向欧拉路径与欧拉回路

欧拉迹是恰好经过每条边一次的路径，首尾相同则是欧拉回路。存在性判定很干净：无向图要求所有非零度点连通，且奇度点数为 0（有回路）或 2（有迹，起点必为某个奇度点）；有向图要求非零度点在忽略方向后连通，且每点入度等于出度（回路），或恰有一点出度比入度多 1（作起点）、一点入度比出度多 1（作终点）。构造用 Hierholzer：维护当前游走的语义栈，沿尚未用过的边走到底，再按弹栈顺序记录边和点，最后反转得到欧拉迹；无向图额外用 `used[]` 标记边是否已走。最后检查回溯出的点数等于 $m+1$，否则说明图不连通、无欧拉迹，返回空。`lex=true` 时先给每个邻接表按目标点排序，得到字典序最小的迹。

```cpp
struct EulerTrail
{
    // 依次经过的点与原边编号。
    vector<int> v, id;
};

optional<EulerTrail> dirEuler(int n, const vector<pair<int, int>> &edges, bool lex = false)
{
    // n 是点数，edges 是有向边，lex 控制是否取最小点序列；无欧拉迹时返回 nullopt。
    if (!n)
    {
        return edges.empty() ? optional<EulerTrail>(EulerTrail{}) : nullopt;
    }
    int m = edges.size(), s = m ? edges[0].first : 0, plus = 0, minus = 0;
    vector<int> in(n), out(n);
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < m; i++)
    {
        auto [u, v] = edges[i];
        g[u].push_back({v, i});
        out[u]++;
        in[v]++;
    }
    for (int u = 0; u < n; u++)
    {
        if (out[u] - in[u] == 1)
        {
            s = u;
            plus++;
        }
        else if (in[u] - out[u] == 1)
        {
            minus++;
        }
        else if (in[u] != out[u])
        {
            return nullopt;
        }
    }
    if (!((plus == 1 && minus == 1) || (!plus && !minus)))
    {
        return nullopt;
    }
    if (!plus)
    {
        for (int u = 0; u < n; u++)
        {
            if (out[u])
            {
                s = u;
                break;
            }
```

```
        }
    }
    if (lex)
    {
        for (auto &a : g)
        {
            sort(a.begin(), a.end());
        }
    }
    vector<int> it(n), rv, re, st{s}, enter{-1};
    while (!st.empty())
    {
        int u = st.back();
        if (it[u] < (int)g[u].size())
        {
            auto [v, id] = g[u][it[u]++];
            st.push_back(v);
            enter.push_back(id);
        }
        else
        {
            rv.push_back(u);
            if (enter.back() != -1) re.push_back(enter.back());
            st.pop_back();
            enter.pop_back();
        }
    }
    if ((int)rv.size() != m + 1)
    {
        return nullopt;
    }
    reverse(rv.begin(), rv.end());
    reverse(re.begin(), re.end());
    return EulerTrail{rv, re};
}

optional<EulerTrail> undirEuler(int n, const vector<pair<int, int>> &edges, bool lex = false)
{
    // n 是点数，edges 是无向边，lex 控制是否取最小点序列；无欧拉迹时返回 nullopt。
    if (!n)
    {
        return edges.empty() ? optional<EulerTrail>(EulerTrail{}) : nullopt;
    }
    int m = edges.size(), s = 0;
    vector<vector<pair<int, int>>> g(n);
    vector<int> deg(n);
    for (int i = 0; i < m; i++)
    {
        auto [u, v] = edges[i];
        g[u].push_back({v, i});
        g[v].push_back({u, i});
        deg[u]++;
```

```
        deg[v]++;
    }
    vector<int> odd;
    for (int u = 0; u < n; u++)
    {
        if (deg[u] & 1)
        {
            odd.push_back(u);
        }
    }
    if (odd.size() != 0 && odd.size() != 2)
    {
        return nullopt;
    }
    if (odd.size() == 2)
    {
        s = odd[0];
    }
    else
    {
        for (int u = 0; u < n; u++)
        {
            if (deg[u])
            {
                s = u;
                break;
            }
        }
    }
    if (lex)
    {
        for (auto &a : g)
        {
            sort(a.begin(), a.end());
        }
    }
    vector<int> it(n), rv, re, st{s}, enter{-1};
    vector<bool> used(m);
    while (!st.empty())
    {
        int u = st.back();
        while (it[u] < (int)g[u].size() && used[g[u][it[u]].second])
        {
            it[u]++;
        }
        if (it[u] < (int)g[u].size())
        {
            auto [v, id] = g[u][it[u]++];
            used[id] = true;
            st.push_back(v);
            enter.push_back(id);
        }
```

```
        else
        {
            rv.push_back(u);
            if (enter.back() != -1) re.push_back(enter.back());
            st.pop_back();
            enter.pop_back();
        }
    }
    if ((int)rv.size() != m + 1)
    {
        return nullopt;
    }
    reverse(rv.begin(), rv.end());
    reverse(re.begin(), re.end());
    return EulerTrail{rv, re};
}
```

`dirEuler(n, edges, lex)` 与 `undirEuler(n, edges, lex)` 返回 `optional<EulerTrail>`；边的数组下标就是原编号。无边时，$n > 0$ 返回只含点 0 的空迹，$n = 0$ 返回空点列。`lex=true` 时，开路径从度数强制的起点出发，回路从最小的非零度点出发，再按目标点升序取边，因而比较的是所有合法起点与走法中的最小点序列。排序前 $O(n+m)$，排序后 $O(n+m\log m)$，空间 $O(n+m)$。连通性由最终是否恰好回溯 $m+1$ 个点统一检查。5.9.002 直接复用本接口；计数见 5.9.003 BEST 定理与有向欧拉回路计数。

下面依次检查有向回路、有向开路径、无向奇度路径、自环与平行边、度数合法但不连通的图，以及字典序最小点序列。

```
signed main()
{
    vector<pair<int, int>> circuit = {{0, 1}, {1, 2}, {2, 0}};
    auto a = dirEuler(3, circuit);
    assert(a && a->v == vector<int>({0, 1, 2, 0}) && a->id == vector<int>({0, 1, 2}));

    vector<pair<int, int>> open = {{0, 1}, {1, 2}, {2, 1}};
    auto b = dirEuler(3, open);
    assert(b && b->v == vector<int>({0, 1, 2, 1}) && b->id == vector<int>({0, 1, 2}));

    vector<pair<int, int>> undirected = {{0, 0}, {0, 1}, {0, 1}, {1, 2}};
    auto c = undirEuler(3, undirected);
    assert(c && c->v == vector<int>({1, 0, 0, 1, 2}) && c->id == vector<int>({1, 0, 2, 3}));

    assert(!dirEuler(4, {{0, 1}, {1, 0}, {2, 3}, {3, 2}}).has_value());
    assert(!undirEuler(6, {{0, 1}, {1, 2}, {2, 0}, {3, 4}, {4, 5}, {5, 3}}).has_value());

    auto lex = dirEuler(3, {{0, 2}, {0, 1}, {1, 0}, {2, 0}}, true);
    assert(lex.has_value() && lex->v == vector<int>({0, 1, 0, 2, 0}));
    puts("OK"); // OK
    return 0;
}
```

5.9.002 字典序最小欧拉路径

求字典序最小的欧拉迹（点序列尽量小）用的正是 5.9.001 有向/无向欧拉路径与欧拉回路 里的 Hierholzer-lex：把每个邻接表按目标点升序排序，再跑迭代 Hierholzer，回溯后反转即得字典序最小的迹。因此本条不另立实现，直接调用 `dirEuler(n, edges, true)`（无向图用 `undirEuler(n, edges, true)`），返回的 `.v` 就是字典序最小的点序列、`.id` 是对应边序列。举例：有向边 $0\to2, 0\to1, 1\to0, 2\to0$ 的字典序最小欧拉回路点序列为 $0,1,0,2,0$。排序 $O(m\log m)$、Hierholzer $O(n+m)$，空间 $O(n+m)$。平行边终点相同时按原边编号先后消歧。只求任意一条迹（不要求字典序）省掉排序即可；统计回路数量而非构造用 5.9.003 BEST 定理与有向欧拉回路计数。

5.9.003 BEST 定理与有向欧拉回路计数

数有向欧拉图的欧拉回路条数是 BEST 定理的经典应用。前提是图欧拉：每点入度等于出度且所有非零度点连通。此时以任意根 r 固定一条首出边后，从 r 出发的欧拉回路数为 $t_r\prod_v(\deg^+(v)-1)!$，其中 t_r 是「所有边都朝向根 r」的内向生成树（arborescence）数目——BEST 定理保证这个乘积与根的选择无关。内向树形图计数正是把每条边反向后跑 8.2.011 Matrix-Tree / Kirchhoff 生成树计数 的 `arboCount`（对反向图求以 r 为根的外向树形图，即原图的内向树形图）。若只固定起点而不固定首边，还要乘 $\deg^+(r)$，因为每条首出边各对应一族回路。度数不平衡、非零度点不连通，或有边而 `root` 为孤立点时返回 0；与边无关的额外孤立点不影响计数。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 8.2.011 Matrix-Tree / Kirchhoff 生成树计数
Z bestTheorem(int n, int root, const vector<pair<int, int>> &e, bool fixFirst = false)
{
    // n 是点数，root 是起点，e 是有编号有向边；fixFirst 表示是否固定首边。
    if (n == 0)
    {
        return e.empty() ? Z(1) : Z(0);
    }
    assert(0 <= root && root < n); // 调试检查，可删。
    vector<int> out(n), in(n);
    for (auto [u, v] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        out[u]++;
        in[v]++;
    }
    for (int u = 0; u < n; u++)
    {
        if (out[u] != in[u])
        {
            return 0;
        }
    }
    if (e.empty())
    {
        return 1;
    }
```

```
    if (out[root] == 0)
    {
        return 0;
    }

    vector<int> id(n, -1), active;
    for (int u = 0; u < n; u++)
    {
        if (out[u] > 0)
        {
            id[u] = active.size();
            active.push_back(u);
        }
    }
    vector<pair<int, int>> rev;
    for (auto [u, v] : e)
    {
        rev.push_back({id[v], id[u]});
    }
    Z ans = arboCount(active.size(), id[root], rev);
    for (int u : active)
    {
        for (int k = 1; k < out[u]; k++)
        {
            ans *= k;
        }
    }
    if (!fixFirst)
    {
        ans *= out[root];
    }
    return ans;
}
```

调用前先备好 8.2.011 Matrix-Tree / Kirchhoff 生成树计数 的 `arboCount`。`bestTheorem(n, root, e, false)` 统计从 `root` 出发、首边任取的有编号边序列；若已指定 `root` 的某条首边，末参传 `true`。只有非零度点参与 Matrix-Tree，额外孤立点不会把答案误判成 0；有边而 `root` 孤立时才返回 0。全图无边约定空回路计数为 1。两点间各有两条双向平行边时，固定一条首边答案为 2，只固定起点答案为 4。设非零度点数为 k，时间 $O(k^3+m)$、空间 $O(k^2+m)$。

5.9.004 无向图最小环

非负权简单无向图的最小环可用 Floyd 变形求出。依次把顶点 k 加入中转点集合；在用 k 松弛前，$d_{i,j}$ 只允许经过编号小于 k 的内部点，因此对每个 $i<j<k$，路径 $i\rightsquigarrow j$ 加直连边 $i-k$、$k-j$ 就枚举了一个以 k 为最大编号点的简单环。

实现直接复用 5.2.004 Floyd 与最小环 的 `minCycle(a)`，不再维护第二份类型接口。输入是对称非负权矩阵，空值表示无边，无环返回空；调用者必须保证路径和环权可由矩阵的元素类型表示。算法要求简单图，平行边形成的二边环不属于该接口。稠密图 $O(n^3)$ 时间、$O(n^2)$ 空间；固定长度短环见 5.9.006 三元环、

四元环计数，全部简单环见 5.9.007 简单环存在性、输出与计数。

5.9.005 最小平均环 / 最小密度环

最小平均环求有向图中「环权和 / 环边数」的最小值，Karp 定理给出优雅的 $O(nm)$ 解法。设 $F_k(v)$ 是从某个虚拟源出发、恰好走 k 条边到达 v 的最小权（初始 $F_0(\cdot)=0$，逐层用所有边松弛），则最小平均环 $\mu^* = \min_v \max_{0\le k\le n-1} \dfrac{F_n(v)-F_k(v)}{n-k}$。直觉是：对最优环上任一点，$F_n$ 相对某个较短层 F_k 的「差商」恰好被环的平均权卡住，外层取 $\min$、内层取 $\max$ 正好抓出这个界。最小密度环（比率环）则求环上 $\sum a / \sum b$ 的最小值（要求每条边代价 $b>0$）：二分候选比率 x，把边权改成 $a-xb$ 判是否存在负环，可行则收紧上界。这是参数二分，不是按当前比率迭代更新的 Dinkelbach 算法。

```cpp
struct MeanEdge
{
    // 端点与边权。
    int u, v;
    long double w;
};

optional<long double> minMeanCycle(int n, const vector<MeanEdge> &e)
{
    // n 是点数，e 是有向边；返回最小平均环权，无环返回空。
    vector f(n + 1, vector<optional<long double>>(n));
    for (int v = 0; v < n; v++)
    {
        f[0][v] = 0;
    }
    for (int k = 1; k <= n; k++)
    {
        for (auto [u, v, w] : e)
        {
            assert(0 <= u && u < n && 0 <= v && v < n && isfinite(w)); // 调试检查，可删。
            if (f[k - 1][u].has_value())
            {
                long double z = *f[k - 1][u] + w;
                if (!f[k][v].has_value() || z < *f[k][v])
                {
                    f[k][v] = z;
                }
            }
        }
    }
    optional<long double> ans;
    for (int v = 0; v < n; v++)
    {
        if (!f[n][v].has_value())
        {
            continue;
        }
        optional<long double> cur;
```

```
        for (int k = 0; k < n; k++)
        {
            if (f[k][v].has_value())
            {
                long double z = (*f[n][v] - *f[k][v]) / (n - k);
                if (!cur.has_value() || z > *cur)
                {
                    cur = z;
                }
            }
        }
        if (cur.has_value() && (!ans.has_value() || *cur < *ans))
        {
            ans = *cur;
        }
    }
    return ans;
}

struct RatioEdge
{
    // 端点、收益与严格为正的代价。
    int u, v;
    long double a, b;
};

// b 必须严格为正；求环上 sum(a)/sum(b) 的最小值。
optional<long double> minRatCycle(int n, const vector<RatioEdge> &e, int it = 100)
{
    // n 是点数，e 是有向边，it 是二分轮数；返回最小环比率，无环返回空。
    if (e.empty())
    {
        return nullopt;
    }
    assert(it >= 0); // 调试检查，可删。
    vector<vector<int>> g(n);
    vector<int> deg(n);
    long double lo = 0, hi = 0;
    bool first = true;
    for (auto [u, v, a, b] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n &&
               isfinite(a) && isfinite(b) && b > 0); // 调试检查，可删。
        g[u].push_back(v);
        deg[v]++;
        long double z = a / b;
        if (first)
        {
            lo = hi = z;
            first = false;
        }
        else
```

```
        {
            lo = min(lo, z);
            hi = max(hi, z);
        }
    }
    queue<int> q;
    for (int u = 0; u < n; u++)
    {
        if (deg[u] == 0)
        {
            q.push(u);
        }
    }
    int seen = 0;
    while (!q.empty())
    {
        int u = q.front();
        q.pop();
        seen++;
        for (int v : g[u])
        {
            if (--deg[v] == 0)
            {
                q.push(v);
            }
        }
    }
    if (seen == n)
    {
        return nullopt;
    }
    auto has = [&](long double x)
    {
        vector<long double> d(n);
        for (int k = 0; k < n; k++)
        {
            bool any = false;
            for (auto [u, v, a, b] : e)
            {
                if (d[v] > d[u] + a - x * b)
                {
                    d[v] = d[u] + a - x * b;
                    any = true;
                }
            }
            if (!any)
            {
                return false;
            }
            if (k == n - 1)
            {
                return true;
```

```
            }
        }
        return false;
    };
    for (int z = 0; z < it; z++)
    {
        long double m = midpoint(lo, hi);
        if (has(m))
        {
            hi = m;
        }
        else
        {
            lo = m;
        }
    }
    return hi;
}
```

点从 0 编号。`minMeanCycle` 用 Karp 层数 DP 返回最小平均权；不可达状态与合法浮点值分开保存。`minRatCycle` 要求每条边 `b>0`，先用拓扑删除准确判定是否存在有向环，再在全部单边比率的最小值与最大值之间二分；中点用 `midpoint`，不会因同号大数相加溢出，也不依赖可能满足 `hi+1==hi` 的探测值。两函数都返回 `nullopt` 表示无环。Karp 算法 $O(nm)$ 时间、$O(n^2)$ 空间；比率环为 $O(it\,nm)$ 时间、$O(n+m)$ 空间。浮点输入与中间运算需保持有限，答案按题目误差要求比较。

5.9.006 三元环、四元环计数

三元环计数的经典技巧是「按度数定向」：把每条无向边定向为从度数小的端点指向度数大的（度数相同时按编号），得到 DAG。这样每个点的出度不超过 $O(\sqrt{m})$，枚举 $u \to v \to w$ 再检查 u, w 是否有边（`vis` 标记 u 的出邻居）即可，每个三角形在其定向后唯一的「源点」处被恰好数一次，总量 $O(m\sqrt{m})$。四元环则换一种思路：环 $a\text{—}b\text{—}c\text{—}d\text{—}a$ 的两条对角点对 (a,c)、(b,d) 各是「一对拥有 ≥ 2 个公共邻居」的点对；对每对顶点 $u < v$ 用 bitset 求公共邻居数 c，累加 $\binom{c}{2}$，每个四元环被两条对角点对各数一次，故最后除以 2。

```
// 依赖 1.1.008 静态取模类
// 依赖 7.1.001 快速幂
unsigned long long triangles(int n, const vector<pair<int, int>> &e)
{
    // e 是简单无向图边集；返回三元环数量。
    vector<int> d(n);
    for (auto [u, v] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v); // 调试检查，可删。
        d[u]++;
        d[v]++;
    }
    vector<vector<int>> g(n);
    for (auto [u, v] : e)
    {
```

```cpp
        if (pair{d[u], u} > pair{d[v], v})
        {
            swap(u, v);
        }
        g[u].push_back(v);
    }
    vector<int> vis(n);
    unsigned long long ans = 0;
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u])
        {
            vis[v] = 1;
        }
        for (int v : g[u])
        {
            for (int w : g[v])
            {
                ans += vis[w];
            }
        }
        for (int v : g[u])
        {
            vis[v] = 0;
        }
    }
    return ans;
}

long long weightedTri(int n, const vector<long long> &val, const vector<pair<int, int>> &e,
 ↪ long long mod)
{
    // n 是点数，val 是每点权值，e 是简单无向边集，mod 是正模数；返回所有三元环三点权乘积之和模 mod。
    assert((int)val.size() == n && mod > 0); // 调试检查，可删。
    vector<int> d(n);
    for (auto [u, v] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u ≠ v); // 调试检查，可删。
        d[u]++;
        d[v]++;
    }
    vector<vector<int>> g(n);
    for (auto [u, v] : e)
    {
        if (pair{d[u], u} > pair{d[v], v})
        {
            swap(u, v);
        }
        g[u].push_back(v);
    }
    vector<int> mark(n, -1);
    long long ans = 0;
```

```
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u])
        {
            mark[v] = u;
        }
        for (int v : g[u])
        {
            for (int w : g[v])
            {
                if (mark[w] == u)
                {
                    long long cur = mulMod(val[u], val[v], mod);
                    ans += mulMod(cur, val[w], mod);
                    if (ans >= mod)
                    {
                        ans -= mod;
                    }
                }
            }
        }
    }
    return (ans + mod) % mod;
}

inline int triWeight998(int n, const vector<int> &value,
                        const vector<pair<int, int>> &edges)
{
    // value 是点权，edges 是简单无向边；返回三元环三点权乘积和模 998244353。
    assert((int)value.size() == n); // 调试检查，可删。
    vector<int> deg(n), ord(n), rk(n);
    for (auto [u, v] : edges)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v); // 调试检查，可删。
        deg[u]++, deg[v]++;
    }
    iota(ord.begin(), ord.end(), 0);
    sort(ord.begin(), ord.end(), [&](int u, int v)
    {
        return pair{deg[u], u} < pair{deg[v], v};
    });
    for (int i = 0; i < n; i++)
    {
        rk[ord[i]] = i;
    }
    vector<Z> w(n);
    for (int i = 0; i < n; i++)
    {
        w[i] = value[ord[i]];
    }
    vector<vector<int>> g(n);
    for (auto [u, v] : edges)
```

```
    {
        u = rk[u], v = rk[v];
        if (u > v) swap(u, v);
        g[v].push_back(u);
    }
    vector<int> mark(n, -1);
    Z ans = 0;
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u])
        {
            mark[v] = u;
        }
        Z sum = 0;
        for (int v : g[u])
        {
            Z cur = 0;
            for (int x : g[v])
            {
                if (mark[x] == u) cur += w[x];
            }
            sum += cur * w[v];
        }
        ans += sum * w[u];
    }
    return ans.val();
}

unsigned long long fourCycles(int n, const vector<pair<int, int>> &e)
{
    // e 是简单无向图边集；返回四元环数量。
    int blocks = (n + 63) / 64;
    vector a(n, vector<unsigned long long>(blocks));
    for (auto [u, v] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v); // 调试检查，可删。
        a[u][v >> 6] |= 1ULL << (v & 63);
        a[v][u >> 6] |= 1ULL << (u & 63);
    }
    unsigned long long ans = 0;
    for (int u = 0; u < n; u++)
    {
        for (int v = u + 1; v < n; v++)
        {
            long long c = 0;
            for (int k = 0; k < blocks; k++)
            {
                c += popcount(a[u][k] & a[v][k]);
            }
            ans += (unsigned long long)c * (c - 1) / 2;
        }
    }
```

```cpp
    return ans / 2;
}

vector<long long> fourCycleCnt(int n, const vector<pair<int, int>> &edges)
{
    // n 是点数，edges 是允许重边的无向边；返回每条原边参与的四元环数量。
    int m = edges.size();
    vector<int> deg(n), ord(n), rk(n);
    for (auto [u, v] : edges)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v); // 调试检查，可删。
        deg[u]++;
        deg[v]++;
    }
    iota(ord.begin(), ord.end(), 0);
    stable_sort(ord.begin(), ord.end(), [&](int u, int v)
    {
        return deg[u] < deg[v];
    });
    for (int i = 0; i < n; i++)
    {
        rk[ord[i]] = i;
    }
    map<pair<int, int>, int> pos;
    vector<pair<int, int>> es;
    vector<vector<int>> group;
    for (int id = 0; id < m; id++)
    {
        auto [u, v] = edges[id];
        u = rk[u];
        v = rk[v];
        if (u < v) swap(u, v);
        auto [it, fresh] = pos.emplace(pair{u, v}, (int)es.size());
        if (fresh)
        {
            es.push_back({u, v});
            group.push_back({});
        }
        group[it->second].push_back(id);
    }
    vector<long long> mul(es.size()), ans(es.size());
    vector<vector<pair<int, int>>> g(n);
    for (int id = 0; id < (int)es.size(); id++)
    {
        auto [u, v] = es[id];
        mul[id] = group[id].size();
        g[u].push_back({v, id});
        g[v].push_back({u, id});
    }
    for (auto &a : g)
    {
        sort(a.begin(), a.end());
```

```
    }
    vector<long long> path(n);
    for (int top = n - 1; top >= 0; top--)
    {
        for (auto [mid, a] : g[top])
        {
            if (mid >= top) break;
           int lim = lower_bound(g[mid].begin(), g[mid].end(), pair{top, -1}) - g[mid].begin();
            for (int i = 0; i < lim; i++)
            {
                auto [bot, b] = g[mid][i];
                path[bot] += mul[a] * mul[b];
            }
        }
        for (auto [mid, a] : g[top])
        {
            if (mid >= top) break;
           int lim = lower_bound(g[mid].begin(), g[mid].end(), pair{top, -1}) - g[mid].begin();
            for (int i = 0; i < lim; i++)
            {
                auto [bot, b] = g[mid][i];
                long long other = path[bot] - mul[a] * mul[b];
                ans[a] += other * mul[b];
                ans[b] += other * mul[a];
            }
        }
        for (auto [mid, a] : g[top])
        {
            if (mid >= top) break;
           int lim = lower_bound(g[mid].begin(), g[mid].end(), pair{top, -1}) - g[mid].begin();
            for (int i = 0; i < lim; i++)
            {
                path[g[mid][i].first] = 0;
            }
        }
    }
    vector<long long> res(m);
    for (int i = 0; i < (int)es.size(); i++)
    {
        for (int id : group[i])
        {
            res[id] = ans[i];
        }
    }
    return res;
}
```

`triangles` 与 `fourCycles` 都要求简单无向图，不允许自环和重边，返回总数 `unsigned long long`。`weightedTri` 使用完全相同的定向枚举，但对每个环唯一地累加 `val[u]val[v]val[w]`；固定模数 998244353 时可用 `triWeight998`，它按度序重排后汇总公共邻点权，时间仍为 $O(m\sqrt{m})$，但减少循环内取模。`fourCycles` 适合顶点数不大但图较稠密的情形，时间为 $O(n^3/64)$。

`fourCycleCnt` 面向允许重边、要求每条输入边分别参与多少个四元环的场景；不允许自环。它先按度数给点重编号并把同端点的边压成一条带重数的边，再以最高编号点为顶点，汇总两条严格向下路径的重数积。每个长度三的折线只处理一次，第二轮减去当前折线后便能把其余同端点折线贡献回写给两条边，最后将压缩边答案复制回每条原边。令 m' 为压缩后简单图的边数，此实现排序和压缩为 $O(m \log m)$，路径枚举为 $O(m'\sqrt{m'})$，额外空间为 $O(n+m)$；在题目保证的 $n, m \le 3 \times 10^5$ 下应使用该接口而不是 bitset 总数接口。三种接口的结果都必须能放入 64 位有符号或无符号整数；若只做符号计算或图规模已使答案越界，调用者须另选数值类型。任意长度的简单环见 5.9.007 简单环存在性、输出与计数，最小权环见 5.9.004 无向图最小环。

5.9.007 简单环存在性、输出与计数

找环和数环是两类难度悬殊的任务，这里分开实现。找一条有向环用三色 DFS：白点未访问、灰点在当前递归栈上、黑点已回溯完毕；遇到指向灰点的边（返祖边）就找到了环，沿 `parent` 从当前点回溯到那个灰点即可复原环上顶点序列。而数出全部简单环是 #P 难的指数问题，只能在小图上用状态压缩 DP：为避免同一环因起点旋转或方向翻转被重复计数，固定「环上编号最小的顶点」为起点 s，DP 状态是已访问点集 `mask`（恒含 s）与当前落脚点 u，转移只走编号在 $[s, n)$ 的点，回到 s 且环长 ≥ 3 时累加；每个环被顺、逆两个方向各数一次，故 $/2$。

```cpp
vector<int> dirCycle(const vector<vector<int>> &g)
{
    // g 是有向邻接表；返回一个简单环，无环返回空。
    int n = g.size();
    vector<int> c(n), p(n, -1), ans;
    auto dfs = [&](auto &&self, int u) -> bool
    {
        c[u] = 1;
        for (int v : g[u])
        {
            assert(0 <= v && v < n); // 调试检查，可删。
            if (c[v] == 0)
            {
                p[v] = u;
                if (self(self, v))
                {
                    return true;
                }
            }
            else if (c[v] == 1)
            {
                ans = {v};
                for (int x = u; x != v; x = p[x])
                {
                    ans.push_back(x);
                }
                reverse(ans.begin() + 1, ans.end());
                return true;
            }
        }
```

```
        c[u] = 2;
        return false;
    };
    for (int s = 0; s < n; s++)
    {
        if (!c[s] && dfs(dfs, s))
        {
            return ans;
        }
    }
    return {};
}

vector<int> findDirCycle(int n, const vector<pair<int, int>> &edges)
{
    // n 是点数，edges 是有向边；返回一条环的原边编号顺序，无环返回空。
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < (int)edges.size(); i++)
    {
        auto [u, v] = edges[i];
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        g[u].push_back({v, i});
    }
    vector<int> state(n), it(n), par(n, -1), pe(n, -1), st;
    for (int s = 0; s < n; s++)
    {
        if (state[s]) continue;
        st.push_back(s);
        state[s] = 1;
        while (!st.empty())
        {
            int u = st.back();
            if (it[u] == (int)g[u].size())
            {
                state[u] = 2;
                st.pop_back();
                continue;
            }
            auto [v, id] = g[u][it[u]++];
            if (!state[v])
            {
                par[v] = u;
                pe[v] = id;
                state[v] = 1;
                st.push_back(v);
            }
            else if (state[v] == 1)
            {
                vector<int> ans{id};
                for (int x = u; x != v; x = par[x])
                {
                    ans.push_back(pe[x]);
```

```
                }
                reverse(ans.begin(), ans.end());
                return ans;
            }
        }
    }
    return {};
}

vector<int> findUndirCyc(int n, const vector<pair<int, int>> &edges)
{
    // n 是点数，edges 是无向边；返回一条环的原边编号顺序，无环返回空。
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < (int)edges.size(); i++)
    {
        auto [u, v] = edges[i];
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        g[u].push_back({v, i});
        if (u != v) g[v].push_back({u, i});
    }
    vector<int> state(n), it(n), par(n, -1), pe(n, -1), st;
    for (int s = 0; s < n; s++)
    {
        if (state[s]) continue;
        st.push_back(s);
        state[s] = 1;
        while (!st.empty())
        {
            int u = st.back();
            if (it[u] == (int)g[u].size())
            {
                state[u] = 2;
                st.pop_back();
                continue;
            }
            auto [v, id] = g[u][it[u]++];
            if (id == pe[u]) continue;
            if (!state[v])
            {
                par[v] = u;
                pe[v] = id;
                state[v] = 1;
                st.push_back(v);
            }
            else if (state[v] == 1)
            {
                vector<int> ans{id};
                for (int x = u; x != v; x = par[x])
                {
                    ans.push_back(pe[x]);
                }
                reverse(ans.begin(), ans.end());
```

```cpp
                return ans;
            }
        }
    }
    return {};
}

unsigned __int128 simpleCycles(const vector<unsigned long long> &g)
{
    // g 是简单无向图的对称邻接位掩码；返回全部简单环数量。
    int n = g.size();
    assert(n <= 22); // 调试检查，可删：状态数必须能承受。
    for (int u = 0; u < n; u++)
    {
        assert(!(g[u] >> u & 1)); // 调试检查，可删。
        for (int v = 0; v < n; v++)
        {
            assert((g[u] >> v & 1) == (g[v] >> u & 1)); // 调试检查，可删。
        }
    }
    unsigned __int128 ans = 0;
    for (int s = 0; s < n; s++)
    {
        int m = n - s;
        size_t states = 1ULL << m;
        vector<unsigned long long> dp(states * m);
        dp[m] = 1;
        for (size_t mask = 1; mask < states; mask += 2)
        {
            for (int u = 0; u < m; u++)
            {
                unsigned long long x = dp[mask * m + u];
                if (!x)
                {
                    continue;
                }
                for (int v = 1; v < m; v++)
                {
                    if (!(mask >> v & 1) && (g[s + u] >> (s + v) & 1))
                    {
                        dp[((mask | 1ULL << v) * m) + v] += x;
                    }
                }
                if (popcount((unsigned)mask) >= 3 && (g[s + u] >> s & 1))
                {
                    ans += x;
                }
            }
        }
    }
    return ans / 2;
}
```

`dirCycle(g)` 用递归 DFS 返回一个简单有向环，首点不在末尾重复；无环返回空。需要在线性时间输出原边编号且避开递归深度限制时，用 `findDirCycle(n, edges)` 或 `findUndirCyc(n, edges)`：两者维护显式 DFS 语义栈，返回能按顺序首尾相接的一条环边编号，无环返回空，支持重边和自环，均为 $O(n+m)$ 时间与空间。`simpleCycles(g)` 接受 $n \le 22$ 的对称位掩码，返回 `unsigned __int128` 环数；DP 的单状态路径数在该范围内不超过 $20!$，可安全存于 `unsigned long long`，最终总数再用 128 位累加。时间 $O(n^2 2^n)$、峰值空间 $O(n2^n)$，实际内存限制往往比断言更早成为瓶颈。定长短环见 5.9.006 三元环、四元环计数，带权最小环见 5.9.004 无向图最小环。

5.9.008 基环树

基环树是恰有 n 条边的连通简单无向图，等价于一棵树多一条边。反复删除度数不超过 1 的点，未被删除的点恰为唯一环；随后以全部环点为根做多源 BFS，即可给每个树枝点确定所属环点、父点和深度。这里的接口按简单图定义，不处理自环或平行边形成的一、二元环。

```cpp
struct Unicycle
{
    // 点数与无向邻接表。
    int n;
    vector<vector<int>> e;
    // cycle 按顺序保存唯一环上的顶点；root[v] 记 v 所挂的环点；parent[v]、dep[v] 是 v 在挂树内的父亲与深度。
    vector<int> cycle, root, parent, dep;
    // on[u] 表示 u 是否位于环上（剥叶子后仍未被删）。
    vector<bool> on;

    Unicycle(int n = 0) : n(n), e(n)
    {
        // n 是点数；构造空图。
    }

    void add(int u, int v)
    {
        // 加入简单无向边 u-v。
        assert(0 <= u && u < n && 0 <= v && v < n && u != v); // 调试检查，可删。
        e[u].push_back(v);
        e[v].push_back(u);
    }

    bool build()
    {
        // 重建环与挂树信息；输入不是基环树时返回 false。
        cycle.clear();
        root.clear();
        parent.clear();
        dep.clear();
        on.clear();
        int m = 0;
        vector<int> d(n);
        queue<int> q;
```

```
        for (int u = 0; u < n; u++)
        {
            d[u] = e[u].size();
            m += d[u];
            if (d[u] <= 1)
            {
                q.push(u);
            }
        }
        if (m ≠ 2 * n)
        {
            return false;
        }
        on.assign(n, true);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            on[u] = false;
            for (int v : e[u])
            {
                if (--d[v] == 1)
                {
                    q.push(v);
                }
            }
        }
        int s = find(on.begin(), on.end(), true) - on.begin();
        if (s == n)
        {
            return false;
        }
        for (int u = 0; u < n; u++)
        {
            if (!on[u])
            {
                continue;
            }
            int cnt = 0;
            for (int v : e[u])
            {
                cnt += on[v];
            }
            if (cnt ≠ 2)
            {
                return false;
            }
        }
        cycle = {s};
        int p = -1, u = s;
        while (true)
        {
```

```
            int v = -1;
            for (int x : e[u])
            {
                if (on[x] && x != p)
                {
                    v = x;
                    break;
                }
            }
            if (v == s)
            {
                break;
            }
            if (v < 0 || (int)cycle.size() > n)
            {
                return false;
            }
            cycle.push_back(v);
            p = u;
            u = v;
        }
        root.assign(n, -1);
        parent.assign(n, -1);
        dep.assign(n, 0);
        for (int x : cycle)
        {
            root[x] = x;
            q.push(x);
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int v : e[u])
            {
                if (on[v] || root[v] != -1)
                {
                    continue;
                }
                parent[v] = u;
                root[v] = root[u];
                dep[v] = dep[u] + 1;
                q.push(v);
            }
        }
        return count(on.begin(), on.end(), true) == (int)cycle.size() &&
               count(root.begin(), root.end(), -1) == 0;
    }
};
```

先 `Unicycle uc(n)`、逐边 `add`，再调用 `build()`。只有边数等于点数、唯一剩余核是一个简单环且所有点都从该环可达时才返回 `true`；成功后 `cycle`、`root`、`parent`、`dep`、`on` 有效。`build` 每次都会清空旧结果，

可在继续加边或手工修改邻接表后重新调用。全过程 $O(n)$ 时间与空间。多个连通块各带一环时应分块处理；树上算法见第 6 章，只需找任意环见 5.9.007 简单环存在性、输出与计数。

5.9.009 函数图与后继跳跃

函数图是每个点恰有一条出边的有向图：每个弱连通块恰含一个环，其余点组成指向环的内向树。先反复删除入度为 0 的点（拓扑剥离），剩下的点恰好都在环上；再按删除顺序的逆序处理树上点，它们的后继总是先被处理，于是入环点 `entry`、到环深度 `depth`、所在块 `comp` 都可直接继承。

后继跳跃用倍增：`up[j][v]` 是走 2^j 步后的点，同时维护这 2^j 步离开的点权和与最小值，层数 L 满足 $2^L > n$。步数 $k \le n$ 时直接倍增；$k > n$ 时必然先走 $\text{depth}(v)$ 步到达入环点 e，再绕环至少一整圈，剩余 $k - \text{depth}(v) = q \cdot len + r$ 步的贡献是 q 倍整圈加上 r 步前缀，所以 k 可以取到 $2^{64}-1$ 而表只有 $O(n \log n)$。

```
struct FuncGraph
{
    int n; // n 是点数，点编号为 [0,n)。
    // to[v] 是 v 的唯一后继；comp[v] 是所在连通块（即所在环）的编号。
    vector<int> to, comp;
    // pos[v] 是环上点在 cycles[comp[v]] 中的下标，树上点为 -1；entry[v] 是 v 第一次到达的环点。
    vector<int> pos, entry;
    // depth[v] 是 v 走到环上所需的步数（环上点为 0）；len[v] 是所在环的长度。
    vector<int> depth, len;
    vector<vector<int>> cycles; // cycles[i] 按后继方向列出第 i 个环。
    int L; // L 是倍增层数，满足 2^L > n。
    // up[j][v] 是 v 走 2^j 步后的点；sum、mn 是这 2^j 步离开的各点权值之和与最小值。
    vector<vector<int>> up, sum, mn;

    FuncGraph(vector<int> f, vector<int> w = {}) : n(f.size()), to(move(f)), comp(n, -1),
↪   pos(n, -1),
        entry(n), depth(n), len(n), L(bit_width((unsigned long long)n)), up(L, vector<int>(n)),
        sum(L, vector<int>(n)), mn(L, vector<int>(n))
    {
        // f[v] 是 v 的后继，w[v] 是点权（为空时全为 0），须满足 n*max|w| 不超过 LLONG_MAX；
        // 建立环分解与 L 层倍增表。
        if (w.empty())
        {
            w.assign(n, 0);
        }
        assert(w.size() == to.size()); // 调试检查，可删。
        // 反复删入度为 0 的点，剩下的恰是环；order 是被删点的顺序。
        vector<int> deg(n), order;
        for (int v : to)
        {
            deg[v]++;
        }
        queue<int> q;
        for (int i = 0; i < n; i++)
        {
            if (!deg[i])
```

```
            {
                q.push(i);
            }
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            order.push_back(u);
            if (!--deg[to[u]])
            {
                q.push(to[u]);
            }
        }
        for (int s = 0; s < n; s++)
        {
            if (deg[s] && comp[s] < 0)
            {
                int id = cycles.size(), u = s;
                vector<int> c;
                do
                {
                    comp[u] = id;
                    pos[u] = c.size();
                    entry[u] = u;
                    c.push_back(u);
                    u = to[u];
                } while (u != s);
                for (int v : c)
                {
                    len[v] = c.size();
                }
                cycles.push_back(move(c));
            }
        }
        // 逆删点顺序处理树上点：后继总是先被处理，可直接继承其信息。
        reverse(order.begin(), order.end());
        for (int u : order)
        {
            int v = to[u];
            comp[u] = comp[v];
            entry[u] = entry[v];
            depth[u] = depth[v] + 1;
            len[u] = len[v];
        }
        for (int i = 0; i < n; i++)
        {
            up[0][i] = to[i];
            sum[0][i] = w[i];
            mn[0][i] = w[i];
        }
        for (int j = 1; j < L; j++)
```

```
        {
            for (int i = 0; i < n; i++)
            {
                int v = up[j - 1][i];
                up[j][i] = up[j - 1][v];
                sum[j][i] = sum[j - 1][i] + sum[j - 1][v];
                mn[j][i] = min(mn[j - 1][i], mn[j - 1][v]);
            }
        }
    }

    struct Walk
    {
        int end;          // end 是走完后到达的点。
        __int128 total;   // total 是途中离开的各点权值之和。
        optional<int> minimum; // minimum 是途中离开的各点权值最小值，不走时为空。
    };

    Walk lift(int v, int k) const
    {
        // 0<=k<2^L；用倍增表从 v 出发走 k 步。
        Walk r{v, 0, nullopt};
        for (int j = 0; j < L; j++)
        {
            if (k >> j & 1)
            {
                r.total += sum[j][r.end];
                r.minimum = r.minimum ? min(*r.minimum, mn[j][r.end]) : mn[j][r.end];
                r.end = up[j][r.end];
            }
        }
        return r;
    }

    Walk walk(int v, unsigned long long k) const
    {
        // 从 v 出发走 k 步；返回终点以及离开的 k 个点（含 v，不含终点）的权值和与最小值。
        if (k <= (unsigned long long)n)
        {
            return lift(v, k);
        }
        // k>n 时必然绕环至少一整圈：先走到入环点 e，再把整圈的和乘上圈数。
        Walk a = lift(v, depth[v]);
        int e = a.end;
        unsigned long long rest = k - depth[v], q = rest / len[e];
        Walk c = lift(e, len[e]), b = lift(e, rest % len[e]);
        a.total += (__int128)q * c.total + b.total;
        a.minimum = a.minimum ? min(*a.minimum, *c.minimum) : *c.minimum;
        a.end = b.end;
        return a;
    }

```

```
    int jump(int v, unsigned long long k) const
    {
        // 返回 v 走 k 步后的点。
        return walk(v, k).end;
    }

    optional<unsigned long long> meet(int u, int v) const
    {
        // u、v 同时每步各走一条边；返回第一次落在同一点时的步数，永不相遇时返回空。
        if (comp[u] != comp[v])
        {
            return nullopt;
        }
        // 都走到环上之后位置差不再改变，所以相遇只可能发生在前 max(depth) 步内。
        unsigned long long hi = max(depth[u], depth[v]);
        if (jump(u, hi) != jump(v, hi))
        {
            return nullopt;
        }
        unsigned long long lo = 0;
        while (lo < hi)
        {
            unsigned long long m = lo + (hi - lo) / 2;
            if (jump(u, m) == jump(v, m))
            {
                hi = m;
            }
            else
            {
                lo = m + 1;
            }
        }
        return lo;
    }
};
```

`FuncGraph f(to, w)` 中 `to[v]` 是后继，`w` 是可选点权，须满足 $n \cdot \max|w|$ 不超过 `LLONG_MAX`。`walk(v,k)` 返回走 k 步后的终点 `end`、离开的 k 个点（含起点、不含终点）的权值和 `total` 与最小值 `minimum`（$k=0$ 时为空）；`jump(v,k)` 只返回终点。`meet(u,v)` 让两点每步各走一条边，返回第一次重合的步数：不在同一块时永不相遇；两点都进入环后位置差不再变化，所以只需检查前 $\max(\mathrm{depth}(u),\mathrm{depth}(v))$ 步，而一旦重合之后一直重合，可以二分。

预处理 $O(n \log n)$ 时间与空间，`walk` 为 $O(\log n)$，`meet` 为 $O(\log^2 n)$。只需找环、不需要跳跃时用 5.9.008 基环树 的拆环方式或直接拓扑剥离；树上的 k 级祖先见 6.1.006 树上距离、K 级祖先。下面的图中 $0 \to 1 \to 2 \to 3 \to 1$ 构成尾巴 0 和环 $\{1,2,3\}$，另有 $4 \to 2$：

```
signed main()
{
    FuncGraph f({1, 2, 3, 1, 2}, {5, -1, 2, 7, 3});
    auto w = f.walk(0, 10000000000000000ULL); // 离开 0 一次，再绕环 333333333333333333 圈。
    assert(w.end == 1 && w.total == 5 + (__int128)8 * 333333333333333333 && *w.minimum == -1);
```

```
    assert(f.jump(0, 4) == 1 && f.depth[0] == 1 && f.len[0] == 3 && f.entry[4] == 2);
    assert(f.meet(0, 3) == 1 && !f.meet(0, 4)); // 0、4 进入环后始终相差一步。
}
```

5.10 特殊图与图论结论

5.10.001 弦图、完美消除序列与判错

弦图是每个长度 ≥ 4 的环都有弦（连接环上不相邻两点的边）的图，等价于存在完美消除序列（PEO）：按此序逆向删点时，每个点在删除瞬间其尚未删除的邻居构成一个团。最大势搜索（MCS）能高效产生候选 PEO——维护每点「已被选入序列的邻居数」w，每步贪心选 w 最大的未选点、并给它未选邻居的 w 加一；把选出顺序反转即得候选 PEO。这个序不一定真是 PEO（图可能非弦），所以还要验证：按 PEO 正序，对每个点 u 找它在序中最靠前的后继邻居 p，检查 u 的其余后继邻居是否都与 p 相邻（用 `mark` 标记 p 的邻居后 $O(1)$ 查），一旦某个 v 不与 p 相邻就找到反例三元组 (u, p, v)，判定非弦图。

```
struct Chordal
{
    // ok 表示图是否为弦图。
    bool ok;
    // peo 保存最大势搜索得到的完美消除序列，弦图时有效。
    vector<int> peo;
    // 非弦图时，bad 是破坏候选 PEO 的三元组 (u,p,v)。
    array<int, 3> bad{-1, -1, -1};
};

Chordal chordal(const vector<vector<int>> &g)
{
    // g 是简单无向图邻接表；返回弦图判定、候选 PEO 和失败三元组。
    int n = g.size();
    vector<int> w(n), pos(n), peo;
    vector<bool> used(n);
    for (int k = 0; k < n; k++)
    {
        int u = -1;
        for (int i = 0; i < n; i++)
        {
            if (!used[i] && (u == -1 || w[i] > w[u]))
            {
                u = i;
            }
        }
        used[u] = true;
        peo.push_back(u);
        for (int v : g[u])
        {
            if (!used[v])
            {
                w[v]++;
            }
```

```
        }
    }
    reverse(peo.begin(), peo.end());
    for (int i = 0; i < n; i++)
    {
        pos[peo[i]] = i;
    }
    vector<int> mark(n, -1);
    for (int i = 0; i < n; i++)
    {
        int u = peo[i], p = -1;
        for (int v : g[u])
        {
            if (pos[v] > i && (p == -1 || pos[v] < pos[p]))
            {
                p = v;
            }
        }
        if (p == -1)
        {
            continue;
        }
        for (int v : g[p])
        {
            mark[v] = u;
        }
        for (int v : g[u])
        {
            if (v != p && pos[v] > i && mark[v] != u)
            {
                return {false, peo, {u, p, v}};
            }
        }
    }
    return {true, peo};
}

struct ChordalCert
{
    // ok 表示图是否为弦图。
    bool ok;
    // peo 是弦图的一组完美消除序列；非弦图时为空。
    vector<int> peo;
    // cycle 是非弦图的一条诱导环；弦图时为空。
    vector<int> cycle;
};

ChordalCert chordalCert(int n, const vector<pair<int, int>> &e)
{
    // n 是点数，e 是简单无向边；返回 PEO 或长度至少四的诱导环证书。
    assert(n >= 0); // 调试检查，可删。
    vector<set<int>> g(n);
```

```
    for (auto [u, v] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v); // 调试检查，可删。
        g[u].insert(v);
        g[v].insert(u);
    }
    vector<int> w(n), pos(n), ord;
    set<pair<int, int>> cand;
    for (int u = 0; u < n; u++)
    {
        cand.insert({0, u});
    }
    while (!cand.empty())
    {
        int u = prev(cand.end())->second;
        cand.erase(prev(cand.end()));
        pos[u] = ord.size();
        ord.push_back(u);
        w[u] = -1;
        for (int v : g[u])
        {
            if (w[v] != -1)
            {
                cand.erase({w[v], v});
                cand.insert({++w[v], v});
            }
        }
    }
    for (int u : ord)
    {
        int pre = -1;
        for (int v : g[u])
        {
            if (pos[v] < pos[u]) pre = max(pre, pos[v]);
        }
        if (pre == -1) continue;
        int p = ord[pre];
        for (int s : g[u])
        {
            if (pos[s] >= pos[p] || g[p].contains(s)) continue;
            vector<int> from(n, -1);
            queue<int> q;
            from[s] = s;
            q.push(s);
            while (!q.empty() && from[p] == -1)
            {
                int x = q.front();
                q.pop();
                for (int v : g[x])
                {
                    if (v == u || (v != p && g[u].contains(v)) || from[v] != -1) continue;
                    from[v] = x;
```

```
                    q.push(v);
                }
            }
            assert(from[p] != -1); // MCS 反例保证该路径存在。
            vector<int> cyc;
            for (int x = p; x != s; x = from[x])
            {
                cyc.push_back(x);
            }
            cyc.push_back(s);
            cyc.push_back(u);
            return {false, {}, move(cyc)};
        }
    }
    reverse(ord.begin(), ord.end());
    return {true, move(ord), {}};
}

ChordalCert chordalCertFast(int n, const vector<pair<int, int>> &edges)
{
    // edges 是简单无向边；使用桶式 MCS 与 CSR 邻接，返回 PEO 或诱导环证书。
    vector<int> deg(n), off(n + 1);
    for (auto [u, v] : edges)
    {
        deg[u]++, deg[v]++;
    }
    for (int i = 0; i < n; i++)
    {
        off[i + 1] = off[i] + deg[i];
    }
    vector<int> to(2 * edges.size()), cur = off;
    for (auto [u, v] : edges)
    {
        to[cur[u]++] = v, to[cur[v]++] = u;
    }
    vector<int> head(n + 1, -1), nx(n, -1), prv(n, -1), wt(n);
    auto erase = [&](int u)
    {
        int x = wt[u];
        if (prv[u] == -1) head[x] = nx[u]; else nx[prv[u]] = nx[u];
        if (nx[u] != -1) prv[nx[u]] = prv[u];
    };
    auto insert = [&](int u)
    {
        int x = wt[u];
        prv[u] = -1, nx[u] = head[x];
        if (nx[u] != -1) prv[nx[u]] = u;
        head[x] = u;
    };
    for (int i = 0; i < n; i++)
    {
        insert(i);
```

```
    }
    vector<int> ord;
    vector<char> used(n);
    int best = 0;
    while ((int)ord.size() < n)
    {
        while (best >= 0 && head[best] == -1)
        {
            best--;
        }
        int u = head[best];
        erase(u), used[u] = 1, ord.push_back(u);
        for (int i = off[u]; i < off[u + 1]; i++)
        {
            if (!used[to[i]])
{
int v = to[i];
erase(v), ++wt[v], insert(v), best = max(best, wt[v]);
}
        }
    }
    vector<int> pos(n), par(n, -1), child(n, -1), sibling(n, -1);
    for (int i = 0; i < n; i++)
    {
        pos[ord[i]] = i;
    }
    for (int u = 0; u < n; u++)
    {
        for (int i = off[u]; i < off[u + 1]; i++)
{
int v = to[i];
if (pos[v] < pos[u] && (par[u] == -1 || pos[par[u]] < pos[v])) par[u] = v;
}
    }
    for (int u = 0; u < n; u++)
    {
        if (par[u] ≠ -1) sibling[u] = child[par[u]], child[par[u]] = u;
    }
    vector<int> mark(n, -1);
    int bad = -1, parent = -1, other = -1;
    for (int u = 0; u < n; u++)
    {
        for (int i = off[u]; i < off[u + 1]; i++)
        {
            mark[to[i]] = u;
        }
        for (int v = child[u]; v ≠ -1; v = sibling[v])
        {
            for (int i = off[v]; i < off[v + 1]; i++)
            {
if (pos[to[i]] < pos[u] && mark[to[i]] ≠ u) bad = v, parent = u, other = to[i];
            }
```

```
        }
    }
    if (bad == -1)
    {
        reverse(ord.begin(), ord.end());
        return {true, move(ord), {}};
    }
    vector<char> ban(n);
    for (int i = off[bad]; i < off[bad + 1]; i++)
    {
        if (pos[to[i]] < pos[bad] && to[i] != parent && to[i] != other) ban[to[i]] = 1;
    }
    vector<int> from(n, -1), q(n);
    int l = 0, r = 0;
    from[other] = other, q[r++] = other;
    while (l < r && from[parent] == -1)
    {
        int u = q[l++];
        for (int i = off[u]; i < off[u + 1]; i++)
        {
            int v = to[i];
            if (pos[v] >= pos[bad] || ban[v] || from[v] != -1) continue;
            from[v] = u, q[r++] = v;
        }
    }
    if (from[parent] == -1) return {false, {}, {bad, parent, other}};
    vector<int> path;
    for (int u = parent;; u = from[u])
    {
        path.push_back(u);
        if (u == other) break;
    }
    reverse(path.begin(), path.end());
    vector<int> cycle{bad};
    cycle.insert(cycle.end(), path.begin(), path.end());
    return {false, {}, move(cycle)};
}
```

`chordal(g)` 是短小的邻接表教学入口：`.ok` 为真时 `.peo` 是 PEO，为假时 `.bad` 给出破坏候选 PEO 的三元组；其朴素 MCS 为 $O(n^2+m)$。顶点数约数千时它更短且便于调试。

`chordalCert(n,e)` 是大图和在线判题接口，输入简单无向边，除了 PEO 还能在非弦图时给出一条长度至少四的诱导环。它把 MCS 的未选点按 (势，编号) 放进有序集合，每次增势和邻接查询分别花 $O(\log n)$；找到 MCS 违反点对后，从其中一个不相邻后继出发，避开当前点及其其余邻居 BFS 到另一个后继，所得路径加上当前点正是一条无弦环。总时间 $O((n+m)\log n)$，额外空间 $O(n+m)$。大图可改用 `chordalCertFast`：桶式 MCS 与 CSR 邻接把排序和集合操作降为线性，仍返回 PEO/诱导环证书。判弦图后求团/独立集直接用 PEO 即可。

5.10.002 欧拉公式与圆上弦分侧

平面图的欧拉公式为 $V - E + F = 1 + C$，其中 C 是连通块数。它能推出简单平面图 $E \le 3V - 6$ 等必要条件，但不能判定一般图是否平面。本条只给两个与平面嵌入有关的短工具：`faceCount` 从欧拉公式反推面数；`chordPlanar` 处理顶点顺次固定在圆周上、每条弦可选画在圆内或圆外的特殊模型。两条弦端点交错时必须异侧，所以问题等价于给弦的交叉图二染色。

```cpp
namespace planar_graph
{
optional<vector<int>> chordPlanar(int n, vector<array<int, 2>> es)
{
    // n 是已按环排列的点数，es[i] 是额外弦端点；返回每条弦放环内或环外的 0/1 方案，无解返回空。
    int m = es.size();
    vector<vector<int>> g(m);
    for (auto &[u, v] : es)
    {
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        if (u > v)
        {
            swap(u, v);
        }
    }
    for (int i = 0; i < m; i++)
    {
        auto [a, b] = es[i];
        for (int j = i + 1; j < m; j++)
        {
            auto [c, d] = es[j];
            bool cross = (a < c && c < b && b < d) || (c < a && a < d && d < b);
            if (cross)
            {
                g[i].push_back(j);
                g[j].push_back(i);
            }
        }
    }
    vector<int> col(m, -1);
    for (int s = 0; s < m; s++)
    {
        if (col[s] >= 0)
        {
            continue;
        }
        col[s] = 0;
        queue<int> q;
        q.push(s);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int v : g[u])
```

```
            {
                if (col[v] < 0)
                {
                    col[v] = col[u] ^ 1;
                    q.push(v);
                }
                else if (col[v] == col[u])
                {
                    return nullopt;
                }
            }
        }
    }
    return col;
}

long long faceCount(long long v, long long e, long long comp = 1)
{
    // v、e、comp 是平面图的点数、边数和连通块数；返回任一平面嵌入的面数。
    if (e >= v)
    {
        return e - v + comp + 1;
    }
    return comp - (v - e) + 1;
}
}
```

`chordPlanar(n, es)` 返回每条弦位于圆内或圆外的 $0/1$ 方案，无解返回空，$O(m^2)$ 时间与空间。共享端点的弦不算交叉。`faceCount(v,e,comp)` 返回 `long long` 面数 $e - v + comp + 1$；调用方须保证该计数在 64 位内，空图应传 `comp=0`，得到唯一的外部面。本条标题刻意不写 “一般平面图判定”，任意图仍需真正的平面性算法；弦分侧也可编码为 5.4.001 Tarjan 2-SAT。

5.10.003 支配树（DAG / 一般图）

从入口 rt 出发，点 w 支配点 v 指的是「从 rt 到 v 的每条路径都经过 w」。所有支配 v 的点排成一条链，链上离 v 最近的那个（$\neq v$）就是 v 的直接支配点 $\mathrm{idom}(v)$；以 idom 为父指针得到的树即支配树，回答「删掉某点后哪些点从入口不可达」这类问题。这里用 Lengauer-Tarjan：先对图 DFS 建树、按 DFS 序 dfn 重标号；对每个点求半支配点 $\mathrm{sdom}(v)$——即所有满足「存在一条到 v 的路径、其中间点 dfn 都大于 v」的起点里 dfn 最小者，它是 idom 的良好近似。按 dfn 逆序处理，用带路径压缩的并查集 `dsu`（`find` 顺带把路径上 sdom 最小的顶点记进标签 `lab`）在已处理部分查询最小 sdom；再借「若 u 的 sdom 等于 v 的 sdom 则 $\mathrm{idom}(v) = \mathrm{sdom}(v)$，否则等于 $\mathrm{idom}(u)$」的定理两趟推出全部 idom。此实现对一般有向图（含环、DAG）都适用，DAG 也可退化成按拓扑序求前驱在支配树上的 LCA，但这份通用版无需区分。

```
struct Dominator
{
    // 点数与本次 DFS 到达的点数。
    int n, ti = 0;
    // 原图、按 DFS 序存的反图与半支配点桶。
```

```
    vector<vector<int>> g, rg, buc;
    // DFS 序及反查、DFS 父点、半支配点、直接支配点、并查集与最优标签。
    vector<int> dfn, rev, par, sdom, dom, dsu, lab;

    Dominator(const vector<vector<int>> &g)
        : n(g.size()), g(g), rg(n + 1), buc(n + 1), dfn(n), rev(n + 1), par(n + 1), sdom(n +
        ↪ 1), dom(n + 1), dsu(n + 1),
          lab(n + 1)
    {
        // g 是有向邻接表；构造求解器。
    }

    void buildDfs(int rt)
    {
        // 从 rt 递归 DFS，并按 DFS 序建立树父点与反图。
        dfn[rt] = ++ti;
        rev[ti] = rt;
        auto dfs = [&](auto &&self, int u) -> void
        {
            for (int v : g[u])
            {
                assert(0 <= v && v < n); // 调试检查，可删。
                if (!dfn[v])
                {
                    dfn[v] = ++ti;
                    rev[ti] = v;
                    par[ti] = dfn[u];
                    self(self, v);
                }
            }
        };
        dfs(dfs, rt);
        for (int u = 0; u < n; u++)
        {
            if (!dfn[u])
            {
                continue;
            }
            for (int v : g[u])
            {
                if (dfn[v])
                {
                    rg[dfn[v]].push_back(dfn[u]);
                }
            }
        }
        for (int i = 1; i <= ti; i++)
        {
            sdom[i] = dsu[i] = lab[i] = i;
        }
    }

```

```
    int find(int u)
    {
        // 返回并查集路径上半支配序最小的标签，并递归压缩路径。
        if (u == dsu[u])
        {
            return u;
        }
        if (dsu[u] == dsu[dsu[u]])
        {
            return lab[u];
        }
        auto compress = [&](auto &&self, int x) -> int
        {
            int p = dsu[x];
            if (p == dsu[p])
            {
                return x;
            }
            int base = self(self, p);
            if (sdom[lab[p]] < sdom[lab[x]])
            {
                lab[x] = lab[p];
            }
            dsu[x] = base;
            return base;
        };
        compress(compress, u);
        return lab[u];
    }

    vector<int> work(int rt)
    {
        // rt 是入口；返回直接支配点，不可达点为 -1。
        assert(0 <= rt && rt < n); // 调试检查，可删。
        ti = 0;
        rg.assign(n + 1, {});
        buc.assign(n + 1, {});
        dfn.assign(n, 0);
        rev.assign(n + 1, 0);
        par.assign(n + 1, 0);
        sdom.assign(n + 1, 0);
        dom.assign(n + 1, 0);
        dsu.assign(n + 1, 0);
        lab.assign(n + 1, 0);
        buildDfs(rt);
        for (int i = ti; i; i--)
        {
            for (int v : rg[i])
            {
                sdom[i] = min(sdom[i], sdom[find(v)]);
            }
            if (i > 1)
```

```
            {
                buc[sdom[i]].push_back(i);
            }
            for (int v : buc[i])
            {
                int u = find(v);
                dom[v] = sdom[u] == sdom[v] ? sdom[v] : u;
            }
            if (i > 1)
            {
                dsu[i] = par[i];
            }
        }
        for (int i = 2; i <= ti; i++)
        {
            if (dom[i] != sdom[i])
            {
                dom[i] = dom[dom[i]];
            }
        }
        vector<int> ans(n, -1);
        ans[rt] = rt;
        for (int i = 2; i <= ti; i++)
        {
            ans[rev[i]] = rev[dom[i]];
        }
        return ans;
    }
};
```

用 `Dominator dm(g)` 建器，`work(rt)` 返回直接支配点数组：`ans[rt]=rt`，不可达点为 -1。每次 `work` 都会重建 DFS 序、反图、桶和并查集状态，因此同一对象可更换入口重复求解；DFS 与并查集路径压缩均直接递归，深链需确认系统栈足够。多个真实入口可先加虚拟源。Lengauer-Tarjan 为近线性时间，常写作 $O((n+m)\alpha(n))$，空间 $O(n+m)$。DAG 也可按拓扑序配合支配树 LCA 增量构造，但通用图直接使用本实现。

下面用菱形 DAG 检查汇合点的直接支配者，再用带环图检查环内支配关系和不可达点。

```
signed main()
{
    vector<vector<int>> a{{1, 2}, {3}, {3}, {}};
    assert(Dominator(a).work(0) == vector<int>({0, 0, 0, 0}));

    vector<vector<int>> b{{1}, {2}, {1, 3}, {}, {}};
    assert(Dominator(b).work(0) == vector<int>({0, 0, 1, 2, -1}));
    return 0;
}
```

5.10.004 竞赛图与 Landau 定理

竞赛图是每对顶点之间恰有一条有向边的图（n 个人两两比赛的胜负关系）。Landau 定理刻画了哪些出度（胜场）多重集是合法的：把出度升序排成 $d_1 \le \dots \le d_n$，则存在对应竞赛图当且仅当每个前缀和 $\sum_{i=1}^{k} d_i \ge \binom{k}{2}$，且总和 $\sum d_i = \binom{n}{2}$。直觉是前 k 小的选手内部至少产生 $\binom{k}{2}$ 场胜负、这些胜场只能落在他们自己头上，故前缀和有下界；总胜场数恒等于总场数。这两个条件不仅必要也充分。

```cpp
bool tourScore(vector<long long> a)
{
    // a 是竞赛图各点出度；返回是否存在一个竞赛图恰好具有该出度多重集。
    sort(a.begin(), a.end());
    int n = a.size();
    __int128 sum = 0;
    for (int k = 1; k <= n; k++)
    {
        if (a[k - 1] < 0 || a[k - 1] >= n)
        {
            return false;
        }
        sum += (__int128)a[k - 1];
        if (sum < (__int128)k * (k - 1) / 2)
        {
            return false;
        }
    }
    return sum == (__int128)n * (n - 1) / 2;
}
```

`tourScore(a)` 接受任意顺序的出度多重集（函数内部会排序），返回是否可图；只判存在性、不构造具体竞赛图（要构造可用逐步满足前缀下界的贪心）。$O(n \log n)$ 时间、$O(n)$ 空间。例如 `{0,1,2}`、`{1,1,1}` 都返回 `true`，`{0,0,3}` 返回 `false`。无向图度序列的可图性是另一套判据，见 5.10.005 度序列可图性 / Erdős-Gallai；竞赛图本身的强连通性、哈密顿路径等结论可结合 5.3.001 Tarjan 强连通分量。

5.10.005 度序列可图性 / Erdős-Gallai

Erdős-Gallai 定理判定一个非负整数多重集能否作为无向简单图（无自环、无重边）的度序列。把度数降序排成 $d_1 \ge \dots \ge d_n$，可图当且仅当总和 $\sum d_i$ 为偶，且对每个 k 有 $\sum_{i=1}^{k} d_i \le k(k-1) + \sum_{i>k} \min(d_i, k)$。右端的含义是：前 k 个高度点的度数，最多有 $k(k-1)$ 连在彼此之间（每对贡献 2），其余每个点最多向它们连 $\min(d_i, k)$ 条。$\sum_{i>k} \min(d_i, k)$ 可用「找到第一个 $d_i < k$ 的位置 p」把它拆成前段整块贡献 k、后段贡献原度数（后缀和），从而每个 k 都 $O(\log n)$ 二分算出，总 $O(n \log n)$。

```cpp
bool graphical(vector<long long> a)
{
    // a 是无向简单图各点度数；返回是否存在一个简单图恰好具有该度数多重集。
    sort(a.begin(), a.end(), greater<>());
    int n = a.size();
    vector<long long> s(n + 1);
    for (int i = 0; i < n; i++)
```

```
    {
        if (a[i] < 0 || a[i] >= n)
        {
            return false;
        }
        if (s[i] > LLONG_MAX - a[i])
        {
            return false;
        }
        s[i + 1] = s[i] + a[i];
    }
    if (s[n] & 1)
    {
        return false;
    }
    for (int k = 1; k <= n; k++)
    {
        int p = lower_bound(a.begin() + k, a.end(), k, greater<>()) - a.begin();
        long long rhs = 1LL * k * (k - 1) + 1LL * (p - k) * k + s[n] - s[p];
        if (s[k] > rhs)
        {
            return false;
        }
    }
    return true;
}
```

`graphical(a)` 接受任意顺序的度数多重集，度数须落在 $[0, n)$ 内且前缀和与判定式在 64 位内，返回是否可图。只判存在性；要真正构造出边集应改用 Havel-Hakimi 贪心（每次取最大度点连向次大的若干点）。$O(n \log n)$ 时间、$O(n)$ 空间。例如 `{2,2,2}`、`{3,3,2,2,2,2}` 返回 `true`，`{3,3,1,1}` 返回 `false`。竞赛图（有向、任意两点一条边）的出度可图性判据不同，见 5.10.004 竞赛图与 Landau 定理；二部图度序列用 Gale-Ryser 定理（本书未单列，思路类似）。

5.10.006 最大团、极大团计数

最大团（任意两点相邻的最大点集）是 NP 难的，但用位运算 + 分支限界在 $n \le 64$ 时很快。`maxClique` 递归维护当前团 `r` 与还能加入的候选集 `p`（都是 64 位掩码），剪枝是「当前团大小 + 候选数 $\le$ 已知最优则回退」；每步选一个枢轴 u（候选中与 `p` 交集最大的点）只对 `p & ~e[u]`（非 u 邻居的候选）分支，因为最大团必含 u 或 u 的某个非邻居，这样能砍掉大量重复分支。补图上的最大团就是原图最大独立集，`maxIndSet` 据此把邻接取反后复用 `maxClique`。枚举全部极大团（无法再加点的团，未必最大）用带枢轴的 Bron-Kerbosch：维护当前团 `r`、候选 `p`、已排除 `x`，`p|x` 为空即输出一个极大团，同样按枢轴分支，最坏 $O(3^{n/3})$ 个极大团。

```
struct Clique64
{
    // n 是图的点数；best 是搜索过程中已知的最大团大小。
    int n, best = 0;
    // ans 是当前最大团的点位集（第 i 位为 1 表示点 i 在团中）。
```

```
    unsigned long long ans = 0;
    // e[u] 的第 v 位为 1 表示存在无向边 u-v。
    vector<unsigned long long> e;

    Clique64(int n = 0) : n(n), e(n)
    {
        // n 是点数；构造空图。
        assert(0 <= n && n <= 64); // 调试检查，可删。
    }

    void add(int u, int v)
    {
        // 加入无向边 u-v；自环与团定义无关，直接忽略。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
        if (u == v)
        {
            return;
        }
        e[u] |= 1ULL << v;
        e[v] |= 1ULL << u;
    }

    void dfs(unsigned long long r, unsigned long long p)
    {
        // r 是已选团，p 是仍可加入的点集；用分支限界更新最大团，无返回值。
        int z = popcount(r);
        if (z + popcount(p) <= best)
        {
            return;
        }
        if (!p)
        {
            best = z;
            ans = r;
            return;
        }
        int u = countr_zero(p);
        for (auto q = p; q; q &= q - 1)
        {
            int v = countr_zero(q);
            if (popcount(p & e[v]) > popcount(p & e[u]))
            {
                u = v;
            }
        }
        unsigned long long q = p & ~e[u];
        while (q)
        {
            int v = countr_zero(q);
            dfs(r | 1ULL << v, p & e[v]);
            p &= ~(1ULL << v);
            q &= q - 1;
```

```
            if (z + popcount(p) <= best)
            {
                return;
            }
        }
    }

    vector<int> maxClique()
    {
        // 无参数；返回当前图的一组最大团顶点。
        best = 0;
        ans = 0;
        dfs(0, n == 64 ? ~0ULL : (1ULL << n) - 1);
        vector<int> a;
        for (int i = 0; i < n; i++)
        {
            if (ans >> i & 1)
            {
                a.push_back(i);
            }
        }
        return a;
    }

    vector<int> maxIndSet()
    {
        // 无参数；返回当前图的一组最大独立集顶点。
        unsigned long long all = n == 64 ? ~0ULL : (1ULL << n) - 1;
        auto old = e;
        for (int i = 0; i < n; i++)
        {
            e[i] = all & ~(old[i] | 1ULL << i);
        }
        auto res = maxClique();
        e = move(old);
        return res;
    }

    template <class F>
    void maxCliques(F f)
    {
        // f 接收一个极大团的点位集；枚举全部极大团并逐个调用 f，无返回值。
        auto go = [&](auto &&self, unsigned long long r, unsigned long long p, unsigned long
        ↪  long x) -> void
        {
            if (!(p | x))
            {
                f(r);
                return;
            }
            int u = countr_zero(p | x);
            unsigned long long q = p & ~e[u];
```

```
            while (q)
            {
                int v = countr_zero(q);
                self(self, r | 1ULL << v, p & e[v], x & e[v]);
                p &= ~(1ULL << v);
                x |= 1ULL << v;
                q &= q - 1;
            }
        };
        go(go, 0, n == 64 ? ~0ULL : (1ULL << n) - 1, 0);
    }
};

long long weightClique(int n, const vector<long long> &val, const vector<pair<int, int>> &e,
 ↪ long long mod)
{
    // n 是点数，val 是点权，e 是简单无向边，mod 是正模数；返回所有非空团点权乘积之和模 mod。
    assert((int)val.size() == n && mod > 0); // 调试检查，可删。
    vector<vector<int>> g(n);
    vector<vector<char>> adj(n, vector<char>(n));
    for (auto [u, v] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u ≠ v && !adj[u][v]); // 调试检查，可删。
        g[u].push_back(v);
        g[v].push_back(u);
        adj[u][v] = adj[v][u] = 1;
    }
    int lim = sqrt(2.0 * e.size()) + 1;
    vector<char> alive(n, 1);
    auto go = [&](auto &&self, int rem) -> long long
    {
        if (rem == 0) return 0;
        int x = -1;
        for (int u = 0; u < n; u++)
        {
            if (alive[u] && (int)g[u].size() < lim)
            {
                x = u;
                break;
            }
        }
        if (x == -1)
        {
            vector<int> vs;
            for (int u = 0; u < n; u++)
            {
                if (alive[u]) vs.push_back(u);
            }
            long long ans = 0;
            for (int mask = 1; mask < (1 << vs.size()); mask++)
            {
                long long cur = 1;
```

```
                bool ok = true;
                for (int i = 0; i < (int)vs.size() && ok; i++)
                {
                    if (!(mask >> i & 1)) continue;
                    cur = (__int128)cur * (val[vs[i]] % mod) % mod;
                    for (int j = 0; j < i; j++)
                    {
                        if ((mask >> j & 1) && !adj[vs[i]][vs[j]])
                        {
                            ok = false;
                            break;
                        }
                    }
                }
                if (ok) ans = (ans + cur) % mod;
            }
            return ans;
        }
        long long ans = 0;
        int d = g[x].size();
        for (int mask = 0; mask < (1 << d); mask++)
        {
            long long cur = val[x] % mod;
            bool ok = true;
            for (int i = 0; i < d && ok; i++)
            {
                if (!(mask >> i & 1)) continue;
                int u = g[x][i];
                if (!alive[u])
                {
                    ok = false;
                    break;
                }
                cur = (__int128)cur * (val[u] % mod) % mod;
                for (int j = 0; j < i; j++)
                {
                    if ((mask >> j & 1) && !adj[u][g[x][j]])
                    {
                        ok = false;
                        break;
                    }
                }
            }
            if (ok) ans = (ans + cur) % mod;
        }
        alive[x] = 0;
        return (ans + self(self, rem - 1)) % mod;
    };
    return (go(go, n) + mod) % mod;
}
```

构造 `Clique64 g(n)`（$n \le 64$）、逐边 `g.add(u, v)`，然后 `maxClique()` 返回一组最大团的点编号、

`maxIndSet()` 返回一组最大独立集、`maxCliques(f)` 对每个极大团把其 64 位点掩码 `r` 交给回调 `f`。求最大团/独立集与极大团枚举最坏都是 $O(3^{n/3})$、$O(n)$ 递归栈，随机稀疏图上剪枝极快。

枚举全部极大团时，回调参数是 64 位点集：

```
signed main()
{
    Clique64 g(3);
    g.add(0, 1);
    g.add(1, 2);
    int cnt = 0;
    g.maxCliques([&](unsigned long long mask)
    {
        cnt++;
        assert(mask);
    });
    assert(cnt == 2);
}
```

`weightClique(n,val,e,mod)` 用于 $n, m \le 100$、需要累加每个非空团的点权乘积的情形。它不断选原图度数小于 $\lfloor\sqrt{2m}\rfloor+1$ 的未删点，枚举其仍存邻居的全部子集，因每个团按最早删点只算一次而不重不漏；若再没有低度点，剩余点数小于该阈值，直接枚举其所有子集。邻接矩阵使子集合法性检查为常数时间，时间为 $O(n2^{\sqrt{2m}+1})$，空间为 $O(n^2+m)$；`mod` 为正，乘积中间量使用 `__int128`。图非常大或有特殊结构时不要硬套指数算法：弦图上最大团用 5.10.001 弦图、完美消除序列与判错 的 PEO 多项式求；二分图最大独立集 = 总点数 − 最大匹配，用 5.6.008 最大独立集。最小点染色数（色数）与最大独立集划分见 5.10.007 最大独立集与点染色，那里给出至多 24 点的精确染色 DP。

5.10.007 最大独立集与点染色

最大独立集直接调用 5.10.006 最大团、极大团计数 的 `Clique64::maxIndSet()`，不再维护第二份补图搜索。点染色用独立集子集 DP：预处理每个点集是否为独立集，再把全集划分成尽量少的独立集。固定让当前集合的最低位属于新取的独立集，可消除颜色类排列造成的重复枚举而不丢最优解。它有明确的 $O(3^n)$ 最坏界，适合 $n \le 24$ 且内存充足。

```
int chromaticNum(const vector<unsigned long long> &e)
{
    // e 是简单无向图邻接位集；返回最小点染色数。
    int n = e.size();
    assert(n <= 24); // 调试检查，可删：状态数必须能承受。
    size_t z = 1ULL << n;
    vector<bool> ind(z);
    ind[0] = true;
    for (size_t s = 1; s < z; s++)
    {
        int v = countr_zero((unsigned long long)s);
        size_t t = s ^ (1ULL << v);
        ind[s] = ind[t] && !(e[v] & t);
    }
    vector<int> f(z, n);
```

```
    f[0] = 0;
    for (size_t s = 1; s < z; s++)
    {
        size_t b = s & -s;
        for (size_t t = s; t; t = (t - 1) & s)
        {
            if ((t & b) && ind[t])
            {
                chmin(f[s], f[s ^ t] + 1);
            }
        }
    }
    return f.back();
}
```

`chromaticNum(e)` 返回最小色数，$n \le 24$，时间 $O(3^n)$、空间 $O(2^n)$；它不恢复方案，但运行时间和峰值内存只由 n 决定，表现较稳定。稀疏或结构明显、并且需要一组具体染色时，可改用 5.10.008 DSATUR 实践型精确点染色。

5.10.008 DSATUR 实践型精确点染色

DSATUR 仍做精确搜索，但分支顺序由当前部分染色决定：优先选择「已染邻居使用的颜色种类最多」的点，再以未染邻居数打破平局。先贪心得到一组合法染色作为上界，随后枚举已有颜色或至多一个新颜色，并用当前颜色数剪枝。它是确定性算法且能恢复方案，通常比子集 DP 更快，但最坏时间仍为指数级，不能把 64 位邻接掩码的容量误当成保证可通过的规模。

```
struct Coloring
{
    int k;              // 使用的颜色数。
    vector<int> color; // 每个点的颜色，编号从 0 开始。
};

class Dsatur
{
    int n, best; // n 是点数，best 是当前最优颜色数。
    vector<unsigned long long> e; // e[u] 是 u 的邻接位集。
    vector<int> col, ans; // col 是搜索中的染色（-1 未染），ans 是最优染色。

    unsigned long long forbidden(int u) const
    {
        // u 是点；返回 u 的已染色邻居所用颜色的位集。
        unsigned long long mask = 0, q = e[u];
        while (q)
        {
            int v = countr_zero(q);
            q &= q - 1;
            if (col[v] >= 0)
            {
                mask |= 1ULL << col[v];
```

```
            }
        }
        return mask;
    }

    int pick() const
    {
        // 无参数；返回饱和度最大、其次未染色邻居最多的未染色点。
        int at = -1, sat = -1, deg = -1;
        unsigned long long left = 0;
        for (int u = 0; u < n; u++)
        {
            if (col[u] < 0)
            {
                left |= 1ULL << u;
            }
        }
        for (int u = 0; u < n; u++)
        {
            if (col[u] >= 0)
            {
                continue;
            }
            int s = popcount(forbidden(u));
            int d = popcount(e[u] & left);
            if (s > sat || (s == sat && d > deg))
            {
                at = u;
                sat = s;
                deg = d;
            }
        }
        return at;
    }

    void dfs(int done, int used)
    {
        // done 是已染色点数，used 是已用颜色数；分支定界搜索，更新 best 与 ans。
        if (used >= best)
        {
            return;
        }
        if (done == n)
        {
            best = used;
            ans = col;
            return;
        }
        int u = pick();
        unsigned long long ban = forbidden(u);
        for (int c = 0; c < used; c++)
        {
```

```
            if (ban >> c & 1)
            {
                continue;
            }
            col[u] = c;
            dfs(done + 1, used);
            col[u] = -1;
        }
        if (used + 1 < best)
        {
            col[u] = used;
            dfs(done + 1, used + 1);
            col[u] = -1;
        }
    }

public:
    Dsatur(vector<unsigned long long> e) : n(e.size()), best(n + 1), e(move(e)), col(n, -1)
    {
        // e 是简单无向图邻接位集；构造精确染色器。
        assert(n <= 64); // 调试检查，可删。
        for (int u = 0; u < n; u++)
        {
            assert(!(this->e[u] >> u & 1)); // 调试检查，可删。
            for (int v = 0; v < n; v++)
            {
                assert((this->e[u] >> v & 1) == (this->e[v] >> u & 1)); // 调试检查，可删。
            }
        }
    }

    Coloring solve()
    {
        // 返回最少颜色数和一组染色方案。
        fill(col.begin(), col.end(), -1);
        int used = 0;
        for (int step = 0; step < n; step++)
        {
            int u = pick();
            unsigned long long ban = forbidden(u);
            int c = 0;
            while (ban >> c & 1)
            {
                c++;
            }
            col[u] = c;
            used = max(used, c + 1);
        }
        best = used;
        ans = col;
        fill(col.begin(), col.end(), -1);
        dfs(0, 0);
```

```
        return {best, ans};
    }
};
```

`Dsatur(e).solve()` 返回精确色数与方案；同一层只新增一个颜色编号，消除颜色置换造成的重复。最坏时间仍为指数级，额外空间为 $O(n)$ 递归栈和状态；实现不随机、没有重试语义，只依赖 C++20 位运算。稀疏或结构明显的图通常优先试本版；需要 $n \le 24$ 下由规模决定的稳定时间和内存界时用 5.10.007 最大独立集与点染色。弦图色数可由 5.10.001 弦图、完美消除序列与判错 多项式求，二分图只需 BFS 二染色。

```
signed main()
{
    vector<unsigned long long> e(5);
    for (int u = 0; u < 5; u++)
    {
        int v = (u + 1) % 5;
        e[u] |= 1ULL << v;
        e[v] |= 1ULL << u;
    }
    Coloring res = Dsatur(e).solve();
    assert(res.k == 3 && res.color == vector<int>({0, 1, 0, 1, 2}));
}
```

5.10.009 有向图最小点基

有向图的最小点基是「选最少的起点，使得从它们出发能到达图中所有点」的一个最小点集。做法是先用 5.3.001 Tarjan 强连通分量 的 `tarjanSCC` 把图缩成 SCC-DAG：同一强连通分量内部互相可达，只需选其中一点即可覆盖整块。缩点后统计每个分量的入度，凡入度为 0 的分量都没有任何外部点能到达它，必须自己选一个起点；而所有入度非零的分量都能被某个入度为 0 的分量沿 DAG 路径到达，无需另选。因此最小点基的大小等于入度为 0 的分量数，从每个这样的分量各取一个代表点（这里取分量内最小编号点）即得一组方案。

```
// 依赖 5.3.001 Tarjan 强连通分量
vector<int> minPtBasis(const vector<vector<int>> &g)
{
    // g 是有向图邻接表；返回升序最小点基，每个源 SCC 取最小编号点。
    auto id = tarjanSCC(g);
    if (g.empty())
    {
        return {};
    }
    int n = g.size(), m = *max_element(id.begin(), id.end()) + 1;
    vector<int> in(m), mn(m, n);
    for (int u = 0; u < n; u++)
    {
        mn[id[u]] = min(mn[id[u]], u);
        for (int v : g[u])
        {
```

```
            if (id[u] != id[v])
            {
                in[id[v]]++;
            }
        }
    }
    vector<int> ans;
    for (int i = 0; i < m; i++)
    {
        if (!in[i])
        {
            ans.push_back(mn[i]);
        }
    }
    sort(ans.begin(), ans.end());
    return ans;
}
```

调用前需备好 5.3.001 Tarjan 强连通分量 的 `tarjanSCC`。`minPtBasis(g)` 吃邻接表（点从 0 编号），返回升序的代表点列表；空图返回空。缩点与统计都是线性的，整体 $O(n+m)$ 时间与空间。注意「最小点基」只保证覆盖可达性、且大小唯一，但代表点的选择不唯一（这里固定取分量最小编号）。若要的是「最少边使全图强连通」（强连通化）则是另一问题，答案为 $\max(\text{源分量数}, \text{汇分量数})$，同样基于 SCC-DAG；缩点后的其它 DAG 分析（拓扑、最长路）也从 5.3.001 Tarjan 强连通分量 出发。

5.10.010 有向无环定向与色多项式

图 G 的色多项式 $P_G(x)$ 是「用 x 种颜色作合法点染色的方案数」，它是关于 x 的 n 次多项式。关键观察：每个颜色类必是一个独立集，于是「用恰好 k 种有标号颜色染全图」的方案数等于把全集划分/覆盖成 k 个（可空）独立集的有序方式数。用子集 DP 求：`ind[s]` 标记点集 s 是否独立（增量判定），`dp` 每轮与「加入一个新颜色类 = 任选一个独立子集」做子集卷积，k 轮后 `dp[full]` 就是 $P_G(k)$。这样得到 $n+1$ 个样本 $P_G(0),\dots,P_G(n)$，因 $\deg P_G = n$ 足以唯一确定它。再用 Newton 前向差分把样本转成下降幂基系数，边算边乘 $x(x-1)\cdots$ 的展开多项式 `cur`，累加成普通幂次系数。有了色多项式，Stanley 定理立刻给出无环定向数 $= (-1)^n P_G(-1)$。

```
// 依赖 1.1.008 静态取模类
vector<array<Z, 21>> rankedZeta(vector<Z> f, int n)
{
    // f 是大小为 2^n 的子集函数；返回按子集大小分层后的子集 Zeta 变换。
    vector<array<Z, 21>> a(f.size());
    for (int s = 0; s < (int)f.size(); s++)
    {
        a[s][popcount((unsigned)s)] = f[s];
    }
    for (int i = 0; i < n; i++)
    {
        for (int s = 0; s < (1 << n); s++)
        {
            if (s >> i & 1)
```

```
            {
                for (int d = 0; d <= n; d++)
                {
                    a[s][d] += a[s ^ (1 << i)][d];
                }
            }
        }
    }
    return a;
}

vector<Z> rankedMobius(vector<array<Z, 21>> a, int n)
{
    // a 是按子集大小分层后的 Zeta 值；返回对应的子集函数。
    for (int i = 0; i < n; i++)
    {
        for (int s = 0; s < (1 << n); s++)
        {
            if (s >> i & 1)
            {
                for (int d = 0; d <= n; d++)
                {
                    a[s][d] -= a[s ^ (1 << i)][d];
                }
            }
        }
    }
    vector<Z> f(1 << n);
    for (int s = 0; s < (1 << n); s++)
    {
        f[s] = a[s][popcount((unsigned)s)];
    }
    return f;
}

vector<Z> subsetConv(const vector<Z> &a, const vector<Z> &b)
{
    // a、b 是相同 n 位全集上的子集函数；返回 (a*b)[S]=sum_{T subset S}a[T]b[S-T]。
    int n = countr_zero((unsigned)a.size());
    assert(a.size() == b.size() && n <= 20); // 调试检查，可删。
    auto x = rankedZeta(a, n), y = rankedZeta(b, n);
    for (int s = 0; s < (1 << n); s++)
    {
        for (int d = n; d >= 0; d--)
        {
            Z z = 0;
            for (int i = 0; i <= d; i++)
            {
                z += x[s][i] * y[s][d - i];
            }
            x[s][d] = z;
        }
```

```
    }
    return rankedMobius(move(x), n);
}

vector<Z> transSubConv(vector<Z> s, vector<Z> x)
{
    // s、x 是同一位数的子集函数；返回映射 a->a*s 对 x 的转置作用结果。
    reverse(x.begin(), x.end());
    x = subsetConv(x, s);
    reverse(x.begin(), x.end());
    return x;
}

vector<Z> powerProjEgf(vector<Z> wt, const vector<Z> &s, int n)
{
    // wt 是线性泛函权，s[0] 必须为 0；返回 sum_T wt[T](s^k/k!)[T]，k=0..n。
    vector<Z> ans(n + 1), dp = move(wt);
    ans[0] = dp[0];
    for (int i = 0; i < n; i++)
    {
        vector<Z> nxt(1 << (n - 1 - i));
        for (int j = 0; j < n - i; j++)
        {
            int l = 1 << j, r = 2 << j;
            vector<Z> a(s.begin() + l, s.begin() + r);
            vector<Z> b(dp.begin() + l, dp.begin() + r);
            b = transSubConv(move(a), move(b));
            for (int k = 0; k < (int)b.size(); k++)
            {
                nxt[k] += b[k];
            }
        }
        dp = move(nxt);
        ans[i + 1] = dp[0];
    }
    return ans;
}

vector<Z> chromPoly(const vector<unsigned long long> &g)
{
    // g 是允许自环的无向图邻接位集；返回按次数升序的色多项式系数，自环图返回全零，支持至多 20 点。
    int n = g.size(), N = 1 << n;
    assert(n <= 20 && n < Z::mod()); // 调试检查，可删：模数须为大于 n 的素数。
    for (int u = 0; u < n; u++)
    {
        if (g[u] >> u & 1) return vector<Z>(n + 1);
    }
    vector<char> ind(N, true);
    for (int s = 1; s < N; s++)
    {
        int u = countr_zero((unsigned)s);
        ind[s] = ind[s ^ (1 << u)] && !(g[u] & (s ^ (1 << u)));
```

```
    }
    vector<Z> f(N), wt(N);
    for (int s = 0; s < N; s++)
    {
        if (ind[s]) f[s] = 1;
    }
    wt[N - 1] = 1;
    Z c = f[0];
    f[0] -= c;
    vector<Z> egf = powerProjEgf(wt, f, n);
    vector<Z> val(n + 1), pw(n + 1), fact(n + 1, 1);
    for (int i = 1; i <= n; i++)
    {
        fact[i] = fact[i - 1] * i;
    }
    for (int k = 0; k <= n; k++)
    {
        if (k == 0) pw[k] = 1;
        else pw[k] = pw[k - 1] * c;
        Z sum = 0;
        for (int i = 0; i <= k; i++)
        {
            sum += egf[i] * pw[k - i] / fact[k - i];
        }
        val[k] = sum * fact[k];
    }
    vector<Z> ans(n + 1), cur{1};
    Z fac = 1;
    for (int k = 0; k <= n; k++)
    {
        Z d = val[0] / fac;
        for (int i = 0; i < (int)cur.size(); i++)
        {
            ans[i] += d * cur[i];
        }
        for (int i = 0; i + 1 < (int)val.size(); i++)
        {
            val[i] = val[i + 1] - val[i];
        }
        val.pop_back();
        if (k == n)
        {
            break;
        }
        fac *= k + 1;
        vector<Z> nxt(cur.size() + 1);
        for (int i = 0; i < (int)cur.size(); i++)
        {
            nxt[i] -= cur[i] * k;
            nxt[i + 1] += cur[i];
        }
        cur.swap(nxt);
```

```
    }
    return ans;
}

inline void tutteRanked(vector<Z> &a, int n, bool inverse)
{
    // a 按子集和次数压平存储，n 是位数；执行分层 Zeta 或 Mobius 变换。
    int N = 1 << n, w = n + 1;
    for (int bit = 0; bit < n; bit++)
    {
        for (int s = 0; s < N; s++)
        {
            if (s >> bit & 1)
{
Z *to = a.data() + (size_t)s * w;
Z *from = a.data() + (size_t)(s ^ (1 << bit)) * w;
for (int d = 0; d <= n; d++)
{
if (inverse) to[d] -= from[d]; else to[d] += from[d];
}
}
        }
    }
}

inline vector<Z> tutteSetExp(const vector<Z> &f, int n)
{
    // f 是常数项为 0 的 n 位集合幂级数；返回集合卷积意义下的 exp(f)。
    int N = 1 << n, w = n + 1;
    vector<Z> a((size_t)N * w);
    for (int s = 0; s < N; s++)
    {
        a[(size_t)s * w + popcount((unsigned)s)] = f[s];
    }
    tutteRanked(a, n, false);
    vector<Z> log(w);
    for (int s = 0; s < N; s++)
    {
        Z *v = a.data() + (size_t)s * w;
        copy(v, v + w, log.begin());
        v[0] = 1;
        for (int d = 1; d <= n; d++)
        {
            Z sum = 0;
            for (int k = 1; k <= d; k++)
            {
                sum += Z(k) * log[k] * v[d - k];
            }
            v[d] = sum / Z(d);
        }
    }
    tutteRanked(a, n, true);
```

```
    vector<Z> ans(N);
    for (int s = 0; s < N; s++)
    {
        ans[s] = a[(size_t)s * w + popcount((unsigned)s)];
    }
    return ans;
}

inline Z tutteEval(const vector<unsigned long long> &g, Z x, Z y)
{
    // g 是无自环无向图邻接位集；x、y 是 Tutte 多项式点值，点数不超过 20。
    int n = g.size(), N = 1 << n;
    assert(0 < n && n <= 20); // 调试检查，可删。
    vector<Z> connected(N), temp(N);
    vector<Z> geo(n + 1);
    for (int i = 1; i <= n; i++)
    {
        geo[i] = geo[i - 1] * y + Z(1);
    }
    for (int u = 0; u < n; u++)
    {
        int M = 1 << u;
        for (int s = 0; s < M; s++)
        {
            temp[s] = connected[s] * geo[popcount(g[u] & (unsigned long long)s)];
        }
        vector<Z> add(temp.begin(), temp.begin() + M);
        add = tutteSetExp(add, u);
        for (int s = 0; s < M; s++)
        {
            connected[M | s] = add[s];
        }
    }
    vector<char> vis(n), hasRoot(N);
    for (int start = 0; start < n; start++)
    {
        if (!vis[start])
{
vector<int> q{start};
vis[start] = 1;
for (int h = 0; h < (int)q.size(); h++)
{
int u = q[h];
for (int v = 0; v < n; v++)
{
    if ((g[u] >> v & 1) && !vis[v])
vis[v] = 1, q.push_back(v);
}
}
for (int s = 0; s < N; s++)
{
    if (s >> start & 1) hasRoot[s] = 1;
```

```
}
}
    }
    for (int s = 1; s < N; s++)
    {
        if (!hasRoot[s]) connected[s] *= x - Z(1);
    }
    return tutteSetExp(connected, n).back();
}
```

先准备 1.1.008 静态取模类的素数域类型 `Z`，并要求模数严格大于 n。`chromPoly(g)` 返回色多项式系数；只需 Tutte 多项式在一点 (x,y) 的值时调用 `tutteEval(g,x,y)`，它复用分层 Zeta/Mobius 与集合幂级数指数，允许图不连通。两者均要求无自环简单无向图、$n \le 20$，时间 $O(n^2 2^n)$、空间 $O(n2^n)$。

5.10.011 拟阵交

拟阵交解决「在同一底集上选一个最大的、同时在两个拟阵 M_1、M_2 中都独立的子集」。很多经典问题是它的特例：二分图最大匹配（两侧各一个划分拟阵）、有向图最大公共森林、带颜色约束的生成树等。算法沿「交换图」增广：当前公共独立集为 I，对每个 $y \notin I$ 若 $I+y$ 在 M_1 中独立就作为源、在 M_2 中独立就作为汇；图中的边表示交换关系——删掉 I 中某个 x 后能否换进 y（保持某个拟阵独立）。找一条从源到汇的最短增广路，沿路翻转元素归属，$|I|$ 加一；找不到路时 I 即为最大。正确性由拟阵交的 min-max（König 型）定理保证。此实现用独立性回调 `first`、`second`（各接收一个 `vector<bool>` 选择标记、返回是否独立），对任意拟阵通用。

```
template <class M1, class M2>
vector<int> matroidMeet(int n, M1 first, M2 second)
{
    // n 是底集大小，first、second 判定选择集在两个拟阵中是否独立；返回升序最大公共独立集。
    vector<bool> in(n);
    while (true)
    {
        vector<int> pre(n, -2), q;
        for (int y = 0; y < n; y++)
        {
            if (!in[y])
            {
                auto a = in;
                a[y] = true;
                if (first(a))
                {
                    pre[y] = -1;
                    q.push_back(y);
                }
            }
        }
        int last = -1;
        for (int h = 0; h < (int)q.size() && last < 0; h++)
        {
            int x = q[h];
```

```
            if (!in[x])
            {
                auto a = in;
                a[x] = true;
                if (second(a))
                {
                    last = x;
                    break;
                }
                for (int y = 0; y < n; y++)
                {
                    if (in[y] && pre[y] == -2)
                    {
                        a[y] = false;
                        if (second(a))
                        {
                            pre[y] = x;
                            q.push_back(y);
                        }
                        a[y] = true;
                    }
                }
            }
            else
            {
                for (int y = 0; y < n; y++)
                {
                    if (!in[y] && pre[y] == -2)
                    {
                        auto a = in;
                        a[x] = false;
                        a[y] = true;
                        if (first(a))
                        {
                            pre[y] = x;
                            q.push_back(y);
                        }
                    }
                }
            }
        }
        if (last < 0)
        {
            break;
        }
        for (int x = last; x >= 0; x = pre[x])
        {
            in[x] = !in[x];
        }
    }
    vector<int> ans;
    for (int i = 0; i < n; i++)
```

```
    {
        if (in[i])
        {
            ans.push_back(i);
        }
    }
    return ans;
}
```

`matroidMeet(n, first, second)` 中 n 是底集大小、两个回调是独立性谕示（oracle），返回最终公共独立集的元素编号。增广轮数为 $O(r)$（r 是答案大小），每轮 $O(n^2)$ 次建边、每次建边调用一次独立性谕示，故独立性调用次数约 $O(r\,n^2)$，实际耗时取决于谕示自身复杂度（多为 $O(n)$ 或更高）。这是无权版；带权拟阵交需要给交换边赋代价并维护势函数，反复在交换图上求最短增广路，不能只按当前 BFS 层数或元素数贪心，本实现未含权。若两个拟阵恰是二分图两侧的划分拟阵，直接用更快的 5.6.003 Hopcroft-Karp；三个拟阵的交是 NP 难，无多项式算法。

最小调用可用两个 “集合大小不超过 2” 的拟阵谕示：

```
signed main()
{
    auto first = [](const vector<bool> &a)
    {
        return count(a.begin(), a.end(), true) <= 2;
    };
    auto second = [](const vector<bool> &a)
    {
        return !(a[0] && a[1]);
    };
    auto ans = matroidMeet(3, first, second);
    assert(ans.size() == 2);
}
```

5.10.012 小规模暴力双极定向

双极定向（bipolar orientation / st-orientation）把一个含边 s-t 的双连通无向图定向成无环图，使得 s 是唯一源（无入边）、t 是唯一汇（无出边），其余每个点都同时有入边和出边。它等价于求一个 st-编号：把点排成序列 $s = p_0, p_1, \dots, p_{n-1} = t$，使每个中间点 p_i 既有编号更小的邻居又有编号更大的邻居，然后每条边由编号小的一端指向大的一端。理论上 st-编号可由耳分解或 Lempel-Even-Cederbaum 在 $O(n+m)$ 内构造；这里给的是小规模（$n \le 12$）的直接实现——DFS 枚举从 s 到 t 的排列，末尾校验每个中间点两侧都有邻居，成立即按位置定向。它逻辑短，适合小数据直接求解，规模大时应换线性算法。

```
vector<pair<int, int>> bipolarOrd(const vector<vector<int>> &g, int s, int t)
{
    // g 是简单无向图；返回以 s 为唯一源、t 为唯一汇的边方向，无解返回空。
    int n = g.size();
    assert(n <= 12 && 0 <= s && s < n && 0 <= t && t < n && s != t); // 调试检查，可删。
    vector<unsigned long long> adj(n);
    for (int u = 0; u < n; u++)
```

```
    {
        for (int v : g[u])
        {
            assert(0 <= v && v < n && u != v); // 调试检查，可删。
            adj[u] |= 1ULL << v;
        }
    }
    for (int u = 0; u < n; u++)
    {
        for (int v = 0; v < n; v++)
        {
            assert((adj[u] >> v & 1) == (adj[v] >> u & 1)); // 调试检查，可删。
        }
    }
    if (!(adj[s] >> t & 1))
    {
        return {};
    }
    vector<int> p{s};
    vector<bool> use(n);
    use[s] = true;
    use[t] = true;
    auto dfs = [&](auto &&self) -> bool
    {
        if ((int)p.size() == n - 1)
        {
            p.push_back(t);
            unsigned long long left = 1ULL << s;
            for (int i = 1; i + 1 < n; i++)
            {
                int u = p[i];
                unsigned long long right = 0;
                for (int j = i + 1; j < n; j++)
                {
                    right |= 1ULL << p[j];
                }
                if (!(adj[u] & left) || !(adj[u] & right))
                {
                    p.pop_back();
                    return false;
                }
                left |= 1ULL << u;
            }
            return true;
        }
        for (int v = 0; v < n; v++)
        {
            if (!use[v])
            {
                use[v] = true;
                p.push_back(v);
                if (self(self))
```

```
                {
                    return true;
                }
                p.pop_back();
                use[v] = false;
            }
        }
        return false;
    };
    if (!dfs(dfs))
    {
        return {};
    }
    vector<int> at(n);
    for (int i = 0; i < n; i++)
    {
        at[p[i]] = i;
    }
    vector<pair<int, int>> ans;
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u])
        {
            if (u < v)
            {
                ans.push_back(at[u] < at[v] ? pair{u, v} : pair{v, u});
            }
        }
    }
    return ans;
}
```

`bipolarOrd(g, s, t)` 要求简单无向图含边 $s-t$ 且 $n \le 12$，返回 `{u,v}` 表示定向边 $u \to v$；无解返回空。有效 st-编号中每个中间点都有更小和更大的邻点，按编号从小到大定向后自然无环，并以 s,t 为唯一源汇。暴力最坏 $O(n!(n+m))$，只适合小图；它直接返回有向边，逻辑短，适合小数据或复核。规模较大时使用下一条线性版本；双连通分解见 5.3.005 割点与点双连通分量。

5.10.013 线性 st 编号

一般的 st 编号不必先假定整张图双连通：只要存在一个排列使 `s` 为首点、`t` 为末点，其余每点都同时有更小和更大的邻点即可。算法先把 `s` 标为已访问，再从 `t` 开始 DFS，使 DFS 序固定以 `s,t` 开头；`low[v]` 保存从 `v` 的 DFS 子树经至多一条返祖边能到达的最早顶点。初始双向链表只有 `s,t`，随后按 DFS 序处理其余点：由 `low[v]` 的符号决定把 `v` 插在父点前还是父点后，并把相反符号传给父点。lowpoint 的嵌套关系保证存在 st 编号时插入位置始终有效。最后把链表转成排名，并重新检查每个非 `s` 点有更小邻点、每个非 `t` 点有更大邻点；检查失败返回空。

```
optional<vector<int>> stNumbering(const vector<vector<int>> &g, int s, int t)
{
    // g 是简单无向图；返回 st 编号的排名，无法构造时返回空。
```

```
    int n = g.size();
    assert(n >= 2 && 0 <= s && s < n && 0 <= t && t < n && s != t); // 调试检查，可删。
    vector<int> pre(n, -1), low(n), par(n, -1), order;
    pre[s] = 0;
    low[s] = s;
    order.push_back(s);
    pre[t] = 1;
    low[t] = t;
    order.push_back(t);
    auto dfs = [&](auto &&self, int u) -> void
    {
        for (int v : g[u])
        {
            assert(0 <= v && v < n && v != u); // 调试检查，可删。
            if (pre[v] == -1)
            {
                pre[v] = order.size();
                low[v] = v;
                par[v] = u;
                order.push_back(v);
                self(self, v);
                if (pre[low[v]] < pre[low[u]])
                {
                    low[u] = low[v];
                }
            }
            else if (pre[v] < pre[low[u]])
            {
                low[u] = v;
            }
        }
    };
    dfs(dfs, t);
    if ((int)order.size() != n)
    {
        return nullopt;
    }

    vector<int> prev(n, -1), next(n, -1), sign(n);
    next[s] = t;
    prev[t] = s;
    sign[s] = -1;
    for (int i = 2; i < n; i++)
    {
        int v = order[i], p = par[v];
        if (p == -1)
        {
            return nullopt;
        }
        if (sign[low[v]] == -1)
        {
            int q = prev[p];
```

```
            if (q == -1)
            {
                return nullopt;
            }
            next[q] = v;
            next[v] = p;
            prev[v] = q;
            prev[p] = v;
            sign[p] = 1;
        }
        else
        {
            int q = next[p];
            if (q == -1)
            {
                return nullopt;
            }
            next[p] = v;
            next[v] = q;
            prev[v] = p;
            prev[q] = v;
            sign[p] = -1;
        }
    }

    vector<int> rank(n, -1);
    int u = s;
    for (int i = 0; i < n && u != -1; i++)
    {
        rank[u] = i;
        u = next[u];
    }
    if (u != -1 || rank[t] != n - 1 || count(rank.begin(), rank.end(), -1))
    {
        return nullopt;
    }
    for (int v = 0; v < n; v++)
    {
        bool left = false, right = false;
        for (int w : g[v])
        {
            assert(0 <= w && w < n && w != v); // 调试检查，可删。
            left |= rank[w] < rank[v];
            right |= rank[v] < rank[w];
        }
        if ((v != s && !left) || (v != t && !right))
        {
            return nullopt;
        }
    }
    return rank;
}
```

`stNumbering(g, s, t)` 返回长度为 n 的排名数组，`rank[s]=0`、`rank[t]=n-1`；按排名从小到大给每条边定向即可得到以 `s` 为唯一源、`t` 为唯一汇的无环定向。图不连通或不存在这种编号时返回空。实现确定、无随机失败，DFS 使用递归 lambda，时间 $O(n+m)$、空间 $O(n+m)$，深链需确认系统栈足够。它比上一条暴力版适用范围更广、复杂度更低，但只返回排名，不直接整理有向边；需要边表时遍历原边比较两端排名即可。

5.10.014 宽度 2 树分解

树分解把原图每个点放进若干个 bag，要求每条原边的两端同处某个 bag，且包含同一原图点的 bag 在分解树上连通；最大 bag 大小减一就是宽度。树宽至多 2 的图可用低度消元识别并构造：反复取当前度数不超过 2 的点 `v`，记录由 `v` 和剩余邻点组成的 bag；若恰有两个邻点，先给它们补边，使这两个点成为后续 bag 中必须共同出现的分隔集。若仍有存活点却再找不到低度点，则不存在宽度 2 分解。逆序恢复时，单邻点 bag 挂到一个包含该点的既有 bag，双邻点 bag 挂到一个同时包含这条补边两端的既有 bag；不同连通块的根 bag 可任意相连。

```cpp
struct TreeDecomp2
{
    // bag 是各树节点包含的原图点集，edge 是 bag 树的边。
    vector<vector<int>> bag;
    vector<pair<int, int>> edge;
};

optional<TreeDecomp2> treeDecomp2(int n, const vector<pair<int, int>> &edge)
{
    // n 是点数，edge 是简单无向边；返回宽度至多 2 的树分解，不存在时返回空。
    vector<set<int>> g(n);
    for (auto [u, v] : edge)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v); // 调试检查，可删。
        g[u].insert(v);
        g[v].insert(u);
    }
    struct Event
    {
        int d, v, a, b; // d 是删除时度数，v 是被删点，a、b 是它的邻居（不存在为 -1）。
    };
    vector<Event> ev;
    vector<bool> alive(n, true);
    queue<int> q;
    for (int u = 0; u < n; u++)
    {
        q.push(u);
    }
    while (!q.empty())
    {
        int v = q.front();
        q.pop();
        if (!alive[v] || g[v].size() >= 3)
        {
```

```
            continue;
        }
        int d = g[v].size(), a = -1, b = -1;
        if (d >= 1)
        {
            a = *g[v].begin();
        }
        if (d == 2)
        {
            b = *g[v].rbegin();
            g[a].insert(b);
            g[b].insert(a);
        }
        ev.push_back({d, v, a, b});
        alive[v] = false;
        for (int u : g[v])
        {
            g[u].erase(v);
            q.push(u);
        }
    }
    if (count(alive.begin(), alive.end(), true))
    {
        return nullopt;
    }

    reverse(ev.begin(), ev.end());
    TreeDecomp2 ans;
    vector<int> one(n, -1);
    map<pair<int, int>, int> two;
    for (auto [d, v, a, b] : ev)
    {
        int id = ans.bag.size(), p = -1;
        ans.bag.push_back({v});
        if (d >= 1)
        {
            ans.bag.back().push_back(a);
        }
        if (d == 2)
        {
            ans.bag.back().push_back(b);
            auto it = two.find(minmax(a, b));
            if (it == two.end())
            {
                return nullopt;
            }
            p = it->second;
        }
        else if (d == 1)
        {
            p = one[a];
            if (p < 0)
```

```
            {
                return nullopt;
            }
        }
        else if (id > 0)
        {
            p = id - 1;
        }
        if (p >= 0)
        {
            ans.edge.push_back({p, id});
        }
        for (int x : ans.bag.back())
        {
            one[x] = id;
        }
        for (int x : ans.bag.back())
        {
            for (int y : ans.bag.back())
            {
                if (x < y)
                {
                    two[{x, y}] = id;
                }
            }
        }
    }
    return ans;
}
```

`treeDecomp2(n, edge)` 要求简单无向图；成功时每个 bag 至多含 3 点，`.edge` 连接 bag 编号并构成一棵树，空图返回一个有效的空分解。失败返回空，说明图的树宽大于 2。`set` 使实现确定且不依赖哈希，时间 $O((n+m)\log n)$、空间 $O(n+m)$；若题目严格卡线性复杂度，可把动态邻接集合换成专门的小度邻接结构。返回结果可直接用于在树宽 2 图上做 bag DP。

5.10.015 Ramsey 定理

Ramsey 数 $R(s,t)$ 是「使得任意对完全图 K_n 的边二染色（红/蓝）都必然出现红色 K_s 或蓝色 K_t」的最小 n。它把「足够大的结构必含规则子结构」量化：$R(3,3)=6$ 是最著名的小值（6 个人中必有 3 人两两相识或两两不识），另有 $R(4,4)=18$，而 $R(5,5)$ 至今未知（只知在 $[43,46]$ 区间）。一般上界 $R(s,t)\le\binom{s+t-2}{s-1}$。竞赛里通常不需要算 Ramsey 数本身，而是用它的存在性保证：在规模达标的完全图上一定能找到单色团。下面给出对应 $R(3,3)=6$ 的构造性验证——在任意 $0/1$ 边染色的完全图里枚举三点找同色三角形，$n\ge 6$ 时必然找到。切忌把未确定的 Ramsey 数当精确常数用。

```
namespace ramsey
{
optional<array<int, 3>> monoTriangle(const vector<vector<int>> &col)
{
    // col 是完全图的对称 0/1 边染色；返回一个单色三角形，不存在时返回空。
```

```
    int n = col.size();
    for (int i = 0; i < n; i++)
    {
        assert((int)col[i].size() == n); // 调试检查，可删。
    }
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            assert((col[i][j] == 0 || col[i][j] == 1) &&
                   col[i][j] == col[j][i]); // 调试检查，可删。
            for (int k = j + 1; k < n; k++)
            {
                if (col[i][j] == col[i][k] && col[i][j] == col[j][k])
                {
                    return array<int, 3>{i, j, k};
                }
            }
        }
    }
    return nullopt;
}
}
```

`monoTriangle(col)` 传对称的 $0/1$ 边染色矩阵（`col[i][j]` 为该边颜色，对角线任意），返回一个同色三角形的三个顶点，不存在则返回 `nullopt`；当 $n \geq 6$ 时由 $R(3,3)=6$ 保证一定非空。枚举三点 $O(n^3)$、$O(1)$ 额外空间。要找更大的单色团（对应 $R(s,t)$）时枚举不再可行，得转成一般 5.10.006 最大团、极大团计数 分别在红图和蓝图上求团；构造证明 Ramsey 下界的极端染色（如 $R(4,4)$ 的 Paley 图）则是另一类专门构造。

5.10.016 外环加弦平面性判定（2-SAT）

这里的 “平面性” 有明确限制：原图是一条固定圆环，输入的每条额外边都是一条弦，只需要决定每条弦画在环内还是环外。两条端点交错的弦不能在同一侧，否则必相交；端点包含或互不交错的弦在同一侧可以嵌套或分离。于是对每一对交错弦加入两个子句，强制它们选相反侧，再调用 5.4.001 Tarjan 2-SAT。这不是任意图的平面图识别，不能把它推广成普适结论。

```
// 依赖 5.4.001 Tarjan 2-SAT
optional<vector<bool>> outerChordOK(int n, const vector<pair<int, int>> &chord)
{
    // n 是环上顶点数；chord[i] 是第 i 条弦的两个不同端点，编号按环顺序。
    assert(n >= 3);
    TarjanTwoSAT sat(chord.size());
    vector<pair<int, int>> a = chord;
    for (auto &[u, v] : a)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        if (u > v)
        {
```

```
            swap(u, v);
        }
    }
    for (int i = 0; i < (int)a.size(); i++)
    {
        for (int j = i + 1; j < (int)a.size(); j++)
        {
            auto [u, v] = a[i];
            auto [x, y] = a[j];
            bool cross = (u < x && x < v && v < y) || (x < u && u < y && y < v);
            if (cross)
            {
                sat.addOr(i, false, j, false);
                sat.addOr(i, true, j, true);
            }
        }
    }
    return sat.solve();
}
```

`outerChordOK(n,chord)` 返回每条弦的一组内外侧选择;无解表示无论怎样分侧都会有交叉。建模 $O(m^2)$，2-SAT 求解为 $O(m^2)$，空间 $O(m^2)$。例如环 $0,1,2,3$ 的弦 $(0,2)$ 与 $(1,3)$ 必须分到两侧:

```
signed main()
{
    auto side = outerChordOK(4, {{0, 2}, {1, 3}});
    assert(side && (*side)[0] != (*side)[1]);
}
```

5.10.017 Havel-Hakimi / Gale-Ryser 构造

Havel-Hakimi：非负整数序列 d 可简单图化，当且仅当去掉最大项 d_u 并把剩下最大的 d_u 项各减 1 后的序列仍可简单图化。交换论证：存在一种实现使度数最大的点恰好连向度数最大的那些点。用大根堆每次取剩余度数最大的点，按该规则连边即可得到一张简单图。

Gale-Ryser：二分图左侧度数 a、右侧度数 b 可实现为简单二分图，当且仅当 $\sum a = \sum b$，且把 a 降序排列后对每个 k 有

$$\sum_{i=1}^{k} a_i \le \sum_{j} \min(b_j, k).$$

`galeRyser` 把 b 升序后用双指针累计右端。条件成立时，左侧按度数从大到小、每个点连向右侧剩余度数最大的若干点，这个贪心总能完成构造（`bipDegree`）。

```
optional<vector<pair<int, int>>> havelHakimi(const vector<int> &d)
{
    // d[i] 是点 i 要求的度数；能构造简单无向图时返回其边表，否则返回空。
    int n = d.size();
    priority_queue<pair<int, int>> q; // q 存 (剩余度数, 点)，每次取剩余度数最大的点。
    vector<pair<int, int>> e;
```

```
    for (int i = 0; i < n; i++)
    {
        if (d[i] < 0 || d[i] >= n)
        {
            return nullopt;
        }
        if (d[i])
        {
            q.push({d[i], i});
        }
    }
    while (!q.empty())
    {
        auto [k, u] = q.top();
        q.pop();
        if (k > (int)q.size())
        {
            return nullopt;
        }
        // u 连向剩余度数最大的 k 个点，各自度数减一后放回。
        vector<pair<int, int>> tmp;
        while (k--)
        {
            auto [s, v] = q.top();
            q.pop();
            e.push_back({u, v});
            if (s > 1)
            {
                tmp.push_back({s - 1, v});
            }
        }
        for (auto x : tmp)
        {
            q.push(x);
        }
    }
    return e;
}

bool galeRyser(vector<int> a, vector<int> b)
{
    // a、b 是二分图左右两侧各点要求的度数；返回是否存在对应的简单二分图。
    __int128 sa = 0, sb = 0;
    for (int x : a)
    {
        if (x < 0 || x > (int)b.size())
        {
            return false;
        }
        sa += x;
    }
    for (int x : b)
```

```
    {
        if (x < 0 || x > (int)a.size())
        {
            return false;
        }
        sb += x;
    }
    if (sa != sb)
    {
        return false;
    }
    // a 降序后检查：前 k 大之和 <= sum min(b_j, k)。b 升序以便双指针累计 min。
    sort(a.rbegin(), a.rend());
    sort(b.begin(), b.end());
    __int128 lhs = 0, small = 0; // small 是 b 中小于 k 的那些度数之和。
    int j = 0;
    for (int k = 1; k <= (int)a.size(); k++)
    {
        lhs += a[k - 1];
        while (j < (int)b.size() && b[j] < k)
        {
            small += b[j++];
        }
        if (lhs > small + (__int128)k * (b.size() - j))
        {
            return false;
        }
    }
    return true;
}

optional<vector<pair<int, int>>> bipDegree(const vector<int> &a, const vector<int> &b)
{
    // a、b 同 galeRyser；可构造时返回边表（左点，右点），否则返回空。
    if (!galeRyser(a, b))
    {
        return nullopt;
    }
    vector<pair<int, int>> left, e;
    priority_queue<pair<int, int>> q; // q 存右侧（剩余度数，点）。
    for (int i = 0; i < (int)a.size(); i++)
    {
        left.push_back({a[i], i});
    }
    for (int j = 0; j < (int)b.size(); j++)
    {
        if (b[j])
        {
            q.push({b[j], j});
        }
    }
    // 左侧按度数从大到小，每个点连向右侧剩余度数最大的若干点。
```

```
    sort(left.rbegin(), left.rend());
    for (auto [k, u] : left)
    {
        vector<pair<int, int>> tmp;
        while (k--)
        {
            auto [s, v] = q.top();
            q.pop();
            e.push_back({u, v});
            if (s > 1)
            {
                tmp.push_back({s - 1, v});
            }
        }
        for (auto x : tmp)
        {
            q.push(x);
        }
    }
    return e;
}
```

`havelHakimi(d)` 可图化时返回一张简单无向图的边表 (u, v)，否则返回空；`galeRyser(a,b)` 只判定，`bipDegree(a,b)` 可实现时返回边表（左点, 右点）。度数为负或超过另一侧点数时直接不可实现。设边数为 m，`havelHakimi` 与 `bipDegree` 的时间 $O((n+m)\log n)$，`galeRyser` 为 $O(n \log n)$。

只判定一般图度序列时，Erdős-Gallai 条件见 5.10.005 度序列可图性 / Erdős-Gallai；竞赛图比分序列用 5.10.004 竞赛图与 Landau 定理。允许重边或要求连通等附加条件时贪心不再适用，通常改用网络流构造（5.8.005 有上下界可行流、最大流、最小流）。

```
signed main()
{
    auto e = havelHakimi({3, 3, 2, 2, 2});
    assert(e && e->size() == 6 && !havelHakimi({3, 3, 1, 1}));
    auto b = bipDegree({2, 1, 1}, {2, 2});
    assert(b && b->size() == 4 && !galeRyser({2, 2}, {2, 1, 0}));
    vector<int> x(3), y(2);
    for (auto [u, v] : *b)
    {
        x[u]++;
        y[v]++;
    }
    assert(x == vector<int>({2, 1, 1}) && y == vector<int>({2, 2}));
}
```

5.10.018 小图一般图同构

`graphIso` 判定两张小图是否同构并给出映射 p，使 $a_{u,v} = b_{p(u),p(v)}$ 对所有 u, v 成立。邻接矩阵可以有向、带自环，元素是边数或边权。回溯时先按（出度和，入度和）降序固定点的顺序，只在度数与自环值相同的点之间尝试，每确定一个点就检查它与所有已确定点之间的两个方向的边。

graphCanon 枚举全部 $n!$ 个顶点排列，取重排后按行展开的字典序最小矩阵作为规范编码；两图同构当且仅当编码相同，所以可以把很多小图按编码去重或分类。

```
optional<vector<int>> graphIso(const vector<vector<int>> &a, const vector<vector<int>> &b)
{
    // a、b 是方阵邻接矩阵（可有向、带自环与重边计数）；同构时返回映射 p，
    // 满足 a[u][v]==b[p[u]][p[v]]，否则返回空。
    int n = a.size();
    if ((int)b.size() != n)
    {
        return nullopt;
    }
    // da[i]、db[i] 是 (出度和, 入度和)，只在度数相同的点之间尝试匹配。
    vector<pair<__int128, __int128>> da(n), db(n);
    for (int i = 0; i < n; i++)
    {
        assert((int)a[i].size() == n && (int)b[i].size() == n); // 调试检查，可删。
        for (int j = 0; j < n; j++)
        {
            da[i].first += a[i][j];
            da[i].second += a[j][i];
            db[i].first += b[i][j];
            db[i].second += b[j][i];
        }
    }
    vector<int> p(n, -1), order(n);
    vector<char> used(n);
    // 度数大的点先定，剪枝更早生效。
    iota(order.begin(), order.end(), 0);
    sort(order.begin(), order.end(), [&](int x, int y)
    {
        return da[x] > da[y];
    });
    auto dfs = [&](auto &&self, int k) -> bool
    {
        // 已为 order[0..k) 确定像，尝试为 order[k] 选像。
        if (k == n)
        {
            return true;
        }
        int u = order[k];
        for (int v = 0; v < n; v++)
        {
            if (used[v] || da[u] != db[v] || a[u][u] != b[v][v])
            {
                continue;
            }
            bool ok = true;
            for (int j = 0; j < k && ok; j++)
            {
                int x = order[j];
                ok = a[u][x] == b[v][p[x]] && a[x][u] == b[p[x]][v];
```

```
            }
            if (!ok)
            {
                continue;
            }
            p[u] = v;
            used[v] = true;
            if (self(self, k + 1))
            {
                return true;
            }
            used[v] = false;
        }
        return false;
    };
    if (dfs(dfs, 0))
    {
        return p;
    }
    return nullopt;
}

pair<vector<int>, vector<int>> graphCanon(const vector<vector<int>> &a)
{
    // a 是 n<=10 的方阵邻接矩阵；枚举全部顶点排列，返回字典序最小的重排矩阵（按行展开）
    // 以及取到它的顶点顺序。两图同构当且仅当规范编码相同。
    int n = a.size();
    assert(n <= 10); // 调试检查，可删。
    vector<int> p(n), best, order;
    iota(p.begin(), p.end(), 0);
    do
    {
        vector<int> code;
        for (int u : p)
        {
            for (int v : p)
            {
                code.push_back(a[u][v]);
            }
        }
        if (order.empty() || code < best)
        {
            best = move(code);
            order = p;
        }
    } while (next_permutation(p.begin(), p.end()));
    return {best, order};
}
```

`graphIso(a,b)` 要求两个矩阵都是方阵，阶数不同直接返回空。最坏时间 $O(n!\,n^2)$，度数剪枝对随机图通常有效，但正则图等对称结构仍可能接近最坏。`graphCanon(a)` 要求 $n \le 10$，时间 $O(n!\,n^2)$，返回编码与取到它的顶点顺序。

树同构见 6.1.008 树哈希与基环树哈希；大规模一般图同构需要 nauty/Traces 一类的划分细化算法，这里只适合小图和打表。下面把有向路 $0\to1\to2$ 与 $2\to0\to1$ 比较：

```cpp
signed main()
{
    vector<vector<int>> a{{0, 1, 0}, {0, 0, 1}, {0, 0, 0}}, b{{0, 1, 0}, {0, 0, 0}, {1, 0, 0}};
    auto p = graphIso(a, b);
    assert(p && *p == vector<int>({2, 0, 1}));
    assert(graphCanon(a).first == graphCanon(b).first);
    assert(!graphIso(a, {{0, 1, 1}, {0, 0, 0}, {0, 0, 0}}));
}
```

6. 树论与动态树

6.1 树的基础性质

6.1.001 树的直径、重心、中心

树上三个 “中心性” 对象定义互不相同，混用会直接出错：直径是最远点对及其路径，重心是删除后最大连通块最小的点，中心是到所有点最大距离最小的点。`treeDiameter` 用两遍搜索——先从任意点找到最远点 s，再从 s 出发找最远点 t，则 s-t 就是一条直径，这套两遍法只在非负边权下成立。`centroids` 按子树大小判断重心；`treeCenters` 反复剥去叶子，最后剩下中心。重心可能有一或两个，与直径奇偶没有直接对应关系；按边数计的无权树直径长度为偶数时有一个中心，为奇数时有两个中心。

```cpp
template <class Dist = long long>
struct Dia
{
    // dist 是直径长度，path 是一条直径上的完整点列。
    Dist dist;
    vector<int> path;
};

template <class Dist = long long>
Dia<Dist> treeDiameter(const vector<vector<pair<int, long long>>> &g)
{
    // g 是非负边权树；返回直径长度与一条直径路径。
    static_assert(numeric_limits<Dist>::is_integer && numeric_limits<Dist>::is_signed &&
                  sizeof(Dist) >= sizeof(long long));
    int n = g.size();
    if (!n)
    {
        return Dia<Dist>{0, {}};
    }
    auto far = [&](int s)
    {
        vector<int> p(n, -1);
        vector<Dist> d(n, -1);
        d[s] = 0;
        auto dfs = [&](auto &&self, int u) -> void
        {
            for (auto [v, w] : g[u])
            {
                if (v == p[u])
                {
                    continue;
                }
                assert(w >= 0); // 调试检查，可删。
                p[v] = u;
                d[v] = d[u] + (Dist)w;
                self(self, v);
            }
```

```
        };
        dfs(dfs, s);
        int u = max_element(d.begin(), d.end()) - d.begin();
        return tuple{u, move(p), move(d)};
    };
    auto first = far(0);
    int s = get<0>(first);
    auto second = far(s);
    auto [t, p, d] = move(second);
    vector<int> path;
    for (int u = t; u != -1; u = p[u])
    {
        path.push_back(u);
    }
    reverse(path.begin(), path.end());
    return Dia<Dist>{d[t], path};
}

vector<int> centroids(const vector<vector<int>> &g)
{
    // g 是无权树；返回一个或两个重心。
    int n = g.size();
    if (!n)
    {
        return {};
    }
    vector<int> sz(n), ans;
    auto dfs = [&](auto &&self, int u, int p) -> void
    {
        sz[u] = 1;
        int mx = 0;
        for (int v : g[u])
        {
            if (v == p)
            {
                continue;
            }
            self(self, v, u);
            sz[u] += sz[v];
            chmax(mx, sz[v]);
        }
        chmax(mx, n - sz[u]);
        if (2 * mx <= n)
        {
            ans.push_back(u);
        }
    };
    dfs(dfs, 0, -1);
    return ans;
}

vector<int> treeCenters(const vector<vector<int>> &g)
```

```
{
    // g 是无权树；返回一个或两个中心。
    int n = g.size();
    if (n <= 2)
    {
        vector<int> a(n);
        iota(a.begin(), a.end(), 0);
        return a;
    }
    vector<int> d(n);
    queue<int> q;
    for (int u = 0; u < n; u++)
    {
        d[u] = g[u].size();
        if (d[u] == 1)
        {
            q.push(u);
        }
    }
    int left = n;
    while (left > 2)
    {
        int z = q.size();
        left -= z;
        while (z--)
        {
            int u = q.front();
            q.pop();
            for (int v : g[u])
            {
                if (--d[v] == 1)
                {
                    q.push(v);
                }
            }
        }
    }
    vector<int> ans;
    while (!q.empty())
    {
        ans.push_back(q.front());
        q.pop();
    }
    return ans;
}
```

`treeDiameter` 的 `Dist` 默认是 `long long`，且不得窄于输入边权类型；调用者须保证直径长度能由 `Dist` 表示，若可能超过 64 位可用 `treeDiameter<__int128>`。`centroids`、`treeCenters` 分别返回无权树的重心和中心。三者均为 $O(n)$ 时间与空间；仅对无权树，直径中间的一个或两个顶点才是中心。

6.1.002 最近公共祖先：倍增

倍增法预处理 `up[k][u]` 为 u 的 2^k 级祖先，满足 $up[k][u] = up[k-1][up[k-1][u]]$。查询时先用进出时间戳判祖先关系（$in[u] \le in[v] \wedge out[v] \le out[u]$），若互不为祖先，就把 u 从高位往低位跳到“仍不是 v 祖先”的最高点，其父亲即 LCA。它面向静态有根树的在线询问；一旦连边断边就必须整表重建。根的 2^k 级祖先取自身（自环），使 `jump` 越界安全；`jump` 越过根返回 -1，`pathKth` 把路径第 k 点按 LCA 拆成两段各做一次 `jump`。

```
struct LCA
{
    // n 是点数，lg 是倍增层数，tim 是前序计时器。
    int n, lg, tim = 0;
    // dep 是深度，in/out 是子树前序区间的两端。
    vector<int> dep, in, out;
    // up[k][u] 是 u 的 2^k 级祖先。
    vector<vector<int>> up;

    LCA(const vector<vector<int>> &g, int root = 0)
       : n(g.size()), lg(bit_width((unsigned long long)max<int>(1, n))), dep(n), in(n), out(n),
         up(lg, vector<int>(n))
    {
        // g 是树，root 是根；空树只建立空表。
        assert(n == 0 || (0 <= root && root < n)); // 调试检查，可删
        if (!n)
        {
            return;
        }
        auto dfs = [&](auto &&self, int u, int p) -> void
        {
            in[u] = tim++;
            up[0][u] = p;
            for (int k = 1; k < lg; k++)
            {
                up[k][u] = up[k - 1][up[k - 1][u]];
            }
            for (int v : g[u])
            {
                if (v == p)
                {
                    continue;
                }
                dep[v] = dep[u] + 1;
                self(self, v, u);
            }
            out[u] = tim;
        };
        dfs(dfs, root, root);
    }

    bool anc(int u, int v) const
    {
```

```
        // 返回 u 是否为 v 的祖先，节点编号须合法。
        return in[u] <= in[v] && out[v] <= out[u];
    }

    int jump(int u, int d) const
    {
        // 返回 u 的第 d 级祖先；越过根返回 -1。
        assert(d >= 0); // 调试检查，可删
        if (d > dep[u])
        {
            return -1;
        }
        for (int k = 0; d; k++, d >>= 1)
        {
            if (d & 1)
            {
                u = up[k][u];
            }
        }
        return u;
    }

    int lca(int u, int v) const
    {
        // 返回 u、v 的最近公共祖先。
        if (anc(u, v))
        {
            return u;
        }
        if (anc(v, u))
        {
            return v;
        }
        for (int k = lg - 1; k >= 0; k--)
        {
            if (!anc(up[k][u], v))
            {
                u = up[k][u];
            }
        }
        return up[0][u];
    }

    int dist(int u, int v) const
    {
        // 返回 u、v 间的边数。
        int p = lca(u, v);
        return dep[u] + dep[v] - 2 * dep[p];
    }

    int pathKth(int u, int v, int k) const
    {
```

```
        // 返回 u 到 v 路径上从 0 开始的第 k 个点；越界返回 -1。
        int p = lca(u, v), a = dep[u] - dep[p], b = dep[v] - dep[p];
        if (k < 0 || k > a + b)
        {
            return -1;
        }
        return k <= a ? jump(u, k) : jump(v, a + b - k);
    }
};
```

构造 `LCA(g, root)`；`lca`、`dist`、`jump`、`pathKth` 均 $O(\log n)$，其中 `dist` 返回边数（带权距离需另维护根距），`jump`/`pathKth` 越界返回 -1。预处理 $O(n \log n)$ 时间与空间。这里的 `in`/`out` 时间戳判祖先技巧被 6.3.003 虚树 直接复用；只读且询问量极大时改用 6.1.004 最近公共祖先：ST 表 / RMQ 的 $O(1)$ 查询，若能离线则用 6.1.005 最近公共祖先：Tarjan 离线 省去每次 log。

倍增、RMQ 与离线 Tarjan 三版共用的固定检查放在 6.1.005 最近公共祖先：Tarjan 离线 后，期望值均可直接手算。

6.1.003 最近公共祖先：树链剖分

已经为换根或路径查询建好重链剖分时，求 LCA 不必再开倍增表：沿重链把链顶更深的一点跳到其链顶的父亲，两点进入同一条链后深度较小者即 LCA，每次询问 $O(\log n)$、零额外预处理。本条直接复用 6.3.002 换根树剖 的 `RerootHLD`，调用其 `lca(u, v)` 即可；若只要 LCA 而不需换根，6.3.001 轻重链剖分 / HLD 的 `HLD::lca` 更轻。相比 6.1.002 最近公共祖先：倍增，树剖版常数更小、省去 $O(n \log n)$ 空间，但同样只适用于静态树。

6.1.004 最近公共祖先：ST 表 / RMQ

把 LCA 归约成欧拉序上的区间最小值：DFS 游走记录 (深度，节点) 序列与每点首次出现位置 `first`，两点 LCA 就是它们首现位置之间深度最小的那一项。区间最小用幂等的 ST 表（比较器 `MinDepth` 取 `pair` 的 `min`，深度在前保证按深度比较），因此查询 $O(1)$。`ST` 只适用于 `min`/`max`/`gcd`/`and`/`or` 这类幂等运算，查询区间约定为 $[l, r)$。它适合询问量巨大、可接受 $O(n \log n)$ 预处理空间的只读场景。

```
template <class T, class F> struct ST
{
    // n 是序列长度，f 是幂等合并函数，a[k][i] 维护长 2^k 的区间。
    int n;
    F f;
    vector<vector<T>> a;

    ST(const vector<T> &v, F f = {}) : n(v.size()), f(f)
    {
        // v 是原序列，f 合并两个区间结果。
        int m = n ? bit_width((unsigned long long)n) : 0;
        a.assign(m, vector<T>(n));
        if (!n)
        {
```

```
            return;
        }
        a[0] = v;
        for (int k = 1; k < m; k++)
        {
            for (int i = 0; i + (1LL << k) <= n; i++)
            {
                a[k][i] = f(a[k - 1][i], a[k - 1][i + (1LL << (k - 1))]);
            }
        }
    }

    // 仅适用于幂等运算（min/max/gcd/and/or），查询 [l,r)。
    T query(int l, int r) const
    {
        // 返回半开区间 [l,r) 的合并结果。
        assert(0 <= l && l < r && r <= n); // 调试检查，可删
        int k = bit_width((unsigned long long)(r - l)) - 1;
        return f(a[k][l], a[k][r - (1LL << k)]);
    }
};

struct MinDepth
{
    pair<int, int> operator()(pair<int, int> a, pair<int, int> b) const
    {
        // 返回深度较小的一项；深度相同则取节点编号较小者。
        return min(a, b);
    }
};

struct LCARMQ
{
    // first[u] 是 u 在欧拉游走序列中的首次出现位置。
    vector<int> first;
    // st 维护欧拉序上的最小深度项。
    ST<pair<int, int>, MinDepth> st;

    static pair<vector<int>, vector<pair<int, int>>> tour(const vector<vector<int>> &g, int
    ↪ root)
    {
        // 返回 {首次出现位置，(深度，节点) 欧拉游走序列}。
        assert(!g.empty() && 0 <= root && root < (int)g.size()); // 调试检查，可删
        vector<int> first(g.size(), -1);
        vector<pair<int, int>> a;
        auto dfs = [&](auto &&self, int u, int p, int d) -> void
        {
            first[u] = a.size();
            a.push_back({d, u});
            for (int v : g[u])
            {
                if (v == p)
```

```
                {
                    continue;
                }
                self(self, v, u, d + 1);
                a.push_back({d, u});
            }
        };
        dfs(dfs, root, -1, 0);
        return {first, a};
    }

    LCARMQ(const vector<vector<int>> &g, int root = 0) : LCARMQ(tour(g, root))
    {
        // g 是非空树，root 是根。
    }

    int lca(int x, int y) const
    {
        // 返回 x、y 的最近公共祖先。
        int l = first[x], r = first[y];
        if (l > r)
        {
            swap(l, r);
        }
        return st.query(l, r + 1).second;
    }

  private:
    LCARMQ(pair<vector<int>, vector<pair<int, int>>> p) : first(move(p.first)), st(p.second,
    ↪ MinDepth{})
    {
        // p 保存首次出现位置与欧拉游走序列。
    }
};
```

构造 `LCARMQ(g, root)` 后 `lca(x, y)` 为 $O(1)$；欧拉序长 $2n-1$，ST 构造 $O(n\log n)$ 时间与空间。相比 6.1.002 最近公共祖先：倍增 查询更快，但空间常数更大，且不直接给 `dist`/`jump`；需要 K 级祖先或路径第 k 点仍回到 6.1.002 或 6.1.006 树上距离、K 级祖先。

单独使用本条 `ST` 时也要显式传入幂等合并对象：

```
signed main()
{
    vector<pair<int, int>> a{{2, 0}, {1, 3}, {4, 2}};
    ST<pair<int, int>, MinDepth> st(a, MinDepth{});
    assert((st.query(0, 3) == pair<int, int>{1, 3}));
}
```

固定检查与倍增版合并放在 6.1.005 最近公共祖先：Tarjan 离线 后。

6.1.005 最近公共祖先：Tarjan 离线

若全部询问预先给定，Tarjan 离线用一遍 DFS 加并查集一次性回答：DFS 回溯完一个儿子就把它并入父亲、并把该集合的祖先指向父亲；处理到某点时，凡另一端已访问完毕的询问，其所在集合的 `anc[find(v)]` 就是最近公共祖先。它必须离线——遍历结束后不能再追加询问。`col` 用 0/1/2 三态区分未访问、在栈中、已完成。

```cpp
// 依赖 6.1.002 最近公共祖先：倍增
// 依赖 6.1.004 最近公共祖先：ST 表 / RMQ
vector<int> offlineLCA(const vector<vector<int>> &g, int rt, const vector<pair<int, int>> &q)
{
    // g 是非空树，rt 是根，q 是询问端点对；按原顺序返回 LCA。
    int n = g.size();
    assert(n > 0 && 0 <= rt && rt < n); // 调试检查，可删
    vector<vector<pair<int, int>>> ask(n);
    for (int i = 0; i < (int)q.size(); i++)
    {
        auto [u, v] = q[i];
        ask[u].push_back({v, i});
        ask[v].push_back({u, i});
    }
    vector<int> p(n, -1), anc(n), ans(q.size(), -1), col(n);
    auto find = [&](auto &&self, int x) -> int
    {
        return p[x] < 0 ? x : p[x] = self(self, p[x]);
    };
    auto merge = [&](int x, int y)
    {
        x = find(find, x);
        y = find(find, y);
        if (x == y)
        {
            return x;
        }
        if (p[x] > p[y])
        {
            swap(x, y);
        }
        p[x] += p[y];
        p[y] = x;
        return x;
    };
    auto dfs = [&](auto &&self, int u, int f) -> void
    {
        anc[u] = u;
        col[u] = 1;
        for (int v : g[u])
        {
            if (v == f)
            {
                continue;
```

```
            }
            self(self, v, u);
            anc[merge(u, v)] = u;
        }
        col[u] = 2;
        for (auto [v, id] : ask[u])
        {
            if (col[v] == 2)
            {
                ans[id] = anc[find(find, v)];
            }
        }
    };
    dfs(dfs, rt, -1);
    return ans;
}
```

`offlineLCA(g, rt, q)` 按输入顺序返回每对询问的 LCA，q 是 `{u, v}` 列表。总计 $O((n+q)\alpha(n))$ 时间、$O(n+q)$ 空间，常数极小。需要在线追加询问时改用 6.1.002 最近公共祖先：倍增 或 6.1.004 最近公共祖先：ST 表 / RMQ。

下面让三种 LCA 分别核对同一组固定答案；各实现之间不互相比较：

```
signed main()
{
    vector<vector<int>> g(5);
    for (auto [u, v] : vector<pair<int, int>>{{0, 1}, {0, 2}, {1, 3}, {1, 4}})
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    LCA a(g);
    LCARMQ b(g);
    vector<pair<int, int>> q{{3, 4}, {2, 4}};
    vector<int> want{1, 0};
    assert(offlineLCA(g, 0, q) == want);
    for (int i = 0; i < (int)q.size(); i++)
    {
        auto [u, v] = q[i];
        assert(a.lca(u, v) == want[i]);
        assert(b.lca(u, v) == want[i]);
    }
    assert(a.jump(4, 2) == 0 && a.pathKth(3, 2, 2) == 0);
}
```

6.1.006 树上距离、K 级祖先

两点距离靠根距与 LCA：$dis(u,v) = dis(u) + dis(v) - 2\,dis(\mathrm{lca})$。K 级祖先则把 k 按二进制拆开，在倍增表 `parent[level][u]` 上逐位上跳，越过根用 -1 明确失败。本条 `KthAncestor` 只维护父倍增表与深度，并用 BFS 序（`order`）自顶向下建表，避免深树递归爆栈。

```cpp
class KthAncestor
{
    // parent[k][u] 是 u 的 2^k 级祖先，越过根记 -1。
    vector<vector<int>> parent;
    // depth[u] 是 u 的深度。
    vector<int> depth;

  public:
    KthAncestor(const vector<vector<int>> &tree, int root = 0) : depth(tree.size())
    {
        // tree 是非空树，root 是根。
        assert(!tree.empty() && 0 <= root && root < (int)tree.size()); // 调试检查，可删
        int levels = bit_width((unsigned long long)tree.size());
        parent.assign(levels, vector<int>(tree.size(), -1));
        vector<int> order{root};
        for (int i = 0; i < (int)order.size(); ++i)
        {
            int u = order[i];
            for (int v : tree[u])
            {
                if (v == parent[0][u])
                {
                    continue;
                }
                parent[0][v] = u;
                depth[v] = depth[u] + 1;
                order.push_back(v);
            }
        }
        for (int level = 1; level < levels; ++level)
        {
            for (int u = 0; u < (int)tree.size(); ++u)
            {
                int p = parent[level - 1][u];
                parent[level][u] = p == -1 ? -1 : parent[level - 1][p];
            }
        }
    }

    int query(int u, int distance) const
    {
        // 返回 u 的第 distance 级祖先；越过根返回 -1。
        assert(distance >= 0); // 调试检查，可删
        if (distance > depth[u])
        {
            return -1;
        }
        for (int bit = 0; distance; ++bit, distance >>= 1)
        {
            if (distance & 1)
            {
                u = parent[bit][u];
```

```
            }
        }
        return u;
    }
};
```

构造 `KthAncestor(tree, root)` 后 `query(u, k)` 返回 u 的第 k 级祖先，$k > dep_u$ 时返回 -1，单次 $O(\log k)$；构造 $O(n \log n)$ 时间与空间。带权距离需另配根距数组。6.2.005 长链剖分 的 $O(1)$ K 级祖先建立在同一倍增预处理之上；只求 LCA 则无需建此表，见 6.1.002 最近公共祖先：倍增。

6.1.007 树上路径交

树上简单路径是 “凸” 的：两条路径的交若非空，仍是一条路径。把两路径四个端点与它们两两的 LCA 作为候选点，凡同时落在两条路径上的点（用 $dis(a,x)+dis(x,b)=dis(a,b)$ 判定 x 在 a-b 上），取其中距离最远的一对即交段端点；候选全不满足则两路径不相交。本条只依赖 6.1.002 最近公共祖先：倍增 的 `lca` 与 `dist`。

```
optional<pair<int, int>> pathIsect(int a, int b, int c, int d, const LCA &l)
{
    // 返回路径 a-b 与 c-d 的公共段端点；不相交时返回 nullopt。
    vector<int> p = {a, b, c, d};
    for (int x : vector<int>{a, b})
    {
        for (int y : vector<int>{c, d})
        {
            p.push_back(l.lca(x, y));
        }
    }
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    vector<int> q;
    for (int x : p)
    {
        bool u = l.dist(a, x) + l.dist(x, b) == l.dist(a, b);
        bool v = l.dist(c, x) + l.dist(x, d) == l.dist(c, d);
        if (u && v)
        {
            q.push_back(x);
        }
    }
    if (q.empty())
    {
        return nullopt;
    }
    pair<int, int> ans{q[0], q[0]};
    for (int x : q)
    {
        for (int y : q)
        {
```

```
            if (l.dist(x, y) > l.dist(ans.first, ans.second))
            {
                ans = {x, y};
            }
        }
    }
    return ans;
}
```

`pathIsect(a, b, c, d, l)` 传入两条路径端点与已建好的 `LCA`，返回 `optional<pair>` 表示交段端点，无交返回 `nullopt`。候选点常数个、每次 LCA $O(\log n)$，总 $O(\log n)$ 时间、$O(1)$ 额外空间。用 dis 等式判点在路径上，省去了显式构造路径本身。

6.1.008 树哈希与基环树哈希

给无根树或基环树算一个与编号无关的结构指纹，用于同构分组。有根树哈希自底向上：每点把儿子哈希排序后经 `map` 映射成一个整数（AHU 编码），保证同构子树得同一值。无根树先取重心（一或两个）为根分别哈希，再取规范集合；基环树先剥叶找出唯一环，对环上每个点以 “临时禁掉环其余点” 的方式哈希其挂树，再把环序列用最小表示 `minRot` 消旋转、`reverse` 消翻转取规范。这里用 `map` 编码而非随机哈希，是确定性无碰撞的（代价是每次 log）。

```
struct TreeIso
{
    // mp 在当前对象内把儿子编码多重集映射为唯一整数。
    map<vector<int>, int> mp;

    int get(vector<int> a)
    {
        // a 是儿子子树编码；返回当前共享编码表中的 AHU 编号。
        sort(a.begin(), a.end());
        return mp.emplace(move(a), mp.size() + 1).first->second;
    }

    vector<int> ids(const vector<vector<int>> &g, int rt, const vector<bool> *ban = nullptr)
    {
        // 返回从 rt 可达的各点有根子树编码；ban 中的点不参与遍历。
        int n = g.size();
        vector<int> p(n, -2), q{rt};
        p[rt] = -1;
        for (int k = 0; k < (int)q.size(); k++)
        {
            int u = q[k];
            for (int v : g[u])
            {
                if (v == p[u] || p[v] != -2 || (ban && (*ban)[v]))
                {
                    continue;
                }
                p[v] = u;
```

```
                q.push_back(v);
            }
        }
        vector<int> h(n);
        for (int k = (int)q.size() - 1; k >= 0; k--)
        {
            int u = q[k];
            vector<int> a;
            for (int v : g[u])
            {
                if (p[v] == u)
                {
                    a.push_back(h[v]);
                }
            }
            h[u] = get(move(a));
        }
        return h;
    }

    int rooted(const vector<vector<int>> &g, int rt = 0)
    {
        // 返回以 rt 为根的整棵树编码，g 必须非空。
        assert(!g.empty() && 0 <= rt && rt < (int)g.size()); // 调试检查，可删
        return ids(g, rt)[rt];
    }

    vector<int> centroids(const vector<vector<int>> &g)
    {
        // 返回非空树的一个或两个重心。
        assert(!g.empty()); // 调试检查，可删
        int n = g.size();
        vector<int> p(n, -1), sz(n, 1), q{0};
        for (int k = 0; k < (int)q.size(); k++)
        {
            int u = q[k];
            for (int v : g[u])
            {
                if (v == p[u])
                {
                    continue;
                }
                p[v] = u;
                q.push_back(v);
            }
        }
        vector<int> ans;
        for (int k = n - 1; k >= 0; k--)
        {
            int u = q[k];
            for (int v : g[u])
            {
```

```
                if (p[v] != u)
                {
                    continue;
                }
                sz[u] += sz[v];
            }
            int mx = n - sz[u];
            for (int v : g[u])
            {
                if (p[v] != u)
                {
                    continue;
                }
                mx = max(mx, sz[v]);
            }
            if (mx * 2 <= n)
            {
                ans.push_back(u);
            }
        }
        return ans;
    }

    vector<int> unrooted(const vector<vector<int>> &g)
    {
        // 返回无根树在当前共享编码表中的规范编码。
        vector<int> ans;
        for (int r : centroids(g))
        {
            ans.push_back(rooted(g, r));
        }
        sort(ans.begin(), ans.end());
        return ans;
    }

    vector<int> minRot(const vector<int> &a)
    {
        // 返回循环序列 a 的字典序最小循环表示。
        int n = a.size(), i = 0, j = 1, k = 0;
        while (i < n && j < n && k < n)
        {
            int x = a[(i + k) % n], y = a[(j + k) % n];
            if (x == y)
            {
                k++;
                continue;
            }
            if (x > y)
            {
                i += k + 1;
            }
            else
```

```
            {
                j += k + 1;
            }
            if (i == j)
            {
                j++;
            }
            k = 0;
        }
        int s = min(i, j);
        vector<int> b;
        for (int x = 0; x < n; x++)
        {
            b.push_back(a[(s + x) % n]);
        }
        return b;
    }

    vector<int> unicycle(const vector<vector<int>> &g)
    {
        // 返回非空简单连通基环图在当前共享编码表中的规范编码。
        assert(!g.empty()); // 调试检查，可删
        int n = g.size();
        vector<int> d(n);
        queue<int> q;
        for (int i = 0; i < n; i++)
        {
            d[i] = g[i].size();
            if (d[i] == 1)
            {
                q.push(i);
            }
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            d[u] = 0;
            for (int v : g[u])
            {
                if (d[v] && --d[v] == 1)
                {
                    q.push(v);
                }
            }
        }
        vector<int> cyc;
        for (int i = 0; i < n; i++)
        {
            if (d[i])
            {
                cyc.push_back(i);
```

```
            }
        }
        if (cyc.empty())
        {
            return {};
        }
        vector<bool> ban(n);
        for (int x : cyc)
        {
            ban[x] = true;
        }
        vector<int> a;
        int s = cyc[0], p = -1, u = s;
        do
        {
            ban[u] = false;
            a.push_back(ids(g, u, &ban)[u]);
            ban[u] = true;
            int v = -1;
            for (int x : g[u])
            {
                if (d[x] && x != p)
                {
                    v = x;
                }
            }
            p = u;
            u = v;
        } while (u != s);
        auto x = minRot(a);
        reverse(a.begin(), a.end());
        return min(x, minRot(a));
    }
};
```

`TreeIso` 用精确 AHU 编码，不存在随机碰撞；`unrooted(g)`、`unicycle(g)` 返回规范整数序列。整数编号由子树形态在 `mp` 中首次出现的顺序决定，因此只有同一个 `TreeIso` 对象编码出的结果才能直接比较；两个各自从空 `mp` 开始的对象也不能比较编号。单棵树 $O(n \log n)$ 时间、$O(n)$ 空间，基环图须简单、连通且恰有一个环。

下面固定检查路径的重标号以及路径与星。所有待比较图都交给同一个 `TreeIso` 对象编码：

```
signed main()
{
    auto tree = [](int n, initializer_list<pair<int, int>> e)
    {
        vector<vector<int>> g(n);
        for (auto [u, v] : e)
        {
            g[u].push_back(v);
            g[v].push_back(u);
```

```
        }
        return g;
    };
    auto path = tree(5, {{0, 1}, {1, 2}, {2, 3}, {3, 4}});
    auto relabeled = tree(5, {{3, 1}, {1, 4}, {4, 0}, {0, 2}});
    auto star = tree(5, {{0, 1}, {0, 2}, {0, 3}, {0, 4}});
    TreeIso iso;
    assert(iso.unrooted(path) == iso.unrooted(relabeled));
    assert(iso.unrooted(path) != iso.unrooted(star));
}
```

6.1.009 Prüfer 序列与树的计数

Prüfer 序列在 n 个带标号点的树与长度 $n-2$ 的序列间建立双射：编码时反复删去编号最小的叶并记录其唯一邻点，解码时用度数数组与当前最小可用叶还原。由此点 i 在序列中出现次数恰为 $\deg(i)-1$，带标号树共 n^{n-2} 棵（Cayley 公式），给定度数 d_i 的树数是多项式系数 $\binom{n-2}{d_1-1,\dots,d_n-1}$。`pruferCode` 用小根堆取最小叶，`pruferTree` 逆流程重建边集。

```
vector<int> pruferCode(const vector<vector<int>> &g)
{
    // 返回 n>=2 的带标号树对应的 Prüfer 序列。
    int n = g.size();
    assert(n >= 2); // 调试检查，可删
    if (n == 2)
    {
        return {};
    }
    vector<int> d(n);
    priority_queue<int, vector<int>, greater<>> q;
    for (int i = 0; i < n; i++)
    {
        d[i] = g[i].size();
        if (d[i] == 1)
        {
            q.push(i);
        }
    }
    vector<int> a;
    for (int k = 0; k < n - 2; k++)
    {
        int u = q.top();
        q.pop();
        int v = -1;
        for (int x : g[u])
        {
            if (d[x])
            {
                v = x;
                break;
            }
```

```
        }
        a.push_back(v);
        d[u]--;
        if (--d[v] == 1)
        {
            q.push(v);
        }
    }
    return a;
}

vector<pair<int, int>> pruferTree(const vector<int> &a)
{
    // 返回 Prüfer 序列还原出的 n-1 条无向边；空序列表示两点一边。
    int n = a.size() + 2;
    vector<int> d(n, 1);
    for (int x : a)
    {
        assert(0 <= x && x < n); // 调试检查，可删
        d[x]++;
    }
    priority_queue<int, vector<int>, greater<>> q;
    for (int i = 0; i < n; i++)
    {
        if (d[i] == 1)
        {
            q.push(i);
        }
    }
    vector<pair<int, int>> e;
    for (int v : a)
    {
        int u = q.top();
        q.pop();
        e.push_back({u, v});
        if (--d[v] == 1)
        {
            q.push(v);
        }
    }
    int u = q.top();
    q.pop();
    e.push_back({u, q.top()});
    return e;
}
```

`pruferCode(g)` 的定义域是 $n \geq 2$ 的无向带标号树，`pruferTree(a)` 输入长 $n-2$ 的序列并返回 $n-1$ 条边；在此定义域内两者互逆。空序列固定表示两点一边，单点树不属于这组双射。编解码各 $O(n \log n)$ 时间、$O(n)$ 空间。它是随机生成树、按度数计数的基础工具，与 8.1.001 阶乘、逆阶乘与组合数 的多项式系数配合即可直接数给定度数序列的树数。

下面用一条含重复标签的序列检查编解码往返，并保留二点树特例：

```cpp
signed main()
{
    vector<int> a{3, 0, 3, 1};
    int n = a.size() + 2;
    auto e = pruferTree(a);
    vector<vector<int>> g(n);
    for (auto [u, v] : e)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    assert(pruferCode(g) == a);
    assert((pruferTree({}) == vector<pair<int, int>>{{0, 1}}));
}
```

6.1.010 树的邻接表转前序表

把邻接表重排成“父先于所有子孙”的前序表，同时给出父亲与每棵子树对应的连续区间，这是所有“子树问题转区间问题”的地基。递归 DFS 首次访问某点时分配前序编号，回溯时累加子树大小，于是每棵子树恰对应区间 $[pos[x],\ pos[x]+siz[x])$。

```cpp
struct TreeOrder
{
    vector<int> par; // par[x] 是 x 的父亲，根的父亲为 -1。
    vector<int> ord; // ord 按 DFS 前序保存全部点。
    vector<int> pos; // pos[x] 是 x 在 ord 中的下标。
    vector<int> siz; // siz[x] 是以 x 为根的子树大小，因此子树对应 ord 的连续区间。
};

TreeOrder preorderTab(const vector<vector<int>> &g, int root = 0)
{
    // g 是无向树，root 是根；空树返回四张空表。
    auto n = (int)g.size();
    if (!n)
    {
        return {};
    }
    assert(0 <= root && root < n); // 调试检查，可删
    TreeOrder z{vector<int>(n, -1), {}, vector<int>(n), vector<int>(n, 1)};
    z.ord.reserve(n);
    auto dfs = [&](auto &&self, int x, int p) -> void
    {
        z.par[x] = p;
        z.pos[x] = z.ord.size();
        z.ord.push_back(x);
        for (auto y : g[x])
        {
            if (y != p)
            {
                self(self, y, x);
```

```
                z.siz[x] += z.siz[y];
            }
        }
    };
    dfs(dfs, root, -1);
    return z;
}
```

`preorderTab(g, root)` 返回 `TreeOrder{par, ord, pos, siz}`：`par` 根为 -1，`pos[x]` 是 x 的前序下标，子树对应 `ord` 上区间 $[pos[x], pos[x]+siz[x])$。时间与额外空间均为 $O(n)$，递归深度最坏为 $O(n)$。这个连续子树区间正是 6.3.004 树上路径/子树数据结构 把子树查询交给一维数据结构的前提。

6.1.011 公共区间分解树

排列的公共区间是位置连续且值集合也连续的区间。所有强公共区间形成唯一的包含树：子区间的值域相邻时为线性节点，否则为素节点。算法从左向右插入排列元素，用两个单调栈维护区间最小值、最大值变化，并让区间加、区间最大值线段树判断栈顶段是否恰构成连续值域；一旦成立就合并为素节点，值域端点相邻的节点则直接合并为线性节点。

```
struct CommonNode
{
    int l, r, parent; // l、r 是闭区间端点，parent 是父节点编号，根为 -1。
    bool linear; // linear 为真表示线性节点，为假表示素节点。
};

vector<CommonNode> commonIntTr(const vector<int> &p)
{
    // p 是 0..n-1 的排列；返回先序编号的公共区间分解树，父节点编号严格更小。
    struct MaxAddSeg
    {
        int n; // n 是叶子数。
        vector<int> mx, tag; // mx 是区间最大值，tag 是整段加法懒标记。
        MaxAddSeg(int n) : n(n), mx(n * 4), tag(n * 4)
        {
            // n 是叶子数；初始每个叶子为 0。
        }
        void add(int ql, int qr, int x, int u, int l, int r)
        {
            // [ql,qr) 加 x，u 覆盖 [l,r)。
            if (qr <= l || r <= ql)
            {
                return;
            }
            if (ql <= l && r <= qr)
            {
                mx[u] += x;
                tag[u] += x;
                return;
            }
```

```
            int m = (l + r) / 2;
            add(ql, qr, x, u * 2, l, m);
            add(ql, qr, x, u * 2 + 1, m, r);
            mx[u] = tag[u] + max(mx[u * 2], mx[u * 2 + 1]);
        }
        void set(int p, int x, int u, int l, int r)
        {
            // 把位置 p 赋为 x，u 覆盖 [l,r)。
            if (r - l == 1)
            {
                mx[u] = x;
                tag[u] = 0;
                return;
            }
            int m = (l + r) / 2;
            if (p < m)
            {
                set(p, x - tag[u], u * 2, l, m);
            }
            else
            {
                set(p, x - tag[u], u * 2 + 1, m, r);
            }
            mx[u] = tag[u] + max(mx[u * 2], mx[u * 2 + 1]);
        }
        int prod(int ql, int qr, int u, int l, int r) const
        {
            // 返回 [ql,qr) 的最大值，u 覆盖 [l,r)。
            if (qr <= l || r <= ql)
            {
                return numeric_limits<int>::min() / 4;
            }
            if (ql <= l && r <= qr)
            {
                return mx[u];
            }
            int m = (l + r) / 2;
            return tag[u] + max(prod(ql, qr, u * 2, l, m), prod(ql, qr, u * 2 + 1, m, r));
        }
        void add(int l, int r, int x) { add(l, r, x, 1, 0, n); }
        void set(int p, int x) { set(p, x, 1, 0, n); }
        int prod(int l, int r) const { return prod(l, r, 1, 0, n); }
    };
    struct Node
    {
        int l, r, mn, mx; // l、r 是半开位置区间，mn、mx 是对应值半开区间。
        bool linear; // 是否为线性节点。
        vector<int> ch; // 从左到右的子节点编号。
    };
    int n = p.size();
    if (n == 0)
    {
```

```
        return {};
    }
    MaxAddSeg seg(n);
    vector<int> mn, mx, stk;
    vector<Node> tr;
    auto makeLeaf = [&](int i)
    {
        tr.push_back({i, i + 1, p[i], p[i] + 1, true, {}});
        return (int)tr.size() - 1;
    };
    auto merge = [&](vector<int> ch, bool linear)
    {
        int l = tr[ch[0]].l, r = tr[ch.back()].r;
        int lo = tr[ch[0]].mn, hi = tr[ch[0]].mx;
        for (int x : ch)
        {
            lo = min(lo, tr[x].mn);
            hi = max(hi, tr[x].mx);
        }
        tr.push_back({l, r, lo, hi, linear, move(ch)});
        return (int)tr.size() - 1;
    };
    for (int i = 0; i < n; i++)
    {
        int r = i;
        while (!mn.empty() && p[mn.back()] > p[i])
        {
            mn.pop_back();
            int l = mn.empty() ? 0 : mn.back() + 1;
            seg.add(l, r, p[i] - p[r - 1]);
            r = l;
        }
        mn.push_back(i);
        r = i;
        while (!mx.empty() && p[mx.back()] < p[i])
        {
            mx.pop_back();
            int l = mx.empty() ? 0 : mx.back() + 1;
            seg.add(l, r, -p[i] + p[r - 1]);
            r = l;
        }
        mx.push_back(i);
        seg.add(0, i, 1);
        int cur = makeLeaf(i);
        while (true)
        {
            if (!stk.empty() && tr[stk.back()].linear && !tr[stk.back()].ch.empty() &&
                (tr[cur].mn == tr[tr[stk.back()].ch.back()].mx || tr[cur].mx ==
                ↪ tr[tr[stk.back()].ch.back()].mn))
            {
                tr[stk.back()].ch.push_back(cur);
                tr[stk.back()].r = tr[cur].r;
```

```
                tr[stk.back()].mn = min(tr[stk.back()].mn, tr[cur].mn);
                tr[stk.back()].mx = max(tr[stk.back()].mx, tr[cur].mx);
                cur = stk.back();
                stk.pop_back();
                continue;
            }
            if (!stk.empty() && (tr[stk.back()].mx == tr[cur].mn || tr[stk.back()].mn ==
            ↪ tr[cur].mx))
            {
                cur = merge({stk.back(), cur}, true);
                stk.pop_back();
                continue;
            }
            stk.push_back(cur);
            if (stk.size() == 1)
            {
                break;
            }
            if (seg.prod(0, tr[stk.back()].l) != 0)
            {
                break;
            }
            vector<int> ch{stk.back()};
            stk.pop_back();
            int l = tr[ch[0]].l, r2 = tr[ch[0]].r, lo = tr[ch[0]].mn, hi = tr[ch[0]].mx;
            while (true)
            {
                int x = stk.back();
                stk.pop_back();
                ch.push_back(x);
                l = min(l, tr[x].l);
                r2 = max(r2, tr[x].r);
                lo = min(lo, tr[x].mn);
                hi = max(hi, tr[x].mx);
                if (r2 - l == hi - lo)
                {
                    break;
                }
            }
            reverse(ch.begin(), ch.end());
            tr.push_back({l, r2, lo, hi, false, move(ch)});
            cur = tr.size() - 1;
        }
        seg.set(i, 0);
    }
    assert(stk.size() == 1);
    vector<CommonNode> ans;
    auto dfs = [&](auto &&self, int u, int fa) -> void
    {
        int id = ans.size();
        ans.push_back({tr[u].l, tr[u].r - 1, fa, tr[u].linear});
        for (int v : tr[u].ch)
```

```
        {
            self(self, v, id);
        }
    };
    dfs(dfs, stk.back(), -1);
    return ans;
}
```

`commonIntTr(p)` 要求 `p` 是 `0..n-1` 的排列，返回先序编号的节点表；节点区间是闭区间，根的 `parent=-1`，`linear=false` 表示素节点。构造时间 $O(n \log n)$、空间 $O(n)$，可覆盖 $n \le 5 \times 10^5$。它分解的是排列公共区间，不是输入图上的树；普通树的 DFS 序仍使用 6.1.010 树的邻接表转前序表。

6.2 树上分治与启发式合并

6.2.001 点分治

点分治按重心分层统计跨子树的路径：任意点对在分治过程中恰有一层首次跨过当前重心，于是“经过重心的合法点对”只需在这一层数一次。做法是收集重心到各点的距离，对全体排序后双指针数出距离和 $\le lim$ 的对，再减去每个儿子子树内部（并未真正跨重心）被多算的对以去重，删去重心后对每个子块递归。每层都从当前块重新求重心，保证子块规模减半。

```
template <class Dist = long long>
class CentPairs
{
    static_assert(numeric_limits<Dist>::is_integer && numeric_limits<Dist>::is_signed &&
                  sizeof(Dist) >= sizeof(long long));
    // n 是树点数，g[u] 的每项依次是邻点和非负边权。
    int n;
    vector<vector<array<int, 2>>> g;
    // siz、par 是当前连通块的临时子树大小与父亲，ban 标记已选重心。
    vector<int> siz, par;
    vector<bool> ban;

    int cen(int s)
    {
        // 返回 s 所在未删除连通块的重心。
        vector<int> ord;
        auto dfs = [&](auto &&self, int u, int p) -> void
        {
            par[u] = p;
            siz[u] = 1;
            ord.push_back(u);
            for (auto x : g[u])
            {
                int v = x[0];
                if (v == p || ban[v])
                {
                    continue;
                }
```

```
                self(self, v, u);
                siz[u] += siz[v];
            }
        };
        dfs(dfs, s, -1);
        int total = ord.size(), c = s, best = total;
        for (int u : ord)
        {
            int mx = total - siz[u];
            for (auto x : g[u])
            {
                int v = x[0];
                if (!ban[v] && par[v] == u)
                {
                    chmax(mx, siz[v]);
                }
            }
            if (mx < best)
            {
                best = mx;
                c = u;
            }
        }
        return c;
    }

    vector<Dist> distances(int s, int p, long long w) const
    {
        // 返回 s 所在分支内各点到当前重心的距离。
        vector<Dist> a;
        auto dfs = [&](auto &&self, int u, int f, Dist d) -> void
        {
            a.push_back(d);
            for (auto [v, ew] : g[u])
            {
                if (v != f && !ban[v])
                {
                    self(self, v, u, d + (Dist)ew);
                }
            }
        };
        dfs(dfs, s, p, (Dist)w);
        return a;
    }

    int pairs(vector<Dist> a, Dist lim) const
    {
        // a 是一组到同一根的距离，lim 是距离上限；返回无序点对距离和不超过 lim 的数量。
        sort(a.begin(), a.end());
        int ans = 0;
        for (int l = 0, r = (int)a.size() - 1; l < r;)
        {
```

```
            if (a[r] <= lim && a[l] <= lim - a[r])
            {
                ans += r - l;
                l++;
            }
            else
            {
                r--;
            }
        }
        return ans;
    }

    int solve(int s, Dist lim)
    {
        // s 是当前未删除连通块内一点，lim 是距离上限；返回该块经当前层首次统计的点对数。
        int c = cen(s);
        ban[c] = true;
        vector<Dist> all{0};
        int ans = 0;
        for (auto [v, w] : g[c])
        {
            if (ban[v])
            {
                continue;
            }
            auto cur = distances(v, c, w);
            ans -= pairs(cur, lim);
            all.insert(all.end(), cur.begin(), cur.end());
        }
        ans += pairs(all, lim);
        for (auto x : g[c])
        {
            int v = x[0];
            if (!ban[v])
            {
                ans += solve(v, lim);
            }
        }
        return ans;
    }

  public:
    CentPairs(const vector<vector<array<int, 2>>> &g) : n(g.size()), g(g), siz(n), par(n),
    ban(n)
    {
        // g 是 0 下标加权树；保存输入并初始化点分治状态，无返回值。
    }

    int countAtMost(Dist lim)
    {
        // lim 是非负距离上限；返回距离不超过 lim 的无序不同点对数量。
```

```
        fill(ban.begin(), ban.end(), false);
        return n ? solve(0, lim) : 0;
    }
};
```

构造 `CentPairs<Dist> q(g)` 后调用 `countAtMost(lim)`；`Dist` 必须是至少与输入同宽的有符号整数，边权和 `lim` 均须非负，且调用者须保证树上距离能由 `Dist` 表示；若可能超过 64 位，应改用 `CentPairs<__int128>`。时间 $O(n \log^2 n)$，空间 $O(n)$。

下面用一棵六点带权树检查零距离、局部点对和全部点对边界：

```
signed main()
{
    vector<vector<array<int, 2>>> g(6);
    vector<array<int, 3>> e{{0, 1, 2}, {0, 2, 3}, {1, 3, 1}, {1, 4, 4}, {2, 5, 2}};
    for (auto [u, v, w] : e)
    {
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
    CentPairs q(g);
    assert(q.countAtMost(0) == 0);
    assert(q.countAtMost(2) == 3);
    assert(q.countAtMost(4) == 6);
    assert(q.countAtMost(5) == 9);
    assert(q.countAtMost(11) == 15);
}
```

6.2.002 点分树

反复删重心并记录父子关系，得到点分树。因重心删除后每个子块不超过原块一半，每个原点至多出现在 $O(\log n)$ 层，故点分树树高 $O(\log n)$。注意点分树的父子边表示“包含块”关系，不是原树的边；它常配合“每点到各级点分树祖先的距离”支持带修改的路径统计。

```
class CentroidTree
{
    // n 是树点数，g 是原树邻接表。
    int n;
    vector<vector<int>> g;
    // siz、tmp 是当前块的临时子树大小与父亲，ban 标记已选重心。
    vector<int> siz, tmp;
    vector<bool> ban;
    // par[u]、dep[u] 是点 u 在点分树中的父节点与深度。
    vector<int> par, dep;

    int cen(int s)
    {
        // 返回 s 所在未删除连通块的重心。
        vector<int> ord;
        auto dfs = [&](auto &&self, int u, int p) -> void
```

```
        {
            tmp[u] = p;
            siz[u] = 1;
            ord.push_back(u);
            for (int v : g[u])
            {
                if (v == p || ban[v])
                {
                    continue;
                }
                self(self, v, u);
                siz[u] += siz[v];
            }
        };
        dfs(dfs, s, -1);
        int total = ord.size(), c = s, best = total;
        for (int u : ord)
        {
            int mx = total - siz[u];
            for (int v : g[u])
            {
                if (!ban[v] && tmp[v] == u)
                {
                    chmax(mx, siz[v]);
                }
            }
            if (mx < best)
            {
                best = mx;
                c = u;
            }
        }
        return c;
    }

    int build(int s, int p, int d)
    {
        // s 是当前块内一点，p、d 是点分树父点和深度；返回当前块重心。
        int c = cen(s);
        par[c] = p;
        dep[c] = d;
        ban[c] = true;
        for (auto v : g[c])
        {
            if (!ban[v])
            {
                build(v, c, d + 1);
            }
        }
        return c;
    }

```

```
  public:
    CentroidTree(const vector<vector<int>> &g)
        : n(g.size()), g(g), siz(n), tmp(n), ban(n), par(n, -1), dep(n)
    {
        // g 是 0 下标树；建立点分树，无返回值。
        if (n)
        {
            build(0, -1, 0);
        }
    }

    const vector<int> &parents() const
    {
        // 无参数；返回点分树父节点数组，根的父节点为 -1。
        return par;
    }

    const vector<int> &depths() const
    {
        // 无参数；返回每个原树点在点分树中的深度。
        return dep;
    }
};
```

构造 `CentroidTree tr(g)`，`parents()`（根为 -1）与 `depths()` 给出点分树拓扑。当前实现每层重新求子树大小，$O(n \log n)$ 时间、$O(n)$ 空间。要在点分树上做距离查询还需另行预处理原树两点距离（如 6.1.002 最近公共祖先：倍增）。静态一次性路径统计直接用 6.2.001 点分治 更省。

下面在 7 点链、6 点星和 7 点完全二叉树上检查点分树父亲与深度：

```
signed main()
{
    auto makeTree = [](int n, initializer_list<pair<int, int>> e)
    {
        vector<vector<int>> g(n);
        for (auto [u, v] : e)
        {
            g[u].push_back(v);
            g[v].push_back(u);
        }
        return g;
    };
    CentroidTree chain(makeTree(7, {{0, 1}, {1, 2}, {2, 3}, {3, 4}, {4, 5}, {5, 6}}));
    assert((chain.parents() == vector<int>{1, 3, 1, -1, 5, 3, 5}));
    assert((chain.depths() == vector<int>{2, 1, 2, 0, 2, 1, 2}));
    CentroidTree star(makeTree(6, {{0, 1}, {0, 2}, {0, 3}, {0, 4}, {0, 5}}));
    assert((star.parents() == vector<int>{-1, 0, 0, 0, 0, 0}));
    CentroidTree binary(makeTree(7, {{0, 1}, {0, 2}, {1, 3}, {1, 4}, {2, 5}, {2, 6}}));
    assert((binary.parents() == vector<int>{-1, 0, 0, 1, 1, 2, 2}));
    cout << "OK\n";
    return 0;
}
```

6.2.003 边分治

边分治先用零权虚点把树“二叉化”（把每个原点的多个儿子串成一条零权链，度数降到 ≤ 3 且保持任意原点间距离不变），再反复选一条使两侧原点数尽量均衡的边切开。二叉化后必存在一条边令两侧原点数都在常数比例内，故递归深度 $O(\log n)$。它擅长那些天然按边分割、而点分治难以合并的贡献（如某些路径计数）。编号 $\ge$ `origNodes()` 的是虚点，统计时不计入。

```
class EdgeDecomp
{
    // original 是原树点数；扩展图中编号 >= original 的点都是零权虚点。
    int original;
    // g[u] 的每项依次是邻点、边权、扩展树边编号。
    vector<vector<array<int, 3>>> g;
    // real[u] 表示该扩展点代表几个原点，ban[id] 标记已经切断的扩展边。
    vector<int> real, sub, par, pe;
    vector<bool> ban;
    // cut[i][0..2] 依次是父切边编号、当前扩展边两个端点。
    vector<array<int, 3>> cut;

    int node(bool isReal)
    {
        // isReal 表示是否对应原点；新增扩展点并返回编号。
        g.push_back({});
        real.push_back(isReal);
        sub.push_back(0);
        par.push_back(-1);
        pe.push_back(-1);
        return g.size() - 1;
    }

    void edge(int u, int v, int w)
    {
        // u、v 是扩展点，w 是边权；加入无向边，无返回值。
        int id = ban.size();
        ban.push_back(false);
        g[u].push_back({v, w, id});
        g[v].push_back({u, w, id});
    }

    void build(int s, int cutPar)
    {
        // s 是当前块内一点，cutPar 是分治树父切边记录；递归建立边分树，无返回值。
        vector<int> ord;
        auto dfs = [&](auto &&self, int u, int p, int pid) -> void
        {
            par[u] = p;
            pe[u] = pid;
            sub[u] = real[u];
            ord.push_back(u);
            for (auto x : g[u])
            {
```

```
                int v = x[0], id = x[2];
                if (v == p || ban[id])
                {
                    continue;
                }
                self(self, v, u, id);
                sub[u] += sub[v];
            }
        };
        dfs(dfs, s, -1, -1);
        int total = sub[s];
        if (total <= 1)
        {
            return;
        }
        array<int, 4> best{INF, -1, -1, -1};
        for (int v : ord)
        {
            if (v == s)
            {
                continue;
            }
            array<int, 4> cur{max(sub[v], total - sub[v]), pe[v], par[v], v};
            if (cur[0] < best[0])
            {
                best = cur;
            }
        }
        assert(best[1] != -1); // 调试检查，可删
        int id = cut.size();
        cut.push_back({cutPar, best[2], best[3]});
        ban[best[1]] = true;
        build(best[2], id);
        build(best[3], id);
    }

  public:
    EdgeDecomp(const vector<vector<array<int, 2>>> &tree) : original(tree.size())
    {
        // tree[u] 的每项依次是邻点和边权；二叉化后建立边分治树，无返回值。
        for (int i = 0; i < original; i++)
        {
            node(true);
        }
        vector<int> p(original, -1);
        vector<vector<array<int, 2>>> child(original);
        if (original)
        {
            auto dfs = [&](auto &&self, int u) -> void
            {
                for (auto [v, w] : tree[u])
                {
```

```
                    if (v == p[u])
                    {
                        continue;
                    }
                    p[v] = u;
                    child[u].push_back({v, w});
                    self(self, v);
                }
            };
            dfs(dfs, 0);
        }
        for (int u = 0; u < original; u++)
        {
            int cur = u;
            for (int i = 0; i < (int)child[u].size(); i++)
            {
                auto [v, w] = child[u][i];
                if (i + 1 == (int)child[u].size())
                {
                    edge(cur, v, w);
                }
                else
                {
                    int z = node(false);
                    edge(cur, z, 0);
                    edge(z, v, w);
                    cur = z;
                }
            }
        }
        if (original)
        {
            build(0, -1);
        }
    }

    const vector<array<int, 3>> &cuts() const
    {
        // 无参数；每项依次是父切分记录编号、当前切边的两个扩展点，根记录的父编号为 -1。
        return cut;
    }

    int origNodes() const
    {
        // 无参数；返回原树点数，用于区分扩展图虚点。
        return original;
    }
};
```

构造 `EdgeDecomp q(tree)`（`tree[u]` 每项 `{v, w}`），`cuts()` 返回边分树，每条记录父切边与当前切边两端点。二叉化后点边仍 $O(n)$，建树 $O(n \log n)$ 时间、$O(n)$ 空间。跨子树路径能合并时优先 6.2.001 点分治（常数更小、无虚点开销）；只有必须按边划分时才用边分治。

6.2.004 DSU on Tree

DSU on Tree 离线回答每个子树的 “频率型” 询问：先处理并清空所有轻儿子，再处理重儿子并保留其统计，最后把所有轻子树暴力加入当前点后回答。每个点只在跨越轻边时被重复加入，而任一点到根的轻边数 $O(\log n)$，故总加删次数 $O(n \log n)$。`add`/`remove`/`answer` 三个回调把 “如何计入一个点、如何撤销、如何取答案” 留给调用者，本体只管调度。

```
class DsuOnTree
{
    // n 是点数，tree 是树邻接表。
    int n;
    vector<vector<int>> tree;
    // parent、size、heavy 是父亲、子树大小与重儿子；ord 是前序点列，pos[u] 是 u 在 ord 中的下标。
    vector<int> parent, size, heavy, ord, pos;

    void prepare(int root)
    {
        // 按 root 重建父亲、前序、子树大小与重儿子。
        fill(parent.begin(), parent.end(), -1);
        fill(size.begin(), size.end(), 1);
        fill(heavy.begin(), heavy.end(), -1);
        ord.clear();
        auto dfs = [&](auto &&self, int u, int p) -> void
        {
            parent[u] = p;
            pos[u] = ord.size();
            ord.push_back(u);
            for (int v : tree[u])
            {
                if (v != p)
                {
                    self(self, v, u);
                    size[u] += size[v];
                    if (heavy[u] == -1 || size[v] > size[heavy[u]])
                    {
                        heavy[u] = v;
                    }
                }
            }
        };
        dfs(dfs, root, -1);
    }

    template <class F> void visitSubtree(int u, F &&f) const
    {
        // 按前序遍历 u 的整棵子树，并对每个点调用 f。
        for (int i = pos[u]; i < pos[u] + size[u]; i++)
        {
            f(ord[i]);
        }
    }
```

```cpp

  public:
    explicit DsuOnTree(vector<vector<int>> tree)
        : n(tree.size()), tree(move(tree)), parent(n), size(n), heavy(n, -1), pos(n)
    {
        // 保存树；根相关信息在 run 时重建。
    }

    template <class Add, class Remove, class Answer> void run(Add &&add, Remove &&remove,
    ↪  Answer &&answer, int root = 0)
    {
        // add/remove 增删单点贡献，answer(u) 在 u 的子树状态齐备时读取答案。
        assert(n > 0 && 0 <= root && root < n); // 调试检查，可删
        prepare(root);
        auto dfs = [&](auto &&self, int u, bool keep) -> void
        {
            for (int v : tree[u])
            {
                if (v != parent[u] && v != heavy[u])
                {
                    self(self, v, false);
                }
            }
            if (heavy[u] != -1)
            {
                self(self, heavy[u], true);
            }
            for (int v : tree[u])
            {
                if (v != parent[u] && v != heavy[u])
                {
                    visitSubtree(v, add);
                }
            }
            add(u);
            answer(u);
            if (!keep)
            {
                visitSubtree(u, remove);
            }
        };
        dfs(dfs, root, false);
    }
};
```

构造 `DsuOnTree q(tree)` 后 `run(add, remove, answer, root)`：`add(u)`/`remove(u)` 增删单点贡献、`answer(u)` 在 u 子树贡献齐备时读答案。全程 $O(n \log n)$ 次回调、$O(n)$ 结构自身空间（不含回调内部数据结构）。它要求询问可离线且贡献可增量增删；可减性不成立或需在线时须另择方案。相比 6.2.001 点分治 面向路径统计，DSU on Tree 面向子树内的可加删信息。

例如统计每个子树的点权和：

```cpp
signed main()
{
    vector<vector<int>> tree{{1, 2}, {0}, {0}};
    vector<int> w{1, 2, 3}, ans(3);
    int sum = 0;
    auto add = [&](int u) { sum += w[u]; };
    auto remove = [&](int u) { sum -= w[u]; };
    auto answer = [&](int u) { ans[u] = sum; };
    DsuOnTree q(tree);
    q.run(add, remove, answer);
    assert((ans == vector<int>{6, 2, 3}));
}
```

6.2.005 长链剖分

长链剖分按 “最深儿子” 划分长链，与轻重链按子树大小划分互补，专治状态只沿深度平移一位的树形递推：重儿子的深度型 DP 数组可让父节点直接偏移一位复用，轻儿子逐项暴力合并，总合并量由各长链长度之和 $O(n)$ 控制。它有两个经典用途分别落在别处——以深度为下标的 DP 合并见 6.5.003 长链剖分 DP 合并；$O(1)$ 查询 K 级祖先则在长链基础上再配一张倍增表，本条直接复用 6.1.006 树上距离、K 级祖先 的 `KthAncestor` 倍增预处理即可。相较 6.2.004 DSU on Tree 的子树频率合并，长链剖分只对 “深度平移” 型状态有效，两者不可互换。

6.2.006 毛毛虫剖分

毛毛虫剖分把树拆成少量 “主干路径 + 挂在其上一层附属点” 的毛毛虫块，用于满足毛毛虫结构约束的路径覆盖或 DP；它的成立高度依赖题目对路径、距离或度数的特殊约束，没有统一模板，通常在轻重链框架上按题目附加规则切链。本条复用 6.3.001 轻重链剖分 / HLD 的 `HLD` 作为主干与 DFS 序底座，再在其重链结构上按题意归并附属点。一般的路径修改查询直接用 6.3.001 即可，无需毛毛虫特化。

6.3 树链与虚树

6.3.001 轻重链剖分 / HLD

轻重链剖分把任意树路径拆成 $O(\log n)$ 段 DFS 序连续区间：按子树大小选重儿子，重链在 DFS 序中连续，任一路径跨越的轻边至多 $O(\log n)$ 条。配一维数据结构即可支持路径修改/查询、子树查询与 LCA。`path(u, v, op, edge)` 把路径拆段回调 `op(l, r, rev)`，`rev` 标记该段应逆序读取——这对非交换信息（矩阵、仿射变换）至关重要；`edge = true` 时处理边权（跳过 LCA 自身那一点）。

```cpp
struct HLD
{
    // n 是点数，tim 是 DFS 序计数器。
    int n, tim = 0;
    // fa、dep、sz、son、top、in 依次是父亲、深度、子树大小、重儿子、链顶和 DFS 序位置。
    // rev[i] 是 DFS 序位置 i 对应的原节点。
    vector<int> fa, dep, sz, son, top, in, rev;
```

```cpp

    HLD(const vector<vector<int>> &g, int root = 0)
        : n(g.size()), fa(n, -1), dep(n), sz(n, 1), son(n, -1), top(n), in(n), rev(n)
    {
        // g 是非空树，root 是根。
        assert(n > 0 && 0 <= root && root < n); // 调试检查，可删
        auto dfs1 = [&](auto &&self, int u, int p) -> void
        {
            fa[u] = p;
            for (int v : g[u])
            {
                if (v == p)
                {
                    continue;
                }
                dep[v] = dep[u] + 1;
                self(self, v, u);
                sz[u] += sz[v];
                if (son[u] < 0 || sz[v] > sz[son[u]])
                {
                    son[u] = v;
                }
            }
        };
        auto dfs2 = [&](auto &&self, int u, int h) -> void
        {
            top[u] = h;
            rev[tim] = u;
            in[u] = tim++;
            if (son[u] >= 0)
            {
                self(self, son[u], h);
            }
            for (int v : g[u])
            {
                if (v != fa[u] && v != son[u])
                {
                    self(self, v, v);
                }
            }
        };
        dfs1(dfs1, root, -1);
        dfs2(dfs2, root, root);
    }

    int lca(int u, int v) const
    {
        // 返回 u、v 的最近公共祖先。
        while (top[u] != top[v])
        {
            if (dep[top[u]] < dep[top[v]])
            {
```

```
                swap(u, v);
            }
            u = fa[top[u]];
        }
        return dep[u] < dep[v] ? u : v;
    }

    // op(l,r,rev): 区间为 [l,r)，rev 表示沿路径应逆序读取。
    template <class F> void path(int u, int v, F op, bool edge = false) const
    {
        // 按 u 到 v 的顺序回调各段；edge=true 时每条边存到较深端点。
        vector<pair<int, int>> right;
        while (top[u] != top[v])
        {
            if (dep[top[u]] >= dep[top[v]])
            {
                op(in[top[u]], in[u] + 1, true);
                u = fa[top[u]];
            }
            else
            {
                right.push_back({in[top[v]], in[v] + 1});
                v = fa[top[v]];
            }
        }
        if (dep[u] >= dep[v])
        {
            int l = in[v] + edge, r = in[u] + 1;
            if (l < r)
            {
                op(l, r, true);
            }
        }
        else
        {
            int l = in[u] + edge, r = in[v] + 1;
            if (l < r)
            {
                right.push_back({l, r});
            }
        }
        reverse(right.begin(), right.end());
        for (auto [l, r] : right)
        {
            op(l, r, false);
        }
    }

    pair<int, int> subtree(int u) const
    {
        // 返回 u 的子树对应的 DFS 序半开区间。
        return {in[u], in[u] + sz[u]};
```

```
    }
};
```

构造 `HLD(g, root)`；`lca` $O(\log n)$、`subtree(u)` 返回子树区间 $O(1)$、`path` 拆成 $O(\log n)$ 段回调，构造 $O(n)$ 时间、$O(n)$ 空间。把 `op` 接到线段树/树状数组即得完整路径数据结构，示例见 6.3.004 树上路径/子树数据结构；只保留询问关键点时压成 6.3.003 虚树。非交换信息务必用 `rev` 标记分别维护左右两半再合并，不能直接把两段区间并起来。

最小调用只记录所有拆分段的长度：

```
signed main()
{
    vector<vector<int>> tree{{1}, {0, 2}, {1}};
    HLD hld(tree);
    int len = 0;
    hld.path(0, 2, [&](int l, int r, bool rev)
    {
        (void)rev;
        len += r - l;
    });
    assert(len == 3);
}
```

6.3.002 换根树剖

在不重建剖分的前提下回答“动态根”下的子树范围。固定一次剖分后，查询 u 在当前根 `root` 下的子树分三类：$u = root$ 时是全树；`root` 不在 u 的原子树内时仍是原区间；否则答案是全树挖去“通向 `root` 的那个儿子”的子树，即至多两段区间。`childOnPath` 沿重链找出 u 到 `root` 路径上紧邻 u 的那个儿子。

```
class RerootHLD
{
    // n 是点数，timer 是 DFS 序计数器。
    int n, timer = 0;
    // tree 是树邻接表。
    vector<vector<int>> tree;
    // parent、depth、size、heavy、head、position 依次是父亲、深度、子树大小、重儿子、链顶和 DFS 序位置。
    // vertexAt[i] 是 DFS 序位置 i 对应的原节点。
    vector<int> parent, depth, size, heavy, head, position, vertexAt;

    void firstDfs(int u)
    {
        // 从 u 出发递归计算父亲、深度、子树大小和重儿子。
        auto dfs = [&](auto &&self, int x, int p) -> void
        {
            parent[x] = p;
            for (int v : tree[x])
            {
                if (v == p)
                {
```

```
                    continue;
                }
                depth[v] = depth[x] + 1;
                self(self, v, x);
                size[x] += size[v];
                if (heavy[x] == -1 || size[v] > size[heavy[x]])
                {
                    heavy[x] = v;
                }
            }
        };
        fill(size.begin(), size.end(), 1);
        dfs(dfs, u, -1);
    }

    void secondDfs(int u, int top)
    {
        // 从 u 开始递归分配链顶和 DFS 序。
        auto dfs = [&](auto &&self, int x, int top) -> void
        {
            head[x] = top;
            position[x] = timer++;
            vertexAt[position[x]] = x;
            if (heavy[x] != -1)
            {
                self(self, heavy[x], top);
            }
            for (int v : tree[x])
            {
                if (v != parent[x] && v != heavy[x])
                {
                    self(self, v, v);
                }
            }
        };
        dfs(dfs, u, top);
    }

    int childOnPath(int anc, int desc) const
    {
        // anc 是 desc 的真祖先；返回路径上紧邻 anc 的儿子。
        while (head[anc] != head[desc])
        {
            if (parent[head[desc]] == anc)
            {
                return head[desc];
            }
            desc = parent[head[desc]];
        }
        return vertexAt[position[anc] + 1];
    }

```

```
73    public:
74      RerootHLD(vector<vector<int>> tree, int root = 0)
75          : n(tree.size()), tree(move(tree)), parent(n, -1), depth(n), size(n), heavy(n, -1),
            ↪  head(n), position(n),
76            vertexAt(n)
77      {
78          // tree 是非空树，root 是预处理根。
79          assert(n > 0 && 0 <= root && root < n); // 调试检查，可删
80          firstDfs(root);
81          secondDfs(root, root);
82      }
83
84      vector<pair<int, int>> pathSegments(int u, int v) const
85      {
86          // 返回覆盖 u-v 路径的 DFS 序区间；区间顺序不表示路径方向。
87          vector<pair<int, int>> result;
88          while (head[u] ≠ head[v])
89          {
90              if (depth[head[u]] < depth[head[v]])
91              {
92                  swap(u, v);
93              }
94              result.push_back({position[head[u]], position[u] + 1});
95              u = parent[head[u]];
96          }
97          if (depth[u] > depth[v])
98          {
99              swap(u, v);
100         }
101         result.push_back({position[u], position[v] + 1});
102         return result;
103     }
104
105     int lca(int u, int v) const
106     {
107         // 返回 u、v 的最近公共祖先。
108         while (head[u] ≠ head[v])
109         {
110             if (depth[head[u]] < depth[head[v]])
111             {
112                 swap(u, v);
113             }
114             u = parent[head[u]];
115         }
116         return depth[u] < depth[v] ? u : v;
117     }
118
119     int distance(int u, int v) const
120     {
121         // 返回 u、v 间的边数。
122         int p = lca(u, v);
123         return depth[u] + depth[v] - 2 * depth[p];
```

```
    }

    vector<pair<int, int>> subtree(int u, int root) const
    {
        // 返回以 root 为当前根时 u 子树对应的至多两个 DFS 序区间。
        if (u == root)
        {
            return {{0, n}};
        }
        if (!(position[u] <= position[root] && position[root] < position[u] + size[u]))
        {
            return {{position[u], position[u] + size[u]}};
        }
        int child = childOnPath(u, root);
        vector<pair<int, int>> result;
        if (position[child])
        {
            result.push_back({0, position[child]});
        }
        if (position[child] + size[child] < n)
        {
            result.push_back({position[child] + size[child], n});
        }
        return result;
    }
};
```

构造 `RerootHLD(tree, root)`；`pathSegments`、`lca`、`distance` 及带动态根的 `subtree(u, root)`（返回至多 2 段 DFS 序区间）。构造 $O(n)$，`pathSegments`/`subtree` 均 $O(\log n)$，空间 $O(n)$。本类的 `lca` 也是 6.1.003 最近公共祖先：树链剖分 所引用的实现。仅静态根时用 6.3.001 轻重链剖分 / HLD 更直接。

6.3.003 虚树

虚树只保留询问关键点及其两两必要的 LCA，把规模压到 $O(k)$，供单次关键点 DP 在压缩树上运行。做法：关键点按 DFS 序 `in` 排序，相邻两点补入它们的 LCA 后再排序去重，用单调栈按祖先关系连边——栈中始终是当前点到根路径上的祖先链，弹到栈顶是当前点祖先为止再连边。返回点数不超过 $2k-1$。本条只依赖 6.1.002 最近公共祖先：倍增 的 `in` 与 `lca`/`anc`。

```
struct VTree
{
    // root 是虚树根；空虚树时为 -1。
    int root = -1;
    // nodes 按原树 DFS 序保存虚树点，edges 按儿子 DFS 序保存从父亲指向儿子的边。
    vector<int> nodes;
    vector<pair<int, int>> edges;
};

VTree virtualTree(vector<int> a, const LCA &lca)
{
```

```
    // a 是关键点列表，lca 是原树 LCA；返回关键点及必要 LCA 构成的虚树。
    if (a.empty())
    {
        return {};
    }
    sort(a.begin(),
         a.end(),
         [&](int x, int y)
         {
             return lca.in[x] < lca.in[y];
         });
    a.erase(unique(a.begin(), a.end()), a.end());
    int n = a.size();
    for (int i = 1; i < n; i++)
    {
        a.push_back(lca.lca(a[i - 1], a[i]));
    }
    sort(a.begin(),
         a.end(),
         [&](int x, int y)
         {
             return lca.in[x] < lca.in[y];
         });
    a.erase(unique(a.begin(), a.end()), a.end());
    vector<int> st;
    VTree ans;
    ans.root = a[0];
    ans.nodes = a;
    for (int u : a)
    {
        while (!st.empty() && !lca.anc(st.back(), u))
        {
            st.pop_back();
        }
        if (!st.empty())
        {
            ans.edges.push_back({st.back(), u});
        }
        st.push_back(u);
    }
    return ans;
}
```

`virtualTree(a, lca)` 传入关键点集与已建好的 `LCA`，返回 `VTree{root, nodes, edges}`（`a` 为空返回空树）。排序 $O(k \log k)$，补 LCA 与建树 $O(k \log n)$，点数 $\le 2k-1$。虚树与原树同构地保留了关键点间的祖先/路径关系，但边“长度”需按原树 `dist` 另算。多次不同关键点集询问时每次重建。

下面用一棵八点树检查关键点去重、补 LCA、连边和空集合（需同时粘贴 6.1.002 的 `LCA`）：

```
signed main()
{
    vector<vector<int>> g(8);
```

```
    vector<pair<int, int>> e{{0, 1}, {0, 2}, {1, 3}, {1, 4}, {2, 5}, {5, 6}, {5, 7}};
    for (auto [u, v] : e)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    LCA lca(g);
    VTree vt = virtualTree({3, 4, 6, 7, 3}, lca);
    assert(vt.root == 0);
    assert((vt.nodes == vector<int>{0, 1, 3, 4, 5, 6, 7}));
    vector<pair<int, int>> edges{{0, 1}, {1, 3}, {1, 4}, {0, 5}, {5, 6}, {5, 7}};
    assert(vt.edges == edges);
    vt = virtualTree({}, lca);
    assert(vt.root == -1 && vt.nodes.empty() && vt.edges.empty());
}
```

6.3.004 树上路径/子树数据结构

静态树上“单点赋值 + 路径点权和 + 固定根子树点权和”的组合板子：路径交给重链剖分（每条根路径 $\leq O(\log n)$ 条重链、链内 DFS 序连续），子树交给 DFS 序连续区间，点权放在 DFS 序上的树状数组里。它不支持动态连边断边——那需要 6.4.001 Link-Cut Tree。

```
template <class Acc = long long>
class TreePathSum
{
    // n 保存点数，ti 保存 DFS 序计数器。
    int n, ti = 0;
    // g 保存静态树邻接表。
    vector<vector<int>> g;
    // fa、dep、siz、son、top、in 依次保存父节点、深度、子树大小、重儿子、链顶和 DFS 序。
    vector<int> fa, dep, siz, son, top, in;
    // val 保存各点当前权值，bit 保存 DFS 序上的前缀和。
    vector<Acc> val, bit;

    void build(int root)
    {
        // 递归建立父亲、重儿子、链顶与 DFS 序。
        auto dfs1 = [&](auto &&self, int u, int p) -> void
        {
            fa[u] = p;
            for (int v : g[u])
            {
                if (v == p)
                {
                    continue;
                }
                dep[v] = dep[u] + 1;
                self(self, v, u);
                siz[u] += siz[v];
                if (son[u] == -1 || siz[v] > siz[son[u]])
```

```
                {
                    son[u] = v;
                }
            }
        };
        auto dfs2 = [&](auto &&self, int u, int h) -> void
        {
            top[u] = h;
            in[u] = ti++;
            if (son[u] != -1)
            {
                self(self, son[u], h);
            }
            for (int v : g[u])
            {
                if (v != fa[u] && v != son[u])
                {
                    self(self, v, v);
                }
            }
        };
        dfs1(dfs1, root, -1);
        dfs2(dfs2, root, root);
    }

    void addBit(int p, Acc v)
    {
        // p 是 0 下标 DFS 序位置，v 是增量；更新树状数组，无返回值。
        for (p++; p <= n; p += p & -p)
        {
            bit[p] += v;
        }
    }

    Acc sumBit(int p) const
    {
        // p 是右端点；返回半开前缀 [0,p) 的点权和。
        Acc ans = 0;
        for (; p; p -= p & -p)
        {
            ans += bit[p];
        }
        return ans;
    }

  public:
    TreePathSum(const vector<vector<int>> &g, const vector<Acc> &a, int root = 0)
        : n(g.size()), g(g), fa(n, -1), dep(n), siz(n, 1), son(n, -1), top(n), in(n), val(a),
        ↪ bit(n + 1)
    {
        // g 是静态树，a 是各点初值，root 是根；完成重链剖分与树状数组初始化，无返回值。
        assert(n > 0 && (int)a.size() == n && 0 <= root && root < n); // 调试检查，可删
```

```
        build(root);
        for (int u = 0; u < n; u++)
        {
            addBit(in[u], val[u]);
        }
    }

    void set(int u, Acc v)
    {
        // u 是节点编号，v 是新点权；执行单点赋值，无返回值。
        assert(0 <= u && u < n); // 调试检查，可删
        addBit(in[u], v - val[u]);
        val[u] = v;
    }

    Acc pathSum(int u, int v) const
    {
        // u、v 是路径两端；返回简单路径上全部点权之和。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删
        Acc ans = 0;
        while (top[u] != top[v])
        {
            if (dep[top[u]] < dep[top[v]])
            {
                swap(u, v);
            }
            ans += sumBit(in[u] + 1) - sumBit(in[top[u]]);
            u = fa[top[u]];
        }
        if (dep[u] > dep[v])
        {
            swap(u, v);
        }
        return ans + sumBit(in[v] + 1) - sumBit(in[u]);
    }

    Acc subtreeSum(int u) const
    {
        // u 是子树根；返回固定构造根下 u 的整棵子树点权和。
        assert(0 <= u && u < n); // 调试检查，可删
        return sumBit(in[u] + siz[u]) - sumBit(in[u]);
    }
};
```

构造 `TreePathSum q(g, a, root)`；默认累加类型为 `long long`，点权、`set` 参数及两个求和函数均使用 `Acc`。必须保证路径和、子树和以及 `set` 产生的点权差不超出 `Acc`；可能超过 64 位时使用 `TreePathSum<__int128>`，并传入 `vector<__int128>` 点权。预处理 $O(n)$，`set`/`subtreeSum` $O(\log n)$、`pathSum` $O(\log^2 n)$，空间 $O(n)$。求和满足可减性才能用树状数组；最值或非交换信息改用线段树接到 6.3.001 轻重链剖分 / HLD 的 `path` 回调。

下面在固定 7 点分叉树上检查同点路径、跨重链路径、根与内部子树，并在修改根、内部点和叶子后复查：

```cpp
signed main()
{
    vector<vector<int>> g(7);
    for (auto [u, v] : vector<pair<int, int>>{{0, 1}, {0, 2}, {1, 3}, {1, 4}, {2, 5}, {5, 6}})
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<long long> a{5, -2, 4, 7, 1, -3, 6};
    TreePathSum q(g, a);
    assert(q.pathSum(4, 4) == 1);
    assert(q.pathSum(3, 6) == 17);
    assert(q.subtreeSum(0) == 18);
    assert(q.subtreeSum(1) == 6);
    q.set(0, 10);
    assert(q.pathSum(3, 6) == 22);
    q.set(1, 8);
    assert(q.pathSum(3, 6) == 32);
    assert(q.subtreeSum(1) == 16);
    q.set(6, -4);
    assert(q.pathSum(3, 6) == 22);
    assert(q.subtreeSum(5) == -7);
    cout << "OK\n";
    return 0;
}
```

6.3.005 树上差分

静态树上有许多次路径加、最后才一次性询问全部点或边时，无需树剖逐次修改。固定根并令 $p=\operatorname{lca}(u,v)$：路径点加 x 在差分数组中执行 $d_u+=x, d_v+=x, d_p-=x$，若 p 不是根再令 $d_{\text{parent}(p)}-=x$；路径边加则执行 $d_u+=x, d_v+=x, d_p-=2x$。全部修改结束后按 DFS 序逆序把儿子累加到父亲，点差分直接落到节点，边差分在节点 u 处表示边 $(\text{parent}(u),u)$ 的答案。下面复用 6.1.002 最近公共祖先：倍增 的 `LCA`，不重复祖先表实现。

```cpp
// 依赖 6.1.002 最近公共祖先：倍增
template <class T = long long> class TreeDiff
{
    // n 是点数，root 是固定树根。
    int n, root;
    // lca 保存倍增祖先、深度与 DFS 序。
    LCA lca;
    // order 按父亲先于儿子的 DFS 序保存节点。
    vector<int> order;
    // dv 与 edgeDelta 分别保存路径点加、路径边加的树上差分。
    vector<T> dv, edgeDelta;

    vector<T> collect(vector<T> value) const
    {
        // value 是一份树上差分；返回自底向上累加后的每点结果。
```

```
        for (int i = n - 1; i > 0; i--)
        {
            int u = order[i];
            value[lca.up[0][u]] += value[u];
        }
        return value;
    }

  public:
    TreeDiff(const vector<vector<int>> &g, int root = 0)
        : n(g.size()), root(root), lca(g, root), order(n), dv(n), edgeDelta(n)
    {
        // g 是非空无向树，root 是固定根；预处理祖先与汇总顺序。
        assert(n > 0 && 0 <= root && root < n); // 调试检查，可删
        for (int u = 0; u < n; u++)
        {
            order[lca.in[u]] = u;
        }
    }

    void clear()
    {
        // 无参数；清空已经记录的点路径与边路径增量，无返回值。
        fill(dv.begin(), dv.end(), T{});
        fill(edgeDelta.begin(), edgeDelta.end(), T{});
    }

    void addVtxPath(int u, int v, const T &delta)
    {
        // u、v 是路径两端，delta 是增量；给简单路径上的每个点增加 delta。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删
        int p = lca.lca(u, v);
        dv[u] += delta;
        dv[v] += delta;
        dv[p] -= delta;
        if (p != root)
        {
            dv[lca.up[0][p]] -= delta;
        }
    }

    void addEdgePath(int u, int v, const T &delta)
    {
        // u、v 是路径两端，delta 是增量；给简单路径上的每条边增加 delta。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删
        int p = lca.lca(u, v);
        edgeDelta[u] += delta;
        edgeDelta[v] += delta;
        edgeDelta[p] -= delta;
        edgeDelta[p] -= delta;
    }

```

```
    vector<T> val() const
    {
        // 无参数；返回全部点累计的路径点增量，按节点编号排列。
        return collect(dv);
    }

    vector<T> edgeValues() const
    {
        // 无参数；返回每点到父亲边的累计增量，根位置为零。
        auto ans = collect(edgeDelta);
        ans[root] = T{};
        return ans;
    }
};
```

构造 `TreeDiff<T>(g,root)` 后用 `addVtxPath` 与 `addEdgePath` 分别记录两套互不干扰的增量；`val()`、`edgeValues()` 都在副本上汇总，可重复调用且不会清空记录，`clear()` 才重置。预处理沿用 LCA 的 $O(n \log n)$ 时间与空间，每次路径修改 $O(\log n)$，一次汇总 $O(n)$；`T` 须支持默认零值、`+=` 与 `-=`，并能容纳累计结果。在线穿插查询用 6.3.004 树上路径/子树数据结构，动态连边断边则用 6.4.001 Link-Cut Tree。

```
signed main()
{
    vector<vector<int>> g{{1, 2}, {0, 3}, {0}, {1}};
    TreeDiff<int> d(g);
    d.addVtxPath(2, 3, 5);
    d.addVtxPath(1, 3, 2);
    d.addEdgePath(2, 3, 5);
    d.addEdgePath(1, 3, 2);
    assert(d.val() == vector<int>({5, 7, 5, 7}));
    // 根为 0，边答案依次对应空边、0-1、0-2、1-3。
    assert(d.edgeValues() == vector<int>({0, 5, 5, 7}));
}
```

6.4 动态树

6.4.001 Link-Cut Tree

LCT 维护由 `link`/`cut` 动态变化的森林，支持连通性、点权修改与路径点权和。核心是用 Splay 维护每条“首选路径”：`access(x)` 把根到 x 的路径改成首选路径并暴露在一棵 Splay 里，`makeRoot` 用翻转标记换根，`findRoot` 找路径最左端。Splay 旋转只重排辅助树、不改变它表示的原森林；路径一旦暴露，辅助树根的 `sum` 就是整条路径的答案。`isRoot` 判某点是否为其所在 Splay 的根（看父亲是否认它作儿子）。

```
template <class Acc = long long>
class LinkCutTree
{
    struct Node
    {
```

```
        // ch[0..1] 依次保存辅助树左右儿子编号。
        array<int, 2> ch{};
        // fa 保存辅助树父节点，rev 标记辅助子树翻转。
        int fa = 0;
        bool rev = false;
        // val 保存当前点权，sum 保存辅助树中路径节点的聚合和。
        Acc val = 0, sum = 0;
    };

    // t[0] 是空节点，t[u+1] 对应外部 0 下标节点 u。
    vector<Node> t;

    bool isRoot(int x) const
    {
        // x 是内部节点编号；返回 x 是否为当前辅助树根。
        int f = t[x].fa;
        return !f || (t[f].ch[0] != x && t[f].ch[1] != x);
    }

    void pull(int x)
    {
        // x 是内部节点编号；由两个儿子重新计算路径聚合和，无返回值。
        t[x].sum = t[t[x].ch[0]].sum + t[x].val + t[t[x].ch[1]].sum;
    }

    void reverseNode(int x)
    {
        // x 是内部节点编号；翻转辅助子树并记录懒标记，无返回值。
        if (!x)
        {
            return;
        }
        swap(t[x].ch[0], t[x].ch[1]);
        t[x].rev ^= 1;
    }

    void push(int x)
    {
        // x 是内部节点编号；下传翻转标记，无返回值。
        if (!t[x].rev)
        {
            return;
        }
        reverseNode(t[x].ch[0]);
        reverseNode(t[x].ch[1]);
        t[x].rev = false;
    }

    void rotate(int x)
    {
        // x 是要上旋的内部节点；执行一次 Splay 旋转并维护聚合，无返回值。
        int y = t[x].fa, z = t[y].fa, k = t[y].ch[1] == x, w = t[x].ch[k ^ 1];
```

```
        if (!isRoot(y))
        {
            t[z].ch[t[z].ch[1] == y] = x;
        }
        t[x].fa = z;
        t[x].ch[k ^ 1] = y;
        t[y].fa = x;
        t[y].ch[k] = w;
        if (w)
        {
            t[w].fa = y;
        }
        pull(y);
        pull(x);
    }

    void splay(int x)
    {
        // x 是内部节点；把它旋到当前辅助树根，无返回值。
        vector<int> stk{x};
        for (int y = x; !isRoot(y); y = t[y].fa)
        {
            stk.push_back(t[y].fa);
        }
        while (!stk.empty())
        {
            push(stk.back());
            stk.pop_back();
        }
        while (!isRoot(x))
        {
            int y = t[x].fa, z = t[y].fa;
            if (!isRoot(y))
            {
                rotate((t[y].ch[1] == x) == (t[z].ch[1] == y) ? y : x);
            }
            rotate(x);
        }
    }

    void access(int x)
    {
        // x 是内部节点；把动态树根到 x 的路径改为首选路径，无返回值。
        int y = 0;
        while (x)
        {
            splay(x);
            int p = t[x].fa;
            t[x].ch[1] = y;
            pull(x);
            y = x;
            x = p;
```

```
        }
    }

    void makeRootNode(int x)
    {
        // x 是内部节点；把它改为所在动态树的根，无返回值。
        access(x);
        splay(x);
        reverseNode(x);
    }

    int findRootNode(int x)
    {
        // x 是内部节点；返回所在动态树根的内部编号。
        access(x);
        splay(x);
        while (true)
        {
            push(x);
            if (!t[x].ch[0])
            {
                break;
            }
            x = t[x].ch[0];
        }
        splay(x);
        return x;
    }

public:
    LinkCutTree(const vector<Acc> &a = {}) : t(a.size() + 1)
    {
        // a 是各节点初始点权；建立 n 个互不连通的节点，无返回值。
        for (int i = 0; i < (int)a.size(); i++)
        {
            t[i + 1].val = a[i];
            t[i + 1].sum = a[i];
        }
    }

    bool same(int u, int v)
    {
        // u、v 是外部节点编号；返回两点是否连通。
        return connected(u, v);
    }

    bool link(int u, int v)
    {
        // u、v 是外部节点编号；不连通时连边并返回 true，否则不修改并返回 false。
        u++;
        v++;
        makeRootNode(u);
```

```
        if (findRootNode(v) == u)
        {
            return false;
        }
        t[u].fa = v;
        return true;
    }

    bool cut(int u, int v)
    {
        // u、v 是外部节点编号；边存在时删除并返回 true，否则返回 false。
        u++;
        v++;
        makeRootNode(u);
        access(v);
        splay(v);
        if (t[v].ch[0] != u || t[u].ch[1])
        {
            return false;
        }
        t[v].ch[0] = 0;
        t[u].fa = 0;
        pull(v);
        return true;
    }

    bool connected(int u, int v)
    {
        // u、v 是外部节点编号；返回两点当前是否属于同一棵动态树。
        return findRootNode(u + 1) == findRootNode(v + 1);
    }

    void set(int u, Acc v)
    {
        // u 是外部节点编号，v 是新点权；执行单点赋值，无返回值。
        u++;
        access(u);
        splay(u);
        t[u].val = v;
        pull(u);
    }

    Acc pathSum(int u, int v)
    {
        // u、v 必须连通；返回两点简单路径上的点权和。
        assert(connected(u, v)); // 调试检查，可删
        u++;
        v++;
        makeRootNode(u);
        access(v);
        splay(v);
        return t[v].sum;
```

```
    }
};
```

构造 `LinkCutTree q(a)`；默认点权、`set` 参数和 `pathSum` 返回值均为 `long long`。必须保证任意时刻维护的路径和能装入 `Acc`；可能超过 64 位时使用 `LinkCutTree<__int128>`，并传入 `vector<__int128>` 点权。`link`/`cut` 失败时返回 `false`，`same`/`connected` 判连通；路径和要求两点连通。各操作均摊 $O(\log n)$、空间 $O(n)$，单次最坏 $O(n)$。它只维护首选路径、不含虚儿子子树信息；要子树聚合用 6.4.002 带子树信息的 LCT。

下面用六个点检查连边、断边、连通性、点权修改、路径和及失败操作：

```
signed main()
{
    LinkCutTree q({1, 2, 3, 4, 5, 6});
    assert(q.link(0, 1));
    assert(q.link(1, 2));
    assert(q.link(1, 3));
    assert(q.link(3, 4));
    assert(!q.link(2, 4));
    assert(q.pathSum(2, 4) == 14);
    q.set(1, -2);
    assert(q.pathSum(2, 4) == 10);
    assert(q.cut(1, 3));
    assert(!q.same(2, 4));
    assert(!q.cut(1, 3));
    assert(q.link(2, 5));
    assert(q.link(5, 4));
    assert(q.pathSum(0, 4) == 13);
    assert(!q.cut(0, 2));
}
```

6.4.002 带子树信息的 LCT

在 LCT 上额外维护虚儿子贡献，从而同时回答路径和与“指定根方向下的子树和”。关键在 `access` 更换首选右儿子时：旧右儿子转为虚儿子、新右儿子移出虚集合，用 `vir += 旧.sum - 新.sum` 维持所有非首选子树的贡献；`link` 时把接入子树的 `sum` 计入父亲的 `vir`。`path` 是首选路径和、`sum` 是含虚儿子的整个辅助子树和。

```
template <class Acc = long long>
class SubtreeLCT
{
    struct Node
    {
        // ch[0..1] 依次保存辅助树左右儿子编号。
        array<int, 2> ch{};
        // fa 保存辅助树父节点，rev 标记辅助子树翻转。
        int fa = 0;
        bool rev = false;
        // val、path、vir、sum 依次保存点权、首选路径和、虚儿子贡献、整棵辅助子树贡献。
```

```
        Acc val = 0, path = 0, vir = 0, sum = 0;
    };

    // t[0] 是空节点，t[u+1] 对应外部节点 u。
    vector<Node> t;

    bool isRoot(int x) const
    {
        // x 是内部节点编号；返回 x 是否为当前辅助树根。
        int f = t[x].fa;
        return !f || (t[f].ch[0] != x && t[f].ch[1] != x);
    }

    void pull(int x)
    {
        // x 是内部节点；重算首选路径和与包含虚儿子的总和，无返回值。
        auto [l, r] = t[x].ch;
        t[x].path = t[l].path + t[x].val + t[r].path;
        t[x].sum = t[l].sum + t[x].val + t[r].sum + t[x].vir;
    }

    void reverseNode(int x)
    {
        // x 是内部节点；翻转辅助子树并记录标记，无返回值。
        if (!x)
        {
            return;
        }
        swap(t[x].ch[0], t[x].ch[1]);
        t[x].rev ^= 1;
    }

    void push(int x)
    {
        // x 是内部节点；下传翻转标记，无返回值。
        if (!t[x].rev)
        {
            return;
        }
        reverseNode(t[x].ch[0]);
        reverseNode(t[x].ch[1]);
        t[x].rev = false;
    }

    void rotate(int x)
    {
        // x 是要上旋的内部节点；执行一次 Splay 旋转并维护信息，无返回值。
        int y = t[x].fa, z = t[y].fa, k = t[y].ch[1] == x, w = t[x].ch[k ^ 1];
        if (!isRoot(y))
        {
            t[z].ch[t[z].ch[1] == y] = x;
        }
```

```
        t[x].fa = z;
        t[x].ch[k ^ 1] = y;
        t[y].fa = x;
        t[y].ch[k] = w;
        if (w)
        {
            t[w].fa = y;
        }
        pull(y);
        pull(x);
    }

    void splay(int x)
    {
        // x 是内部节点；把它旋到辅助树根，无返回值。
        vector<int> stk{x};
        for (int y = x; !isRoot(y); y = t[y].fa)
        {
            stk.push_back(t[y].fa);
        }
        while (!stk.empty())
        {
            push(stk.back());
            stk.pop_back();
        }
        while (!isRoot(x))
        {
            int y = t[x].fa, z = t[y].fa;
            if (!isRoot(y))
            {
                rotate((t[y].ch[1] == x) == (t[z].ch[1] == y) ? y : x);
            }
            rotate(x);
        }
    }

    void access(int x)
    {
        // x 是内部节点；切换首选儿子并同步虚儿子贡献，无返回值。
        int y = 0, z = x;
        while (z)
        {
            splay(z);
            int p = t[z].fa;
            t[z].vir += t[t[z].ch[1]].sum - t[y].sum;
            t[z].ch[1] = y;
            pull(z);
            y = z;
            z = p;
        }
        splay(x);
    }
```

```

    void makeRootNode(int x)
    {
        // x 是内部节点；把它改为所在动态树的根，无返回值。
        access(x);
        reverseNode(x);
    }

    int findRootNode(int x)
    {
        // x 是内部节点；返回所在动态树根的内部编号。
        access(x);
        while (true)
        {
            push(x);
            if (!t[x].ch[0])
            {
                break;
            }
            x = t[x].ch[0];
        }
        splay(x);
        return x;
    }

public:
    SubtreeLCT(const vector<Acc> &a = {}) : t(a.size() + 1)
    {
        // a 是各点初始权值；建立 n 个互不连通节点，无返回值。
        for (int i = 0; i < (int)a.size(); i++)
        {
            t[i + 1].val = a[i];
            t[i + 1].path = a[i];
            t[i + 1].sum = a[i];
        }
    }

    bool link(int u, int v)
    {
        // u、v 是外部节点编号；不连通时连边并返回 true，否则返回 false。
        u++;
        v++;
        makeRootNode(u);
        if (findRootNode(v) == u)
        {
            return false;
        }
        access(v);
        t[u].fa = v;
        t[v].vir += t[u].sum;
        pull(v);
        return true;
```

```
    }

    bool cut(int u, int v)
    {
        // u、v 是外部节点编号；边存在时删除并返回 true，否则返回 false。
        u++;
        v++;
        makeRootNode(u);
        access(v);
        if (t[v].ch[0] != u || t[u].ch[1])
        {
            return false;
        }
        t[v].ch[0] = 0;
        t[u].fa = 0;
        pull(v);
        return true;
    }

    bool connected(int u, int v)
    {
        // u、v 是外部节点编号；返回两点当前是否属于同一棵动态树。
        return findRootNode(u + 1) == findRootNode(v + 1);
    }

    void set(int u, Acc v)
    {
        // u 是外部节点编号，v 是新点权；执行单点赋值，无返回值。
        u++;
        access(u);
        t[u].val = v;
        pull(u);
    }

    Acc pathSum(int u, int v)
    {
        // u、v 必须连通；返回两点简单路径上的点权和。
        assert(connected(u, v)); // 调试检查，可删
        u++;
        v++;
        makeRootNode(u);
        access(v);
        return t[v].path;
    }

    Acc subtreeSum(int root, int u)
    {
        // root 是有根树根，u 是子树根；返回该根方向下 u 的子树点权和。
        assert(connected(root, u)); // 调试检查，可删
        root++;
        u++;
        makeRootNode(root);
```

```
        access(u);
        return t[u].val + t[u].vir;
    }
};
```

构造 `SubtreeLCT q(a)`；默认点权、`set` 参数、路径和与子树和均为 `long long`。必须保证所有路径聚合、虚儿子贡献和子树和能装入 `Acc`；可能超过 64 位时使用 `SubtreeLCT<__int128>`，并传入 `vector<__int128>` 点权。`subtreeSum(root, u)` 要求两点连通，先把 `root` 置根再返回 u 的有根子树和。各操作均摊 $O(\log n)$、空间 $O(n)$。子树和依赖 `vir` 的差量维护，故聚合须可减。

下面用六个点检查路径和、换根子树、点权修改与动态连边断边：

```
signed main()
{
    SubtreeLCT q({1, 2, 3, 4, 5, 6});
    assert(q.link(0, 1));
    assert(q.link(1, 2));
    assert(q.link(1, 3));
    assert(q.link(3, 4));
    assert(q.link(3, 5));
    assert(q.pathSum(2, 5) == 15);
    assert(q.subtreeSum(0, 1) == 20);
    assert(q.subtreeSum(2, 1) == 18);
    q.set(3, -4);
    assert(q.subtreeSum(0, 1) == 12);
    assert(q.cut(1, 3));
    assert(!q.connected(0, 5));
    assert(q.subtreeSum(4, 3) == 2);
    assert(q.link(2, 5));
    assert(q.pathSum(0, 4) == 13);
    assert(q.subtreeSum(0, 1) == 12);
    assert(!q.link(0, 4));
}
```

6.4.003 动态割边、割点 LCT

这里给两套桥/割点维护。`OnlineBridge` 只支持加边并实时维护桥数：用 “连通块 DSU + 边双 DSU + 生成森林” 三件套，新边跨连通块则桥数 $+1$，成环则把环上（两端到 LCA）的桥全部收缩进一个边双、相应减少桥数；`makeRoot` 靠翻父边实现路径反转。`DynamicCut` 是通用基准版，按边编号加删，每次 `analyze` 重跑一遍 Tarjan 求 `low` 值判桥与割点，简单但每次线性。

```
class OnlineBridge
{
    // d2、dc 分别保存边双代表元和连通块代表元。
    vector<int> d2, dc;
    // fa 保存当前生成森林父节点，siz 保存连通块大小。
    vector<int> fa, siz;
    // vis 保存最近一次 mergePath 的访问轮次。
    vector<int> vis;
```

```
    // ti 是访问轮次，cnt 是当前桥数。
    int ti = 0, cnt = 0;

    int find2(int x)
    {
        // x 是节点编号；返回其当前边双连通分量代表元。
        if (x == -1)
        {
            return -1;
        }
        return d2[x] == x ? x : d2[x] = find2(d2[x]);
    }

    int findC(int x)
    {
        // x 是节点编号；返回其当前连通块代表元。
        if (x == -1)
        {
            return -1;
        }
        x = find2(x);
        return dc[x] == x ? x : dc[x] = findC(dc[x]);
    }

    void makeRoot(int x)
    {
        // x 是节点编号；翻转生成森林中 x 到原根的父边，无返回值。
        x = find2(x);
        int root = x, pre = -1;
        while (x != -1)
        {
            int p = find2(fa[x]);
            fa[x] = pre;
            dc[x] = root;
            pre = x;
            x = p;
        }
        siz[root] = siz[pre];
    }

    void mergePath(int a, int b)
    {
        // a、b 已在同一连通块；压缩新环上的全部桥，无返回值。
        ti++;
        vector<int> pa, pb;
        int lca = -1;
        while (lca == -1)
        {
            if (a != -1)
            {
                a = find2(a);
                pa.push_back(a);
```

```
                if (vis[a] == ti)
                {
                    lca = a;
                    break;
                }
                vis[a] = ti;
                a = fa[a];
            }
            swap(a, b);
            swap(pa, pb);
        }
        for (auto x : pa)
        {
            d2[x] = lca;
            if (x == lca)
            {
                break;
            }
            cnt--;
        }
        for (auto x : pb)
        {
            d2[x] = lca;
            if (x == lca)
            {
                break;
            }
            cnt--;
        }
    }

public:
    OnlineBridge(int n) : d2(n), dc(n), fa(n, -1), siz(n, 1), vis(n)
    {
        // n 是固定点数；建立没有边的无向图，无返回值。
        iota(d2.begin(), d2.end(), 0);
        iota(dc.begin(), dc.end(), 0);
    }

    void addEdge(int a, int b)
    {
        // a、b 是无向边两端；加入边并更新当前桥数，无返回值，允许重边和自环。
        a = find2(a);
        b = find2(b);
        if (a == b)
        {
            return;
        }
        int ca = findC(a), cb = findC(b);
        if (ca != cb)
        {
            cnt++;
```

```
            if (siz[ca] > siz[cb])
            {
                swap(a, b);
                swap(ca, cb);
            }
            makeRoot(a);
            fa[a] = b;
            dc[a] = b;
            siz[cb] += siz[a];
        }
        else
        {
            mergePath(a, b);
        }
    }

    int bridges() const
    {
        // 无参数；返回当前无向图中的桥数。
        return cnt;
    }
};

class DynamicCut
{
    // n 保存固定点数。
    int n;
    // e[id][0..1] 依次保存第 id 条边的两个端点，on 标记边当前是否存在。
    vector<array<int, 2>> e;
    vector<char> on;

  public:
    DynamicCut(int n) : n(n)
    {
        // n 是固定点数；建立空无向图，无返回值。
    }

    int addEdge(int u, int v)
    {
        // u、v 是边的两个端点；加入边并返回稳定边编号。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删
        e.push_back({u, v});
        on.push_back(1);
        return e.size() - 1;
    }

    bool eraseEdge(int id)
    {
        // id 是 addEdge 返回的边编号；边存在时删除并返回 true，否则返回 false。
        if (id < 0 || id >= (int)e.size() || !on[id])
        {
            return false;
```

```
        }
        on[id] = 0;
        return true;
    }

    pair<vector<int>, vector<int>> analyze() const
    {
        // 无参数；返回 {全部桥的边编号, 全部割点编号}，两部分都按编号升序排列。
        vector<vector<array<int, 2>>> g(n);
        for (int id = 0; id < (int)e.size(); id++)
        {
            if (!on[id])
            {
                continue;
            }
            auto [u, v] = e[id];
            g[u].push_back({v, id});
            g[v].push_back({u, id});
        }
        vector<int> dfn(n), low(n), cut(n), bridge;
        int ti = 0;
        auto dfs = [&](auto &&self, int u, int pe) -> void
        {
            dfn[u] = low[u] = ++ti;
            int child = 0;
            for (auto [v, id] : g[u])
            {
                if (id == pe)
                {
                    continue;
                }
                if (!dfn[v])
                {
                    child++;
                    self(self, v, id);
                    chmin(low[u], low[v]);
                    if (low[v] > dfn[u])
                    {
                        bridge.push_back(id);
                    }
                    if (pe != -1 && low[v] >= dfn[u])
                    {
                        cut[u] = 1;
                    }
                }
                else
                {
                    chmin(low[u], dfn[v]);
                }
            }
            if (pe == -1)
            {
```

```
                cut[u] = child > 1;
            }
        };
        for (int s = 0; s < n; s++)
        {
            if (dfn[s])
            {
                continue;
            }
            dfs(dfs, s, -1);
        }
        vector<int> point;
        for (int u = 0; u < n; u++)
        {
            if (cut[u])
            {
                point.push_back(u);
            }
        }
        sort(bridge.begin(), bridge.end());
        return {bridge, point};
    }
};
```

只加边且只问桥数用 `OnlineBridge::addEdge/bridges`（m 次加边总 $O((n+m)\log n)$、空间 $O(n)$，允许重边自环）；要删边或要割点列表用 `DynamicCut::addEdge/eraseEdge/analyze`，`analyze` 返回按编号升序的（桥边编号，割点编号），更新 $O(1)$、每次分析 $O(n+m)$。其中 Tarjan 的递归深度最坏为 $O(n)$，深链能否直接运行取决于当前评测机的线程栈容量。频繁删边而只要连通性时另有线段树分治 + 可撤销并查集（见图论章），本条 `DynamicCut` 只作正确性基准。

`DynamicCut` 用固定图覆盖链、环、平行边、自环和删边；`OnlineBridge` 用固定加边顺序覆盖跨连通块、成环、重边和自环：

```
signed main()
{
    {
        DynamicCut q(4);
        int e0 = q.addEdge(0, 1);
        int e1 = q.addEdge(1, 2);
        int e2 = q.addEdge(2, 0);
        int e3 = q.addEdge(2, 3);
        q.addEdge(3, 3);
        assert((q.analyze() == pair{vector<int>{e3}, vector<int>{2}}));
        assert(q.eraseEdge(e2));
        assert((q.analyze() == pair{vector<int>{e0, e1, e3}, vector<int>{1, 2}}));
    }
    {
        DynamicCut q(3);
        int e0 = q.addEdge(0, 1);
        int e1 = q.addEdge(0, 1);
        int e2 = q.addEdge(1, 2);
```

```
        assert((q.analyze() == pair{vector<int>{e2}, vector<int>{1}}));
        assert(q.eraseEdge(e1));
        assert((q.analyze() == pair{vector<int>{e0, e2}, vector<int>{1}}));
    }
    {
        OnlineBridge q(5);
        q.addEdge(0, 1);
        q.addEdge(1, 2);
        q.addEdge(2, 3);
        assert(q.bridges() == 3);
        q.addEdge(0, 2);
        assert(q.bridges() == 1);
        q.addEdge(1, 3);
        assert(q.bridges() == 0);
        q.addEdge(3, 4);
        assert(q.bridges() == 1);
        q.addEdge(3, 4);
        assert(q.bridges() == 0);
        q.addEdge(4, 4);
        assert(q.bridges() == 0);
    }
}
```

6.4.004 全局平衡二叉树

全局平衡二叉树把静态树的每条重链按 “轻子树质量” 选加权中点建成二叉树，使全局深度 $O(\log n)$，为树 DP 的单点修改提供统一的向上更新链。重链上点 u 的权重取 $siz[u]-siz[son[u]]$（挂在该位置的轻子树总质量），每次取累计质量的中点作根，保证向任一儿子下降时剩余质量至少减半；轻链递归建成子树后用虚父边挂到对应重链点上。

```
class GlobalTree
{
    // n 保存点数，root 保存全局平衡树根。
    int n, root;
    // g 保存原树邻接表。
    vector<vector<int>> g;
    // pa、siz、son 依次保存原树父节点、子树大小和重儿子。
    vector<int> pa, siz, son;
    // ch[u][0..1] 保存全局平衡树左右儿子，fa 保存平衡树父边或轻边父亲。
    vector<array<int, 2>> ch;
    vector<int> fa;

    void dfs(int root)
    {
        // 递归计算原树父亲、子树大小和重儿子。
        auto work = [&](auto &&self, int u) -> void
        {
            siz[u] = 1;
            for (int v : g[u])
            {
```

```
                if (v == pa[u])
                {
                    continue;
                }
                pa[v] = u;
                self(self, v);
                siz[u] += siz[v];
                if (son[u] == -1 || siz[v] > siz[son[u]])
                {
                    son[u] = v;
                }
            }
        };
        work(work, root);
    }

    int buildSeq(const vector<int> &v, const vector<int> &pre, int l, int r)
    {
        // v 是一条重链，pre 是轻子树质量前缀和，l、r 是当前闭区间；返回加权中点根。
        if (l > r)
        {
            return -1;
        }
        int need = (pre[l] + pre[r + 1]) / 2;
        int m = lower_bound(pre.begin() + l + 1, pre.begin() + r + 2, need) - pre.begin() - 1;
        m = clamp(m, l, r);
        int u = v[m];
        ch[u][0] = buildSeq(v, pre, l, m - 1);
        ch[u][1] = buildSeq(v, pre, m + 1, r);
        if (ch[u][0] != -1)
        {
            fa[ch[u][0]] = u;
        }
        if (ch[u][1] != -1)
        {
            fa[ch[u][1]] = u;
        }
        return u;
    }

    int buildChain(int x)
    {
        // x 是一条重链链顶；递归建立轻链并返回该链的平衡树根。
        vector<int> v;
        for (int u = x; u != -1; u = son[u])
        {
            v.push_back(u);
        }
        for (auto u : v)
        {
            for (auto w : g[u])
            {
```

```
                if (w == pa[u] || w == son[u])
                {
                    continue;
                }
                int z = buildChain(w);
                fa[z] = u;
            }
        }
        vector<int> pre(v.size() + 1);
        for (int i = 0; i < (int)v.size(); i++)
        {
            pre[i + 1] = pre[i] + siz[v[i]] - (son[v[i]] == -1 ? 0 : siz[son[v[i]]]);
        }
        return buildSeq(v, pre, 0, v.size() - 1);
    }

  public:
    GlobalTree(const vector<vector<int>> &g, int rt = 0)
        : n(g.size()), root(-1), g(g), pa(n, -1), siz(n), son(n, -1), ch(n, array<int, 2>{-1,
        ↪ -1}), fa(n, -1)
    {
        // g 是静态树，rt 是原树根；建立全局平衡二叉树，无返回值。
        assert(n > 0 && 0 <= rt && rt < n); // 调试检查，可删
        dfs(rt);
        root = buildChain(rt);
    }

    int getRoot() const
    {
        // 无参数；返回全局平衡二叉树根节点编号。
        return root;
    }

    const vector<array<int, 2>> &children() const
    {
        // 无参数；返回每个节点在平衡二叉树中的左右儿子。
        return ch;
    }

    const vector<int> &parents() const
    {
        // 无参数；返回平衡树父边与轻边共同组成的父指针。
        return fa;
    }

    const vector<int> &heavy() const
    {
        // 无参数；返回原树中每个节点的重儿子。
        return son;
    }

    const vector<int> &parTree() const
```

```
    {
        // 无参数；返回原树父节点数组。
        return pa;
    }
};
```

构造 `GlobalTree tr(g, root)`；`children()` 给二叉树实儿子、`parents()` 给 “实儿子父边 + 连接轻链的虚父边” 的混合父指针，`heavy()`/`parTree()` 给原树重儿子与父亲。构造 $O(n \log n)$ 时间、$O(n)$ 空间，混合父链长 $O(\log n)$。它只是拓扑底座、不含题目信息，动态 DP 用法见 6.4.005 动态 DP。相比用 LCT 做动态 DP，全局平衡树是静态结构、常数更小，但不支持连边断边。

6.4.005 动态 DP

维护固定树、点权可变时的最大权独立集（相邻不同选）。状态 “不选/选当前点”，重儿子转移写成 2×2 的 max-plus 矩阵 $\begin{pmatrix} g_0 & g_0 \\ g_1 & -\infty \end{pmatrix}$（$g$ 为并入轻儿子后的节点常量），沿重链是矩阵连乘；轻儿子贡献并进 g。挂在 6.4.004 全局平衡二叉树 上后，一次单点修改只需沿混合父链重算 $O(\log n)$ 个矩阵，跨轻边时先扣除旧贡献、重算后再加回。允许选空集，故全负时答案为 0。

```
// 依赖 6.4.004 全局平衡二叉树
template <class Acc = long long>
class DynamicMWIS
{
    struct Mat
    {
        // nullopt 表示不可达，其余项保存 Acc 类型的 max-plus 值。
        array<array<optional<Acc>, 2>, 2> a{};
    };

    // tr 保存全局平衡二叉树拓扑。
    GlobalTree tr;
    // ch、fa、son、pa 分别引用平衡树儿子、混合父指针、重儿子和原树父节点。
    vector<array<int, 2>> ch;
    vector<int> fa, son, pa;
    // w 是点权，g[u] 是不含重儿子的“不选/选 u”贡献。
    vector<Acc> w;
    vector<array<Acc, 2>> g;
    // s 保存每个平衡树子树合并后的 max-plus 矩阵。
    vector<Mat> s;
    // tree 保存原树邻接表。
    vector<vector<int>> tree;

    static Mat mul(const Mat &x, const Mat &y)
    {
        // x、y 是两个 max-plus 矩阵；返回按顺序合并后的矩阵。
        Mat z;
        for (int i = 0; i < 2; i++)
        {
            for (int j = 0; j < 2; j++)
            {
```

```
                for (int k = 0; k < 2; k++)
                {
                    if (!x.a[i][k] || !y.a[k][j])
                    {
                        continue;
                    }
                    Acc v = *x.a[i][k] + *y.a[k][j];
                    if (!z.a[i][j] || *z.a[i][j] < v)
                    {
                        z.a[i][j] = v;
                    }
                }
            }
        }
        return z;
    }

    bool isChild(int x) const
    {
        // x 是节点编号；返回 x 到 fa[x] 是否为平衡二叉树实儿子边。
        return fa[x] != -1 && (ch[fa[x]][0] == x || ch[fa[x]][1] == x);
    }

    void pull(int x)
    {
        // x 是节点编号；由自身转移和左右儿子重算矩阵，无返回值。
        Mat cur;
        cur.a[0][0] = g[x][0];
        cur.a[0][1] = g[x][0];
        cur.a[1][0] = g[x][1];
        if (ch[x][0] != -1)
        {
            cur = mul(s[ch[x][0]], cur);
        }
        if (ch[x][1] != -1)
        {
            cur = mul(cur, s[ch[x][1]]);
        }
        s[x] = cur;
    }

    void init(int root)
    {
        // 递归求静态 DP，并初始化各点的轻儿子贡献。
        vector<array<Acc, 2>> f(w.size());
        auto dfs = [&](auto &&self, int u) -> void
        {
            f[u] = {0, w[u]};
            for (int v : tree[u])
            {
                if (pa[v] != u)
                {
```

```
                    continue;
                }
                self(self, v);
                f[u][0] += max(f[v][0], f[v][1]);
                f[u][1] += f[v][0];
                if (v != son[u])
                {
                    g[u][0] += max(f[v][0], f[v][1]);
                    g[u][1] += f[v][0];
                }
            }
            g[u][1] += w[u];
        };
        dfs(dfs, root);
    }

    void buildMat()
    {
        // 对平衡二叉树森林作递归后序，建立全部矩阵。
        auto dfs = [&](auto &&self, int u) -> void
        {
            for (int v : ch[u])
            {
                if (v != -1)
                {
                    self(self, v);
                }
            }
            pull(u);
        };
        for (int u = 0; u < (int)w.size(); u++)
        {
            if (!isChild(u))
            {
                dfs(dfs, u);
            }
        }
    }

public:
    DynamicMWIS(const vector<vector<int>> &tree, const vector<Acc> &w, int root = 0)
        : tr(tree, root), ch(tr.children()), fa(tr.parents()), son(tr.heavy()),
        ↪  pa(tr.parTree()), w(w), g(w.size()),
          s(w.size()), tree(tree)
    {
        // tree 是静态树，w 是可修改点权，root 是固定根；初始化动态最大权独立集，无返回值。
        assert(tree.size() == w.size()); // 调试检查，可删
        init(root);
        buildMat();
    }

    void set(int x, Acc v)
```

```
    {
        // x 是节点编号，v 是新点权；更新点权并维护全树最大权独立集，无返回值。
        assert(0 <= x && x < (int)w.size()); // 调试检查，可删
        g[x][1] += v - w[x];
        w[x] = v;
        while (fa[x] != -1)
        {
            if (isChild(x))
            {
                pull(x);
            }
            else
            {
                int p = fa[x];
                Acc old0 = max(*s[x].a[0][0], *s[x].a[1][0]);
                Acc old1 = *s[x].a[0][0];
                g[p][0] -= old0;
                g[p][1] -= old1;
                pull(x);
                g[p][0] += max(*s[x].a[0][0], *s[x].a[1][0]);
                g[p][1] += *s[x].a[0][0];
            }
            x = fa[x];
        }
        pull(x);
    }

    Acc answer() const
    {
        // 无参数；返回当前整棵树最大权独立集的权值，允许选择空集。
        int r = tr.getRoot();
        return max(*s[r].a[0][0], *s[r].a[1][0]);
    }
};
```

构造 `DynamicMWIS q(tree, w, root)`；默认点权、`set` 参数及 `answer()` 返回值均为 `long long`。必须保证所有有限 DP 值、轻儿子贡献和矩阵乘加结果能装入 `Acc`；可能超过 64 位时使用 `DynamicMWIS<__int128>`，并传入 `vector<__int128>` 点权。预处理 $O(n \log n)$、单点修改 $O(\log n)$、查询 $O(1)$、空间 $O(n)$。它要求树结构固定、只改点权。改成其他矩阵化树 DP 时，除了重新定义矩阵乘法，还必须保证轻子树旧贡献可以撤销；一般半环没有减法，不能直接照搬本实现。

下面用一棵六点树检查修改根、内部点、叶子与全负权：

```
signed main()
{
    {
        vector<vector<int>> g(6);
        vector<pair<int, int>> e{{0, 1}, {0, 2}, {1, 3}, {1, 4}, {2, 5}};
        for (auto [u, v] : e)
        {
            g[u].push_back(v);
```

```
            g[v].push_back(u);
        }
        DynamicMWIS q(g, {5, 4, 3, 2, 6, 7});
        assert(q.answer() == 20);
        q.set(0, -10);
        assert(q.answer() == 15);
        q.set(4, -8);
        assert(q.answer() == 11);
        q.set(5, -20);
        assert(q.answer() == 7);
        q.set(1, 100);
        assert(q.answer() == 103);
        q.set(1, -4);
        q.set(2, -3);
        q.set(3, -2);
        assert(q.answer() == 0);
    }
}
```

6.4.006 离线动态图连通块点权和

一般图上边可以加删、点权可以修改、询问某点所在连通块点权和；删边会拆开环，在线的 LCT 无法直接处理，操作全部已知时用线段树分治。把时刻 $0 \dots q-1$ 建成线段树：每条边在它存在的时间区间 $[$加入，删除$)$ 内有效，每次“点 u 加 x”在 $[i, q)$ 内有效，把它们挂到覆盖对应区间的 $O(\log q)$ 个结点上。DFS 线段树时，进入结点就用并查集合并挂在上面的边、把加值加到 u 当前所在块的根上，到叶子 i 回答该时刻的查询，退出结点时撤销本结点的全部修改。

点加值挂在结点上时直接加到当时的块根：更深的结点只会继续合并，块只会变大，所以这个加值始终被包含在 u 所在块的和里。撤销要求并查集不做路径压缩，只按大小合并，每次修改前把被改点的旧状态压栈。

```
vector<int> offlineCSum(const vector<int> &a, const vector<array<int, 4>> &op)
{
    // a 是初始点权；op[i] 的四维依次是类型、u、v、x：类型 0 加边 u-v，1 删边 u-v，
    // 2 给点 u 加 x，3 查询 u 所在连通块的点权和。加边时该边不存在，删边时该边存在。
    // 返回全部类型 3 查询的结果，顺序与操作出现顺序相同。
    int n = a.size(), q = op.size();
    // seg[p] 是时间线段树结点 p 上的事件，每个事件的四维依次是类型（0 合并、1 加值）、u、v、x。
    vector<vector<array<int, 4>>> seg(max(1LL, 4 * q));
    auto add = [&](auto &&self, int p, int l, int r, int ql, int qr, array<int, 4> e) -> void
    {
        // 把事件 e 挂到覆盖时间区间 [ql,qr) 的结点上；p 管辖 [l,r)。
        if (qr <= l || r <= ql)
        {
            return;
        }
        if (ql <= l && r <= qr)
        {
            seg[p].push_back(e);
```

```
            return;
        }
        int m = (l + r) / 2;
        self(self, p * 2, l, m, ql, qr, e);
        self(self, p * 2 + 1, m, r, ql, qr, e);
    };
    map<pair<int, int>, int> born; // born[{u,v}] 是当前存在的边的加入时刻。
    for (int i = 0; i < q; i++)
    {
        auto [type, u, v, x] = op[i];
        if (type == 0)
        {
            born[minmax(u, v)] = i;
        }
        else if (type == 1)
        {
            auto e = minmax(u, v);
            add(add, 1, 0, q, born[e], i, {0, e.first, e.second, 0});
            born.erase(e);
        }
        else if (type == 2)
        {
            // 点加值从此刻起一直有效，相当于在 [i,q) 内给所在连通块加 x。
            add(add, 1, 0, q, i, q, {1, u, 0, x});
        }
    }
    for (auto [e, l] : born)
    {
        add(add, 1, 0, q, l, q, {0, e.first, e.second, 0});
    }
    // fa 是不压缩路径的按大小合并并查集，根的 fa 为 -大小；sum[r] 是以 r 为根的连通块点权和。
    vector<int> fa(n, -1), sum = a, ans;
    // his 是修改记录，三维依次是被改的点、旧 fa、旧 sum。
    vector<array<int, 3>> his;
    auto root = [&](int u)
    {
        // 返回 u 所在连通块的根。
        while (fa[u] >= 0)
        {
            u = fa[u];
        }
        return u;
    };
    auto save = [&](int u)
    {
        // 记录点 u 修改前的状态。
        his.push_back({u, fa[u], sum[u]});
    };
    auto rollback = [&](int at)
    {
        // 撤销到 his 只有 at 条记录时的状态。
        while ((int)his.size() > at)
```

```
        {
            auto [u, f, s] = his.back();
            his.pop_back();
            fa[u] = f;
            sum[u] = s;
        }
    };
    auto solve = [&](auto &&self, int p, int l, int r) -> void
    {
        // 进入结点 p（管辖时刻 [l,r)）时应用其事件，到叶子回答查询，退出时回滚。
        int at = his.size();
        for (auto [type, u, v, x] : seg[p])
        {
            u = root(u);
            if (type == 1)
            {
                save(u);
                sum[u] += x;
                continue;
            }
            v = root(v);
            if (u == v)
            {
                continue;
            }
            if (fa[u] > fa[v])
            {
                swap(u, v);
            }
            save(u);
            save(v);
            fa[u] += fa[v];
            fa[v] = u;
            sum[u] += sum[v];
        }
        if (r - l == 1)
        {
            if (op[l][0] == 3)
            {
                ans.push_back(sum[root(op[l][1])]);
            }
        }
        else
        {
            int m = (l + r) / 2;
            self(self, p * 2, l, m);
            self(self, p * 2 + 1, m, r);
        }
        rollback(at);
    };
    if (q)
    {
```

```
        solve(solve, 1, 0, q);
    }
    return ans;
}
```

`offlineCSum(a,op)` 中 `op[i]` 的四维依次是类型、u、v、x：类型 0 加边 u-v（此时该边不存在），1 删边 u-v（此时该边存在），2 给点 u 加 x，3 查询 u 所在块的点权和；未用到的维写 0。返回全部查询结果，顺序与操作出现顺序相同。允许自环（合并时自动忽略），不允许同一时刻存在两条相同的边。设操作数为 q，每个事件挂到 $O(\log q)$ 个结点，按大小合并使 `root` 为 $O(\log n)$，总时间 $O(q \log q \log n)$，空间 $O(q \log q + n)$。

只有加边没有删边时直接用普通并查集；在森林上在线维护路径或子树信息时分别用 6.4.001 Link-Cut Tree 与 6.4.002 带子树信息的 LCT，在线维护森林的连通块整体信息用 6.4.007 Euler Tour Tree。下面的操作序列含加边、点加、删边与查询：

```
signed main()
{
    vector<int> a{1, 2, 3};
    vector<array<int, 4>> op{{3, 0, 0, 0}, {0, 0, 1, 0}, {3, 1, 0, 0}, {2, 0, 0, 5}, {3, 1, 0,
    ↪ 0},
                             {0, 1, 2, 0}, {3, 2, 0, 0}, {1, 0, 1, 0}, {3, 0, 0, 0}};
    assert(offlineCSum(a, op) == vector<int>({1, 3, 8, 11, 6}));
}
```

6.4.007 Euler Tour Tree

Euler Tour Tree 维护一片动态森林：每棵树存成一条欧拉序，换根就是把序列循环移位，连边与删边都只是序列的拼接与切分。序列用带父指针的隐式 treap 维护：

- `reroot(u)`：把 u 所在序列从 u 的点结点处切开、前后对调，u 成为序列开头；
- `link(u,v)`：分别换根后按 $E_u, (u \to v), E_v, (v \to u)$ 拼接；
- `cut(u,v)`：序列形如 $A\,(u \to v)\,B\,(v \to u)\,C$（两个边结点也可能顺序相反），$B$ 是被切下的子树，A 与 C 拼回剩下的树。

点结点带权、边结点权恒为 0，结点维护子树点数、点权和与整棵子树加的懒标记，因此可以 $O(\log n)$ 查询或修改整个连通块。父指针让任一结点能找到所在序列的根和自己的下标。

```
struct EulerTour
{
    // 每棵树存成一条欧拉序，序列用带父指针的隐式 treap 维护；节点 0 为空，1..n 是点节点，
    // 之后每条树边对应两个边节点 (u->v 与 v->u)。点节点带权，边节点权恒为 0。
    // l、r、p 是左右儿子与父亲；sz 是子树节点数，cnt 是子树内点节点数。
    vector<int> l, r, p, sz, cnt;
    vector<char> own;                  // own[x] 表示 x 是点节点。
    vector<unsigned long long> pri;    // pri 是 treap 随机优先级。
    vector<__int128> val, sum, lz;     // val 是点权，sum 是子树权和，lz 是整棵子树待加的值。
    map<pair<int, int>, pair<int, int>> edge; // edge[{u,v}] (u<v) 是该边的两个边节点。
    mt19937_64 rng{712367};

```

```
    EulerTour(int n)
    {
        // n 是点数，点编号为 [0,n)；初始为 n 个孤立点，点权为 0。
        make(false);
        sz[0] = 0; // 节点 0 是空节点，大小为 0。
        for (int i = 0; i < n; i++)
        {
            make(true);
        }
    }

    int make(bool v)
    {
        // 新建一个节点，v 表示是否为点节点；返回节点编号。
        for (auto *a : {&l, &r, &p})
        {
            a->push_back(0);
        }
        sz.push_back(1);
        cnt.push_back(v);
        own.push_back(v);
        pri.push_back(rng());
        val.push_back(0);
        sum.push_back(0);
        lz.push_back(0);
        return l.size() - 1;
    }

    void apply(int x, __int128 d)
    {
        // 给 x 的子树内所有点节点加 d。
        if (x)
        {
            val[x] += own[x] * d;
            sum[x] += d * cnt[x];
            lz[x] += d;
        }
    }

    void push(int x)
    {
        // 把 x 的懒标记下传给儿子。
        if (lz[x])
        {
            apply(l[x], lz[x]);
            apply(r[x], lz[x]);
            lz[x] = 0;
        }
    }

    void pull(int x)
    {
```

```
        // 由儿子重算 x 的子树信息，并修正儿子的父指针。
        sz[x] = 1 + sz[l[x]] + sz[r[x]];
        cnt[x] = own[x] + cnt[l[x]] + cnt[r[x]];
        sum[x] = val[x] + sum[l[x]] + sum[r[x]];
        p[l[x]] = p[r[x]] = x;
        p[0] = 0;
    }

    int merge(int a, int b)
    {
        // 拼接序列 a、b，返回新根（其父指针为 0）。
        if (!a || !b)
        {
            p[a | b] = 0;
            return a | b;
        }
        int x;
        if (pri[a] > pri[b])
        {
            push(a);
            r[a] = merge(r[a], b);
            x = a;
        }
        else
        {
            push(b);
            l[b] = merge(a, l[b]);
            x = b;
        }
        pull(x);
        p[x] = 0;
        return x;
    }

    pair<int, int> split(int x, int k)
    {
        // 把序列 x 拆成前 k 个节点和其余部分，两个根的父指针都为 0。
        if (!x)
        {
            return {0, 0};
        }
        push(x);
        pair<int, int> res;
        if (sz[l[x]] >= k)
        {
            auto [a, b] = split(l[x], k);
            l[x] = b;
            res = {a, x};
        }
        else
        {
            auto [a, b] = split(r[x], k - sz[l[x]] - 1);
```

```
            r[x] = a;
            res = {x, b};
        }
        pull(x);
        p[res.first] = p[res.second] = 0;
        return res;
    }

    int root(int x) const
    {
        // 返回 x 所在序列的根。
        while (p[x])
        {
            x = p[x];
        }
        return x;
    }

    int index(int x) const
    {
        // 返回 x 在所在序列中的下标（从 0 开始）。
        int i = sz[l[x]];
        for (; p[x]; x = p[x])
        {
            if (x == r[p[x]])
            {
                i += sz[l[p[x]]] + 1;
            }
        }
        return i;
    }

    int reroot(int u)
    {
        // 把点 u 所在树的欧拉序旋转为以 u 开头，返回序列根。
        int x = u + 1;
        auto [a, b] = split(root(x), index(x));
        return merge(b, a);
    }

    bool connected(int u, int v) const
    {
        // 返回 u、v 是否在同一棵树中。
        return root(u + 1) == root(v + 1);
    }

    bool link(int u, int v)
    {
        // 连边 u-v；已连通时不操作并返回 false。
        if (connected(u, v))
        {
            return false;
```

```
        }
        int a = reroot(u), b = reroot(v), uv = make(false), vu = make(false);
        edge[minmax(u, v)] = {uv, vu};
        merge(merge(merge(a, uv), b), vu);
        return true;
    }

    bool cut(int u, int v)
    {
        // 删除树边 u-v；边不存在时返回 false。
        auto it = edge.find(minmax(u, v));
        if (it == edge.end())
        {
            return false;
        }
        auto [x, y] = it->second;
        int i = index(x), j = index(y);
        if (i > j)
        {
            swap(i, j);
        }
        // 序列形如 A [e1] B [e2] C：B 是被切下的子树，A+C 是剩下的树。
        auto [a, rest] = split(root(x), i);
        auto [e1, tail] = split(rest, 1);
        auto [b, last] = split(tail, j - i - 1);
        auto [e2, c] = split(last, 1);
        merge(a, c);
        edge.erase(it);
        return true;
    }

    __int128 compSum(int u) const
    {
        // 返回 u 所在树的点权和。
        return sum[root(u + 1)];
    }

    int compSize(int u) const
    {
        // 返回 u 所在树的点数。
        return cnt[root(u + 1)];
    }

    void compAdd(int u, int d)
    {
        // 给 u 所在树的所有点加 d。
        apply(root(u + 1), d);
    }

    void add(int u, int d)
    {
        // 给点 u 加 d：先自顶向下下传标记，再自底向上重算。
```

```
        int x = u + 1;
        vector<int> path;
        for (int y = x; y; y = p[y])
        {
            path.push_back(y);
        }
        for (int i = path.size() - 1; i >= 0; i--)
        {
            push(path[i]);
        }
        val[x] += d;
        for (int y : path)
        {
            pull(y);
        }
    }
};
```

`EulerTour t(n)` 初始为 n 个点权为 0 的孤立点。`link(u,v)` 在已连通时返回 `false`，`cut(u,v)` 在边不存在时返回 `false`；`connected`、`compSize`、`compSum` 查询连通性、块大小与块点权和，`compAdd(u,d)` 给 u 所在块全体加 d，`add(u,d)` 只给点 u 加。每次操作期望 $O(\log n)$，空间 $O(n+m)$，m 为历史连过的边数（删边不回收结点）；`edge` 用 `map` 按无序点对查边，所以不支持重边。

它只维护连通块整体信息，不支持路径聚合；需要路径信息时用 6.4.001 Link-Cut Tree，同时需要子树信息时用 6.4.002 带子树信息的 LCT。操作离线已知时，6.4.006 离线动态图连通块点权和 更简单。下面的操作序列核对整块加、删边后重新连边：

```
signed main()
{
    EulerTour t(5);
    for (int i = 0; i < 5; i++)
    {
        t.add(i, i + 1); // 点 i 的初始权值为 i+1。
    }
    t.link(0, 1), t.link(1, 2), t.link(3, 4);
    t.compAdd(1, 10); // 0、1、2 各加 10。
    assert(t.compSum(2) == 36 && t.compSize(0) == 3);
    t.cut(0, 1);
    t.link(2, 3);
    assert(t.compSum(4) == 34 && !t.connected(0, 4) && !t.link(1, 4));
}
```

6.5 树形 DP

6.5.001 树上背包

树上背包基础版：$f_u[j]$ 为 u 子树中恰选 j 个点的最大权值，合并一棵儿子子树时对左右所选个数做 max-plus 卷积，每点可不选也可选，所选点之间无连通或依赖约束。数组长度只开到当前有效上界（各边两侧有效状态数之积求和才是真实复杂度，这正是经典 “子树背包上界为 $O(nk)$” 的来源）。

```cpp
template <class Acc = long long>
vector<optional<Acc>> treePack(const vector<vector<int>> &g, const vector<Acc> &val, int root,
    ↪ int k)
{
    // 返回 f[j]= 全树恰选 j 点的最大权值；nullopt 表示不可达。
    int n = g.size();
    assert(n > 0 && (int)val.size() == n && 0 <= root && root < n && k >= 0); // 调试检查，可删
    auto dfs = [&](auto &&self, int u, int p) -> vector<optional<Acc>>
    {
        vector<optional<Acc>> f(min<int>(1, k) + 1);
        f[0] = 0;
        if (k)
        {
            f[1] = val[u];
        }
        for (int v : g[u])
        {
            if (v == p)
            {
                continue;
            }
            auto h = self(self, v, u);
            vector<optional<Acc>> nf(min<int>(k, (int)f.size() + (int)h.size() - 2) + 1);
            for (int i = 0; i < (int)f.size(); i++)
            {
                for (int j = 0; j < (int)h.size() && i + j <= k; j++)
                {
                    if (f[i] && h[j])
                    {
                        Acc v = *f[i] + *h[j];
                        if (!nf[i + j] || *nf[i + j] < v)
                        {
                            nf[i + j] = v;
                        }
                    }
                }
            }
            f.swap(nf);
        }
        return f;
    };
    return dfs(dfs, root, -1);
}
```

`treePack(g, val, root, k)` 要求非空树且 $k \geq 0$，默认返回 `optional<long long>` 数组，`nullopt` 表示该选择数不可达；必须保证可达状态的价值累加能装入 `Acc`。可能超过 64 位时使用 `treePack<__int128>`，并传入 `vector<__int128>` 点权。$O(nk^2)$ 保守时间、$O(nk)$ 空间；链形树的递归深度为 $O(n)$，实际可承受深度取决于评测环境的线程栈。例如链 `0-1-2`、点权 `{5, -2, 4}`、$k = 3$ 时四个状态依次为 `0 5 9 7`。有父子依赖时改用 6.5.004 树上依赖背包。

6.5.002 单调队列优化树上背包

每点挂一类 “有体积、价值、件数上限” 的物品，用按体积余数分组的单调队列把本点多重背包的枚举件数一维消掉，再与儿子卷积。固定余数 r 后，容量 $r, r+w, r+2w, \dots$ 上的转移是长度 $cnt+1$ 的滑动窗口最大值：把 $f[r+jw]-jv$ 入队即可线性处理一类物品。该优化只作用于本点物品，儿子之间仍是树上背包卷积。

```cpp
template <class Acc = long long>
vector<optional<Acc>> treePack(const vector<vector<int>> &g,
                               const vector<array<long long, 3>> &a,
                               int root,
                               int cap)
{
    // a[u] 是 {体积, 价值, 件数}；返回恰用各容量的最大价值，nullopt 表示不可达。
    int n = g.size();
    assert(n > 0 && (int)a.size() == n && 0 <= root && root < n && cap >= 0); // 调试检查，可删
    auto dfs = [&](auto &&self, int u, int p) -> vector<optional<Acc>>
    {
        int w = a[u][0];
        Acc v = (Acc)a[u][1];
        int cnt = a[u][2];
        assert(w > 0 && cnt >= 0); // 调试检查，可删
        vector<optional<Acc>> pre(cap + 1), f(cap + 1);
        pre[0] = (Acc)0;
        for (int r = 0; r < w && r <= cap; r++)
        {
            deque<int> q;
            for (int j = 0, x = r; x <= cap; j++, x += w)
            {
                while (!q.empty() && q.front() < j - cnt)
                {
                    q.pop_front();
                }
                if (pre[x])
                {
                    Acc cur = *pre[x] - (Acc)j * v;
                    while (!q.empty())
                    {
                        int y = r + q.back() * w;
                        Acc old = *pre[y] - (Acc)q.back() * v;
                        if (old > cur)
                        {
                            break;
                        }
                        q.pop_back();
                    }
                    q.push_back(j);
                }
                if (!q.empty())
                {
                    int y = r + q.front() * w;
```

```
                    f[x] = *pre[y] + (Acc)(j - q.front()) * v;
                }
            }
        }
        for (int x : g[u])
        {
            if (x == p)
            {
                continue;
            }
            auto h = self(self, x, u);
            auto nf = f;
            for (int i = 0; i <= cap; i++)
            {
                for (int j = 1; i + j <= cap; j++)
                {
                    if (f[i] && h[j])
                    {
                        Acc cur = *f[i] + *h[j];
                        if (!nf[i + j] || *nf[i + j] < cur)
                        {
                            nf[i + j] = cur;
                        }
                    }
                }
            }
            f.swap(nf);
        }
        return f;
    };
    return dfs(dfs, root, -1);
}
```

`treePack(g, a, root, cap)` 要求非空树、$cap \geq 0$、体积为正且件数非负，各点物品独立。默认价值乘法、队列比较、卷积和状态表均使用 `long long`，必须保证这些中间值可表示；风险题显式调用 `treePack<__int128>`。不可达状态为 `nullopt`。$O(nC^2)$ 保守时间、$O(nC)$ 空间；链形树的递归深度为 $O(n)$，实际可承受深度取决于评测环境的线程栈。例如单点、物品 `{2,3,3}`、容量 7 时容量 `0,2,4,6` 的值为 `0,3,6,9`，其余不可达。

6.5.003 长链剖分 DP 合并

这是长链剖分 DP 合并的样例——求每个子树中出现次数最多的相对深度（并列取较小深度）。以深度为下标的 DP 让长儿子的数组与父节点错位一位直接继承（`at[h] = at[u]+1` 共享同一段内存 `f`），其余轻儿子只在其有效深度范围内逐项合并；分别核算每个轻子树的扫描长度，其总和是 $O(n)$ 而非平方。指针 `ptr` 线性分配各长链的内存段。

```
vector<int> depthMode(const vector<vector<int>> &g, int root = 0)
{
    // 返回每个子树中出现次数最多的相对深度，并列取较小者。
```

```
    int n = g.size();
    if (!n)
    {
        return {};
    }
    assert(0 <= root && root < n); // 调试检查，可删
    vector<int> len(n, 1), son(n, -1), ans(n), at(n), f(n + 1);
    auto dfs1 = [&](auto &&self, int u, int p) -> void
    {
        for (int v : g[u])
        {
            if (v == p)
            {
                continue;
            }
            self(self, v, u);
            if (len[v] + 1 > len[u])
            {
                len[u] = len[v] + 1;
                son[u] = v;
            }
        }
    };
    dfs1(dfs1, root, -1);
    int ptr = len[root];
    auto dfs2 = [&](auto &&self, int u, int p) -> void
    {
        int h = son[u];
        if (h != -1)
        {
            at[h] = at[u] + 1;
            self(self, h, u);
            ans[u] = ans[h] + 1;
        }
        f[at[u]] = 1;
        if (f[at[u] + ans[u]] <= 1)
        {
            ans[u] = 0;
        }
        for (int v : g[u])
        {
            if (v == p || v == h)
            {
                continue;
            }
            at[v] = ptr;
            ptr += len[v];
            self(self, v, u);
            for (int d = 0; d < len[v]; d++)
            {
                f[at[u] + d + 1] += f[at[v] + d];
                int x = f[at[u] + d + 1], y = f[at[u] + ans[u]];
```

```
                if (x > y || (x == y && d + 1 < ans[u]))
                {
                    ans[u] = d + 1;
                }
            }
        }
    };
    dfs2(dfs2, root, -1);
    assert(ptr == n); // 调试检查，可删
    return ans;
}
```

`depthMode(g, root)` 返回 `ans[u]` 为 u 子树中众数相对深度（并列取较小）。两遍 DFS 加长链内存复用，总 $O(n)$ 时间与空间——这是长链剖分把 “深度平移” 型 DP 从 $O(n^2)$ 降到线性的典型。前置的长链划分思想见 6.2.005 长链剖分；非深度平移型的子树合并回到 6.5.001 树上背包。

下面用分叉树、链、非零根与空树检查固定结果：

```
signed main()
{
    auto makeTree = [](int n, initializer_list<pair<int, int>> e)
    {
        vector<vector<int>> g(n);
        for (auto [u, v] : e)
        {
            g[u].push_back(v);
            g[v].push_back(u);
        }
        return g;
    };
    auto g = makeTree(8, {{0, 1}, {0, 2}, {1, 3}, {1, 4}, {2, 5}, {5, 6}, {5, 7}});
    assert((depthMode(g) == vector<int>{2, 1, 2, 0, 0, 1, 0, 0}));
    g = makeTree(4, {{0, 1}, {1, 2}, {2, 3}});
    assert((depthMode(g) == vector<int>{0, 0, 0, 0}));
    g = makeTree(4, {{0, 1}, {0, 2}, {0, 3}});
    assert((depthMode(g, 1) == vector<int>{1, 2, 0, 0}));
    assert(depthMode({}).empty());
}
```

6.5.004 树上依赖背包

树上依赖背包：每件物品有父子依赖，选任一非根物品前必须先选其父物品；求容量上限内的最大价值。$f_u[c]$ 为满足 u 子树依赖且恰用容量 c 的最大价值；关键约束是 “非虚根节点在 $c=0$ 时禁止合并非空儿子状态”——没选父亲就不能选后代，正由此表达。用一个虚根 `rt` 把依赖森林统一成一棵树。

```
template <class Acc = long long>
vector<optional<Acc>> treeDepPack(const vector<int> &w,
                                  const vector<Acc> &val,
                                  const vector<int> &par,
                                  int cap)
```

```cpp
{
    // 返回恰用容量 c 且满足父子依赖的最大价值；nullopt 表示不可达。
    int n = w.size(), rt = n;
    assert((int)val.size() == n && (int)par.size() == n && cap >= 0); // 调试检查，可删
    vector<vector<int>> son(n + 1);
    for (int u = 0; u < n; u++)
    {
        assert(w[u] > 0 && (-1 <= par[u] && par[u] < n)); // 调试检查，可删
        son[par[u] < 0 ? rt : par[u]].push_back(u);
    }
    int seen = 0;
    auto dfs = [&](auto &&self, int u) -> vector<optional<Acc>>
    {
        seen++;
        vector<optional<Acc>> f(cap + 1);
        if (u == rt)
        {
            f[0] = (Acc)0;
        }
        else if (w[u] <= cap)
        {
            f[w[u]] = val[u];
        }
        for (int v : son[u])
        {
            auto h = self(self, v);
            auto nf = f;
            for (int i = 0; i <= cap; i++)
            {
                if (!f[i] || (u != rt && i == 0))
                {
                    continue;
                }
                for (int j = 1; i + j <= cap; j++)
                {
                    if (h[j])
                    {
                        Acc cur = *f[i] + *h[j];
                        if (!nf[i + j] || *nf[i + j] < cur)
                        {
                            nf[i + j] = cur;
                        }
                    }
                }
            }
            f.swap(nf);
        }
        return f;
    };
    auto ans = dfs(dfs, rt);
    assert(seen == n + 1); // 调试检查，可删：par 必须构成森林
    return ans;
```

```
}
```

`treeDepPack(w, val, par, cap)` 要求 $cap \ge 0$ 且 `par` 构成森林，根记 -1；默认返回 `optional<long long>` 数组，必须保证所有价值和能装入 `Acc`。需要宽状态时调用 `treeDepPack<__int128>` 并传入 `vector<__int128>` 价值。$O(nC^2)$ 保守时间、$O(nC)$ 空间；链形依赖的递归深度为 $O(n)$，实际可承受深度取决于评测环境的线程栈。例如体积 `{2,2,3}`、价值 `{3,5,8}`、父亲 `{-1,0,0}`、容量 5 时最大值为 `11`（选点 0 与 2）。

6.5.005 有根树最小逆序拓扑序

根必须先于所有后代；对合法拓扑序 p，最小化 $\sum_i \sum_{j<i} c_{p_i} d_{p_j}$。把一段连续块记作 $(D, C) = (\sum d, \sum c)$，块 X 接在 Y 前产生的跨块代价是 $D_X C_Y$，交换论证给出最优顺序按 C/D 非增排列。树约束下，每次取当前比率最大的非根块并紧接到其父块后，再把两块收缩；DSU 找当前父块，循环链表用一次 `swap` 完成拼接。块和使用 `__int128` 累加，避免多个 `long long` 点值相加时溢出；交叉乘积和题目要求的答案也必须能由 `__int128` 表示。

```
vector<int> minInvOrder(const vector<int> &par, vector<long long> c,
                                vector<long long> d, int root = 0)
{
    // par 是父亲数组（根为 -1），c、d 是非负点值，root 是根；
    // 返回祖先先于后代、使 sum_i sum_{j<i} c[p_i] d[p_j] 最小的一个排列。
    int n = par.size();
    assert((int)c.size() == n && (int)d.size() == n && par[root] == -1); // 调试检查，可删。

    struct DSU
    {
        vector<int> p, id; // p 是并查集父亲（根为负的集合大小），id[r] 是集合 r 当前对应的块代表。

        DSU(int n) : p(n, -1), id(n)
        {
            // n 是点数；每个点自成一个集合，代表为自身。
            iota(id.begin(), id.end(), 0);
        }

        int find(int x)
        {
            // x 是点；返回所在集合的根，带路径压缩。
            return p[x] < 0 ? x : p[x] = find(p[x]);
        }

        int operator[](int x)
        {
            // x 是点；返回所在集合当前对应的块代表。
            return id[find(x)];
        }

        void merge(int x, int y, int z)
        {
            // x、y 是两点；合并其集合并把代表设为 z。
```

```
            x = find(x);
            y = find(y);
            if (x == y)
            {
                id[x] = z;
                return;
            }
            if (-p[x] < -p[y])
            {
                swap(x, y);
            }
            p[x] += p[y];
            p[y] = x;
            id[x] = z;
        }
    };

    struct Block
    {
        __int128 d, c; // d、c 是块内 d 值之和与 c 值之和。
        int root, ver; // root 是块代表点，ver 是入堆时的版本号，过期版本出堆时跳过。

        static bool ratioLess(const Block &x, const Block &y)
        {
            // x、y 是块；返回 c/d 比值是否 x<y，全零块视为最小。
            if (y.c == 0 && y.d == 0)
            {
                return false;
            }
            if (x.c == 0 && x.d == 0)
            {
                return true;
            }
            return (__int128)x.c * y.d < (__int128)x.d * y.c;
        }

        bool operator<(const Block &o) const
        {
            // o 是另一块；先比 c/d 比值，再比 (root,ver)，使大根堆先取比值最大的块。
            if (ratioLess(*this, o))
            {
                return true;
            }
            if (ratioLess(o, *this))
            {
                return false;
            }
            return pair(root, ver) < pair(o.root, o.ver);
        }
    };

    DSU dsu(n);
```

```
    priority_queue<Block> q;
    vector<int> nxt(n), ver(n);
    vector<__int128> sumC(n), sumD(n);
    iota(nxt.begin(), nxt.end(), 0);
    for (int u = 0; u < n; u++)
    {
        sumC[u] = c[u];
        sumD[u] = d[u];
        if (u != root)
        {
            assert(0 <= par[u] && par[u] < n); // 调试检查，可删。
            q.push({sumD[u], sumC[u], u, 0});
        }
    }
    while (!q.empty())
    {
        auto x = q.top();
        q.pop();
        int u = x.root;
        if (ver[u] != x.ver)
        {
            continue;
        }
        int v = dsu[par[u]];
        sumD[v] += sumD[u];
        sumC[v] += sumC[u];
        dsu.merge(u, v, v);
        if (v != root)
        {
            q.push({sumD[v], sumC[v], v, ++ver[v]});
        }
        swap(nxt[u], nxt[v]);
    }

    vector<int> ans{nxt[root]};
    while (ans.back() != root)
    {
        ans.push_back(nxt[ans.back()]);
    }
    reverse(ans.begin(), ans.end());
    return ans;
}
```

`minInvOrder(par,c,d,root)` 要求 `par[root]=-1`、其余父亲构成一棵以 `root` 为根的树，且 $c_i, d_i \geq 0$；返回一个最优排列，最小值按题意扫描排列即可。块和、任意两个块的交叉乘积以及题目要求的代价都必须能由 `__int128` 表示；输入点值仍可使用 `long long`，函数会在合并前提升为宽类型。堆中每个有效块只合并一次，时间 $O(n \log n)$、空间 $O(n)$。

下面用一棵三点树检查祖先必须在前、以及两个兄弟的最优交换顺序：

```cpp
__int128 orderValue(const vector<int> &p, const vector<long long> &c,
                    const vector<long long> &d)
{
    // p 是排列，c、d 是点值；返回 sum_i sum_{j<i} c[p_i] d[p_j]。
    __int128 ans = 0, sum = 0;
    for (int u : p)
    {
        ans += sum * c[u];
        sum += d[u];
    }
    return ans;
}

signed main()
{
    vector<int> par{-1, 0, 0};
    vector<long long> c{0, 4, 1}, d{1, 1, 5};
    auto got = minInvOrder(par, c, d);
    assert(got == vector<int>({0, 1, 2}));
    assert(orderValue(got, c, d) == 6);
}
```

6.5.006 换根 DP / Rerooting

固定根树形 DP 只得到一个根的答案；若每个点都要作为根，逐根重算会退化到 $O(n^2)$。设 `merge` 是带单位元 `identity` 的结合运算，`base[u]` 是点 u 自身贡献，`lift(value,from,to,edgeId)` 把 `from` 一侧的聚合跨边转成对 `to` 的贡献。第一遍 DFS 自底向上求 `down[u]`；第二遍对每个点按邻接表顺序建立贡献的前缀积与后缀积，给儿子时合并除该儿子外的全部方向，再跨父子边传下去。这样每条有向边只转移常数次，不要求 `merge` 可逆；若 `merge` 不可交换，结果按给定邻接表顺序定义。

```cpp
// RerootEdge[0] 是邻点，RerootEdge[1] 是无向边编号。
using RerootEdge = array<int, 2>;

template <class T, class Merge, class Lift>
vector<T> rerooting(
    const vector<vector<RerootEdge>> &g, const vector<T> &base, const T &identity, Merge
    ↪ merge, Lift lift, int root = 0)
{
    // g 是带边号无向树，base 是各点自身贡献，identity 是 merge 单位元；merge 合并贡献，lift 令贡献跨
    ↪ 边。
    int n = g.size();
    assert(n > 0 && (int)base.size() == n && 0 <= root && root < n); // 调试检查，可删
    vector<T> down(n, identity), fromPar(n, identity), ans(n, identity);

    auto dfsDown = [&](auto &&self, int u, int p) -> void
    {
        down[u] = base[u];
        for (auto [v, edgeId] : g[u])
        {
```

```
            if (v == p)
            {
                continue;
            }
            self(self, v, u);
            down[u] = merge(down[u], lift(down[v], v, u, edgeId));
        }
    };
    dfsDown(dfsDown, root, -1);

    auto dfsUp = [&](auto &&self, int u, int p) -> void
    {
        int degree = g[u].size();
        vector<T> part(degree, identity), prefix(degree + 1, identity), suffix(degree + 1,
↪   identity);
        for (int i = 0; i < degree; i++)
        {
            auto [v, edgeId] = g[u][i];
            part[i] = v == p ? fromPar[u] : lift(down[v], v, u, edgeId);
            prefix[i + 1] = merge(prefix[i], part[i]);
        }
        for (int i = degree - 1; i >= 0; i--)
        {
            suffix[i] = merge(part[i], suffix[i + 1]);
        }
        ans[u] = merge(base[u], prefix[degree]);
        for (int i = 0; i < degree; i++)
        {
            auto [v, edgeId] = g[u][i];
            if (v == p)
            {
                continue;
            }
            T withoutV = merge(base[u], merge(prefix[i], suffix[i + 1]));
            fromPar[v] = lift(withoutV, u, v, edgeId);
            self(self, v, u);
        }
    };
    dfsUp(dfsUp, root, -1);
    return ans;
}
```

`g[u]` 的每项为 `{v,edgeId}`，每条无向边须在两端各出现一次且使用同一边号；`base` 与 `g` 等长。`merge` 必须满足结合律并以 `identity` 为双侧单位元，`lift` 的返回类型为 `T`，可按方向与边号读取外部边权。设一次回调为 $O(1)$，总时间与额外空间均为 $O(n)$，递归调用栈最坏 $O(n)$。只需要固定根子树状态时普通树形 DP 更短；动态修改点权后反复询问全树最优值属于 6.4.005 动态 DP，不是本条的静态全根问题。

```
signed main()
{
    vector<vector<RerootEdge>> g(4);
    vector<int> w{2, 3, 1};
```

```
    auto add = [&](int u, int v, int id)
    {
        g[u].push_back({v, id});
        g[v].push_back({u, id});
    };
    add(0, 1, 0);
    add(1, 2, 1);
    add(1, 3, 2);
    // State[0] 是点数，State[1] 是到当前根的距离和。
    using State = array<int, 2>;
    auto merge = [](State a, State b)
    {
        return State{a[0] + b[0], a[1] + b[1]};
    };
    auto lift = [&](State a, int from, int to, int id)
    {
        (void)from;
        (void)to;
        return State{a[0], a[1] + a[0] * w[id]};
    };
    auto ans = rerooting(g, vector<State>(4, {1, 0}), State{0, 0}, merge, lift);
    assert((vector<int>{ans[0][1], ans[1][1], ans[2][1], ans[3][1]}
            == vector<int>{10, 6, 12, 8}));
}
```

7. 数论

7.1 整数运算与模运算

7.1.001 快速幂

指数二进制分解：底数每轮平方，遇到二进制置位就把它乘进答案，$O(\log(b+1))$ 次乘法。只用到乘法结合律和单位元，所以矩阵快速幂、幺半群快速幂都是同一套框架，换掉乘法即可。

`mulMod` 在模数不超过 `3037000499` 时直接用 64 位乘法，接近 2^{63} 的模数才对这一条乘法局部使用 `__int128`，因此任意 $m < 2^{63}$ 都不溢出；`powerMod` 的 `r` 初值取 $1 \bmod m$ 是为兼容 $m = 1$（此时答案恒为 0）。`mulMod` 是本章各处模乘的公共底座，防溢出细节见 7.1.011 Long Long 防溢出乘法。

```
long long mulMod(long long a, long long b, long long m)
{
    // a、b 是乘数，m 是正模数；返回 a*b mod m 的最小非负余数。
    assert(m > 0); // 调试检查，可删
    a %= m;
    b %= m;
    if (a < 0)
    {
        a += m;
    }
    if (b < 0)
    {
        b += m;
    }
    constexpr long long limit = 3037000499LL;
    if (m <= limit)
    {
        return a * b % m;
    }
    return (__int128)a * b % m;
}

long long powerMod(long long a, unsigned long long b, long long m)
{
    // a 是可为负的底数，b 是非负指数，m 是正模数；返回 a^b mod m。
    assert(m > 0); // 调试检查，可删
    long long r = 1 % m;
    a %= m;
    if (a < 0)
    {
        a += m;
    }
    while (b)
    {
        if (b & 1)
        {
            r = mulMod(r, a, m);
```

```
        }
        a = mulMod(a, a, m);
        b >>= 1;
    }
    return r;
}
```

复杂度 $O(\log(b+1))$ 次模乘。矩阵幂见线性代数章，模数为素数时求逆即 a^{m-2}，见 7.1.006 模逆元：快速幂、exgcd、线性与批量。

7.1.002 GCD、快速 GCD、LCM

标准 `gcd(a, b)`（$O(\log \max(|a|,|b|))$）一般够用，但参数含最小负数时其幅值无法用 `long long` 表示，应改用 `binaryGcd`。二进制 GCD 只用减法和移位、避开取模，取模昂贵时略快。求最小公倍数一律先除后乘 $\operatorname{lcm}(a,b) = a/\gcd(a,b)\cdot b$ 以压小中间值；`safeLcm` 再用 `unsigned __int128` 验算结果是否超出无符号 64 位，溢出返回空。约定 $\gcd(0,0)=0$。

```
unsigned long long magnitude(long long x)
{
    // 返回 x 的无符号幅值，最小负数也可表示
    auto u = (unsigned long long)x;
    return x < 0 ? 0ULL - u : u;
}

unsigned long long binaryGcd(long long a, long long b)
{
    // 返回 gcd(|a|,|b|)，约定 gcd(0,0)=0
    auto x = magnitude(a);
    auto y = magnitude(b);
    if (x == 0)
    {
        return y;
    }
    if (y == 0)
    {
        return x;
    }
    int z = countr_zero(x | y);
    x >>= countr_zero(x);
    do
    {
        y >>= countr_zero(y);
        if (x > y)
        {
            swap(x, y);
        }
        y -= x;
    } while (y ≠ 0);
    return x << z;
}
```

```

optional<unsigned long long> safeLcm(long long a, long long b)
{
    // 返回非负 lcm(|a|,|b|)，超过 64 位无符号范围时返回空
    if (a == 0 || b == 0)
    {
        return 0;
    }
    auto x = magnitude(a);
    auto y = magnitude(b);
    auto g = gcd(x, y);
    unsigned __int128 v = (unsigned __int128)(x / g) * y;
    if (v > numeric_limits<unsigned long long>::max())
    {
        return nullopt;
    }
    return (unsigned long long)v;
}
```

`binaryGcd` 与 `safeLcm` 返回无符号幅值，因而能表示 $gcd(-2^{63},0)=2^{63}$；`safeLcm` 返回空表示结果超过 $2^{64}-1$。二者均为 $O(\log\max(|a|,|b|))$ 时间。

7.1.003 扩展欧几里得

欧几里得迭代 $\gcd(a,b)=\gcd(b,a \bmod b)$,同时维护余数对原数的线性表示,结束时即得到一组 Bézout 系数 x,y，满足 $ax+by=\gcd(a,b)$。两个直接应用：任意互素模数下的逆元 `invMod`（$\gcd(a,m)\neq 1$ 时无逆元，返回空），以及线性丢番图方程 $ax+by=c$（有解当且仅当 $\gcd(a,b)\mid c$）。`exgcd` 返回 `[gcd, x, y]`，gcd 取非负。`invMod`、`exgcd` 是后续同余方程、CRT 的公共底座。

```
using Exgcd = array<__int128, 3>; // [0]=gcd(a,b)，[1]=x，[2]=y，后二者是一组 Bézout 系数

Exgcd exgcd(long long a, long long b)
{
    // a、b 是原方程系数；返回 {gcd(a,b),x,y}，满足 a*x+b*y=gcd(a,b)。
    __int128 r0 = a;
    __int128 r1 = b;
    __int128 x0 = 1;
    __int128 x1 = 0;
    __int128 y0 = 0;
    __int128 y1 = 1;
    while (r1)
    {
        __int128 q = r0 / r1;
        __int128 nr = r0 - q * r1;
        __int128 nx = x0 - q * x1;
        __int128 ny = y0 - q * y1;
        r0 = r1;
        r1 = nr;
        x0 = x1;
        x1 = nx;
```

```
        y0 = y1;
        y1 = ny;
    }
    if (r0 < 0)
    {
        r0 = -r0;
        x0 = -x0;
        y0 = -y0;
    }
    return {r0, x0, y0};
}

optional<long long> invMod(long long a, long long m)
{
    // a 是待求逆整数，m 是大于 1 的模数；返回最小非负逆元，不可逆时返回空。
    assert(m > 1); // 调试检查，可删
    auto [g, x, y] = exgcd(a, m);
    if (g != 1)
    {
        return nullopt;
    }
    x %= m;
    if (x < 0)
    {
        x += m;
    }
    return (long long)x;
}
```

复杂度 $O(\log \max(|a|,|b|))$，迭代实现只用 $O(1)$ 额外空间。`exgcd` 用 `__int128` 保存非负 gcd 与系数，能覆盖两个完整 `long long` 输入；`invMod` 要求 $m>1$。合数模、非互素情形的方程求解见 7.1.004 线性同余方程与同余方程组。

7.1.004 线性同余方程与同余方程组

单个方程 $ax \equiv b \pmod m$ 有解当且仅当 $\gcd(a,m) \mid b$。此时约去公因子 $g=\gcd(a,m)$，得到唯一剩余类 $x \equiv (b/g)\,(a/g)^{-1} \pmod{m/g}$，周期为 m/g。`congruence` 返回这一对 `[余数, 周期]`，无解返回空；多个约束的合并交给 7.1.005 CRT / exCRT。依赖 7.1.002 GCD、快速 GCD、LCM 的 `binaryGcd`、7.1.003 扩展欧几里得 的 `invMod` 与 7.1.001 快速幂 的 `mulMod`。

```
// 依赖 7.1.001 快速幂
// 依赖 7.1.003 扩展欧几里得
using Cong = array<int, 2>;   // [0]= 最小非负余数, [1]= 正周期（模）

optional<Cong> congruence(long long a, long long b, long long m)
{
    // 解 ax≡b (mod m)，返回 {最小非负解, 周期}；无解返回空。
    assert(m > 0); // 调试检查，可删
    long long g = (long long)binaryGcd(a, m);
```

```
    if (b % g)
    {
        return nullopt;
    }
    long long q = m / g;
    if (q == 1)
    {
        return Cong{0, 1};
    }
    long long r = mulMod(b / g, *invMod(a / g, q), q);
    return Cong{r, q};
}
```

复杂度 $O(\log m)$。

7.1.005 CRT / exCRT

合并两个同余式 $x \equiv a_0 \pmod{a_1}$、$x \equiv b_0 \pmod{b_1}$：令 $g = \gcd(a_1, b_1)$，差 $b_0 - a_0$ 能被 g 整除时有解，否则无解。用 `exgcd` 解出步长 k，新解 $x = a_0 + a_1 k$，新模为 $\operatorname{lcm}(a_1, b_1)$。逐个 `crt` 折叠即得任意多式的 exCRT；模数两两互素时退化为经典 CRT。`crt` 在方程无解或新模超过 `long long` 正数范围时均返回空。依赖 7.1.003 扩展欧几里得 的 `exgcd`。

```
// 依赖 7.1.004 线性同余方程与同余方程组
optional<Cong> crt(Cong a, Cong b)
{
    // 合并 x≡a[0] (mod a[1]) 与 x≡b[0] (mod b[1]); 返回 {最小非负解,lcm 模}, 无解或新模越界时返回空。
    assert(a[1] > 0 && b[1] > 0); // 调试检查，可删
    auto [g, x, y] = exgcd(a[1], b[1]);
    __int128 d = (__int128)b[0] - a[0];
    if (d % g)
    {
        return nullopt;
    }
    long long q = (long long)((__int128)b[1] / g);
    __int128 k = 0;
    if (q > 1)
    {
        k = d / g * x % q;
        if (k < 0)
        {
            k += q;
        }
    }
    __int128 m = (__int128)a[1] / g * b[1];
    if (m > numeric_limits<long long>::max())
    {
        return nullopt;
    }
    __int128 r = ((__int128)a[1] * k + a[0]) % m;
    if (r < 0)
```

```
    {
        r += m;
    }
    return Cong{(long long)r, (long long)m};
}
```

折叠 n 个方程共 $O(n \log V)$，V 为模数上界。若已知模数两两互素，也可直接乘出总模再逐项配逆元，常数更小但溢出风险更高，一般仍用上面的两两合并。

7.1.006 模逆元：快速幂、exgcd、线性与批量

素数模 p 下由费马小定理 $a^{-1} \equiv a^{p-2} \pmod p$，`invPrime` 用快速幂求单个逆元。批量求 $1..n$ 的逆元用经典线性递推：由 $p = \lfloor p/i \rfloor\, i + (p \bmod i)$ 两边模 p 整理得 $i^{-1} \equiv -\lfloor p/i \rfloor\, (p \bmod i)^{-1} \pmod p$，一趟 $O(n)$ 填满 `invTable`。任意给定数组则用 `batchInv`：先算前缀积，只求一次总积逆元，再逆序撤掉当前元素。三者都要求模数为素数、被求逆元素非零；任意互素合数模的单个逆元见 7.1.003 扩展欧几里得 的 `invMod`。

```
// 依赖 7.1.001 快速幂
int invPrime(int a, int mod)
{
    // 返回非零剩余 a 在素数模 mod 下的逆元
    assert(0 < a && a < mod); // 调试检查，可删
    int ans = 1;
    int b = mod - 2;
    while (b)
    {
        if (b & 1)
        {
            ans = mulMod(ans, a, mod);
        }
        a = mulMod(a, a, mod);
        b >>= 1;
    }
    return ans;
}

vector<int> invTable(int n, int mod)
{
    // 返回 1..n 在素数模 mod 下的逆元表，位置 0 保持 0
    assert(0 <= n && n < mod); // 调试检查，可删
    vector<int> inv(n + 1);
    if (n >= 1)
    {
        inv[1] = 1;
    }
    for (int i = 2; i <= n; i++)
    {
        inv[i] = mod - mulMod(mod / i, inv[mod % i], mod);
    }
```

```
    return inv;
}

vector<int> batchInv(const vector<int> &a, int mod)
{
    // 返回 a 中每个非零剩余在素数模 mod 下的逆元
    int n = a.size();
    vector<int> pre(n + 1, 1);
    for (int i = 0; i < n; i++)
    {
        assert(0 < a[i] && a[i] < mod); // 调试检查，可删
        pre[i + 1] = mulMod(pre[i], a[i], mod);
    }
    int cur = invPrime(pre[n], mod);
    vector<int> ans(n);
    for (int i = n - 1; i >= 0; i--)
    {
        ans[i] = mulMod(cur, pre[i], mod);
        cur = mulMod(cur, a[i], mod);
    }
    return ans;
}
```

单个 `invPrime` 为 $O(\log p)$；`invTable` 为 $O(n)$；`batchInv` 为 $O(n+\log p)$，预处理后读取任一数组元素的逆元才是 $O(1)$。两种批量接口均用 $O(n)$ 空间。固定模 `998244353` 且有大量在线逆元询问时，可用本条 HPP 附带的 `InvTable<998244353>::get(x)`：预处理 $O(2^{22})$、约 20 MiB 常驻表，之后单次 $O(1)$；它是专项接口，不替代通用 `invMod`、`invPrime` 与批量逆元。

7.1.007 静态取模类

静态取模类是跨章节的基础值类型，完整实现、常见模数的加减乘除验证与复杂度统一放在 1.1.008 静态取模类。本编号只保留旧目录和外部引用的兼容入口，不重复维护第二份实现。需要运行期模数时使用 7.1.008 动态取模类，需要接近 64 位上界的运行时模数时使用 7.1.001 快速幂 的 `mulMod`，不能把固定取模类硬套到不同语义上。

7.1.008 动态取模类

模数在运行期才确定、或一道题要分阶段换模数时用 `DynMod`。模数是全类共享的静态状态，`setMod` 一改，所有 `DynMod` 对象的含义随之改变——因此不同模数的对象不能同时存活、混算。核心是 `Barrett` 约减：预存 $\lceil 2^{64}/m \rceil$ 级别的定点倒数，把 $z \bmod m$ 换成一次乘法估商加一步修正，避开慢速硬件除法（推导见 7.1.009 Barrett 约减）。要求 $m < 2^{31}$，从而 $a, b < 2^{31}$、乘积 $< 2^{62}$ 落在 `unsigned long long` 内。

```
struct Barrett
{
    unsigned m;             // 当前约减使用的正模数
    unsigned long long im;  // m>1 时为 ceil(2^64/m)，作估商用的定点倒数

```

```
    Barrett(unsigned m = 1) : m(m), im(0)
    {
        // m 为正模数，构造时预计算定点倒数 im。
        assert(m > 0); // 调试检查，可删
        im = ~0ULL / m + 1;
    }

    unsigned mul(unsigned a, unsigned b) const
    {
        // 返回 a*b mod m，要求 a、b 均在 [0,m) 内。
        if (m == 1)
        {
            return 0;
        }
        unsigned long long z = 1ULL * a * b;
        unsigned long long x = (unsigned __int128)z * im >> 64;
        unsigned v = z - x * m;
        if (m <= v)   // 估商偏大 1 时 v 因无符号回绕呈现为大值，补一个 m 收回 [0,m)
        {
            v += m;
        }
        return v;
    }
};

struct DynMod
{
    inline static Barrett bt;  // 全类共享的约减器，由 setMod 设定当前模数
    unsigned x = 0;            // 当前模数下的最小非负代表

    static void setMod(unsigned m)
    {
        // 把后续所有 DynMod 对象的模数设为 m
        assert(m > 0 && m < (1U << 31)); // 调试检查，可删
        bt = Barrett(m);
    }
    static unsigned mod()
    {
        // 无参数；返回当前设定的模数。
        return bt.m;
    }

    DynMod(long long v = 0)
    {
        // v 规范到 [0,mod) 建立剩余类。
        v %= mod();
        if (v < 0)
        {
            v += mod();
        }
        x = v;
    }
```

```
58
59      DynMod &operator+=(DynMod b)
60      {
61          // b 是同模剩余类；原地完成模加并返回当前对象引用。
62          x += b.x;
63          if (x >= mod())
64          {
65              x -= mod();
66          }
67          return *this;
68      }
69      DynMod &operator-=(DynMod b)
70      {
71          // b 是同模剩余类；原地完成模减并返回当前对象引用。
72          x += mod() - b.x;
73          if (x >= mod())
74          {
75              x -= mod();
76          }
77          return *this;
78      }
79      DynMod &operator*=(DynMod b)
80      {
81          // b 是同模剩余类；原地完成模乘并返回当前对象引用。
82          x = bt.mul(x, b.x);
83          return *this;
84      }
85      friend DynMod operator+(DynMod a, DynMod b)
86      {
87          // a、b 是同模剩余类；返回 a+b。
88          return a += b;
89      }
90
91      friend DynMod operator-(DynMod a, DynMod b)
92      {
93          // a、b 是同模剩余类；返回 a-b。
94          return a -= b;
95      }
96
97      friend DynMod operator*(DynMod a, DynMod b)
98      {
99          // a、b 是同模剩余类；返回 a*b。
100         return a *= b;
101     }
102     bool operator==(const DynMod &b) const = default;
103
104     DynMod pow(unsigned long long n) const
105     {
106         // n 是非负指数；返回当前剩余类的 n 次幂。
107         DynMod a = *this;
108         DynMod r = 1;
109         while (n)
```

```
        {
            if (n & 1)
            {
                r *= a;
            }
            a *= a;
            n >>= 1;
        }
        return r;
    }
};
```

用法：`DynMod::setMod(m)` 后即可像普通整数一样加减乘与 `pow`。不提供除法（合数模一般不可逆）；素数模要逆元用 `pow(mod() - 2)`。加减乘 $O(1)$，`pow(n)` 为 $O(\log(n+1))$，对象 $O(1)$ 空间。

7.1.009 Barrett 约减

Barrett 约减用乘法代替除法算 $z \bmod m$。当 $m > 1$ 时取定点倒数 $\mathrm{im} = \lceil 2^{64}/m \rceil$，它不小于真倒数，于是估商 $\hat{q} = \lfloor z \cdot \mathrm{im}/2^{64} \rfloor$ 满足 $\hat{q} \in \{q,\, q+1\}$（$q = \lfloor z/m \rfloor$）。把 $v = z - \hat{q}\,m$ 截断到 32 位无符号：$\hat{q} = q$ 时 $v \in [0, m)$；$\hat{q} = q+1$ 时真值落在 $[-m, 0)$，按无符号回绕表现为一个 $\geq m$ 的大数。因此 `if (m <= v) v += m;` 用一次回绕加法把两种情形都收回 $[0, m)$；$m = 1$ 直接返回 0。要求 $m < 2^{31}$，乘积 $z = ab < 2^{62}$，中间量 $z \cdot \mathrm{im}$ 用 `unsigned __int128` 承载。

实现就是 7.1.008 动态取模类 里的 `struct Barrett`，此处不再重复打印。与 7.1.010 Montgomery 模乘 相比，Barrett 不要求模数为奇、无需进出域的编解码，代价是每次约减多一步条件修正。

7.1.010 Montgomery 模乘

Montgomery 把元素 a 编码成 $aR \bmod P$（本实现 $R = 2^{32}$），使模乘只用乘、加、移位而无需硬件除法。核心是 `reduce`：给定 $x < PR$，用 P 关于 R 的负逆 `r`（$Pr \equiv -1 \pmod R$）补齐低 32 位使 $x + ((x\,r) \bmod R)P$ 整除 R，右移 32 位即得 $xR^{-1} \bmod P$（落在 $[0, 2P)$，`val` 再规约一次）。于是一次模乘 = 一次普通乘 + 一次 `reduce`；构造时乘以 $r2 = R^2 \bmod P$ 把普通值编码进域。要求 P 为奇数（负逆 `r` 才存在）且 $P < 2^{30}$（保证 $2P$ 不溢出 32 位）。奇素数下大量模乘通常快于 7.1.009 Barrett 约减。

```
template <unsigned P> struct Mont
{
    unsigned x = 0;   // Montgomery 域表示，即 (真值 * 2^32) mod P，范围 [0,2P)

    static_assert(P > 0 && (P & 1));
    static_assert(P < (1U << 30));

    static constexpr unsigned inv()
    {
        // 返回 P 关于 2^32 的负逆 r：满足 P*r ≡ -1 (mod 2^32)，牛顿迭代 5 次收敛。
        unsigned x = P;
        for (int i = 0; i < 5; i++)
```

```
        {
            x *= 2 - P * x;
        }
        return -x;
    }

    static constexpr unsigned r = inv(); // r 是 -P^{-1} mod 2^32，用于 Montgomery 约减。
    static constexpr unsigned r2 = -(unsigned long long)P % P; // r2 是 2^64 mod P，用于转入
    ↪ Montgomery 表示。

    static constexpr unsigned reduce(unsigned long long x)
    {
        // 输入 x < 2^32 * P，返回 x * 2^-32 mod P（结果在 [0,2P) 内）。
        return (x + (unsigned long long)((unsigned)x * r) * P) >> 32;
    }

    Mont(long long v = 0)
    {
        // 把普通整数 v 编码进 Montgomery 域
        v %= P;
        if (v < 0)
        {
            v += P;
        }
        x = reduce((unsigned long long)v * r2);
    }

    unsigned val() const
    {
        // 无参数；返回当前剩余类的最小非负整数代表。
        unsigned v = reduce(x);
        return v >= P ? v - P : v;
    }

    Mont &operator+=(const Mont &o)
    {
        // o 是同模 Montgomery 数；原地完成域内加法并返回当前对象引用。
        x += o.x;
        if (x >= 2 * P)
        {
            x -= 2 * P;
        }
        return *this;
    }

    Mont &operator-=(const Mont &o)
    {
        // o 是同模 Montgomery 数；原地完成域内减法并返回当前对象引用。
        x -= o.x;
        if (x >= 2 * P)
        {
            x += 2 * P;
```

```
        }
        return *this;
    }

    Mont &operator*=(const Mont &o)
    {
        // o 是同模 Montgomery 数；原地完成域内乘法并返回当前对象引用。
        x = reduce((unsigned long long)x * o.x);
        return *this;
    }

    Mont pow(long long n) const
    {
        // n 是非负指数；返回当前元素的 n 次幂。
        assert(n >= 0); // 调试检查，可删
        Mont a = *this;
        Mont r = 1;
        while (n)
        {
            if (n & 1)
            {
                r *= a;
            }
            a *= a;
            n >>= 1;
        }
        return r;
    }

    Mont invMod() const
    {
        // 当前元素必须非零且 P 必须为素数；返回乘法逆元。
        assert(val() != 0); // 调试检查，可删
        return pow(P - 2);
    }

    Mont &operator/=(const Mont &o)
    {
        // o 是非零同模 Montgomery 数；原地除以 o 并返回当前对象引用。
        return *this *= o.invMod();
    }

    friend Mont operator+(Mont a, const Mont &b)
    {
        // a、b 是同模 Montgomery 数；返回 a+b。
        return a += b;
    }
    friend Mont operator-(Mont a, const Mont &b)
    {
        // a、b 是同模 Montgomery 数；返回 a-b。
        return a -= b;
    }
```

```
    friend Mont operator*(Mont a, const Mont &b)
    {
        // a、b 是同模 Montgomery 数；返回 a*b。
        return a *= b;
    }
    friend Mont operator/(Mont a, const Mont &b)
    {
        // a、b 是同模 Montgomery 数且 b 非零；返回 a/b。
        return a /= b;
    }
    friend bool operator==(const Mont &a, const Mont &b)
    {
        // a、b 是同模 Montgomery 数；返回二者是否代表同一剩余类。
        return a.val() == b.val();
    }
};
```

用 `Mont<P>` 要求 `P` 为奇数且 $P < 2^{30}$。加减乘除 $O(1)$，`pow`/`invMod` $O(\log P)$，对象 $O(1)$ 空间；除法与 `invMod` 走费马小定理，额外要求 `P` 为素数。

7.1.011 Long Long 防溢出乘法

计算 $a \cdot b \bmod m$ 而中间不溢出。若两个规范余数的乘积能放进 `long long`，直接乘再取模最快；否则把这一条乘法升到 `__int128`，即 7.1.001 快速幂 的 `mulMod`，$O(1)$ 且无精度问题。若编译器不支持 `__int128`，退路有二：龟速乘（把 b 二进制拆开、逐位倍增累加，$O(\log(b+1))$）；或浮点估商法 $ab - \lfloor ab/m \rfloor m$，用 `long double` 估商后校正余数，$O(1)$ 但需留意精度。竞赛环境几乎总有 `__int128`，完整 64 位模数直接用 `mulMod` 即可。

7.1.012 快速整除判定

判断一个可能超过机器字长、以字符串给出的十进制整数能否被 m 整除。做法是 Horner 逐位取模：从高位到低位维护 $r = (10r + d_i) \bmod m$，扫完若 $r = 0$ 即整除。支持前导正负号，m 取绝对值且须非零，$O(|s|)$ 一趟完成，不需高精度除法。

```
// 依赖 7.1.002 GCD、快速 GCD、LCM
namespace fastDiv
{
bool divisible(const string &s, int m)
{
    // s 是带可选正负号的十进制整数串，m 是非零除数；返回 s 是否能被 m 整除。
    assert(m != 0); // 调试检查，可删
    auto d = magnitude(m);
    unsigned long long r = 0;
    bool small = d <= 3037000499ULL;
    int p = !s.empty() && (s[0] == '+' || s[0] == '-');
    assert(p < (int)s.size()); // 调试检查，可删
    for (; p < (int)s.size(); p++)
    {
```

```
        assert(isdigit((unsigned char)s[p])); // 调试检查，可删
        unsigned digit = (unsigned)(s[p] - '0');
        if (small)
        {
            r = (r * 10 + digit) % d;
        }
        else
        {
            r = ((unsigned __int128)r * 10 + digit) % d;
        }
    }
    return r == 0;
}
}
```

复杂度 $O(|s|)$ 时间、$O(1)$ 空间；小除数的逐位 Horner 只用 64 位运算，除数绝对值超过 `3037000499` 时才对单步 `10r+d_i` 使用 `unsigned __int128`。

7.1.013 模意义真分数还原

已知某既约真分数 p/q 对模 m 的值 $x \equiv p\,q^{-1} \pmod m$，且有界 $|p| \le a$、$0 < q \le b$，要还原出 p, q。做法是对 (m, x) 跑扩展欧几里得（等价于 x/m 的连分数展开）并同步维护 t 系数：当余数 r_1 首次不超过分子界 a 时停下，(r_1, t_1) 即候选分子、分母。再校验 $t_1 \neq 0$、$|t_1| \le b$、$\gcd(r_1, t_1) = 1$ 且 $x\,t_1 \equiv r_1 \pmod m$，全通过才唯一还原，否则返回空；当 $2ab < m$ 时解必唯一。典型用途：模意义下做完一串有理运算后把结果读回真分数。

```
optional<pair<long long, long long>> ratRebuild(long long x, long long m, long long a, long
↪ long b)
{
    // x 是模 m 的剩余，a、b 是分子绝对值与分母上界；在 2ab<m 时返回唯一既约真分数。
    assert(m > 0 && a >= 0 && b > 0); // 调试检查，可删
    assert((__int128)2 * a * b < m); // 调试检查，可删
    x %= m;
    if (x < 0)
    {
        x += m;
    }
    __int128 r0 = m;
    __int128 r1 = x;
    __int128 t0 = 0;
    __int128 t1 = 1;
    auto mag = [](__int128 v)
    {
        return v < 0 ? -v : v;
    };
    while (mag(r1) > a)
    {
        if (!r1)
        {
```

```
            return nullopt;
        }
        __int128 q = r0 / r1;
        __int128 nr = r0 - q * r1;
        __int128 nt = t0 - q * t1;
        r0 = r1;
        r1 = nr;
        t0 = t1;
        t1 = nt;
    }
    if (!t1 || mag(t1) > b)
    {
        return nullopt;
    }
    if (t1 < 0)
    {
        r1 = -r1;
        t1 = -t1;
    }
    if (mag(r1) >= t1 || gcd((long long)mag(r1), (long long)t1) != 1)
    {
        return nullopt;
    }
    if (((__int128)x * t1 - r1) % m)
    {
        return nullopt;
    }
    return pair{(long long)r1, (long long)t1};
}
```

`ratRebuild(x, m, a, b)` 要求 $2ab < m$，该界保证满足条件的既约真分数至多一个；返回 `{分子, 分母}`（分母恒正），不存在时返回空。复杂度 $O(\log m)$ 时间、$O(1)$ 空间。

7.1.014 整数 k 次方根

求最大整数 x 使 $x^k \le n$，即 $\lfloor n^{1/k} \rfloor$。这里在 $[1,\ 2^{64/k+1}]$ 上二分，判定 `ok(x)` 用 `unsigned __int128` 边乘边比、一超过 n 立即返回假，从而避免 x^k 溢出。$k = 1$ 或 $n \le 1$ 直接返回 n，$k \ge 64$ 时答案必为 1。相比先浮点估根再修正，二分更短且无精度坑。

```
unsigned long long kthRoot(unsigned long long n, int k)
{
    // n 是被开方的非负整数，k 是正整数次数；返回 floor(n^(1/k))。
    assert(k > 0); // 调试检查，可删
    if (k == 1 || n <= 1)
    {
        return n;
    }
    if (k >= 64)
    {
        return 1;
```

```
    }
    auto ok = [&](unsigned long long x)
    {
        unsigned __int128 y = 1;
        for (int i = 0; i < k; i++)
        {
            y *= x;
            if (y > n)
            {
                return false;
            }
        }
        return true;
    };
    unsigned long long l = 1;
    unsigned long long r = min(n, 1ULL << (64 / k + 1));
    while (l < r)
    {
        unsigned long long m = l + (r - l + 1) / 2;
        if (ok(m))
        {
            l = m;
        }
        else
        {
            r = m - 1;
        }
    }
    return l;
}
```

`kthRoot(n, k)` 接收无符号 n、正整数 k，返回 $\lfloor n^{1/k} \rfloor$。复杂度 $O(k \log(n+1))$ 时间、$O(1)$ 空间。

7.1.015 线性同余不等式

求最小非负整数 x 使 $l \le dx \bmod m \le r$，常用于「带模线性函数首次落入某区间」的问题。核心是 Euclid 风格递归：$2d > m$ 时借 $dx \bmod m$ 与 $(m-d)x \bmod m$ 的对称把系数翻小；若当前步 $\lfloor (l-1)/d \rfloor < \lfloor r/d \rfloor$ 说明区间内已含 d 的倍数，直接取 $\lfloor (l-1)/d \rfloor + 1$；否则把问题递归到模 d。每层至少对较大参数取模一次，深度 $O(\log m)$。

```
int minModInt(int m, int d, int l, int r)
{
    // 参数满足 m>0 且 0<=d,l<=r<m；返回使 l<=d*x mod m<=r 的最小非负 x，无解返回 -1。
    assert(m > 0 && 0 <= d && d < m && 0 <= l && l <= r && r < m); // 调试检查，可删
    if (l == 0)
    {
        return 0;
    }
    if (d == 0)
    {
```

```
        return -1;
    }
    if (d > m - d)
    {
        return minModInt(m, m - d, m - r, m - l);
    }
    if ((l - 1) / d < r / d)
    {
        return (l - 1) / d + 1;
    }
    auto k = minModInt(d, (-m % d + d) % d, l % d, r % d);
    if (k == -1)
    {
        return -1;
    }
    return (int)(((__int128)k * m + l - 1) / d + 1);
}
```

`minModInt(m, d, l, r)` 要求 $m > 0$、$0 \le d < m$、$0 \le l \le r < m$，返回满足条件的最小非负 x，无解返回 `-1`。比较 $2d > m$ 改写为无溢出的 `d>m-d`，最终乘法在 `__int128` 中完成；复杂度 $O(\log m)$ 时间与递归栈。

7.1.016 基于值域优化的 GCD

当所有查询数都落在 $[1, n]$ 且需要海量 gcd 时，可 $O(n)$ 预处理后单次 $O(1)$ 作答。关键：把每个 $x \le n$ 分解成三个因子 $x = f_0 f_1 f_2$，每个因子要么 $\le \sqrt{n}$、要么本身为质数（线性筛时顺带维护）；再打出 $\le \sqrt{n}$ 两数的 gcd 小表 `tab`。查询 $\gcd(x, y)$ 时对 x 的三个因子依次与 y 求 gcd 并从 y 中除去：小因子查 `tab`，质因子只需判 y 能否被其整除，三步相乘即答案。依据是任意数都能写成三个「各自 $\le \sqrt{n}$ 或为质数」的因子之积。

```
struct ValueGcd
{
    int n;                      // 允许查询的最大值
    int b;                      // 小因子表边界 floor(sqrt(n))+1
    vector<char> prime;         // prime[x] 表示 x 是否为质数
    vector<array<int, 3>> fac; // x 的三个受限因子
    vector<vector<int>> tab;    // 小值两两 gcd 表

    ValueGcd(int n) : n(n), b(sqrtl(n) + 1), prime(n + 1, true), fac(n + 1), tab(b + 1,
    ↪ vector<int>(b + 1))
    {
        // 参数 n 是闭值域上界；预处理分解和小值 GCD 表，无返回值。
        prime[0] = false;
        if (n)
        {
            prime[1] = false;
        }
        for (int i = 0; i <= b; i++)
        {
```

```
            tab[i][0] = i;
            tab[0][i] = i;
        }
        for (int i = 1; i <= b; i++)
        {
            for (int j = i; j <= b; j++)
            {
                tab[i][j] = tab[i][j % i];
                tab[j][i] = tab[i][j];
            }
        }
        vector<int> ps;
        if (n >= 1)
        {
            fac[1] = {1, 1, 1};
        }
        for (int i = 2; i <= n; i++)
        {
            if (prime[i])
            {
                ps.push_back(i);
                fac[i] = {1, 1, i};
            }
            for (auto p : ps)
            {
                if (i > n / p)
                {
                    break;
                }
                prime[i * p] = false;
                fac[i * p] = fac[i];
                auto at = min_element(fac[i * p].begin(), fac[i * p].end());
                *at *= p;
                if (i % p == 0)
                {
                    break;
                }
            }
        }
    }

    int small(int x, int y) const
    {
        // 参数 x 是不超过 b 或为质数的因子，y 在 [0,n]；返回 gcd(x,y)。
        if (x == 1)
        {
            return 1;
        }
        auto t = y % x;
        if (prime[x])
        {
            return t ? 1 : x;
```

```
        }
        return tab[t][x];
    }

    int get(int x, int y) const
    {
        // 参数 x、y 均在 [1,n]；返回二者最大公约数，查询阶段为常数次查表。
        assert(1 <= x && x <= n && 1 <= y && y <= n); // 调试检查，可删
        auto g0 = small(fac[x][0], y);
        y /= g0;
        auto g1 = small(fac[x][1], y);
        y /= g1;
        auto g2 = small(fac[x][2], y);
        return g0 * g1 * g2;
    }
};
```

构造 `ValueGcd(n)` 预处理值域上界 n，再 `get(x, y)` 查询，要求 $1 \le x, y \le n$（`get` 内含断言，不接受 0）。预处理 $O(n + (\sqrt{n})^2) = O(n)$ 时间与空间，单次查询 $O(1)$。查询数超出值域时退化用标准 `gcd` 即可。海量小值 gcd 才值得上此结构，否则直接 `gcd` 更省事。

7.1.017 高斯整数与最大公约数

高斯整数环 $\mathbb{Z}[i]$ 的元素写作 $x + yi$，范数 $N(x + yi) = x^2 + y^2$ 满足乘法性。对非零 b，把复商 $(a\bar{b})/N(b)$ 的实、虚部分别舍入到最近整数得到商 q，则余数 $a - qb$ 的两个坐标误差都不超过 $1/2$，因而其范数严格小于 $N(b)$。这使普通欧几里得算法在 $\mathbb{Z}[i]$ 中同样终止；最大公约数只确定到乘以 $\pm 1, \pm i$ 的单位元，本接口不强行规定代表元。

```
struct GaussianInt
{
    int x, y; // x、y 分别是实部与虚部。

    GaussianInt(int x = 0, int y = 0) : x(x), y(y)
    {
        // x、y 是高斯整数 x+yi 的两个系数。
    }

    bool zero() const
    {
        // 无参数；返回当前高斯整数是否为 0。
        return x == 0 && y == 0;
    }

    GaussianInt conjugate() const
    {
        // 无参数；返回共轭 x-yi。
        return {x, -y};
    }

```

```
    GaussianInt &operator+=(GaussianInt b)
    {
        // b 是加数；原地加 b 并返回自身。
        x += b.x;
        y += b.y;
        return *this;
    }

    GaussianInt &operator-=(GaussianInt b)
    {
        // b 是减数；原地减 b 并返回自身。
        x -= b.x;
        y -= b.y;
        return *this;
    }

    GaussianInt &operator*=(GaussianInt b)
    {
        // b 是乘数；原地乘 b 并返回自身，题目范围保证结果可由 64 位表示。
        __int128 nx = (__int128)x * b.x - (__int128)y * b.y;
        __int128 ny = (__int128)x * b.y + (__int128)y * b.x;
        x = (int)nx;
        y = (int)ny;
        return *this;
    }

    friend GaussianInt operator+(GaussianInt a, GaussianInt b)
    {
        // a、b 是高斯整数；返回 a+b。
        return a += b;
    }

    friend GaussianInt operator-(GaussianInt a, GaussianInt b)
    {
        // a、b 是高斯整数；返回 a-b。
        return a -= b;
    }

    friend GaussianInt operator*(GaussianInt a, GaussianInt b)
    {
        // a、b 是高斯整数；返回 a*b。
        return a *= b;
    }

    static int nearestDiv(__int128 a, __int128 b)
    {
        // a 是被除数，b 是正除数；返回距 a/b 最近的整数，半整数向正无穷取整。
        assert(b > 0);
        return (int)(a >= 0 ? (a + b / 2) / b : -((-a + b / 2) / b));
    }

    GaussianInt div(GaussianInt b) const
```

```
    {
        // b 非零；返回使余数范数严格小于 b 范数的一组商。
        assert(!b.zero());
        __int128 d = (__int128)b.x * b.x + (__int128)b.y * b.y;
        return {nearestDiv((__int128)x * b.x + (__int128)y * b.y, d),
                nearestDiv((__int128)y * b.x - (__int128)x * b.y, d)};
    }

    GaussianInt mod(GaussianInt b) const
    {
        // b 非零；返回当前值除以 b 的欧几里得余数。
        return *this - div(b) * b;
    }

    static GaussianInt gcd(GaussianInt a, GaussianInt b)
    {
        // a、b 是高斯整数；返回任意一个高斯整数最大公约数。
        while (!b.zero())
        {
            GaussianInt r = a.mod(b);
            a = b;
            b = r;
        }
        return a;
    }
};
```

`GaussianInt::gcd(a,b)` 返回任意单位元等价的高斯整数 GCD；`div(b)` 的 `b` 必须非零，`mod(b)` 返回欧几里得余数。LC 的坐标绝对值不超过 10^9，中间乘积以 `__int128` 计算，最终输入输出在 `long long` 范围内；单次复杂度为欧几里得迭代次数，通常为 $O(\log \max(N(a), N(b)))$，额外空间 $O(1)$。普通整数只需 7.1.001 GCD 与扩展 GCD，不要为整数 GCD 引入高斯除法。

7.2 素数与因数分解

7.2.001 埃氏筛、线性筛

线性筛（欧拉筛）让每个合数只被它的最小质因子筛掉一次，$O(n)$ 求出 $[2, n]$ 的全部质数，并顺带填出最小质因子表 `minp`。有了 `minp`，任意 $x \le n$ 都能在 $O(\log x)$ 内反复除最小质因子完成分解。相比埃氏筛的 $O(n \log \log n)$，线性筛不重复标记、还免费带出 `minp`，本书作默认；只要素数列表、值域也不大时埃氏筛更短。

```
struct Sieve
{
    vector<int> minp;   // minp[x] 是 x 的最小质因子
    vector<int> primes; // 升序质数表

    Sieve(int n) : minp(n + 1)
    {
        // n 是非负筛表上界；线性筛 [2,n] 并填充 minp 与 primes。
```

```
        for (int i = 2; i <= n; i++)
        {
            if (!minp[i])
            {
                minp[i] = i;
                primes.push_back(i);
            }
            for (int p : primes)
            {
                if (p > minp[i] || i > n / p)
                {
                    break;
                }
                minp[i * p] = p;
            }
        }
    }

    vector<pair<int, int>> factor(int x) const
    {
        // x 在 [1,minp.size()) 内；返回按质因子升序的 {质因子, 指数}。
        assert(1 <= x && x < (int)minp.size()); // 调试检查，可删
        vector<pair<int, int>> ans;
        while (x > 1)
        {
            int p = minp[x];
            int c = 0;
            while (x % p == 0)
            {
                x /= p;
                c++;
            }
            ans.push_back({p, c});
        }
        return ans;
    }
};

vector<int> enumPrimes(int n)
{
    // n 是非负上界；仅标记奇数并返回 [2,n] 的全部质数。
    if (n < 2)
    {
        return {};
    }
    vector<bool> composite(n / 2 + 1);
    for (int p = 3; p <= n / p; p += 2)
    {
        if (composite[p / 2])
        {
            continue;
        }
```

```
        for (int x = p * p; x <= n; x += p * 2)
        {
            composite[x / 2] = true;
        }
    }
    vector<int> primes{2};
    for (int x = 3; x <= n; x += 2)
    {
        if (!composite[x / 2])
        {
            primes.push_back(x);
        }
    }
    return primes;
}
```

构造 `Sieve(n)` 后 `primes` 即质数表、`factor(x)` 分解 $x \le n$。筛 $O(n)$ 时间与空间，`factor` 为 $O(\log x)$。只需枚举大上界内质数时用 `enumPrimes(n)`：它只存奇数合数位，筛标记占约 $n/2$ 位，时间 $O(n \log\log n)$；返回表仍占 $O(\pi(n))$ 个整数。单个大整数分解改用 7.2.004 Pollard-Rho。

7.2.002 区间筛

当区间右端 r 很大（如 10^{12}）而区间长度 $r-l$ 仍可存下时，用区间筛求 $[l,r)$ 内的质数。先普通筛出 $\le \sqrt{r}$ 的质数，再对每个这样的质数 p，从 $\ge p^2$ 的首个 p 倍数起在长度 $r-l$ 的布尔数组上标记合数；$0,1$ 需显式排除。平方根依赖 7.1.014 整数 k 次方根 的 `kthRoot`。

```
vector<long long> segPrimes(long long l, long long r)
{
    // 返回半开区间 [l,r) 内的全部素数
    l = max(l, 2LL);
    if (l >= r)
    {
        return {};
    }
    int n = kthRoot((unsigned long long)(r - 1), 2);
    vector<bool> a(n + 1, true);
    vector<bool> b((size_t)(r - l), true);
    vector<int> p;
    for (int i = 2; i <= n; i++)
    {
        if (!a[i])
        {
            continue;
        }
        p.push_back(i);
        if (1LL * i * i <= n)
        {
            for (int j = i * i; j <= n; j += i)
            {
                a[j] = false;
```

```
            }
        }
    }
    for (long long x : p)
    {
        long long q = l / x + (l % x != 0);
        long long first = q > (r - 1) / x ? r : q * x;
        long long square = x > (r - 1) / x ? r : x * x;
        long long s = max(square, first);
        for (long long y = s; y < r; )
        {
            b[(size_t)(y - l)] = false;
            if (y > r - x)
            {
                break;
            }
            y += x;
        }
    }
    vector<long long> ans;
    for (long long x = l; x < r; x++)
    {
        if (b[x - l])
        {
            ans.push_back(x);
        }
    }
    return ans;
}
```

`segPrimes(l, r)` 返回 $[l, r)$ 内的全部质数（升序）；要求 $r > l$ 且 `r-l` 能转为无符号 64 位，区间长度 `r-l` 能作为容器大小。首个倍数和平方均用 64 位除法边界计算，不在循环中使用 128 位下标。设 $m = r - l$，时间 $O(\sqrt{r}\log\log r + m\log\log r)$，空间 $O(\sqrt{r} + m)$。整体值域不大时直接用 7.2.001 埃氏筛、线性筛。

7.2.003 Miller-Rabin

确定性判素：把 $n - 1 = d \cdot 2^s$，对每个底数 a 检查 $a^d \bmod n$ 是否为 1，或连续平方过程中是否出现 $n - 1$；两者都不满足则 n 是合数。对整个 64 位范围使用固定底数

$$2, 325, 9375, 28178, 450775, 9780504, 1795265022.$$

这组底数已被验证能排除全部强伪素数，因此结论确定、无需随机重复。乘法用 `unsigned __int128` 防溢出；`mul64`/`pow64` 也供 7.2.004 Pollard-Rho 复用。

```
unsigned long long mul64(unsigned long long a, unsigned long long b, unsigned long long mod)
{
    // a、b 是乘数，mod 是正模数；返回 a*b 对 mod 的最小非负余数。
```

```
    return (unsigned __int128)a * b % mod;
}

unsigned long long pow64(unsigned long long a, unsigned long long b, unsigned long long mod)
{
    // 返回 a^b 对正模数 mod 的最小非负余数
    assert(mod > 0); // 调试检查，可删
    unsigned long long ans = 1 % mod;
    a %= mod;
    while (b)
    {
        if (b & 1)
        {
            ans = mul64(ans, a, mod);
        }
        a = mul64(a, a, mod);
        b >>= 1;
    }
    return ans;
}

bool isPrime(unsigned long long n)
{
    // n 是 64 位无符号整数；返回 n 是否为质数，本底数集合在完整 64 位范围内确定。
    if (n < 2)
    {
        return false;
    }
    for (auto p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL, 23ULL, 29ULL, 31ULL,
    ↪  37ULL})
    {
        if (n % p == 0)
        {
            return n == p;
        }
    }
    int s = countr_zero(n - 1);
    auto d = (n - 1) >> s;
    for (auto a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL})
    {
        if (a % n == 0)
        {
            continue;
        }
        auto x = pow64(a % n, d, n);
        if (x == 1 || x == n - 1)
        {
            continue;
        }
        bool ok = false;
        for (int r = 1; r < s; r++)
        {
```

```
            x = mul64(x, x, n);
            if (x == n - 1)
            {
                ok = true;
                break;
            }
        }
        if (!ok)
        {
            return false;
        }
    }
    return true;
}
```

`isPrime(n)` 接收 `unsigned long long`；7 个底数各 $O(\log n)$ 次 u128 模乘，$O(1)$ 空间。合数要拆因子用 7.2.004 Pollard-Rho。快速检查分两行写：

- $2, 17, 2305843009213693951$ 判为质数；
- $1, 341, 3825123056546413051$ 判为合数。

7.2.004 Pollard-Rho

对 64 位合数随机找一个非平凡因子。在伪随机序列 $x \leftarrow x^2 + c \pmod n$ 上用 Brent 倍增区间判环；每 128 步把差值乘积取一次 `gcd`，将逐步 `gcd` 的开销摊薄。若批量乘积直接得到 n，再从该块起点逐步回退，保证不会漏掉块内较早出现的非平凡因子。拿到因子后递归分解两侧，配合 7.2.003 Miller-Rabin 的 `isPrime` 在质数处停下、`mul64` 做模乘，最终得到保留重数的升序质因子表。

```
unsigned long long pollard(unsigned long long n)
{
    // n 是合数且 n>1；用 Brent 分块乘积返回非平凡因子，依赖 7.2.003 的 mul64。
    if (n % 2 == 0)
    {
        return 2;
    }
    static mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    while (true)
    {
        auto c = rng() % (n - 1) + 1;
        auto y = rng() % (n - 1) + 1;
        auto f = [&](unsigned long long z)
        {
            return (unsigned long long)(((unsigned __int128)mul64(z, z, n) + c) % n);
        };
        unsigned long long x = 0, ys = 0, g = 1, q = 1, r = 1;
        constexpr unsigned long long m = 128;
        while (g == 1)
        {
            x = y;
            q = 1;
```

```
            for (unsigned long long i = 0; i < r; i++)
            {
                y = f(y);
            }
            for (unsigned long long k = 0; k < r && g == 1; k += m)
            {
                ys = y;
                auto lim = min(m, r - k);
                for (unsigned long long i = 0; i < lim; i++)
                {
                    y = f(y);
                    auto d = x > y ? x - y : y - x;
                    q = mul64(q, d, n);
                }
                g = gcd(q, n);
            }
            r <<= 1;
        }
        if (g == n)
        {
            do
            {
                ys = f(ys);
                auto d = x > ys ? x - ys : ys - x;
                g = gcd(d, n);
            } while (g == 1);
        }
        if (g ≠ n)
        {
            return g;
        }
    }
}

vector<unsigned long long> factor64(unsigned long long n)
{
    // n 是正整数；返回 n 的全部质因子并保留重数，结果升序，依赖 7.2.003 的 isPrime。
    assert(n > 0); // 调试检查，可删
    vector<unsigned long long> ans;
    auto dfs = [&](auto &&self, unsigned long long x) -> void
    {
        if (x == 1)
        {
            return;
        }
        if (isPrime(x))
        {
            ans.push_back(x);
            return;
        }
        auto d = pollard(x);
        self(self, d);
```

```
        self(self, x / d);
    };
    dfs(dfs, n);
    sort(ans.begin(), ans.end());
    return ans;
}
```

依赖 7.2.003 Miller-Rabin 的 `mul64`、`isPrime`。`factor64(n)` 返回升序质因子（含重数），$n = 1$ 返回空。Brent 版仍为期望约 $O(n^{1/4})$ 次模运算，但每块只做一次 `gcd`；递归栈 $O(\log n)$，无有限最坏保证。验证：`factor64(600851475143)` 得 $71, 839, 1471, 6857$。

7.2.005 质因数分解

对单个中小整数试除到 $\sqrt{n}$：合数必有 $\le \sqrt{n}$ 的质因子，逐个除尽后剩下大于 1 的部分只能是质数。只试 2 与奇数即可。批量分解筛表内整数用 7.2.001 埃氏筛、线性筛 的 `minp`，接近 64 位的大数用 7.2.004 Pollard-Rho。

```
vector<pair<int, int>> trialFactor(int n)
{
    // n 是正整数；返回按质因子递增排列的 {质因子, 指数}，适合单次分解中小整数。
    assert(n > 0); // 调试检查，可删
    vector<pair<int, int>> ans;
    for (int p = 2; p <= n / p; p += p == 2 ? 1 : 2)
    {
        if (n % p)
        {
            continue;
        }
        int e = 0;
        while (n % p == 0)
        {
            n /= p;
            e++;
        }
        ans.push_back({p, e});
    }
    if (n > 1)
    {
        ans.push_back({n, 1});
    }
    return ans;
}
```

`trialFactor(n)` 返回升序 `{质因子, 指数}`。$O(\sqrt{n})$ 时间、输出外 $O(1)$ 空间，适合单次分解不大的整数。验证：`trialFactor(360)` 得 $(2,3),(3,2),(5,1)$，`trialFactor(1000000007)` 得 $(1000000007, 1)$。

7.2.006 约数枚举、约数个数和

枚举单个整数的全部正约数：成对枚举到 $\sqrt{n}$，每找到 d 就同时得到 n/d，最后拼成升序表。若已有质因数分解 $n=\prod p_i^{e_i}$，则约数个数为 $\prod(e_i+1)$、约数和为 $\prod \frac{p_i^{e_i+1}-1}{p_i-1}$（各质数幂的等比和之积），`divCntSum` 直接由指数算出、不再分解。批量场景先用 7.2.001 埃氏筛、线性筛 筛 `minp` 再分解。

```cpp
vector<int> divisors(int n)
{
    // n 是正整数；返回 n 的全部正约数，已升序且不重复。
    assert(n > 0); // 调试检查，可删
    vector<int> a;
    vector<int> b;
    for (int d = 1; d <= n / d; d++)
    {
        if (n % d == 0)
        {
            continue;
        }
        a.push_back(d);
        if (d != n / d)
        {
            b.push_back(n / d);
        }
    }
    reverse(b.begin(), b.end());
    a.insert(a.end(), b.begin(), b.end());
    return a;
}

pair<int, int> divCntSum(const vector<pair<int, int>> &fac)
{
    // fac 的每项是不同质因子 p 及其指数 e；返回约数个数与约数和，结果须能放入有符号 64 位。
    int cnt = 1;
    int sum = 1;
    for (auto [p, e] : fac)
    {
        int pw = 1;
        int cur = 1;
        for (int i = 1; i <= e; i++)
        {
            pw *= p;
            cur += pw;
        }
        cnt *= e + 1;
        sum *= cur;
    }
    return {cnt, sum};
}
```

`divisors(n)` 返回升序约数表，$O(\sqrt{n})$ 时间、$O(d(n))$ 空间；`divCntSum(fac)` 输入 `{p,e}` 列表、返回 `{约数个数，约数和}`，$O(\sum e_i)$，本身不负责分解。

7.2.007 因数最多的数

求不超过 lim 且约数个数最多的最小正整数（反素数 / 高合成数）。关键观察：最优数的质因子必是从 2 起若干连续最小质数，且指数非增。于是按质数递增、指数非增地 DFS，当前约数数为各指数加一之积，乘积超界即剪枝；前 16 个质数足以覆盖 64 位范围。

```
namespace highlyComp
{
array<int, 2> mostDivisors(int lim)
{
    // lim 是正上界；返回的两维依次是最小整数及其因数个数。
    assert(lim >= 1); // 调试检查，可删
    constexpr array<int, 16> ps = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53};
    int best = 1;
    int bestCnt = 1;
    auto dfs = [&](auto &&self, int p, int last, int x, int cnt) -> void
    {
        // self 是递归自身，p 是质数下标，last 是指数上界，x 是当前乘积，cnt 是因数个数。
        if (cnt > bestCnt || (cnt == bestCnt && x < best))
        {
            best = x;
            bestCnt = cnt;
        }
        if (p == (int)ps.size())
        {
            return;
        }
        for (int e = 1; e <= last && x <= lim / ps[p]; e++)
        {
            x *= ps[p];
            self(self, p + 1, e, x, cnt * (e + 1));
        }
    };
    dfs(dfs, 0, 63, 1, 1);
    return {best, bestCnt};
}
}
```

`mostDivisors(lim)` 返回 {最小整数，其约数个数}。搜索节点数即满足非增指数约束的候选数，远小于逐个枚举，递归深度 $O(\log \mathrm{lim})$。

7.2.008 阶乘最后非零位

求 $n!$ 去掉末尾全部零后的个位数字（十进制）。末尾零由成对的 $2,5$ 产生，而 2 远多于 5，故零的个数等于 $n!$ 中 5 的指数。递归地把 $\lfloor n/5 \rfloor$ 段的贡献、本段 $n \bmod 5$ 的尾积、以及多出来的 2 的 4 周期相乘，$O(\log_5 n)$ 得结果；表 $d = \{1,1,2,6,4\}$ 是 $0!$ 到 $4!$ 的末位。

```
namespace lastNzDigit
{
```

```
int lastDigit(int n)
{
    // n 是非负整数；返回 n! 去掉末尾全部零后的个位数字。
    assert(n >= 0); // 调试检查，可删
    constexpr array<int, 5> d = {1, 1, 2, 6, 4};
    if (n < 5)
    {
        return d[n];
    }
    int p = n / 5;
    int two = array<int, 4>{6, 2, 4, 8}[p % 4];
    return lastDigit(p) * d[n % 5] * two % 10;
}
}
```

`lastDigit(n)` 返回 $n!$ 去尾零后的个位，$O(\log_5 n)$ 时间与递归栈。

7.2.009 阶乘模合数

计算 $n! \bmod p^e$（p 质数），是扩展 Lucas 的底层步骤。把 $n!$ 拆成质因子 p 的幂与不含 p 的单位部分两块：单位部分按完整 p^e 周期递归，`unitPrefix[i]` 预存一个周期内 $1..i$ 中删去 p 倍数后的前缀积，$\text{unitFact}(n) = \text{unitPrefix}[p^e]^{\lfloor n/p^e \rfloor} \cdot \text{unitPrefix}[n \bmod p^e] \cdot \text{unitFact}(\lfloor n/p \rfloor)$；$p$ 的幂指数由 Legendre 公式 $v_p(n!) = \sum_{j\ge 1} \lfloor n/p^j \rfloor$ 得到，指数达到 e 时整体为 0。本版直接预处理整个周期，只用于 $p^e \le 5 \times 10^6$；更大模数不能只删掉容量断言继续使用。依赖 7.1.001 快速幂 的 `powerMod`。

```
class PPFactorial
{
    // p 是质数，e 是指数，mod=p^e。
    int p, e, mod = 1;
    // unitPrefix[i] 是 1..i 中删去 p 的倍数后的乘积模 mod。
    vector<int> unitPrefix;

    int unitFact(int n) const
    {
        // n 是非负整数；返回 n! 删除全部质因子 p 后的单位部分模 mod。
        if (n == 0)
        {
            return 1;
        }
        int z = mulMod(powerMod(unitPrefix[mod], n / mod, mod), unitPrefix[n % mod], mod);
        return mulMod(z, unitFact(n / p), mod);
    }

  public:
    PPFactorial(int p, int e) : p(p), e(e)
    {
        // p 是质数，e 是正指数；预处理模 p^e 的单位前缀乘积，无返回值。
        assert(p >= 2 && e >= 1); // 调试检查，可删
        for (int i = 0; i < e; i++)
```

```
        {
            assert(mod <= 5000000 / p); // 调试检查，可删
            mod *= p;
        }
        unitPrefix.assign(mod + 1, 1);
        for (int i = 1; i <= mod; i++)
        {
            unitPrefix[i] = mulMod(unitPrefix[i - 1], i % p ? i : 1, mod);
        }
    }

    int modulus() const
    {
        // 无参数；返回构造时确定的质数幂模数 p^e。
        return mod;
    }

    int exponent(int n) const
    {
        // n 是非负整数；返回 n! 中质因子 p 的指数。
        assert(n >= 0); // 调试检查，可删
        int ans = 0;
        while (n)
        {
            n /= p;
            ans += n;
        }
        return ans;
    }

    int fact(int n) const
    {
        // n 是非负整数；返回 n! mod p^e。
        assert(n >= 0); // 调试检查，可删
        int q = exponent(n);
        if (q >= e)
        {
            return 0;
        }
        return mulMod(unitFact(n), powerMod(p, q, mod), mod);
    }

    array<int, 2> split(int n) const
    {
        // n 是非负整数；返回 {n! 的 p-adic 指数，删除 p 因子后的单位部分}。
        assert(n >= 0); // 调试检查，可删
        return {exponent(n), unitFact(n)};
    }
};
```

构造 `PPFactorial q(p, e)`，要求 p 质数、当前打印实现限制 $p^e \le 5 \times 10^6$（预处理开一个 p^e 长的前缀积表）。`modulus()` 返回构造时的 p^e；`fact(n)` 返回 $n! \bmod p^e$，`split(n)` 返回 {p-adic 指数，单位部

分}。预处理 $O(p^e)$ 时空，单次查询 $O(\log_p n)$。验证：模 $2^4 = 16$ 时 `fact(3)=6`、`fact(4)=8`、`fact(6)=0`，`split(6)` 的指数为 4。

7.2.010 哥德巴赫猜想及竞赛用结论

对不小于 4 的偶数 n，筛出 $[0, n]$ 的素数表后从小到大枚举 p，命中第一个使 p 与 $n-p$ 同为素数的拆分即返回。竞赛中猜想在题目给定范围内已由打表验证，这里只做确定性构造、不作为正确性证明。

```
vector<char> primeTable(int n)
{
    // n 是非负闭上界；返回 [0,n] 的素数标记表。
    assert(n >= 0); // 调试检查，可删
    vector<char> is(n + 1, true);
    is[0] = false;
    if (n >= 1)
    {
        is[1] = false;
    }
    for (int i = 2; i <= n / i; i++)
    {
        if (!is[i])
        {
            continue;
        }
        for (int j = i * i; j <= n; j += i)
        {
            is[j] = false;
        }
    }
    return is;
}

pair<int, int> goldbach(int n)
{
    // 参数 n 是不小于 4 的偶数；返回一组和为 n 的素数，未找到时返回 {-1,-1}。
    if (n < 4 || n % 2)
    {
        return {-1, -1};
    }
    auto is = primeTable(n);
    for (int p = 2; p <= n; p++)
    {
        if (is[p] && is[n - p])
        {
            return {p, n - p};
        }
    }
    return {-1, -1};
}
```

`goldbach(n)` 要求 n 为不小于 4 的偶数，返回一组和为 n 的素数 `{p, n-p}`，非法或未找到时返回 `{-1,-1}`。

筛表 $O(n \log \log n)$、搜索 $O(n)$、空间 $O(n)$。

7.2.011 麦乐鸡定理 / 两数 Frobenius 问题

对互质正整数 a, b，不能写成 $ax + by$（$x, y \geq 0$）的最大整数由两数 Frobenius 定理直接给出，为 $ab - a - b$；若 $\gcd(a, b) \neq 1$，则不可表示的数有无穷多个、无最大值。

```
__int128 frobenius2(int a, int b)
{
    // 互质正整数 a、b 的最大不可表示数
    assert(a > 0 && b > 0); // 调试检查，可删
    assert(gcd(a, b) == 1); // 调试检查，可删
    return (__int128)a * b - a - b;
}
```

`frobenius2(a, b)` 要求 a, b 为互质正整数；返回类型为 `__int128`，避免 $ab - a - b$ 在乘法时溢出。非互质输入没有最大不可表示数，不属于接口定义域。公式本身为 $O(1)$ 时间、$O(1)$ 空间；启用调试断言时，互质检查另需 $O(\log \min(a, b))$ 时间。

7.3 算术函数与筛法

7.3.001 欧拉函数

线性筛顺带求 $\varphi(1..n)$：质数 p 处 $\varphi(p) = p-1$；用最小质因子 p 扩展 $i \cdot p$ 时，若 $p \mid i$ 则 $\varphi(ip) = \varphi(i) \cdot p$，否则 $\varphi(ip) = \varphi(i) \cdot (p - 1)$，即 φ 在质数幂上的乘性递推。

```
vector<int> phiTable(int n)
{
    // n 是非负筛表上界；返回 0..n 的欧拉函数值，phi[1]=1。
    vector<int> phi(n + 1), minp(n + 1), primes;
    if (n >= 1)
    {
        phi[1] = 1;
    }
    for (int i = 2; i <= n; i++)
    {
        if (minp[i] == 0)
        {
            minp[i] = i;
            primes.push_back(i);
            phi[i] = i - 1;
        }
        for (auto p : primes)
        {
            if (p > minp[i] || i > n / p)
            {
                break;
            }
```

```
            minp[i * p] = p;
            if (i % p == 0)
            {
                phi[i * p] = phi[i] * p;
                break;
            }
            phi[i * p] = phi[i] * (p - 1);
        }
    }
    return phi;
}
```

`phiTable(n)` 返回 `phi[0..n]`、`phi[1]=1`，$O(n)$ 时空。一般积性函数用 7.3.003 积性函数，整除偏序上的累计与反演见 7.3.005 莫比乌斯反演，大范围前缀和见 7.3.006 杜教筛。

7.3.002 莫比乌斯函数

线性筛求 $\mu(1..n)$：扩展 $i \cdot p$ 时若 $p \mid i$ 则出现平方因子、$\mu(ip) = 0$，否则 $\mu(ip) = -\mu(i)$，符号由不同质因子个数的奇偶决定。

```
vector<int> mobiusTable(int n)
{
    // n 是非负筛表上界；返回 0..n 的莫比乌斯函数值，mu[1]=1。
    vector<int> mu(n + 1), minp(n + 1), primes;
    if (n >= 1)
    {
        mu[1] = 1;
    }
    for (int i = 2; i <= n; i++)
    {
        if (minp[i] == 0)
        {
            minp[i] = i;
            primes.push_back(i);
            mu[i] = -1;
        }
        for (auto p : primes)
        {
            if (p > minp[i] || i > n / p)
            {
                break;
            }
            minp[i * p] = p;
            if (i % p == 0)
            {
                mu[i * p] = 0;
                break;
            }
            mu[i * p] = -mu[i];
        }
```

```
    }
    return mu;
}
```

`mobiusTable(n)` 返回 `mu[0..n]`、`mu[1]=1`，位置 0 无数论含义，$O(n)$ 时空。反演见 7.3.005 莫比乌斯反演。

7.3.003 积性函数

给定 f 在质数幂上的值 $f(p^e)$（只需积性、不必完全积性），线性筛批量求 $f(1..n)$。维护每个数的最小质因子 p、其当前指数 e 与幂 p^e：若 $p \mid x$ 就在同一质数上升幂，$f(y) = f(x/p^e) \cdot f(p^{e+1})$；否则新质因子，$f(y) = f(x) \cdot f(p)$。

```
template <class T = long long, class F> vector<T> multTable(int n, F primePower)
{
    // n 是非负上界；primePower(p,e) 返回 f(p^e)，且 f 必须是积性函数；返回 f[0..n]，位置 0 不使用。
    assert(n >= 0); // 调试检查，可删
    vector<T> f(n + 1);
    vector<int> minp(n + 1);
    vector<int> exp(n + 1);
    vector<int> pp(n + 1);
    vector<int> primes;
    if (n >= 1)
    {
        f[1] = 1;
        pp[1] = 1;
    }
    for (int x = 2; x <= n; x++)
    {
        if (minp[x] == 0)
        {
            minp[x] = x;
            exp[x] = 1;
            pp[x] = x;
            primes.push_back(x);
            f[x] = primePower(x, 1);
        }
        for (auto p : primes)
        {
            if (p > minp[x] || x > n / p)
            {
                break;
            }
            int y = x * p;
            minp[y] = p;
            if (p == minp[x])
            {
                exp[y] = exp[x] + 1;
                pp[y] = pp[x] * p;
                f[y] = f[x / pp[x]] * primePower(p, exp[y]);
```

```
                break;
            }
            exp[y] = 1;
            pp[y] = p;
            f[y] = f[x] * primePower(p, 1);
        }
    }
    return f;
}
```

`multTable<T>(n, primePower)` 的回调返回类型 `T` 的 $f(p^e)$；所有质数幂函数值和中间乘积都必须能由 `T` 表示，模意义函数直接令 `T` 为模整数。下标、最小质因子和 $p^e \le n$ 仍使用普通 `int`，不随结果类型扩大。时间、空间均为 $O(n)$；φ、μ 的专用版见 7.3.001 欧拉函数、7.3.002 莫比乌斯函数。

例如令 $f(p^e) = e + 1$，即可线性筛出约数个数：

```
signed main()
{
    auto primePower = [](int p, int e)
    {
        (void)p;
        return e + 1;
    };
    auto d = multTable<int>(6, primePower);
    assert((d == vector<int>{0, 1, 2, 2, 3, 2, 4}));
}
```

7.3.004 Dirichlet 卷积

计算两个算术函数的 Dirichlet 卷积 $(f*g)(n) = \sum_{d\mid n} f(d)\,g(n/d)$。枚举因子 d 再遍历其倍数 $dk \le n$ 累加 $f(d)g(k)$，即调和级数式的 $O(n \log n)$ 求和；单位元 ε 只在 1 处为 1。

```
template <class T> vector<T> dirichlet(const vector<T> &a, const vector<T> &b)
{
    // a、b 在下标 1..n 保存两个算术函数；返回同长度的 Dirichlet 卷积，位置 0 不使用。
    assert(!a.empty() && !b.empty()); // 调试检查，可删
    int n = min(a.size(), b.size()) - 1;
    vector<T> c(n + 1);
    for (int d = 1; d <= n; d++)
    {
        for (int k = 1; k <= n / d; k++)
        {
            c[d * k] += a[d] * b[k];
        }
    }
    return c;
}
```

下标 $1..n$ 保存函数值、位置 0 留空；`dirichlet(a, b)` 两数组长度不同时只算公共上界，$O(n \log n)$ 时间、$O(n)$ 空间。若只需整除前缀和 / 反演，用 7.3.005 莫比乌斯反演 的原地变换更省空间。

7.3.005 莫比乌斯反演

整除偏序上的一对互逆变换。Zeta 为 $g(x)=\sum_{d\mid x} f(d)$。不能按所有 d 直接把已经更新过的 `a[d]` 再加到倍数，否则同一原始约数会沿 $1\mid 2\mid 4$ 等链重复累计；正确原地做法是逐个质数处理一个指数维度。对每个质数 p，按 x 递增加 `a[x]` 到 `a[xp]`，便恰好补上指数少一的状态；逆变换反序减回。

```
template <class T> void divisorZeta(vector<T> &a)
{
    // a 在下标 1..n 保存原函数；原地改成每个位置的约数和，位置 0 不使用。
    assert(!a.empty()); // 调试检查，可删
    int n = a.size() - 1;
    vector<char> comp(n + 1);
    vector<int> p;
    for (int x = 2; x <= n; x++)
    {
        if (!comp[x])
        {
            p.push_back(x);
        }
        for (auto q : p)
        {
            if (q > n / x)
            {
                break;
            }
            comp[x * q] = true;
            if (x % q == 0)
            {
                break;
            }
        }
    }
    for (auto q : p)
    {
        for (int x = 1; x <= n / q; x++)
        {
            a[x * q] += a[x];
        }
    }
}

template <class T> void divisorMu(vector<T> &a)
{
    // a 是 divisorZeta 的结果；原地恢复下标 1..n 的原函数，位置 0 不使用。
    assert(!a.empty()); // 调试检查，可删
    int n = a.size() - 1;
    vector<char> comp(n + 1);
    vector<int> p;
    for (int x = 2; x <= n; x++)
    {
        if (!comp[x])
```

```
        {
            p.push_back(x);
        }
        for (auto q : p)
        {
            if (q > n / x)
            {
                break;
            }
            comp[x * q] = true;
            if (x % q == 0)
            {
                break;
            }
        }
    }
    for (auto it = p.rbegin(); it != p.rend(); it++)
    {
        int q = *it;
        for (int x = n / q; x >= 1; x--)
        {
            a[x * q] -= a[x];
        }
    }
}
```

下标 $1..n$，`divisorZeta(a)` 求约数和、`divisorMu(a)` 由其恢复原函数，二者严格互逆。筛素数加各质数维更新为 $O(n \log\log n)$ 时间，素数表与标记为 $O(n)$ 额外空间。固定例 `a={0,1,0,0,0}` 变换后应为 `{0,1,1,1,1}`，可直接识别旧实现对位置 4 的重复累计。

7.3.006 杜教筛

求积性函数 f 的大范围前缀和 $S(n)=\sum_{i=1}^{n} f(i)$ 而不必线性筛到 n。选一个便于求和的 g，由 $\sum_{i\le n}(f*g)(i)=\sum_{d\le n} g(d)\,S(\lfloor n/d\rfloor)$ 得 $g(1)S(n)=\sum_{i\le n}(f*g)(i)-\sum_{d\ge 2} g(d)\,S(\lfloor n/d\rfloor)$，右端对 d 整除分块递归。此处 μ,φ 都取 $g=1$：$\mu*1=\varepsilon$ 故 $\sum(f*g)=1$，$\varphi*1=\mathrm{id}$ 故 $\sum(f*g)=\frac{n(n+1)}{2}$。预筛到约 $n^{2/3}$，并对 $\lfloor n/i\rfloor$ 的 $O(\sqrt{n})$ 种取值记忆化。

```
struct DuSieve
{
    int n;                          // 预筛上界
    vector<int> p;                  // 预筛范围内的素数
    vector<int> mu;                 // 莫比乌斯函数值
    vector<int> phi;                // 欧拉函数值
    vector<bool> vis;               // 线性筛合数标记
    vector<long long> smu;          // 莫比乌斯函数前缀和
    vector<__int128> sphi;          // 欧拉函数前缀和
    unordered_map<long long, long long> cmu; // 大参数 Mertens 值
    unordered_map<long long, __int128> cphi; // 大参数欧拉函数前缀和

```

```
    DuSieve(long long m)
    {
        // m 是最大查询上界；预筛约 m^(2/3) 内的 mu、phi 及其前缀和。
        assert(m >= 0); // 调试检查，可删
        n = max(1LL, (long long)(powl(m, 2.0L / 3) + 2));
        mu.resize(n + 1);
        phi.resize(n + 1);
        vis.resize(n + 1);
        smu.resize(n + 1);
        sphi.resize(n + 1);
        mu[1] = phi[1] = 1;
        for (int i = 2; i <= n; i++)
        {
            if (!vis[i])
            {
                p.push_back(i);
                mu[i] = -1;
                phi[i] = i - 1;
            }
            for (int q : p)
            {
                if (1LL * i * q > n)
                {
                    break;
                }
                vis[i * q] = true;
                if (i % q == 0)
                {
                    phi[i * q] = phi[i] * q;
                    break;
                }
                mu[i * q] = -mu[i];
                phi[i * q] = phi[i] * (q - 1);
            }
        }
        for (int i = 1; i <= n; i++)
        {
            smu[i] = smu[i - 1] + mu[i];
            sphi[i] = sphi[i - 1] + phi[i];
        }
    }

    long long sumMu(long long x)
    {
        // x 是查询点；返回 sum_{i=1}^x mu(i)，超出预筛范围时整除分块递归并记忆化。
        assert(x >= 0); // 调试检查，可删
        if (x <= n)
        {
            return smu[x];
        }
        if (cmu.contains(x))
        {
```

```
            return cmu[x];
        }
        __int128 ans = 1;
        long long l = 2;
        while (l <= x)
        {
            long long r = x / (x / l);
            ans -= (__int128)(r - l + 1) * sumMu(x / l);
            if (r == x)
            {
                break;
            }
            l = r + 1;
        }
        return cmu[x] = (long long)ans;
    }

    __int128 sumPhi(long long x)
    {
        // x 是查询点；返回 sum_{i=1}^x phi(i)，超出预筛范围时整除分块递归并记忆化。
        assert(x >= 0); // 调试检查，可删
        if (x <= n)
        {
            return sphi[x];
        }
        if (cphi.contains(x))
        {
            return cphi[x];
        }
        __int128 ans = (__int128)x * ((__int128)x + 1) / 2;
        long long l = 2;
        while (l <= x)
        {
            long long r = x / (x / l);
            ans -= (__int128)(r - l + 1) * sumPhi(x / l);
            if (r == x)
            {
                break;
            }
            l = r + 1;
        }
        return cphi[x] = ans;
    }
};
```

构造 `DuSieve S(n)`（n 是最大查询上界）后调用 `S.sumMu(x)` / `S.sumPhi(x)`，要求 $0 \le x \le n$。`sumPhi` 返回 `__int128`，三角和、块长乘法和缓存都保持宽类型；`sumMu` 的中间差也先提升再转回其必在 $[-x, x]$ 内的结果。预筛 $O(n^{2/3})$ 时空；缓存后同参查询 $O(1)$。更一般的积性函数前缀和用 7.3.007 Min_25 筛。

7.3.007 Min_25 筛

求积性函数 f 的前缀和 $\sum_{i=1}^{n} f(i)$，要求 f 在质数处等于一个低次多项式、在质数幂处可快速求值。分两步：先对每个不同的 $\lfloor n/i \rfloor$ 求「不超过它的素数的幂和」——`primeCount` 求素数个数、`primeSum` 求素数一次和，逐个质数扣掉最小质因子恰为该质数的合数贡献；再 `dfs` 枚举每个质数幂，把质数幂上的函数值与其后缀素数贡献拼起来。本模板取质数处 $f(p) = A + Bp$，由 `linearPPow(A,B)` 求和。

```
struct Min25
{
    long long n;   // 前缀和上界
    long long sq;  // sq 是 floor(sqrt(n))。
    long long hi;  // 大商与小商的分界
    long long s;   // 不同整除商的数组长度
    vector<int> p; // 不超过 sq 的素数
    vector<Z> buf; // 各离散商处的素数 f 值前缀和
    Z A;           // f(p^e)=A*e+B*p 的指数系数
    Z B;           // f(p^e)=A*e+B*p 的质数系数
    Z ans;         // 当前查询的答案

    Min25(long long n) : n(n), sq(0)
    {
        // n 是前缀和上界（n>0）；筛出不超过 sqrt(n) 的素数并预置离散商的分界。
        assert(n > 0); // 调试检查，可删
        sq = sqrtl(n);
        while ((sq + 1) <= n / (sq + 1))
        {
            sq++;
        }
        while (sq > n / sq)
        {
            sq--;
        }
        hi = n ? n / sq : 0;
        if (hi != 1 && n / (hi - 1) == sq)
        {
            hi--;
        }
        s = hi + sq;
        vector<bool> isp(sq + 1, true);
        for (int i = 2; i <= sq; i++)
        {
            if (isp[i])
            {
                p.push_back(i);
                if (1LL * i * i <= sq)
                {
                    for (int j = i * i; j <= sq; j += i)
                    {
                        isp[j] = false;
                    }
                }
```

```
            }
        }
    }

    int id(long long x) const
    {
        // x 是某个整除商 n/l；返回它在离散数组中的下标。
        return x <= sq ? s - x : n / x;
    }

    vector<Z> primeCount() const
    {
        // 无参数；返回各离散整除商对应的不超过该值的素数个数。
        vector<Z> h(s);
        for (long long x = 1; x < s; x++)
        {
            long long v = x < hi ? n / x : s - x;
            h[x] = v - 1;
        }
        int cnt = 0;
        for (long long q : p)
        {
            for (long long x = 1; x < s; x++)
            {
                long long v = x < hi ? n / x : s - x;
                if (v < q * q)
                {
                    break;
                }
                h[x] -= h[id(v / q)] - cnt;
            }
            cnt++;
        }
        return h;
    }

    vector<Z> primeSum() const
    {
        // 无参数；返回各离散整除商对应的不超过该值的素数和。
        vector<Z> h(s);
        for (long long x = 1; x < s; x++)
        {
            long long v = x < hi ? n / x : s - x;
            if (v & 1)
            {
                h[x] = Z(v) * Z(v / 2 + 1) - 1;
            }
            else
            {
                h[x] = Z(v / 2) * Z(v + 1) - 1;
            }
        }
```

```
        for (long long q : p)
        {
            Z before = h[id(q - 1)];
            for (long long x = 1; x < s; x++)
            {
                long long v = x < hi ? n / x : s - x;
                if (v < q * q)
                {
                    break;
                }
                h[x] -= Z(q) * (h[id(v / q)] - before);
            }
        }
        return h;
    }

    Z f(long long q, int e) const
    {
        // q 是质数 p，e 是指数；返回质数幂上的函数值 f(p^e)=A·e+B·p。
        return A * e + B * q;
    }

    void dfs(int i, int e, long long prod, Z cur)
    {
        // i、e 指当前质数及指数，prod 是已选乘积，cur 是此前质因子贡献；递归累加后续贡献。
        ans += cur * f(p[i], e + 1);
        long long lim = n / prod;
        if (lim >= 1LL * p[i] * p[i])
        {
            dfs(i, e + 1, prod * p[i], cur);
        }
        cur *= f(p[i], e);
        ans += cur * (buf[id(lim)] - buf[id(p[i])]);
        for (int j = i + 1; j < (int)p.size() && 1LL * p[j] * p[j] <= lim; j++)
        {
            dfs(j, 1, prod * p[j], cur);
        }
    }

    Z linearPPow(Z a, Z b)
    {
        // 对 f(p^e)=a*e+b*p，返回 sum_{i=1}^n f(i)
        A = a;
        B = b;
        auto c = primeCount();
        auto d = primeSum();
        buf.resize(s);
        for (int i = 0; i < s; i++)
        {
            buf[i] = c[i] * A + d[i] * B;
        }
        ans = buf[id(n)] + 1;
```

```
        for (int i = 0; i < (int)p.size(); i++)
        {
            dfs(i, 1, p[i], 1);
        }
        return ans;
    }
};
```

依赖 1.1.008 静态取模类的取模类 `Z`（本节不重复打印）。构造 `Min25 q(n)` 后，`q.linearPPow(A,B)` 只适用于完整定义为 $f(p^e)=Ae+Bp$ 的积性函数；只知道质数处 $f(p)=A+Bp$ 不足以确定质数幂，不能据此套用。离散商按对应数值递减扫描，低于 p^2 后立即结束该质数的更新，表长 $O(\sqrt{n})$，本通用递归版约 $O\left(\frac{n^{3/4}}{\log n}\right)$ 时间、$O(\sqrt{n})$ 空间。只求 μ,φ 前缀和时 7.3.006 杜教筛 更简单。

7.3.008 Meissel-Lehmer 质数计数

单点求 $\pi(n)$（不超过 n 的素数个数）而不枚举全部素数，适合少量大整数查询。核心是 Legendre 函数 $\phi(x,a)$——$[1,x]$ 中不被前 a 个素数整除的数的个数，满足 $\phi(x,a)=\phi(x,a-1)-\phi(\lfloor x/p_a\rfloor,a-1)$。Lehmer 公式把 $\pi(x)$ 写成 $\phi(x,a)$ 加上跨 $\sqrt[4]{x},\sqrt[3]{x},\sqrt{x}$ 若干区间的修正项，从而递归到更小规模；不超过 N 的部分预筛 `pi` 直接查表。

```
// 依赖 7.1.014 整数 k 次方根
struct LehmerPi
{
    // N 是预筛上界，不超过 N 的查询直接用 pi 表回答。
    static constexpr int N = 5000000;
    // p 是不超过 N 的素数表；pi[x] 是不超过 x 的素数个数（x<=N）。
    vector<int> p, pi;
    // memPhi 缓存 Legendre 函数 phi(x,s) 的值。
    unordered_map<unsigned long long, long long> memPhi;
    // memPi 缓存大参数处的 pi(x)。
    unordered_map<long long, long long> memPi;

    LehmerPi() : pi(N + 1)
    {
        // 无参数；线性筛出不超过 N 的素数及其计数前缀 pi。
        vector<bool> c(N + 1);
        for (int i = 2; i <= N; i++)
        {
            if (!c[i])
            {
                p.push_back(i);
            }
            for (int q : p)
            {
                if (1LL * i * q > N)
                {
                    break;
                }
                c[i * q] = true;
```

```
                if (i % q == 0)
                {
                    break;
                }
            }
            pi[i] = p.size();
        }
    }

    long long phi(long long x, int s)
    {
        // Legendre phi: 返回 [1,x] 中不被前 s 个素数整除的整数个数。
        assert(x >= 0 && 0 <= s && s <= (int)p.size()); // 调试检查，可删
        if (!x)
        {
            return 0;
        }
        if (!s)
        {
            return x;
        }
        if (s == 1)
        {
            return x - x / 2;
        }
        if (x <= N && 1LL * p[s - 1] * p[s - 1] > x)
        {
            return max(1LL, pi[x] - s + 1);
        }
        if (s < 100 && x < 100000)
        {
            unsigned long long k = (unsigned long long)x << 7 | s;
            if (auto it = memPhi.find(k); it != memPhi.end())
            {
                return it->second;
            }
            return memPhi[k] = phi(x, s - 1) - phi(x / p[s - 1], s - 1);
        }
        return phi(x, s - 1) - phi(x / p[s - 1], s - 1);
    }

    long long operator()(long long x)
    {
        // x 是查询点；返回 pi(x)= 不超过 x 的素数个数。
        assert(0 <= x && x <= (long long)N * N); // 调试检查，可删
        if (x <= N)
        {
            return pi[x];
        }
        if (auto it = memPi.find(x); it != memPi.end())
        {
            return it->second;
```

```
        }
        long long a = (*this)(kthRoot((unsigned long long)x, 4));
        long long b = (*this)(kthRoot((unsigned long long)x, 2));
        long long c = (*this)(kthRoot((unsigned long long)x, 3));
        long long ans = phi(x, a) + (b + a - 2) * (b - a + 1) / 2;
        for (long long i = a; i < b; i++)
        {
            long long w = x / p[i];
            ans -= (*this)(w);
            if (i < c)
            {
                long long z = (*this)(kthRoot((unsigned long long)w, 2));
                for (long long j = i; j < z; j++)
                {
                    ans -= (*this)(w / p[j]) - j;
                }
            }
        }
        return memPi[x] = ans;
    }
};
```

构造 `LehmerPi pi;` 一次性预筛 $N = 5 \times 10^6$，随后 `pi(x)` 返回 $\pi(x)$。固定表版本要求 $0 \le x \le N^2 = 2.5 \times 10^{13}$，保证外层访问的全部 $\le \sqrt{x}$ 素数已经筛出；更大的查询必须把 N 随最大输入提高，不能沿用本表。各整数根调用 7.1.014 整数 k 次方根 的 `kthRoot` 校正。预筛 $O(N)$ 时空，单次查询约 $O\left(\frac{x^{2/3}}{\log x}\right)$。

7.3.009 整除分块

把 $\lfloor n/i \rfloor$ 相等的连续下标并成一块：固定 n 时商在 $[l,\ \lfloor n/\lfloor n/l \rfloor \rfloor]$ 内不变，每次跳到该块右端加一，只需枚举约 $2\sqrt{n}$ 个块。`divBlocks` 返回每块 $[l, r]$ 及其公共商，`quotients` 只取不同商值（升序）。

```
struct DivBlock
{
    // l、r 是该块闭区间的左右端点；q 是块内公共商 floor(n/l)。
    long long l, r, q;
    bool operator==(const DivBlock &) const = default;
};

vector<DivBlock> divBlocks(long long n)
{
    // n 是正整数；返回所有极大闭区间 [l,r]，段内 floor(n/i) 同为 q。
    assert(n > 0); // 调试检查，可删
    vector<DivBlock> a;
    long long l = 1;
    while (l <= n)
    {
        long long q = n / l;
        long long r = n / q;
        a.push_back({l, r, q});
        if (r == n)
```

```
        {
            break;
        }
        l = r + 1;
    }
    return a;
}

vector<long long> quotients(long long n)
{
    // n 是正整数；返回 floor(n/i) 的全部不同取值（升序）。
    vector<long long> a;
    for (const auto &block : divBlocks(n))
    {
        a.push_back(block.q);
    }
    reverse(a.begin(), a.end());
    return a;
}
```

`divBlocks(n)` 返回 `{l, r, q}` 列表，每段对整除型求和贡献 $(r-l+1)F(q)$；`quotients(n)` 返回 $\lfloor n/i \rfloor$ 的不同取值。块数不超过 $2\lfloor\sqrt{n}\rfloor$，$O(\sqrt{n})$ 时间与空间。它是 7.3.006 杜教筛 递归的基础。

7.3.010 调和级数分块

这里的“调和级数分块”与 7.3.009 整除分块 是同一分解——把 $\lfloor n/l \rfloor$ 相同的下标合成极大闭区间；名字只是强调按每段贡献 $\sum_q F(q)\cdot(\text{段长})$ 求和时，段数与调和级数求和的阶数相当。直接复用该条的 `divBlocks(n)`（每段 `{l, r, q}`，$q=\lfloor n/l \rfloor$），整段贡献 $(r-l+1)F(q)$。例如 $n=10$ 依次得到 $[1,1,10],[2,2,5],[3,3,3],[4,5,2],[6,10,1]$。

> 备注：本条不再单独实现。原资料这里有一份返回 `array<int,3>`、参数为 `int` 的 `divBlocks`，但在本章 `#define int long long` 下 `int` 与 `long long` 是同一类型，它与 7.3.009 整除分块 的 `divBlocks(long long)` 构成同签名重定义、无法同时编译，故合并为引用。

7.3.011 素数、$\omega(n)$ 与 $d(n)$

线性筛同时递推三个量：不同质因子数 ω、含重数质因子数 Ω、约数个数 d。用最小质因子 p 扩展 $y=xp$ 时，若 $p \mid x$（升幂，旧指数 e）则 ω 不变、$\Omega+1$，$d(y)=d(x)/(e+1)\cdot(e+2)$；否则遇新质因子，$\omega,\Omega$ 各 $+1$、d 翻倍。

```
vector<array<int, 3>> factorStats(int n)
{
    // n 是筛表上界；返回 s[x]={omega(x),Omega(x),d(x)}，依次为不同质因子数、含重数质因子数、约数数。
    vector<array<int, 3>> s(n + 1);
    vector<int> minp(n + 1), exp(n + 1), primes;
    if (n >= 1)
    {
        s[1] = {0, 0, 1};
```

```
    }
    for (int x = 2; x <= n; x++)
    {
        if (minp[x] == 0)
        {
            minp[x] = x;
            exp[x] = 1;
            primes.push_back(x);
            s[x] = {1, 1, 2};
        }
        for (auto p : primes)
        {
            if (p > minp[x] || x > n / p)
            {
                break;
            }
            int y = x * p;
            minp[y] = p;
            if (p == minp[x])
            {
                exp[y] = exp[x] + 1;
                s[y] = {s[x][0], s[x][1] + 1, s[x][2] / (exp[x] + 1) * (exp[y] + 1)};
                break;
            }
            exp[y] = 1;
            s[y] = {s[x][0] + 1, s[x][1] + 1, s[x][2] * 2};
        }
    }
    return s;
}
```

`factorStats(n)` 返回 `s[0..n]`，`s[x]={omega(x), Omega(x), d(x)}`，`s[1]={0,0,1}`，$O(n)$ 时空。单个大整数可先用 7.2.004 Pollard-Rho 分解，再由质因子表直接算出这些统计量。

7.3.012 商集上的 Dirichlet 卷积与除法

固定上界 N 后，$\lfloor N/i \rfloor$ 只有 $O(\sqrt{N})$ 个不同值。本条的数组在这些商值上保存算术函数前缀和：第 `i` 项表示 $F(v_i)=\sum_{k\le v_i} f(k)$，而不是 `f(i)`。利用小参数与大商值分治，可直接得到 $f*g$ 的同类前缀和表；若 $f(1)$ 可逆，还能由 $H=F*G$ 反解 G。这适合杜教筛、Min_25 一类只访问整除商集的批量运算。

```
// 依赖 1.1.008 静态取模类
template <int Mod> using StaticModInt = MInt<(int)Mod>;

struct QuotDirich
{
    unsigned long long lim, sq, t; // lim 是 N, sq 是 floor(sqrt N), t 是小值段 1..t 的上界。
    int n; // n 是 lim/i 的不同取值个数（表长）。

```

```
    static unsigned long long isqrt(unsigned long long x)
    {
        // x 是非负整数；返回 floor(sqrt(x))。
        unsigned long long r = sqrtl((long double)x);
        while (r + 1 <= x / (r + 1))
        {
            r++;
        }
        while (r && r > x / r)
        {
            r--;
        }
        return r;
    }

    static unsigned long long icbrt(unsigned long long x)
    {
        // x 是非负整数；返回 floor(cuberoot(x))。
        unsigned long long r = cbrtl((long double)x);
        auto ok = [&](unsigned long long y)
        {
            return y == 0 || y <= x / y / y;
        };
        while (ok(r + 1))
        {
            r++;
        }
        while (!ok(r))
        {
            r--;
        }
        return r;
    }

    QuotDirich(unsigned long long N) : lim(N)
    {
        // N 是固定查询上界；建立其全部整除商值的双端下标。
        assert(1 <= N && N <= (1ULL << 50)); // 调试检查，可删。
        sq = isqrt(N);
        t = sq * sq + sq <= N ? sq : sq - 1;
        n = t + sq + 1;
    }

    int id(unsigned long long x) const
    {
        // x 必须是 lim/i 的某个取值；返回它在表中的下标。
        return x <= t ? x : n - lim / x;
    }

    unsigned long long value(int i) const
    {
        // i 是表下标；返回对应商值。
```

```
        return i <= (int)t ? i : lim / (n - i);
    }

    template <class T, class F> vector<T> sumTable(F prefix) const
    {
        // prefix(x) 返回原函数在 1..x 的前缀和；生成商集表。
        vector<T> a(n);
        for (int i = 1; i < n; i++)
        {
            a[i] = prefix(value(i));
        }
        return a;
    }

    template <class T> vector<T> conv(vector<T> F, vector<T> G) const
    {
        // F、G 是两函数的商集前缀和表；返回 Dirichlet 卷积的商集前缀和表。
        assert((int)F.size() == n && (int)G.size() == n); // 调试检查，可删。
        if (lim == 1)
        {
            return {T(0), F[1] * G[1]};
        }
        vector<T> f(n), g(n), H(n);
        for (int i = 1; i < n; i++)
        {
            f[i] = F[i] - F[i - 1];
            g[i] = G[i] - G[i - 1];
        }
        unsigned long long k = icbrt(lim);
        unsigned long long s = k * k;
        for (unsigned long long a = 1; a <= k; a++)
        {
            H[id(a * a)] += f[a] * g[a];
            if (a * (a + 1) <= t)
            {
                for (unsigned long long b = a + 1; b <= t / a; b++)
                {
                    H[a * b] += f[a] * g[b] + f[b] * g[a];
                }
            }
            unsigned long long q = min(s / a, t);
            for (unsigned long long b = max(a, t / a) + 1; b <= q; b++)
            {
                H[id(a * b)] += f[a] * g[b] + f[b] * g[a];
            }
            if (lim / sq <= s / a)
            {
                unsigned long long p = lim / (s / a + 1) + 1;
                for (unsigned long long b = p; b <= sq; b++)
                {
                    H[id(a * (lim / b))] += f[a] * g[n - b] + g[a] * f[n - b];
                }
```

```
            }
        }
        for (int i = 1; i < n; i++)
        {
            H[i] += H[i - 1];
        }
        for (unsigned long long z = 1; lim / z > s; z++)
        {
            unsigned long long m = lim / z;
            unsigned long long u = isqrt(m);
            H[n - z] = T(0);
            for (unsigned long long a = 1; a <= u; a++)
            {
                int j = id(m / a);
                H[n - z] += f[a] * G[j] + g[a] * F[j];
            }
            H[n - z] -= F[u] * G[u];
        }
        return H;
    }

    template <class T> vector<T> divide(vector<T> H, vector<T> F) const
    {
        // H、F 是商集前缀和表且 f(1) 可逆；返回满足 H=F*G 的 G 表。
        assert((int)H.size() == n && (int)F.size() == n && F[1] != T(0)); // 调试检查，可删。
        if (lim == 1)
        {
            return {T(0), H[1] / F[1]};
        }
        T inv = T(1) / F[1];
        for (auto &x : F)
        {
            x *= inv;
        }
        vector<T> f(n), g(n), h(n);
        for (int i = 1; i < n; i++)
        {
            f[i] = F[i] - F[i - 1];
            h[i] = H[i] - H[i - 1];
        }
        unsigned long long k = icbrt(lim);
        unsigned long long s = max(sq, k * k);
        g[1] = H[1];
        for (int i = 2; i < n; i++)
        {
            unsigned long long a = value(i);
            if (a > s)
            {
                break;
            }
            g[i] = h[i] - g[1] * f[i];
            if (a * a <= s)
```

```
            {
                h[id(a * a)] -= f[i] * g[i];
            }
            unsigned long long u = min<unsigned long long>(i - 1, s / a);
            for (unsigned long long b = 2; b <= u; b++)
            {
                h[id(a * b)] -= f[i] * g[b] + f[b] * g[i];
            }
        }
        vector<T> G(n);
        for (int i = 1; i < n; i++)
        {
            G[i] = G[i - 1] + g[i];
        }
        for (unsigned long long z = lim / (s + 1); z > 0; z--)
        {
            unsigned long long m = lim / z;
            unsigned long long u = isqrt(m);
            G[n - z] = H[n - z] - g[1] * F[n - z] + F[u] * G[u];
            for (unsigned long long a = 2; a <= u; a++)
            {
                int j = id(m / a);
                G[n - z] -= f[a] * G[j] + g[a] * F[j];
            }
        }
        for (auto &x : G)
        {
            x *= inv;
        }
        return G;
    }
};
```

先构造 `QuotDirich q(N)`，再用 `q.sumTable<T>(prefix)` 生成表；`q.conv(F,G)` 求卷积，`q.divide(H,F)` 反解且要求 $f(1)$ 在 `T` 中可逆。固定素数模直接使用 1.1.008 静态取模类的 `MInt<P>`；`StaticModInt<Mod>` 只是兼容旧调用名的单行别名，不维护第二份取模实现。这里公开限制 $1 \le N \le 2^{50}$，时间约 $O(N^{2/3})$、空间 $O(\sqrt{N})$；实际常数和最优分界会随平台变化。小 N 直接使用 7.3.004 Dirichlet 卷积 更简单。

下面取常数函数 $f(n)=g(n)=1$；其卷积为约数个数 $d(n)$，并由卷积表反解除原函数：

```
signed main()
{
    QuotDirich q(10);
    auto F = q.sumTable<Z>([](unsigned long long x)
    {
        return Z(x);
    });
    auto H = q.conv(F, F);
    assert(q.divide(H, F) == F);
    assert(H[q.id(1)].val() == 1);
    assert(H[q.id(2)].val() == 3);
```

```
    assert(H[q.id(3)].val() == 5);
    assert(H[q.id(5)].val() == 10);
    assert(H[q.id(10)].val() == 27);
}
```

7.3.013 无平方因子数计数

无平方因子数的指标为 $\sum_{k^2|x}\mu(k)$，故不超过 n 的个数

$$Q(n)=\sum_{k\le\sqrt{n}}\mu(k)\left\lfloor\frac{n}{k^2}\right\rfloor.$$

取 $I\approx n^{1/5}$、$D=\lfloor\sqrt{n/I}\rfloor$，$k\le D$ 直接求和；余下部分按 $i=\lfloor n/k^2\rfloor<I$ 分组并望远镜化为 $\sum_{i=1}^{I-1}M(\lfloor\sqrt{n/i}\rfloor)-(I-1)M(D)$。这些大 Mertens 值再用 $M(x)=1-\sum_{j=2}^{x}M(\lfloor x/j\rfloor)$ 递推；所需的大商恰好已经按 i 从大到小算出。为控制 $n\le10^{18}$ 时的内存，μ 用 `int`，$M(0..D)$ 用 `int`。

```
long long sqrtFloor(long long n)
{
    // n 是非负整数；返回 floor(sqrt(n))，先用浮点估计再整数校正。
    long long x = sqrtl((long double)n);
    while ((x + 1) <= n / (x + 1))
    {
        x++;
    }
    while (x > n / x)
    {
        x--;
    }
    return x;
}

vector<int> squarefreeMu(int n)
{
    // n 是上界；返回 Mobius 函数值，mu[1..n] 有效（mu[0] 无意义）。
    vector<int> mu(n + 1, 1);
    vector<bool> composite(n + 1);
    for (int p = 2; p <= n; p++)
    {
        if (composite[p])
        {
            continue;
        }
        mu[p] = -1;
        for (int j = p * 2; j <= n; j += p)
        {
            composite[j] = true;
            mu[j] = -mu[j];
        }
        if (p <= n / p)
        {
```

```
            for (int j = p * p; j <= n; j += p * p)
            {
                mu[j] = 0;
            }
        }
    }
    return mu;
}

long long countSqFree(long long n)
{
    // n 是上界；返回 [1,n] 中无平方因子数的个数，n<=0 时为 0。
    if (n <= 0)
    {
        return 0;
    }
    int I = max<long long>(1, powl((long double)n, 0.2L));
    int D = sqrtFloor(n / I);
    auto mu = squarefreeMu(D);
    vector<int> mertens(D + 1);
    long long direct = 0;
    for (int k = 1; k <= D; k++)
    {
        mertens[k] = mertens[k - 1] + mu[k];
        direct += (long long)mu[k] * (n / k / k);
    }
    mu.clear();
    mu.shrink_to_fit();

    vector<long long> large;
    large.reserve(I - 1);
    long long sum = 0;
    for (int i = I - 1; i >= 1; i--)
    {
        long long x = sqrtFloor(n / i);
        long long s = sqrtFloor(x);
        long long cur = 1;
        for (int q = 1; q <= x / (s + 1); q++)
        {
            cur -= (x / q - x / (q + 1)) * mertens[q];
        }
        for (int j = 2; j <= s; j++)
        {
            long long y = x / j;
            if (y <= D)
            {
                cur -= mertens[y];
            }
            else
            {
                cur -= large[I - j * j * i - 1];
            }
```

```
        }
        large.push_back(cur);
        sum += cur;
    }
    return direct + sum - (I - 1) * (long long)mertens[D];
}
```

`countSqFree(n)` 返回 $[1,n]$ 中无平方因子的正整数个数，$n \le 0$ 返回 0。时间约 $O(n^{2/5}\log\log n)$、空间 $O(n^{2/5})$；$n = 10^{18}$ 时结果仍不超过 n，64 位足够。小上界已经有 7.3.002 莫比乌斯函数 的线性筛，本条用于无法筛到 $\sqrt{n}$ 的范围。

下面检查小范围边界，并保留 10^{18} 的已核对结果：

```
signed main()
{
    assert(countSqFree(0) == 0);
    assert(countSqFree(1) == 1);
    assert(countSqFree(10) == 7);
    assert(countSqFree(100) == 61);
    assert(countSqFree(1000000000000000000LL) == 607927101854022750LL);
}
```

7.3.014 Lucy 素数前缀筛

Lucy 筛只保留 $\lfloor n/i \rfloor$ 的不同取值，同时维护

$$G_0(x) = x - 1, \qquad G_1(x) = \frac{x(x+1)}{2} - 1,$$

它们分别是候选数 $2, \dots, x$ 的个数与和。按素数 p 递推时，删除最小素因子恰为 p 的合数：

$$G_k(x) \leftarrow G_k(x) - p^k\big(G_k(\lfloor x/p \rfloor) - G_k(p-1)\big), \quad x \ge p^2.$$

处理完 $p \le \sqrt{n}$ 后，得到 $\pi(x)$ 与不超过 x 的素数和。它是 Min_25 筛中最常用的第一步，适合只需要素数个数或素数和的整除分块问题。

```
// 依赖 7.1.014 整数 k 次方根
struct PrimePrefix
{
    int x; // 商集中的 floor(n/i)。
    int cnt; // 不超过 x 的素数个数。
    __int128 sum; // 不超过 x 的素数和，初始化的三角和需要宽整数。
};

vector<PrimePrefix> lucyPrime(int n)
{
    // n 是非负上界；返回商集内的素数个数与素数和，按 x 升序；n=0 返回空表。
    assert(n >= 0);
    if (n == 0) return {};
    int rt = kthRoot(n, 2);
    vector<PrimePrefix> a;
    for (int l = 1; l <= n; )
```

```
    {
        int x = n / l;
        a.push_back({x, x - 1, (__int128)x * ((__int128)x + 1) / 2 - 1});
        if (x == 1) break;
        l = n / x + 1;
    }
    reverse(a.begin(), a.end());
    int m = a.size();
    auto id = [&](int x)
    {
        // x 必须属于 n 的商集；返回其在升序表中的下标。
        return x <= rt ? x - 1 : m - n / x;
    };
    for (int p = 2; p <= rt; p++)
    {
        if (a[id(p)].cnt == a[id(p - 1)].cnt) continue;
        auto pre = a[id(p - 1)];
        for (int i = m - 1; i >= 0 && a[i].x >= p * p; i--)
        {
            const auto &v = a[id(a[i].x / p)];
            a[i].cnt -= v.cnt - pre.cnt;
            a[i].sum -= (__int128)p * (v.sum - pre.sum);
        }
    }
    return a;
}
```

`lucyPrime(n)` 返回的向量大小为 $O(\sqrt{n})$，可以按 `x` 二分查询任意 $x = \lfloor n/i \rfloor$。以整除商下标代替哈希，筛选时间为 $O(n^{3/4}/\log n)$ $(n \to \infty)$，空间复杂度为 $O(\sqrt{n})$；`sum` 用 `__int128` 承载三角和，输入满足 $0 \le n \le 2^{63}-1$，实际规模还受 $O(\sqrt{n})$ 内存与筛选时间限制。$n = 0$ 返回空表。示例中 $\pi(20) = 8$、素数和为 77：

```
signed main()
{
    // 无参数；验证素数个数与素数和，返回 0。
    auto a = lucyPrime(20);
    auto it = find_if(a.begin(), a.end(), [](const PrimePrefix &v) { return v.x == 20; });
    assert(it != a.end() && it->cnt == 8 && it->sum == 77);
}
```

7.3.015 Powerful Number 筛

若每个素因子的指数都至少为 2，称正整数为 Powerful 数，约定 1 也在其中。每个 Powerful 数唯一写成 $x = a^2b^3$，其中 b 无平方因子：将每个指数唯一写成 $e = 2u + 3v$，其中 $v = e \bmod 2$，分别向 a, b 放入 p^u, p^v。因此可以用 7.3.002 莫比乌斯函数 的 `mobiusTable` 与 7.1.014 整数 k 次方根 的 `kthRoot` 枚举全部值。

Powerful Number 筛用于积性函数求和。选择易求前缀和的积性函数 g，满足 $f(p) = g(p)$，并令

$f = h * g$，其中 $*$ 是 Dirichlet 卷积。由 $f(1) = g(1) = h(1) = 1$，得 $h(p) = 0$，且对 $e \geq 2$ 有

$$h(p^e) = f(p^e) - \sum_{i=0}^{e-1} h(p^i) g(p^{e-i}).$$

于是 h 只在 Powerful 数处可能非零，交换求和顺序得到

$$\sum_{i=1}^{n} f(i) = \sum_{\substack{d \leq n \\ d \text{ 为 Powerful 数}}} h(d) G(\lfloor n/d \rfloor), \qquad G(m) = \sum_{i=1}^{m} g(i).$$

`powerfulSum` 按递增素因子递归枚举这些 d，由调用者给出 `hp(p,e)` 与 `gs(m)`；不需要生成或排序全部数。`powerfulNums` 则用于确实需要有序列表的构造题。

```
// 依赖 7.1.014 整数 k 次方根
// 依赖 7.2.001 埃氏筛、线性筛
// 依赖 7.3.002 莫比乌斯函数
vector<int> powerfulNums(int n)
{
    // n 是非负上界；返回不超过 n 的全部 Powerful 数，按升序排列；n>=1 时包含 1。
    assert(n >= 0);
    int lim = kthRoot(n, 3);
    auto mu = mobiusTable(lim);
    vector<int> ans;
    for (int b = 1; b <= lim; b++)
    {
        if (mu[b] == 0) continue;
        int base = (int)((__int128)b * b * b);
        int end = kthRoot(n / base, 2);
        for (int a = 1; a <= end; a++)
        {
            ans.push_back((int)((__int128)a * a * base));
        }
    }
    sort(ans.begin(), ans.end());
    return ans;
}

template <class T, class H, class G> T powerfulSum(int n, H hp, G gs)
{
    // n 是非负上界；hp(p,e) 返回积性函数 h 的素数幂值，要求 h(1)=1、h(p)=0；
    // gs(m) 返回 sum_{i=1..m} g(i)；返回 sum_{i=1..n} (h*g)(i)，T 承载求和结果。
    assert(n >= 0);
    if (n == 0) return T(0);
    auto p = Sieve(kthRoot(n, 2)).primes;
    auto dfs = [&](auto &&self, int x, int pos, T val) -> T
    {
        // x 是已选素数幂之积，pos 是下一素数下标，val=h(x)；返回该分支贡献。
        T ans = val * gs(n / x);
        for (int i = pos; i < (int)p.size() && p[i] <= (n / x) / p[i]; i++)
        {
            int q = p[i] * p[i];
            for (int e = 2; q <= n / x; e++)
            {
```

```
                ans += self(self, x * q, i + 1, val * hp(p[i], e));
                if (q > (n / x) / p[i]) break;
                q *= p[i];
            }
        }
        return ans;
    };
    return dfs(dfs, 1, 0, T(1));
}
```

例如 $f(n)=\sum_{d^2\mid n} d^2$，取 $g(n)=1$，则 $G(m)=m$，而 $h(p^e)$ 在 e 为偶数时等于 p^e、奇数时为零：

```
signed main()
{
    // 无参数；验证 Powerful 枚举与平方约数和，返回 0。
    auto hp = [](int p, int e) -> __int128
    {
        // p 是素数，e>=2 是指数；返回本例的 h(p^e)。
        if (e & 1) return 0;
        __int128 v = 1;
        for (int i = 0; i < e; i++)
        {
            v *= p;
        }
        return v;
    };
    auto gs = [](int m) -> __int128
    {
        // m 是前缀上界；本例 g(i)=1，返回 G(m)=m。
        return m;
    };
    assert(powerfulNums(100).size() == 14);
    assert(powerfulSum<__int128>(10, hp, gs) == 27);
}
```

令 $K=\Theta(\sqrt{n})$ 为不超过 n 的 Powerful 数数量。`powerfulNums` 枚举加排序需 $O(\sqrt[3]{n}\log n+K\log K)$ 时间、$O(\sqrt[3]{n}+K)$ 空间，开根的 $O(\log n)$ 因子来自精确二分。`powerfulSum` 的线性筛与枚举需 $O(\sqrt{n})$ 次基本操作，并调用 K 次 `gs`、$K-1$ 次 `hp`；回调成本必须另计。其空间为 $O(\sqrt{n})$（含素数筛），递归深度至多 $O(\log n)$。输入要求 $0\le n\le 2^{63}-1$，中间乘积与结果由 `T` 及回调负责承载；$n=0$ 返回空表或零。一般积性函数若难以选择 g，可比较 7.3.007 Min_25 筛；不能仅因枚举量少就忽略 `G` 的求值成本。

7.4 离散对数与剩余

7.4.001 BSGS / exBSGS

求最小非负 x 使 $a^x\equiv b\pmod m$。BSGS 要求 $\gcd(a,m)=1$：取 $n=\lceil\sqrt{m}\rceil$，哈希表存小步 $a^j\ (0\le j<n)$，再按大步枚举 i 匹配 $b\cdot a^{-ni}$，命中即 $x=ni+j$。exBSGS 去掉互质要求——反复约去 $\gcd(a,m)$ 并累计指数偏移，直到底数与模互质再转 BSGS。依赖 7.1.001 快速幂 的 `mulMod`、`powerMod`，

7.1.002 GCD、快速 GCD、LCM 的 `binaryGcd`，7.1.003 扩展欧几里得 的 `invMod` 及 7.1.014 整数 k 次方根 的 `kthRoot`。

```
// 依赖 7.1.002 GCD、快速 GCD、LCM
// 依赖 7.1.003 扩展欧几里得
// 依赖 7.1.014 整数 k 次方根
optional<long long> bsgs(long long a, long long b, long long m,
                         long long ord = -1)
{
    // 在 [0,ord) 内求最小解 x；ord=-1 时使用通用上界 m，无解返回空。
    assert(m > 0); // 调试检查，可删
    if (m == 1)
    {
        return 0;
    }
    a %= m;
    b %= m;
    if (a < 0)
    {
        a += m;
    }
    if (b < 0)
    {
        b += m;
    }
    assert(binaryGcd(a, m) == 1); // 调试检查，可删
    if (ord == -1)
    {
        ord = m;
    }
    assert(ord > 0); // 调试检查，可删
    long long n = kthRoot((unsigned long long)ord, 2);
    if (n <= (ord - 1) / n && n * n < ord)
    {
        n++;
    }
    unordered_map<long long, long long> baby;
    long long x = 1;
    for (long long j = 0; j < n; j++)
    {
        if (!baby.contains(x))
        {
            baby[x] = j;
        }
        x = mulMod(x, a, m);
    }
    long long step = powerMod(*invMod(a, m), n, m);
    x = b;
    long long rounds = ord / n + (ord % n != 0);
    for (long long i = 0; i < rounds; i++)
    {
        if (auto it = baby.find(x); it != baby.end())
```

```
        {
            long long j = it->second;
            if (j < ord && i <= (ord - 1 - j) / n)
            {
                return i * n + j;
            }
        }
        x = mulMod(x, step, m);
    }
    return nullopt;
}

optional<long long> exbsgs(long long a, long long b, long long m)
{
    // a 是底数，b 是右端值，m 是模数（不要求与 a 互质）；返回满足 a^x=b (mod m) 的最小非负 x，不存在时返回空。
    assert(m > 0); // 调试检查，可删
    a %= m;
    b %= m;
    if (a < 0)
    {
        a += m;
    }
    if (b < 0)
    {
        b += m;
    }
    if (m == 1 || b == 1 % m)
    {
        return 0;
    }
    long long k = 1;
    long long z = 0;
    while (true)
    {
        long long g = (long long)binaryGcd(a, m);
        if (g == 1)
        {
            break;
        }
        if (b == k)
        {
            return z;
        }
        if (b % g)
        {
            return nullopt;
        }
        b /= g;
        m /= g;
        z++;
        k = mulMod(k, a / g, m);
```

```
    }
    if (m == 1)
    {
        return z;
    }
    auto x = bsgs(a, mulMod(b, *invMod(k, m), m), m);
    if (!x)
    {
        return nullopt;
    }
    return *x + z;
}
```

`bsgs(a,b,m)` 要求 $\gcd(a,m)=1$，默认在 $[0,m)$ 搜索；已知底数阶不超过 `ord` 时传第四参，把时空降为 $O(\sqrt{ord})$。模 1 的最小解约定为 0。`exbsgs` 无互素要求，返回最小非负解或空。哈希操作为期望 $O(1)$；固定群阶可分解时用 7.4.002 Pohlig-Hellman。

7.4.002 Pohlig-Hellman

当模素数 p 的乘法群阶 $p-1=\prod q_i^{e_i}$ 只含较小质因子时，把离散对数分解到各质数幂阶子群求解：对每个 $q_i^{e_i}$，用 $g^{(p-1)/q_i^{e_i}}$、$h^{(p-1)/q_i^{e_i}}$ 在 $q_i^{e_i}$ 阶子群里跑 BSGS 得 $x \bmod q_i^{e_i}$，再逐步 CRT 合并成 $x \bmod (p-1)$。依赖 7.1.001 快速幂 的 `powerMod`、7.1.003 扩展欧几里得 的 `invMod` 与 7.4.001 BSGS / exBSGS 的 `bsgs`。

```
// 依赖 7.4.001 BSGS / exBSGS
optional<int> PH(int g, int h, int p)
{
    // g 是模素数 p 的原根，h 是非零剩余；返回最小非负 x 使 g^x=h，无解返回空。
    assert(p >= 2); // 调试检查，可删
    int left = p - 1;
    int x = 0;
    int curMod = 1;
    vector<int> fac;
    for (int q = 2; q <= left / q; q++)
    {
        if (left % q)
        {
            continue;
        }
        int pe = 1;
        while (left % q == 0)
        {
            left /= q;
            pe *= q;
        }
        fac.push_back(pe);
    }
    if (left > 1)
    {
        fac.push_back(left);
    }
```

```
    for (auto pe : fac)
    {
        int a = powerMod(g, (p - 1) / pe, p);
        int b = powerMod(h, (p - 1) / pe, p);
        auto y = bsgs(a, b, p, pe);
        if (!y)
        {
            return nullopt;
        }
        int t = (*y - x) % pe;
        if (t < 0)
        {
            t += pe;
        }
        auto inv = invMod(curMod % pe, pe);
        if (!inv)
        {
            return nullopt;
        }
        t = mulMod(t, *inv, pe);
        x += curMod * t;
        curMod *= pe;
        x %= curMod;
    }
    h %= p;
    if (h < 0)
    {
        h += p;
    }
    return powerMod(g, x, p) == h ? optional<int>(x) : nullopt;
}
```

`PH(g, h, p)`：p 素数、g 为模 p 原根、h 非零，返回最小非负 x 使 $g^x \equiv h$，无解返回 `nullopt`。时间约 $O(\sum_i \sqrt{q_i^{e_i}})$、空间 $O(\max_i \sqrt{q_i^{e_i}})$；$p-1$ 含大质因子时该质数幂上的 BSGS 仍可能慢。原根由 7.4.003 原根与全部原根筛 求。

7.4.003 原根与全部原根筛

模素数 p 的原根 g 是满足「对 $p-1$ 的每个不同质因子 q 都有 $g^{(p-1)/q} \neq 1$」的最小正整数；枚举 $g = 2, 3, \ldots$ 逐个验证即可（最小原根通常很小）。取到一个原根后，全部原根为 g^k（$\gcd(k, p-1) = 1$），共 $\varphi(p-1)$ 个。依赖 7.1.001 快速幂 的 `powerMod` 和 7.2.004 Pollard-Rho 的 `factor64`；64 位输入不能用试除法分解 $p-1$。

```
// 依赖 7.2.004 Pollard-Rho
long long primRoot(long long p)
{
    // p 是素数；返回模 p 的最小正原根，p=2 时返回 1。
    assert(p >= 2); // 调试检查，可删
    if (p == 2)
```

```
    {
        return 1;
    }
    long long n = p - 1;
    auto raw = factor64(n);
    raw.erase(unique(raw.begin(), raw.end()), raw.end());
    for (long long g = 2;; g++)
    {
        bool ok = true;
        for (auto d : raw)
        {
            if (powerMod(g, n / d, p) == 1)
            {
                ok = false;
            }
        }
        if (ok)
        {
            return g;
        }
    }
}

vector<long long> primRoots(long long p)
{
    // p 是素数；返回模 p 的全部原根并按数值升序排列。
    long long g = primRoot(p);
    long long n = p - 1;
    vector<long long> a;
    for (long long k = 1; k <= n; k++)
    {
        if (gcd(k, n) == 1)
        {
            a.push_back(powerMod(g, k, p));
        }
    }
    sort(a.begin(), a.end());
    return a;
}
```

`primRoot(p)` 返回模素数 p 的最小原根（$p=2$ 时为 1）；`primRoots(p)` 返回全部 $\varphi(p-1)$ 个原根（升序）。Pollard-Rho 分解 $p-1$ 的期望时间通常约为最小质因子四次根量级；若最小原根为 g，逐个验证为 $O(g\,\omega(p-1)\log p)$，实践中 g 通常很小；枚举并排序全部原根为 $O(p\log p)$。仅需离散对数时用 7.4.001 BSGS / exBSGS 或 7.4.002 Pohlig-Hellman。

7.4.004 二次剩余

素数模 p 下求 $x^2\equiv n$ 的一个根。$p=2$ 时根就是 $n \bmod 2$；奇素数先用 Euler 判据 $n^{(p-1)/2}$ 判定是否为二次剩余，$p\equiv 3 \pmod 4$ 时直接取 $x=n^{(p+1)/4}$，其余情况用 Tonelli-Shanks。写 $p-1=q\cdot 2^s$（q 奇），取一个非剩余 z，以 $c=z^q$ 逐步消去 $t=n^q$ 的 2-幂阶误差。返回 $\min(x,p-x)$，另一根为

$p - x$。依赖 7.1.001 快速幂 的 `mulMod`、`powerMod`。

```
optional<long long> sqrtMod(long long n, long long p)
{
    // n 是被开方数，p 是素数模；返回较小的平方根，非二次剩余返回空。
    assert(p >= 2); // 调试检查，可删
    n %= p;
    if (n < 0)
    {
        n += p;
    }
    if (n == 0)
    {
        return 0;
    }
    if (p == 2)
    {
        return n;
    }
    if (powerMod(n, (p - 1) / 2, p) != 1)
    {
        return nullopt;
    }
    if (p % 4 == 3)
    {
        long long x = powerMod(n, (p + 1) / 4, p);
        return min(x, p - x);
    }
    long long q = p - 1;
    long long s = 0;
    while (!(q & 1))
    {
        q >>= 1;
        s++;
    }
    long long z = 2;
    while (powerMod(z, (p - 1) / 2, p) != p - 1)
    {
        z++;
    }
    long long c = powerMod(z, q, p);
    long long x = powerMod(n, (q + 1) / 2, p);
    long long t = powerMod(n, q, p);
    long long m = s;
    while (t != 1)
    {
        long long i = 1;
        long long y = mulMod(t, t, p);
        while (y != 1)
        {
            y = mulMod(y, y, p);
            i++;
```

```
        }
        long long b = powerMod(c, 1ULL << (m - i - 1), p);
        x = mulMod(x, b, p);
        c = mulMod(b, b, p);
        t = mulMod(t, c, p);
        m = i;
    }
    return min(x, p - x);
}
```

`sqrtMod(n, p)`（p 为素数）返回较小的那个根或 `nullopt`；奇素数下另一根为 $p-x$，$n\equiv 0$ 时返回 0。给定二次非剩余后，Tonelli-Shanks 主体用 $O(\log^2 p)$ 次模乘；当前代码从 2 起试找非剩余，这一步最坏检查 $O(p)$ 个候选、每次需 $O(\log p)$ 次模乘，实践中通常很快。额外空间 $O(1)$。一般 k 次剩余用 7.4.005 K 次剩余。

7.4.005 K 次剩余

求素数模 p 下 $x^k\equiv a$ 的一组解。`kthRootBSGS` 取原根并离散对数，适合少量查询；默认的 `kthResidue` 先解掉与 $p-1$ 互素的指数部分，再对 $\gcd(k,p-1)$ 的每个质数幂逐层提根，适合批量查询。依赖 7.1.001 快速幂 的 `mulMod`、`powerMod` 与 7.1.003 扩展欧几里得 的 `invMod`；简单版本还依赖 7.4.001 BSGS / exBSGS 和 7.4.003 原根与全部原根筛。

```
// 依赖 7.4.003 原根与全部原根筛
optional<int> kthRootBSGS(int a, int k, int p)
{
    // a 是目标剩余，k 是非负指数，p 是素数；返回一组 x 使 x^k=a mod p，无解返回空。
    assert(k >= 0 && p >= 2); // 调试检查，可删
    a %= p;
    if (a < 0)
    {
        a += p;
    }
    if (k == 0)
    {
        return a == 1 ? optional<int>(0) : nullopt;
    }
    if (a == 0)
    {
        return 0;
    }
    int g = primRoot(p);
    auto b = bsgs(g, a, p);
    if (!b)
    {
        return nullopt;
    }
    int d = gcd(k, p - 1);
    if (*b % d)
    {
```

```
        return nullopt;
    }
    int mod = (p - 1) / d;
    if (mod == 1)
    {
        return 1;
    }
    auto inv = invMod(k / d, mod);
    if (!inv)
    {
        return nullopt;
    }
    int y = mulMod(*b / d, *inv, mod);
    return powerMod(g, y, p);
}
```

头文件随后实现 `ppRoot` 与默认 `kthResidue`：后者把指数约化到 $p-1$，检查 $a^{(p-1)/\gcd(k,p-1)}=1$，再逐质数幂提根。`kthResidue(a,k,p)`（p 素数、$k\geq 0$）返回任意一根或 `nullopt`；按 $0^0=1$ 约定，$k=0$ 时仅 $a\equiv 1 \pmod p$ 有解并返回 0。简单的 `kthRootBSGS` 为 $O(\sqrt{p})$ 期望时间空间；默认接口避免整群离散对数，单个质数幂步骤约需 $O(\sqrt{eq}\log p)$ 次模运算，其中 $q^e \mid \gcd(k,p-1)$。测试对小素数全部暴力比对，并覆盖 LC 的零指数语义。

7.4.006 扩展欧拉定理与欧拉降幂

扩展欧拉定理允许非互素底数在指数足够大时把指数化为 $(b \bmod \varphi(m))+\varphi(m)$。幂塔由下向上沿 φ 链递归，模数在 $O(\log m)$ 层后降到 1；关键是不能只保留指数余数，还要知道真实指数是否已经达到当前模数。`Enc` 分开保存余数与 `large` 标志，`towerCapped` 用带上限的乘幂判断真实塔值是否达到阈值；零底数则按 $0^0=1$ 单独处理奇偶层。模幂依赖 7.1.001 快速幂 的 `powerMod`。

```
class PowerTower
{
    struct Enc
    {
        long long rem = 0; // 对当前模数的余数
        bool large = false; // 真实值是否不小于当前模数
    };

    static long long phi(long long n)
    {
        // 返回正整数 n 的 Euler phi 值
        long long ans = n;
        for (long long p = 2; p <= n / p; p++)
        {
            if (n % p)
            {
                continue;
            }
            while (n % p == 0)
            {
```

```
            n /= p;
        }
        ans = ans / p * (p - 1);
    }
    if (n > 1)
    {
        ans = ans / n * (n - 1);
    }
    return ans;
}

static long long mulCap(long long a, long long b, long long limit)
{
    // 返回 min(a*b,limit)，参数均非负
    if (!a || !b)
    {
        return 0;
    }
    if (a > (limit - 1) / b)
    {
        return limit;
    }
    return min(limit, a * b);
}

static long long powerCapped(long long base, long long exponent, long long limit)
{
    // 返回 min(base^exponent,limit)
    long long result = 1;
    while (exponent && result < limit)
    {
        if (exponent & 1)
        {
            result = mulCap(result, base, limit);
        }
        exponent >>= 1;
        if (exponent)
        {
            base = mulCap(base, base, limit);
        }
    }
    return result;
}

static long long towerCapped(long long base, long long height,
                             long long limit)
{
    // 返回 height 层幂塔与 limit 的较小值，约定 0^0=1
    if (height == 0)
    {
        return min(1LL, limit);
    }
```

```
        if (base == 0)
        {
            long long v = height & 1 ? 0 : 1;
            return min(v, limit);
        }
        if (base == 1)
        {
            return min(1LL, limit);
        }
        if (limit <= 1)
        {
            return limit;
        }

        long long need = 0;
        long long v = 1;
        while (v < limit)
        {
            v = mulCap(v, base, limit);
            need++;
        }
        long long e = towerCapped(base, height - 1, need);
        if (e >= need)
        {
            return limit;
        }
        return powerCapped(base, e, limit);
    }

    static Enc evaluate(long long base, long long height, long long mod)
    {
        // 返回幂塔对 mod 的余数，并独立记录真实值是否越过 mod
        if (height == 0)
        {
            return {1 % mod, 1 >= mod};
        }
        if (base == 0)
        {
            long long v = height & 1 ? 0 : 1;
            return {v % mod, v >= mod};
        }
        if (mod == 1)
        {
            return {0, towerCapped(base, height, 1) == 1};
        }
        long long ph = phi(mod);
        auto e = evaluate(base, height - 1, ph);
        unsigned long long exponent = (unsigned long long)e.rem;
        if (e.large)
        {
            exponent += (unsigned long long)ph;
        }
```

```
        long long rem = powerMod(base, exponent, mod);
        bool large = towerCapped(base, height, mod) == mod;
        return {rem, large};
    }

  public:
    // 计算 height 层 base 幂塔对 mod 的余数。
    static long long eval(long long base, long long height, long long mod)
    {
        // 返回 height 层 base 幂塔对 mod 的余数
        assert(base >= 0 && height >= 0 && mod > 0); // 调试检查，可删
        return evaluate(base, height, mod).rem;
    }
};
```

`PowerTower::eval(base,height,mod)` 要求三者分别非负、非负、正，$height = 0$ 视作 1，并约定 $0^0 = 1$；因此零底数时奇数层为 0、正偶数层为 1。余数与“真实值是否达到模数”分成 `Enc` 两个字段，不再用 `residue+mod` 塞进可能溢出的 `long long`。递归层数不超过 $\min(height, O(\log mod))$，每层试除求 φ 最坏 $O(\sqrt{mod})$，栈 $O(\log mod)$。

7.4.007 Carmichael 函数

Carmichael 函数 $\lambda(n)$ 是模 n 既约剩余类群中所有元素阶的最小公倍数，也是对任意 $\gcd(a, n) = 1$ 都满足 $a^m \equiv 1 \pmod{n}$ 的最小正指数 m（它整除 $\varphi(n)$，往往更小）。分解 $n = \prod p^k$ 后逐个素数幂取值再合并：奇素数幂与 $2, 4$ 处 $\lambda = \varphi$，而 $\lambda(2^k) = 2^{k-2}\ (k \geq 3)$；最终 $\lambda(n) = \mathrm{lcm}$ 各素数幂之值。

```
long long carmichael(long long n)
{
    // n>=1；试除分解后返回 Carmichael 函数 lambda(n)。
    assert(n >= 1); // 调试检查，可删
    long long ans = 1;
    for (long long p = 2; p <= n / p; p++)
    {
        if (n % p)
        {
            continue;
        }
        long long q = 1;
        int k = 0;
        while (n % p == 0)
        {
            n /= p;
            q *= p;
            k++;
        }
        long long z = q / p * (p - 1);
        if (p == 2 && k >= 3)
        {
            z /= 2;
```

```
        }
        ans = lcm(ans, z);
    }
    if (n > 1)
    {
        ans = lcm(ans, n - 1);
    }
    return ans;
}
```

`carmichael(n)`（$n \geq 1$）返回 $\lambda(n)$，试除分解 $O(\sqrt{n})$ 时间、$O(1)$ 额外空间；各质数幂及逐步最小公倍数均不超过原 n，故正 `long long` 输入下不会溢出。验证：$n = 8, 15, 16, 561$ 依次得 $2, 4, 4, 80$（$\lambda(561) = \operatorname{lcm}(\lambda(3), \lambda(11), \lambda(17)) = \operatorname{lcm}(2, 10, 16) = 80$）。

7.4.008 固定模批量离散对数

素数 p 与原根 g 固定、查询 a_i 很多时，不应为每个询问各做一次 BSGS。取 $B \approx \sqrt{(p-1)/Q}$，预存 g^{Bx}；随后对 $0 \leq y < B$ 共享扫描全部查询，若 $a_i g^y = g^{Bx}$，则答案为 $Bx - y \pmod{p-1}$。表大小与扫描次数分别约为 p/B、QB，平衡后总期望时间和空间量级均为 $O(\sqrt{pQ})$。下面的开放寻址表把非零域元素和指数压在同一个 64 位槽中，避免 `unordered_map` 在大询问下的节点开销；它与 1.4.004 防 hack 哈希用途不同。

```
// 依赖 7.1.001 快速幂
struct DLogHash
{
    // tab 是开放寻址表，非空槽高 32 位存域元素、低 32 位存指数，0 表示空槽（域元素非零）。
    vector<unsigned long long> tab;

    DLogHash(int n) : tab(max(3LL, n * 10 / 7 + 3))
    {
        // n 是预计插入的键数；表长取约 1.43n，装载率不超过 0.7。
    }

    unsigned long long hash(unsigned long long x) const
    {
        // x 是键；返回 splitmix64 混合后的散列值。
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }

    void insert(int key, int val)
    {
        // key 是 [1,p) 内的域元素，val 是正指数；同一键只保留最小的指数。
        int p = hash(key) % tab.size();
        while (tab[p] && (int)(tab[p] >> 32) != key)
        {
            if (++p == (int)tab.size())
```

```
            {
                p = 0;
            }
        }
        if (!tab[p] || val < (int)(tab[p] & 0xffffffffULL))
        {
            tab[p] = ((unsigned long long)key << 32) | val;
        }
    }

    int find(int key) const
    {
        // 返回 key 对应的最小指数，不存在时返回 -1。
        int p = hash(key) % tab.size();
        while (tab[p])
        {
            if ((int)(tab[p] >> 32) == key)
            {
                return (int)(tab[p] & 0xffffffffULL);
            }
            if (++p == (int)tab.size())
            {
                p = 0;
            }
        }
        return -1;
    }
};

vector<int> batchDLog(int p, int g, const vector<int> &a)
{
    // p 是不超过 1e9 的素数，g 是模 p 的原根，a[i] 在 [1,p) 内；
    // 返回 ans[i] 为满足 g^x = a[i] (mod p) 的最小 x，范围 [0,p-2]。
    assert(p >= 2 && 1 <= g && g < p); // 调试检查，可删。
    int qn = a.size();
    vector<int> ans(qn, -1);
    if (qn == 0)
    {
        return ans;
    }
    if (p == 2)
    {
        fill(ans.begin(), ans.end(), 0);
        return ans;
    }
    // 巨步表存 g^{Bx}（1<=x<=lim），小步对所有查询共享 g^y（0<=y<B）。
    int B = sqrtl((long double)(p - 1) / qn) + 1;
    int lim = (p - 2 + B - 1) / B;
    DLogHash tab(lim);
    int step = powerMod(g, B, p), cur = step;
    for (int x = 1; x <= lim; x++)
    {
```

```
        tab.insert(cur, x);
        cur = cur * step % p;
    }
    int gy = 1; // gy 是 g^y。
    for (int y = 0; y < B; y++)
    {
        for (int i = 0; i < qn; i++)
        {
            assert(1 <= a[i] && a[i] < p); // 调试检查，可删。
            if (a[i] == 1)
            {
                ans[i] = 0;
                continue;
            }
            // a[i]*g^y = g^{Bx} 说明 a[i] = g^{Bx-y}。
            int x = tab.find(a[i] * gy % p);
            if (x != -1)
            {
                int e = ((x * B - y) % (p - 1) + p - 1) % (p - 1);
                if (ans[i] == -1 || e < ans[i])
                {
                    ans[i] = e;
                }
            }
        }
        gy = gy * g % p;
    }
    return ans;
}
```

`batchDLog(p,g,a)` 要求 $p \le 10^9$ 为素数、g 为原根且每个查询在 $[1,p)$，返回唯一的 $x_i \in [0,p-2]$ 使 $g^{x_i} \equiv a_i \pmod p$；空查询直接返回空。$Q$ 极大时表可能占数百 MiB，应按题目内存限制评估；单次或少量查询直接用 7.4.001 BSGS / exBSGS，群阶光滑时用 7.4.002 Pohlig-Hellman。

下面在模 17、原根 3 下检查单位元及一批可直接快速幂复核的指数：

```
signed main()
{
    assert(batchDLog(17, 3, {1, 3, 9, 10, 16}) ==
           vector<int>({0, 1, 2, 3, 8}));
}
```

7.4.009 Hensel 简单根提升

Hensel 引理把多项式 $f \in \mathbb{Z}[x]$ 模素数 p 的单根提升为模 p^k 的根。设 $f(x_0) \equiv 0 \pmod q$（$q = p^i$）且 $f'(x_0) \not\equiv 0 \pmod p$，令 $x_1 = x_0 + tq$，由 Taylor 展开

$$f(x_0 + tq) \equiv f(x_0) + tq\, f'(x_0) \pmod{q^2},$$

在模 qp 下要求 $\frac{f(x_0)}{q}+t\,f'(x_0)\equiv 0\pmod p$，于是

$$t\equiv -\frac{f(x_0)}{q}\cdot f'(x_0)^{-1}\pmod p.$$

$f'(x_1)\equiv f'(x_0)\pmod p$，所以逆元只需在模 p 下求一次；每轮把模数乘 p，提升后的根在模 p^k 下唯一。

```cpp
// 依赖 7.2.003 Miller-Rabin
optional<int> hensel(const vector<int> &a, int x, int p, int k)
{
    // a 是整系数多项式 f，低次在前；p 是素数，x 是 f 模 p 的根；p^k 不超过 LLONG_MAX。
    // 当 x 是单根（f'(x) 模 p 非零）时返回 f 模 p^k 的唯一提升根（在 [0,p^k) 内），否则返回空。
    assert(isPrime(p) && k >= 1); // 调试检查，可删。
    auto eval = [&](int y, int mod, bool der)
    {
        // 返回 f(y) 或 f'(y)（der 为真）模 mod 的非负值，秦九韶求值。
        __int128 s = 0;
        for (int i = (int)a.size() - 1; i >= der; i--)
        {
            s = (s * y + (__int128)a[i] * (der ? i : 1)) % mod;
        }
        return (int)((s + mod) % mod);
    };
    x = (x % p + p) % p;
    int d = eval(x, p, true);
    if (eval(x, p, false) || !d)
    {
        return nullopt;
    }
    // 牛顿迭代：若 f(x)≡0 (mod q)，则 x - f(x)/f'(x) 是模 qp 的根，且 f'(x) 的逆只需模 p。
    int inv = pow64(d, p - 2, p), q = p;
    for (int i = 1; i < k; i++)
    {
        assert((__int128)q * p <= LLONG_MAX); // 调试检查，可删。
        int nq = q * p, f = eval(x, nq, false);
        int t = (__int128)(p - f / q) * inv % p;
        x += t * q;
        q = nq;
    }
    return x;
}
```

`hensel(a,x,p,k)` 中 `a` 是系数（低次在前），`p` 为素数，`x` 是模 p 的根，p^k 须不超过 `LLONG_MAX`；返回 $[0,p^k)$ 内的提升根。若 x 不是模 p 的根，或 $f'(x)\equiv 0\pmod p$（重根，可能无解也可能有多个提升），返回空。求值用 `__int128` 防止乘积溢出。时间 $O(k\deg f)$，额外空间 $O(1)$。素性判定与模幂复用 7.2.003 Miller-Rabin 的 `isPrime` 与 `pow64`。

模 p 的平方根本身用 7.4.004 二次剩余 求。下面把 $x^2\equiv 2\pmod 7$ 的根 3 提升到模 $7^4=2401$：

```cpp
signed main()
{
```

```
    auto r = hensel({-2, 0, 1}, 3, 7, 4);
    assert(r && *r == 2166 && (*r * *r - 2) % 2401 == 0);
    assert(!hensel({0, 0, 1}, 0, 7, 3)); // f'(0)=0，不是单根。
}
```

7.5 类欧几里得

7.5.001 `floor_sum`

计算 $\sum_{i=0}^{n-1}\left\lfloor\frac{ai+b}{m}\right\rfloor$，把 $O(n)$ 枚举降到对数级。做法同 AtCoder Library：先扣除 a,b 中 $\geq m$ 的整商贡献，再令 $y=an+b$，利用格点三角形关于直线的对称把问题交换 a 与 m 递归，参数如辗转相除般缩小，直到 $an+b<m$ 终止；开头两段先把 $a<0$、$b<0$ 规约为非负。

```
__int128 floorSum(long long n, long long m, long long a, long long b)
{
    // n 是项数，m 是正除数，a、b 是线性系数；返回取整和，要求精确中间量与答案可由 __int128 表示。
    assert(n >= 0 && m > 0); // 调试检查，可删
    __int128 N = n;
    __int128 M = m;
    __int128 A = a;
    __int128 B = b;
    __int128 ans = 0;
    auto divFloor = [](__int128 x, __int128 y)
    {
        __int128 q = x / y;
        __int128 r = x % y;
        return q - (r < 0);
    };
    __int128 q = divFloor(A, M);
    ans += N * (N - 1) / 2 * q;
    A -= q * M;
    q = divFloor(B, M);
    ans += N * q;
    B -= q * M;
    while (true)
    {
        if (A >= M)
        {
            ans += N * (N - 1) / 2 * (A / M);
            A %= M;
        }
        if (B >= M)
        {
            ans += N * (B / M);
            B %= M;
        }
        __int128 y = A * N + B;
        if (y < M)
        {
```

```
            break;
        }
        N = y / M;
        B = y % M;
        swap(A, M);
    }
    return ans;
}
```

`floorSum(n,m,a,b)` 要求 $m>0$、$n\ge 0$，a,b 可取完整 `long long`；负数规约、三角和与 $AN+B$ 均用 `__int128` 计算，但接口仍要求所有精确中间量及最终取整和落在 `__int128` 范围内，极端的 $n^2|a|/m$ 可能超界。复杂度 $O(\log\max(|a|,m))$ 时间、$O(1)$ 空间。需要聚合更多格点量时用 7.5.003 万能欧几里得；求 $(ai+b) \bmod m$ 的最小值见 7.5.004 Min of Mod of Linear。

7.5.002 类欧几里得格点统计

基础线性取整和 $\sum\lfloor\frac{ai+b}{m}\rfloor$ 由 7.5.001 floor_sum 的 `floorSum` 提供，本条不重复实现。若要同时聚合横步数、纵步数及其交叉项等格点统计量（一次矩、覆盖面积等），改用 7.5.003 万能欧几里得 的 `floorPath`：把每种待累加量定义成一个幺半群元素，一次遍历即可得到全部统计。

> 备注：原资料本条重复粘贴了一份与 7.5.001 floor_sum 完全相同的 `floorSum`（且只算和、并未真的算矩），属重复代码；同名同签名与 7.5.001 冲突，故删去，改按上面的分工引用。

7.5.003 万能欧几里得

万能欧几里得把直线 $y=\frac{pi+r}{q}$ 下方的取整阶梯看成一串操作：i 从 1 走到 $length$，每当 $\lfloor\frac{pi+r}{q}\rfloor$ 增加 1 就写一个纵步 U，每到一个整数 i 就写一个横步 R。要求的统计量只要能写成某个幺半群上的乘积 $\prod(\text{步})$，就能像类欧几里得一样递归：$p\ge q$ 时每个 R 前必然先有 $\lfloor p/q\rfloor$ 个 U，把 R 换成 $U^{\lfloor p/q\rfloor}R$ 后令 $p\leftarrow p \bmod q$；$p<q$ 时交换 U、R 的角色（相当于把图形沿 $y=x$ 翻转），问题规模像辗转相除一样下降。这里的 `Euclid` 以 (x,y,xy) 记录横步数、纵步数以及“每个横步之前已有的纵步数之和”，合并时交叉项补 $y\cdot x'$，于是 $xy=\sum_{i=1}^{length}\lfloor\frac{pi+r}{q}\rfloor$。

`floorMoment` 求更一般的矩 $T_{p,q}=\sum_{i=0}^{n} i^p f(i)^q$，$f(i)=\lfloor\frac{ai+b}{c}\rfloor$，一次给出所有 $p+q\le d$ 的表：

- $a=0$ 时 f 是常数，$T_{p,q}=S_p(n)\,(b/c)^q$，其中 $S_p(x)=\sum_{i=0}^{x} i^p$ 是 $p+1$ 次多项式，由 $\sum_{k\le p}\binom{p+1}{k}S_k(x)=(x+1)^{p+1}$ 逐个求出。
- $a\ge c$ 或 $b\ge c$ 时 $f(i)=\lfloor a/c\rfloor i+\lfloor b/c\rfloor+g(i)$，$g$ 用 $(a \bmod c, b \bmod c)$；把 f^q 按三项式展开，归约到 g 的表。
- $a,b<c$ 时令 $h=f(n)$，利用 $f(i)^q=\sum_{j<f(i)}((j+1)^q-j^q)$ 以及 $f(i)>j \iff i>g(j)$，$g(j)=\lfloor\frac{cj+c-b-1}{a}\rfloor$，交换求和次序得

$$T_{p,q}=h^q S_p(n)-\sum_{u<q}\binom{q}{u}\sum_{r}[x^r]S_p\cdot\sum_{j=0}^{h-1}j^u g(j)^r,$$

右边是参数为 $(h-1,c,c-b-1,a)$ 的同类表，参数像辗转相除一样变小。

```cpp
// 依赖 1.1.008 静态取模类
struct Euclid
{
    __int128 x = 0;  // 已拼接路径的横步总数
    __int128 y = 0;  // 已拼接路径的纵步总数
    __int128 xy = 0; // 每个横步之前的纵步数之和

    Euclid operator+(const Euclid &other) const
    {
        // 把 other 路径接在当前路径后，返回合并结果。
        return {x + other.x, y + other.y,
                xy + other.xy + y * other.x};
    }
};

inline Euclid monoidPower(Euclid value, long long exponent)
{
    // 返回 value 在幺半群 + 下的 exponent 次幂。
    assert(exponent >= 0); // 调试检查，可删
    Euclid res;
    while (exponent)
    {
        if (exponent & 1)
        {
            res = res + value;
        }
        value = value + value;
        exponent >>= 1;
    }
    return res;
}

inline Euclid floorPath(long long p, long long q, long long r,
                        long long length, Euclid up, Euclid right)
{
    // p、q、r 定义取整直线，length 是横坐标上界；up、right 是纵步与横步元素，返回整段聚合。
    assert(p >= 0 && q > 0 && 0 <= r && r < q && length >= 0); // 调试检查，可删
    if (!length)
    {
        return {};
    }
    if (p >= q)
    {
        return floorPath(p % q, q, r, length, up,
                         monoidPower(up, p / q) + right);
    }
    long long height = (long long)(((__int128)length * p + r) / q);
    if (!height)
    {
        return monoidPower(right, length);
    }
    long long first = (long long)(((__int128)q - r - 1) / p);
```

```
    long long suffix = (long long)((__int128)length -
                                   ((__int128)q * height - r - 1) / p);
    return monoidPower(right, first) + up +
           floorPath(q, p, (q - r - 1) % p, height - 1,
                     right, up) +
           monoidPower(right, suffix);
}

using Z7 = MInt<1000000007>; // 矩表固定模 1e9+7，换模数只改这里。
constexpr int floorDeg = 10;  // floorDeg 是 p+q 的上界。
// FloorTable[p][q] 存 sum i^p floor(...)^q，只有 p+q<=d 的项有效。
using FloorTable = array<array<Z7, floorDeg + 1>, floorDeg + 1>;

struct FloorPre
{
    // C[n][k] 是组合数；S[p] 是 S_p(x)=sum_{i=0}^{x} i^p 的系数（低次在前，0^0 记为 1）。
    array<array<Z7, floorDeg + 2>, floorDeg + 2> C;
    array<vector<Z7>, floorDeg + 1> S;

    FloorPre()
    {
        // 无参数；由 sum_k C(p+1,k) S_k(x) = (x+1)^{p+1} 逐个求出幂和多项式。
        for (int n = 0; n <= floorDeg + 1; n++)
        {
            C[n][0] = 1;
            for (int k = 1; k <= n; k++)
            {
                C[n][k] = C[n - 1][k - 1] + C[n - 1][k];
            }
        }
        for (int p = 0; p <= floorDeg; p++)
        {
            S[p].assign(p + 2, 0);
            for (int j = 0; j <= p + 1; j++)
            {
                S[p][j] = C[p + 1][j];
            }
            for (int k = 0; k < p; k++)
            {
                for (int j = 0; j < (int)S[k].size(); j++)
                {
                    S[p][j] -= C[p + 1][k] * S[k][j];
                }
            }
            Z7 iv = Z7(p + 1).inv();
            for (auto &x : S[p])
            {
                x *= iv;
            }
        }
    }

```

```
    Z7 eval(int p, int x) const
    {
        // 返回 S_p(x)，x>=0。
        Z7 ans = 0;
        for (int i = (int)S[p].size() - 1; i >= 0; i--)
        {
            ans = ans * x + S[p][i];
        }
        return ans;
    }
};

FloorTable floorMoment(int n, int a, int b, int c, int d)
{
    // n>=0，a,b>=0，c>0，0<=d<=floorDeg；记 f(i)=floor((a*i+b)/c)，
    // 返回表 T，T[p][q] = sum_{i=0}^{n} i^p f(i)^q（p+q<=d），均模 1e9+7。
    assert(n >= 0 && a >= 0 && b >= 0 && c > 0 && 0 <= d && d <= floorDeg); // 调试检查，可删。
    static const FloorPre pre;
    FloorTable ans{};
    if (!a)
    {
        // f 恒为 b/c。
        Z7 v = b / c;
        for (int p = 0; p <= d; p++)
        {
            for (int q = 0; p + q <= d; q++)
            {
                ans[p][q] = pre.eval(p, n) * v.pow(q);
            }
        }
        return ans;
    }
    int da = a / c, db = b / c;
    if (da || db)
    {
        // f(i) = da*i + db + g(i)，g 用 (a%c，b%c)；按三项式展开 f^q。
        auto sub = floorMoment(n, a % c, b % c, c, d);
        vector<Z7> pa(d + 1, 1), pb(d + 1, 1);
        for (int i = 1; i <= d; i++)
        {
            pa[i] = pa[i - 1] * da;
            pb[i] = pb[i - 1] * db;
        }
        for (int p = 0; p <= d; p++)
        {
            for (int q = 0; p + q <= d; q++)
            {
                for (int j = 0; j <= q; j++)
                {
                    for (int t = 0; t + j <= q; t++)
                    {
                        // g^j (da*i)^t db^{q-j-t} 的系数是 C(q,j) C(q-j,t)。
```

```
                        ans[p][q] += pre.C[q][j] * pre.C[q - j][t] * pa[t] * pb[q - j - t] *
↪ sub[p + t][j];
                    }
                }
            }
        }
        return ans;
    }
    // a<c 且 b<c：交换坐标轴。h=f(n)，f(i)>j 当且仅当 i>g(j)，g(j)=floor((c*j+c-b-1)/a)。
    int h = (int)(((__int128)a * n + b) / c);
    for (int p = 0; p <= d; p++)
    {
        ans[p][0] = pre.eval(p, n);
    }
    if (!h)
    {
        return ans;
    }
    // f(i)^q = sum_{j<f(i)} ((j+1)^q - j^q)，交换求和后 i 取 (g(j), n]。
    auto sub = floorMoment(h - 1, c, c - b - 1, a, d);
    for (int p = 0; p <= d; p++)
    {
        for (int q = 1; p + q <= d; q++)
        {
            Z7 w = ans[p][0] * Z7(h).pow(q);
            for (int u = 0; u < q; u++)
            {
                for (int r = 0; r <= p + 1; r++)
                {
                    w -= pre.C[q][u] * pre.S[p][r] * sub[u][r];
                }
            }
            ans[p][q] = w;
        }
    }
    return ans;
}

vector<int> floorMoments(const vector<array<int, 6>> &qs)
{
    // qs[i] 的六维依次是 n、a、b、c、p、q（p+q<=floorDeg）；
    // 返回各询问的 sum_{i=0}^{n} i^p floor((a*i+b)/c)^q 模 1e9+7。
    vector<int> ans;
    for (auto [n, a, b, c, p, q] : qs)
    {
        assert(p >= 0 && q >= 0 && p + q <= floorDeg); // 调试检查，可删。
        ans.push_back(floorMoment(n, a, b, c, p + q)[p][q].val());
    }
    return ans;
}
```

`floorPath(p,q,r,length,up,right)` 要求 $p \geq 0$、$q > 0$、$0 \leq r < q$，返回 $1 \leq i \leq length$ 的阶梯

路径聚合值；传 `up={0,1,0}`、`right={1,0,0}` 时 `xy` 就是 $\sum_{i=1}^{length}\lfloor\frac{pi+r}{q}\rfloor$，三个字段须落在 `__int128` 范围内。递归 $O(\log\max(p,q))$ 层，每层用 $O(\log length)$ 次合并做快速幂。换别的统计量只需改 `Euclid` 的字段与 `operator+`，保持结合律即可。

`floorMoment(n,a,b,c,d)` 要求 $n\ge 0$、$a,b\ge 0$、$c>0$、$d\le 10$，返回 `FloorTable`，只有 $p+q\le d$ 的项有效；`floorMoments(qs)` 逐个回答 `{n,a,b,c,p,q}` 询问。模数固定为 10^9+7（改 `Z7` 即可换素数模），依赖 1.1.008 静态取模类。递归 $O(\log\max(a,c))$ 层，每层 $O(d^4)$。只求 $\sum\lfloor\frac{ai+b}{c}\rfloor$ 时用更短的 7.5.001 floor_sum。下面先用小例子核对，再给一个大参数的实测值：

```
signed main()
{
    auto e = floorPath(4, 11, 3, 5, {0, 1, 0}, {1, 0, 0}); // 依次为 floor((4i+3)/11), i=1..5: 0,1,1,1,2。
    assert(e.x == 5 && e.y == 2 && e.xy == 5);
    auto t = floorMoment(10, 3, 2, 5, 3);
    assert(t[1][2] == 1069 && t[0][0] == 11);
    auto v = floorMoments({{100, 7, 3, 4, 2, 3}, {1000000000, 123456789, 987654321, 1000000007, 3, 4}});
    assert(v[0] == 279549271);
    cout << v[1] << '\n';
    // 期望输出：738884790
}
```

7.5.004 `Min of Mod of Linear`

在 $0\le i<n$ 中求 $(ai+b)\bmod m$ 的最小值。余数 $r_i=(ai+b)-m\lfloor\frac{ai+b}{m}\rfloor$，而 “$r_i\le x$ 的下标个数” 等于两次 `floorSum` 之差且关于 x 单调，于是二分最小可行的 x 即为答案。依赖 7.5.001 floor_sum 的 `floorSum`。

```
int minModLinear(int n, int m, int a, int b)
{
    // n、m 为正，a、b 是整数；返回 0<=i<n 时 (a*i+b) mod m 的最小值。
    assert(n > 0 && m > 0); // 调试检查，可删
    a %= m;
    b %= m;
    if (a < 0)
    {
        a += m;
    }
    if (b < 0)
    {
        b += m;
    }
    auto count = [&](int x)
    {
        // x 是余数上界；返回余数落在 [0,x] 的下标数量。
        return floorSum(n, m, a, b) - floorSum(n, m, a, b - x - 1);
    };
    int l = -1;
```

```
    int r = m - 1;
    while (r - l > 1)
    {
        int x = midpoint(l, r);
        if (count(x))
        {
            r = x;
        }
        else
        {
            l = x;
        }
    }
    return r;
}
```

`minModLinear(n,m,a,b)` 要求 $n,m>0$，a,b 可负，返回值在 $[0,m)$。计数调用 7.5.001 floor_sum 的宽返回值，二分中点使用 `midpoint`；a,b 先规约到 $[0,m)$，故时间复杂度为 $O(\log^2 m)$，额外空间 $O(1)$。固定例 $n=5,m=11,a=4,b=3$ 的余数为 $3,7,0,4,8$，答案为 0。

7.5.005 Stern-Brocot Tree / Farey 序列

在正有理数的 Stern-Brocot 表示与既约分数间互转，并枚举 Farey 序列。Stern-Brocot 树以左右邻分数的 mediant 导航正有理数，同向的连续步用连分数商压成一段（`L`/`R` + 次数）；`farey` 用相邻分数行列式恒为 1 的递推 $\frac{p}{q},\frac{r}{s}\to k=\left\lfloor\frac{n+q}{s}\right\rfloor$ 逐个生成 n 阶序列。

```
vector<pair<char, long long>> sternPath(long long p, long long q)
{
    // p、q 是既约正分数的分子与分母；返回该分数在 Stern-Brocot 树中的路径编码（每段为 L/R 及重复次数）。
    assert(p > 0 && q > 0 && gcd(p, q) == 1); // 调试检查，可删
    vector<pair<char, long long>> a;
    while (p != q)
    {
        char c;
        long long k;
        if (p < q)
        {
            c = 'L';
            k = (q - 1) / p;
            q -= k * p;
        }
        else
        {
            c = 'R';
            k = (p - 1) / q;
            p -= k * q;
        }
        a.push_back({c, k});
    }
```

```cpp
    return a;
}

pair<long long, long long> sternValue(const vector<pair<char, long long>> &a)
{
    // a 是合法路径编码，且结果可由 long long 表示；返回对应的既约正分数 {p,q}。
    long long p = 1;
    long long q = 1;
    for (auto it = a.rbegin(); it != a.rend(); it++)
    {
        auto [c, k] = *it;
        assert((c == 'L' || c == 'R') && k > 0); // 调试检查，可删
        if (c == 'L')
        {
            assert(k <= (numeric_limits<long long>::max() - q) / p); // 调试检查，可删
            q += k * p;
        }
        else
        {
            assert(k <= (numeric_limits<long long>::max() - p) / q); // 调试检查，可删
            p += k * q;
        }
    }
    return {p, q};
}

vector<pair<int, int>> farey(int n)
{
    // n 是阶数；返回 n 阶 Farey 序列 F_n，分数按值递增排列。
    assert(n >= 1); // 调试检查，可删
    vector<pair<int, int>> a{{0, 1}};
    int p = 0;
    int q = 1;
    int r = 1;
    int s = n;
    while (r <= n)
    {
        a.push_back({r, s});
        __int128 k = ((__int128)n + q) / s;
        int x = (int)(k * r - p);
        int y = (int)(k * s - q);
        p = r;
        q = s;
        r = x;
        s = y;
    }
    return a;
}
```

`sternPath` 要求 $p, q > 0$ 且 $\gcd(p, q) = 1$；`sternValue` 逆变换回既约分数；`farey(n)` 返回 F_n。路径编解码 $O(\log(p+q))$ 段与空间，Farey 序列 $O(|F_n|)$ 时间空间。

`ratApprox(n,x,y)` 返回分子、分母均不超过 `n` 的最紧下界和上界；不存在的方向分别用 `{0,1}`、`{1,0}` 表示。要求 `n>0,x>=0,y>0`，中间运算使用 `__int128`。

7.5.006 Pell 方程

求非平方正整数 d 的 Pell 方程 $x^2 - dy^2 = 1$ 的最小正整数解（基本解）。展开 $\sqrt{d}$ 的周期连分数并同步递推收敛分数 $\frac{p}{q}$，第一个满足 $p^2 - dq^2 = 1$ 的收敛分数即基本解；分子分母用 `__int128` 递推，以便在越过 `long long` 前及时判溢出。平方根依赖 7.1.014 整数 k 次方根 的 `kthRoot`。

```
optional<pair<long long, long long>> pell(long long d)
{
    // d 是正整数；返回 x^2-d*y^2=1 的基本解，d 为平方数或解越界时返回空。
    assert(d > 0); // 调试检查，可删
    long long a0 = kthRoot((unsigned long long)d, 2);
    if ((__int128)a0 * a0 == d)
    {
        return nullopt;
    }
    __int128 D = d;
    __int128 m = 0;
    __int128 q = 1;
    __int128 a = a0;
    __int128 p0 = 1;
    __int128 p1 = a;
    __int128 q0 = 0;
    __int128 q1 = 1;
    constexpr unsigned __int128 IMAX = ((unsigned __int128)1 << 127) - 1;
    while (true)
    {
        auto uq1 = (unsigned __int128)q1;
        if (q1 && uq1 > IMAX / (unsigned __int128)D / uq1)
        {
            return nullopt;
        }
        if (p1 * p1 - D * q1 * q1 == 1)
        {
            break;
        }
        m = q * a - m;
        q = (D - m * m) / q;
        a = (a0 + m) / q;
        __int128 p2 = a * p1 + p0;
        __int128 q2 = a * q1 + q0;
        if (p2 > numeric_limits<long long>::max() || q2 > numeric_limits<long long>::max())
        {
            return nullopt;
        }
        p0 = p1;
        p1 = p2;
        q0 = q1;
```

```
        q1 = q2;
    }
    return pair{(long long)p1, (long long)q1};
}
```

`pell(d)`（d 正整数）返回基本解，d 为完全平方数或解越过 `long long` 时返回空。若 $\sqrt{d}$ 的连分数周期长度为 L，找到基本解至多检查 $2L$ 个周期项，额外保存 $O(1)$ 个宽整数。验证：$d=2,3,5,6,7$ 依次得 $(3,2),(2,1),(9,4),(5,2),(8,3)$，$d=4$ 返回空。

7.5.007 圆上整点

`twoSqRep(n)` 依据两平方和定理、Cornacchia 和高斯整数乘法返回全部非负表示；$p\equiv 3\pmod 4$ 的奇指数返回空，`n=0` 返回 `{0,0}`。枚举 $x^2+y^2=n$ 的全部整数解，即原点为心、半径平方为 n 的圆周整点。枚举非负 x、判 $n-x^2$ 是否完全平方，再由坐标交换与正负号生成全部解，用 `set` 消去坐标为零或 $|x|=|y|$ 时的重复。

```
vector<pair<long long, long long>> circlePoints(long long n)
{
    // n 是非负半径平方；返回满足 x*x+y*y=n 的全部整数点。
    assert(n >= 0); // 调试检查，可删
    set<pair<long long, long long>> s;
    for (long long x = 0; x == 0 || x <= n / x; x++)
    {
        long long y2 = n - x * x;
        long long y = kthRoot((unsigned long long)y2, 2);
        if (y * y != y2)
        {
            continue;
        }
        for (int sx : {-1, 1})
        {
            for (int sy : {-1, 1})
            {
                s.insert({sx * x, sy * y});
                s.insert({sx * y, sy * x});
            }
        }
    }
    return {s.begin(), s.end()};
}
```

`circlePoints(n)`（$n\ge 0$）依赖 7.1.014 整数 k 次方根 的 `kthRoot`，返回去重并按字典序排列的全部整点。时间为 $O(\sqrt{n}(\log n+\log(k+1)))$、空间为 $O(k)$（k 为整点数）；非原点圆心整体平移结果即可。验证：$n=25$ 得 12 个点（$(0,\pm5),(\pm5,0),(\pm3,\pm4),(\pm4,\pm3)$），$n=3$ 为空，$n=0$ 只有 $(0,0)$。

8. 组合数学

8.1 组合数

8.1.001 阶乘、逆阶乘与组合数

预处理阶乘表 `fac` 与逆阶乘表 `ifac` 后，$\binom{n}{k}=\frac{n!}{k!(n-k)!}$、$A(n,k)=\frac{n!}{(n-k)!}$ 都退化为常数次乘法。逆阶乘不必逐个求逆：先用一次除法得到最大的 $(n!)^{-1}$，再借 $((i-1)!)^{-1}=(i!)^{-1}\cdot i$ 反向递推，全表只用一次求逆。`init` 支持增量扩表，`fac.size()` 已够时直接返回，扩容时只补算新增前缀，所以系数类型 `T` 必须支持除法（模素数或有理数），且所有用到的 $n!$ 在该类型下可逆——否则某个 $i!\equiv 0$ 会让 `T(1)/fac[n]` 失去意义。

```
// 依赖 1.1.008 静态取模类
template <class T> class Comb
{
    // fac[i]、ifac[i] 分别保存 i! 与其乘法逆元。
    vector<T> fac{1}, ifac{1};

  public:
    void init(int n)
    {
        // n 是要支持的最大上标；把阶乘表扩展到 n，无返回值。
        assert(n >= 0); // 调试检查，可删。
        if ((int)fac.size() > n)
        {
            return;
        }
        int old = fac.size();
        fac.resize(n + 1);
        ifac.resize(n + 1);
        for (int i = old; i <= n; i++)
        {
            fac[i] = fac[i - 1] * T(i);
        }
        ifac[n] = T(1) / fac[n];
        for (int i = n; i >= old; i--)
        {
            ifac[i - 1] = ifac[i] * T(i);
        }
    }

    T C(int n, int k)
    {
        // n 是对象数，k 是选择数；返回组合数，k 不在 [0,n] 时返回 0。
        if (k < 0 || k > n)
        {
            return T(0);
        }
        init(n);
```

```
        return fac[n] * ifac[k] * ifac[n - k];
    }

    T A(int n, int k)
    {
        // n 是对象数，k 是有序选取数；返回排列数，k 不在 [0,n] 时返回 0。
        if (k < 0 || k > n)
        {
            return T(0);
        }
        init(n);
        return fac[n] * ifac[n - k];
    }
};
```

以 `Comb<Z> c` 实例化，`Z` 需支持整数构造与四则运算，`c.C(n,k)`、`c.A(n,k)` 在 $k \notin [0,n]$ 时返回 0；`Z` 为模整数时模数必须大于所有用到的 n。扩表到 n 为 $O(n)$ 时间与空间，之后每次查询 $O(1)$。这是素数模且 n 小于模数时的首选；一旦 n 超过素模 p，某个 $p! \equiv 0$ 破坏可逆性，须改用 8.1.003 Lucas 定理 把上下标按 p 进制拆开，合数模则只能走 8.1.004 扩展 Lucas。分母不可逆或只需要整行组合数时换 8.1.002 杨辉三角 的纯加法递推。

准备 1.1.008 静态取模类后，可用同一个较大的组合数同时检查扩表、逆阶乘和三次模乘；下面两个结果分别是 $\binom{132}{66}$ 在最常用的两个模数下的已知值：

```
signed main()
{
    // 无参数；核对两个常见模数下的组合数与边界，返回 0。
    using Z998 = MInt<998244353>;
    using Z1e9 = MInt<1000000007>;
    Comb<Z998> a;
    Comb<Z1e9> b;
    assert(a.C(132, 66).val() == 241200029);
    assert(b.C(132, 66).val() == 598375978);
    assert(a.C(0, 0).val() == 1);
    assert(a.C(10, 11).val() == 0);
    return 0;
}
```

8.1.002 杨辉三角

对是否选中某个指定元素分类，就得到 Pascal 恒等式 $\binom{i}{j} = \binom{i-1}{j-1} + \binom{i-1}{j}$，每行两端恒为 1。整张表只用加法递推，不涉及除法，因此模数不保证分母可逆、或需要一整行组合数时它比 8.1.001 阶乘、逆阶乘与组合数 更合适；代价是行数不能太大。

```
template <class T> vector<vector<T>> pascalTri(int n)
{
    // n 是最大行号；返回第 0..n 行杨辉三角，其中 a[i][j]=C(i,j)。
    assert(n >= 0); // 调试检查，可删。
    vector<vector<T>> a(n + 1);
```

```
    for (int i = 0; i <= n; i++)
    {
        a[i].assign(i + 1, T(1));
        for (int j = 1; j < i; j++)
        {
            a[i][j] = a[i - 1][j - 1] + a[i - 1][j];
        }
    }
    return a;
}
```

`pascalTri<T>(n)` 返回第 $0..n$ 行，`T` 只需支持加法和从 1 构造，`a[i][j]` 即 $\binom{i}{j}$。$n=5$ 时第 5 行为 `1 5 10 10 5 1`。整表 $O(n^2)$ 时间与空间，只求单个组合数时不该建整表：那种场景用 8.1.001 阶乘、逆阶乘与组合数 的 $O(1)$ 查询，或结果不取模、能放进 `__int128` 时用 8.1.010 球盒模型与鸽巢原理 里的逐项约分 `comb`。

8.1.003 Lucas 定理

Lucas 定理来自把 $(1+x)^n$ 在模素数 p 下按 p 进制分块：$(1+x)^p \equiv 1+x^p \pmod p$，于是 $n=\sum n_i p^i$ 时 $(1+x)^n \equiv \prod_i (1+x^{p^i})^{n_i}$，比较 x^k 系数即得 $\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod p$，任一位 $k_i > n_i$ 则整体为 0。它只处理素数模，把远大于 p 的上标拆成 $O(\log_p n)$ 位、每位组合数在长度 p 的阶乘表上 $O(1)$ 取得。

```
int powPrime(int a, int b, int p)
{
    // a 是底数，b 是非负指数，p 是素数；返回 a^b mod p。
    assert(b >= 0 && p >= 2); // 调试检查，可删。
    a %= p;
    if (a < 0)
    {
        a += p;
    }
    int ans = 1 % p;
    while (b)
    {
        if (b & 1)
        {
            ans = combMulMod(ans, a, p);
        }
        a = combMulMod(a, a, p);
        b >>= 1;
    }
    return ans;
}

int lucas(int n, int k, int p)
{
    // n、k 是组合数参数，p 是素数；返回 C(n,k) mod p。
    assert(n >= 0 && p >= 2); // 调试检查，可删。
```

```
    if (k < 0 || k > n)
    {
        return 0;
    }
    vector<int> fac(p, 1), ifac(p, 1);
    for (int i = 1; i < p; i++)
    {
        fac[i] = combMulMod(fac[i - 1], i, p);
    }
    ifac[p - 1] = powPrime(fac[p - 1], p - 2, p);
    for (int i = p - 1; i; i--)
    {
        ifac[i - 1] = combMulMod(ifac[i], i, p);
    }
    int ans = 1;
    while (n || k)
    {
        int a = n % p, b = k % p;
        if (b > a)
        {
            return 0;
        }
        ans = combMulMod(combMulMod(ans, fac[a], p), ifac[b], p);
        ans = combMulMod(ans, ifac[a - b], p);
        n /= p;
        k /= p;
    }
    return ans;
}
```

`lucas(n,k,p)` 要求 `n>=0`，且 `p>=2` 是能完整建 p 长阶乘表的素数；实测 `lucas(100,20,7)=0`、`lucas(100,30,7)=0`、`lucas(100,50,7)=4`。每次 $O(p+\log_p n)$ 时间、$O(p)$ 空间，同一 `p` 反复查询时应把 `fac`/`ifac` 提到外层复用。若 $n<p$，可直接用 8.1.001 阶乘、逆阶乘与组合数；若 $n\geq p$ 且 p 大到不能建表，普通阶乘表同样会在 $p!$ 处失去逆元，必须改为逐位按较小一侧乘除、按需缓存阶乘或采用大素模阶乘算法。模数为合数时改用 8.1.004 扩展 Lucas 按质数幂分别处理再 CRT 合并。

8.1.004 扩展 Lucas

合数模 $m=\prod p_i^{e_i}$ 下 $\binom{n}{k}$ 的困难在于阶乘含 p 因子、逆元不存在，扩展 Lucas 把它拆到每个 p^e 上单独算再 CRT 合并。关键是把 $n!$ 分成 p 的幂部分与「单位部分」：$n!=p^{v_p(n!)}\cdot u(n)$，其中 $v_p(n!)=\sum_{i\geq 1}\lfloor n/p^i\rfloor$（`exponent`），$u(n)$ 是删去所有 p 因子后的乘积模 p^e（`split` 的第二项，靠 $[1,p^e]$ 内非 p 倍数之积的周期性快速幂加余项递归求得）。于是 $\binom{n}{k}$ 在 p^e 下的 p 指数为 $v_p(n!)-v_p(k!)-v_p((n-k)!)$，$\geq e$ 则该分量为 0，否则单位部分可逆，乘上 $p^{\text{指数}}$ 即得；各 $p_i^{e_i}$ 两两互质，`crt` 用 $\sum \text{get}\cdot\frac{m}{m_i}\cdot(\frac{m}{m_i})^{-1}$ 合并。本条依赖 7.1.001 快速幂 的 `mulMod`、`powerMod`、`invMod`，并复用 7.2.009 阶乘模合数 的 `PPFactorial::split`、`fact` 与 `modulus`；只新增组合数与 CRT 层，不能另写同义的质数幂单位阶乘。

```
// 依赖 7.1.003 扩展欧几里得
// 依赖 7.2.009 阶乘模合数
```

```
struct BinomMod
{
    struct PP
    {
        // p 是质因子，e 是 p 在总模数中的指数。
        int p, e;
        // fac 维护模 p^e 的单位阶乘与 p-adic 指数。
        PPFactorial fac;

        PP(int p, int e) : p(p), e(e), fac(p, e)
        {
            // p 是质因子，e 是正指数；建立对应的质数幂阶乘表。
        }

        int modulus() const
        {
            // 无参数；返回本分量的质数幂模数 p^e。
            return fac.modulus();
        }

        int comb(int n, int k) const
        {
            // n、k 是组合数参数；返回 C(n,k) mod p^e，k 不在 [0,n] 时返回 0。
            assert(n >= 0); // 调试检查，可删。
            if (k < 0 || k > n)
            {
                return 0;
            }
            auto [en, un] = fac.split(n);
            auto [ek, uk] = fac.split(k);
            auto [er, ur] = fac.split(n - k);
            int q = en - ek - er;
            if (q >= e)
            {
                return 0;
            }
            int mod = modulus();
            int den = mulMod(uk, ur, mod);
            int z = mulMod(un, *invMod(den, mod), mod);
            return mulMod(z, powerMod(p, q, mod), mod);
        }

        int fact(int n) const
        {
            // n 是非负整数；返回 n! mod p^e。
            return fac.fact(n);
        }
    };

    // mod 是查询所用的正模数。
    int mod;
    // a 保存 mod 的全部质数幂分量。
```

```
    vector<PP> a;

    explicit BinomMod(int mod) : mod(mod)
    {
        // mod 是正模数且每个质数幂分量不超过 5e6；分解 mod 并建立单位阶乘表。
        assert(mod > 0); // 调试检查，可删。
        int n = mod;
        for (int p = 2; p <= n / p; p++)
        {
            if (n % p == 0)
            {
                int e = 0;
                while (n % p == 0)
                {
                    n /= p;
                    e++;
                }
                a.emplace_back(p, e);
            }
        }
        if (n > 1)
        {
            a.emplace_back(n, 1);
        }
    }

    template <class F> long long crt(F get) const
    {
        // get(q) 返回模质数幂 q.mod 的余数；返回 CRT 合并后的模 mod 最小非负余数。
        if (mod == 1)
        {
            return 0;
        }
        long long ans = 0;
        for (const auto &q : a)
        {
            int qmod = q.modulus();
            int m = mod / qmod;
            int x = mulMod(get(q), m, mod);
            x = mulMod(x, *invMod(m, qmod), mod);
            ans = ans >= mod - x ? ans - (mod - x) : ans + x;
        }
        return ans;
    }

    int C(int n, int k) const
    {
        // n、k 是组合数参数；返回 C(n,k) mod mod，k 不在 [0,n] 时返回 0。
        assert(n >= 0); // 调试检查，可删。
        return crt(
            [&](const auto &q)
            {
```

```
                return q.comb(n, k);
            });
    }

    int fact(int n) const
    {
        // n 是非负整数；返回 n! mod mod。
        assert(n >= 0); // 调试检查，可删。
        return crt(
            [&](const auto &q)
            {
                return q.fact(n);
            });
    }
};
```

构造 `BinomMod q(mod)` 后调用 `q.C(n,k)` 或 `q.fact(n)`。构造时分解 `mod`、为每个质数幂 p^e 建长度 p^e 的 `f` 表；当前实现用调试断言要求每个 $p^e \le 5 \times 10^6$，避免误建无法承受的表。构造为 $O(\sqrt{m} + \sum p^e)$ 时间与 $O(\sum p^e)$ 空间，单次查询 $O(\sum \log_p n)$。素数模且 n 小于模数时直接用 8.1.001 阶乘、逆阶乘与组合数，素数模且 n 超模数时用 8.1.003 Lucas 定理；合数模才使用本条。需要按 `n,k,mod` 一次性调用而不持有对象时见 8.1.005 任意模数组合数 的薄封装。

公开的 `crt(get)` 可合并调用者按各质数幂计算的余数，例如重建 $23 \bmod 12$：

```
signed main()
{
    BinomMod q(12);
    int x = q.crt([](const auto &part)
    {
        return 23 % part.modulus();
    });
    assert(x == 11);
}
```

下面固定覆盖模 1、质数幂、混合质数幂和越界排名：

```
signed main()
{
    assert(BinomMod(1).C(20, 7) == 0);
    assert(BinomMod(8).C(10, 3) == 0);
    assert(BinomMod(72).C(20, 6) == 24);
    BinomMod q(60);
    assert(q.C(40, 41) == 0);
}
```

8.1.005 任意模数组合数

任意正模数下的组合数与扩展 Lucas 是同一数学算法，这里不重复底层类，只依赖 8.1.004 扩展 Lucas 的 `BinomMod`，包一层按 `n,k,mod` 一次性调用的入口；区别只在于是否预先持有模数对象。

```cpp
// 依赖 7.1.003 扩展欧几里得
// 依赖 7.2.009 阶乘模合数
int binomAnyMod(int n, int k, int mod)
{
    // n、k 是组合数参数，mod 是正模数且每个质数幂分量不超过 5e6；返回 C(n,k) mod mod。
    return BinomMod(mod).C(n, k);
}
```

`binomAnyMod(n,k,mod)` 每次都新建 `BinomMod`，因此同样要求 `mod` 的每个质数幂分量不超过 5×10^6；实测 `binomAnyMod(10,3,12)=0`、`binomAnyMod(10,4,1000)=210`。单次构造加查询的开销与 8.1.004 扩展 Lucas 相同，只适合零散查询；同一 `mod` 反复调用时应持有一个 `BinomMod` 复用预处理。

8.1.006 Vandermonde 卷积

从 m 个第一类元素与 n 个第二类元素中共选 r 个，按第一类恰选了 k 个分类，各类互斥且并起来是全部方案，于是有 Vandermonde 卷积 $\sum_k \binom{m}{k}\binom{n}{r-k}=\binom{m+n}{r}$。下面这段既是它的直接实现，也可当作核对组合数接口的验证：把左侧逐项求和与右侧 $\binom{m+n}{r}$ 比对。求和下标已收紧到 $k\in[\max(0,r-n),\min(m,r)]$，落在此外的组合数必为 0。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 8.1.001 阶乘、逆阶乘与组合数
template <class C> auto vandermonde(int m, int n, int r, C comb)
{
    // m、n 是两组大小，r 是总选择数，comb(x,y) 返回 C(x,y)；返回 Vandermonde 左侧求和。
    assert(m >= 0 && n >= 0); // 调试检查，可删。
    using T = decltype(comb(0, 0));
    T ans{};
    for (int k = max<int>(0, r - n); k <= min(m, r); k++)
    {
        ans += comb(m, k) * comb(n, r - k);
    }
    return ans;
}
```

把任一组合数接口作为回调传给 `vandermonde(m,n,r,comb)`，`comb(x,y)` 对合法参数返回同一系数类型，返回值类型由 `decltype(comb(0,0))` 推断；右侧直接写 `comb(m+n,r)` 即可，固定查验值为 `vandermonde(2,3,2,comb)==10`。共 $O(\min(m,r)-\max(0,r-n)+1)$ 次组合数调用、$O(1)$ 额外空间。`comb` 通常取 8.1.001 阶乘、逆阶乘与组合数 的 `Comb<Z>::C`（素数模）或 8.1.004 扩展 Lucas 的 `BinomMod::C`（合数模）；本条本身只做求和，不预处理阶乘。

固定素数模下可把 `Comb<Z>::C` 包成 Lambda：

```cpp
signed main()
{
    Comb<Z> c;
    auto comb = [&](int n, int k)
    {
        return c.C(n, k);
    };
```

```
    assert(vandermonde(2, 3, 2, comb) == Z(10));
}
```

8.1.007 容斥原理

容斥把并集计数写成所有非空交集按集合个数奇加偶减：$\left|\bigcup A_i\right| = \sum_{\emptyset \neq S} (-1)^{|S|+1} \left|\bigcap_{i \in S} A_i\right|$，前提是各交集计数针对同一全集。这里的示例把它落到「$[1, n]$ 中至少被某个 d_i 整除的整数个数」：子集 S 对应除数的最小公倍数 lcm，其倍数个数为 $\lfloor n / \operatorname{lcm} \rfloor$，按 `popcount(s)` 的奇偶加减。累乘 `lcm` 时用 `l > n / (d[i]/g)` 提前判溢出并跳过该子集（此时 $\operatorname{lcm} > n$，贡献本就为 0），避免乘积炸掉 `int`。

```
namespace incl_excl
{
int countMult(int n, vector<int> d)
{
    // n 是闭区间上界，d 是正除数表；返回 [1,n] 中至少被一个 d[i] 整除的整数个数。
    assert(n >= 0 && d.size() < 63); // 调试检查，可删。
    for (int x : d)
    {
        assert(x > 0); // 调试检查，可删。
    }
    int ans = 0;
    for (unsigned long long s = 1; s < (1ULL << d.size()); s++)
    {
        int l = 1;
        bool ok = true;
        for (int i = 0; i < (int)d.size(); i++)
        {
            if (!(s >> i & 1))
            {
                continue;
            }
            int g = gcd(l, d[i]);
            if (l > n / (d[i] / g))
            {
                ok = false;
                break;
            }
            l *= d[i] / g;
        }
        if (ok)
        {
            ans += (popcount(s) & 1 ? 1 : -1) * (n / l);
        }
    }
    assert(0 <= ans && ans <= n); // 调试检查，可删。
    return ans;
}
}
```

`countMult(n, d)` 要求 `n>=0`、每个除数为正且个数 $k < 63$（否则子集掩码溢出），返回 $[1, n]$ 内至少被

一个 d_i 整除的整数个数；还须保证容斥过程的带符号部分和能由 `int` 表示。枚举共 $O(2^k k)$ 时间、$O(1)$ 额外空间。除数很多时，先删掉被更小除数整除的冗余项；值域不大可筛出所有倍数，具有额外数论结构时再用 Möbius 或倍数 DP，不能用 8.1.009 单位根反演 直接替代任意除数集合的并集。第 k 大与子集最小值和之间的转换见 8.1.008 扩展 min-max 容斥与 KMAX-MIN 反演。

8.1.008 扩展 min-max 容斥与 KMAX-MIN 反演

普通 min-max 容斥是 $\max(S)=\sum_{\emptyset\neq T\subseteq S}(-1)^{|T|+1}\min(T)$；扩展版把「第 k 大」写成各子集最小值的带二项式系数组合：$\operatorname{kthmax}(S)=\sum_T(-1)^{|T|-k}\binom{|T|-1}{k-1}\min(T)$。按大小聚合后只依赖每个尺寸 j 的子集最小值之和 mn_j，得 第k 大 $=\sum_{j\ge k}(-1)^{j-k}\binom{j-1}{k-1}mn_j$。为避免巨大交错部分和，代码分别在三个互素大素数下递推二项式系数，再用 CRT 恢复唯一的 64 位有符号答案；三模乘积远大于整个 64 位值域。

```cpp
namespace minmax_inv
{
int kthMaxMinSum(const vector<int> &mn, int k)
{
    // mn[j] 是所有大小为 j 的子集最小值之和且 mn[0] 不用，k 从 1 开始；返回第 k 大元素。
    int n = (int)mn.size() - 1;
    assert(1 <= k && k <= n); // 调试检查，可删。
    constexpr array<long long, 3> ps{998244353, 1000000007, 1000000009};
    assert(n < ps[0]); // 调试检查，可删。
    array<long long, 3> rem{};
    for (int z = 0; z < 3; z++)
    {
        long long mod = ps[z];
        vector<long long> inv(n - k + 1);
        if (n > k)
        {
            inv[1] = 1;
        }
        for (int i = 2; i <= n - k; i++)
        {
            inv[i] = mod - (mod / i) * inv[mod % i] % mod;
        }
        long long c = 1;
        long long ans = 0;
        for (int j = k; j <= n; j++)
        {
            if (j > k)
            {
                c = c * (j - 1) % mod * inv[j - k] % mod;
            }
            long long x = mn[j] % mod;
            if (x < 0)
            {
                x += mod;
            }
            long long v = c * x % mod;
```

```
            if ((j - k) & 1)
            {
                ans -= v;
                if (ans < 0)
                {
                    ans += mod;
                }
            }
            else
            {
                ans += v;
                if (ans >= mod)
                {
                    ans -= mod;
                }
            }
        }
        rem[z] = ans;
    }
    __int128 ans = rem[0];
    __int128 m = ps[0];
    for (int z = 1; z < 3; z++)
    {
        long long mod = ps[z];
        long long a = rem[z] - (long long)(ans % mod);
        if (a < 0)
        {
            a += mod;
        }
        long long b = m % mod;
        long long e = mod - 2;
        long long iv = 1;
        while (e)
        {
            if (e & 1)
            {
                iv = iv * b % mod;
            }
            b = b * b % mod;
            e >>= 1;
        }
        ans += m * (a * iv % mod);
        m *= mod;
    }
    if (ans > m / 2)
    {
        ans -= m;
    }
    assert(numeric_limits<int>::min() <= ans && ans <= numeric_limits<int>::max()); // 调试检查，
    ↪ 可删。
    return (int)ans;
}
```

```
}
```

`kthMaxMinSum(mn, k)` 中 `mn[j]` 是所有大小为 j 的子集最小值之和（`mn[0]` 不用），每个和与最终答案都必须能放进 `int`；`k` 从 1 计，数组长度小于三个模数。三次线性递推与常数次 CRT 共 $O(n)$ 时间、$O(n)$ 额外空间，不会因最终抵消前的部分和而溢出。这套反演用于「各尺寸子集最小值和已知、要还原第 k 名」；只需并集大小时用 8.1.007 容斥原理。

8.1.009 单位根反演

单位根的正交性给出指示函数 $[m \mid t] = \frac{1}{m}\sum_{j=0}^{m-1}\omega^{jt}$（$\omega$ 是 m 次本原单位根）。把它套到生成函数 $A(x)=\sum_k a_k x^k$ 上，取 $t = k - r$ 就能滤出指数模 m 余 r 的那些项：$\sum_{k\equiv r\,(m)} a_k = \frac{1}{m}\sum_{j=0}^{m-1}\omega^{-jr}A(\omega^j)$，非目标指数在求和中相位抵消。这里在模 998244353 的有限域里做，$\omega = 3^{(p-1)/m}$ 要求 $m \mid p-1$；`rootFilter` 对每个 ω^j 用 Horner（`a.rbegin()` 起）求 $A(\omega^j)$，乘上 ω^{-jr} 累加，最后乘 m^{-1}。

```
namespace roots_unity
{
constexpr int mod = 998244353;

int qpow(int a, int n)
{
    // a 是底数，n 是非负指数；返回模 mod 幂。
    assert(n >= 0); // 调试检查，可删。
    a %= mod;
    if (a < 0)
    {
        a += mod;
    }
    int r = 1;
    while (n)
    {
        if (n & 1)
        {
            r = r * a % mod;
        }
        a = a * a % mod;
        n >>= 1;
    }
    return r;
}

int rootFilter(const vector<int> &a, int m, int rem)
{
    // a 是多项式系数，m 是整除 mod-1 的正整数，rem 是余数；返回次数同余 rem 的系数和。
    assert(m > 0 && (mod - 1) % m == 0); // 调试检查，可删。
    rem %= m;
    if (rem < 0)
    {
```

```
        rem += m;
    }
    int w = qpow(3, (mod - 1) / m);
    int ans = 0;
    for (int j = 0; j < m; j++)
    {
        int x = qpow(w, j);
        int y = 0;
        for (auto it = a.rbegin(); it != a.rend(); ++it)
        {
            int c = *it % mod;
            if (c < 0)
            {
                c += mod;
            }
            y = (y * x + c) % mod;
        }
        int e = ((m - rem) % m) * j % (mod - 1);
        ans = (ans + y * qpow(w, e)) % mod;
    }
    return ans * qpow(m, mod - 2) % mod;
}
}
```

`rootFilter(a, m, rem)` 中 `a` 是多项式系数（`a[i]` 为 x^i 系数），`m` 必须整除 $p-1$，`rem` 是目标余数（内部先规约到 $[0,m)$）；返回次数同余 `rem` 的系数和。直接求值 $O(mn)$ 时间、$O(1)$ 额外空间（n 为多项式次数）。当 m 很小而需要频繁分组，或题目本身要卷积时，用第 10 章的 NTT 把多点求值降到 $O(n\log n)$ 更划算；这里的朴素版胜在实现短、无需搬多项式全家桶。它与 8.1.007 容斥原理 都在做「按同余类筛选」，但单位根反演处理的是幂级数系数而非集合并。

8.1.010 球盒模型与鸽巢原理

球盒计数先分清球、盒是否有标号，能否为空，有无容量限制。本条实现最常用的隔板法：无标号球放进有标号盒，允许空盒时为 $\binom{b+m-1}{m-1}$，要求非空时为 $\binom{b-1}{m-1}$。`comb` 每轮先约分再做一次受检乘法，默认用 64 位；需要更宽结果时由调用者显式选择类型。鸽巢下界用商与余数计算，不形成可能溢出的 `objects+boxes-1`。

```
namespace balls_boxes
{
template <class T = long long>
T comb(T n, int k)
{
    // n 是非负上标，k 是下标；返回二项式系数。
    static_assert(numeric_limits<T>::is_integer && numeric_limits<T>::is_signed &&
                  sizeof(T) >= sizeof(long long));
    assert(n >= 0); // 调试检查，可删。
    if (k < 0 || (T)k > n)
    {
        return T(0);
    }
```

```
    if ((T)k > n - k)
    {
        k = (int)(n - k);
    }
    T ans = 1;
    for (int i = 1; i <= k; i++)
    {
        T x = n - k + i;
        int y = i;
        int g = gcd((int)(x % y), y);
        x /= g;
        y /= g;
        assert(ans % y == 0); // 调试检查，可删。
        ans /= y;
        ans *= x;
    }
    return ans;
}

template <class T = long long>
T unlabBoxDist(int balls, int boxes, bool nonempty)
{
    // balls 是球数，boxes 是盒数，nonempty 表示每盒非空；返回方案数。
    static_assert(numeric_limits<T>::is_integer && numeric_limits<T>::is_signed &&
                  sizeof(T) >= sizeof(long long));
    assert(balls >= 0 && boxes >= 0); // 调试检查，可删。
    if (boxes == 0)
    {
        return T(balls == 0);
    }
    if (nonempty)
    {
        return balls < boxes ? T(0) : comb<T>((T)balls - 1, boxes - 1);
    }
    T b = balls, m = boxes;
    return comb<T>(b + m - 1, boxes - 1);
}

int pigeonBound(int objects, int boxes)
{
    // objects 是对象数，boxes 是正盒数；返回必有一盒至少包含的对象数。
    assert(objects >= 0 && boxes > 0); // 调试检查，可删。
    return objects / boxes + (objects % boxes != 0);
}
}
```

两个计数函数默认返回 `long long`；调用者须保证上标和答案能由 `T` 表示，需要 128 位精确值时调用 `comb<__int128>` 或 `unlabBoxDist<__int128>`。`comb` 为 $O(\min(k, n-k))$ 时间、$O(1)$ 空间。超过 128 位用 14.1.001 高精度整数（BigInt），只求模意义答案则用 8.1.001 阶乘、逆阶乘与组合数 或 8.1.004 扩展 Lucas。

8.1.011 q-二项式与 q-Lucas

Gaussian 二项式由 $\begin{bmatrix} n \\ k \end{bmatrix}_q = \prod_{i=1}^{k} \frac{[n-k+i]_q}{[i]_q}$ 定义，其中 $[i]_q = 1 + q + \cdots + q^{i-1}$。模素数 p 时，直接除法会在某个 $[i]_q = 0$ 后失效；令 d 为首个正的零 q-整数，若 $q = 0$ 或遍历到 $p-1$ 仍无零则取 $d = p$。对 $0 \le n, k < p$ 有 q-Lucas 分解 $\begin{bmatrix} n \\ k \end{bmatrix}_q = \binom{\lfloor n/d \rfloor}{\lfloor k/d \rfloor} \begin{bmatrix} n \bmod d \\ k \bmod d \end{bmatrix}_q \pmod p$。低位的 q-阶乘均非零，高位是普通组合数；$q = 0$ 时合法组合数恒为 1，$q = 1$ 时退化为普通组合数，二者都无需另写特判。

```
class QBinomPrime
{
    // p 是素数模，q 是规范到 [0,p) 的参数，d 是首个零 q-整数的位置；无零时 d=p。
    int p, q, d;
    // fac/ifac 是 0..d-1 的 q-阶乘及逆元，cfac/cifac 是高位普通阶乘及逆元。
    vector<int> fac, ifac, cfac, cifac;

    int mul(int a, int b) const
    {
        // a、b 是模 p 乘数；返回 ab mod p。
        return combMulMod(a, b, p);
    }

    int power(int a, int n) const
    {
        // a 是模 p 底数，n 是非负指数；返回 a^n mod p。
        int r = 1;
        while (n)
        {
            if (n & 1)
            {
                r = mul(r, a);
            }
            a = mul(a, a);
            n >>= 1;
        }
        return r;
    }

    int small(int n, int k) const
    {
        // 0<=n<d，k 是下标；返回对应低位 q-二项式，k 不在 [0,n] 时返回 0。
        if (k < 0 || k > n)
        {
            return 0;
        }
        return mul(mul(fac[n], ifac[k]), ifac[n - k]);
    }

    int ordinary(int n, int k) const
    {
        // n 是高位上标，k 是高位下标；返回普通 C(n,k) mod p。
        if (k < 0 || k > n)
        {
```

```
            return 0;
        }
        return mul(mul(cfac[n], cifac[k]), cifac[n - k]);
    }

public:
    QBinomPrime(int p, int q) : p(p), q(q)
    {
        // p 是可承担 O(p) 预处理的素数，q 是 q-二项式参数；建立 q-Lucas 查询表。
        assert(p >= 2); // 调试检查，可删。
        this->q %= p;
        if (this->q < 0)
        {
            this->q += p;
        }
        fac = {1};
        vector<int> num;
        int x = 1;
        for (int i = 1; i < p; i++)
        {
            if (x == 0)
            {
                break;
            }
            num.push_back(x);
            fac.push_back(mul(fac.back(), x));
            x = (mul(this->q, x) + 1) % p;
        }
        d = fac.size();
        ifac.resize(d);
        ifac[d - 1] = power(fac[d - 1], p - 2);
        for (int i = d - 1; i >= 1; i--)
        {
            ifac[i - 1] = mul(ifac[i], num[i - 1]);
        }

        int m = (p - 1) / d;
        cfac.assign(m + 1, 1);
        cifac.assign(m + 1, 1);
        for (int i = 1; i <= m; i++)
        {
            cfac[i] = mul(cfac[i - 1], i);
        }
        cifac[m] = power(cfac[m], p - 2);
        for (int i = m; i >= 1; i--)
        {
            cifac[i - 1] = mul(cifac[i], i);
        }
    }

    int C(int n, int k) const
    {
```

```
        // n、k 是 q-二项式参数且 0<=n<p；返回 [n choose k]_q mod p，k 越界时返回 0。
        assert(0 <= n && n < p); // 调试检查，可删。
        if (k < 0 || k > n)
        {
            return 0;
        }
        return mul(ordinary(n / d, k / d), small(n % d, k % d));
    }
};
```

构造 `QBinomPrime cb(p,q)` 后用 `cb.C(n,k)`，要求 `p` 为素数、$0 \le n < p$，并且 p 小到能承受线性预处理。固定查验值为 `QBinomPrime(7,2).C(6,3)==2`。设 q-整数周期为 d，构造 $O(d + p/d)$ 时间与空间，单次查询 $O(1)$；同一 `p,q` 的多组询问必须复用对象。若 $n \ge p$，还要继续按 p 进制处理普通组合数高位，不能直接套本接口；普通二项式见 8.1.003 Lucas 定理，多项式形式的快速运算见第 10 章。

8.1.012 二项式恒等式与反演

约定 $n \ge 0$ 且 $\binom{n}{k} = 0$ 对所有 $k \notin [0, n]$ 成立。二项式定理及其系数形式为

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^{n-k} y^k,$$
$$[x^k](1+x)^n = \binom{n}{k}.$$

分别代入 $x = 1$ 与 $x = -1$，得到

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n, \qquad \sum_{k=0}^{n} (-1)^k \binom{n}{k} = [n = 0].$$

对 $(1+x)^n$ 求导后乘 x，再令 $x = 1$，可依次得到加权和。下式要求 $n \ge 2$；$n = 0, 1$ 直接代入求和式。

$$\sum_{k} k \binom{n}{k} = n 2^{n-1},$$
$$\sum_{k} k(k-1) \binom{n}{k} = n(n-1) 2^{n-2},$$
$$\sum_{k} k^2 \binom{n}{k} = n(n+1) 2^{n-2}.$$

Vandermonde 卷积见 8.1.006 Vandermonde 卷积。令其中一侧退化，可得 hockey-stick 恒等式；其条件为 $n \ge r \ge 0$：

$$\sum_{k=r}^{n} \binom{k}{r} = \binom{n+1}{r+1}.$$

二项式反演必须成对使用同一符号约定。对任意序列，若

$$b_n=\sum_{k=0}^{n}\binom{n}{k}a_k,$$

则恰有

$$a_n=\sum_{k=0}^{n}(-1)^{n-k}\binom{n}{k}b_k.$$

这些等式是整数恒等式，取模时可直接逐项取模，不涉及除法；按 $(-1)^k$ 书写的等价反演式需要同时调整两边符号，不能只替换一个求和式。

8.1.013 斯特林反演

第一类、第二类斯特林数分别记为 $s(n,k)$、$S(n,k)$。两组反演关系是

$$f_n=\sum_{k=0}^{n}S(n,k)g_k\iff g_n=\sum_{k=0}^{n}s(n,k)f_k,$$

以及

$$f_n=\sum_{k=0}^{n}s(n,k)g_k\iff g_n=\sum_{k=0}^{n}S(n,k)f_k.$$

它们来自多项式基变换：$x^{\underline{n}}=\sum_k s(n,k)x^k$、$x^n=\sum_k S(n,k)x^{\underline{k}}$。竞赛中看到幂和与下降幂和互换时，可以直接套用；系数的符号只出现在第一类数中。

递推式为

$$s(n,k)=s(n-1,k-1)-(n-1)s(n-1,k),\qquad S(n,k)=S(n-1,k-1)+kS(n-1,k),$$

边界为 $s(0,0)=S(0,0)=1$，其余越界项为 0。若只需固定 k，使用 8.2.014 固定 k 的第一类 Stirling 数 或 8.2.015 固定 k 的第二类 Stirling 数；需要全部 $0\le k\le n$ 时用 $O(n^2)$ 动态规划。

```cpp
// 依赖 8.2.005 Stirling 数
template <class T> vector<T> stirlingTr(const vector<T> &a, bool inv = false)
{
    // a[0..n] 是输入序列；inv=false 用第二类数变换，inv=true 用有符号第一类数反演。
    // 返回 b[i]=sum_{j=0..i} t[i][j]*a[j]，其中 t 为对应数表；空序列返回空表。
    if (a.empty()) return {};
    int n = a.size() - 1;
    auto t = inv ? stirling1<T>(n) : stirling2<T>(n);
    vector<T> b(n + 1);
    for (int i = 0; i <= n; i++)
    {
        for (int j = 0; j <= i; j++)
        {
            b[i] += t[i][j] * a[j];
        }
```

```
    }
    return b;
}
```

`stirlingTr(a)` 计算第二类变换，`stirlingTr(a,true)` 计算其逆变换；底层数表复用 8.2.005 Stirling 数，不另写递推。`T` 需要支持整数构造、加减乘，整数结果须避免溢出；模意义下的恒等式不要求模数为素数，使用既有模整数类型即可。

```
signed main()
{
    // 无参数；验证第二类变换与有符号第一类反演，返回 0。
    vector<Z> a{1, 2, 3, 4};
    auto b = stirlingTr(a);
    assert((b == vector<Z>{1, 2, 5, 15}));
    assert(stirlingTr(b, true) == a);
}
```

长度为 $n+1$ 时，完整数表与变换共需 $O(n^2)$ 时间、$O(n^2)$ 额外空间。只需 Stirling 数某一行时，直接选择底层条目的单行快速算法；固定 k 则使用 8.2.014 或 8.2.015，不必构造整个数表。

8.2 经典数列与计数

8.2.001 Fibonacci / Lucas 数

矩阵快速幂 $\begin{bmatrix}1 & 1\\ 1 & 0\end{bmatrix}^n$ 展开后可消去冗余乘法，得到倍增恒等式 $F_{2n}=F_n(2F_{n+1}-F_n)$、$F_{2n+1}=F_n^2+F_{n+1}^2$。`fibPair` 每次递归把下标折半、同时维护相邻对 $\{F_n,F_{n+1}\}$，比通用 2×2 矩阵幂常数更小；Lucas 数则由 $L_n=2F_{n+1}-F_n$ 直接换算。模乘调用 `combMulMod`，常见小模数直接使用 64 位乘法，接近 `long long` 上界时才局部使用 `__int128`；$n=0$ 时返回 $\{0, 1 \bmod mod\}$，兼容 `mod=1`。

```
array<int, 2> fibPair(unsigned long long n, int mod)
{
    // n 是下标，mod 是正模数；返回 {F_n,F_{n+1}} mod mod。
    assert(mod > 0); // 调试检查，可删。
    if (n == 0)
    {
        return {0, 1 % mod};
    }
    auto [a, b] = fibPair(n >> 1, mod);
    int t = b >= mod - b ? b - (mod - b) : b + b;
    t -= a;
    if (t < 0)
    {
        t += mod;
    }
    int c = combMulMod(a, t, mod);
    int x = combMulMod(a, a, mod);
    int y = combMulMod(b, b, mod);
```

```
    int d = x >= mod - y ? x - (mod - y) : x + y;
    if (n & 1)
    {
        int next = c >= mod - d ? c - (mod - d) : c + d;
        return {d, next};
    }
    return {c, d};
}

int lucasNumber(unsigned long long n, int mod)
{
    // n 是下标，mod 是正模数；返回 Lucas 数 L_n mod mod。
    assert(mod > 0); // 调试检查，可删。
    if (n == 0)
    {
        return 2 % mod;
    }
    auto [fn, fn1] = fibPair(n, mod);
    int ans = fn1 >= mod - fn1 ? fn1 - (mod - fn1) : fn1 + fn1;
    ans -= fn;
    return ans < 0 ? ans + mod : ans;
}
```

`fibPair(n,mod)` 返回 $\{F_n, F_{n+1}\} \bmod mod$，`lucasNumber(n,mod)` 返回 $L_n \bmod mod$；模 10^9+7 下 `fibPair(10)={55,89}`、`lucasNumber(10)=123`。$O(\log n)$ 时间、$O(\log n)$ 递归栈。需要精确大整数时沿用倍增式，改以 14.1.001 高精度整数（BigInt）保存两项并去掉取模。这是唯一按下标 $O(\log n)$ 求值的数列；下面 Catalan、Motzkin、Bell 等都是 $O(n^2)$ 或 $O(n\sqrt{n})$ 的整段递推，求单个高位项时反而应回到对应的闭式或矩阵幂。

8.2.002 Catalan 数

合法括号序列、凸多边形三角剖分、二叉树形态等都由 Catalan 数计数。按第一个匹配位置（或根）把对象拆成互相独立的左右两块，得卷积递推 $C_i = \sum_{j=0}^{i-1} C_j C_{i-1-j}$，$C_0 = 1$；闭式 $C_n = \frac{1}{n+1}\binom{2n}{n}$ 只在分母可逆或整数约分时才直接用。下面返回 $C_0, \dots, C_n$，`T` 只需支持加法、乘法和从 1 构造。

```
template <class T> vector<T> catalan(int n)
{
    // n 是要计算的最大下标；返回 C_0 到 C_n 的 Catalan 数。
    assert(n >= 0); // 调试检查，可删。
    vector<T> a(n + 1);
    a[0] = 1;
    for (int i = 1; i <= n; i++)
    {
        for (int j = 0; j < i; j++)
        {
            a[i] += a[j] * a[i - 1 - j];
        }
    }
    return a;
```

```
}
```

`catalan<T>(n)` 返回 $C_0, \dots, C_n$，卷积递推为 $O(n^2)$ 时间、$O(n)$ 输出空间。模素数且 $2n$ 小于模数时，用 8.1.001 阶乘、逆阶乘与组合数 按闭式预处理整段只需 $O(n)$；用形式幂级数时则由 $C(x) = 1 + xC(x)^2$ 做 Newton 迭代。图的生成树计数不属于这类一维递推，见 8.2.011 Matrix-Tree / Kirchhoff 生成树计数。

前若干项 $C_0, C_1, \dots$：$1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900$（64 位有符号内到 $C_{35} = 3116285494907301262$ 不溢出，C_{36} 溢出）。

8.2.003 Motzkin 数

Motzkin 数统计不越过横轴、每步为上/下/平的格路（也对应圆上非交弦等结构）。按首步分类：若首步是水平步，剩下是长度 $i-1$ 的 Motzkin 路；若首步上升，则到首次回零之间夹一段 Motzkin 路、之后再接一段，于是 $M_i = M_{i-1} + \sum_{j=0}^{i-2} M_j M_{i-2-j}$，$M_0 = 1$，即一个线性项加 Catalan 型卷积。

```
template <class T> vector<T> motzkin(int n)
{
    // n 是要计算的最大下标；返回 M_0 到 M_n 的 Motzkin 数。
    assert(n >= 0); // 调试检查，可删。
    vector<T> a(n + 1);
    a[0] = 1;
    for (int i = 1; i <= n; i++)
    {
        a[i] = a[i - 1];
        for (int j = 0; j + 2 <= i; j++)
        {
            a[i] += a[j] * a[i - 2 - j];
        }
    }
    return a;
}
```

`motzkin<T>(n)` 返回 $M_0, \dots, M_n$，卷积递推 $O(n^2)$ 时间、$O(n)$ 空间。与 8.2.002 Catalan 数 同族但多了水平步：只有上、下步且必须回到起点时是 Catalan，允许平步则是 Motzkin。

前若干项 $M_0, M_1, \dots$：$1, 1, 2, 4, 9, 21, 51, 127, 323, 835, 2188, 5798, 15511$。

8.2.004 Bell 数

Bell 数 B_n 数把 n 个带标号元素划分成若干非空无序集合的方案。递推 $B_{n+1} = \sum_{k=0}^{n} \binom{n}{k} B_k$ 来自按「含最后一个元素的块」大小分类。代码用等价的 Bell 三角实现：每行首项抄上一行末项，同行向右做前缀累加 $a[i][j] = a[i][j-1] + a[i-1][j-1]$，每行首项 `a[i][0]` 即 B_i，避免显式算组合数。

```
template <class T> vector<T> bell(int n)
{
    // n 是要计算的最大下标；返回 B_0 到 B_n 的 Bell 数。
    assert(n >= 0); // 调试检查，可删。
```

```
    vector<vector<T>> a(n + 1, vector<T>(n + 1));
    vector<T> b(n + 1);
    a[0][0] = 1;
    b[0] = 1;
    for (int i = 1; i <= n; i++)
    {
        a[i][0] = a[i - 1][i - 1];
        b[i] = a[i][0];
        for (int j = 1; j <= i; j++)
        {
            a[i][j] = a[i][j - 1] + a[i - 1][j - 1];
        }
    }
    return b;
}
```

`bell<T>(n)` 返回 $B_0,\dots,B_n$，Bell 三角为 $O(n^2)$ 时间与空间。Bell 数是第二类 Stirling 数按块数求和 $B_n=\sum_k S(n,k)$，需要按块数分开计数时用 8.2.005 Stirling 数 的第二类；只关心总划分数则本条足够。模素数下需要整段大规模结果时，可由 EGF e^{e^x-1} 调用第 10 章形式幂级数指数。

前若干项 $B_0,B_1,\dots$：$1,1,2,5,15,52,203,877,4140,21147,115975,678570$。

8.2.005 Stirling 数

第一类 Stirling 数按下标计排列的环数，第二类计集合划分成指定块数的方案。加入第 i 个新元素时，无符号第一类满足 $\left[\begin{smallmatrix}i\\j\end{smallmatrix}\right]=\left[\begin{smallmatrix}i-1\\j-1\end{smallmatrix}\right]+(i-1)\left[\begin{smallmatrix}i-1\\j\end{smallmatrix}\right]$；代码采用对应下降幂 $x^{\underline{n}}$ 系数的有符号版本 $s(i,j)=s(i-1,j-1)-(i-1)s(i-1,j)$，需要环数时取绝对值。第二类中，新元素要么独占新块，要么并入已有 j 块之一，故 $\left\{\begin{smallmatrix}i\\j\end{smallmatrix}\right\}=\left\{\begin{smallmatrix}i-1\\j-1\end{smallmatrix}\right\}+j\left\{\begin{smallmatrix}i-1\\j\end{smallmatrix}\right\}$。

```
template <class T> vector<vector<T>> stirling1(int n)
{
    // n 是最大行号；返回有符号第一类 Stirling 数 s(i,j) 的第 0..n 行。
    assert(n >= 0); // 调试检查，可删。
    vector<vector<T>> a(n + 1, vector<T>(n + 1));
    a[0][0] = 1;
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= i; j++)
        {
            a[i][j] = a[i - 1][j - 1] - T(i - 1) * a[i - 1][j];
        }
    }
    return a;
}

template <class T> vector<vector<T>> stirling2(int n)
{
    // n 是最大行号；返回第二类 Stirling 数 S(i,j) 的第 0..n 行。
    assert(n >= 0); // 调试检查，可删。
    vector<vector<T>> a(n + 1, vector<T>(n + 1));
```

```
    a[0][0] = 1;
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= i; j++)
        {
            a[i][j] = a[i - 1][j - 1] + T(j) * a[i - 1][j];
        }
    }
    return a;
}
```

`stirling1<T>(n)`、`stirling2<T>(n)` 各返回一张 $(n+1)\times(n+1)$ 的三角表，$a[i][j]$ 为对应 Stirling 数；均 $O(n^2)$ 时间与空间。只查一个第二类 $S(n,k)$ 且模素数时，可用 $S(n,k)=\frac{1}{k!}\sum_j(-1)^j\binom{k}{j}(k-j)^n$ 配 8.1.001 阶乘、逆阶乘与组合数，以 $O(k\log n)$ 时间求值；需要小规模整表时用本条。集合划分不限块数就求和成 8.2.004 Bell 数。

在 $m,n\ge 0$ 时，交错幂和与第二类 Stirling 数满足下式；当 $m=n=0$ 时约定 $0^0=1$：

$$\sum_{k=0}^{n}(-1)^k\binom{n}{k}k^m=(-1)^n n!\left\{{m\atop n}\right\}.$$

因此 $m<n$ 时为 0，$m=n$ 时为 $(-1)^n n!$。当模素数 p 且 $n<p$ 时，因 $n!$ 可逆，可以反解为

$$\left\{{m\atop n}\right\}=\frac{1}{n!}\sum_{k=0}^{n}(-1)^{n-k}\binom{n}{k}k^m\pmod p.$$

若模数为合数，或 $p\mid n!$，不能直接使用这个分式形式。

前几行速查。代码返回有符号第一类 $s(n,k)$，下表只展示无符号绝对值 $|s(n,k)|$：

$n\backslash k$	0	1	2	3	4	5	6	7
0	1	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0
2	0	1	1	0	0	0	0	0
3	0	2	3	1	0	0	0	0
4	0	6	11	6	1	0	0	0
5	0	24	50	35	10	1	0	0
6	0	120	274	225	85	15	1	0
7	0	720	1764	1624	735	175	21	1

第二类 $\left\{{n\atop k}\right\}$（划分成 k 个非空块）：

$n\backslash k$	0	1	2	3	4	5	6	7
0	1	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0

$n\backslash k$	0	1	2	3	4	5	6	7
2	0	1	1	0	0	0	0	0
3	0	1	3	1	0	0	0	0
4	0	1	7	6	1	0	0	0
5	0	1	15	25	10	1	0	0
6	0	1	31	90	65	15	1	0
7	0	1	63	301	350	140	21	1

8.2.006 Eulerian 数

Eulerian 数 $\left\langle {n \atop k} \right\rangle$ 计 n 元排列中恰有 k 个下降位置（$a_i > a_{i+1}$）的排列数，下标 k 从 0 起。把最大元素插入长度 $i-1$ 排列的 i 个空隙：插进已有的某个下降处或末尾（保持下降数，共 $j+1$ 种），或插进上升处新增一个下降（$i-j$ 种），得递推 $\left\langle {i \atop j} \right\rangle = (j+1)\left\langle {i-1 \atop j} \right\rangle + (i-j)\left\langle {i-1 \atop j-1} \right\rangle$。有些资料把递增段数 $k+1$ 作为列下标，本实现始终用下降位置数 k。

```cpp
template <class T> vector<vector<T>> eulerian(int n)
{
    // n 是最大排列长度；返回第 0..n 行 Eulerian 数，列下标是下降位置数。
    assert(n >= 0); // 调试检查，可删。
    vector<vector<T>> a(n + 1, vector<T>(n + 1));
    a[0][0] = 1;
    for (int i = 1; i <= n; i++)
    {
        for (int j = 0; j < i; j++)
        {
            if (j)
            {
                a[i][j] += T(i - j) * a[i - 1][j - 1];
            }
            a[i][j] += T(j + 1) * a[i - 1][j];
        }
    }
    return a;
}
```

`eulerian<T>(n)` 返回三角表，下标为元素数与下降数，$O(n^2)$ 时间与空间。Eulerian 数与 Worpitzky 恒等式、$\sum k^n$ 幂和相关联；若只需幂和本身，8.2.007 Bernoulli 数与幂和 的 Faulhaber 公式更直接。它与第二类 Stirling 数可互相线性表出，按题目要「下降数分布」还是「块数分布」二选一。

前几行速查 $\left\langle {n \atop k} \right\rangle$（$k$ 为下降位置数，从 0 起；每行对称、行和为 $n!$）：

$n\backslash k$	0	1	2	3	4	5	6
1	1						
2	1	1					
3	1	4	1				
4	1	11	11	1			

$n\backslash k$	0	1	2	3	4	5	6
5	1	26	66	26	1		
6	1	57	302	302	57	1	
7	1	120	1191	2416	1191	120	1

8.2.007 Bernoulli 数与幂和

本条采用 $B_1=+\frac{1}{2}$ 的 Bernoulli 约定，由差分表以 $a[j]=\frac{1}{j+1}$ 起、反复做 $a[j-1]\leftarrow j(a[j-1]-a[j])$ 得到。对应的 Faulhaber 公式是 $\sum_{i=1}^{n} i^m=\frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k}B_k n^{m+1-k}$；例如 $m=1$ 给出 $\frac{n^2+n}{2}$。因递推含除法，系数类型 `T` 必须支持除法，模素数时各分母须可逆。

```
template <class T> vector<T> bernoulli(int n)
{
    // n 是要计算的最大下标；返回 B_0 到 B_n，采用 B_1=+1/2 的约定。
    assert(n >= 0); // 调试检查，可删。
    vector<T> a(n + 1), b(n + 1);
    for (int m = 0; m <= n; m++)
    {
        a[m] = T(1) / T(m + 1);
        for (int j = m; j; j--)
        {
            a[j - 1] = T(j) * (a[j - 1] - a[j]);
        }
        b[m] = a[0];
    }
    return b;
}
```

`bernoulli<T>(n)` 返回 $B_0,\dots,B_n$，$O(n^2)$ 时间、$O(n)$ 空间。本条符号约定下前四项为 $1,\frac{1}{2},\frac{1}{6},0$，可优先用 $B_1=+\frac{1}{2}$ 排查约定混用。要算 $\sum_{i=1}^{n} i^m$ 时，先取 $B_0,\dots,B_m$ 再代入 Faulhaber 公式；系数预处理后可把它作为关于 n 的 $m+1$ 次多项式求值。需要更快地求整段 Bernoulli 数时，用 EGF $\frac{x}{e^x-1}$ 配 10.2.003 乘法逆元。

8.2.008 错排数

错排数 D_n 是没有任何元素留在原位的排列数（空排列计为 1）。考虑元素 1 被放到位置 j：若 j 上的元素回放到位置 1（对换），剩下是 $n-2$ 个元素的错排；否则把「位置 1 禁放 1」的约束转嫁给 j，化为 $n-1$ 个元素的错排。j 有 $n-1$ 种取法，故 $D_n=(n-1)(D_{n-1}+D_{n-2})$，$D_0=1,D_1=0$。

```
template <class T> vector<T> derangements(int n)
{
    // n 是最大规模；返回 d[0..n]，d[i] 是 i 个元素无固定点排列数。
    assert(n >= 0); // 调试检查，可删。
    vector<T> d(n + 1);
    d[0] = 1;
    if (n >= 1)
```

```
    {
        d[1] = 0;
    }
    for (int i = 2; i <= n; i++)
    {
        d[i] = T(i - 1) * (d[i - 1] + d[i - 2]);
    }
    return d;
}
```

`derangements<T>(n)` 返回 `d[0..n]`，`T` 支持加法、乘法与整数构造即可（大整数或模整数）；$O(n)$ 时间与输出空间。前七项为 $1, 0, 1, 2, 9, 44, 265$。等价的容斥闭式是 $D_n = n!\sum_{k=0}^{n}\frac{(-1)^k}{k!}$，模素数下想 $O(1)$ 单点查询就配 8.1.001 阶乘、逆阶乘与组合数 预处理；只需整段小 n 时本条线性递推更短。

8.2.009 分拆数与五边形数

分拆数 $p(n)$ 数把 n 写成无序正整数之和的方案数，生成函数为 $\prod_{k\ge1}(1-x^k)^{-1}$。对其倒数 $\prod(1-x^k)$ 用 Euler 五边形定理 $\prod_{k\ge1}(1-x^k) = \sum_j(-1)^j x^{j(3j-1)/2}$（$j$ 取全体整数），只在广义五边形数 $g=\frac{k(3k\pm1)}{2}$ 处非零，于是得正负交替递推 $p(n)=\sum_{k\ge1}(-1)^{k-1}(p(n-g_k^-)+p(n-g_k^+))$。每个 n 只枚举到 $g\le n$ 的 $O(\sqrt{n})$ 个五边形数，把朴素 $O(n^2)$ 降到 $O(n\sqrt{n})$。

```
template <class T> vector<T> partitions(int n)
{
    // n 是要计算的最大整数；返回 p(0) 到 p(n)。
    assert(n >= 0); // 调试检查，可删。
    vector<T> a(n + 1);
    a[0] = 1;
    for (int i = 1; i <= n; i++)
    {
        for (int k = 1;; k++)
        {
            int x = k / 2;
            int y = 3 * k - 1;
            if (k & 1)
            {
                x = k;
                y /= 2;
            }
            if (x > i / y)
            {
                break;
            }
            int g = x * y;
            if (k & 1)
            {
                a[i] += a[i - g];
            }
            else
            {
```

```
                a[i] -= a[i - g];
            }
            if (g <= i - k)
            {
                g += k;
                if (k & 1)
                {
                    a[i] += a[i - g];
                }
                else
                {
                    a[i] -= a[i - g];
                }
            }
        }
    }
    return a;
}
```

`partitions<T>(n)` 返回 $p(0),\dots,p(n)$，固定查验值为 $p(0)=1$、$p(5)=7$、$p(10)=42$；$O(n\sqrt{n})$ 时间、$O(n)$ 空间。这是无限制的整数分拆；若限制份数或每份大小（如「至多 k 份」「各份互异」），改用完全背包式的二维 DP 或对应生成函数。模素数下需要更长前缀时，可按五边形定理写出分母的稀疏系数，再调用 10.2.003 乘法逆元在 $O(n\log n)$ 时间求逆。

8.2.010 有根树、无根树计数

树计数要先分清是否有根、节点是否带标号、儿子是否有序。本条实现最干净的一类——带标号无根树，即 Cayley 公式：n 个带标号点的无根树共 n^{n-2} 棵；若根标号预先固定，每棵无根树只有一种置根方式，数量仍为 n^{n-2}；若把“根是哪一个标号点”也视为结构的一部分，数量才是 $n\cdot n^{n-2}=n^{n-1}$。`cayley` 返回无根树数和后一种“根被区分”的有根树数，$n=1$ 单独处理。无标号树不在本条闭式范围内。

```
namespace tree_count
{
int qpow(int a, int n, int mod)
{
    // a、n、mod 是底数、非负指数和正模数；返回模幂。
    assert(n >= 0 && mod > 0); // 调试检查，可删。
    a %= mod;
    if (a < 0)
    {
        a += mod;
    }
    int r = 1 % mod;
    while (n)
    {
        if (n & 1)
        {
            r = combMulMod(r, a, mod);
        }
```

```
        a = combMulMod(a, a, mod);
        n >>= 1;
    }
    return r;
}

array<int, 2> cayley(int n, int mod)
{
    // n 是有标号点数，mod 是正模数；返回 {无根树数，根标号被区分的有根树数}。
    assert(n >= 1 && mod > 0); // 调试检查，可删。
    int unrooted = n == 1 ? 1 % mod : qpow(n % mod, n - 2, mod);
    int rooted = qpow(n % mod, n - 1, mod);
    return {unrooted, rooted};
}
}
```

`cayley(n, mod)` 返回的两维依次是无根树数、根标号被区分的有根树数，固定查验值为 `cayley(4,1000000007)` 返回 `{16,64}`。两次模幂共 $O(\log n)$ 时间、$O(1)$ 空间；若根固定为某个已知标号，答案取第一项。Cayley 公式只适用于完全图的标号树；给定图上数生成树要用 8.2.011 Matrix-Tree / Kirchhoff 生成树计数 的 Laplacian 行列式，带度数限制的标号树则用 Prüfer 序列计数。

8.2.011 Matrix-Tree / Kirchhoff 生成树计数

Matrix-Tree（Kirchhoff）定理把生成树计数化成行列式。无向图的 Laplacian $L = D - A$（D 度数对角阵、A 邻接矩阵）任意删掉同编号的一行一列，剩下 $(n-1)$ 阶子式的行列式就是生成树数量；证明来自 Cauchy–Binet 展开到关联矩阵。有向图则按朝向选入度或出度 Laplacian，`arboCount` 用入度版统计从 `root` 向外、每个非根点恰有一条入边的树形图数。平行边按重数累加；自环在 Laplacian 的度数项与邻接项中抵消，不属于生成树。行列式在有限域 `Z` 上用高斯消元 `detField` 求：逐列找非零主元（找不到说明奇异，返回 0），换行时翻转符号，消元后累乘对角元。

```
// 依赖 1.1.008 静态取模类
Z detField(vector<vector<Z>> a)
{
    // a 是有限域上的方阵；返回其行列式。
    int n = a.size();
    for (const auto &row : a)
    {
        assert((int)row.size() == n); // 调试检查，可删。
    }
    Z ans = 1;
    for (int c = 0; c < n; c++)
    {
        int p = c;
        while (p < n && a[p][c] == Z(0))
        {
            p++;
        }
        if (p == n)
        {
```

```
            return 0;
        }
        if (p != c)
        {
            swap(a[p], a[c]);
            ans = -ans;
        }
        ans *= a[c][c];
        Z iv = a[c][c].inv();
        for (int i = c + 1; i < n; i++)
        {
            Z q = a[i][c] * iv;
            for (int j = c; j < n; j++)
            {
                a[i][j] -= q * a[c][j];
            }
        }
    }
    return ans;
}

Z spanTreeCnt(int n, const vector<pair<int, int>> &e)
{
    // n 是点数，e 是无向边端点列表且允许重边；返回生成树数量。
    assert(n >= 0); // 调试检查，可删。
    for (auto [u, v] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
    }
    if (n <= 1)
    {
        return 1;
    }
    vector a(n - 1, vector<Z>(n - 1));
    for (auto [u, v] : e)
    {
        if (u < n - 1)
        {
            a[u][u] += 1;
        }
        if (v < n - 1)
        {
            a[v][v] += 1;
        }
        if (u < n - 1 && v < n - 1)
        {
            a[u][v] -= 1;
            a[v][u] -= 1;
        }
    }
    return detField(move(a));
}
```

```

Z arboCount(int n, int root, const vector<pair<int, int>> &e)
{
    // n 是点数，root 是根，e 是有向边端点列表且允许重边；返回从 root 向外的生成树数量。
    assert(n >= 1 && 0 <= root && root < n); // 调试检查，可删。
    for (auto [u, v] : e)
    {
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
    }
    if (n == 1)
    {
        return 1;
    }
    vector<int> id(n, -1);
    for (int u = 0, k = 0; u < n; u++)
    {
        if (u != root)
        {
            id[u] = k++;
        }
    }
    vector a(n - 1, vector<Z>(n - 1));
    for (auto [u, v] : e)
    {
        if (v == root)
        {
            continue;
        }
        a[id[v]][id[v]] += 1;
        if (u != root)
        {
            a[id[v]][id[u]] -= 1;
        }
    }
    return detField(move(a));
}
```

先准备 1.1.008 静态取模类的素数域 `Z`。`spanTreeCnt(n, e)` 接受点编号为 `0..n-1` 的无向边，`arboCount(n, root, e)` 接受有向边 $u \to v$ 并统计从 `root` 向外的树形图；空无向图 `n=0` 约定返回 1，有向接口要求 `n>=1`。自环可保留在边表中并自动抵消，平行边分别计数，非单点图存在孤立点时行列式为 0。整体 $O(n^3 + m)$ 时间、$O(n^2)$ 额外空间。完全图上有闭式时优先 8.2.010 有根树、无根树计数 的 Cayley 公式；只有一般图才动用行列式。5.9.003 BEST 定理与有向欧拉回路计数 需要朝根的树形图，因此应反转原图全部边后调用 `arboCount`。

下面用三角形、孤立点和有向图各核对一个已知答案：

```
signed main()
{
    assert(spanTreeCnt(3, {{0, 1}, {1, 2}, {2, 0}}).val() == 3);
    assert(spanTreeCnt(3, {{0, 1}}).val() == 0);
    assert(arboCount(3, 0, {{0, 1}, {0, 2}, {1, 2}}).val() == 2);
```

```
}
```

8.2.012 Pólya / Burnside 置换计数

Burnside 引理说群 G 作用下的轨道数（本质不同方案数）等于各元素不动点数的平均：$|X/G| = \frac{1}{|G|}\sum_{g\in G}|\mathrm{Fix}(g)|$。用 c 种颜色给位置染色时，一个置换 g 的染色不动点要求同一循环上位置同色，故 $|\mathrm{Fix}(g)| = c^{\mathrm{cyc}(g)}$，这就是 Pólya 计数把循环长度代入库存多项式的常数版本。`burnside` 对群里每个位置置换数出循环数 `cyc`，累加 c^{cyc}，最后除以 $|G|$。传入的 `group` 必须是真正的群（含单位元、对合成封闭），否则平均无意义。

```
namespace polya
{
template <class T = long long>
T burnside(const vector<vector<int>> &group, int colors)
{
    // group 中每项是位置置换，colors 是颜色数；返回本质不同染色数。
    static_assert(numeric_limits<T>::is_integer && numeric_limits<T>::is_signed &&
                  sizeof(T) >= sizeof(long long));
    assert(!group.empty() && colors >= 0); // 调试检查，可删。
    int n = group[0].size();
    T sum = 0;
    for (const auto &p : group)
    {
        assert((int)p.size() == n); // 调试检查，可删。
        vector<int> cnt(n);
        for (int x : p)
        {
            assert(0 <= x && x < n); // 调试检查，可删。
            cnt[x]++;
        }
        for (int x : cnt)
        {
            assert(x == 1); // 调试检查，可删。
        }
        vector<int> vis(n);
        int cyc = 0;
        for (int i = 0; i < n; i++)
        {
            if (vis[i])
            {
                continue;
            }
            cyc++;
            int x = i;
            while (!vis[x])
            {
                vis[x] = 1;
                x = p[x];
            }
```

```
        }
        T ways = 1;
        for (int i = 0; i < cyc; i++)
        {
            ways *= colors;
        }
        sum += ways;
    }
    int sz = group.size();
    assert(sum % sz == 0); // 调试检查，可删。
    return sum / sz;
}
}
```

`burnside(group, colors)` 要求 `group` 是完整有限群，并返回本质不同染色数；调用者须保证幂和总和能由 `T` 表示，需要 128 位精确计数时调用 `burnside<__int128>`。算法访问 $O(|G|n)$ 个置换位置。结果超过 128 位时换 14.1.001 高精度整数（BigInt）；群规模本身指数级时应改用循环指标解析求和。

四个位置只考虑旋转、使用两种颜色时共有 6 个轨道，可用这组固定群检查循环分解与平均数：

```
signed main()
{
    // 无参数；核对四点旋转群的二染色轨道数，返回 0。
    vector<vector<int>> rot{
        {0, 1, 2, 3}, {1, 2, 3, 0},
        {2, 3, 0, 1}, {3, 0, 1, 2}};
    assert(polya::burnside(rot, 2) == 6);
    return 0;
}
```

8.2.013 杨氏矩阵与钩子公式

形状为分拆 λ（行长非增）的 Young 图，其标准 Young 表（每行每列严格递增地填 $1..n$）数量由钩长公式给出：$f^\lambda = \dfrac{n!}{\prod_{格 c} h(c)}$，其中格 c 的钩长 $h(c) = 1 +$ 右侧同行格数 $+$ 下方同列格数。直接形成 $n!$ 可能在除掉钩长前溢出。代码用线性筛得到最小质因子，先按 Legendre 公式统计 $n!$ 的质因数指数，再逐格分解钩长并扣除指数，最后只乘答案真正保留的质因子；列高 `col[j]` 让钩长可在 $O(1)$ 时间写成 `row[i]-j+col[j]-i-1`。

```
namespace young_tab
{
template <class T = long long>
T youngTab(const vector<int> &row)
{
    // row 是非增的行长分拆；返回标准 Young 表数量。
    static_assert(numeric_limits<T>::is_integer && numeric_limits<T>::is_signed &&
                  sizeof(T) >= sizeof(long long));
    size_t cells = 0;
    for (int i = 0; i < (int)row.size(); i++)
```

```
    {
        assert(row[i] >= 0); // 调试检查，可删。
        if (i)
        {
            assert(row[i - 1] >= row[i]); // 调试检查，可删。
        }
        cells += row[i];
    }
    int n = (int)cells;
    vector<int> spf(n + 1), ps;
    for (int i = 2; i <= n; i++)
    {
        if (!spf[i])
        {
            spf[i] = i;
            ps.push_back(i);
        }
        for (int p : ps)
        {
            if (p > spf[i] || i > n / p)
            {
                break;
            }
            spf[i * p] = p;
        }
    }
    vector<int> e(n + 1);
    for (int p : ps)
    {
        for (int x = n / p; x; x /= p)
        {
            e[p] += x;
        }
    }
    vector<int> col(row.empty() ? 0 : row[0]);
    for (int x : row)
    {
        for (int j = 0; j < x; j++)
        {
            col[j]++;
        }
    }
    for (int i = 0; i < (int)row.size(); i++)
    {
        for (int j = 0; j < row[i]; j++)
        {
            int x = row[i] - j + col[j] - i - 1;
            while (x > 1)
            {
                int p = spf[x];
                e[p]--;
                x /= p;
```

```
            }
        }
    }
    T ans = 1;
    for (int p : ps)
    {
        assert(e[p] >= 0); // 调试检查，可删。
        for (int i = 0; i < e[p]; i++)
        {
            ans *= p;
        }
    }
    return ans;
}

}
```

`youngTab(row)` 要求行长非负且非增，空形状返回 1；调用者须保证结果能由 `T` 表示，需要 128 位时调用 `youngTab<__int128>`。线性筛、列高与指数表使用 $O(n+\lambda_1)$ 空间，全部分解最坏为 $O(n\log n)$ 时间。超过 128 位的精确答案换 14.1.001 高精度整数（BigInt）。

下面同时检查非矩形小形状与 2×10 矩形；后者的标准 Young 表数恰为 Catalan 数 $C_{10}=16796$：

```
signed main()
{
    // 无参数；核对两个 Young 图形状的标准表数量，返回 0。
    assert(young_tab::youngTab({2, 1}) == 2);
    assert(young_tab::youngTab({10, 10}) == 16796);
    return 0;
}
```

8.2.014 固定 k 的第一类 Stirling 数

有符号第一类 Stirling 数的指数生成函数为 $\frac{\log(1+x)^k}{k!}=\sum_{n\ge k}s(n,k)\frac{x^n}{n!}$。先写出 $\log(1+x)=\sum_{i\ge 1}(-1)^{i-1}\frac{x^i}{i}$，调用多项式快速幂，最后把第 n 项乘回 $n!/k!$，即可一次得到固定 `k` 的整列；符号与 8.2.005 Stirling 数 的 `stirling1` 完全一致。

```
// 依赖 1.1.008 静态取模类
// 依赖 7.4.004 二次剩余
// 依赖 10.2.002 乘法、除法、取模
// 依赖 10.2.004 导数、积分
// 依赖 10.2.005 ln、exp、开根、快速幂
vector<Z> stirling1K(int n, int k)
{
    // n 是最大行号，k 是固定列号；返回 s(k,k) 到 s(n,k)，模数由 Z 决定。
    assert(0 <= k && k <= n && n < mod); // 调试检查，可删。
    Poly f(n + 1);
    for (int i = 1; i <= n; i++)
    {
```

```
        f[i] = (i & 1 ? Z(1) : Z(-1)) / Z(i);
    }
    Poly g = fpsPow(f, k, n + 1);
    Z fk = 1;
    for (int i = 1; i <= k; i++)
    {
        fk *= i;
    }
    Z ifk = fk.inv();
    Z fac = 1;
    vector<Z> ans(n - k + 1);
    for (int i = 0; i <= n; i++)
    {
        if (i >= k)
        {
            ans[i - k] = g[i] * fac * ifk;
        }
        fac *= i + 1;
    }
    return ans;
}
```

本函数依赖 10.2.001 多项式封装的 `Poly/Z` 与 10.2.005 `ln`、`exp`、开根、快速幂的 `fpsPow`。`stirling1K(n,k)[i-k]` 等于 $s(i,k)$，要求 $n <$ mod 以保证积分分母和阶乘可逆；时间 $O(M(n))$、输出与工作空间 $O(n)$。若只查一个小 `n,k`，直接用 8.2.005 Stirling 数 的二维递推更简单。

8.2.015 固定 k 的第二类 Stirling 数

固定块数的第二类 Stirling 数满足普通生成函数 $\sum_{n\ge k} S(n,k)x^{n-k} = \prod_{i=1}^{k}(1-ix)^{-1}$。代码用平衡乘积树构造分母，避免从左到右相乘退化成二次复杂度，再调用 FPS 逆元截断到 `n-k+1` 项；第 t 项就是 $S(k+t,k)$。

```
// 依赖 8.2.014 固定 k 的第一类 Stirling 数
// 依赖 10.2.003 乘法逆元
vector<Z> stirling2K(int n, int k)
{
    // n 是最大行号，k 是固定列号；返回 S(k,k) 到 S(n,k)，模数由 Z 决定。
    assert(0 <= k && k <= n); // 调试检查，可删。
    auto build = [&](auto &&self, int l, int r) -> Poly
    {
        // self 是递归自身，l,r) 是因子下标；返回这一段 (1-ix) 的乘积。
        if (r - l == 1)
        {
            return Poly{1, -Z(l)};
        }
        int m = (l + r) / 2;
        return self(self, l, m) * self(self, m, r);
    };
    Poly den = k == 0 ? Poly{1} : build(build, 1, k + 1);
    Poly ans = fpsInv(den, n - k + 1);
```

```
    return vector<Z>(ans.begin(), ans.end());
}
```

本函数依赖 [10.2.001 多项式封装的乘法和 10.2.003 乘法逆元的 `fpsInv`。构造乘积树为 $O(M(k)\log k)$，求逆为 $O(M(n-k))$，峰值空间 $O(n+k)$；`k=0` 返回 $S(0,0)=1$ 及其后的全零项。若只需固定 `n` 的整行，仍用 8.2.005 Stirling 数 或第二类的有限和公式。

下面直接检查四条可手算的固定列，不在正文再放二维递推作参考实现：

```
signed main()
{
    assert(stirling1K(5, 1) == vector<Z>({1, -1, 2, -6, 24}));
    assert(stirling1K(6, 2) == vector<Z>({1, -3, 11, -50, 274}));
    assert(stirling2K(5, 1) == vector<Z>({1, 1, 1, 1, 1}));
    assert(stirling2K(6, 2) == vector<Z>({1, 3, 7, 15, 31}));
    return 0;
}
```

8.2.016 小素数大下标第一类 Stirling 数

当素数 p 很小而 n,k 可到 64 位时，二维递推不再可行。先用 Pascal 表保存 $0\le a,b<p$ 的组合数，`comb` 按 Lucas 定理逐位查询。对 $n=ip+j$，再把 $k-i=a(p-1)+b$ 调整为公式要求的余数范围，有 $s(n,k)=(-1)^{i+a}\binom{i}{a}s(j,b)\pmod p$；越界条件直接贡献 0。低位 $s(j,b)$ 只需预处理到 $p-1$。

```
class SmallPComb
{
  protected:
    // p 是小素数，c[i][j] 是 C(i,j) mod p，其中 0<=j<=i<p。
    int p;
    vector<vector<int>> c;

    int comb(int n, int k) const
    {
        // n、k 是非负 64 位组合数参数；返回 C(n,k) mod p，k 越界时返回 0。
        if (k < 0 || k > n)
        {
            return 0;
        }
        int ans = 1;
        while (n)
        {
            int a = n % p;
            int b = k % p;
            if (b > a)
            {
                return 0;
            }
            ans = ans * c[a][b] % p;
            n /= p;
            k /= p;
```

```
        }
        return ans;
    }

  public:
    explicit SmallPComb(int p) : p(p)
    {
        // p 是可承担 O(p^2) 表的素数；建立一位 Lucas 所需的 Pascal 三角。
        assert(2 <= p && p <= numeric_limits<int>::max()); // 调试检查，可删。
        c.resize(p);
        c[0] = {1};
        for (int i = 1; i < p; i++)
        {
            c[i].assign(i + 1, 0);
            for (int j = 0; j <= i; j++)
            {
                int x = 0;
                if (j < i)
                {
                    x += c[i - 1][j];
                }
                if (j)
                {
                    x += c[i - 1][j - 1];
                }
                c[i][j] = x % p;
            }
        }
    }
};

class Stirling1P : public SmallPComb
{
    // s[i][j] 是有符号第一类 Stirling 数 s(i,j) mod p，其中 0<=j<=i<p。
    vector<vector<int>> s;

  public:
    explicit Stirling1P(int p) : SmallPComb(p), s(p)
    {
        // p 是小素数；建立第一类低位表与 Lucas 表。
        s[0] = {1};
        for (int i = 1; i < p; i++)
        {
            s[i].assign(i + 1, 0);
            for (int j = 0; j <= i; j++)
            {
                int x = 0;
                if (j)
                {
                    x += s[i - 1][j - 1];
                }
                if (j < i)
```

```
                {
                    x += s[i - 1][j] * (p - i + 1) % p;
                }
                s[i][j] = x % p;
            }
        }
    }

    int get(int n, int k) const
    {
        // n、k 是非负 64 位下标；返回有符号 s(n,k) mod p，k>n 时返回 0。
        assert(n >= 0); // 调试检查，可删。
        if (k < 0 || k > n)
        {
            return 0;
        }
        int i = n / p;
        int j = n % p;
        if (i > k)
        {
            return 0;
        }
        int a = (k - i) / (p - 1);
        int b = (k - i) % (p - 1);
        if (b == 0 && j > 0)
        {
            b += p - 1;
            a--;
        }
        if (a < 0 || i < a || b > j)
        {
            return 0;
        }
        int ans = comb(i, a) * s[j][b] % p;
        if (((i + a) & 1) && ans)
        {
            ans = p - ans;
        }
        return ans;
    }
};
```

构造 `Stirling1P q(p)` 后 `q.get(n,k)` 在 $O(\log_p n)$ 时间返回有符号 $s(n,k) \bmod p$；预处理为 $O(p^2)$ 时间与空间，表项显式使用 `int`，避免 `#define int long long` 把 $p = 5000$ 时的内存翻倍。`p` 必须是不超过 `int` 上界的小素数，且内存要能容纳两张三角表；中等规模整表仍优先 8.2.005 Stirling 数。

8.2.017 小素数大下标第二类 Stirling 数

第二类需要不同的周期式，不能复用第一类的符号公式。对 $n < p$ 直接查低位表；否则写 $k = ip + j$，再把 $n - i = a(p-1) + b$ 调整到 $1 \le b \le p-1$。当 $b < p-1$ 时答案为 $\binom{a}{i} S(b, j)$；当 $b = p-1$

时，`j=0` 的答案是 $\binom{a}{i-1}$，否则为 $\binom{a}{i}S(p-1,j)$。

```
// 依赖 8.2.016 小素数大下标第一类 Stirling 数
class Stirling2P : public SmallPComb
{
    // s[i][j] 是第二类 Stirling 数 S(i,j) mod p，其中 0<=j<=i<p。
    vector<vector<int>> s;

  public:
    explicit Stirling2P(int p) : SmallPComb(p), s(p)
    {
        // p 是小素数；建立第二类低位表与 Lucas 表。
        s[0] = {1};
        for (int i = 1; i < p; i++)
        {
            s[i].assign(i + 1, 0);
            for (int j = 0; j <= i; j++)
            {
                int x = 0;
                if (j)
                {
                    x += s[i - 1][j - 1];
                }
                if (j < i)
                {
                    x += s[i - 1][j] * j % p;
                }
                s[i][j] = x % p;
            }
        }
    }

    int get(int n, int k) const
    {
        // n、k 是非负 64 位下标；返回 S(n,k) mod p，k>n 时返回 0。
        assert(n >= 0); // 调试检查，可删。
        if (k < 0 || k > n)
        {
            return 0;
        }
        if (n < p)
        {
            return s[n][k];
        }
        int i = k / p;
        int j = k % p;
        int a = (n - i) / (p - 1);
        int b = (n - i) % (p - 1);
        if (b == 0)
        {
            b = p - 1;
            a--;
```

```
        }
        if (a < 0 || j > b)
        {
            return 0;
        }
        if (b < p - 1)
        {
            return comb(a, i) * s[b][j] % p;
        }
        if (j == 0)
        {
            return comb(a, i - 1);
        }
        return comb(a, i) * s[p - 1][j] % p;
    }
};
```

`Stirling2P q(p)` 同样以 $O(p^2)$ 时间与空间预处理，`q.get(n,k)` 为 $O(\log_p n)$；它与 8.2.016 小素数大下标第一类 Stirling 数 只共享 `SmallPComb` 的 Lucas 表，低位递推和高位公式彼此独立。固定 k 的整列输出用 8.2.015 固定 k 的第二类 Stirling 数，不要为单次大下标查询构造长多项式。

模 5 时先用小值核对定义递推，再用 $s(n,n-1)=-\binom{n}{2}$ 与 $S(n,2)=2^{n-1}-1$ 检查大下标分支：

```
signed main()
{
    int n = 1000000000000000003LL;
    Stirling1P a(5);
    Stirling2P b(5);
    assert(a.get(6, 2) == 4);
    assert(a.get(n, n - 1) == 2);
    assert(b.get(6, 3) == 0);
    assert(b.get(n, 2) == 3);
    cout << "OK\n";
    return 0;
}
```

8.2.018 LGV 引理与不相交格路计数

在有限有向无环图上，给定有序且各自互异的起点 $A_1,\dots,A_k$ 与终点 $B_1,\dots,B_k$。令 M_{ij} 为 A_i 到 B_j 的路径权和，路径权是各边权之积，空路径权为 1。行列式展开中，两条相交路径在首个交点交换后缀，权值不变、排列符号相反，故相交项两两抵消：

$$\det M=\sum_{\pi\in S_k}\operatorname{sgn}(\pi)\sum_{\substack{P_i:A_i\to B_{\pi(i)}\\ P_i\text{ 两两顶点不交}}}\prod_i w(P_i).$$

只有恒等排列能够形成不相交路径组时，结果才是对应配对的普通计数；边不相交不足以满足此结论。网格单调格路通常按边界顺序排列端点以保证该条件。实现先对每个起点做 DAG 路径 DP，再复用 8.2.011 Matrix-Tree 的有限域行列式 `detField`。

```cpp
// 依赖 8.2.011 Matrix-Tree / Kirchhoff 生成树计数
Z lgvPaths(const vector<vector<pair<int, Z>>> &g, const vector<int> &ord,
           const vector<int> &s, const vector<int> &t)
{
    // g[u] 保存 DAG 的 {终点, 边权}, ord 是全部顶点的正向拓扑排列;
    // s、t 是等长有序起终点表; 返回顶点不相交路径组的带符号权和。
    int n = g.size(), k = s.size();
    assert((int)ord.size() == n && (int)t.size() == k);
    vector a(k, vector<Z>(k));
    for (int i = 0; i < k; i++)
    {
        vector<Z> dp(n);
        dp[s[i]] = 1;
        for (int u : ord)
        {
            for (auto [v, w] : g[u])
            {
                dp[v] += dp[u] * w;
            }
        }
        for (int j = 0; j < k; j++)
        {
            a[i][j] = dp[t[j]];
        }
    }
    return detField(move(a));
}
```

`g` 的顶点编号为 $0, n)$，`ord` 必须是全部顶点的正向拓扑排列，边权使用 [1.1.008 静态取模类的 `Z`。设图有 m 条边、k 对端点，时间为 $O(k(n+m)+k^3)$，额外空间为 $O(n+k^2)$；空端点表返回 1。若单条路径数已有组合数闭式，直接构造 M 后调用 `detField` 即可，省去建图和 DP。

下面是两行三列、只向右或向下走的网格。起点为左侧两格，终点为右侧两格；唯一顶点不交路径组是两条水平线。

```cpp
signed main()
{
    // 无参数; 验证有序端点的单调网格路径组计数, 返回 0。
    vector<vector<pair<int, Z>>> g(6);
    g[0] = {{1, 1}, {3, 1}};
    g[1] = {{2, 1}, {4, 1}};
    g[2] = {{5, 1}};
    g[3] = {{4, 1}};
    g[4] = {{5, 1}};
    assert(lgvPaths(g, {0, 1, 2, 3, 4, 5}, {0, 3}, {2, 5}) == Z(1));
}
```

8.3 生成与构造

8.3.001 排列组合生成

这四种枚举都短到不该封装，直接展开在使用处即可。排序后反复 `next_permutation` 无重地枚举多重集合的不同排列；`0..2^n-1` 的二进制掩码天然枚举全部子集；Gosper Hack（`x&-x` 取最低位、`s+lowbit` 进位后补齐低位）在相同 popcount 的掩码间按数值递增跳转，等价于组合的 colex 顺序，并非元素下标序列的字典序；Gray 编码 $g = x \oplus (x \gg 1)$ 使相邻编号只差一位，解码则对不断右移的前缀连续异或还原。下面的 `main` 把四段写在一起，末尾 `assert(y == x)` 验证 Gray 编解码互逆。

```
signed main()
{
    // 无参数；直接展示排列、子集、固定大小子集与 Gray Code 的展开写法，返回 0。
    vector<int> a{1, 1, 2};
    sort(a.begin(), a.end());
    do
    {
        // 此处直接处理当前字典序排列 a。
    } while (next_permutation(a.begin(), a.end()));

    int n = 5;
    for (int s = 0; s < (1LL << n); s++)
    {
        // s 的第 i 位表示是否选择编号 i 的元素。
    }

    int k = 3;
    if (k == 0)
    {
        int s = 0;
        // 空集是唯一的 0 元子集。
    }
    else
    {
        int s = (1LL << k) - 1;
        while (s < (1LL << n))
        {
            // 此处直接处理当前恰含 k 个元素的子集 s。
            int x = s & -s;
            int y = s + x;
            s = y | ((s ^ y) >> 2) / x;
        }
    }

    int x = 23;
    int g = x ^ (x >> 1);
    int y = g;
    for (int t = y >> 1; t; t >>= 1)
    {
        y ^= t;
    }
```

```
    assert(y == x);
}
```

直接把对应循环抄到 `solve`/`main` 里，不要包装成回调。示例要求 $0 \le n < 63$、$0 \le k \le n$，Gray 编解码按非负整数处理。设不同排列数为 P，排列枚举 $O(Pn)$；全部子集 $O(2^n)$；固定大小子集 $O(\binom{n}{k})$；Gray 编码 $O(1)$、解码 $O(\log x)$，都只用 $O(1)$ 额外空间。只需要「第几个排列」而非全部枚举时用 8.3.002 康托展开与逆展开 直接跳到目标；子集枚举里要按权和分组则配合状压 DP。

8.3.002 康托展开与逆展开

康托展开把排列映到它的字典序排名（从 0 起）：第 i 位的权是 $(n-1-i)!$，系数是「p_i 右边比它小的元素个数」，累加即排名。逆展开反过来，从高位到低位用 $k/(i-1)!$ 选出当前剩余元素里第几小的、再取模递降。`cantor` 用树状数组维护右侧更小计数把逐位统计降到 $O(n \log n)$；`invCantor` 每步从候选 `vector` 删除选中元素，故为 $O(n^2)$。两向都要求元素互异，且排名从 0 还是 1 起必须全程统一。

```
unsigned long long cantor(const vector<int> &p)
{
    // p 是由 0 到 n-1 组成的排列；返回其从零开始的字典序排名。
    int n = p.size();
    assert(n <= 20); // 调试检查，可删。
    vector<int> seen(n);
    for (int x : p)
    {
        assert(0 <= x && x < n && !seen[x]); // 调试检查，可删。
        seen[x] = 1;
    }
    vector<unsigned long long> f(n + 1, 1);
    for (int i = 1; i <= n; i++)
    {
        f[i] = f[i - 1] * i;
    }
    vector<int> bit(n + 1);
    auto add = [&](int x)
    {
        // x 是要加入的排列值；把它在树状数组中的计数加一。
        for (++x; x <= n; x += x & -x)
        {
            bit[x]++;
        }
    };
    auto sum = [&](int x)
    {
        // x 是值域右端；返回已加入值中严格小于 x 的个数。
        int ans = 0;
        for (; x; x -= x & -x)
        {
            ans += bit[x];
        }
        return ans;
```

```
    };
    unsigned long long ans = 0;
    for (int i = n - 1; i >= 0; i--)
    {
        ans += f[n - 1 - i] * sum(p[i]);
        add(p[i]);
    }
    return ans;
}

vector<int> invCantor(int n, unsigned long long k)
{
    // n 是排列长度，k 是从零开始的排名；返回由 0 到 n-1 组成的对应排列。
    assert(0 <= n && n <= 20); // 调试检查，可删。
    vector<unsigned long long> f(n + 1, 1);
    for (int i = 1; i <= n; i++)
    {
        f[i] = f[i - 1] * i;
    }
    assert(k < f[n]); // 调试检查，可删。
    vector<int> a(n), ans;
    iota(a.begin(), a.end(), 0);
    for (int i = n; i; i--)
    {
        unsigned long long q = k / f[i - 1];
        k %= f[i - 1];
        assert(q < a.size()); // 调试检查，可删。
        ans.push_back(a[q]);
        a.erase(a.begin() + q);
    }
    return ans;
}
```

`cantor(p)` 接受由 $0, 1, \ldots, n-1$ 组成的排列并返回其康托排名，`invCantor(n, k)` 由排名还原排列。`cantor` 为 $O(n \log n)$、`invCantor` 为 $O(n^2)$，都 $O(n)$ 空间；排名用 `unsigned long long` 存，故 $n \le 20$（$21!$ 已溢出 64 位）。更大的精确排名改用 14.1.001 高精度整数（BigInt）保存阶乘与答案；若只求排名模某个数，则把 `f[i]` 和累加过程取模，但模排名不能再唯一逆展开。只需按字典序枚举而非跳转到指定名次时用 8.3.001 排列组合生成 的 `next_permutation`。

下面用三组已知排名检查重复换位、首尾边界与逆展开：

```
signed main()
{
    assert(cantor({0, 1, 2, 3}) == 0);
    assert(cantor({1, 0, 3, 2}) == 7);
    assert(cantor({3, 2, 1, 0}) == 23);
    assert((invCantor(4, 7) == vector<int>{1, 0, 3, 2}));
    cout << "OK\n";
    return 0;
}
```

8.3.003 勾股数枚举

Euclid 参数化给出全部本原勾股数组：取 $m > n \geq 1$、$\gcd(m, n) = 1$ 且 m, n 一奇一偶，则 $(m^2 - n^2,\ 2mn,\ m^2 + n^2)$ 是本原三元组，且每个本原解恰对应一组这样的 (m, n)；再乘正整数就得到全部非本原解。`pythagorean` 按此枚举斜边 $c = m^2 + n^2 \leq c_{\max}$ 的本原组，`(m-n)%2==0`（同奇偶）或 $\gcd \neq 1$ 时跳过，并把两条直角边排成 $a \leq b$。

```cpp
// PTriple 的三个分量依次是两条直角边 a、b 与斜边 c。
using PTriple = array<int, 3>;

vector<PTriple> pythagorean(long long cmax)
{
    // cmax 是非负斜边上界；返回全部满足 a<=b、c<=cmax 的本原三元组 {a,b,c}。
    assert(cmax >= 0); // 调试检查，可删。
    vector<PTriple> ans;
    for (long long m = 2; m <= (cmax - 1) / m; m++)
    {
        long long mm = m * m;
        for (long long n = 1; n < m; n++)
        {
            if ((m - n) % 2 == 0 || gcd(m, n) != 1)
            {
                continue;
            }
            long long nn = n * n;
            if (mm > cmax - nn)
            {
                break;
            }
            long long a = mm - nn;
            long long b = 2 * m * n;
            long long c = mm + nn;
            if (a > b)
            {
                swap(a, b);
            }
            ans.push_back({a, b, c});
        }
    }
    return ans;
}
```

`pythagorean(cmax)` 返回斜边不超过 `cmax` 的全部本原三元组，`PTriple[0..2]` 为两直角边与斜边。m 到约 $\sqrt{c_{\max}}$、内层 $n < m$，实际有效 (m, n) 对约 $O(c_{\max})$，空间为答案数。需要全部（含非本原）解时把每个本原组乘 $1, 2, \ldots$ 直到超限；只统计数量而不列举时用数论方法按 c 的因子结构计数，不必逐个枚举。

8.3.004 阿达马矩阵

Sylvester 阿达马矩阵元素全为 ± 1、任意两行正交，用于正交编码和构造题。从 $H_1 = [1]$ 出发按分块递推 $H_{2m} = \begin{bmatrix} H_m & H_m \\ H_m & -H_m \end{bmatrix}$ 倍增：若 $H_m H_m^T = mI$，则展开可验证 $H_{2m} H_{2m}^T = 2mI$，故正交性对全部 2 的幂阶成立。代码每轮把 $m \times m$ 的块复制到左上、右上、左下，右下取相反数，直到边长达到 n。

```cpp
vector<vector<int>> hadamard(int n)
{
    // n 是正的 2 的幂；返回元素只取 1 和 -1、任意两条不同行正交的 Sylvester 阿达马矩阵。
    assert(n > 0 && has_single_bit((unsigned long long)n)); // 调试检查，可删。
    assert((unsigned long long)n <= numeric_limits<size_t>::max()); // 调试检查，可删。
    size_t side = (size_t)n;
    assert(side <= numeric_limits<size_t>::max() / sizeof(int) / side); // 调试检查，可删。
    vector<vector<int>> a{{1}};
    while ((int)a.size() < n)
    {
        int m = a.size();
        vector b(2 * m, vector<int>(2 * m));
        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < m; j++)
            {
                b[i][j] = a[i][j];
                b[i][j + m] = a[i][j];
                b[i + m][j] = a[i][j];
                b[i + m][j + m] = -a[i][j];
            }
        }
        a.swap(b);
    }
    return a;
}
```

`hadamard(n)` 要求 `n` 是正的 2 的幂（`has_single_bit` 断言），返回 $n \times n$ 的 ± 1 矩阵；$O(n^2)$ 时间与输出空间。$n = 4$ 时每个元素为 ± 1，每行与自身内积为 4、与其他行内积为 0。题目要 $0/1$ 矩阵时把 -1 映射成 0，但映射后普通整数内积不再为 0（需改用异或/汉明距离度量正交）。非 2 的幂阶的阿达马矩阵（如 12、20 阶）需要 Paley 等其他构造，本条不覆盖；填数构造题里的幻方见 8.3.005 幻方。

8.3.005 幻方

幻方要把 $1, 2, \ldots, n^2$ 各恰用一次填进 $n \times n$ 方阵，使每行、每列、两条主对角线的和都等于幻和 $\frac{n(n^2+1)}{2}$。除 $n = 2$ 无解外，按阶数奇偶分三套经典构造：奇数阶用 Siamese（罗伯法），从中上格填 1，每步向右上移动，越界则绕环、遇占用则改为下移一格；双偶阶（$4 \mid n$）先按行填 $1, 2, \ldots, n^2$，再对满足 $i \bmod 4 = j \bmod 4$ 或 $(i \bmod 4) + (j \bmod 4) = 3$ 的格取补数 $n^2 + 1 - v$；单偶阶（$n \equiv 2 \pmod 4$）用 Strachey/LUX 法，把方阵分成四个 $m = \frac{n}{2}$ 阶子块，各子块填同一个奇数阶幻方 b 加偏移 $0, 2m^2, 3m^2, m^2$（左上、右上、左下、右下），再交换左上/左下块的前 $k = \frac{n-2}{4}$ 列（跳过中心行首列并单独换中心行的第 k 列）与后 $k - 1$ 列，使四块的行列和重新配平。

```cpp
vector<vector<int>> magicSquare(int n)
{
    // n 是阶数；返回恰好填入 1 到 n^2 的幻方，n=2 时无解并返回空矩阵。
    assert(n > 0); // 调试检查，可删。
    assert(n <= numeric_limits<int>::max() / n); // 调试检查，可删。
    assert((unsigned long long)n <= numeric_limits<size_t>::max()); // 调试检查，可删。
    size_t side = (size_t)n;
    assert(side <= numeric_limits<size_t>::max() / sizeof(int) / side); // 调试检查，可删。
    int total = n * n;
    if (n == 2)
    {
        return {};
    }
    if (n & 1)
    {
        vector a(n, vector<int>(n));
        int x = 0;
        int y = n / 2;
        for (int v = 1; v <= total; v++)
        {
            a[x][y] = v;
            int nx = (x + n - 1) % n;
            int ny = (y + 1) % n;
            if (a[nx][ny])
            {
                x = (x + 1) % n;
            }
            else
            {
                x = nx;
                y = ny;
            }
        }
        return a;
    }
    if (n % 4 == 0)
    {
        vector a(n, vector<int>(n));
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                int v = i * n + j + 1;
                bool rev = i % 4 == j % 4 || i % 4 + j % 4 == 3;
                a[i][j] = rev ? total + 1 - v : v;
            }
        }
        return a;
    }
    int m = n / 2;
    int k = (n - 2) / 4;
    int z = m * m;
```

```
    auto b = magicSquare(m);
    vector a(n, vector<int>(n));
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < m; j++)
        {
            a[i][j] = b[i][j];
            a[i][j + m] = b[i][j] + 2 * z;
            a[i + m][j] = b[i][j] + 3 * z;
            a[i + m][j + m] = b[i][j] + z;
        }
    }
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < k; j++)
        {
            if (i != m / 2 || j)
            {
                swap(a[i][j], a[i + m][j]);
            }
        }
        for (int j = n - k + 1; j < n; j++)
        {
            swap(a[i][j], a[i + m][j]);
        }
    }
    swap(a[m / 2][k], a[m / 2 + m][k]);
    return a;
}
```

`magicSquare(n)` 要求 `n>0` 且 n^2、所有块偏移和填充值均能由当前 64 位 `int` 表示；代码用除法断言检查乘法前边界，并另行检查矩阵元素总数可由 `size_t` 分配。$n=1$ 得 `{{1}}`，$n=2$ 返回空矩阵表示无解，其余阶数均有构造。时间与输出空间均为 $O(n^2)$；同为矩阵构造的 ±1 正交阵见 8.3.004 阿达马矩阵。

```
signed main()
{
    assert(magicSquare(2).empty());
    auto a = magicSquare(3);
    assert(a.size() == 3 && a[0].size() == 3 && a[1][1] == 5);
}
```